



مجموعه مهندسی کامپیوتر — ارشد ☒ دکتری ☒

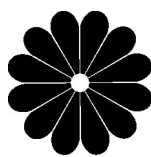
چاپ چهاردهم

ساختمان داده‌ها

هادی یوسفی



به نام خداوند جان و خرد



پوران پژوهش

کتاب ارشد و دکتری

مهندسی کامپیوتر و IT

ساختمان داده‌ها

چاپ چهاردهم

مؤلف:

هادی یوسفی

پاییز ۱۳۹۶

سرشناسه	: یوسفی، هادی، ۱۳۵۴ -
عنوان و نام پدیدآور	: ساختمان داده‌ها
مشخصات نشر	: تهران: پوران پژوهش، ۱۳۹۴.
مشخصات ظاهری	: ۴۱۲ ص.
فروست	: کتاب ارشد، مهندسی کامپیوتر و IT
شابک	: 978-964-184-482-2
وضعیت فهرست‌نویسی	: فنیای مختصر.
یادداشت	: فهرست‌نویسی کامل این اثر در نشانی: http://opac.nlai.ir قابل دسترسی است.
یادداشت	: چاپ یازدهم.
شماره کتابشناسی ملی	: ۲۸۴۱۰۱۸

انتشارات پوران پژوهش

نام کتاب:	ساختمان داده‌ها
تألیف:	هادی یوسفی
ناشر:	پوران پژوهش
حروفچینی:	پوران پژوهش
چاپ و صحافی:	دالاهو - ولیعصر
شمارگان:	۲۰۰۰ نسخه
نوبت چاپ:	چهاردهم - پاییز ۱۳۹۶
شابک:	۹۷۸-۹۶۴-۱۸۴-۴۸۲-۲ ISBN: 978-964-184-482-2

دفتر مرکزی: میدان انقلاب - ابتدای کارگر جنوبی - کوچه مهدیزاده - پلاک ۹ - واحد ۵۴ تلفن: ۶۶۹۲۷۰۴۰

این اثر، مشمول قانون حمایت مؤلفان و مصنفان و هنرمندان مصوب ۱۳۴۸ است، هر کس تمام یا قسمتی از این اثر را بدون اجازه مؤلف و ناشر، نشر یا پخش یا عرضه کند مورد پیگرد قانونی قرار خواهد گرفت.

به نام تنها پرستیدنی که قلم را آفرید

پیش‌گفتار انتشارات

نگاهی به شمار داوطلبان آزمون کارشناسی ارشد نشان می‌دهد که در این سال‌ها درخواست تحصیل در دوره‌های تحصیلات تکمیلی دانشگاه‌ها افزایش چشمگیری داشته است. دشواری پیش روی بیشتر داوطلبان، گوناگونی منابع درسی و نبود دسترسی به آنها و همچنین نمونه آزمون‌های مناسب برای تمرین و فهم بیشتر مفاهیم درسی است.

مدیریت بنیاد انتشاراتی پوران پژوهش، آقای دکتر احمد هژبر و همسرشان بانو افسانه عبدی با بیش از ۲۰ سال کوشش در راستای برآورده‌سازی نیاز داوطلبان، با سیاست کلی چاپ کتاب‌های ارزنده با بهایی درخور، آماده‌سازی و گردآوری چهار مجموعه‌ی گوناگون با انگیزه‌های گوناگون را در دستور کار قرار داده است. مجموعه‌ی نخست با نام کتاب ارشد (با جلد آبی‌رنگ) که تاکنون به دست داوطلبان رسیده، با پذیرش چشمگیری همراه بوده است. در هر عنوان کتاب ارشد، پس از شرح کامل درس در هر فصل، پرسش‌های چهارگزینه‌ای آزمون‌های سراسری و آزاد چند سال گذشته با پاسخ‌های تشریحی آورده شده است. شرح درس در هر کتاب از این مجموعه به گونه‌ای است که برای دانشجویان سال‌های پایین‌تر سودمند است و نیز یک منبع درسی مناسب برای دانشجویان و استادان دانشگاه‌ها می‌باشد. کتاب ارشد نخستین بار در مهرماه سال ۱۳۸۰ در قالب پانزده عنوان به داوطلبان شناسانده شد و هم‌اینک بیش از ۳۰۰ عنوان را دربرمی‌گیرد. مجموعه‌ی دوم با نام آزمون‌های جامع ارشد و دکتری (با جلد سیاه رنگ) به گونه‌ای گردآوری شده است که دانشجو دفترچه‌های آزمون‌های سراسری کارشناسی ارشد و دکتری سال‌های گذشته را با پاسخ‌های تشریحی در یک کتاب خواهد داشت.

مجموعه‌ی سوم با نام بانک تست ارشد (با جلد نارنجی رنگ) در دروس پایه و تخصصی هر رشته، یک کتاب کار به شمار می‌رود که در آن پرسش‌های طبقه‌بندی‌شده به همراه پاسخ‌های تشریحی آورده شده است تا دانشجو با حل و بررسی آن‌ها توانایی بایسته برای پاسخ‌گویی به آزمون‌ها را به دست آورد. مجموعه‌ی چهارم که با جلد قهوه‌ای رنگ عرضه شده و به عنوان کتاب‌های مرجع در دانشگاه‌ها آموخته می‌شود. این کتاب‌ها شامل تألیفات اساتید برجسته کشور و همچنین ترجمه آثار گران‌بهای مؤلفان بزرگ خارج از کشور می‌باشد.

در پایان از خوانندگان محترم تقاضا می‌شود پیشنهادات و انتقادات خود را به پست الکترونیکی انتشارات پوران پژوهش info@pouran.net ارسال فرمایند.

به نام خدا

مقدمه مولف

یک «ساختمان داده» راهی است برای ذخیره و سازماندهی داده‌ها به منظور تسهیل دسترسی و پیرایش آنها. هیچ ساختمان داده‌ای برای هر منظوری مناسب نیست پس نیاز است که نقاط قوت و ضعف ساختمان داده‌های مختلف را بدانیم.

کتاب حاضر، حاصل سال‌ها تدریس و مطالعه است. این کتاب چند بار ویرایش شده است و تلاش شده که مطالب به روز و مطابق با سرفصل‌های وزارت علوم باشد. این کتاب برای آمادگی کنکور کارشناسی ارشد مهندسی کامپیوتر، مهندسی فناوری اطلاعات و علوم کامپیوتر تألیف شده است و همچنین می‌توان از این کتاب به عنوان منبع درس ساختمان داده‌ها در دانشگاه استفاده کرد. این کتاب در ۸ فصل تنظیم شده است که توصیه می‌شود برای درک بهتر و عمیق‌تر فصل‌های ۱ تا ۷ کتاب طراحی الگوریتم همین مؤلف نیز مطالعه شود. در انتهای کتاب تست‌های کنکور ۹۱، ۹۲ و ۹۳ و ۹۴ و ۹۵ و دکتری ۹۵ به همراه پاسخ تشریحی و تست‌های تکمیلی آمده است. الگوریتم‌هایی که در کتاب آمده است از علامت‌ها و قواعد برخی زبان‌ها مثل پاسکال و C بهره برده‌اند. از آنجایی که در تست‌های کنکور معمولاً از شبه کد استفاده می‌شود که در آنها علامت‌های مختلفی بکار می‌رود، در این کتاب نیز چنین است. مثلاً در برخی الگوریتم‌ها علامت $=$ به معنی انتساب و علامت $==$ به معنی مقایسه است. و یا در برخی الگوریتم‌ها علامت $:=$ یا $\leftarrow$ به معنی انتساب و علامت $=$ به معنی مقایسه است.

این کتاب مطالب را به طور کامل پوشش داده است و اگر با دقت مطالعه شود، نیاز به منبع دیگری نیست. از دانشجویان گرامی تقاضا می‌کنم اگر پیشنهاد یا انتقادی در جهت بهبود کتاب دارند با اینجانب در میان گذارند.

دوستان عزیزی در آماده‌سازی کتاب تلاش کرده‌اند که از همه این عزیزان سپاسگزارم.

هادی یوسفی

پاییز ۹۶

Hadi_yusefi@yahoo.com

۰۹۱۲-۱۷۸۸۵۳۴

@yuseficlass

فهرست مطالب

فصل اول. الگوریتم	۱
پیچیدگی الگوریتم، رشد توابع، نمادهای جانبی	
فصل دوم. الگوریتم‌های بازگشتی	۲۹
حل روابط بازگشتی، الگوریتم‌های بازگشتی، درخت بازگشت	
فصل سوم. آرایه، لیست پیوندی، صف، پشته	۶۷
فصل چهارم. جداول درهم‌سازی	۱۲۵
فصل پنجم. درخت ریشه‌دار	۱۳۹
درخت دودویی، پیمایش، درخت نخ‌ی، درخت عمومی، تبدیل درخت عمومی به دودویی، الگوریتم‌های درخت	
فصل ششم. درخت‌های ویژه	۱۹۹
BST ، درخت متوازن، دوران درخت، درخت ۲-۳-۴، درخت قرمز سیاه، $heap$ ، $Deap$ ، $minmaxheap$ ، درخت و هیپ دو جمله‌ای، هافمن، $treap$	
فصل هفتم. گراف	۲۹۱
نمایش گراف، پیمایش گراف، درخت پوشای مینیمم، نمایش مجموعه‌ها، کوتاه‌ترین مسیر هم مبدأ، کوتاه‌ترین مسیر بین تمام زوج گره‌ها، ترتیب توپولوژیکی	
فصل هشتم. مرتب‌سازی	۳۴۱
ضمیمه ۱. تست‌های تکمیلی	۳۸۱
ضمیمه ۲. سوال‌های آزمون سراسری (۹۱ تا ۹۴)	۳۹۱
سوال‌های آزمون دکتری ۱۳۹۵	۴۱۴
سوال‌های آزمون سراسری ۱۳۹۵	۴۱۸
سوال‌های آزمون سراسری ۱۳۹۶	۴۲۳

الگوریتم فصل ۱

الگوریتم

الگوریتم دنباله‌ای از دستورات خوش تعریف است که هیچ یا چند پارامتر ورودی دارد و حداقل یک خروجی دارد. برای حل یک مسئله ممکن است الگوریتم‌های مختلفی وجود داشته باشد که هر الگوریتم ویژگی منحصر به فردی دارد. در هر الگوریتم دو عامل حافظه مصرفی و زمان اجرا اهمیت دارد که زمان اجرا بسیار اهمیت بیشتری دارد.

در این بخش نحوه تحلیل زمان اجرای الگوریتم‌ها را بررسی می‌کنیم. برای این منظور نیازمند دانستن برخی خواص و قوانین ریاضی هستیم که آنها را یادآوری می‌کنیم.

قواعد ریاضی

هنگام محاسبه تعداد اجرای حلقه‌ها به دنباله‌ها برمی‌خوریم که برخی از آنها را یادآوری می‌کنیم.

$$۱) \sum_{i=1}^n i = 1 + 2 + \dots + n = \frac{n(n+1)}{2} \sim \frac{1}{2}n^2 = \theta(n^2)$$

(علامت $\sim$ به معنی هم ارزی است. علامت θ را تعریف خواهیم کرد. به عنوان مثال برای

$$\text{محاسبه حد } \lim_{n \rightarrow \infty} \frac{1+2+\dots+n}{2n^2+n+1} = \frac{1}{2} \text{ از هم ارزی } \frac{1}{2}n^2 \text{ استفاده کردیم)}$$

$$۲) \sum_{i=1}^n i^2 = 1^2 + 2^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6} \sim \frac{1}{3}n^3 = \theta(n^3)$$

$$۳) \sum_{i=1}^n i^۳ = 1^۳ + ۲^۳ + \dots + n^۳ = \left(\frac{n(n+1)}{۲}\right)^۲ \sim \frac{1}{۴} n^۴ = \theta(n^۴)$$

$$۴) \sum_{i=1}^n i^k = 1^k + ۲^k + \dots + n^k \sim \frac{1}{k+1} n^{k+1} = \theta(n^{k+1}) \quad (k > ۰ \text{ عدد ثابت})$$

$$۵) \sum_{i=1}^n \frac{1}{i} = 1 + \frac{1}{۲} + \frac{1}{۳} + \dots + \frac{1}{n} \sim Lnn = \theta(Lgn)$$

(سری همسازه یا هارمونیک است که با Lnn تقریباً مساوی است. ثابت خواهیم کرد که پایه لگاریتم‌ها اگر ثابت باشد در رشد تابع تأثیری ندارد و به همین خاطر بجای Lnn نوشته‌ایم Lgn . در ضمن $Logn$ یا Lgn یعنی $Log_۲n$)

$$۶) \sum_{k=۰}^{n-1} ax^k = a + ax + ax^۲ + \dots + ax^{n-1} = a \frac{1-x^n}{1-x} = a \frac{x^n - 1}{x - 1}$$

(سری هندسی با قدر نسبت x که اگر $n \rightarrow \infty$ و $|x| < 1$ باشد داریم:

$$\sum_{k=۰}^{\infty} ax^k = a + ax + ax^۲ + \dots = \frac{a}{1-x}$$

$$۷) \sum_{k=۰}^{n-1} (a + kd) = a + (a + d) + (a + ۲d) + \dots + (a + (n-1)d)$$

$$= \frac{\text{تعداد جملات} \times (\text{جمله آخر} + \text{جمله اول})}{۲} = \frac{(a + a + (n-1)d)n}{۲}$$

(تصادد حسابی با قدر نسبت d است)

$$۸) \sum_{i=m}^n a = a + a + \dots + a = (n - m + 1)a$$

(a به i وابسته نیست)

$$۹) \sum_{k=۰}^{\infty} kx^k = \frac{x}{(1-x)^۲}, \quad |x| < 1 \quad (\text{با مشتق‌گیری از سری هندسی اثبات می‌شود})$$

$$۱۰) \sum_{k=1}^n (a_k - a_{k-1}) = a_n - a_۰$$

$$\left(\sum_{k=1}^{n-1} \frac{1}{k(k+1)} = \sum_{k=1}^{n-1} \left(\frac{1}{k} - \frac{1}{k+1} \right) = 1 - \frac{1}{n} \right) \quad (\text{سری تلسکوپی گویند به عنوان مثال})$$

لگاریتم در این درس اهمیت زیادی دارد. لگاریتم معکوس تابع توان است یعنی اگر $f(x) = a^x$ آنگاه $f^{-1}(x) = \text{Log}_a^x$. به عبارتی اگر $a^b = c$ آنگاه $\text{Log}_a^c = b$ و برعکس. برخی خواص لگاریتم عبارت است از:

$$۱) \text{Log}_c^{(ab)} = \text{Log}_c^a + \text{Log}_c^b$$

$$۲) \text{Log}_c^{\left(\frac{a}{b}\right)} = \text{Log}_c^a - \text{Log}_c^b$$

$$۳) \text{Log}_{b^n}^{a^m} = \frac{m}{n} \text{Log}_b^a$$

$$۴) \text{Log}_b^a = \frac{\text{Log}_c^a}{\text{Log}_c^b}$$

$$۵) \text{Log}_b^a = \frac{1}{\text{Log}_a^b}$$

$$۶) a^{\text{Log}_a^b} = b$$

$$۷) a^{\text{Log}_c^b} = b^{\text{Log}_c^a}$$

رشد توابع

می‌خواهیم بررسی کنیم وقتی n را به ∞ می‌بریم $f(n) = 4n^2$ زودتر به ∞ می‌رود یا $g(n) = n^3$. درواقع می‌خواهیم بررسی کنیم به ازای n های خیلی بزرگ کدام تابع بزرگتر می‌شود. برای مقایسه رشد توابع می‌توان از حد کمک گرفت.

$$\lim_{n \rightarrow \infty} \frac{4n^2}{n^3} = \lim_{n \rightarrow \infty} \frac{4}{n} = 0 \Rightarrow \text{رشد } 4n^2 \text{ از } n^3 \text{ کمتر است.}$$

به طور کلی حد $\frac{f(n)}{g(n)}$ وقتی $n \rightarrow \infty$ سه حالت دارد:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \begin{cases} 0 & \leftrightarrow \text{رشد } f(n) < \text{رشد } g(n) \\ \infty & \leftrightarrow \text{رشد } f(n) > \text{رشد } g(n) \\ 0 < k < \infty & \leftrightarrow \text{رشد } f(n) = \text{رشد } g(n) \end{cases}$$

لازم به ذکر است که توابع مورد بررسی مثبت هستند ($f(n) > 0$, $g(n)$) ولی می‌توانند نزولی^۱ باشند که در این صورت گوییم رشد منفی دارند. مثلاً برای مقایسه رشد توابع نزولی $f(n) = \frac{1}{n}$ و $g(n) = \frac{1}{n^2}$ داریم:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{\frac{1}{n}}{\frac{1}{n^2}} = \lim_{n \rightarrow \infty} (n) = \infty$$

پس رشد $\frac{1}{n}$ از $\frac{1}{n^2}$ بیشتر است.

با کمک حد می‌توان به نتایج زیر رسید:

(۱) ضریب ثابت تأثیری در رشد ندارد مثلاً رشد $4n^2$ با n^2 مساوی است.

(۲) رشد یک چند جمله‌ای با رشد جمله با بیشترین درجه‌اش مساوی است یعنی:

$$a_p n^p + a_{p-1} n^{p-1} + \dots + a_1 n + a_0 \sim a_p n^p = \theta(n^p)$$

(۳) پایه ثابت لگاریتم تأثیری در رشد ندارد یعنی $\log_a n$ و $\log_b n$ هم رشد هستند ($a, b > 1$ ثابت)

(۴) رشد توابع نمایی از چند جمله‌ای بیشتر است یعنی رشد a^n از n^b بیشتر است (a, b ثابت و $a > 1$)

(۵) رشد a^n از $\log^b n = (\log n)^b$ همیشه بیشتر است (a, b ثابت و $a > 0$)

(۶) رشد $(\log n)^a$ از $\log^b \log n = (\log \log n)^b$ همیشه بیشتر است (a, b ثابت و $a > 0$)

(۷) رشد n^n از رشد $n!$ بیشتر است.

(۸) رشد $Lg(n!)$ با رشد $n.Lgn$ مساوی است.

(اثبات این گزاره کمی مشکل است. یک راه اثبات، استفاده از تقریب استرلینگ است که

$$n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \left(1 + \theta\left(\frac{1}{n}\right)\right)$$

می‌گوید

بنابراین ترتیب رشد زیر قابل اثبات است:

$$\frac{1}{n^2} < \frac{1}{n} < 1 < Lg Lg n < \sqrt{Lg n} < Lg n < \sqrt{n} < n < n Lg n$$

۱ - تابع f را صعودی گویند اگر $m < n$ آنگاه $f(m) \leq f(n)$. تابع f را نزولی گویند هرگاه $m < n$ آنگاه $f(m) \geq f(n)$. تابع f را صعودی اکید گویند هرگاه $m < n$ آنگاه $f(m) < f(n)$. تابع f را نزولی اکید گویند هرگاه $m < n$ آنگاه $f(m) > f(n)$.

رشد ۱ یعنی رشد ثابت. یعنی اصلاً زیاد نمی‌شود و به n وابسته نیست. مثلاً حلقه‌ای که همیشه ۱۰۰۰ بار اجرا می‌شود، رشدش از مرتبه $\theta(1)$ است.

❖ **نکته:** اگر رشد $Lg(f(n))$ از رشد $Lg(g(n))$ کمتر باشد آنگاه حتماً رشد $f(n)$ از $g(n)$ کمتر است. مثلاً می‌خواهیم $f(n) = (Lgn)^n$ را با $g(n) = n^{\lg n}$ مقایسه کنیم $Lg(f(n)) = n \cdot Lg Lgn$ و $Lg(g(n)) = Lg^n n$. واضح است که رشد $Lg(g(n))$ از $Lg(f(n))$ کمتر است پس رشد $g(n) = n^{Lgn}$ از $f(n) = (Lgn)^n$ کمتر است.

❖ **نکته:** اگر رشد $Lg(f(n))$ از رشد Lgn کمتر یا مساوی باشد آنگاه گویند $f(n)$ محدود به چند جمله‌ای است ($f(n) \leq n^c$) که c ثابت است، یعنی رشد $f(n)$ از رشد چند جمله‌ای‌ها بیشتر نیست.

مثال: آیا $|Lgn|!$ (یا $|Lgn|$) محدود به چند جمله‌ای است؟

پاسخ: لگاریتم $|Lgn|!$ را گرفته و بررسی می‌کنیم.

$$Lg(|Lgn|!) \sim |Lgn| \cdot Lg |Lgn| \sim Lgn \cdot Lg(Lgn) > Lgn$$

پس $|Lgn|!$ محدود به چند جمله‌ای نیست. (دقت کنید با توجه به اینکه $Lgn!$ با $n \cdot Lgn$ هم‌ارز است پس $|Lgn|!$ با $|Lgn| \cdot Lg |Lgn|$ هم‌ارز است)

مثال: آیا $|Lg Lgn|!$ محدود به چند جمله‌ای است؟

پاسخ: بررسی کنید لگاریتم این تابع رشد کمتری از Lgn دارد پس محدود به چند جمله‌ای است.

نمادهای مجانبی (asymptotic notations)

برای بیان زمان اجرای الگوریتم‌ها و رشد توابع از نمادهای O و Ω و θ و o و w استفاده می‌شود.

نماد O (big oh): گویند تابع $f(n)$ از مرتبه $O(g(n))$ است و می‌نویسیم $f(n) \in O(g(n))$ یا $f(n) = O(g(n))$ هرگاه رشد $f(n)$ از رشد $g(n)$ کمتر یا مساوی باشد. مثلاً $3n^2 + n + 4 \in O(n^2)$ یا $3n^2 + n + 4 \in O(n^3)$ ولی $3n^2 + n + 4 \notin O(n)$.

تعریف دقیق ریاضی نماد O عبارت است از:

$$O(g(n)) = \{f(n) \mid \exists c, n_0 > 0, \forall n \geq n_0 : f(n) \leq c \cdot g(n)\}$$

یعنی $O(g(n))$ شامل همه توابعی است که رشدشان از $g(n)$ کمتر یا مساوی است. مثلاً $O(n^2)$ شامل توابعی است که رشدشان از n^2 کمتر یا مساوی است مثل $1 + 3n^2$ ، $5n$ ، $Lg^4 n$ و $\sqrt{n}$ و $Lg Lg n$ و ...

توجه: اگر $f(n)$ از مرتبه $O(g(n))$ باشد بهتر است بنویسیم $f(n) \in O(g(n))$. زیرا $O(g(n))$ یک مجموعه است و نوشتن $f(n) = O(g(n))$ درست نیست ولی به دلیل استفاده خیلی زیاد، این اشتباه نادیده گرفته شده است و در این کتاب نیز استفاده می‌شود.

نماد Ω (امگای بزرگ): گویند تابع $f(n)$ از مرتبه $\Omega(g(n))$ است و می‌نویسیم $f(n) \in \Omega(g(n))$ یا $f(n) = \Omega(g(n))$ هرگاه رشد $f(n)$ از رشد $g(n)$ بیشتر یا مساوی باشد. مثلاً $3n^2 + n + 4 \in \Omega(n^2)$ یا $3n^2 + n + 5 \in \Omega(n)$ ولی $3n^2 + n + 4 \notin \Omega(n^3)$.
تعریف ریاضی دقیق Ω عبارت است از:

$$\Omega(g(n)) = \{f(n) \mid \exists c, n_0 > 0, \forall n \geq n_0 : f(n) \geq c \cdot g(n)\}$$

یعنی $\Omega(g(n))$ شامل همه توابعی است که رشدشان از $g(n)$ بیشتر یا مساوی است. مثلاً $\Omega(n^2)$ شامل توابعی است که رشدشان از n^2 بیشتر یا مساوی است مثل n^2 ، $3n^2 + n$ ، $n^3 - n$ و 2^n و $n!$ و ...

نماد θ (تتا): گویند تابع $f(n)$ از مرتبه $\theta(g(n))$ است و می‌نویسیم $f(n) \in \theta(g(n))$ یا $f(n) = \theta(g(n))$ هرگاه $f(n)$ و $g(n)$ هم رشد باشند. مثلاً $3n^2 + n + 4 \in \theta(n^2)$ و $3n^2 + n + 4 \notin \theta(n)$ و $3n^2 + n + 4 \notin \theta(n^3)$.
تعریف ریاضی دقیق θ عبارت است از:

$$\theta(g(n)) = \{f(n) \mid \exists c_1, c_2, n_0 > 0, \forall n \geq n_0 : c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)\}$$

به راحتی می‌توان دریافت که اشتراک $O(g(n))$ با $\Omega(g(n))$ برابر $\theta(g(n))$ است.

نماد o (little oh): گویند تابع $f(n)$ از مرتبه $o(g(n))$ است و می‌نویسیم $f(n) \in o(g(n))$ یا $f(n) = o(g(n))$ هرگاه رشد $f(n)$ از رشد $g(n)$ کمتر باشد مثلاً $3n^2 + n + 4 \in o(n^3)$ ولی $3n^2 + n + 4 \notin o(n^2)$.

تعریف ریاضی دقیق نماد o عبارت است از:

$$o(g(n)) = \{f(n) \mid \exists n_0 > 0, \forall c > 0, \forall n \geq n_0 : f(n) < c \cdot g(n)\}$$

توجه کنید تفاوت این تعریف با تعریف O (big) این است که اینجا $\forall c > 0$ ولی در تعریف O (big) داشتیم $\exists c > 0$. به هر حال این تعریف می‌گویید، $o(g(n))$ شامل توابعی است که رشدشان از $g(n)$ کمتر است. مثلاً $o(n^2)$ شامل توابعی مثل $3n + 4$ و Lgn و $Lg^5 n$ و $\sqrt{n}$ و $\frac{1}{n}$ و ... است.

نماد ω (امگای کوچک): گویند تابع $f(n)$ از مرتبه $\omega(g(n))$ است و می‌نویسیم $f(n) \in \omega(g(n))$ یا $f(n) = \omega(g(n))$ هرگاه رشد $f(n)$ از رشد $g(n)$ بیشتر باشد. مثلاً $3n^2 + n + 4 \in \omega(n^2)$ ولی $3n^2 + n + 4 \notin \omega(n^2)$.

تعریف ریاضی دقیق نماد ω عبارت است از:

$$\omega(g(n)) = \{f(n) \mid \exists n_0 > 0, \forall c > 0, \forall n \geq n_0 : f(n) > c \cdot g(n)\}$$

یعنی $\omega(g(n))$ شامل توابعی است که رشدشان از $g(n)$ بیشتر است. مثلاً $\omega(n^2)$ شامل توابعی مثل $3n^3 + 4$ و 2^n و $n!$ و n^n و ... است.

واضح است که $\omega(g(n))$ با $o(g(n))$ هیچ اشتراکی ندارد.

خلاصه تعاریف نمادها را می‌توان به شکل زیر بیان کرد:

$$\begin{aligned} f(n) \in O(g(n)) &\leftrightarrow f \text{ رشد} \leq g \text{ رشد} \\ f(n) \in \Omega(g(n)) &\leftrightarrow f \text{ رشد} \geq g \text{ رشد} \\ f(n) \in \theta(g(n)) &\leftrightarrow f \text{ رشد} = g \text{ رشد} \\ f(n) \in o(g(n)) &\leftrightarrow f \text{ رشد} < g \text{ رشد} \\ f(n) \in \omega(g(n)) &\leftrightarrow f \text{ رشد} > g \text{ رشد} \end{aligned}$$

همچنین می‌توان با توجه به تعریف نمادها و مفهوم حدگیری، روابط زیر را ارائه داد:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \begin{cases} 0 \leftrightarrow f(n) \in o(g(n)) \rightarrow f(n) \in O(g(n)) \\ \infty \leftrightarrow f(n) \in \omega(g(n)) \rightarrow f(n) \in \Omega(g(n)) \\ 0 < k < \infty \leftrightarrow f(n) \in \theta(g(n)) \end{cases}$$

دقت کنید اگر $f(n) \in o(g(n))$ می‌توان نتیجه گرفت $f(n) \in O(g(n))$ ولی عکس این مطلب لزوماً صحیح نیست.

خواص زیر برای نمادها به راحتی قابل بررسی هستند:

(a) بازتابی (reflexive): نمادهای O و Ω و θ بازتاب هستند یعنی

$$f(n) \in O(f(n)), \quad f(n) \in \Omega(f(n)), \quad f(n) \in \theta(f(n))$$

(b) تقارنی (symmetric): نماد θ متقارن است یعنی:

$$f(n) \in \theta(g(n)) \leftrightarrow g(n) \in \theta(f(n))$$

(c) تقارنی ترانهاده (transpose symmetric): نمادهای O و Ω و همچنین o و ω دارای خاصیت تقارنی ترانهاده هستند یعنی:

$$f(n) \in O(g(n)) \leftrightarrow g(n) \in \Omega(f(n))$$

$$f(n) \in o(g(n)) \leftrightarrow g(n) \in \omega(f(n))$$

(d) تعدی (transitive): همه نهادها متعدی هستند یعنی:

$$f(n) \in O(g(n)) , g(n) \in O(h(n)) \rightarrow f(n) \in O(h(n))$$

بجای نماد O می‌توان نمادهای دیگر را قرار داد.

(e) اگر $f(n) = O(g(n))$ و $f(n) = \Omega(g(n))$ آنگاه $f(n) = \theta(g(n))$ و برعکس.

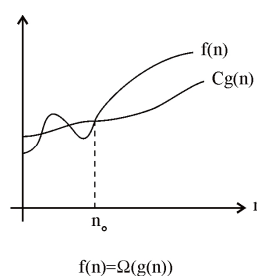
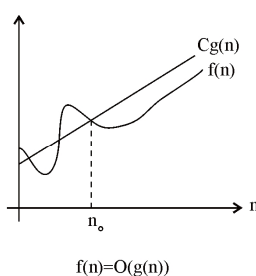
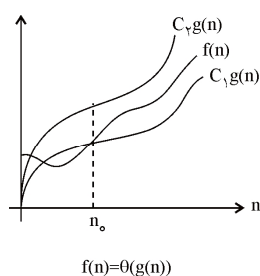
مثال: آیا $O(f(n)) - \Omega(f(n))$ و $o(f(n))$ دو مجموعه مساوی هستند؟

پاسخ: هرچند ظاهراً باید مساوی باشند ولی نیستند. مثلاً تابع $g(n) = \begin{cases} n & \text{زوج } n \\ 1 & \text{فرد } n \end{cases}$ را در نظر

بگیرید. واضح است که $g(n) \in O(n)$ زیرا $g(n)$ کمتر مساوی n است. $g(n) \notin \Omega(n)$ زیرا به ازای n های فرد، $g(n)$ برابر ۱ است که از n بیشتر مساوی نیست. پس $g(n) \in O(n) - \Omega(n)$. از طرفی $g(n) \notin o(n)$ زیرا به ازای n های زوج، $g(n)$ برابر n است که از n کمتر نیست. پس نشان دادیم توابعی وجود دارد که عضو $O(n) - \Omega(n)$ هستند ولی عضو $o(n)$ نیستند. می‌توان نشان داد همیشه:

$$o(f(n)) \subseteq O(f(n)) - \Omega(f(n))$$

توجه: نمودارهای زیر، نمادهای O و Ω و θ را نشان می‌دهند.



محاسبه زمان اجرا

زمان اجرای یک الگوریتم به عوامل مختلفی بستگی دارد مثل نوع سخت افزار، نوع کامپایلر، اندازه ورودی و ... ما می‌خواهیم زمان اجرا را طوری بیان کنیم که مستقل از سخت افزار و نرم افزار باشد. زمان اجرا را برابر تعداد اجرای همه دستورات تعریف می‌کنیم.

مثال: زمان اجرا (تعداد اجرای همه دستورات، گام شماری) تکه برنامه‌های زیر را بدست می‌آوریم:

۱) $for(i = 1 \text{ to } n) // \text{ بار } n + 1$

$write("*"); // \text{ بار } n$

زمان اجرا $= 2n + 1 = \theta(n)$

حلقه for و حلقه $while$ خودشان از جمله داخلشان یک بار بیشتر اجرا می‌شوند البته به شرطی که دستوری مثل $break$ استفاده نشود.

۲) $for(i = a \text{ to } b) // \text{ بار } b - a + 1$

$write("*"); // \text{ بار } b - a + 1$

زمان اجرا $= 2(b - a) + 1 = \theta(b - a)$

۳) $for(i = 1 \text{ to } n) // \text{ بار } n + 1$

$for(j = 1 \text{ to } n) // \text{ بار } n(n + 1)$

$write("*"); // \text{ بار } n^2$

زمان اجرا $= 2n^2 + 2n + 1 = \theta(n^2)$

۴) $for(i = 1 \text{ to } n) // \text{ بار } n + 1$

$for(j = 1 \text{ to } n) // \text{ بار } n(n + 1)$

$for(k = 1 \text{ to } n) // \text{ بار } n^2(n + 1)$

$write("*"); // \text{ بار } n^3$

زمان اجرا $= 2n^3 + 2n^2 + 2n + 1 = \theta(n^3)$

لازم به ذکر است که اگر زمان اجرا به طور کامل پرسش نشود و فقط مرتبه زمان اجرا را برحسب نماد θ بخواهیم، لازم نیست تعداد اجرای همه خطوط را بیابیم بلکه کافی است فقط تعداد اجرای دستور اصلی (دستور $write$ در مثال‌های فوق) را بیابیم.

مثال: در تکه برنامه‌های زیر تعداد اجرای دستور *write* را محاسبه می‌کنیم:

```
۱)  for(i = ۱ to n)
      for(j = ۱ to i)
          write("* ");
```

حلقه دوم به حلقه اول وابسته است. به ازای $i = ۱$ دستور *write* یک بار اجرا می‌شود. به ازای $i = ۲$ دو بار و به همین ترتیب به ازای $i = n$ دستور *write* به تعداد n بار اجرا می‌شود پس تعداد اجرای دستور *write* برابر $\frac{n(n+1)}{۲}$ است.

برای محاسبه تعداد اجرای دستور *write* می‌توان به ازای هر حلقه یک Σ نوشت و به شکل زیر نیز محاسبه کرد:

$$\sum_{i=1}^n \sum_{j=1}^i ۱ = \sum_{i=1}^n i = \frac{n(n+1)}{۲} = \theta(n^۲)$$

```
۲)  for(i = ۱ to n)
      for(j = ۱ to i)
          for(k = ۱ to j)
              write("* ");
```

$$\begin{aligned} \text{تعداد اجرای } write &= \sum_{i=1}^n \sum_{j=1}^i \sum_{k=1}^j ۱ = \sum_{i=1}^n \sum_{j=1}^i j = \sum_{i=1}^n \frac{i(i+1)}{۲} = \frac{۱}{۲} \left[\sum_{i=1}^n i^۲ + \sum_{i=1}^n i \right] \\ &= \frac{۱}{۲} \left[\frac{n(n+1)(۲n+1)}{۶} + \frac{n(n+1)}{۲} \right] = \frac{۱}{۲} \times \frac{n(n+1)(۲n+۴)}{۶} \\ &= \frac{n(n+1)(n+۲)}{۶} = \theta(n^۳) \end{aligned}$$

```
۳)  i = n;
      while(i > ۱)
      {
          write("* "); // بار اجرا می‌شود  $\left\lfloor \frac{n}{۲} \right\rfloor = \theta(n)$ 
          i = i - ۲;
      }
```

از i هر بار ۲ واحد کم می‌شود. مثلاً اگر $n = ۱۰$ باشد دستور $write$ به ازای $i = ۱۱, ۹, ۷, ۵, ۳$ اجرا می‌شود. یا اگر $n = ۱۱$ باشد دستور $write$ به ازای $i = ۱۰, ۸, ۶, ۴, ۲$ اجرا می‌شود. پس دستور $write$ همیشه $\left\lfloor \frac{n}{2} \right\rfloor$ بار اجرا می‌شود.

توجه: علامت $\lfloor x \rfloor$ کف ($floor$) است و علامت $\lceil x \rceil$ سقف ($ceiling$) است. $\lfloor x \rfloor$ برابر بزرگترین عدد صحیح کوچکتر یا مساوی x است و $\lceil x \rceil$ برابر کوچکترین عدد صحیح بزرگتر یا مساوی x است.

```
۴)   i = n;
      while(i > ۱)
      {
        write(" "); // بار  $\lceil Lgn \rceil = \theta(Lgn)$ 
        i =  $\left\lfloor \frac{i}{2} \right\rfloor$ ;
      }
```

دستور $write$ به ازای $i = \frac{n}{2^0}, \frac{n}{2^1}, \frac{n}{2^2}, \dots, \frac{n}{2^{k-1}}$ اجرا می‌شود (یعنی k بار) حال اگر $\frac{n}{2^k} = ۱$ ($k = \lg n$) حلقه پایان می‌یابد. و اگر n توانی از ۲ نباشد به راحتی می‌توان بررسی کرد که علامت $\lceil \rceil$ لازم است.

```
۵)   i = ۱;
      while(i < n)
      {
        write(" "); // بار  $\lceil Lgn \rceil = \theta(Lgn)$ 
        i = i * ۲;
      }
```

همانند قبلی می‌توان بررسی کرد. و یا می‌توان به ازای $n = ۷$ و $n = ۸$ بررسی کرد که ۳ بار اجرا می‌شود و نتیجه گرفت که علامت $\lceil \rceil$ لازم است.

```

۶)   i = ۲;
      while(i < n)
      {
        write("**"); // بار  $\lceil Lg Lgn \rceil = \theta(Lg Lgn)$ 
        i = i * i;
      }

```

دستور $write$ به ازای $2^{2^0}, 2^{2^1}, 2^{2^2}, 2^{2^3}, \dots, 2^{2^{k-1}}$ یعنی k بار اجرا می‌شود و وقتی $2^{2^k} = n \rightarrow 2^k = Lgn \rightarrow k = Lg Lgn$ حلقه پایان می‌یابد پس: می‌توان بررسی کرد اگر $Lg Lgn$ عدد صحیح نباشد علامت $\lceil \rceil$ لازم است. در برخی الگوریتم‌ها زمان اجرا به طور دقیق قابل محاسبه نیست بلکه در حالت‌های مختلف، متفاوت است. سه حالت در تحلیل الگوریتم‌ها مطرح می‌شود. بهترین ($Best$)، متوسط ($Average$) و بدترین ($worst$) حالت.

به عنوان مثال فرض کنید می‌خواهیم مقدار x را در آرایه $c[1..n]$ جستجوی خطی کنیم. عمل اصلی در جستجو، مقایسه است. اگر x در $c[1]$ باشد بهترین حالت است و ۱ مقایسه می‌خواهد ($B(n) = 1$). اگر x در $c[n]$ باشد و یا اصلاً نباشد بدترین حالت است و n مقایسه انجام می‌شود ($\omega(n) = n$). بررسی حالت متوسط معمولاً دشوار است و باید تمام حالات را در نظر گرفت و سپس میانگین گرفت. فعلاً فرض کنید x حتماً در آرایه هست در این صورت احتمال وجود x در

خانه $c[i]$ ($i = 1, 2, \dots, n$) برابر $\frac{1}{n}$ است و زمان اجرای حالت متوسط برابر است با:

$$A(n) = \frac{1}{n} \sum_{i=1}^n i = \frac{1}{n} \frac{n(n+1)}{2} = \frac{n+1}{2}$$

دقت کنید اگر x در $c[1]$ باشد ۱ مقایسه، در $c[2]$ باشد ۲ مقایسه و ...

حال فرض کنید x با احتمال p در آرایه هست پس با احتمال $\frac{p}{n}$ در $c[i]$ است و با احتمال $1-p$ در آرایه نیست در این صورت زمان اجرای حالت متوسط برابر است با:

$$A(n) = \frac{p}{n} \times 1 + \frac{p}{n} \times 2 + \dots + \frac{p}{n} \times n + (1-p) \times n = \frac{p(n+1)}{2} + (1-p)n$$

مثال: فرض کنید با احتمال ۵۰٪ عنصر x در $c[1..n]$ هست. به طور متوسط چه تعداد عنصر آرایه برای یافتن x بررسی می‌شوند؟

پاسخ: $p = \frac{1}{2}$ پس $A(n) = \frac{n+1}{4} + \frac{n}{2} = \frac{3n+1}{4}$ یعنی به طور متوسط $\frac{3}{4}$ عناصر آرایه بررسی می‌شوند.

مسائل:

۱- تابع $Lg^* n$ برابر است با تعداد باری که از n لگاریتم (پایه ۲) بگیریم که حاصل ۱ یا کمتر از ۱ شود. مثلاً $Lg^* ۱۶ = ۳$ ($Lg Lg Lg ۱۶ = ۱$) و یا $Lg^* ۶۵۵۳۶ = ۴$ و $Lg^* ۲۶۵۵۳۶ = ۵$. تابع $Lg^* n$ رشد بسیار ضعیفی دارد و از تمام توابع صعودی که در این بخش دیدیم رشد کمتری دارد. در هر قسمت بررسی کنید ترتیب داده شده صحیح است.

$$a) \quad n^{\frac{1}{Lgn}} < Lg^2 n < Lg(n!) < n^2 < \left(\frac{3}{2}\right)^n < 2^{2^n}$$

(ابتدا بررسی کنید $n^{\frac{1}{Lgn}} = 2$ ثابت است)

$$b) \quad 2^{Lg^* n} < (\sqrt{2})^{Lgn} < n^2 < n! < 2^{2^{n+1}}$$

(بررسی کنید $\sqrt{2}^{Lgn} = \sqrt{n}$)

$$c) \quad Lg^* n < Lg Lgn < \sqrt{Lgn} < Lgn < n^{Lg Lgn} < 2^n < n \cdot 2^n$$

(بررسی کنید $n^{Lg Lgn} = (Lgn)^{Lgn}$)

$$d) \quad \sqrt{Lgn} < 2^{Lgn} < 4^{Lgn} < e^n < (n+2)!$$

$$e) \quad Lg^*(n^n) < 2^{\sqrt{2}^{Lgn}} < n < n Lgn < n^{2+Lgn} < \left(\frac{3}{2}\right)^n$$

$$f) \quad 4^{(Lgn+Lg Lgn)} = n^2 Lg^2 n < n^2 Lg^3 \sqrt{n}$$

۲- درستی یا نادرستی عبارات زیر را مشخص کنید.

$$f(n) + g(n) = \theta(\max\{f(n), g(n)\}) \quad (a)$$

$$b) \quad \text{اگر } f(n) = \Omega(n^2) \text{ و } g(n) = \Omega(n), \text{ در این صورت } f(g(n)) = \Omega(n^3)$$

$$c) \quad f(g(n)) = \theta(g(f(n)))$$

$$d) \quad f(n) = \theta\left(f\left(\frac{n}{2}\right)\right)$$

$$e) \quad \text{اگر } T_1(n) = O(f) \text{ و } T_2(n) = O(f) \text{ آنگاه } T_1(n) + T_2(n) = O(f)$$

$$i) \quad T_1(n) + T_2(n) = O(f)$$

$$ii) \quad T_1(n) * T_2(n) = O(f)$$

$$iii) \quad T_1(n) = O(T_2(n))$$

$$\frac{T_1(n)}{T_7(n)} = O(1) \quad (iv)$$

اگر $f(n) = \Omega(g(n))$ و $g(n) = O(f(n))$ آنگاه $f(n) = \theta(g(n))$ درست است. پاسخ: a

b) نادرست. مثلاً $f(n) = n^2$ و $g(n) = n$ آنگاه $f(g(n)) = n^2$ که $\Omega(n^2)$ نیست.

c) نادرست. مثلاً $f(n) = Lgn$ و $g(n) = n^2$ آنگاه $f(g(n)) = 2Lgn$ و $g(f(n)) = (Lgn)^2$

d) نادرست. مثلاً $f(n) = 4^n$ آنگاه $f(\frac{n}{4}) = 4^{\frac{n}{4}}$

e) فقط قسمت i درست است. برای نقض قسمت ii فرض کنید $T_1(n) = T_7(n) = f = n$

آنگاه $T_1(n) * T_7(n) = n^2 \neq O(n)$ برای نقض قسمت iii و iv فرض کنید $f = n^2$

$$T_1(n) = n \quad \text{و} \quad T_7(n) = n^2$$

f) نادرست. $f(n) = \Omega(g(n))$ با $g(n) = O(f(n))$ معادل است و نتیجه نمی‌شود $f(n) = \theta(g(n))$

۳- آیا می‌توان ادعا کرد که برای هر دو تابع $f(n)$ و $g(n)$ یا $f(n) = O(g(n))$ یا $g(n) = O(f(n))$ یا هر دو؟

پاسخ: خیر. فرض کنید $f(n) = n$ و $g(n) = n^{\sin n}$ بررسی کنید $f(n) \neq O(g(n))$ و $g(n) \neq O(f(n))$

۴- پیچیدگی تکه برنامه‌های زیر را بیابید.

a) `for(i = 1 to n)`

`for(j = 1 to i^2)`

`if(j mod i == 0)`

`for(k = 1 to j)`

`write(" ");`

b) `for(i = 1 to n)`

`if(odd(i)) for(j = i to n) x = x + 1;`

`else for(j = 1 to i) y = y + 1;`

```

c)  i = ۳;
    if(n == ۲ or n == ۳) return true;
    if(n mod ۲ == ۰) return false;

    while(i۳ ≤ n)
    if(n mod i == ۰) return false;
        else i = i + ۲;
    return true;

d)  for(i = ۱ ; i ≤ n ; i = i * ۲)
    for(j = ۱ ; j ≤ n ; j++)
        write("*");

e)  for(i = ۱ ; i ≤ n ; i++)
    for(j = ۱ ; j ≤ n ; j = j * ۲)
        write("*");

```

پاسخ: a به ازای هر i ، داخلی‌ترین حلقه i بار اجرا می‌شود. در واقع به ازای $j = i, ۲i, ۳i, \dots, i * i$ شرط if درست است و حلقه داخلی اجرا می‌شود پس حلقه داخلی به تعداد $i * i = i + ۲i + \dots + i * i = i \frac{i(i+1)}{۲}$ بار اجرا می‌شود که از مرتبه $\theta(i^۳)$ است و چون خود i, n بار تکرار می‌شود، مرتبه کل $\theta(n^۴)$ است.

b چه i زوج باشد و چه فرد مرتبه $\theta(n^۲)$ می‌شود.

c با توجه به شرط حلقه $while$ یعنی $i^۲ \leq n$ مرتبه $O(\sqrt{n})$ است.

۵- اگر f_w و f_a به ترتیب درجه‌های پیچیدگی زمان اجرای یک الگوریتم در بدترین حالت و حالت میانگین باشند، کدام عبارت $f_a = \theta(f_w)$ ، $f_a = o(f_w)$ ، $f_a = O(f_w)$ همواره صحیح است؟

پاسخ: زمان در حالت متوسط کمتر یا مساوی زمان در بدترین حالت است پس همواره $f_a = O(f_w)$

۶- بعضی منابع نماد Ω را با کمی تغییر تعریف می‌کنند. نام این نماد جدید را Ω^∞ (امگای

بی‌نهایت) می‌گذاریم. گوییم $f(n) = \Omega^\infty(g(n))$ هرگاه ثابت مثبت c وجود داشته باشد که برای تعداد بی‌شماری n داشته باشیم $f(n) \geq cg(n)$. نشان دهید برای هر دو تابع مثبت

$f(n)$ و $g(n)$ یا $f(n) = O(g(n))$ یا $f(n) = \Omega^\infty(g(n))$ (یا هر دو)

۷- نماد $\tilde{O}(soft - oh)$ همانند O است که فاکتورهای لگاریتمی را چشم‌پوشی می‌کند.

$$\tilde{O}(g(n)) = \{f(n) : \exists c, k, n_0 > 0, \forall n \geq n_0 : f(n) \leq cg(n)Lg^k(n)\}$$

به همین ترتیب $\tilde{\Omega}$ و $\tilde{\theta}$ نیز تعریف می‌شوند. نشان دهید $f(n) = \tilde{O}(g(n))$ و

$$f(n) = \tilde{\Omega}(g(n)) \text{ اگر و فقط اگر } f(n) = \tilde{\theta}(g(n)).$$

۸- فرض کنید $f(n)$ تابع صعودی است. f_c^* را تعریف می‌کنیم:

$$f_c^*(n) = \min\{i \geq 0 : f^{(i)}(n) \leq c\}$$

$f^{(i)}(n)$ را نیز به صورت مقابل تعریف می‌کنیم:

$$f^{(i)}(n) = \begin{cases} n & \text{if } i = 0 \\ f(f^{(i-1)}(n)) & \text{if } i > 0 \end{cases}$$

مثلاً اگر $f(n) = 2n$ آنگاه $f^{(i)}(n) = 2^i n$ است.

با توجه به جدول مقابل، درستی مقادیر ستون $f_c^*(n)$ را بررسی کنید.

به عنوان مثال فرض کنید $f(n) = \frac{n}{2}$ و $c = 1$:

$$f^{(2)}(n) = \frac{n}{2^2}, \dots, f^{(i)}(n) = \frac{n}{2^i} \leq 1$$

در نتیجه کمترین i برابر Lgn می‌شود.

$f(n)$	c	$f_c^*(n)$
$n - 1$	0	n
Lgn	1	$Lg^* n$
$\frac{n}{2}$	1	$\lceil \lg n \rceil$
$\frac{n}{2}$	2	$\lceil \lg n \rceil - 1$
$\sqrt{n}$	2	$\geq \lg \lg n$
$\sqrt{n}$	1	∞
$\frac{1}{n^2}$	2	$Log_2(Lgn)$
$\frac{n}{Lgn}$	2	$O(\lg n)$

سؤال‌های طبقه‌بندی شده فصل اول

۱- مجموع مراحل خطوط در برنامه زیر چند است؟

```
float sum (int num[ ], int n)
{
    int i , temp=0 ;
    for (i= 0 ; i<n ; i++)
        temp += num [i] ;
    return temp ;
}
```

(۱) $2n + 3$
 (۲) $2n + 1$
 (۳) تعداد مراحل بستگی به n دارد و نامشخص است.
 (۴) $n+1$

۲- در برنامه زیر تعداد دفعات تکرار دستورالعمل ۳ برابر است با... (علوم کامپیوتر ۸۰)

```
۱) for (k = ۰ ; k <= n - ۱ ; k++)
۲) for (i = ۱ ; i <= n - k ; i++)
۳) a[i][i+k] = k ;
```

(۱) n^2
 (۲) $\frac{n(n+1)}{2}$
 (۳) $\frac{n^2}{2}$
 (۴) $\frac{n(n-1)}{2}$

۳- مقدار محاسبه شده برای $TOTAL$ را تعیین کنید. (علوم کامپیوتر ۷۹)

```
TOTAL:= 0 ;
for i:=1 to N do
begin
    K:=N ;
    while (K<>1) do begin
        K:=K div 2 ;
        TOTAL := TOTAL + ۱ ;
    end
end ;
```

(۱) $N (\log_2^N - 1)$
 (۲) $N \log_2^N$
 (۳) $N (\log_2^N + 1)$
 (۴) $\log_2^N + 1$

۴- با توجه به تکه برنامه مقابل مقدار $f(n)$ عبارت است از:

$i = ۱$	$\log_۲^n$ (۱)
$while (i \leq n)$	$\log_۲^n + ۱$ (۲)
{	$n \left(\frac{n+۱}{۲} \right)$ (۳)
$j = ۱$	
$while (j \leq n)$	$n (\log_۲^n + ۱)$ (۴)
{	
$j = j^*$	
}	
$i = i + ۱$	
}	

۵- روال بازگشتی مقابل را در نظر بگیرید مطلوبست M (۳۰ و ۵)

$M(A, B)$	۲۰ (۱)
$P \leftarrow \circ$	۴۰ (۲)
$while A <> 0$	۷۰ (۳)
$Do \{ if A \bmod ۲ = ۱ then P \leftarrow P + B$	۱۵۰ (۴)
$A \leftarrow \left\lfloor \frac{A}{۲} \right\rfloor$	
$B \leftarrow ۲B; \}$	
$Return P$	

۶- اگر M واحد، زمان اجرای $Modul A$ شود و N تعداد داده‌های ورودی باشد، تابع

$I := N$	پیچیدگی زمانی الگوریتم زیر کدام است؟
$while I > ۱$	$MN \log_{\delta}^N$ (۱)
for $j := ۱ to N$	$MN^۲$ (۲)
$Modul A$	$N \log_{\delta}^M$ (۳)
end for	$MN^۲ \log_{\gamma}^N$ (۴)
$I := I / ۵$	
end while	

۷- زمان مصرفی قطعه برنامه زیر در بهترین و بدترین حالت چه مقداری است و مربوط به چه مواردی می‌باشد؟

```
for i ← ۲ to n do
    if k mod i = ۰ then
        for j ← i to n do
```

$\ell \leftarrow \ell^* k$

(۱) در بهترین حالت زمان خطی است و این مربوط به ورودی است که k به n بخش پذیر نباشد. و بدترین حالت زمانی است که k به کلیه اعداد ۱ تا n بخش پذیر باشد در اینصورت زمان $۲n$ است.

(۲) در بهترین حالت زمان ضریب ثابتی از n است و این حالت مربوط به مواردی است که k مضربی از n باشد و بدترین زمان $n^۲$ است که مربوط به باقی حالات می‌شود.

(۳) در هر حال و در کلیه موارد زمان مصرفی $n^۲$ است.

(۴) اگر k ضربی از $n!$ باشد در بدترین حالت قرار می‌گیریم که زمان مصرفی مربوطه $n^۲$ است و بهترین حالت زمانی است که k به هیچکدام از اعداد ۲ تا n بخش پذیر نباشد در این صورت زمان مصرفی خطی است.

۸- مرتبه زمانی شبه کد زیر چیست؟

```
for (i = ۱ ; i <= n ; i++)
{
    for (j = ۱; j <= n; j++)
        log n
    x++;
n--;
}
```

(۱) n
(۲) $n^۲$
(۳) $\log n$
(۴) $n \log n$

۹- مرتبه زمانی شبه کد زیر چیست؟

```
for (i = ۱; i <= n; i++)
    for (j = ۱; j <= n; j++)
        log n
    {
        x++;
    }
n--;
```

(۱) n
(۲) $n^۲$
(۳) $\log n$
(۴) $n \log n$

۱۰- پس از اجرای قطعه کد زیر، مقدار نهایی x چه مقداری خواهد بود؟ (۸۵ IT)

$x=0;$	$n \cdot \lfloor \log_2 n \rfloor$ (۱)
$for (i = 1; i \leq n; i++) \{$	$n^2 + \lfloor \log_2 n \rfloor$ (۲)
$for (j = 1; j \leq n; j++) x++;$	$n^2 + n \lfloor \log_2 n \rfloor$ (۳)
$j = 1;$	$n (1 + \lfloor \log_2 n \rfloor)$ (۴)
$while (j < n) \{$	
$x++; j = j * 2;$	
$\}$	
$\}$	

۱۱- کدامیک از مجموعه توابع زیر برحسب افزایش مرتبه ($Order$) از چپ به راست مرتب هستند؟ (علوم کامپیوتر ۷۹)

$n! و (1/0.5)^n و n^{1000}$ (۲)	$n! و n^{1000} و (1/0.5)^n$ (۱)
$n^{1000} و n! و (1/0.5)^n$ (۴)	$n^{1000} و n! و (1/0.5)^n$ (۳)

۱۲- مرتبه بزرگی ($Order of magnitude$) قطعه کد زیر چیست؟ (آزاد ۸۰)

$k := 0 ;$	$O(n^2)$ (۱)
$for i := 1 to n do begin$	$O(nm)$ (۲)
$for j := 1 to m do$	$O(nm + n^2)$ (۳)
$k := k + 1;$	$O(nm + n \log n)$ (۴)
$j := 1 ;$	
$while j < n do begin$	
$k := k + 1;$	
$j := 2j;$	
end	
$end ;$	

۱۳- هزینه اجرای تابع زیر چه اندازه است؟

$i := n ;$	$O(\log_2^n)$ (۱)
$while (i > 1) do$	$O(n)$ (۲)
$begin$	$O(n \log_2^n)$ (۳)
$i := i \bmod 2;$	$O(1)$ (۴)
$write(i) ;$	
$end;$	

۱۴- گزینه درست را انتخاب کنید. (علوم کامپیوتر ۸۰)

$$(۱) \quad f(n) \in O(g(n)), h(n) \in \Omega(g(n)) \Rightarrow f(n) \in \theta(h(n))$$

$$(۲) \quad f(n) \in \theta(g(n)), g(n) \in O(h(n)) \Rightarrow h(n) \in O(f(n))$$

$$(۳) \quad f(n) \in \Omega(g(n)), h(n) \in O(g(n)) \Rightarrow f(n) \in \theta(g(n))$$

$$(۴) \quad f(n) \in \Omega(g(n)), g(n) \in \theta(h(n)) \Rightarrow f(n) \in \Omega(h(n))$$

۱۵- کدام یک از عبارات زیر صحیح است؟ (علوم کامپیوتر ۸۱)

$$(۱) \quad \sum_{i=0}^n i^2 = O(n^3) \quad (۲) \quad 3^n = O(2^n)$$

$$(۳) \quad n^2 \log n = O(n^2) \quad (۴) \quad \frac{n^2}{\log n} = O(n^2)$$

۱۶- کدام یک از روابط ذیل درست است؟ (آزاد ۸۱)

$$(۱) \quad O(\log n) < O(\sqrt{n}) \quad (۲) \quad O(\log n) > O(\sqrt{n})$$

$$(۳) \quad O(n!) < O(a^n) \quad (۴) \quad O(\sqrt{n^3}) < O(n)$$

۱۷- فرض کنید f و g ، دو تابع دلخواه به شکل $f, g: N \rightarrow R^+$ ، به علاوه فرض کنید

کدام یک از گزاره‌های زیر درست است؟ (مهندسی کامپیوتر دولتی ۸۲) $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = +\infty$

$$(۱) \quad g(n) \in O(f(n)) \text{ و } f(n) \in O(g(n)) \quad (۲) \quad f(n) \in O(g(n)) \text{ و } g(n) \notin \Omega(f(n))$$

$$(۳) \quad f(n) \in \theta(g(n)) \text{ و } g(n) \notin \Omega(f(n)) \quad (۴) \quad f(n) \in \Omega(g(n)) \text{ و } f(n) \notin \theta(g(n))$$

۱۸- کدام عبارت صحیح است؟ (علوم کامپیوتر سراسری ۸۲)

$$(۱) \quad (n+1)(n^2 - 2n + 1) \in O(2^n) \quad (۲) \quad (n+1)(n^2 - 2n + 1) \in \theta(n)$$

$$(۳) \quad (n+1)(n^2 - 2n + 1) \in \Omega(n^4) \quad (۴) \quad (n+1)(n^2 - 2n + 1) \in O(n^2 \log n)$$

۱۹- کدام گزینه صحیح می‌باشد؟ (علوم کامپیوتر سراسری ۸۴)

$$(۱) \quad 0 < \xi < 1 \quad n^2 \log n = O(n^{2+\xi})$$

$$(۲) \quad \sqrt{n} = (\log n)$$

$$(۳) \quad 0 < \xi < 1 \quad n^{1+\xi} = O(n \log n)$$

$$(۴) \quad n^2 = O\left(\frac{n^2}{\log n}\right)$$

۲۰- کدام یک از گزینه‌های زیر صحیح است؟ (طراحی الگوریتم IT ۸۴)

$$n^2 \sin n \in O(n) \quad (۲) \quad n^2 \sin n \in \Omega(n) \quad (۱)$$

$$n \in \Omega(n^2 \sin n) \quad (۴) \quad n \in \Theta(n^2 \sin n) \quad (۳)$$

۲۱- در رشد توابع زیر کدام ترتیب صحیح می‌باشد؟ ($\varepsilon > 0$) (علوم کامپیوتر ۸۵)

$$O(n \log n) \text{ و } O(1 + \varepsilon)^n \text{ و } O\left(\frac{n^2}{\log n}\right) \quad (۱)$$

$$O(1 + \varepsilon)^n \text{ و } O(n \log n) \text{ و } O\left(\frac{n^2}{\log n}\right) \quad (۲)$$

$$O\left(\frac{n^2}{\log n}\right) \text{ و } O(n \log n) \text{ و } O(1 + \varepsilon)^n \quad (۳)$$

$$O(n \log n) \text{ و } O\left(\frac{n^2}{\log n}\right) \text{ و } O(1 + \varepsilon)^n \quad (۴)$$

۲۲- کدام یک از عبارات زیر درست‌اند: (دولتی ۸۵)

$$e^{c\sqrt{n}} = O(e^{\sqrt{n}}). I \quad C \geq 1 \quad \text{فقط II} \quad (۱)$$

$$n^2 = O(n \lg n). II \quad \text{فقط III} \quad (۲)$$

$$n = O(n \lg n). III \quad II \text{ و } I \quad (۳)$$

$$n = O(n \lg n). III \quad III \text{ و } I \quad (۴)$$

۲۳- کدام یک از عبارت‌های زیر نادرست است؟ (IT ۸۵)

$$O(n!) = O(n^n) \quad (۱)$$

$$O(1.6^n) < O(n) < O(n \cdot \log_2 n) \quad (۲)$$

$$\text{هر الگوریتم از مرتبه } \theta(n) \text{ از مرتبه } O(n^2) \text{ نیز هست.} \quad (۳)$$

$$\text{حذف عنصر آخر در یک لیست زنجیره‌ای یک طرفه، با داشتن اشاره گرهای } first \text{ و } last \text{ از مرتبه } O(1) \text{ است.} \quad (۴)$$

۲۴- کامپیوتری در واحد زمان مسأله‌ای به اندازه ۱۶ را که الگوریتم آن از مرتبه زمانی n^2 است حل می‌کند. اگر سرعت کامپیوتر ۱۳۱۰۷۲ برابر گردد این کامپیوتر همان مسأله را با چه اندازه‌ای در واحد زمان حل خواهد کرد؟

(دولتی ۸۶)

$$16 \times 17 \times \log 17 \quad (2) \quad 16 + 17 + \log 17 \quad (1)$$

$$16 + \log 131072 \quad (4) \quad 32 \quad (3)$$

۲۵- کدام گزینه صحیح است؟ (۱/۲ یعنی 1.2)

(علوم کامپیوتر ۸۹)

$$\frac{n}{\lg n} = O(n^{1-x}), 3^n = \Omega(n \cdot 2^n) \quad (1)$$

$$n \left(\log_3^n \right)^\Delta = \Omega(n^{1/2}), \sqrt{n} = O\left((\lg n)^\Delta\right) \quad (2)$$

$$\frac{n}{\lg n} = \Omega(n^{1-x}) \quad 0 < x < 1, 3^n = O(n 2^n) \quad (3)$$

$$n \left(\log_3^n \right)^\Delta = O(n^{1/2}), \sqrt{n} = \Omega\left((\lg n)^\Delta\right) \quad (4)$$

۲۶- میزان زمان لازم برای اجرای قطعه برنامه زیر چگونه برآورد می‌شود؟

(۸۳ IT)

$$\text{for } i \leftarrow 1 \text{ to } n \quad O(n^2) \quad (1)$$

$$\text{for } j \leftarrow n \text{ to } i \quad \theta(n^3) \quad (2)$$

$$\text{for } k \leftarrow 1 \text{ to } n^2 \quad \omega(n^4) \quad (3)$$

$$\text{sum} \leftarrow \text{sum} + A \quad \theta(n^4) \quad (4)$$

۲۷- کدام گزینه مرتبه زمانی کد زیر را نشان می‌دهد؟

(۹۰ و ۸۷ - IT)

$$\text{for } (i = 1; i \leq n; i++) \quad \theta(n^2) \quad (2) \quad \theta(n) \quad (1)$$

$$\text{for } (j = 1; j \leq n; j = j + i) \quad \theta(n^2 \log n) \quad (4) \quad \theta(n \log n) \quad (3)$$

$x++;$

۲۸- در کد زیر دستور $x++$ چند بار تکرار می‌شود فرض کنید $n \geq 3$ است.

(۸۷ - IT)

$$\text{for } (i = 3; i \leq n; i = i^*)$$

$x++;$

$$\left\lceil \frac{(\log n + 1)}{3} \right\rceil \quad (4) \quad \left\lceil \log \frac{n}{3} + 1 \right\rceil \quad (3) \quad \left\lceil \frac{(\log n)}{3} \right\rceil \quad (2) \quad \left\lceil \log \frac{n}{3} \right\rceil \quad (1)$$

(۸۸ - IT)

مرتبه زمانی شبه کد زیر چیست؟ -۲۹

$$\text{for}(i = 1; i \leq n; i = i^2)$$
(۱) $\theta(n)$

$$\text{for}(j = 1; j \leq n; j = j^2)$$
(۲) $\theta(n^2)$

$$\text{for}(k = 1; k \leq j; k++)$$
(۳) $\theta(n \log n)$

$$x++;$$
(۴) $\theta\left(n(\log n)^2\right)$

۳۰- در یک زمستان سرد، خرس قطبی n قطعه گوشت دقیقاً به اندازه‌های ۱، ۲ تا n را در غاری ذخیره کرده است. او هر روز یکی از این قطعه گوشت‌ها را به صورت تصادفی انتخاب می‌کند. اگر اندازه‌ی گوشت عدد فردی بود، آن را کاملاً می‌خورد. اگر زوج بود، آن را دقیقاً نصف می‌کند، یک نصف آن را می‌خورد و نصف دیگر را مجدداً در غار قرار می‌دهد. اگر گوشتی موجود نباشد، خرس می‌میرد. با این الگوریتم، برای n های خیلی بزرگ روزهای باقیمانده از عمر خرس ما تابع کدام یک از گزینه‌ها خواهد بود؟ (مهندسی کامپیوتر ۸۹)

(۱) $\theta(n)$ (۲) $\theta(\log n)$ (۳) $\theta(n \log n)$ (۴) $\theta(n^2)$

۳۱- توابع $f(n) = 4^{\lg n}$ ، $g(n) = l g^{\lg n} n$ و $h(n) = \lg^2 n$ را در نظر بگیرید. کدام یک

(مهندسی کامپیوتر ۸۹)

از گزاره‌های زیر صحیح است؟

(۱) $f(n) \in O(g(n))$, $f(n) \in \Omega(h(n))$ (۲) $f(n) \in \Theta(h(n))$, $g(n) \in \Omega(f(n))$ (۳) $g(n) \in \Omega(h(n))$, $h(n) \in \Omega(f(n))$ (۴) $h(n) \in O(g(n))$, $f(n) \in \Theta(g(n))$

[پاسخنامه سؤال‌های طبقه‌بندی شده فصل اول]

(۱) گزینه (۱). لازم به ذکر است که تعریف متغیر به شرطی جمله اجرایی است که مقدار اولیه گرفته باشد:

تعداد اجرا	جمله
۱	$temp=0$
$n+1$	for
n	$temp+=num[i]$
۱	$return$
$۲n+۳ = \text{جواب}$	

(۲) گزینه (۲).

$$\sum_{k=0}^{n-1} \sum_{i=1}^{n-k} 1 = \sum_{k=0}^{n-1} (n-k) = \sum_{k=0}^{n-1} n - \sum_{k=0}^{n-1} k = n^2 - \frac{n(n-1)}{2} = \frac{n(n+1)}{2}$$

(۳) گزینه (۲). حلقه for , N بار و حلقه $while$, $\log_2^N$ بار اجرا می‌شود. می‌توان به ازای $N=1$ بررسی کرد که در این صورت گزینه ۲ بدست می‌آید. البته باید فرض کنیم که N توانی از ۲ است.

(۴) گزینه (۴). حلقه $while$ داخلی $\log_2^n + 1$ بار اجرا می‌شود و حلقه بیرونی n بار اجرا می‌شود.

(۵) گزینه (۴).

A	۳۰	۱۵	۷	۳	۱	۰
B	۵	۱۰	۲۰	۴۰	۸۰	۱۶۰
P	۰	۰	۱۰	۳۰	۷۰	۱۵۰
$A \bmod ۲$	۰	۱	۱	۱	۱	

(۶) گزینه (۱). حلقه $while$ دارای مرتبه $\log_2^N$ ، حلقه for , N و داخل حلقه for , M است.

(۷) گزینه (۴). اگر k بر اعداد ۲ تا n بخش پذیر باشد آنگاه حلقه for داخلی نیز هر بار اجرا می‌شود و چون هر دو حلقه به n وابسته اند، زمان n^2 می‌شود ولی اگر k بر هیچ یک از اعداد ۲ تا n بخش پذیر نباشد فقط حلقه بیرون اجرا می‌شود که مرتبه آن n است.

(۸) گزینه (۲). به ازای $i=1$ ، جمله $++x$ (جمله اصلی)، n بار اجرا می‌شود. به ازای $i=2$ ، $n-1$ بار (دقت کنید n یک واحد کم شده است). به همین ترتیب به ازای هر

افزایش i مقدار n کم می‌شود وقتی به $\frac{n}{4}$ برسیم کار تمام است. بنابراین تعداد اجرای جمله

$$x + \dots + (n-1) + (n-2) + \dots + \left(\frac{n}{4}\right) = \theta(n^2)$$

گزینه (۱). به ازای $i=1$ ، جمله $x + \dots$ به تعداد $\frac{n}{4}$ بار اجرا می‌شود زیرا به ازای هر

j یکبار $n - j$ انجام می‌شود. به ازای $i=2$ جمله اصلی به تعداد $\frac{n}{4}$ بار اجرا

$$\frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \dots = n\left(\frac{1}{1-\frac{1}{2}}\right) = \theta(n)$$

گزینه (۳). حلقه for داخلی، n بار. حلقه $while$ ، $\lceil \log n \rceil$ و حلقه for بیرونی n بار اجرا می‌شوند.

گزینه (۲). توابع نمایی مثل $(1.05)^n$ از چند جمله‌ای مثل n^{1000} رشد بیشتری دارند.

گزینه (۴). حلقه $while$ دارای مرتبه $O(\log^n)$ و حلقه for داخلی دارای مرتبه $O(m)$

می‌باشد و چون این دو حلقه متوالی هستند دارای مرتبه $O(m + \log^n)$ می‌باشند

بنابراین کل برنامه دارای مرتبه $O(n(m + \log^n))$ می‌باشد.

گزینه (۴). حلقه $while$ به ازای هر n ، تعداد محدودی اجرا می‌شود، چون حاصل $mod 2$ یا ۰ یا ۱ است.

گزینه (۴). $f \in \Omega(g)$ یعنی رشد f بیشتر یا مساوی g است. $g \in \theta(h)$ یعنی g با h هم رشد است. پس رشد f بیشتر یا مساوی h است یعنی $f \in \Omega(h)$

گزینه (۴ و ۱). رشد $\frac{n^2}{\log n}$ از n^2 کمتر است زیرا n^2 از $n^2 \log n$ کمتر است پس

$$\frac{n^2}{\log n} = O(n^2) \text{ ولی اگر } O \text{ را به معنی } \theta \text{ فرض کنیم گزینه ۱ صحیح است.}$$

گزینه (۱). نرخ رشد تابع $\sqrt{n}$ سریع‌تر از $\log n$ است. زیرا n^a از $(\log n)^b$ رشد بیشتری دارد ($a, b > 0$)

گزینه (۳). توجه کنید فرض مسئله نشان می‌دهد نرخ رشد تابع $f(n)$ سریعتر از تابع $g(n)$ است. پس $f(n) \in \Omega(g(n))$ بوده ولی $f(n) \notin \theta(g(n))$.

(۱۸) گزینه (۱). تابع $(n+1)(n^2-2n+1)$ از مرتبه $\theta(n^3)$ است پس می‌توان گفت از مرتبه $O(2^n)$ است.

(۱۹) گزینه (۱). می‌دانیم لگاریتم به هر توان ثابتی از n به هر توان مثبتی، رشد کمتری دارد یعنی $(Lgn)^a < n^b$ پس $Lgn < n^\varepsilon$ ($\varepsilon > 0$) حال طرفین را در n^3 ضرب می‌کنیم $n^3 Lgn < n^{3+\varepsilon}$ در نتیجه $n^3 Lgn = O(n^{3+\varepsilon})$. در مورد سایر گزینه‌ها دقت کنید:

$$\sqrt{n} > Lgn, \quad n^{1+\varepsilon} > n Lgn, \quad n^2 > \frac{n^2}{Lgn}$$

(۲۰) گزینه (۱). اگر n طبیعی فرض شود آنگاه $n^2 \sin n \geq Cn$ به ازای حداقل یک C و n_0 مثبت و همه $n \geq n_0$ صحیح است. البته $\sin n$ باید داخل قدر مطلق باشد.

(۲۱) گزینه (۴). رشد $n \log n$ کمتر از $\frac{n^2}{\log n}$ و کمتر از $(1+\varepsilon)^n$ است (از حد استفاده کنید)

(۲۲) گزینه (۲). مرتبه $e^{c\sqrt{n}}$ از $e^{\sqrt{n}}$ بیشتر است و n^2 از $n \log n$ بیشتر است.

(۲۳) گزینه (۴). گزینه ۴ نیاز به پیمایش دارد تا آدرس گره ماقبل آخر بدست آید که مرتبه آن $O(n)$ می‌شود. البته گزینه ۱ هم اشکال دارد و باید به صورت $n! = O(n^n)$ نوشته شود.

(۲۴) گزینه (۳). (دقت کنید $2^{17} = 131072$) وقتی گفته می‌شود مرتبه زمانی n^{2^n} است می‌توان فرض کرد زمان اجرا cn^{2^n} است.

$$c \times 16 \times 2^{16} = 1 \Rightarrow c = \frac{1}{2^{20}}, \quad \frac{1}{2^{20}} \times \frac{1}{131072} \times n \times 2^n = 1 \Rightarrow n = 32$$

(۲۵) گزینه (۴).

رشد $\sqrt{n}$ از $\lg n$ به هر توانی بیشتر است.

رشد $\log_p^n$ به هر توانی از $n^{o/2}$ کمتر است.

(۲۶) گزینه (۴). حلقه اول و دوم به n و حلقه داخلی به n^2 وابسته‌اند. در حلقه دوم to به معنی *down to* است!

۲۷) گزینه (۳).

i	j	تعداد اجرای $x++$
۱	n و ... و ۲ و ۱	n
۲	$n-1$ و ... و ۳ و ۱	$\frac{n}{2}$
۳	$n-2$ و ... و ۴ و ۱	$\frac{n}{3}$
$\vdots$		
n	۱	$\frac{n}{n}$

$$= n + \frac{n}{2} + \frac{n}{3} + \dots + \frac{n}{n} = n \left(1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} \right) \approx n.Lnn = \theta(n \cdot \lg n)$$

توجه: ثابت می‌شود $\frac{1}{2} + \dots + \frac{1}{n} < Lnn < 1 + \frac{1}{2} + \dots + \frac{1}{n-1}$

۲۸) گزینه (۳). به ازای $n=3$ ، یک بار اجرا می‌شود. و به ازای $n=6$ دوبار که فقط در گزینه ۳ صدق می‌کنند.

۲۹) گزینه (۳). حلقه i را نادیده بگیرید آنگاه تعداد اجرای دو حلقه دیگر برابر $\theta(n)$ است زیرا $2^0 + 2^1 + 2^2 + \dots + 2^m = 2^{m+1} - 1 = \theta(n)$ می‌باشد. پس مرتبه کل الگوریتم $\theta(n \cdot \log n)$ است زیرا حلقه i از مرتبه $\theta(\log n)$ است.

۳۰) گزینه (۱). n قطعه گوشت، خرس را n روز زنده نگه می‌دارد. $\frac{n}{2}$ از این قطعات زوج است که این قطعات مجدداً ذخیره می‌شوند که در نتیجه $\frac{n}{4}$ روز دیگر خرس زنده می‌ماند. $\frac{n}{4}$ از این قطعات زوج است که در نتیجه باقی می‌مانند و $\frac{n}{8}$ روز دیگر خرس زنده می‌ماند. به همین ترتیب تعداد روزهای باقیمانده از عمر خرس برابر است با:

$$n + \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \dots = n \left(1 + \frac{1}{2} + \frac{1}{4} + \dots \right) = n \times \frac{1}{1 - \frac{1}{2}} = 2n$$

۳۱) گزینه (۱). ترتیب رشد توابع به شکل زیر است:

$$\lg^2 n < 4^{\lg n} = n^2 < \lg^{\lg n} n$$

$$4^{\lg n} \in O(\lg^{\lg n} n) \quad , \quad 4^{\lg n} \in \Omega(\lg^2 n)$$

پس

توجه: اگر لگاریتم توابع گفته شده را بگیرید، به همین نتیجه می‌رسید. ولی دقت کنید که اگر تابع f از تابع g رشد کمتری داشته باشد ممکن است lff با lgg هم رشد شود.

الگوریتم‌های بازگشتی

فصل ۲

الگوریتم بازگشتی خود را فراخوانی می‌کند. با یک مثال وارد بحث می‌شویم.

مثال: با توجه به تابع بازگشتی f

```
f(n)
{
    if(n ≤ ۱) return n * n;
    return f(n - ۱) + f(n - ۱) + n;
}
```

الف) $f(۴)$ چند است؟

ب) $f(۴)$ چه تعداد فراخوانی دارد؟

ج) $f(۴)$ چه تعداد عمل + را انجام می‌دهد؟

د) $f(۴)$ چه تعداد عمل * انجام می‌دهد؟

هـ) اگر $T(n)$ تعداد فراخوانی $f(n)$ باشد برای $T(n)$ فرمول بازگشتی بنویسید.

و) اگر $T(n)$ تعداد عمل + برای $f(n)$ باشد برای $T(n)$ فرمول بازگشتی بنویسید.

ز) اگر $T(n)$ تعداد عمل * برای $f(n)$ باشد برای $T(n)$ فرمول بازگشتی بنویسید.

ح) اگر $T(n)$ زمان اجرای تابع $f(n)$ باشد برای $T(n)$ فرمول بازگشتی بنویسید.

پاسخ: الف)

$$f(۱) = ۱, \quad f(۲) = f(۱) + f(۱) + ۲ = ۴,$$

$$f(۳) = f(۲) + f(۲) + ۳ = ۱۱, \quad f(۴) = f(۳) + f(۳) + ۴ = ۲۶$$

ب) $f(4)$ دو بار $f(3)$ را صدا می‌زند. هر $f(3)$ دو بار $f(2)$ را صدا می‌زند. هر $f(2)$ دو بار $f(1)$ را صدا می‌زند پس تعداد فراخوانی جمعاً $15 = 1 + 2 + 4 + 8$ است.

ج) به ازای هر $n > 1$ دو بار عمل + انجام می‌شود. پس تعداد عمل + برابر $14 = 1 + 2 + 4 + 8$ است.

د) به ازای هر $n \leq 1$ یک بار عمل * انجام می‌شود. و چون ۸ بار $f(1)$ فراخوانی می‌شود پس ۸ بار عمل * انجام می‌شود.

ه) هر فراخوانی $f(n)$ دو بار $f(n-1)$ را فراخوانی می‌کند پس $T(n) = 2T(n-1) + 1$.
 $T(n-1)$ بخاطر $f(n-1)$ و عدد ۱ همان فراخوانی اول است. این رابطه بازگشتی مرتبه یک است (یعنی $T(n)$ فقط به $T(n-1)$ وابسته است) و یک شرط اولیه می‌خواهد که $T(1) = 1$ چون به ازای $n = 1$ یک فراخوانی انجام می‌شود.

و) هر فراخوانی $f(n)$ که $n > 1$ دو بار عمل + دارد و دو بار فراخوانی $f(n-1)$ پس $T(n) = 2T(n-1) + 2$ و شرط اولیه $T(1) = 0$ چون به ازای $n = 1$ اصلاً عمل + ندارد.

ز) هر فراخوانی $f(n)$ که $n > 1$ اصلاً عمل * ندارد ولی دو فراخوانی $f(n-1)$ دارد پس $T(n) = 2T(n-1)$ و شرط اولیه $T(1) = 1$ چون به ازای $n = 1$ یک عمل * انجام می‌شود.

ح) زمان اجرا برابر تعداد اجرای همه دستورات است ولی می‌توان زمان اجرا را برابر تعداد اجرای عمل اصلی دانست. عمل اصلی در تابع f می‌تواند تعداد انجام عمل + باشد یا تعداد اجرای if یا تعداد فراخوانی‌ها. اگر زمان را برابر تعداد اجرای if فرض کنیم می‌توان نوشت:

$T(n) = 2T(n-1) + 1$ و $T(1) = 1$. می‌تون ثابت کرد که $T(n) = 2^n - 1$ می‌شود و از مرتبه $\theta(2^n)$ است.

◀ نکته: اگر $T(n) = a \cdot T(n-b) + c$ که a, b, c ثابت هستند آنگاه اگر $a \neq 1$ آنگاه

$T(n) = \theta(a^{\frac{n}{b}})$ و اگر $a = 1$ و $c \neq 0$ آنگاه $T(n) = \theta(n)$. برای اثبات این نکته، حل روابط بازگشتی را بخوانید

مثال: زمان اجرای تابع مقابل از چه مرتبه‌ای است؟

```
g(n)
{
    if (n ≤ 1) return n * n;
    return 2 * g(n-1) + n;
}
```

پاسخ: اگر $T(n)$ را برابر تعداد اجرای if فرض کنیم آنگاه $T(n) = T(n-1) + 1$ و $T(1) = 1$ که $T(n) = \theta(n)$ یعنی مرتبه خطی است. دقت کنید خروجی این تابع با تابع مثال قبل همواره مساوی است ولی زمان اجرای این دو تابع متفاوت است.

حل روابط بازگشتی

برای یافتن مرتبه الگوریتم‌های بازگشتی ابتدا باید عمل اصلی آن الگوریتم را مشخص کنیم سپس برای آن رابطه بازگشتی بنویسیم و سپس رابطه بازگشتی را حل کنیم (البته خیلی مواقع با نکات می‌توان بدون حل مرتبه را یافت) و مرتبه اجرایی را بیابیم. روش‌های حل روابط بازگشتی متنوع هستند که در درس ساختمان گسسته تشریح می‌شوند. در این درس خلاصه‌ای از روش‌های حل را ارائه می‌دهیم. ابتدا حل روابط بازگشتی به فرم

$$a_n = c_1 \cdot a_{n-1} + c_2 \cdot a_{n-2} + \dots + c_k \cdot a_{n-k} + b_1^n \cdot p_1(n) + b_2^n \cdot p_2(n) + \dots$$

که رابطه مرتبه k است را بررسی می‌کنیم. در این رابطه b_i ها ثابت هستند و $p_i(n)$ ها چند جمله‌ای درجه d_i ($p_1(n)$ چند جمله‌ای درجه d_1 است. $p_2(n)$ چند جمله‌ای درجه d_2 است و ...) در ضمن همه b_i ها متمایز هستند. برای حل این رابطه معادله مشخصه تشکیل می‌دهیم که عبارت است از:

$$(x^k - c_1 x^{k-1} - c_2 x^{k-2} - \dots - c_k)(x - b_1)^{d_1+1} (x - b_2)^{d_2+1} \dots = 0.$$

ریشه‌های این معادله را می‌یابیم. اگر دو ریشه حقیقی متمایز x_1 و x_2 وجود داشت. شکل کلی جواب $a_n = \alpha_1 (x_1)^n + \alpha_2 (x_2)^n$ است. اگر سه ریشه حقیقی متمایز x_1 و x_2 و x_3 وجود داشت، شکل کلی جواب $a_n = \alpha_1 (x_1)^n + \alpha_2 (x_2)^n + \alpha_3 (x_3)^n$ است. اگر ریشه مضاعف $x_1 = x_2$ داشت، شکل جواب $a_n = (\alpha_1 + \alpha_2 n)(x_1)^n$ است. اگر ریشه ۳ گانه $x_1 = x_2 = x_3$ داشت شکل جواب $a_n = (\alpha_1 + \alpha_2 n + \alpha_3 n^2)(x_1)^n$ است و به همین ترتیب (α_i ها ثابت هستند که با توجه به شروط اولیه محاسبه می‌شوند)

مثال: رابطه بازگشتی فیبوناچی $F_n = F_{n-1} + F_{n-2}$ ، $F_0 = 0$ و $F_1 = 1$ یک رابطه

مرتبه ۲ است. این رابطه همگن است (یعنی $b_i^n p_i(n)$ ندارد) برای حل، معادله مشخصه تشکیل می‌دهیم که $x^2 - x - 1 = 0$ است. ریشه‌های این معادله $x_1 = \frac{1+\sqrt{5}}{2}$ و $x_2 = \frac{1-\sqrt{5}}{2}$

می‌باشد پس شکل کلی جواب $F_n = \alpha_1 \left(\frac{1+\sqrt{\delta}}{2}\right)^n + \alpha_2 \left(\frac{1-\sqrt{\delta}}{2}\right)^n$ است. برای یافتن α_1 و α_2 ، شروط اولیه را اعمال می‌کنیم:

$$\begin{cases} F_0 = \alpha_1 \left(\frac{1+\sqrt{\delta}}{2}\right)^0 + \alpha_2 \left(\frac{1-\sqrt{\delta}}{2}\right)^0 = 0 \\ F_1 = \alpha_1 \left(\frac{1+\sqrt{\delta}}{2}\right)^1 + \alpha_2 \left(\frac{1-\sqrt{\delta}}{2}\right)^1 = 1 \end{cases}$$

از معادله اول $\alpha_1 + \alpha_2 = 0$ پس $\alpha_2 = -\alpha_1$ که در معادله دوم جایگزین می‌کنیم:

$$\alpha_1 \left(\frac{1+\sqrt{\delta}}{2}\right) - \alpha_1 \left(\frac{1-\sqrt{\delta}}{2}\right) = 1 \rightarrow \alpha_1 = \frac{1}{\sqrt{\delta}}$$

پس $\alpha_2 = \frac{-1}{\sqrt{\delta}}$ در نتیجه:

$$F_n = \frac{1}{\sqrt{\delta}} \left(\frac{1+\sqrt{\delta}}{2}\right)^n - \frac{1}{\sqrt{\delta}} \left(\frac{1-\sqrt{\delta}}{2}\right)^n = \theta \left(\left(\frac{1+\sqrt{\delta}}{2}\right)^n\right)$$

مثال: رابطه $a_n = a_{n-1} + n$ و $a_0 = 0$ مرتبه یک و ناهمگن است. برای حل ابتدا دقت

کنید این رابطه به شکل $a_n = a_{n-1} + 1^n(n)$ است که $b_1 = 1$ و $d_1 = 1$ پس معادله مشخصه

$0 = (x-1)(x-1)^2$ است. سه ریشه مساوی $1 = x_1 = x_2 = x_3$ دارد پس

$a_n = (\alpha_1 + \alpha_2 n + \alpha_3 n^2)(1)^n$. برای یافتن α_1 و α_2 و α_3 نیاز به ۳ شرط اولیه داریم. از

رابطه $a_n = a_{n-1} + n$ و شرط $a_0 = 0$ می‌توان a_1 و a_2 را محاسبه کرد:

$$a_1 = a_0 + 1 = 1 \text{ و } a_2 = a_1 + 2 = 3$$

حال a_0 و a_1 و a_2 را اعمال می‌کنیم:

$$\begin{cases} a_0 = (\alpha_1 + 0 + 0)(1)^0 = 0 \\ a_1 = (\alpha_1 + \alpha_2 + \alpha_3)(1)^1 = 1 \\ a_2 = (\alpha_1 + 2\alpha_2 + 4\alpha_3)(1)^2 = 3 \end{cases}$$

با حل این دستگاه $\alpha_1 = 0$ و $\alpha_2 = \frac{1}{2}$ و $\alpha_3 = \frac{1}{6}$ پس $a_n = \frac{n(n+1)}{2} = \theta(n^2)$

مثال: با توجه به رابطه بازگشتی $T(n) = 4T(n-2) + 2^n + n$ فقط مرتبه $T(n)$ را با

نماد θ بیان کنید.

پاسخ: این رابطه ناهمگن مرتبه ۲ است و به شکل

$$T(n) = T(n-1) + 4T(n-2) + 2^n(1) + 1^n(n)$$

است پس $b_1 = 2$ و $d_1 = 0$ و $b_2 = 1$ و $d_2 = 1$ بنابراین معادله مشخصه عبارت است از:

$$(x^2 - 4)(x - 2)^1(x - 1)^2 = 0$$

ریشه‌های این معادله -۲ و ۲ و ۱ و ۱ می‌باشد پس شکل کلی جواب

$$T(n) = \alpha_1(-2)^n + (\alpha_2 + \alpha_3 n)(2)^n + (\alpha_4 + \alpha_5 n)(1)^n$$

اگر $\alpha_3 \neq 0$ مرتبه این رابطه $T(n) = \theta(n \cdot 2^n)$ است.

مثال: زمان اجرای تابع زیر از چه مرتبه‌ای است؟

```
f(n)
{
    if(n ≤ 1) return n;
    return f(n-1) * f(n-2) + n;
}
```

پاسخ: زمان اجرای تابع f را برابر تعداد اجرای if در نظر می‌گیریم و با $T(n)$ نشان می‌دهیم

پس $T(n) = T(n-1) + T(n-2) + 1$ و $T(0) = 1$ و $T(1) = 1$. که با حل این رابطه

$$T(n) = \theta\left(\left(\frac{1+\sqrt{5}}{2}\right)^n\right) \text{ حاصل می‌شود که نمایی است درواقع } 2^{\frac{n}{2}} < T(n) < 2^n \text{ می‌باشد.}$$

تاکنون روابط بازگشتی خطی را حل کردیم یعنی روابطی که a_n به a_{n-1} و a_{n-2} و ... وابسته هست. حال می‌خواهیم نحوه بیان مرتبه سایر روابط بازگشتی را ارائه دهیم.

قضیه Master: فرض کنید $a \geq 1$ و $b > 1$ ثابت هستند و $f(n)$ یک تابع است. با توجه به

رابطه بازگشتی $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ که $\frac{n}{b}$ می‌تواند به شکل $\left\lfloor \frac{n}{b} \right\rfloor$ یا $\left\lceil \frac{n}{b} \right\rceil$ باشد.

برای یافتن مرتبه $T(n)$ سه حالت پیش می‌آید:

حالت ۱: اگر $f(n) = O(n^{\log_b^{a-\varepsilon}})$ که $\varepsilon > 0$ ثابت است آنگاه $T(n) = \theta(n^{\log_b^a})$

حالت ۲: اگر $f(n) = \theta(n^{\log_b^a})$ آنگاه $f(n) = \theta(n^{\log_b^a} \cdot \lg n)$

حالت ۳: اگر $f(n) = \Omega(n^{\log_b^{a+\varepsilon}})$ که $\varepsilon > 0$ ثابت است و اگر $af\left(\frac{n}{b}\right) \leq cf(n)$ برای

ثابت $c < 1$ و n ‌های به اندازه کافی بزرگ آنگاه $T(n) = \theta(f(n))$.

توجه: اگر $f(n)$ چند جمله‌ای باشد، قضیه *Master* را می‌توان به زبان ساده اینطور بیان کرد که مرتبه $f(n)$ را با $n^{\log_b^a}$ مقایسه کنید اگر $f(n)$ رشد بیشتری داشت آنگاه $T(n) = \theta(f(n))$. اگر $n^{\log_b^a}$ رشد بیشتری داشت آنگاه $T(n) = \theta(n^{\log_b^a})$ و اگر هم‌رشد بودند آنگاه $T(n) = \theta(f(n) \cdot \lg n)$.

مثال:

$$a) \quad T(n) = ۴T\left(\frac{n}{۲}\right) + n, \quad a = ۴, b = ۲, f(n) = n = O(n^{\log_2^4 - \epsilon})$$

که به ازای مثلاً $\epsilon = ۱$ درست است پس $T(n) = \theta(n^۲)$

$$b) \quad T(n) = ۲T\left(\frac{n}{۲}\right) + n, \quad a = b = ۲, f(n) = n = O(n^{\log_2^2})$$

حالت ۲ برقرار است پس $T(n) = \theta(n \cdot \lg n)$

$$c) \quad T(n) = ۲T\left(\frac{n}{۲}\right) + n^۲, \quad a = b = ۲, f(n) = n^۲ = \Omega(n^{\log_2^2 + \epsilon})$$

که به ازای مثلاً $\epsilon = ۱$ درست است. حال شرط $af\left(\frac{n}{b}\right) \leq cf(n)$ را بررسی می‌کنیم:

$$۲f\left(\frac{n}{۲}\right) = ۲\left(\frac{n^۲}{۴}\right) = \frac{۱}{۲}n^۲ \leq cn^۲$$

که به ازای مثلاً $c = \frac{۱}{۲}$ برقرار است پس $T(n) = \theta(n^۲)$ البته چون $f(n) = n^۲$ چند جمله‌ای است نیاز به بررسی شرط فوق نیست.

$$d) \quad T(n) = ۲T\left(\frac{n}{۲}\right) + n \cdot \lg n, \quad a = b = ۲, f(n) = n \cdot \lg n$$

ظاهراً $n \cdot \lg n = n^{\lg 2} = n$ رشد بیشتری دارد و باید حالت ۳ برقرار باشد ولی

$$n \cdot \lg n = \Omega(n^{\log_2^2 + \epsilon})$$

پس نمی‌توان از قضیه *Master* کمک گرفت.

$$e) \quad T(n) = ۴T\left(\frac{n}{۲}\right) + n \cdot \lg n, \quad a = ۴, b = ۲, f(n) = n \cdot \lg n = O(n^{\log_2^4 - \epsilon})$$

به ازای مثلاً $\epsilon = ۱$ درست است پس $T(n) = \theta(n^۲)$

$$f) \quad T(n) = T\left(\frac{n}{3}\right) + 1, \quad a = 1, b = \frac{3}{2}, f(n) = 1 = \theta(n^{\log_{\frac{3}{2}} 1}) = \theta(1)$$

حالت ۲ برقرار است پس $T(n) = \theta(\lg n)$

$$g) \quad T(n) = 2T\left(\frac{n}{2}\right) + n^2 \cdot \lg n, \quad a = b = 2, f(n) = n^2 \cdot \lg n = \Omega(n^{\log_2 2 + \varepsilon})$$

مثلاً به ازای $\varepsilon = 1$ درست است حال شرط $af\left(\frac{n}{b}\right) \leq cf(n)$ را بررسی می‌کنیم.

$$2f\left(\frac{n}{2}\right) = 2\left(\frac{n}{2}\right)^2 \lg \frac{n}{2} \leq \frac{1}{2} n^2 \lg n$$

پس به ازای $c = \frac{1}{2} < 1$ برقرار است بنابراین حالت ۳ درست است و $T(n) = \theta(n^2 \cdot \lg n)$

$$h) \quad T(n) = 2T\left(\frac{n}{2}\right) + \frac{n}{\lg n}, \quad a = b = 2, f(n) = \frac{n}{\lg n} = O(n^{\log_2 2 - \varepsilon})$$

به ازای هیچ $\varepsilon > 0$ برقرار نیست پس نمی‌توان از قضیه Master استفاده کرد.

◀ **نکته:** اگر $T(n) = aT\left(\frac{n}{b}\right) + n^{\log_b a} \cdot \lg^k n$ و $k \geq 0$ ثابت است آنگاه

$$T(n) = \theta(n^{\log_b a} \cdot \lg^{k+1} n)$$

مثال:

$$a) \quad T(n) = 2T\left(\frac{n}{2}\right) + n \cdot \lg n = \theta(n \cdot \lg^2 n)$$

$$b) \quad T(n) = 4T\left(\frac{n}{2}\right) + \lg^2 n! = \theta(n^2 \cdot \lg^3 n)$$

(دقت کنید $lgn!$ با $nlgn$ هم‌رشد است)

برخی روابط بازگشتی با تغییر متغیر به روابط قابل حل تبدیل می‌شوند.

مثال: برای یافتن مرتبه $T(n) = 3T(\sqrt[3]{n}) + \lg n$ ، تعریف می‌کنیم $n = 2^m$ پس:

$$T(2^m) = 3T(2^{\frac{m}{3}}) + m$$

حال $T(2^m)$ را $F(m)$ می‌نامیم پس:

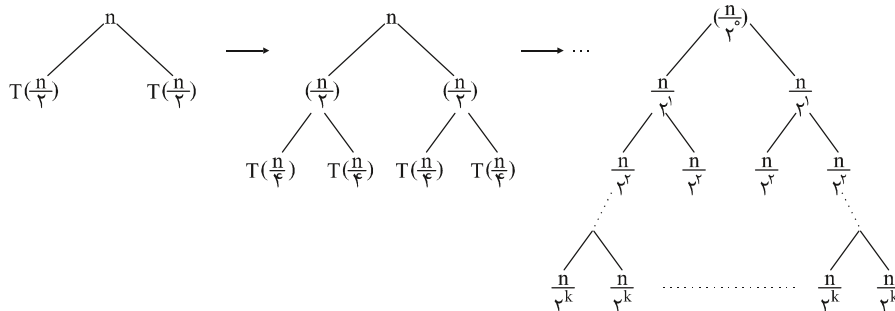
$$F(m) = 3F\left(\frac{m}{3}\right) + m$$

Diagram illustrating the expansion of a recurrence relation $T(n) = T(n/a) + T(n/b) + c$. The first tree shows a root $f(n)$ with two children $T(n/a)$ and $T(n/b)$. An arrow points to a second tree where $f(n/a)$ and $f(n/b)$ are expanded into their own recursive calls: $f(n/a)$ has children $T(n/a^2)$ and $T(n/ab)$, and $f(n/b)$ has children $T(n/ab)$ and $T(n/b^2)$. A final arrow points to an ellipsis, indicating further expansion.

مثال:

$$T(n) = 2T\left(\frac{n}{2}\right) + n = T\left(\frac{n}{2}\right) + T\left(\frac{n}{2}\right) + n$$

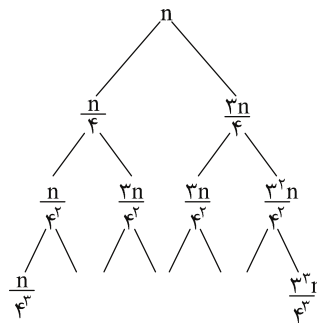
را با درخت می‌یابیم:



درخت وقتی تمام می‌شود که $\frac{n}{2^k} = 1$ یعنی $k = \lg n$ این درخت دارای $k + 1$ سطح است و جمع نودهای هر سطح برابر n است پس جمع تمام نودها $(k + 1)n$ یعنی $(\lg n + 1)n$ است پس $T(n) = \theta(n \cdot \lg n)$

مثال:

$$T(n) = T\left(\frac{n}{4}\right) + T\left(\frac{3n}{4}\right) + n$$



این درخت پر نیست و شاخه سمت چپ زودتر به پایان (به ۱) می‌رسد، طول شاخه سمت چپ $\log_4^n$ و طول شاخه سمت راست $\log_{\frac{4}{3}}^n$ است (دقت کنید شاخه سمت راست به شکل $\frac{3^k}{4^k} n$ حرکت می‌کند و باید $\frac{3^k}{4^k} n = 1$ پس $k = \log_{\frac{4}{3}}^n$). جمع نودهای هر سطح تا سطح $\log_4^n$ ، برابر n است ولی از آن بعد از n کمتر است پس جمع همه نودها $n \cdot \log_{\frac{4}{3}}^n < T(n) < n \cdot \log_4^n$ است پس $T(n) = O(n \cdot \log_{\frac{4}{3}}^n)$ و $T(n) \in \Omega(n \cdot \log_4^n)$ و چون پایه لگاریتم در رشد بی‌اهمیت است پس $T(n) \in \theta(n \cdot \lg n)$

نکته: رابطه بازگشتی $T(n) = T(\alpha n) + T(\beta n) + n$ که α و β ثابت هستند مفروض است، اگر $\alpha + \beta = 1$ آنگاه $T(n) = \theta(n \cdot \lg n)$ و اگر $\alpha + \beta < 1$ آنگاه $T(n) = \theta(n)$

مثال: رابطه بازگشتی $T(n) = T(\frac{n}{5}) + T(\frac{4n}{10}) + n$ از مرتبه $T(n) = \theta(n)$ است چون $\frac{1}{5} + \frac{4}{10} = \frac{9}{10} < 1$ (با درخت بازگشت اثبات کنید)

نکته: به طور کلی $T(n) = T(\frac{n}{a_1}) + T(\frac{n}{a_2}) + \dots + T(\frac{n}{a_k}) + \theta(n)$ اگر $\sum_{i=1}^k \frac{1}{a_i} = 1$ آنگاه $T(n) = \theta(n \cdot \lg n)$ و اگر $\sum_{i=1}^k \frac{1}{a_i} < 1$ آنگاه $T(n) = \theta(n)$

قضیه بمب اتم! (اختیاری): رابطه بازگشتی $T(n) = \sum_{i=1}^k a_i T(\frac{n}{b_i}) + f(n)$ که k ثابت، $a_i > 0$ ثابت و $b_i > 1$ برای هر i ثابت هستند مفروض است. مرتبه این رابطه بازگشتی از فرمول:

$$T(n) = \theta(n^p (1 + \int_1^n \frac{f(x)}{x^{p+1}} dx))$$

بدست می‌آید که p جواب منحصر به فرد معادله $\sum_{i=1}^k \frac{a_i}{b_i^p} = 1$ است.

مثال:

$$a) \quad T(n) = T(\frac{3n}{4}) + T(\frac{n}{4}) + n$$

$$a_1 = a_2 = 1, \quad b_1 = \frac{4}{3}, \quad b_2 = 4, \quad f(n) = n$$

$$\text{معادله } \frac{a_1}{b_1^p} + \frac{a_2}{b_2^p} = 1 \text{ یعنی } \left(\frac{3}{4}\right)^p + \left(\frac{1}{4}\right)^p = 1 \text{ جواب } p = 1 \text{ دارد.}$$

$$T(n) = \theta(n^1 (1 + \int_1^n \frac{x}{x^2} dx)) = \theta(n \cdot \lg n) \quad \text{پس:}$$

توجه کنید:

$$\int_1^n \frac{x}{x^2} dx = \int_1^n \frac{1}{x} dx = \ln n$$

$$b) \quad T(n) = T(\frac{n}{5}) + T(\frac{4n}{10}) + n$$

معادله $\left(\frac{1}{5}\right)^p + \left(\frac{4}{10}\right)^p = 1$ جواب مشخصی ندارد ولی جوابش حتماً $1 < p < \infty$ می‌باشد زیرا $\frac{1}{5} + \frac{4}{10} < 1$ است. حال انتگرال را محاسبه می‌کنیم.

$$\begin{aligned} \int_1^n \frac{f(x)}{x^{p+1}} dx &= \int_1^n \frac{x}{x^{p+1}} dx = \int_1^n x^{-p} dx = \frac{1}{1-p} x^{1-p} \Big|_1^n \\ &= \frac{n^{1-p} - 1}{1-p} = \theta(n^{1-p}) \end{aligned}$$

پس:

$$T(n) = \theta(n^p (1 + \theta(n^{1-p}))) = \theta(n^p \cdot n^{1-p}) = \theta(n)$$

$$c) \quad T(n) = \frac{1}{4} T\left(\frac{n}{4}\right) + \frac{3}{4} T\left(\frac{3n}{4}\right) + 1$$

معادله $\frac{1}{4} \left(\frac{1}{4}\right)^p + \frac{3}{4} \left(\frac{3}{4}\right)^p = 1$ جواب $p = \infty$ دارد پس:

$$T(n) = \theta\left(1 + \int_1^n \frac{1}{x} dx\right) = \theta(\lg n)$$

$$d) \quad T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{4}\right) + 1$$

معادله $\left(\frac{1}{2}\right)^p + \left(\frac{1}{4}\right)^p = 1$ دارای جواب $p = \lg\left(\frac{1+\sqrt{5}}{2}\right) \approx 0.69$ است (کافی است

$x = 2^p$ تعریف کنید و معادله درجه ۲ را حل کنید) حاصل انتگرال را می‌یابیم:

$$\int_1^n \frac{1}{x^{p+1}} dx = \frac{1 - n^{-p}}{p} = \theta(1)$$

$$T(n) = \theta(n^p (1 + \theta(1))) = \theta(n^p) \approx \theta(n^{0.69})$$

پس:

مسائل معروف بازگشتی

۱- مجموع ۱ تا n

$$sum(n) = \underbrace{1 + 2 + \dots + n - 1}_{sum(n-1)} + n = sum(n-1) + n, \quad sum(0) = 0.$$

زمان اجرای این تابع $\theta(n)$ است زیرا زمان اجرای این تابع با تعداد فراخوانی‌ها مساوی است

که $\theta(n)$ تاست. دقت کنید خود این تابع $\theta(n^2) = \frac{n(n+1)}{2}$ است زیرا محاسبه

می‌کند ولی این ربطی به زمان اجرای تابع ندارد.

۲- فاکتوریل

$$fact(n) = n! = n * (n-1)! = n * fact(n-1) \text{ , } fact(0) = 1$$

زمان اجرای این تابع نیز $\theta(n)$ است.

۳- توان

$$pow(m, n) = m^n = m * m^{n-1} = m * pow(m, n-1) \text{ , } pow(m, 0) = 1$$

زمان اجرای این تابع نیز $\theta(n)$ است زیرا $\theta(n)$ بار فراخوانی دارد.

البته برای توان می‌توان تابعی با زمان $\theta(\lg n)$ نیز نوشت. برای n های زوج

$$pow(m, n) = (m^{\lfloor \frac{n}{2} \rfloor})^2 * m \text{ و برای } n \text{ های فرد } pow(m, n) = (m^{\frac{n}{2}})^2$$

پس:

$$pow(m, n) = \begin{cases} pow^2(m, \frac{n}{2}) & n \neq 0 \text{ زوج} \\ m * pow^2(m, \lfloor \frac{n}{2} \rfloor) & n \text{ فرد} \\ 1 & n = 0 \end{cases}$$

زمان اجرای این تابع برای n زوج $T(n) = T(\frac{n}{2}) + 1$ و برای n فرد $T(n) = T(\lfloor \frac{n}{2} \rfloor) + 1$

است که $\theta(\lg n)$ می‌باشد.

۴- خارج قسمت تقسیم

برای محاسبه $div(m, n) = \lfloor \frac{m}{n} \rfloor$ می‌توان از تابع زیر کمک گرفت:

$$div(m, n) = \begin{cases} 0 & m < n \\ div(m-n, n) + 1 & m \geq n \end{cases}$$

تعداد فراخوانی این تابع $1 + \lfloor \frac{m}{n} \rfloor$ است. مثلاً برای محاسبه $div(200, 6)$ تعداد فراخوانی

$$1 + \lfloor \frac{200}{6} \rfloor = 34 \text{ است.}$$

۵- باقیمانده تقسیم

برای محاسبه $m \bmod n$ می‌توان از تابع زیر کمک گرفت:

$$\text{mod}(m, n) = \begin{cases} m & m < n \\ \text{mod}(m-n, n) & m \geq n \end{cases}$$

تعداد فراخوانی این تابع نیز $1 + \left\lfloor \frac{m}{n} \right\rfloor$ است.

۶- بزرگترین مقسوم‌علیه مشترک

$$\gcd(m, n) = \begin{cases} m & n = 0 \\ \gcd(n, m \bmod n) & n \neq 0 \end{cases}$$

۷- ترکیب

برای محاسبه ترکیب $\binom{m}{n}$ می‌توان از فرمول پاسکال استفاده کرد $(m \geq n)$ ،

$$\text{comb}(m, n) = \begin{cases} 1 & , m = n \text{ or } n = 0 \\ \text{comb}(m-1, n-1) + \text{comb}(m-1, n), & \text{otherwise} \end{cases}$$

می‌توان بررسی کرد که تعداد فراخوانی $1 - \binom{m}{n} 2$ است و تعداد باری که عمل + انجام

می‌شود $1 - \binom{m}{n}$ است.

۸- برج‌های هانوی

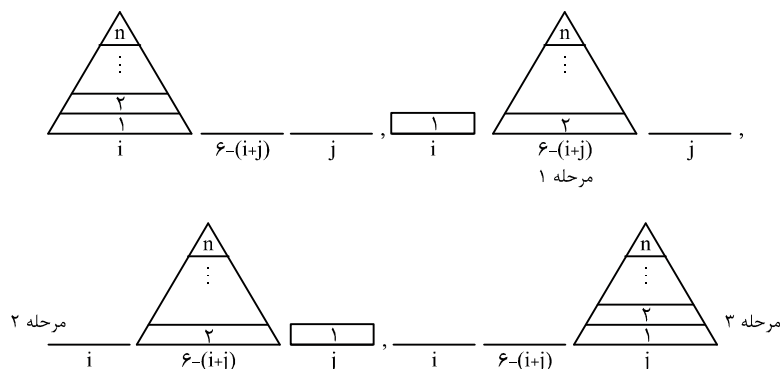
سه میله با شماره‌های ۱ و ۲ و ۳ مفروض است. داخل یکی از میله‌ها n دیسک با اندازه‌های متفاوت از بزرگ به کوچک روی هم قرار گرفته‌اند (دیسک بزرگ پایین است) می‌خواهیم این n دیسک را با کمک یکی از میله‌ها به میله سوم منتقل کنیم به شرطی که اولاً هر بار فقط یک دیسک منتقل شود ثانیاً هیچگاه دیسک بزرگ روی کوچک قرار نگیرد. اگر شماره میله مبدا و مقصد به ترتیب i و j باشد، شماره میله کمکی $6 - (i + j)$ است. برای حل این مسئله باید ۳ مرحله زیر انجام شوند:

(۱) انتقال $n-1$ دیسک از میله i با کمک میله j به میله $6 - (i + j)$

(۲) انتقال یک دیسک از میله i به میله j

(۳) انتقال $n-1$ دیسک از میله $6 - (i + j)$ با کمک میله i به میله j

دقت کنید مراحل ۱ و ۳ به صورت بازگشتی هستند یعنی هر کدام از این مراحل به ۳ مرحله تقسیم می‌شوند.



الگوریتم هانوی را می‌توان به شکل زیر نوشت:

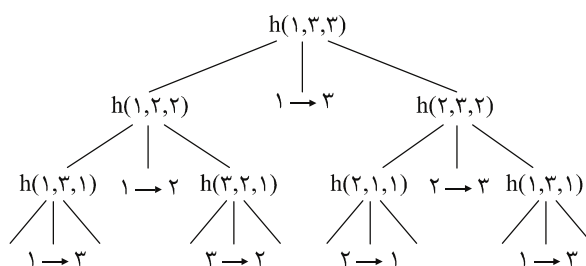
i و j شماره مبدا و مقصد. n تعداد دیسک

$hanoi(i, j, n) //$

```
{
    if( $n > 0$ )
    {
         $hanoi(i, 6 - (i + j), n - 1);$ 
         $write(i \rightarrow j);$ 
         $hanoi(6 - (i + j), j, n - 1);$ 
    }
}
```

به ازای $n = 3$ این الگوریتم را آزمایش می‌کنیم (نام تابع را h می‌گذاریم)

میله مبدا را ۱ و مقصد را ۳ و کمکی را ۲ در نظر می‌گیریم:



حال از چپ به راست، برگ‌ها را چاپ می‌کنیم پس خروجی عبارت است از (چپ به راست)

$1 \rightarrow 3, 1 \rightarrow 2, 3 \rightarrow 2, 1 \rightarrow 3, 2 \rightarrow 1, 2 \rightarrow 3, 1 \rightarrow 3$

$$T(n) = \mathfrak{r}^n - \mathfrak{1} = \theta(\mathfrak{r}^n)$$

تمرین

۱- با هر روشی که می‌شناسید، درستی مرتبه‌های داده شده برای فرمول‌های بازگشتی زیر را بررسی کنید. (فرض کنید شرط اولیه به درستی تعریف شده است)

$$a) \quad T(n) = 2T\left(\frac{n}{2}\right) + \frac{n}{\lg n} = \theta(n \cdot \lg \lg n)$$

$$b) \quad T(n) = aT\left(\frac{n}{a}\right) + \frac{n}{\lg n} = \theta(n \cdot \lg \lg n) \quad (a > 1 \text{ ثابت})$$

$$c) \quad T(n) = T(n-1) + \frac{1}{n} = \theta(\lg n)$$

$$d) \quad T(n) = T\left(\frac{n}{4}\right) + T\left(\frac{3n}{4}\right) + n^2 = \theta(n^2)$$

$$e) \quad T(n) = T\left(\frac{n}{4}\right) + T\left(\frac{n}{2}\right) + n^2 = \theta(n^2)$$

$$f) \quad T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{n}{6}\right) + n^{\sqrt{\lg n}} = \theta(n^{\sqrt{\lg n}})$$

$$g) \quad T(n) = 2^n T(n-1) = \theta(\sqrt{2}^{(2^n + n)})$$

$$h) \quad T(n) = \frac{2}{n} \sum_{k=0}^{n-1} T(k) + n = \theta(n \cdot \lg n)$$

(طرفین را در n ضرب کنید سپس $T(n-1)$ را بیابید و از $T(n)$ کم کنید و ...)

$$i) \quad T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{4}\right) + T\left(\frac{n}{8}\right) + n = \theta(n)$$

$$j) \quad T(n) = 2T\left(\frac{3n}{4}\right) + n^2 = \theta\left(n^{\log_{\frac{4}{3}} 2}\right)$$

$$k) \quad T(n) = T(n-1) + \lg n = \theta(n \cdot \lg n)$$

$$L) \quad T(n) = T(n-2) + 2 \lg n = \theta(n \cdot \lg n)$$

$$m) \quad T(n) = \sqrt{n}T(\sqrt{n}) + n = \theta(n \cdot \lg \lg n)$$

۲- با هر روشی که می‌شناسید مرتبه روابط بازگشتی زیر را بیابید.

$$a) \quad A(n) = \begin{cases} 0 & \text{if } n < 5 \\ A(n-5) + 2 & \text{other wise} \end{cases}$$

$$b) \quad B(n) = B(n-1) + \binom{n}{2}, \quad B(0) = 0.$$

$$c) \quad c(n) = \frac{c(n-1)}{c(n-2)}, \quad c(0) = 1, \quad c(1) = 2$$

$$d) \quad D(n) = D^{\vee}(n-1), \quad D(0) = 2$$

$$e) \quad E(n) = E(n-1) \cdot E(n-2), \quad E(0) = 2, \quad E(1) = 1$$

$$f) \quad F(n) = 1 + \sum_{k=0}^{n-1} (F(k-1) + F(n-k)), \quad F(0) = 1$$

$$g) \quad g(n) = \sum_{k=0}^{n-1} (k \cdot g(k-1)), \quad g(0) = 1$$

$$h) \quad H(n) = \frac{1}{2 - H(n-1)}, \quad H(0) = 0.$$

$$i) \quad I(n) = \max_{1 \leq k \leq n} \{I(k-1) + I(n-k) + n\}$$

$$j) \quad J(n) = \max_{1 \leq k \leq n} \{J(k-1) + J(n-k) + 1\}$$

$$k) \quad k(n) = \min_{1 \leq i \leq n} \{k(i-1) + k(n-i) + n\}$$

$$L) \quad L(n) = L\left(\frac{n}{5}\right) + L\left(\frac{n}{10}\right) + 2L\left(\frac{n}{2}\right) + \sqrt{n}$$

$$m) \quad M(n) = 2M\left(\frac{n}{3}\right) + 2M\left(\frac{2n}{3}\right) + n$$

$$n) \quad N(n) = \sqrt{2n} N(\sqrt{2n}) + \sqrt{n}$$

$$p) \quad P(n) = \sqrt{2n} P(\sqrt{2n}) + n$$

$$q) \quad Q(n) = \sqrt{2n} Q(\sqrt{2n}) + n^2$$

$$r) \quad R(n) = R\left(\frac{n}{2}\right) + R\left(\frac{n}{3}\right) + R\left(\frac{n}{6}\right) + n$$

$$s) \quad S(n) = S\left(\frac{n}{2}\right) + S\left(\frac{n}{3}\right) + S\left(\frac{n}{6}\right) + n^2$$

$$t) \quad T(n) = \sum_{i=1}^{\lg n} T\left(\frac{n}{2^i}\right) + n$$

سؤال‌های طبقه‌بندی شده فصل دوم

۱- تابع بازگشتی زیر را در نظر بگیرید:

```
int test (int n)
{
    if (n <= ۲)
        return ۱;
    else
        return test (n - ۲) * test (n - ۲);
}
```

(دولتی ۷۵)

زمان اجرای تابع فوق برابر است با:

$$O(n \log n) \quad (۲) \quad O\left(\frac{n}{2^2}\right) \quad (۳) \quad O(2^n) \quad (۴)$$

۲- مرتبه اجرایی تابع زیر چیست؟

```
int test (int n)
{
    if (n <= ۱) return ۱;
    else
        return test\left(\frac{n}{2}\right) + test\left(\frac{n}{2}\right);
}
```

(دولتی ۷۵)

۳- در برنامه زیر مقدار $F(۳)$ و $F(۶)$ برابر است با:

```
int F(int m, int n)
{
    if (m == ۱ || n == ۰ || m == n)
        return ۱;
    else
        return F(m - ۱, n) + F(m - ۱, n - ۱);
}
```

(۱) ۲۰

(۲) ۱۰

(۳) ۱۸

(۴) ۴

۴- مقدار تابع زیر به ازای $a=2$ و $n=6$ چیست؟ (آزاد ۷۷)

$function\ DC(a,n);$ (۱) ۱۲
 $begin$ (۲) ۶۴
 $\quad if\ n = 1\ then\ return\ (a);$ (۳) ۱۶
 $\quad if\ n\ is\ even\ then\ return\ \left(DC\left(a, \frac{n}{2}\right) * DC\left(a, \frac{n}{2}\right) \right);$ (۴) ۳۲
 $\quad return\ (a * DC(a, n-1))$
 $end\ ;$

۵- الگوریتم زیر $\binom{n}{m}$ را برای اعداد صحیح $n \geq m$ محاسبه می‌کند: (دوای ۷۷)

$function\ combination\ (n, m: integer) : integer\ ;$
 $begin$
 $\quad if\ (m = n)\ or\ (m = 0)\ then$
 $\quad \quad return\ \ (1)$
 $\quad else$
 $\quad \quad return\ combination\ (n - 1, m) + combination\ (n - 1, m - 1)$
 $end;$

برای $n > m$ دلخواه عمل + چند بار اجرا می‌شود؟

$\binom{n}{m}$ (۱) nm (۲) $\binom{n}{m} - 1$ (۳) $n(n-m)$ (۴)

۶- در فراخوانی تابع زیر برای $n=8$ چند عمل ضرب انجام می‌شود؟ فرض کنید هر عمل

$Square$ نیز یک عمل ضرب نیاز دارد. (دوای ۷۸)

$function\ count(n)$ (۱) ۴
 $\quad if\ n \leq 0\ then\ return\ 1$ (۲) ۸
 $\quad if\ n = 1\ then\ return\ 2$ (۳) ۱۰
 $\quad if\ n = 2\ then\ return\ 3$ (۴) ۹
 $\quad return\ \left(count\ (n - 2) * Square\ (count\ (n - 4)) \right)$

۷- تابع زیر روی عدد طبیعی x چه عملی انجام می‌دهد؟

```
function g(x)
  if x > 1 then
    return x * g(x - 1)
  else
    return 1
  end g
```

$$\sum_{i=1}^x i \quad (۴) \quad x! \quad (۳) \quad x^x \quad (۲) \quad 1 \quad (۱)$$

۸- یک مسئله با استفاده از الگوریتم تقسیم و غلبه حل شده است. این مسئله بزرگ به ۵ مسئله کوچکتر شکسته می‌شود و اندازه هر مسئله شکسته شده $\frac{1}{۳}$ مسئله بزرگتر است. الگوریتم شکستن و ترکیب دارای مرتبه زمانی $O(n)$ می‌باشد. مرتبه بزرگی این الگوریتم چیست؟ (آزاد ۷۹)

$$O\left(n^{\frac{5}{3}}\right) \quad (۴) \quad O\left(n^{\log_3 5}\right) \quad (۳) \quad O(n^2) \quad (۲) \quad O\left(n \log_3 5\right) \quad (۱)$$

۹- اگر A و B دو عدد صحیح نامنفی باشند و تابع M به صورت زیر تعریف شده باشد، حاصل $M(۲۰ و ۲۸)$ چیست؟

$$M(A, B) = \begin{cases} M(B, A) & \text{if } A < B \\ A & \text{if } B = 0 \\ M(B, MOD(A, B)) & \text{else} \end{cases} \quad \begin{matrix} ۸ \quad (۱) \\ ۴ \quad (۲) \\ ۱ \quad (۴) \\ \text{صفر} \quad (۳) \end{matrix}$$

۱۰- با توجه به دو تابع زیر، خروجی $F_۲(۴)$ و $F_۱(۴)$ چیست؟ (دولتی ۸۲)

```
void F۱(int x)
{
  if (x) F۲(x - 1);
  Printf(x) ;
}

void F۲(int y)
{
  if (y){
    printf(y + 1);
    F۱(y - 1);
  }
}
```

- (۱) به ترتیب از چپ به راست برای $F_1(4)$ خروجی ۴۴۲۲۰ و برای $F_2(4)$ خروجی ۵۳۱۱۳ است.
 (۲) به ترتیب از چپ به راست برای $F_1(4)$ خروجی ۲۰۲۴۴ و برای $F_2(4)$ خروجی ۱۳۳۵ است.
 (۳) به ترتیب از چپ به راست برای $F_1(4)$ خروجی ۴۲۲۰۴ و برای $F_2(4)$ خروجی ۵۳۳۱ است.
 (۴) به ترتیب از چپ به راست برای $F_1(4)$ خروجی ۴۲۰۲۴ و برای $F_2(4)$ خروجی ۵۳۱۳ است.

۱۱- تابع زیر برای محاسبه بزرگترین مقسوم علیه مشترک دو عدد نوشته شده است:

```
int gcd (int a , int b)
{
    if (b == ۰)
        return a ;
    else
        return gcd (b , a mod b) ;
}
```

(دوای ۸۰)

در این صورت مرتبه زمانی الگوریتم کدام است؟

$$O\left(\log_2^{(a-b)}\right) \quad (۴) \quad O\left(\log_2^a\right) \quad (۳) \quad O\left(\log_2 \frac{a}{b}\right) \quad (۲) \quad O\left(\log_2 \frac{a}{b}\right) \quad (۱)$$

(دوای ۸۰)

۱۲- در روال بازگشتی زیر مقدار Rec (۳ و ۵) کدام است؟

```
int Rec (int P, int q)
{
    int R;
    if (q <= ۰) return ۱;
    R = Rec (P, q/۲);
    R = R * R ;
    if (q mod ۲ == ۰)
        return R;
    else
        return R * P;
}
```

۱۳- خروجی تابع زیر برای فراخوانی $Test(5 و 2)$ چقدر است؟

$function Test (x,y: Longint): Longint ;$ ۱۰ (۱)
 $begin$ ۴ (۲)
 $if (x \leq y) or (y = 0) then test := x$ ۶ (۳)
 $else if (y = 1) then test := test (x - 1, y) + 1$ ۵ (۴)
 $else test := test (test (y, x), y - 1) + 2$
 $end;$

۱۴- تابع زیر چه فرمولی را محاسبه می‌کند؟ ($n > 0$)

$function f(n:integer, x:real): Real;$ x^n (۱)
 $begin$ x^{n-1} (۲)
 $if n \leq 1 then f := n$ $x!$ (۳)
 $else f := x * f(n - 1, x)$ $(x - 1)!$ (۴)
 $end;$

۱۵- فرض کنید که a و b اعداد صحیح مثبت بوده و تابع Q به صورت زیر به شکل بازگشتی تعریف شده باشد:

$$Q(a, b) = \begin{cases} 0 & \text{if } a < b \\ Q(a - b, b) + 1 & \text{if } b \leq a \end{cases}$$

مقادیر $Q(2, 3)$ و $Q(14, 3)$ را پیدا کنید.

$$Q(2, 3) = 0 \text{ و } Q(14, 3) = 4 \quad (2) \quad Q(2, 3) = 0 \text{ و } Q(14, 3) = 0 \quad (1)$$

$$Q(2, 3) = 4 \text{ و } Q(14, 3) = 3 \quad (4) \quad Q(2, 3) = 0 \text{ و } Q(14, 3) = 3 \quad (3)$$

۱۶- در تست قبل مقدار $Q(159, 5)$ کدام است؟

$$33 \quad (4) \quad 32 \quad (3) \quad 31 \quad (2) \quad 30 \quad (1)$$

۱۷- تابع $A(M, N)$ به شکل زیر را در نظر بگیرید. حاصل $A(1, 2)$ کدام است؟ (دو امتیاز)

$$A(M, N) = \begin{cases} N + 1 & \text{اگر } M = 0 \\ A(M - 1, 1) & \text{اگر } N = 0 \\ A(M - 1, A(M, N - 1)) & \text{بقیه حالات} \end{cases}$$

$$6 \quad (4) \quad 5 \quad (3) \quad 4 \quad (2) \quad 3 \quad (1)$$

۱۸- با اجرای برنامه زیر چه عددی چاپ می‌شود؟

```
#include <Stdio.h>
int func (int, int);
void main ( ) {
    int n = ۲;
    int m = ۳;
    printf ("%d\n" , func(n , m));
}
int func(int m, int n) {
    if (m == ۰) return n + +;
    if (n == ۰) return func(m - ۱, ۱);
    return func((m - ۱), func(m, n - ۱));
}
```

۱ (۱)

۴ (۲)

۷ (۳)

۹ (۴)

۱۹- الگوریتم زیر برای محاسبه سری فیبوناچی است، گزینه صحیح را انتخاب کنید. (علوم کامپیوتر ۷۳)

```
function fibo (n)
begin
    if n ≤ ۲ return (n)
    else return (fibo(n - ۱) + fibo(n - ۲))
end;
```

(۱) این الگوریتم از رده تقسیم و غلبه است و مرتبه آن نمایی است.

(۲) این الگوریتم از رده تقسیم و غلبه است و مرتبه آن خطی است.

(۳) این الگوریتم از رده برنامه‌ریزی پویا است و مرتبه آن خطی است.

(۴) این الگوریتم از رده برنامه‌ریزی پویا است و مرتبه آن نمایی است.

۲۰- برنامه زیر برای محاسبه سری فیبوناچی است. گزینه صحیح را انتخاب کنید. (علوم کامپیوتر ۸۰)

```
function fibo(n:integer): integer;
```

```
Var f, f۱, f۲, i : integer;
```

```
begin
```

```
    f۱ := ۱, f۲ := ۱;
```

```
    for i := ۱ to n do
```

```
    begin
```

```
        f := f۱ + f۲;
```

```
        f۱ := f۲;
```

```
        f۲ := f;
```

```
    end;
```

```
    fibo:=f
```

```
end;
```

(۱) الگوریتم این برنامه از رده برنامه‌ریزی پویا و مرتبه آن خطی است.

(۲) الگوریتم این برنامه از رده تقسیم و غلبه و مرتبه آن خطی است.

(۳) الگوریتم این برنامه از رده برنامه‌ریزی پویا و مرتبه آن بیش از خطی است.

(۴) الگوریتم این برنامه از رده تقسیم و غلبه و مرتبه آن بیش از خطی است.

۲۱- تابع زیر چه کاری را انجام می‌دهد و $L(۲۵)$ را محاسبه کنید. (آزاد ۷۶)

$$L(n) = \begin{cases} 0 & n = 1 \\ L\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + 1 & n > 1 \end{cases}$$

(۱) نصف عدد داده شده $۱+$ و ۱۳

(۲) بزرگترین عدد صحیح مثل L به طوری که $۲^L \leq n$ و ۴

(۳) $L = \lfloor \log_2^n \rfloor$ و ۴

(۴) ۲ و ۳ صحیح است.

۲۲- برنامه زیر برای انتقال n حلقه از برج‌هانوی (برهما) استفاده می‌شود. کدام گزینه دستور

حذف شده را مشخص می‌کند؟ (علوم کامپیوتر ۸۰)

Procedure Tower(n:integer; from, help, to: Char);

begin

if n = ۱ then

Writeln('move a disk from', form, 'to', to)

else begin

Tower(n - ۱, form, to, help);

Writeln('move a disk from', form, 'to', to);

دستور حذف شده

end

end;

Tower(n - ۱, form, to, help); (۱)

Tower(n - ۱, help, from, to); (۲)

Writeln('move a disk from', help, 'to', to); (۳)

Writeln('move a disk from', form, 'to', help); (۴)

۲۳- جواب فرمول بازگشتی $T(n) = 2T(n/2) + \log n!$ برابر است با:

(علوم کامپیوتر سری سراسری ۸۲)

$$(۱) \theta(n^2) \quad (۲) \theta(n \log n) \quad (۳) \theta(n \log^2 n) \quad (۴) \theta(n^2 \log n)$$

۲۴- کدام یک از گزاره‌های زیر در مورد محاسبه جمله n ام سری فیبوناچی صحیح است؟

یادآوری می‌شود که اگر $f(n)$ جمله n ام این سری باشد داریم:

$$f(n) = f(n-1) + f(n-2)$$

(۱) می‌توانیم هم از یک الگوریتم بازگشتی استفاده کنیم و هم از یک حلقه، زمان مصرفی و حافظه مورد نیاز در هر دو حال یکسان است.

(۲) بهتر است به جای حلقه از یک الگوریتم با دو فراخوانی بازگشتی برای $n-1$ و $n-2$ استفاده کنیم زمان در این صورت نصف می‌شود.

(۳) محاسبه با یک حلقه بسیار سریعتر از موردی است که این محاسبه با یک الگوریتم بازگشتی با فراخوانی برای $n-1$ و $n-2$ انجام شود ولی حافظه مورد نیاز بسیار بزرگ است.

(۴) استفاده از یک حلقه هم از لحاظ سرعت و هم از لحاظ میزان حافظه مورد نیاز نسبت به الگوریتم بازگشتی ارجحیت دارد.

۲۵- تابعی بازگشتی برای محاسبه $g(n)$ در نظر می‌گیریم. که مطابق تعریف زیر نوشته شده

باشد برای محاسبه $g(5)$ تابع مربوطه چند بار فراخوانی می‌شود؟

$$g(n) = \begin{cases} n, & n = 1 \text{ یا } n = 2 \\ 2g(n-1) + 3g(n-2), & n > 2 \end{cases}$$

$$(۱) 3 \quad (۲) 5 \quad (۳) 6 \quad (۴) 9$$

۲۶- فرض کنید که n توانی از ۲ باشد. الگوریتم زیر چه عددی را به عنوان جواب برمی‌گرداند؟

function Test(n)

if $n=1$ Then

return 1

return $2 * \text{Test}\left(\frac{n}{2}\right) + 6 * n - 1;$

$$(۱) 6n \log_2 n \quad (۲) 6n \log_2 n + 1$$

$$(۳) 6n \log_2 n - 1 \quad (۴) 6n$$

۲۷- تابع $T(n) = ۲T\left(\left\lfloor \sqrt{n} \right\rfloor\right) + \log n$ را در نظر بگیرید، کدام یک از روابط زیر برای $T(n)$

(طراحی الگوریتم دولتی ۸۵)

درست است؟

$$T(n) = O(\log n \cdot \log \log n) \quad (۲) \qquad T(n) = O(\log \log n) \quad (۱)$$

$$T(n) = O(n \cdot \log n \cdot \log n) \quad (۴) \qquad T(n) = O(n \cdot \log \log n) \quad (۳)$$

۲۸- زیر برنامه روبرو را در نظر بگیریم:

(طراحی الگوریتم هوش مصنوعی دولتی ۸۵)

```

Procedure Time (x) {
    if (x < 8) write ("Message")

    else {
        Time( $\left\lfloor \frac{x}{۲} \right\rfloor$ )
        Time( $\left\lfloor \frac{x}{۴} \right\rfloor$ )
        Time( $\left\lfloor \frac{x}{۸} \right\rfloor$ )
    }
}

```

اگر $t(x)$ تعداد دفعات چاپ پیغام *Message* روی صفحه برای فراخوانی $time(x)$ باشد کدام یک از موارد زیر صحیح است؟

$$t(۹) = ۳; t(۱۶) = ۵; t(۳۶) = ۹ \quad (۲) \qquad t(۹) = ۴; t(۱۶) = ۵; t(۳۶) = ۸ \quad (۱)$$

$$t(۹) = ۴; t(۱۶) = ۶; t(۳۶) = ۹ \quad (۴) \qquad t(۹) = ۳; t(۱۶) = ۶; t(۳۶) = ۸ \quad (۳)$$

۲۹- رابطه‌های بازگشتی زیر برای اعداد صحیح $n > ۲$ تعریف شده‌اند و داریم

$$T(۰) = T(۱) = ۱ \quad \text{کدام یک از این روابط جواب چند جمله‌ای (polynomial) ندارد؟}$$

(دولتی ۸۶)

$$T(N) = T\left(\left\lfloor \frac{۷N}{۸} \right\rfloor\right) + ۸N + ۱ \quad (۲) \qquad T(N) = ۲T(N - ۲) + ۱ \quad (۱)$$

$$T(N) = T(N - ۱) + N^۲ \quad (۴) \qquad T(N) = ۳T\left(\left\lfloor \frac{N}{۲} \right\rfloor\right) + N^۲ \quad (۳)$$

۳۰- اگر پردازش مقابل به صورت $test(۶۴)$ فراخوانی شود خروجی کدام است؟

(۱) اعداد زوج از ۲ تا ۶۴
 (۲) اعداد $۲^۰, ۲^۱, \dots, ۲^۶$ (از راست به چپ)
 (۳) اعداد $۲^۶, ۲^۵, \dots, ۲^۰$ (از راست به چپ)
 (۴) اعداد $۲^۱, ۲^۲, \dots, ۲^۵$ (از راست به چپ)

```

procedure test (b: integer) ;
begin
  if b > \ then test (b div ۲);
  Write(b);
end;
```

۳۱- در پردازش مقابل اگر $b = ۶۴$ باشد خروجی کدام است؟

(۱) اعداد $۲^۰, ۲^۱, \dots, ۲^۶$ (از راست به چپ)
 (۲) اعداد $۲^۶, ۲^۵, \dots, ۲^۰$ (از راست به چپ)
 (۳) ۷ بار عدد ۱ چاپ می‌شود.
 (۴) ۷ بار عدد ۲ چاپ می‌شود.

```

procudure test (var b: integer) ;
begin
  if b > \ then
    begin
      b := b div ۲;
      test(b)
    end;
  write(b);
end;
```

۳۲- پردازش زیر چه می‌کند؟

```

procedure A(var x:integer ; j: integer) ;
begin
  if x <= j then exit ;
  x := x - ۱;
  A(x, j);
  write (x)
end ;
```

- (۱) به تعداد $x-j$ بار j را پشت سر هم چاپ می‌کند ($x > j$)
- (۲) به تعداد x بار j را پشت سر هم چاپ می‌کند ($x > j$)
- (۳) مقدار ۰ تا x را چاپ می‌کند.
- (۴) مقدار ۰ تا j را به تعداد x بار چاپ می‌کند.

۳۳- کدام یک از توابع زیر برای محاسبه جمله n ام فیبوناچی پیچیدگی زمانی بیشتری نسبت به بقیه دارد؟

function $F1(n:byte):Longint$; (۱)

begin

if $(n \leq ۲)$ *then* $F_1 := ۱$

else $F_1 := F_1(n-۱) + F_1(n-۲)$;

end ;

function $F_۲(n : byte) : Longint$; (۲)

Var

$A, B, C, I: Longint$;

begin

$A := ۱$; $B := ۱$; $C := ۱$;

for $I := ۳$ *to* n *do*

begin

$C := A + B$;

$A := B$;

$B := C$;

end ;

$F_۲ := C$;

end ;

Function $F_۳(A, B, I, n : Longint) : Longint$; (۳)

begin

if $I = n$ *then* $F_۳ := A$

else

$F_۳ := F_۳(B, A + B, I + ۱, n)$;

end ;

function $F_۴(n : byte) : Longint$; (۴)

Var

$A : Array[۱..۱۰۰] of Longint$;

$I:byte$;

begin

$A[۱] := ۱$; $A[۲] := ۱$;

for $I := ۳$ *to* n *do*

$A[I] := A[I-۱] + A[I-۲]$;

$F_۴ := A[n]$;

end ;

۳۴- رویه زیر را در نظر بگیرید:

(آزاد ۸۳)

Procedure $f(a : \text{string}, k, n : \text{integer})$

begin

if $k=n$ *then* $\text{print}(a)$

else

for $i:=k$ *to* n *do*

begin

$\text{swap}(a[k], a[i]);$

$f(a, k+1, n);$

end;

end f.

تابع print رشته a را چاپ و تابع swap جای دو عنصر را عوض می‌کند. کدام رابطه زیر

درباره زمان f درست است؟

(۲) $O(n^k)$

(۱) $O(n!)$

(۴) $\Theta(n.n!)$

(۳) $\Theta(n!)$

۳۵- کدام یک از الگوریتم‌های زیر تشخیص می‌دهد که " N " یک عدد اول است یا خیر؟ (آزاد ۷۵)

(۱)

for $k := ۲$ *to* $N - ۱$ *do*

if $N \bmod k <> ۰$ *then* $\text{Writeln}('Prime')$

else $\text{writeln}('not prime');$

(۲)

$PR := \text{False};$

for $k := ۲$ *to* $N - ۱$ *do*

if $(N \bmod k = ۰)$ *then* $PR := \text{False}$

else $PR := \text{True};$

if (PR) *then* $\text{writeln}(N, 'is prime')$

else $\text{writeln}(N, 'is not prime');$

(۳)

```

PR:=True;
for k := ۲ to N-۱ do
    if (N mod k = ۰) then PR := False
if (PR) then writeln('prime')
else writeln ('not prime');

```

(۴)

```

PR:=True;
for k := ۲ to N-۱ do
    if (N mod k = ۰) then PR := False
Else PR:=True;
if (PR) then writeln('prime')
else writeln ('not prime');

```

۳۶- زمان اجرای الگوریتم $T(n)$ که این الگوریتم به صورت زیر است، برابر کدام گزینه است؟
(دوگزینه‌ای)

$$T(n) = \begin{cases} ۱ & n = ۱ \\ n + T(n-۱) & n \geq ۲ \end{cases}$$

$$O(n^۲) \quad (۴) \quad O(n \log n) \quad (۳) \quad O\left(n^{\frac{۲}{۳}}\right) \quad (۲) \quad O(n) \quad (۱)$$

۳۷- زمان اجرای الگوریتمی از رابطه بازگشتی تست ۳۶ بدست می‌آید، مرتبه آن الگوریتم کدام است؟

$$O(n^۲) \quad (۴) \quad O(n \log n) \quad (۳) \quad O\left(n^{\frac{۲}{۳}}\right) \quad (۲) \quad O(n) \quad (۱)$$

۳۸- می‌خواهیم n خط را در یک صفحه رسم کنیم. با فرض اینکه هیچ دو خطی موازی همدیگر نیستند و همچنین بیشتر از دو خط همدیگر را در یک نقطه قطع نمی‌کنند، تعداد نواحی تولید شده توسط این خطوط چیست؟
(آرد ۷۹)

$$n^۲ \quad (۴) \quad ۳n-۱ \quad (۳) \quad \frac{n(n+۱)}{۲} + ۱ \quad (۲) \quad ۲n+۱ \quad (۱)$$

۳۹- جواب رابطه‌ی بازگشتی $T(n) = ۴T\left(\frac{\sqrt{n}}{۳}\right) + \log^۲ n$ کدام است؟ (مهندسی - ۸۸)

$$\Theta(\log^۲ n) \quad (۲) \qquad \Theta(\sqrt{n}) \quad (۱)$$

$$\Theta(\log^۲ n \log \log n) \quad (۴) \qquad \Theta(\log^۳ n) \quad (۳)$$

۴۰- رابطه‌ی بازگشتی زیر را در نظر بگیرید:

$$F(x, \circ) = F(x + ۱, \circ) + F(x + ۱, ۱), \text{ if } x < n$$

$$F(x, ۱) = ۲F(x + ۱, \circ) + F(x + ۱, ۱), \text{ if } x < n$$

$$F(n, \circ) = ۱$$

$$F(n, ۱) = \circ$$

اگر از این رابطه بخواهیم مقدار $F(۱, ۱)$ را به صورت کارا حساب کنیم، چند بار عمل «جمع»

همان + در رابطه‌های فوق را باید انجام دهیم؟ (مهندسی کامپیوتر ۸۹)

$$O(۲^{n-۱}) \quad (۴) \qquad O(n) \quad (۳) \qquad O(n^۲) \quad (۲) \qquad O(۲^n) \quad (۱)$$

۴۱- اگر برای حل رابطه‌ی بازگشتی $T(n, k) = T(n/۲, k) + T(n, k/۴) + kn$ از $T(*, ۱) = T(*, *) = a$ استفاده کنیم، ارتفاع درخت به کدام یک از گزینه‌های زیر نزدیک‌تر است؟

درخت بازگشت استفاده کنیم، ارتفاع درخت به کدام یک از گزینه‌های زیر نزدیک‌تر است؟

(مهندسی کامپیوتر ۹۰)

$$\log_۴ nk \quad (۲) \qquad \log_۴ k \quad (۱)$$

$$\max\{\log_۴ n, \log_۴ k\} \quad (۴) \qquad \log_۴ n + \log_۴ k \quad (۳)$$

پاسخنامه سؤال‌های طبقه‌بندی شده فصل دوم

(۱) گزینه (۳). اگر $T(n)$ را زمان اجرای الگوریتم (تعداد فراخوانی یا تعداد اجرای if) فرض

$$T(n) = \begin{cases} 1, & n \leq 2 \\ 2T(n-2) + 1, & n > 2 \end{cases}$$

کنیم می‌توان رابطه بازگشتی را برای زمان

اجرا نوشت که از مرتبه $O(2^{\frac{n}{2}})$ است.

(۲) گزینه (۲). رابطه بازگشتی برای زمان اجرا به صورت $T(n) = 2T(\frac{n}{2}) + 1$ می‌باشد که

طبق قضیه *master* مرتبه اجرایی $O(n)$ است.

$$f(3, 6) = f(2, 6) + f(2, 5) = 4 \quad \text{گزینه (۴).} \quad (۳)$$

$$f(2, 6) = f(1, 6) + f(1, 5) = 2$$

$$f(2, 5) = f(1, 5) + f(1, 4) = 2$$

$$f(1, 6) = 1$$

$$f(1, 5) = 1$$

$$f(1, 4) = 1$$

$$DC(2, 6) = DC(2, 3) * DC(2, 3) = 8 * 8 = 64 \quad \text{گزینه (۲).} \quad (۴)$$

$$DC(2, 3) = 2 * DC(2, 2) = 2 * 4 = 8$$

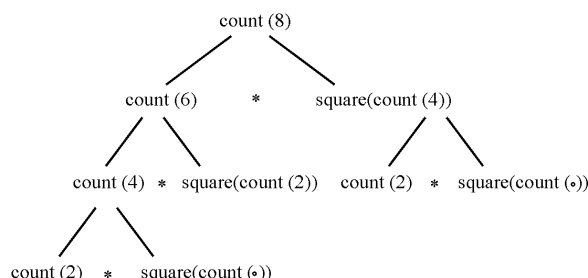
$$DC(2, 2) = DC(2, 1) * DC(2, 1) = 2 * 2 = 4$$

$$DC(2, 1) = 2$$

(۵) گزینه (۳). برای محاسبه $\binom{n}{m}$ ، تابع مورد نظر $\binom{n}{m}$ بار عدد ۱ را با هم جمع می‌زند مثلاً

برای محاسبه $\binom{4}{1}$ حاصل $1+1+1+1$ محاسبه می‌شود یعنی ۳ بار عملگر + تکرار می‌شود.

(۶) گزینه (۲).



(۷) گزینه (۳). به عنوان مثال $g(۳)$ را محاسبه می‌کنیم: $g(۳) = ۳ * g(۲) = ۳ * ۲ * ۱ = ۳!$

$$g(۲) = ۲ * g(۱) = ۲ * ۱$$

$$g(۱) = ۱$$

(۸) گزینه (۳). با توجه به توضیحات مسئله رابطه بازگشتی زیر را می‌توان نوشت:

$$T(n) = \Delta T\left(\frac{n}{۳}\right) + cn$$

طبق قضیه *master*:

$$n^{\log_b^a} = n^{\log_3^{\Delta}} > cn \Rightarrow T(n) = O(n^{\log_3^{\Delta}})$$

(۹) گزینه (۲). بزرگترین مقسوم علیه مشترک ۲۰ و ۲۸ را محاسبه می‌کند

$$M(۲۰, ۲۸) = M(۲۸, ۲۰)$$

$$M(۲۸, ۲۰) = M(۲۰, ۸)$$

$$M(۲۰, ۸) = M(۸, ۴)$$

$$M(۸, ۴) = M(۴, ۰) = ۴$$

(۱۰) گزینه (۴).

$$F_1(۴) \rightarrow \underbrace{F_2(۳)}_{\text{چاپ ۴}} \rightarrow F_1(۲) \rightarrow \underbrace{F_2(۱)}_{\text{چاپ ۲}} \rightarrow F_1(۰)$$

پس از پایان فراخوانی‌ها، تابع F_1 باید جمله $printf(x)$ را اجرا کند. بنابراین ابتدا $F_1(۰)$ ،

سپس $F_1(۲)$ و $F_1(۴)$ به ترتیب ۰ و ۲ و ۴ را چاپ می‌کنند. پس خروجی برابر ۴۲۰۲۴

خواهد بود. به همین ترتیب $F_2(۴)$ قابل بررسی است.

(۱۱) گزینه (۳). می‌توان نشان داد که همیشه مقسوم از دو برابر باقیمانده بزرگتر است، پس

$$(a \bmod b) < \frac{a}{۲} \Leftrightarrow a > ۲(a \bmod b)$$

بنابراین می‌توان گفت تابع gcd در هر بار فراخوانی، a را تقریباً نصف می‌کند بنابراین مرتبه

الگوریتم $O(\log_2^a)$ خواهد بود.

(۱۲) گزینه (۴).

$$\overset{P}{\text{Rec}}(\overset{q}{\Delta}, ۳) \rightarrow R = \overset{P}{\text{Rec}}(\overset{q}{\Delta}, ۱) \rightarrow R = \overset{P}{\text{Rec}}(\overset{q}{\Delta}, ۰) \rightarrow \text{return } ۱$$

بنابراین $R = \text{Rec}(\delta, \circ) = 1$ می‌شود و از پشته خارج می‌شود. حال $\text{Rec}(\delta, 1)$ را باید از پشته برداشته و جملات باقیمانده آن را اجرا کنیم. برای اجرا ابتدا جمله $R = R * R = 1 * 1$ اجرا شده و R یک می‌شود سپس چون $q \bmod 2 = \circ$ نیست حاصل $R * P = 1 * \delta = \delta$ بازگشت می‌شود، پس $R = \text{Rec}(\delta, 1) = \delta$ می‌شود. پس از آن $\text{Rec}(\delta, 3)$ از پشته برداشته می‌شود. ابتدا $R = R * R = \delta * \delta$ اجرا می‌شود و سپس چون $q \bmod 2 = \circ$ نیست حاصل $R * P = 2\delta * \delta$ برگشت داده می‌شود. پس مقدار $\text{Rec}(\delta, 3)$ برابر 12δ است.

(۱۳) گزینه (۲).

$$\overset{x}{test}(\overset{y}{\delta}, \overset{y}{2}) = \overset{x}{test}(\overset{y}{test}(2, \delta), 1) + 2 = \overset{x}{test}(2, 1) + 2 = 2 + 2 = 4$$

$$\overset{x}{test}(\overset{y}{2}, \overset{y}{\delta}) = 2$$

$$\overset{x}{test}(\overset{y}{2}, \overset{y}{1}) = \overset{x}{test}(\overset{y}{1}, \overset{y}{1}) + 1 = 1 + 1 = 2$$

(۱۴) گزینه (۲). به ازای $n = 1$ حاصل برابر ۱ و به ازای $n = 3$ حاصل برابر x^2 است.

(۱۵) گزینه (۲). خارج قسمت a به b را محاسبه می‌کند.

(۱۶) گزینه (۲).

$$Q(159, \delta) = 159 \div \delta = 31$$

(۱۷) گزینه (۳). در تابع آکرمان همیشه $A(1, N) = N + 2$ پس $A(1, 3) = 5$

(۱۸) گزینه (۴).

$$\underset{m}{func}(\underset{n}{2}, \underset{n}{3}) = \underset{m}{func}(1, \underset{n}{func}(2, 2)) = \underset{m}{func}(1, 7) = 9$$

$$\underset{m}{func}(\underset{n}{2}, \underset{n}{2}) = \underset{m}{func}(1, \underset{n}{func}(2, 1)) = \underset{m}{func}(1, \delta) = 7$$

$$\underset{m}{func}(\underset{n}{2}, \underset{n}{1}) = \underset{m}{func}(1, \underset{n}{func}(2, \circ)) = \underset{m}{func}(1, 3) = \delta$$

$$\underset{m}{func}(\underset{n}{2}, \circ) = \underset{m}{func}(1, 1) = 3$$

$$\underset{m}{func}(1, 1) = \underset{m}{func}(\circ, \underset{n}{func}(1, \circ)) = \underset{m}{func}(\circ, 2) = 3$$

$$\underset{m}{func}(1, \circ) = \underset{m}{func}(\circ, 1) = 2$$

$$\underset{m}{func}(\circ, 1) = 2$$

ابتدا فراخوانی‌ها را انجام دادیم تا به $func(\circ, 1)$ رسیدیم. بعد به عقب بازگشت کردیم. البته اگر بخواهیم قواعد زبان C را رعایت کنیم یعنی $return\ n++$ مقدار n را برگرداند، گزینه ۱ صحیح است.

گزینه (۱). الگوریتم بازگشتی است و مرتبه آن از $2^{\frac{n}{2}}$ بیشتر و از 2^n کمتر است. (دقیقاً از مرتبه $\theta\left(\left(\frac{\sqrt{5}+1}{2}\right)^n\right)$ است)

گزینه (۲). الگوریتم غیربازگشتی است و مرتبه آن n است.

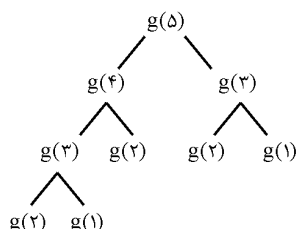
گزینه (۴). جزء صحیح لگاریتم n محاسبه می‌شود یعنی $L(25) = \left\lceil \log_2 25 \right\rceil = 4$

گزینه (۲). در مرحله سوم هانوی باید $n-1$ دیسک از میله کمکی با کمک میله مبدأ به میله مقصد منتقل شود.

گزینه (۳). با توجه به اینکه $\log n!$ و $n \log n$ هم رشد هستند و با توجه به اینکه $n^{\log_b a} = n$ ، پس مرتبه $\theta(n(\log n)^2)$ است.

گزینه (۴). اگر از حلقه استفاده کنیم زمان خطی می‌شود، حال آنکه بازگشتی زمان نمایی است و استفاده از حلقه از نظر حافظه نیز بهتر است چون نیازی به پشته ندارد.

گزینه (۴).



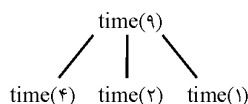
گزینه (۲). به ازای $n=1$ جواب باید ۱ شود (گزینه ۲)

گزینه (۲). تغییر متغیر می‌دهیم

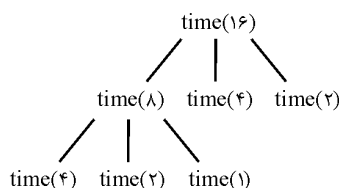
$$n = 2^m \Rightarrow T(2^m) = 2T\left(2^{\frac{m}{2}}\right) + m, \quad T(2^m) = F(m)$$

$$\Rightarrow F(m) = 2F\left(\frac{m}{2}\right) + m = O(m \log m) \Rightarrow T(n) = O(\log n \cdot \log \log n)$$

(۲۸) گزینه (۲).



پس $time(9)$ ۳ بار پیغام را چاپ می‌کند.



پس $time(16)$ ۵ بار پیغام را چاپ می‌کند. به همین ترتیب $time(36)$ ، ۹ بار چاپ می‌کند.

(۲۹) گزینه (۱). مرتبه گزینه ۱، $\theta(2^{\frac{n}{2}})$ می‌باشد.

سایر گزینه‌ها را بدست می‌آوریم:

$$T(N) = T\left(\left\lfloor \frac{9N}{8} \right\rfloor\right) + 8N + 1 = \theta(N)$$

$$T(N) = 3T\left(\left\lfloor \frac{N}{2} \right\rfloor\right) + N^2 = \theta(N^2)$$

دقت کنید برای این دو از قضیه *Master* استفاده کردیم. در این قضیه وجود $\left\lfloor \right\rfloor$ و $\left\lceil \right\rceil$ بی‌تأثیر است.

$$T(N) = T(N-1) + N^2$$

برای حل این رابطه یک شرط اولیه مثلاً $T(1) = 1$ فرض می‌کنیم حال داریم:

$$T(2) = T(1) + 2^2 = 1 + 2^2, \quad T(3) = T(2) + 3^2 = 1 + 2^2 + 3^2$$

$$\dots \text{ و } T(N) = 1 + 2^2 + 3^2 + \dots + N^2 = \frac{N(N+1)(2N+1)}{6} = \theta(N^3)$$

(۳۰) گزینه (۲). باید فراخوانی کنیم، هرگاه فراخوانی‌ها تمام شد، از آخرین فراخوانی شروع کرده و جملات باقیمانده را ($write(b)$) اجرا کنیم:

$$test(64) \rightarrow test(32) \rightarrow test(16) \rightarrow test(8) \rightarrow test(4) \rightarrow test(2) \rightarrow test(1)$$

پس از پایان فراخوانی‌ها، هر یک از فراخوانی‌ها باید جمله $write$ را اجرا کنند پس خروجی مطابق گزینه ۲ خواهد بود.

(۳۱) گزینه (۳). فراخوانی‌ها مطابق تست قبل می‌باشد ولی چون پارامتر b دارای var (reference) می‌باشد، آخرین مقدار b یعنی یک به همه فراخوانی‌ها منتقل می‌شود و همگی عدد یک را چاپ می‌کنند.

(۳۲) گزینه (۱). با $x = 5$ و $j = 3$ فراخوانی می‌کنیم:

$$A(5, 3) \rightarrow A(4, 3) \rightarrow \underbrace{A(3, 3)}_{exit}$$

چون x دارای var است، ۲ بار (۵-۳) عدد ۳ چاپ می‌شود.

(۳۳) گزینه (۱). گزینه‌های ۲ و ۴ غیربازگشتی هستند و مرتبه آنها $O(n)$ می‌باشد. گزینه ۳ با هر فراخوانی مقدار I را یک واحد اضافه می‌کند تا $I=n$ شود. بنابراین اگر در ابتدا $I=1$ باشد، n بار فراخوانی انجام می‌شود. پس گزینه ۳ نیز از مرتبه $O(n)$ است. ولی گزینه ۱ دارای مرتبه نمایی است.

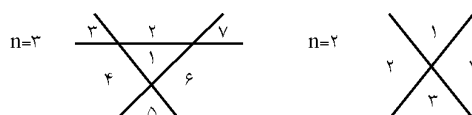
(۳۴) گزینه (۳). این تابع محاسبه تمام جایگشت‌های n عنصر است. اگر تابع $print(a)$ از مرتبه $\theta(1)$ فرض شود، مرتبه این الگوریتم $\theta(n!)$ است. ولی اگر تابع $print(a)$ از مرتبه $\theta(n)$ فرض شود، مرتبه این الگوریتم $\theta(n \cdot n!)$ است.

(۳۵) گزینه (۳). به راحتی می‌توان سایر گزینه‌ها را نقض کرد. مثلاً گزینه ۱ برای عددی مثل $N=9$ چندین بار $prime$ و $not\ prime$ چاپ می‌کند. در گزینه ۲ و ۴ فرض کنید $N=9$ ، آنگاه PR چندین بار $False$ و $ture$ می‌شود ولی در آخرین اجرای حلقه PR برابر $ture$ می‌شود و در این صورت عبارت if جمله $is\ prime$ را چاپ می‌کند که غلط است.

(۳۶) گزینه (۱). این الگوریتم به طور بازگشتی، n بار فراخوانی انجام می‌دهد و مرتبه هر فراخوانی $O(1)$ است بنابراین این n بار فراخوانی $O(n)$ می‌باشد. دقت کنید که این الگوریتم جمع اعداد ۱ تا n را محاسبه می‌کند و این جمع برابر است با $\frac{n(n+1)}{2}$ ولی این مسئله ربطی به مرتبه اجرایی ندارد. در واقع در این تست اگر زمان اجرای الگوریتم را با $g(n)$ نمایش دهیم می‌توان نوشت: $g(n) = g(n-1) + 1$ که این تابع از مرتبه $g(n) = \theta(n)$ می‌باشد.

(۳۷) گزینه (۴). با توجه به اینکه $T(n) = \frac{n(n+1)}{2}$ پس مرتبه آن الگوریتم $O(n^2)$ است. دقت کنید در این تست $T(n)$ خود زمان اجرای یک الگوریتم دیگر است و با توجه به اینکه $T(n) = \frac{n(n+1)}{2}$ پس $T(n) = \theta(n^2)$.

(۳۸) گزینه (۲).



۳۹) گزینه (۴).

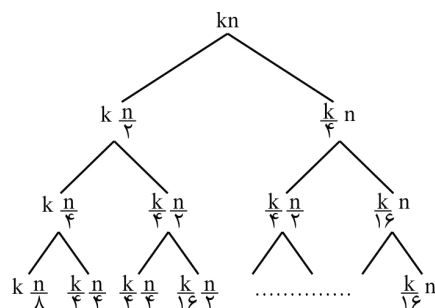
$$n = 3^m \Rightarrow T(3^m) = 4T\left(\frac{m}{3}\right) + \text{Log}^2 3^m$$

$$\Rightarrow F(m) = 4F\left(\frac{m}{3} - 1\right) + m^2 = \theta(m^2 \text{Log} m) \Rightarrow T(n) = \theta(\text{Log}^2 n \cdot \text{Log} \text{Log} n)$$

توجه کنید که می‌توان ۱- را از داخل F حذف کرد و با $Master$ به جواب رسید.

۴۰) گزینه (۳). برای محاسبه (۱ و ۱) اگر از بالا به پایین (تقسیم و غلبه) عمل کنیم یعنی اگر از خود (۱ و ۱) شروع کنیم، تعداد جمع‌ها، نمایی می‌شود. ولی به صورت کارا یعنی اگر از پایین به بالا (پویا) عمل کنیم یعنی ابتدا $F(n, 0)$ و $F(n, 1)$ و $F(n-1, 0)$ و $F(n-1, 1)$ و ... و $F(2, 0)$ و $F(2, 1)$ را محاسبه کنیم و در یک مکانی ذخیره کنیم و در نهایت $F(1, 1)$ را محاسبه کنیم، تعداد عملیات جمع، خطی می‌شود. برای درک بهتر، به ازای $n = 4$ آزمایش کنید.

۴۱) گزینه (۳).



ارتفاع شاخه سمت چپ $\log_4^n$ ، ارتفاع شاخه سمت راست $\log_4^k$ ، ارتفاع شاخه‌های وسط از همه بزرگتر است مثلاً مسیر $kn \rightarrow k \frac{n}{2} \rightarrow \frac{k}{4} \frac{n}{2} \rightarrow \frac{k}{4} \frac{n}{4}$ طولانی‌ترین مسیر است و ارتفاعش $\log_4^k + \log_4^n$ است. به ازای $k = 64$ و $n = 16$ درخت بکشید و آزمایش کنید.

آرایه، لیست پیوندی، پشته، صف

فصل ۳

آرایه از تعدادی عنصر هم‌نوع و هم‌اندازه تشکیل شده که به صورت متوالی در حافظه ذخیره می‌شود. آرایه یک ساختمان داده خطی است. اگر تعریف آرایه یک بعدی به شکل $A[L..U]$ و نوع عناصر $Type$ فرض شود که $sizeof (Type)$ را n بایت فرض کنیم آنگاه تعداد عناصر $U - L + 1$ و حافظه تخصیصی $n * (U - L + 1)$ بایت است.

عنصر $A[i]$ ($L \leq i \leq U$) عنصر $i - L + 1$ ام است و اگر آدرس شروع آرایه در حافظه را α فرض کنیم، آدرس شروع $A[i]$ برابر $\alpha + n * (i - L)$ است. مثلاً در زبان C تعریف $float f[100]$ یک آرایه ۱۰۰ عنصری (۰.۹۹) تعریف می‌کند که ۴۰۰ بایت حافظه اشغال می‌کند ($sizeof (float)$ برابر ۴ بایت است). عنصر $A[20]$ عنصر ۲۱ام ($20 - 0 + 1$) آرایه است و اگر آدرس شروع آرایه در حافظه $\alpha = 1000$ باشد، آدرس شروع $A[20]$ برابر $1000 + 4 * (20 - 0) = 1080$ است.

آرایه‌های چند بعدی به دو طریق ممکن است در حافظه ذخیره شوند: سطری (*Row major*) و ستونی (*Column major*). مثلاً آرایه ۲ بعدی $A[1..3, 1..2]$ را در نظر بگیرید:

[۱,۱]	[۱,۲]
[۲,۱]	[۲,۲]
[۳,۱]	[۳,۲]

شکل ماتریسی

[۱,۱]	[۱,۲]	[۲,۱]	[۲,۲]	[۳,۱]	[۳,۲]

شکل سطری

[۱,۱]	[۲,۱]	[۳,۱]	[۱,۲]	[۲,۲]	[۳,۲]

شکل ستونی

آرایه دو بعدی $A[L_1..U_1, L_2..U_2]$ را در نظر بگیرید. تعداد عناصر $(U_1 - L_1 + 1)(U_2 - L_2 + 1)$ است.

آدرس شروع عنصر $A[i, j]$ ($L_1 \leq i \leq U_1$, $L_2 \leq j \leq U_2$) برابر است با:

در روش سطری: $[(i - L_1)(U_2 - L_2 + 1) + (j - L_2)] * n + \alpha$

در روش ستونی: $[(j - L_2)(U_1 - L_1 + 1) + (i - L_1)] * n + \alpha$

در آرایه ۳ بعدی $A[L_1..u_1, L_2..u_2, L_3..u_3]$ ، آدرس شروع $A[i, j, k]$:

تعداد عناصر ابعاد بعدی

سطری: $[(i - L_1)(u_2 - L_2 + 1)(u_3 - L_3 + 1) + (j - L_2)(u_3 - L_3 + 1) + (k - L_3)] * n + \alpha$

ستونی: $[(k - L_3)(u_2 - L_2 + 1)(u_1 - L_1 + 1) + (j - L_2)(u_1 - L_1 + 1) + (i - L_1)] * n + \alpha$

تعداد عناصر ابعاد قبلی

مثال: در آرایه $float\ f[5][10][15][20]$ که اندیس شروع در هر بعد، صفر است عنصر

$f[2][3][4][6]$ (الف) در روش سطری عنصر چندم است؟

ب) در روش ستونی از چه آدرسی شروع می‌شود؟ فرض کنید آدرس شروع آرایه در حافظه

$\alpha = 1000$ و $n = sizeof(float) = 4$.

پاسخ: الف) برای محاسبه عنصر چندم، در فرمول‌های گفته شده بجای n و α ، عدد یک قرار دهید:

$[(2-0)(10-0)(15-0) + (3-0)(15-0) + (4-0)(15-0) + (6-0)] * 1 + 1000 = 6987$

ب) $[(6-0)(15-0)(10-0) + (4-0)(10-0) + (3-0)(10-0) + (2-0)] * 4 + 1000 = 19868$

ماتریس‌های خاص

با چند مثال، ماتریس‌های مثلثی و ۳ قطری را بررسی می‌کنیم.

مثال: عناصر غیر صفر ماتریس پایین مثلثی $A_{n \times n}$ را به صورت سطری در آرایه یک بعدی

$B[1 \dots \frac{n(n+1)}{2}]$ ذخیره کرده‌ایم، مکان عنصر a_{ij} را محاسبه کنید.

پاسخ:

$$A = \begin{bmatrix} a_{11} & 0 & 0 & 0 & \dots & 0 \\ a_{21} & a_{22} & 0 & 0 & \dots & 0 \\ a_{31} & a_{32} & a_{33} & 0 & \dots & 0 \\ \vdots & & & & & \\ a_{n1} & a_{n2} & & & \dots & a_{nn} \end{bmatrix} \Rightarrow B = [a_{11} \ a_{21} \ a_{22} \ a_{31} \ a_{32} \ a_{33} \ \dots \ a_{nn}]$$

عنصر a_{ij} در سطر i ام است که قبل از آن $i-1$ سطر وجود دارد که در این $i-1$ سطر، $\frac{i(i-1)}{2}$ عنصر غیرصفر هست و در سطر i ام عنصر a_{ij} عنصر j ام است. پس a_{ij} در $B\left[\frac{i(i-1)}{2} + j\right]$ قرار دارد. مثلاً مکان a_{33} در آرایه B خانه $5 = \frac{3(3-1)}{2} + 3$ است.

مثال: در مثال قبل اگر عناصر غیرصفر، ستونی ذخیره شوند، مکان a_{ij} را بیابید.

پاسخ: $B = [a_{11} \ a_{21} \ a_{31} \ \dots \ a_{n1} \ a_{12} \ a_{22} \ a_{32} \ \dots \ a_{nn}]$

عنصر a_{ij} در ستون j ام است و قبل از آن $j-1$ ستون وجود دارد که تعداد عناصر غیرصفر در این ستون‌ها $(n-j+2) + \dots + (n-1) + n$ است و در ستون j ام عنصر a_{ij} عنصر $i-j+1$ ام است پس مکان a_{ij} در B برابر است با:

$$n + (n-1) + \dots + (n-j+2) + i-j+1 = \frac{(n+n-j+2)(j-1)}{2} + i-j+1$$

مثلاً مکان a_{33} برابر است با:

$$\frac{(2n-2+2)(2-1)}{2} + 3-2+1 = n+2$$

توجه: اگر آرایه B از α شروع شود به فرمول‌های گفته شده باید $\alpha-1$ را جمع کنید.

تمرین: فرمول‌های گفته شده برای ماتریس پایین مثلثی را برای بالا مثلثی محاسبه کنید.

تمرین: عناصر غیرصفر ماتریس پایین مثلثی را با شروع از قطر اصلی به صورت قطری در آرایه B ذخیره کنید و مکان a_{ij} را بیابید.

مثال: عناصر غیرصفر ماتریس ۳ قطری $A_{n \times n}$ را به صورت سطری در $B[1..3n-2]$ ذخیره کنید و مکان a_{ij} را بیابید.

پاسخ: در ماتریس ۳ قطری بجز عناصر قطر اصلی و موازی بالا و پایین قطر اصلی، سایر عناصر صفر هستند:

$$A = \begin{bmatrix} a_{11} & a_{12} & 0 & 0 & \dots & 0 \\ a_{21} & a_{22} & a_{23} & 0 & \dots & 0 \\ 0 & a_{32} & a_{33} & a_{34} & \dots & 0 \\ \vdots & & & & & \\ 0 & 0 & \dots & a_{n,n-1} & a_{nn} \end{bmatrix} \Rightarrow B = [a_{11} \ a_{12} \ a_{21} \ a_{22} \ a_{23} \ a_{32} \ a_{33} \ a_{34} \ \dots \ a_{nn}]$$

عنصر a_{ij} در سطر i ام است پس قبل از آن $i-1$ سطر است که در سطر اول ۲ عنصر غیرصفر و در $i-2$ سطر دیگر ۳ عنصر غیرصفر هست. در سطر i ام عنصر a_{ij} عنصر $j-i+2$ ام است پس مکان a_{ij} در B برابر $2i + j - 2 = 2 + 3(i-2) + j - i + 2$ است.

مثلاً مکان $a_{۳۲}$ برابر $۶ = ۲ - ۲ + ۳ \times ۲$ است.

توجه: اگر عناصر غیر صفر A را ستونی ذخیره کنید مکان از $۲ - i + j$ بدست می‌آید.

توجه: اگر آرایه B از α شروع شود به فرمول‌ها $\alpha - ۱$ را اضافه کنید.

عملیات روی آرایه

جمع دو ماتریس: دستورات مقابل جمع دو ماتریس $A[۱..n, ۱..m]$ و $B[۱..n, ۱..m]$ را در $C[۱..n, ۱..m]$ ذخیره می‌کند که از مرتبه $\theta(nm)$ است.

for ($i = ۱$ to n)

for ($j = ۱$ to m)

$$C[i, j] = A[i, j] + B[i, j]$$

ترانهاد (transpose) ماتریس: دستورات مقابل ترانهاد $A[۱..n, ۱..m]$ (تعویض سطر و ستون‌ها) را در $B[۱..m, ۱..n]$ قرار می‌دهد، و از مرتبه $\theta(nm)$ است.

for ($i = ۱$ to n)

for ($j = ۱$ to m)

$$B[j, i] = A[i, j]$$

ضرب دو ماتریس:

$$A[۱..n, ۱..m] \times B[۱..m, ۱..p] = C[۱..n, ۱..p]$$

$$C[i, j] = \sum_{k=۱}^m A[i, k] * B[k, j] \quad , \quad ۱ \leq i \leq n \quad , \quad ۱ \leq j \leq p$$

برای محاسبه $C[i, j]$ ، m بار عمل $*$ و $m - ۱$ بار عمل $+$ انجام می‌شود پس برای محاسبه همه عناصر C ، nmp بار عمل $*$ و $n(m - ۱)p$ بار عمل $+$ انجام می‌شود. دستورات مقابل ضرب دو ماتریس را انجام می‌دهند:

for ($i = ۱$ to n)

for ($j = ۱$ to p)

{

$$C[i, j] = ۰$$

for ($k = ۱$ to m)

$$C[i, j] = C[i, j] + A[i, k] * B[k, j]$$

}

البته در این کد، تعداد باری که عمل + انجام می‌شود با تعداد عمل * مساوی است و برابر mnp است.

برای کاهش عمل + می‌توان بجای $C[i, j] = 0$ دستور $C[i, j] = A[i, 1] * B[1, j]$ را قرار دهیم و حلقه آخر را از $k = 2$ شروع کنیم که در این صورت تعداد عمل + برابر $n(m-1)p$ می‌شود. با توجه به الگوریتم نوشته شده، ضرب دو ماتریس $n \times n$ از مرتبه $\theta(n^3)$ است. البته روش‌هایی برای ضرب با مرتبه بهتر وجود دارد. مثلاً روش ضرب استراسن از مرتبه $\theta(n^{\lg 7})$ است. ثابت می‌شود کران پایین برای ضرب دو ماتریس $n \times n$ ، $\Omega(n^2)$ است ولی هنوز الگوریتمی با مرتبه $\theta(n^2)$ یافت نشده است (برای مطالعه بیشتر به کتاب طراحی الگوریتم رجوع کنید)

مثال: ضرب $A_{10 \times 12} \times B_{12 \times 8} \times C_{8 \times 5}$ حداقل چند عمل ضرب عددی نیاز دارد؟

پاسخ: ضرب ماتریس‌ها خاصیت جابجایی ندارد ولی شرکت‌پذیری دارد. به دو صورت $(AB)C$ و $A(BC)$ می‌توان ضرب را انجام داد. تعداد ضرب عددی هر حالت را محاسبه می‌کنیم:

$(AB)C$: تعداد ضرب عددی: $10 \times 12 \times 8 + 10 \times 8 \times 5 = 1360$.

$A(BC)$: تعداد ضرب عددی: $12 \times 8 \times 5 + 10 \times 12 \times 5 = 1080$.

پس جواب 1080 می‌باشد.

مثال: چه رابطه‌ای بین w و x و y و z برقرار باشد تا ضرب $A_{w \times x} \times B_{x \times y} \times C_{y \times z}$ به صورت $(AB)C$ از $A(BC)$ بهتر باشد (یعنی تعداد ضرب عددی کمتری داشته باشد)

پاسخ: $(AB)C$: تعداد ضرب عددی: $wxy + wyz$

$A(BC)$: تعداد ضرب عددی: $xyz + wxz$

$$wxy + wyz < xyz + wxz \xrightarrow{\div xyz} \frac{1}{z} + \frac{1}{x} < \frac{1}{w} + \frac{1}{y}$$

نکته: تعداد حالات ضرب n ماتریس برابر $\frac{1}{n} \binom{2n-2}{n-1}$ است. مثلاً "تعداد حالات ضرب ۴"

ماتریس برابر $5 = \frac{1}{4} \binom{6}{3}$ است:

$$(AB)(CD) \text{ و } ((AB)C)D \text{ و } (A(BC))D \text{ و } A((BC)D) \text{ و } A(B(CD))$$

جستجوی خطی در آرایه (Linear search): این روش در آرایه‌های نامرتب $A[1..n]$

استفاده می‌شود و روش جستجو به این ترتیب است که عنصر مورد نظر (x) ابتدا با عنصر اول آرایه مقایسه می‌شود و در صورت عدم تساوی با عنصر دوم مقایسه می‌شود و به همین ترتیب جستجو

وقتی پایان می‌یابد که یا به اولین مورد تساوی برسیم و یا به انتهای آرایه برسیم. حداقل و حداکثر تعداد مقایسه به ترتیب ۱ و n است و متوسط تعداد مقایسه $\frac{n+1}{2}$ است (در فصل اول به روش امید ریاضی بدست آمد) بنابراین مرتبه زمانی جستجوی خطی $O(n)$ است.

seqsearch (A [$1..n$] , x)

```
{
    loc = ۱;
    while (loc ≤ n and A[loc] ≠ x)
        loc = loc + ۱;
    if (loc > n) return NIL else return loc;
}
```

اگر x در آرایه نباشد، NIL برمی‌گرداند در غیر این صورت اولین مکان وقوع x را برمی‌گرداند. **جستجوی دودویی** (*binary search*): در آرایه‌های مرتب استفاده می‌شود که فرض می‌کنیم آرایه صعودی مرتب است. در این روش x ابتدا با عنصر وسط آرایه مقایسه می‌شود اگر مساوی بودند، جستجو تمام است. اگر x از عنصر وسط بزرگتر باشد، جستجو با همین شیوه در نیمه راست آرایه ادامه می‌یابد و در غیر این صورت جستجو در نیمه چپ آرایه ادامه می‌یابد. الگوریتم‌های زیر x را در آرایه مرتب صعودی $A[L..u]$ به دو شیوه بازگشتی و غیربازگشتی جستجو می‌کنند:

recbinary (l , u , x)

```
{
    if ( $L > u$ ) return NIL
    mid = ⌊  $\frac{L + u}{2}$  ⌋
    if ( $x == A[mid]$ ) return mid
    if ( $x > A[mid]$ )
        return recbinary (mid + ۱,  $u$ ,  $x$ )
    else
        return recbinary ( $L$ , mid - 1 ,  $x$ )
}
```

Iterativebinary (L , u , x)

```
{
    while ( $L \leq u$ )
    {
```

```

mid = ⌊ $\frac{L+u}{2}$ ⌋
if (x == A[mid]) return mid
if (x > A[mid]) L = mid + 1 else u = mid - 1
}
return NIL
}

```

هر دو الگوریتم NIL می‌دهند اگر x در آرایه نباشد. حداکثر تعداد مقایسه‌ها برای آرایه n عنصری را $T(n)$ می‌نامیم و می‌توان نشان داد $T(1) = 1$ و $T(n) = T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + 1$ که با حل این رابطه $T(n) = \lfloor \lg n \rfloor + 1 = \lceil \lg(n+1) \rceil$ برای مطالعه بیشتر این نوع جستجو به کتاب طراحی الگوریتم رجوع کنید.

ماتریس اسپارس (*Sparse*) (خلوت، پراکنده)

ماتریسی که تعداد صفرهای آن زیاد است. برای نمایش یک ماتریس اسپارس، از یک آرایه دو بعدی به صورت $Sparse = Array[0..t, 0..2] \text{ of items}$ می‌توان استفاده کرد که t تعداد عناصر غیرصفر ماتریس است و $items$ نوع عناصر می‌باشد. سطر اول ماتریس $Sparse$ شامل تعداد سطرها و ستون‌ها و همچنین تعداد عناصر غیرصفر ماتریس اصلی است.

مثال: با توجه به ماتریس A ، ماتریس $B: Sparse$ به صورت زیر است:

$$\begin{array}{c}
 \text{تعداد عناصر غیرصفر} \quad \text{تعداد ستون‌های } A \quad \text{تعداد سطرهاى } A \\
 \swarrow \quad \downarrow \quad \searrow \\
 A = \begin{bmatrix} 5 & 0 & 0 & 10 & 0 & -5 \\ 0 & 12 & 4 & 0 & 0 & 0 \\ 0 & 0 & 0 & -8 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 10 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 45 & 0 & 0 & 0 \end{bmatrix} \Rightarrow B = \begin{bmatrix} 6 & 6 & 8 \\ 1 & 1 & 5 \\ 1 & 4 & 10 \\ 1 & 6 & -5 \\ 2 & 2 & 12 \\ 2 & 3 & 4 \\ 3 & 4 & -8 \\ 5 & 1 & 10 \\ 6 & 3 & 45 \end{bmatrix}
 \end{array}$$

شماره سطر و ستون و مقدار عناصر غیرصفر A به ترتیب سطری در ماتریس B ذخیره شده‌اند. با توجه به نمایش A و B ، در A ، ۳۶ عدد و در B ، ۲۷ عدد ذخیره شده است. اگر ابعاد ماتریس افزایش یابد این وضعیت بهتر نیز می‌شود.

با توجه به نمایش ماتریس مشخص است که $B[0,0]$ تعداد سطرهای ماتریس اولیه (A) ، $B[0,1]$ تعداد ستون‌ها و $B[0,2]$ تعداد عناصر غیرصفر ماتریس A را نشان می‌دهد.

ترانهاده کردن ماتریس اسپارس

هر عنصر از ماتریس اسپارس بوسیله یک سه گانه (i, j, val) ذخیره می‌شود که i شماره سطر و j شماره ستون و val مقدار عنصر است.

برای ترانهاده کردن ماتریس B کافیست عنصر مکان (i, j) به مکان (j, i) منتقل شود. به عنوان مثال، ترانهاده ماتریس مثال قبل ممکن است به صورت زیر باشد:

$$B^t = \begin{bmatrix} 6 & 6 & 8 \\ 1 & 1 & 5 \\ 4 & 1 & 10 \\ 6 & 1 & -5 \\ 2 & 2 & 12 \\ 3 & 2 & 4 \\ 4 & 3 & -8 \\ 1 & 5 & 10 \\ 3 & 6 & 45 \end{bmatrix}$$

اما مشکل این است که ترتیب رعایت نشده است. بنابراین نمایش فوق برای B^t صحیح نیست. بلکه باید B^t به صورت زیر باشد.

یعنی باید عناصر ستون اول ماتریس B^t منظم باشند.

$$B^t = \begin{bmatrix} 6 & 6 & 8 \\ 1 & 1 & 5 \\ 1 & 5 & 10 \\ 2 & 2 & 12 \\ 3 & 2 & 4 \\ 3 & 6 & 45 \\ 4 & 1 & 10 \\ 4 & 3 & -8 \\ 6 & 1 & -5 \end{bmatrix}$$

می‌توان الگوریتم مقابل را برای ترانهاده کردن پیشنهاد داد.

for all elements in column j in order do
place element (i, j, val) in position (j, i, val) ;

مرتبه اجرایی این الگوریتم $O(nt)$ است که t تعداد عناصر غیرصفر ماتریس است و n تعداد ستون‌ها. اگر t خود از مرتبه nm باشد (m تعداد سطرها) آنگاه مرتبه الگوریتم $O(n^2m)$ می‌باشد که خوب نیست.

روال دیگری به نام *fasttranspose* برای ترانهاده کردن وجود دارد که در ابتدا تعداد عناصر را در هر ستون ماتریس تعیین می‌کند. با این عمل تعداد عضوها در هر ردیف ماتریس ترانهاده حاصل مشخص می‌شود. از این عدد به دست آمده، نقطه شروع هر سطر در ماتریس B^t به آسانی بدست می‌آید. حال می‌توان عناصر B را یکی یکی به ترتیب به مکان صحیح‌شان در B^t منتقل کرد. مرتبه این الگوریتم $O(n+t)$ می‌باشد.

❖ **نکته:** ضرب دو ماتریس اسپارس لزوماً اسپارس نیست. الگوریتم ضرب دو ماتریس اسپارس $A_{n \times m}$ و $B_{m \times p}$ که به ترتیب دارای t_1 و t_2 عنصر غیرصفر هستند از مرتبه $O(pt_1 + nt_2)$ است.

نمایش چند جمله‌ای‌ها

برای نمایش چند جمله‌ای $p(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_0$ می‌توان به سادگی ضرایب را در یک آرایه یک بعدی ذخیره ساخت. مثلاً برای ذخیره $5x^4 + 2x^2 - 6x + 3$ داریم:

			۴	۳	۲	۱	۰
۵		۰	۲	-۶	۳		

ولی این روش ممکن است فضای زیادی برای برخی چند جمله‌ای‌ها مثل $7X^{100} - 3X + 4$ هدر دهد. زیرا آرایه‌ای با ۱۰۱ عنصر نیاز است که فقط ۳ عنصر آن غیرصفر است. روش دیگر استفاده از یک ماتریس دو سطری است که یک سطر، ضرایب و سطر دیگر توان‌ها باشد. مثلاً برای نمایش $7X^{100} - 3X + 4$ داریم:

ضرایب	۴	-۳	۷
توان‌ها	۰	۱	۱۰۰

تطابق الگو String matching

می‌خواهیم زیر رشته P را در رشته S جستجو کنیم و اگر P در S بود، اولین اندیس اولین وقوع P را (i) برگردانیم. مثلاً اگر $S = "abcdbe"$ و $P = "bc"$ آنگاه $i=1$ (دقت کنید اولین اندیس رشته صفر است). اگر P در S نبود یا P تهی بود می‌خواهیم ۱- برگردانیم. روش معمولی آن است که کاراکترهای رشته P را تا زمانی که مطابقت دارند با S مقایسه کنیم و اگر به نامساوی رسیدیم، در رشته S یک واحد جلو رویم و P را دوباره مقایسه کنیم. این الگوریتم از مرتبه

$O(m.n)$ است (m طول رشته S و n طول رشته P). روش سریعتر توسط $Pratt$ و $Morris$ و $Knuth$ (KMP) ارائه شده است فرض کنید $P = "abcabcacab"$ و $S = S_0 S_1 \dots S_{m-1}$ می‌خواهیم ببینیم در S_i تطابق وجود دارد یا خیر. اگر $S_i \neq a$ واضح است که باید مقایسه را از S_{i+1} انجام دهیم (یعنی S_{i+1} را با a مقایسه کنیم). اگر $S_i = a$ و $S_{i+1} \neq b$ باید S_{i+1} را با a مقایسه کنیم. اگر $S_i S_{i+1} = ab$ و $S_{i+2} \neq c$ لازم نیست مثل روش معمولی S_{i+1} را با a مقایسه کنیم بلکه کفایت مقایسه را از S_{i+2} ادامه دهیم. چون اکنون می‌دانیم که $S_{i+1} \neq a$ حال فرض کنید $S_i S_{i+1} S_{i+2} S_{i+3} = abca$ ولی $S_{i+4} \neq b$ حال باید مقایسه را بین S_{i+4} و دومین کاراکتر P یعنی b شروع کنیم. برای آنکه بدانیم مقایسه را از کجای S انجام دهیم، تابع f را که به تابع شکست ($failure$) معروف است، تعریف می‌کنیم.

اگر $P = P_0 P_1 \dots P_{n-1}$ یک رشته باشد، تابع شکست (f) آن به صورت زیر تعریف می‌شود:

$$f(j) = \begin{cases} \text{Largest } k < j \text{ such that } P_0 P_1 \dots P_k = P_{j-k} P_{j-k+1} \dots P_j & (\text{اگر چنین } k \text{ باشد}) \\ -1 & \text{و در غیر این صورت} \end{cases}$$

به عنوان مثال برای رشته P داریم:

$$\begin{array}{cccccccccc} j & = & 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \\ P & = & a & b & c & a & b & c & a & c & a & b \\ j & = & -1 & -1 & -1 & 0 & 1 & 2 & 3 & -1 & 0 & 1 \end{array}$$

بنابراین اگر در مقایسه رشته P با S یک تطابق جزئی مثل $S_{i-j} \dots S_{i-1} = P_0 P_1 \dots P_{j-1}$ وجود داشت و $S_i \neq P_j$ در این صورت در مرحله بعدی مقایسه، باید S_i و $P_{f(j-1)+1}$ مقایسه شود (اگر $j \neq 0$). اگر $j = 0$ ، آنگاه S_{i+1} با P_0 مقایسه می‌شود. از نوشتن الگوریتم خودداری می‌کنیم ولی بدانیم الگوریتم KMP از مرتبه $O(n+m)$ است. توجه: برای f می‌توان رابطه زیر را نوشت:

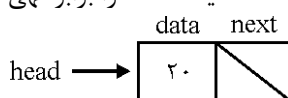
$$f(j) = \begin{cases} -1 & , \text{ if } j = 0 \\ f^m(j-1) + 1 & , \text{ if } P_{f^k(j-1)+1} = P_j \text{ که } K \text{ کوچکترین } K \text{ است} \\ -1 & , \text{ if } K \text{ وجود نداشته باشد.} \end{cases}$$

لیست پیوندی

ساختمان داده‌ای است که عناصر آن به ترتیب خطی هستند ولی برخلاف آرایه که ترتیب خطی با اندیس مشخص می‌شود، در لیست پیوندی این ترتیب با اشاره‌گرها که در هر عنصر است مشخص می‌شود. برای ذخیره تعدادی عنصر هم می‌توان از آرایه استفاده کرد و هم لیست پیوندی. اگر تعداد عناصر از قبل مشخص نباشد، لیست پیوندی بهتر است. برای عملیاتی مثل حذف و درج نیز لیست پیوندی بهتر است. لیست پیوندی انواع دارد: یکطرفه یا دو طرفه و حلقوی یا غیرحلقوی. در لیست یکطرفه هر نود فقط یک اشاره‌گر دارد (*link* یا *next*) و در لیست دو طرفه هر نود دو اشاره‌گر دارد (*next* و *prev*). نحوه تعریف نودهای لیست یک طرفه به زبان *C* و پاسکال می‌تواند به شکل زیر باشد:

پاسکال	زبان <i>C</i>
<i>type</i>	<i>struct node</i>
<i>node = record</i>	{
<i>data : items;</i>	<i>items data;</i>
<i>next : ^node;</i>	<i>node * next;</i>
<i>end;</i>	}

برای ساخت لیست، متغیری به نام *head* (یا *start*) از نوع اشاره‌گر به *node* تعریف می‌کنیم و با دستوری مثل *new(head)* نودی می‌سازیم که *head* به آن اشاره کند و سپس مثلاً با دستور $data(head) = ۲۰$ ($data(head) = ۲۰$ ، $head.data = ۲۰$ ، $head \wedge data = ۲۰$ و $(*head).data = ۲۰$)، مقدار *data* فیلد *head* را برابر ۲۰ قرار می‌دهیم حال با دستوری مثل $next(head) = nil$ فیلد *next* را برابر تهی (*nil* یا *null*) قرار می‌دهیم:



جستجو در لیست پیوندی: می‌خواهیم آدرس اولین نود شامل مقدار *k* را برگردانیم مرتبه $O(n)$ است (برای لیست *n* نودی)

```
search (k)
{
    x = head
    while (x ≠ nil and data(x) ≠ k)
        x = next (x)
    return (x)
}
```

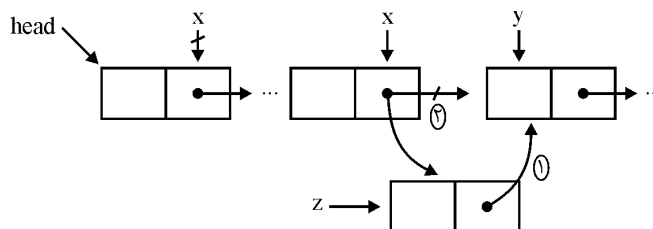

درج نود بعد از نود با آدرس مشخص y : ابتدا با دستور $new(x)$ نودی می‌سازیم که به آن اشاره کند سپس با دستور $next(x) = next(y)$ فیلد اشاره‌گر نود x را به نود بعد از y اشاره می‌دهیم و سپس با دستور $next(y) = x$ ، فیلد اشاره‌گر نود y را به نود جدید x اشاره می‌دهیم. مرتبه $\theta(1)$ است.

حذف نود با آدرس مشخص y : باید لیست پیمایش شود تا به نود قبل از y برسیم و سپس فیلد اشاره‌گر نود قبل از y را به نود بعد از y اشاره دهیم و سپس نود y را حذف کنیم مرتبه $O(n)$ است.

```
delete (y)
{
    x = head
    while (next(x) ≠ y)
        x = next(x)
    next(x) = next(y)
    dispose(y) // free(y)
}
```

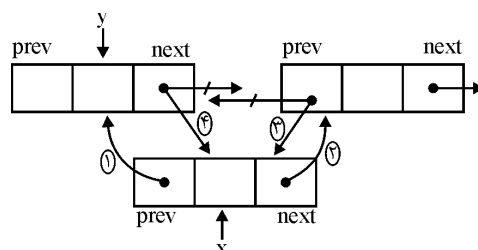
درج نود قبل از نود با آدرس مشخص y : لیست را پیمایش می‌کنیم تا به نود قبل از y برسیم و سپس عمل درج را انجام می‌دهیم، مرتبه $O(n)$ است (یعنی حداکثر $\theta(n)$ است)

```
x = head
while (next(x) ≠ y)
    x = next(x)
new (z)
(۱) next(z) = next(x)
(۲) next(x) = z
```



نتیجه: درج نود ابتدای لیست، حذف نود اول از مرتبه $\theta(1)$ هستند. درج نود انتهای لیست اگر آدرس نود آخر مشخص باشد از مرتبه $\theta(1)$ و اگر مشخص نباشد از مرتبه $\theta(n)$ است. حذف نود آخر از مرتبه $\theta(n)$ است، حتی اگر آدرس نود آخر مشخص باشد.

درج نود بعد از نود با آدرس مشخص y در لیست دو طرفه:



۴ دستور ۱ و ۲ و ۳ و ۴ را می‌توان به ۸ ترتیب مختلف نوشت:
۱۳۲۴ و ۳۱۲۴ و ۳۲۱۴ و ۳۲۴۱ و ۱۲۳۴ و ۲۱۳۴ و ۲۳۱۴ و ۲۳۴۱

```
new (x)
(۱) prev(x) = y
(۲) next(x) = next(y)
(۳) prev(next(y)) = x
(۴) next(y) = x
```

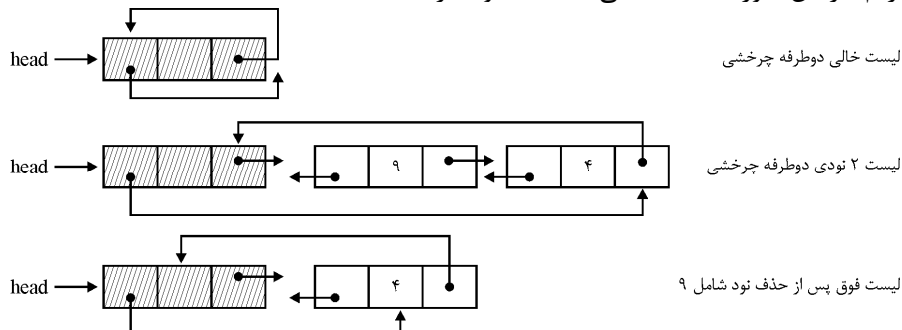
حذف گره با آدرس مشخص y از لیست دو طرفه:

```
delete (y)
{
    if (prev(y) ≠ nil)
        next (prev(y)) = next (y)
    else head = next (y)
    if (next(y) ≠ nil)
        Prev(next(y))=prev (y)
}
```

توجه: اگر بتوان از شرایط مرزی صرفنظر کرد آنگاه الگوریتم فوق را می‌توان ساده‌تر نوشت:

```
delete'(y)
{
    next (prev (y)) = next (y)
    prev (next (y)) = prev (y)
}
```

با استفاده از *sentinel* می‌توان از شرایط مرزی صرفنظر کرد. به این صورت که یک نود که خواص سایر نودها را دارد ولی در فیلد *data* مقدار معتبری ندارد را به عنوان سرلیست در نظر بگیریم. در این صورت لیست خالی فقط یک نود دارد:



الگوریتم جستجوی مقدار k در لیست فوق به شکل مقابل است. اگر نود با مقدار k وجود نداشته باشد، خروجی، $head$ است:

```
search (k)
{
    x = next(head)
    while (x ≠ head and data(x) ≠ k)
        x = next(x)
    return x
}
```

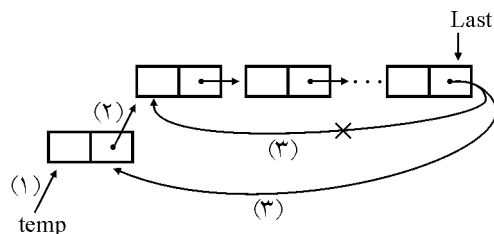
لیست یک طرفه حلقوی

مشابه لیست یک طرفه خطی است با این تفاوت که اشاره‌گر آخرین گره nil نیست، بلکه به اولین گره اشاره می‌کند.

در لیست خطی، برای پیمایش باید از گره اول شروع کرد ولی در لیست حلقوی با شروع از هر گره می‌توان لیست را پیمایش کرد. کلیه الگوریتم‌های لیست حلقوی، شبیه لیست خطی است؛ با این تفاوت که $Link$ گره آخر با $head$ مساوی است و نه با nil ؛ بنابراین بجای مقایسه با nil در الگوریتم‌های گفته شده، باید با $head$ مقایسه کنیم (هرجا که نیاز هست).

❖ **نکته:** در لیست حلقوی، می‌توان به جای حفظ آدرس سرلیست، آدرس گره آخر را نگهداری کرد. در این حالت برای اضافه کردن عنصری به سر لیست و ته لیست، نیاز به پیمایش نیست. به عنوان مثال دستورات زیر، یک گره به ابتدای یک لیست حلقوی غیرتهی با این فرض که اشاره‌گر $Last$ به گره آخر اشاره می‌کند، اضافه می‌کنند:

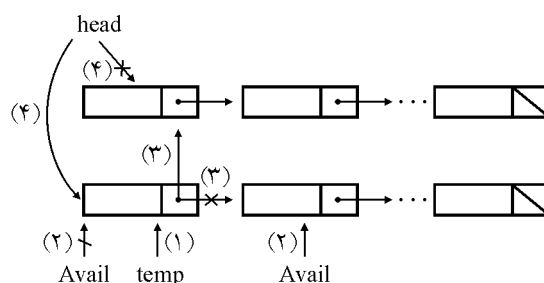
```
(۱) new(temp);
(۲) temp ^ .Link := Last ^ .Link;
(۳) Last ^ .Link := temp;
```



لیست Avail

لیست Avail یا لیست در دسترس حافظه، لیست حاوی گره‌های بلااستفاده است، که اشاره‌گر Avail به اولین گره آن اشاره می‌کند. با وجود لیست Avail فرض بر این است که هر گره‌ای که از لیست اصلی حذف می‌شود به لیست Avail اضافه می‌شود و برای اضافه کردن به لیست اصلی از گره‌های لیست Avail استفاده می‌شود.

- اضافه کردن گره به ابتدای لیست اصلی با وجود لیست Avail



$temp := Avail;$ (۱)

$Avail := Avail \wedge Link;$ (۲)

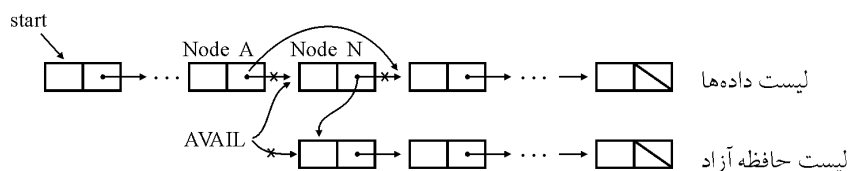
$temp \wedge Link := head;$ (۳)

$head := temp;$ (۴)

سایر الگوریتم‌ها نیز به همین شکل قابل پیاده‌سازی هستند. با وجود لیست Avail، هرگاه این لیست تهی باشد یعنی $Avail = Null$ و بخواهیم یک عنصر جدید به لیست اصلی اضافه کنیم، سرریزی (Overflow) است یعنی نمی‌توانیم گره‌ای به لیست اضافه کنیم. و همچنین اگر بخواهیم اطلاعاتی را از یک لیست خالی ($head = Null$) حذف کنیم، زیرریز (underflow) اتفاق می‌افتد.

حذف گره از لیست پیوندی با وجود AVAIL

گره از لیست اصلی حذف شده، به ابتدای لیست AVAIL اضافه می‌شود. در شکل زیر Node N را حذف و به اول لیست AVAIL افزوده‌ایم:



ملاحظه می‌شود که ۳ آدرس تغییر می‌کند.

الگوریتم DEL گره‌ای را که LOC به آن اشاره می‌کند را حذف کرده و به لیست $AVAIL$ می‌فرستد. در این الگوریتم، $LOCP$ مکان گره قبل از گره LOC می‌باشد. اگر LOC گره اول باشد آنگاه $LOCP = NULL$ خواهد بود:

$DEL(INFO, LINK, START, AVAIL, LOC, LOCP)$

۱. If $LOCP = NULL$, then :

$Set\ START := LINK[START].$ [Delete first node.]

Else:

$Set\ LINK[locp] := LINK[loc].$

[End of If structure.]

۲. $Set\ LINK[LOC] := AVAIL$ and $AVAIL := LOC$.

◀ **نکته:** با وجود لیست $AVAIL$ ، حذف یک عنصر از لیست یکطرفه، ۳ آدرس و اضافه کردن یک عنصر نیز، ۳ آدرس عوض می‌کند.

◀ **نکته:** بدون لیست $AVAIL$ ، حذف یک عنصر از لیست یکطرفه، یک آدرس و اضافه کردن یک عنصر، ۲ آدرس عوض می‌کند.

◀ **نکته:** در لیست دو طرفه، حذف یک عنصر موجب تغییر ۲ آدرس و اضافه کردن یک عنصر، موجب تغییر ۴ آدرس می‌شود.

نمایش چند جمله‌ای با لیست پیوندی

برای این منظور از لیست خطی یکطرفه استفاده می‌شود که شکل گره‌های آن به صورت زیر است.

$type\ polypointer = \wedge polynode;$

$polynode = record$

$coef: integer;$

$exp: integer;$

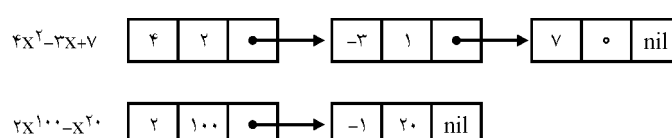
$link: polypointer$

end;

توان ضریب

coef	exp	link
------	-----	------

به عنوان مثال:



❖ **نکته:** الگوریتم جمع دو چند جمله‌ای که دارای m و n جمله هستند، از مرتبه $O(m+n)$ است.

❖ **نکته:** چند جمله‌ای را می‌توان با لیست چرخشی (حلقوی) نیز نشان داد.

❖ **نکته:** ماتریس اسپارس را می‌توان با لیست پیوندی نیز نمایش داد.

لیست عمومی (general list)

لیستی که هر گره آن بتواند شامل یک زیر لیست باشد را گویند.

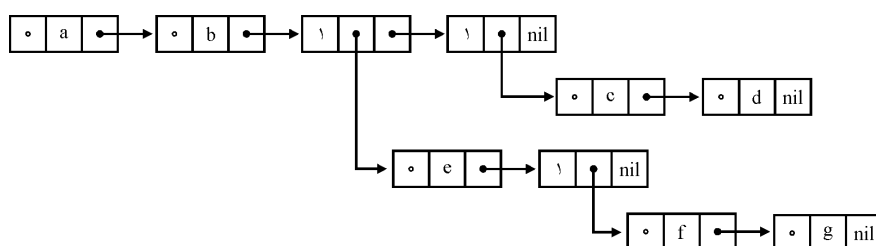
گره‌های لیست‌های عمومی به شکل

tag	data/dlink	link
-----	------------	------

 هستند.

اگر $tag=0$ باشد فیلد وسط به عنوان $data$ و اگر $tag=1$ باشد، فیلد وسط به عنوان اشاره‌گر می‌باشد. از لیست عمومی برای نمایش درخت می‌توان استفاده کرد. به عنوان مثال لیست L را می‌توان به شکل زیر در نظر گرفت:

$$L=(a,b,(e(f,g)),(c,d))$$



پشته و صف (stack, queue)

پشته و صف مجموعه‌های دینامیکی هستند که عناصر به ترتیب از پیش تعریف شده‌ای حذف و درج می‌شوند. در یک پشته، عنصری که حذف می‌شود، آخرین عنصری است که درج شده است، یعنی پشته سیاست *LIFO (Last In First Out)* را پیاده‌سازی می‌کند. در یک صف، عنصری که حذف می‌شود، عنصری است که مدت زمان بیشتری در صف بوده است یعنی صف سیاست *FIFO (First In First Out)* را پیاده‌سازی می‌کند. پشته و صف را می‌توان با آرایه و یا لیست پیوندی پیاده‌سازی کرد.

پشته

عمل اضافه کردن به پشته را *push* و عمل حذف از پشته را *pop* می‌نامیم. برای پیاده‌سازی پشته با آرایه، $s[1..n]$ را تعریف می‌کنیم و متغیر top نشان می‌دهد آخرین عنصری که *push* شده در کدام خانه است. بنابراین همیشه $s[1..top]$ خانه‌های پرپشته هستند.

<pre> push (s , x) { if(top == n) error "over flow" else { top = top + ۱ s[top] = x } } </pre>	<pre> pop(s) { if (top == ۰) error "under flow" else { top = top - ۱ return s[top + ۱] } } </pre>
--	---

این دو تابع از مرتبه $\theta(۱)$ هستند. با توجه به این توابع، top همیشه به خانه پر بالای پشته اشاره می‌کند، می‌توان عملیات را طوری نوشت که top به خانه خالی اشاره کند که به عنوان تمرین انجام دهید.

همچنین عمل pop را می‌توان طوری نوشت که یک متغیر به عنوان پارامتر دریافت کند و عنصر بالای پشته را در متغیر قرار دهد بنابراین pop را می‌توان به شکل مقابل نوشت:

```

Void pop (int & x)
{
    if (top == ۰)
        stack under flow ( );
    else
        x = s[top - -];
}

```

فرض کرده‌ایم $s[۱..n]$ پشته است و عناصر آن از نوع int هستند. دقت کنید $x = s[top - -]$ ابتدا $s[top]$ را به x منتقل می‌کند و سپس top را یک واحد کاهش می‌دهد.

مثال: پشته‌ای داریم که اعداد ۱ تا n داخل آن هستند (۱ پایین و n بالای پشته)، اگر با هر عمل pop پشته سر و ته شود، بررسی کنید آخرین عددی که pop می‌شود $\left\lfloor \frac{n+۱}{۲} \right\rfloor$ است و اگر پشته برعکس باشد (n پایین و ۱ بالای پشته) آنگاه آخرین عدد $\left\lfloor \frac{n+۲}{۲} \right\rfloor$ است.

مثال: اعداد ۱ تا ۵ به ترتیب صعودی در ورودی داده شده‌اند، این اعداد به ترتیب به یک پشته $push$ می‌شوند و هرگاه لازم بود pop می‌شوند و در خروجی نوشته می‌شوند، کدام یک از خروجی‌های زیر، قابل تولید نیست؟

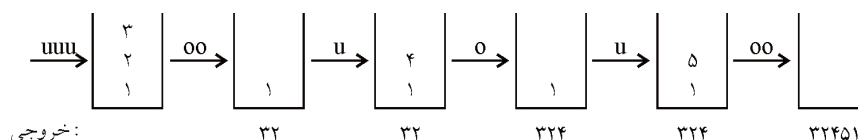
a) ۳ ۲ ۴ ۵ ۱

b) ۳ ۴ ۲ ۵ ۱

c) ۱ ۲ ۵ ۴ ۳

d) ۳ ۴ ۱ ۵ ۲

پاسخ: دنباله (a) را می‌توان به شکل زیر تولید کرد ($pop=o$ و $push=u$):



به همین ترتیب:

b) $uuuuouoooo$ c) $uououuuoooo$ d) $uuuuouo \times$

دنباله (d) قابل تولید نیست زیرا بعد از ۴ عدد ۱ قابل pop شدن نیست.

نکته: اگر ورودی صعودی باشد و در خروجی دنباله $x_1 \dots x_p \dots x_n \dots$ (چپ به راست) وجود داشته باشد که $x_p < x_n < x_1$ آنگاه آن دنباله قابل تولید نیست مثلاً در دنباله (d) اعداد ۲ و ۱ و ۳ دارای این خاصیت هستند ($1 < 2 < 3$).

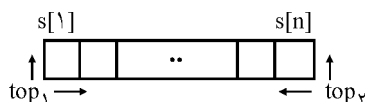
نکته: با n عدد ورودی، تعداد دنباله‌های قابل تولید از پشته برابر $\frac{1}{n+1} \binom{2n}{n}$ (عدد کاتالان)

است. مثلاً با ۳ عدد (۱, ۲, ۳) تعداد دنباله‌های قابل تولید برابر $\frac{1}{4} \binom{6}{3} = 5$ است که عبارتند از ۱۲۳ و ۱۳۲ و ۲۱۳ و ۲۳۱ و ۳۲۱. دنباله ۳۱۲ قابل تولید نیست.

پشته‌های چندگانه

اگر بخواهیم دو پشته را در یک آرایه پیاده‌سازی کنیم، بهترین راه این است که با استفاده از دو متغیر top_1 و top_2 که در جهت عکس هم حرکت می‌کنند، بالای هر پشته را مشخص کنیم. اگر

S یک آرایه n عنصری باشد آنگاه $S[1]$ ابتدای پشته اول و $S[n]$ ابتدای پشته دوم است. در این حالت داریم:



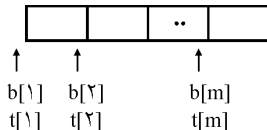
شرط خالی بودن پشته اول $top_1 = 0$

شرط خالی بودن پشته دوم $top_2 = n + 1$

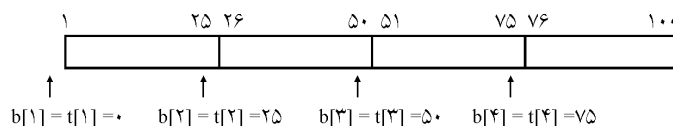
شرط پر بودن آرایه S (پر بودن هر دو پشته) $top_2 = top_1 + 1$

با این پیاده‌سازی از تمام فضای آرایه استفاده می‌شود.

ولی اگر بیش از دو پشته، در یک آرایه پیاده‌سازی شود، در این حالت روش فوق قابل اجرا نیست. در این حالت برای ساخت m پشته در آرایه $S[1..n]$ ، آرایه را m قسمت می‌کنیم. اندازه هر قسمت تقریباً برابر $\left\lfloor \frac{n}{m} \right\rfloor$ است. برای هر پشته i ، $b[i]$ به یکی کمتر از پایین‌ترین عنصر و $t[i]$ به بالاترین عنصر اشاره می‌کند. شرط خالی بودن پشته i ام، $b[i] = t[i]$ است. مقادیر اولیه $b[i]$ و $t[i]$ برابر است با $t[i] = b[i] = \left\lfloor \frac{n}{m} \right\rfloor (i - 1)$



به عنوان مثال اگر آرایه $n = 100$ عنصری باشد و بخواهیم $m=4$ پشته در آن بسازیم:



طبق فرمول داده شده مقادیر اولیه به صورت زیر محاسبه می‌شوند:

$$1 \text{ پشته } : i = 1 \rightarrow b[1] = t[1] = \left\lfloor \frac{100}{4} \right\rfloor (1 - 1) = 0$$

$$2 \text{ پشته } : i = 2 \rightarrow b[2] = t[2] = \left\lfloor \frac{100}{4} \right\rfloor (2 - 1) = 25$$

$$3 \text{ پشته } : i = 3 \rightarrow b[3] = t[3] = \left\lfloor \frac{100}{4} \right\rfloor (3 - 1) = 50$$

$$4 \text{ پشته } : i = 4 \rightarrow b[4] = t[4] = \left\lfloor \frac{100}{4} \right\rfloor (4 - 1) = 75, \quad b[5] = \left\lfloor \frac{100}{4} \right\rfloor (5 - 1) = 100$$

پشته i وقتی پر می‌باشد که $t[i] = b[i + 1]$. برای $push$ کردن در پشته i مقدار $t[i]$ افزایش می‌یابد و برای pop مقدار $t[i]$ کاهش می‌یابد عملیات $push$ و pop در پشته i به صورت زیر است:

Procedure push (i:integer; x:items);

begin

if t[i] = b[i + 1] then stackfull(i)

else begin

$t[i] := t[i] + 1;$

$S[t[i]] := x;$

end

end;

لازم به ذکر است برای آنکه پشته آخر (یعنی پشته m) دچار مشکل نشود، تعریف می‌کنیم $b[m + 1] = n$. که در این صورت شرط پر بودن پشته m نیز صحیح است.

Procedure pop (i:integer; Var x:items);

begin

if t[i] = b[i] then stackempty(i)

else begin

$x := S[t[i]];$

$t[i] := t[i] - 1;$

end

end;

ممکن است یکی از پشته‌ها سریع‌تر از دیگر پشته‌ها پر شود و به همین علت استفاده از فضای حافظه بهینه نخواهد بود. برای رفع این مشکل باید عناصر شیفت داده شوند تا در انتهای پشته پر شده فضای خالی تولید شود که این عمل در بدترین شرایط از مرتبه $O(n)$ خواهد بود.

توجه: در زبان C اگر بخواهیم داخل آرایه $S[n] \dots S[n - 1]$ پشته بسازیم آنگاه،

$t[0 \dots m - 1]$ و $b[0 \dots m]$ خواهد بود و مقدار اولیه $t[i]$ و $b[i]$ برابر $\left\lfloor \frac{n}{m} \right\rfloor - 1$ است و

اندیس شروع پشته شماره i ، برابر $i \times \left\lfloor \frac{n}{m} \right\rfloor$ است.

برای پیاده‌سازی پشته با لیست پیوندی، نودهایی حاوی دو فیلد ($link$ و $data$) تعریف

می‌کنیم. نود اول لیست، عنصر بالای پشته را نگه می‌دارد پس $push$ یعنی درج نود در ابتدای

لیست و *pop* یعنی خواندن *data* از نود اول و حذف آن می‌باشد: (*top* اشاره‌گر به نود اول است).
Type نوع داده‌هاست. هرگاه *top* برابر صفر (*Null*) باشد یعنی پشته خالی است)

node = record

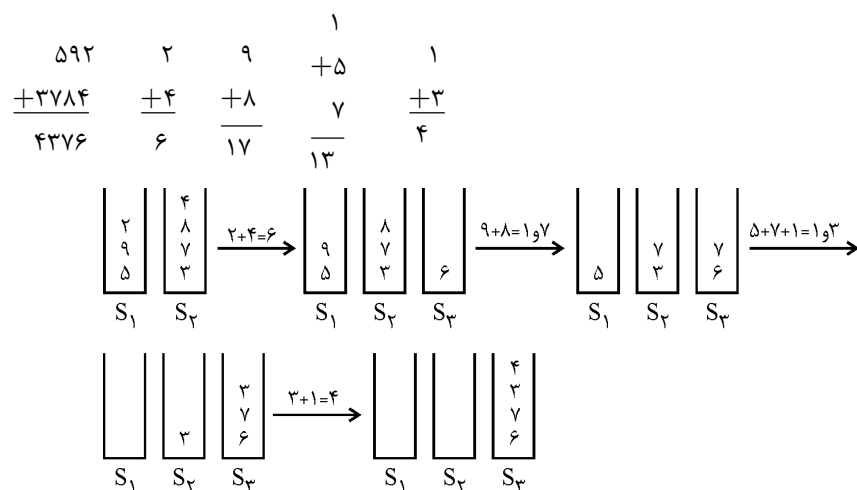
```
{
    Type data;
    Node * link;
}
```

```
push (item)
{
    temp = new node;
    if (temp ≠ ۰)
    {
        (temp → data) = item;
        (temp → link) = top;
        top = temp;
        return true;
    }
    else
    {
        Write ("out of space");
        return false;
    }
}
```

```
pop (item)
{
    if(top == ۰)
    {
        write ("stack is empty");
        return false;
    }
    else
    {
        item = (top → data);
        temp = top;
        top = (top → link);
        delete temp;
        return true;
    }
}
```

یکی از کاربردهای پشته جمع دو عدد بزرگ است، مثلاً جمع دو عدد ۲۰ رقمی. برای این منظور ارقام دو عدد را به دو پشته وارد می‌کنیم و تا زمانی که حداقل یکی از پشته‌ها خالی نشده، از دو پشته ارقام را *pop* کرده جمع کرده و حاصل جمع را به یک پشته دیگر وارد می‌کنیم و رقم نقلی جمع را در یک متغیر ذخیره می‌کنیم تا با ارقام بعدی که *pop* می‌شوند، جمع کنیم.

مثلاً جمع دو عدد ۵۹۲ و ۳۷۸۴ را نشان می‌دهیم (البته این دو عدد بزرگ نیستند ولی برای فهم الگوریتم خوب هستند)



ارزشیابی عبارات

ما برای بیان یک عبارت ریاضی از فرم اصطلاحاً *infix* (میانوندی) استفاده می‌کنیم مثل $a+b$. ولی دو فرم دیگر برای عبارت ریاضی وجود دارد که عبارتند از: *postfix* (پسوندی) که به آن فرم معکوس لهستانی (*RPN: Reverse Polish Notation*) نیز می‌گویند. در این فرم، عملگر (*Operator*) بعد از عملوندها (*Operands*) نوشته می‌شود: $ab+$ و فرم *Prefix* (پیشوندی) که به آن فرم لهستانی (*Polish Notation*) نیز می‌گویند. در این فرم عملگر، قبل از عملوندها نوشته می‌شود: $+ab$.

مزیت فرم‌های پسوندی و پیشوندی نسبت به میانوندی آن است که در این دو فرم اولویت مطرح نیست و نیاز به پرانتزگذاری نیست ولی در فرم میانوندی، پرانتزگذاری نیاز است. مثلاً عبارت میانوندی $a+b*c$ را می‌توان به دو صورت زیر پرانتزگذاری کرد:

$$a + (b * c) \Rightarrow \text{فرم پسوندی: } abc*+ \quad \text{فرم پیشوندی: } +a*bc$$

$$(a + b) * c \Rightarrow \text{فرم پیشوندی: } *+abc \quad \text{فرم پسوندی: } ab+c*$$

همانطور که ملاحظه می‌شود، فرم‌های پسوندی و پیشوندی نیاز به پرانتزگذاری ندارند. اولویت عملگرهایی که در این درس استفاده می‌شوند عبارتند از:

۱- $() [] \{ \}$

۲- *not* قرینه - ~ ! توان ↑

۳- $\% \div *$

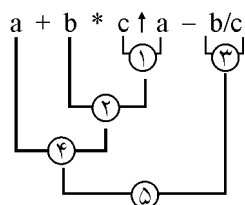
۴- + -

عملگرهای ردیف ۲ در یک عبارت از راست به چپ محاسبه می‌شوند.

مثلاً $not\ a \uparrow b$ به صورت $not\ (a \uparrow b)$ محاسبه می‌شود.

سایر عملگرها در یک عبارت از چپ به راست ارزیابی می‌شوند مثلاً عبارت $a/b * c$ به صورت $(a/b) * c$ محاسبه می‌شود.

مثال: ترتیب عملیات در عبارت زیر مشخص شده است.



تبدیل عبارت میانوندی به پسوندی و پیشوندی

- عبارت را طبق اولویت پرانتزگذاری کنید.

- برای تبدیل به پسوندی، هر عملگر را به سمت راست پرانتز بسته خودش منتقل کنید.

- تبدیل به پیشوندی هر عملگر را به سمت چپ پرانتز باز خودش منتقل کنید.

- پرانتزها را حذف کنید.

مثال: فرم پسوندی و پیشوندی عبارت میانوندی زیر را بیابید.

$$a + b * c \uparrow a - b / c$$

پاسخ: ابتدا پرانتزگذاری می‌کنیم.

$$\left(\left(a + (b * (c \uparrow a)) \right) - (b / c) \right)$$

$$abca \uparrow * + bc / -$$

فرم پسوندی:

$$\left(\left((a + (b * (c \uparrow a))) \right) - (b / c) \right)$$

$$- + a * b \uparrow ca / bc$$

فرم پیشوندی:

نکته: برای تبدیل فرم میانوندی به پسوندی از الگوریتم زیر که از پشته استفاده می‌کند، نیز

می‌توان کمک گرفت:

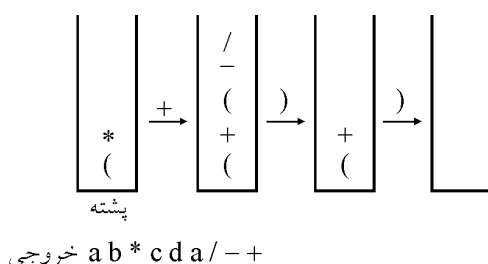
عبارت *infix* را از چپ به راست پیمایش می‌کنیم:

- پرانتزهای باز را به پشته *push* می‌کنیم.

- عملوندها را در خروجی می‌نویسیم.

- اگر به عملگر رسیدیم و روی پشته عملگری بود که اولویت آن بیشتر یا مساوی عملگر جدید بود، ابتدا آن را *pop* می‌کنیم در خروجی می‌نویسیم و سپس عملگر جدید را *push* می‌کنیم.
 - اگر به پرانتز بسته رسیدیم آنگاه عملگرهای بالای پشته را برداشته در خروجی می‌نویسیم تا به پرانتز باز برسیم، آن پرانتز باز را نیز از پشته حذف می‌کنیم ولی در خروجی نمی‌نویسیم.
مثال: عبارت میانوندی $(a*b+(c-d/a))$ را با کمک پشته به پسوندی تبدیل کنید.

پاسخ:



دقت کنید در این مثال ۶ عمل *push* انجام شد (به تعداد عملگرها بعلاوه پرانتزهای باز) و حداقل اندازه پشته لازم باید ۵ باشد.

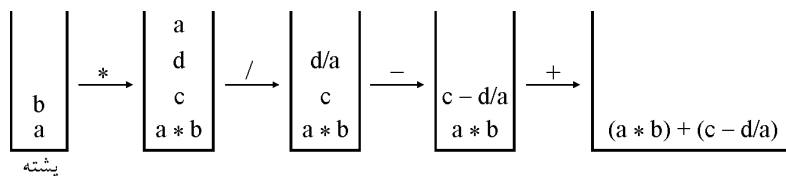
تبدیل پسوندی به میانوندی و پیشوندی

برای تبدیل عبارت پسوندی به میانوندی، می‌توان از پشته استفاده کرد. به این ترتیب که از سمت چپ به راست عبارت را پیمایش کرده و با رسیدن به عملوند آن را *push* می‌کنیم و با رسیدن به عملگر دو عملوندی (مثل +) دو عملوند بالای پشته را برداشته و حاصل عملیات آنها را در پشته قرار می‌دهیم.

مثال: فرم میانوندی عبارت زیر را بیابید:

$$a \ b \ * \ c \ d \ a \ / \ - \ +$$

پاسخ: عبارت را از چپ به راست پیمایش می‌کنیم:



برای تبدیل عبارت پسوندی به پیشوندی می‌توان ابتدا آن را به میانوندی تبدیل کرد. برای تبدیل مستقیم پسوندی به پیشوندی می‌توان عبارت را پرانتزگذاری کرد. به صورت (عملگر عملوندعملوند) یعنی هر دو عملوند و یک عملگر سمت راست آنها را یک پرانتز فرض کنیم. حال هر پرانتز خود یک عملوند محسوب می‌شود. سپس عملگر را به سمت چپ پرانتز باز خود منتقل کنیم.

مثال: عبارت پسوندی مثال قبل را با پرانتزگذاری به پیشوندی تبدیل کنید.

پاسخ: ابتدا پرانتزگذاری می‌کنیم:

$$\begin{aligned} & ((a \ b \ *) \ (c \ (d \ a \ /) \ -) \ +) \\ \Rightarrow & \ + \ * \ a \ b \ - \ c \ / \ d \ a \end{aligned}$$

نکته: از روش گفته شده در مثال قبل می‌توان برای تبدیل به میانوندی نیز استفاده کرد.

تبدیل پیشوندی به میانوندی و پسوندی

برای تبدیل به میانوندی می‌توان عبارت را از راست به چپ بررسی کرده، عملوندها را به پشته *push* کنیم. با رسیدن به یک عملگر دو عملوندی، دو مقدار بالای پشته را برداشته و حاصل عمل آنها را در پشته قرار دهیم. دقت کنید در این حالت عملوند بالایی پشته، عملوند اول محسوب می‌شود.

مثال ۲: عبارت پیشوندی $*+AB-CD$ را به میانوندی تبدیل کنید.

پاسخ: از راست به چپ پیمایش می‌کنیم:

$$\begin{array}{|c|} \hline C \\ \hline D \\ \hline \end{array} \Rightarrow \begin{array}{|c|} \hline A \\ B \\ C-D \\ \hline \end{array} \xRightarrow{+} \begin{array}{|c|} \hline A+B \\ C-D \\ \hline \end{array} \xRightarrow{*} (A+B)*(C-D)$$

برای تبدیل پیشوندی به پسوندی می‌توان عبارت را پرانتزگذاری کرد به صورت (عملوند عملوند عملگر) سپس هر عملگر را به بعد از پرانتز بسته خود منتقل کنیم. دقت کنید هر پرانتز، خود یک عملوند محسوب می‌شود.

مثال ۳: عبارت پیشوندی $^{\wedge} - * + ABC - DE + FG$ را پسوندی کنید. ($^{\wedge}$ توان است)

پاسخ: ابتدا پرانتزگذاری می‌کنیم.

$$\begin{aligned} & (^{\wedge} \ (\ - \ (\ * \ (\ + \ A \ B \) \ C \) \ - \ DE \) \ + \ FG \) \) \\ \Rightarrow & \ A \ B \ + \ C \ * \ DE \ - \ - \ F \ G \ + \ ^{\wedge} \end{aligned}$$

نکته: کاربردهای پشته عبارتند از ۱- ارزیابی عبارات ریاضی ۲- ذخیره آدرس بازگشت هنگام فراخوانی‌ها.

صف (Queue)

صف لیست مرتبی است که عمل اضافه کردن (*addq* یا *Enqueue*) از یک طرف آن که انتها (*rear*) نام دارد، انجام می‌شود و عمل حذف (*delq* یا *dequeue*) از طرف دیگر که ابتدا (*front*) نام دارد، انجام می‌شود. به صف لیست *FIFO* (*First In First out*) نیز می‌گویند. یعنی اولین عنصر ورودی، اولین عنصر خروجی است.

صف مثل پشته در حل بسیاری از مسائل کامپیوتر کاربرد دارد. بیشترین کاربرد صف در کامپیوتر، برای زمانبندی برنامه‌ها است. در پردازش دسته‌ای (*batch*)، برای خواندن و اجرای برنامه‌ها، بایستی آنها را به صف کرد تا یکی پس از دیگری اجرا شوند.

برای نمایش صف هم می‌توان از آرایه استفاده کرد و هم از لیست پیوندی. در اینجا آرایه بررسی می‌شود. متغیر *front* اول صف و *rear* انتهای صف را مشخص می‌کند.

ایجاد صف $q : \text{Array}[1..n] \text{ of items};$

$front, rear : 0..n$

مقدار اولیه $front = rear = 0;$

صف خالی است $\rightarrow front = rear$ if

صف پر است $\rightarrow rear = n$ if

عملیات حذف و اضافه روی صف به صورت زیر است:

Procedure addq (*x: items*) ;

begin

if *rear*=*n* *then*

queuefull ()

else begin

rear := *rear* + ۱;

q[rear]:=*x*;

end

end;

procedure delq (*Var x:items*);

begin

if *front*=*rear* *then*

queueempty ()

else begin

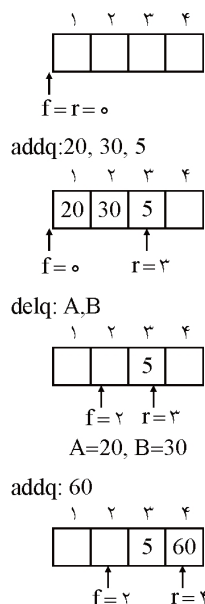
front:=*front*+۱ ;

x:=*q[front]*;

end;

end;

مثال ۴: دستورات روبرو را روی یک صف $n=4$ عنصری اجرا کنید.



با توجه به مثال اگر اکنون یک $addq$ دیگر انجام دهیم اعلام می‌شود صف پر است زیرا $r=n$ شده است، ولی آرایه پر نیست و این اشکال صف خطی ($Linear$) است. در واقع در صف خطی آرایه فقط یک بار قابل استفاده است. برای رفع این مشکل از صف حلقوی استفاده می‌شود.

صف دوار (حلقوی)

برای تعریف صف حلقوی، آرایه را بهتر است به صورت $q[0..n-1]$ تعریف کنیم. در این حالت وقتی $rear = n-1$ شد عنصر بعدی در $q[0]$ قرار می‌گیرد. در صف حلقوی نیز هرگاه $front=rear$ یعنی صف خالی است. مقداردهی اولیه به صورت $front = rear = 0$ می‌باشد. شرط پر بودن صف حلقوی آن است که $front = (rear + 1) \bmod n$ باشد، این شرط تضمین می‌کند که یک خانه خالی است (علت آن را خواهیم دید).

برای اضافه کردن به صف حلقوی باید هرگاه $rear = n-1$ بود صفر شود و در غیر این صورت $rear$ یک واحد اضافه شود که می‌توان با جمله $rear := (rear + 1) \bmod n$ این عمل را انجام داد. برای حذف از صف حلقوی باید هرگاه $front$ برابر $n-1$ شد، صفر شود. پس می‌توان جمله $front := (front + 1) \bmod n$ را موقع حذف نوشت. می‌توان صف حلقوی را به صورت زیر نمایش داد:

$front$ و $rear$ در جهت حرکت عقربه‌های ساعت حرکت می‌کنند.

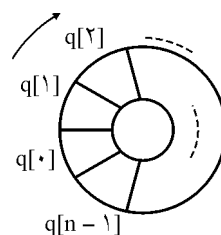
زیر برنامه‌های حذف و اضافه در صف حلقوی به صورت زیر است:

```

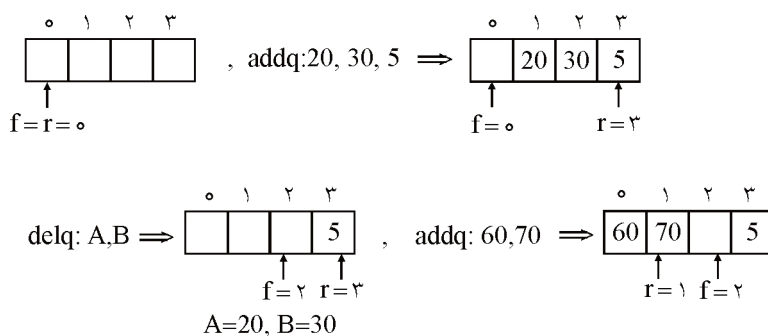
procedure addq (x: items);
begin
  if front = (rear + ۱) mod n then
    queuefull
  else begin
    rear := (rear + ۱) mod n;
    q[rear] := x
  end
end;

Procedure delq (Var x:items);
begin
  if front=rear then
    queueempty
  else begin
    front := (front+۱) mod n;
    x := q [front]
  end
end;

```



مثال ۵: عملیات زیر از چپ به راست روی یک صف حلقوی که $n=4$ ، نشان داده شده است:

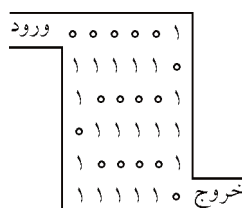


صف سو ندری

تمرین

۱- مسئله مسیر پرپیچ و خم ($MAZE$). یک موش را وارد جعبه‌ای بزرگ و روباز می‌کنیم. بسیاری از مسیرها داخل جعبه، بن‌بست است. موش باید چنان از پیچ و خم‌ها بگذرد تا به خروجی برسد (فقط یک راه خروج وجود دارد).

($MAZE$) را می‌توان با یک آرایه دوبعدی با مقادیر ۰ و ۱ نشان داد. صفر یعنی راه باز است و ۱ یعنی راه بسته است. اگر آرایه $Maze[1..m, 1..p]$ باشد، موش از $Maze[1, 1]$ وارد می‌شود و از $Maze[m, p]$ خارج می‌شود. لازم به ذکر است که هنگام پیاده سازی، یک حاشیه اطراف آرایه که همه عناصر آن یک هستند در نظر گرفته می‌شود یعنی $Maze[0..m+1, 0..p+1]$ می‌باشد همچنین آرایه کمکی به نام $Mark[0..m+1, 0..p+1]$ نیاز است که در ابتدا همه عناصر آن صفر هستند و وقتی به محل (i, j) برسیم $Mark[i, j]$ یک می‌شود. همچنین آرایه کمکی دیگری به نام $Move$ برای مشخص کردن جهت حرکت نیاز است. الگوریتم یافتن مسیر، از پشته استفاده می‌کند اندازه پشته باید mp باشد. در شکل زیر یک مارپیچ با طولانی‌ترین مسیر نشان داده شده است. الگوریتم $Maze$ را بنویسید.



در ضمن الگوریتم یافتن مسیر پرپیچ و خم از مرتبه $O(mp)$ خواهد بود، (m تعداد سطرها و P تعداد ستون‌ها است).

۲- پشته، حذف و اضافه را فقط از یک سمت انجام می‌دهد و صف از یک سمت حذف و از سمت دیگر درج می‌کند. یک $deque$ ($double\ ended\ queue$) حذف و درج را از دو طرف اجازه می‌دهد. ۴ رویه با مرتبه $O(1)$ که حذف و درج را در دو طرف $deque$ که با آرایه ساخته شده، بنویسید.

۳- با دو پشته یک صف بسازید، طوریکه هزینه n عمل، $O(n)$ باشد.

۴- با دو صف یک پشته بسازید.

۵- توضیح دهید چگونه می‌توان یک لیست دو طرفه را با استفاده از فقط یک اشاره‌گر $x \cdot np$ در هر نود x پیاده‌سازی کرد (در حالت عادی هر نود ۲ اشاره‌گر $next$ و $prev$ می‌خواهد). فرض کنید اشاره‌گر یک عدد صحیح k بیتی است و تعریف کنید: $XOR \ x \cdot prev \ x \cdot np = x \cdot next$

الگوریتم‌های $SEARCH$ و $INSERT$ و $DELETE$ را روی این لیست پیاده‌سازی کنید. همچنین نشان دهید چگونه می‌توان این لیست را با مرتبه $O(1)$ معکوس کرد.

۶- برای هر یک از ۴ نوع لیست جدول زیر، زمان اجرای بدترین حالت برای هر یک از عملیات را مشخص کنید.

	لیست دو طرفه مرتب	لیست دو طرفه نامرتب	لیست یک طرفه مرتب	لیست یک طرفه نامرتب
$SEARCH(L, K)$				
$INSERT(L, X)$				
$DELETE(L, X)$				
$SUCCESSOR(L, X)$				
$PREDECESSOR(L, X)$				
$MINIMUM(L)$				
$MAXIMUM(L)$				

در الگوریتم‌های $SEARCH$ ، k کلیدی است که جستجو می‌کنیم. در الگوریتم $INSERT$ ، x اشاره‌گر به عنصری است که می‌خواهیم درج کنیم و کلید این عنصر قبلاً مقداردهی شده است. در الگوریتم $DELETE$ ، x اشاره‌گر به نودی است که باید حذف شود. $SUCCESSOR(L, x)$ عنصر با آدرس x را دریافت می‌کند و آدرس عنصری که کلیدش کوچکترین کلید بزرگتر از کلید x است را برمی‌گرداند.

۷- ماتریس $A_{m \times n}$ را آرایه $Monge$ گویند اگر برای هر i, j, k, l که $1 \leq i \leq k \leq m$ و $1 \leq j \leq L \leq n$ داشته باشیم.

$$A[i, j] + A[k, L] \leq A[i, L] + A[k, j]$$

به عبارت دیگر اگر ۴ عنصر از ماتریس به شکل

$$\begin{array}{cc} a & \dots & b \\ \vdots & & \vdots \\ c & \dots & d \end{array}$$

در نظر بگیریم $a + d \leq b + c$

باشد. مثلاً ماتریس 7×5 زیر $Monge$ است:

(a) ثابت کنید یک آرایه $Monge$ است اگر و فقط اگر برای هر $i = 1, 2, \dots, m-1$ و $j = 1, 2, \dots, n-1$ داشته باشیم:

$$A[i, j] + A[i+1, j+1] \leq A[i, j+1] + A[i+1, j]$$

۱۰	۱۷	۱۳	۲۸	۲۳
۱۷	۲۲	۱۶	۲۹	۲۳
۲۴	۲۸	۲۲	۳۴	۲۴
۱۱	۱۳	۱۶	۱۷	۷
۴۵	۴۴	۳۲	۳۷	۲۳
۳۶	۳۳	۱۹	۲۱	۶
۷۵	۶۶	۵۱	۵۳	۳۴

b فرض کنید $f(i)$ اندیس ستون شامل سمت چپ‌ترین عنصر می‌نیمم در سطر i باشد. ثابت

$$\text{کنید } f(1) \leq f(2) \leq \dots \leq f(m)$$

c یک الگوریتم تقسیم و غلبه که سمت چپ‌ترین می‌نیمم را در هر سطر $A_{m \times n}$ می‌یابد عبارت است از:

سطرهای شماره زوج A را در ماتریس جدید A' قرار دهید و به طور بازگشتی چپ‌ترین می‌نیمم را برای هر سطر A' بیابید. سپس چپ‌ترین می‌نیمم را در سطرهای فرد A بیابیم.

توضیح دهید چگونه چپ‌ترین می‌نیمم در سطرهای فرد A را با فرض اینکه چپ‌ترین می‌نیمم در سطرهای زوج A معلوم است، با مرتبه $O(m+n)$ بیابیم.

d نشان دهید زمان اجرای الگوریتم قسمت c از مرتبه $O(m+n \cdot \lg m)$ است.

سؤال‌های طبقه‌بندی شده فصل سوم

۱- اگر آرایه $A[1..n, 1..m]$ در آرایه $B[1..m \times n]$ به روش سطر به سطر ذخیره شده

باشد، آدرس عنصر $A[i, j]$ در آرایه B کدام است؟

$$(1) \quad (i-1) * m + j \quad (2) \quad (j-1) * n + i$$

$$(3) \quad (i-1) * n + j \quad (4) \quad (j-1) * m + i$$

۲- آرایه ۳ بعدی $A[1..m, 1..n, 1..p]$ در یک آرایه یک بعدی $B[1..m \times n \times p]$ به روش

سطر به سطر ذخیره شده است. آدرس عنصر $A[I, J, K]$ در آرایه B کدام است؟

$$(1) \quad (I-1)np + (J-1)m + (K-1)$$

$$(2) \quad (I-1)np + (J-1)p + (K-1)$$

$$(3) \quad mnp + np + 1$$

$$(4) \quad Inp + Jp + K$$

۳- اگر آرایه A به صورت $\text{int } A[20][10][5]$ تعریف شده باشد (یعنی محدوده اندیس‌ها به صورت

$A[0..19][0..9][0..4]$ باشد). با فرض این که هر عدد int چهار بایت لازم دارد و آدرس شروع

آرایه صفر باشد، آدرس عنصر $A[2][4][2]$ در حالت ستون محور (*column major*) عبارت

است از:

$$(1) \quad 1934 \quad (2) \quad 1932 \quad (3) \quad 484 \quad (4) \quad 483$$

۴- تعریف آرایه $\text{long } L[10][20][30]$ را در زبان C در نظر بگیرید. اگر هر خانه از نوع long

چهار بایت حافظه را اشغال کند و این آرایه از آدرس واقعی ۱۰۰۰۰ در مبنای ده شروع شود

و تخصیص حافظه به صورت سطری (*row major*) باشد، برای دسترسی به عنصر

$L[5][10][20]$ این آرایه، آدرس اولین بایت مورد دسترسی توسط کدام گزینه مشخص

شده است؟

(۸۳ IT)

$$(1) \quad 13280 \quad (2) \quad 23280 \quad (3) \quad 3320 \quad (4) \quad 13320$$

۵- آرایه $M(30 \times 20 \times 10)$ را که هر عنصر آن ۴ بایت حافظه نیاز دارد به صورت ستونی

(*Column major*) از بایت ۲۰۰۰۰ (مبنای ۱۰) در حافظه ذخیره می‌نماییم (یعنی به

ترتیب ۱۰ ماتریس 30×20 که هر ماتریس خود به صورت ستونی ذخیره شده است) در

این صورت آدرس خانه (۹ و ۱۱ و ۲۱) M از چه بایتی شروع می‌شود؟

(دولتی ۷۸)

$$(1) \quad 20480 \quad (2) \quad 40480 \quad (3) \quad 40840 \quad (4) \quad 20840$$

۶- می‌خواهیم حاصلضرب $ABCD$ یک ماتریس 5×13 و B یک ماتریس 89×5 و C یک ماتریس 3×89 و D یک ماتریس 3×34 می‌باشد را پیدا کنیم بطوری که کمترین تعداد عمل ضرب انجام گیرد. ترتیب ضرب ماتریس‌ها عبارت است از: (دولتی ۷۹)

$$(A(BC))D \quad (1) \quad A(BC)D \quad (2) \quad (AB)(CD) \quad (3) \quad ((AB)C)D \quad (4)$$

۷- ۴ ماتریس زیر با ابعاد داده شده را در نظر بگیرید: (طراحی الگوریتم و IT ۸۴)

$$A_{10 \times 2}, B_{2 \times 25}, C_{25 \times 3}, D_{3 \times 4}$$

کدام یک از گزینه‌های زیر مربوط به پرانتزبندی بهینه برای محاسبه حاصلضرب آنهاست؟

$$\begin{aligned} (1) \quad & (A \times B) \times (C \times D) \quad (2) \quad (A \times (B \times C)) \times D \\ (3) \quad & A \times ((B \times C) \times D) \quad (4) \quad ((A \times B) \times C) \times D \end{aligned}$$

۸- قطعه برنامه زیر برای ضرب دو ماتریس مربع A و B با مرتبه n مورد نظر است. برای تکمیل این برنامه، دستور حذف شده به صورت است. (علوم کامپیوتر ۷۹)

for $i := 1$ to n do

for $j := 1$ to n do

begin

$C[i, j] := 0;$

for $K := 1$ to n do

دستور حذف شده

end;

$$C[i, j] := A[i, K] * B[K, j] + C[i, j]; \quad (1)$$

$$C[i, j] := A[i, j] * B[K, j]; \quad (2)$$

$$C[i, j] := A[i, k] * B[k, j]; \quad (3)$$

$$C[i, j] := A[i, j] * B[k, j] + C[i, j]; \quad (4)$$

۹- فرض کنید A یک ماتریس $m \times n$ خلوت (*Sparse*) باشد عناصر غیرصفر آن را به ترتیب سطری در یک آرایه، $3 \times t$ که t تعداد عناصر غیرصفر ماتریس است ذخیره می‌کنیم. بهترین تابع زمان عمل ترانهاده‌گیری برابر است با (علوم کامپیوتر ۸۰)

$$O(tm) \quad (2) \quad O(t+m) \quad (1)$$

$$O(t+n) \quad (4) \quad O(tn) \quad (3)$$

۱۰- زمان ترانهاده گرفتن از یک ماتریس خلوت (*Sparse*) چقدر است اگر n تعداد سطرها و m تعداد ستون‌ها و t تعداد عناصر غیرصفر ماتریس باشد. (آزاد ۸۲)

$$O(nm) \quad (۴) \quad O(mnt) \quad (۳) \quad O(mt) \quad (۲) \quad O(nt) \quad (۱)$$

۱۱- در یک آرایه دو بعدی ۵×۵ مقادیر خانه‌های سطر اول همگی یک می‌باشد. اگر محتوای بقیه خانه‌های ستون‌های ۱ و ۵ برابر با صفر بوده و داشته باشیم:

$$A(I, J) = \frac{1}{2}(A(I-1, J-1) + A(I-1, J+1))$$

برای I از ۲ تا ۵ و J از ۲ تا ۴

محتوای خانه‌های سطر سوم این آرایه چند خواهد بود؟ (آزاد ۷۱ و ۷۲)

$$\begin{aligned} & \begin{matrix} ۱ & ۱ & ۱ & ۰ & (۲) \\ ۰ & \frac{1}{2} & \frac{1}{2} & \frac{1}{2} & ۰ & (۳) \\ ۰ & \frac{1}{4} & \frac{1}{2} & ۱ & ۰ & (۴) \end{matrix} \end{aligned}$$

۱۲- فرض کنی (i, j) و (K, L) دو عنصر در یک صفحه ۸×۸ باشند (مثل صفحه شطرنج) کدام یک از گزینه‌های زیر هم قطر بودن آنها را تعیین می‌کند؟ (تهدید دو مهره فیل) (علوم کامپیوتر ۷۹)

$$if (i = K) \text{ or } (j = L) \text{ then } (۱)$$

$$if (i - K = j - L) \text{ or } (i - K = L - j) \text{ then } (۲)$$

$$if (i - K = j - L) \text{ then } (۳)$$

$$if (i - K = j - L) \text{ or } (i - K = L + ۸ - j) \text{ then } (۴)$$

۱۳- آرایه مرتب A داده شده است. تابع زیر برای جستجوی دودویی در A (از اندیس L تا R) برای پیدا کردن x نوشته شده است:

function search(*x:real*; *L, R:integer*): *Boolean*;

Var

M:integer;

Begin

$M := (L + R) \text{ div } 2;$

if ($x = A[M]$) *then return* (*true*);

if ($x < A[M]$) *and* ($M > L$) *then return* (*search*($x, L, M - 1$));

if ($x > A[M]$) *and* ($M < R$) *then return* (*search*($x, M + 1, R$));

return false;

end;

کدامیک از گزینه‌های زیر در مورد الگوریتم فوق برای جستجوی دودویی در $A[1..N]$ غلط است؟ (منظور از مقایسه، مقایسه x با یک عنصر در A است) (دهلی ۷۷)

- (۱) اگر x در $A[1..N]$ باشد، همواره با حداکثر $O(\log_2^N)$ پیدا می‌شود.
 (۲) اگر x در $A[1..N]$ نباشد، پاسخ $false$ همواره با حداکثر $O(\log_2^N)$ مقایسه بدست می‌آید.
 (۳) در جستجوی ناموفق برای دو مقدار متفاوت x (که در A موجود نیستند) همواره تعداد مقایسه‌ها برابر است.
 (۴) حداکثر تعداد مقایسه‌ها (چه موفق و چه ناموفق) همواره از $O(N)$ کمتر است.

۱۴- فرض کنید یک آرایه ۲۰۰ عنصری مرتب شده باشد. زمان اجرای بدترین $(F(n))$ برای پیدا کردن عنصر معلوم x در آرایه A با استفاده از جستجوی دودویی چیست؟ (دهلی ۷۷)

(۱) ۲۰۰ (۲) ۸ (۳) ۷ (۴) ۱۲

۱۵- اگر A آرایه‌ای مرتب از اعداد صحیح ۱ الی ۱۰۲۴ باشد، الگوریتم جستجوی دودویی با چند بار تکرار عدد ۴ را پیدا می‌کند؟

(۱) ۸ (۲) ۷ (۳) ۹ (۴) ۱۰

۱۶- در صورتیکه آرایه مورد جستجو در جستجوی دودویی به صورت ۷ و ۶ و ۵ و ۴ و ۳ و ۲ و ۱ و ۰ و ۱- باشد، متوسط تعداد مقایسه‌ها برای جستجوی موفق چیست؟ (دهلی ۷۹)

- (۱) $\frac{۲۷}{۹}$ (۲) $\frac{۲۵}{۹}$ (۳) $\frac{۳۱}{۹}$ (۴) هیچکدام

۱۷- فرض کنید آرایه مورد جستجو توسط جستجوی دودویی به صورت (۱۲۰ و ۱۰۱ و ۸۲ و ۵۴ و ۹ و ۷ و ۳ و ۲ و ۰ و ۶-) باشد متوسط تعداد مقایسه‌های مورد نیاز برای حالت جستجوی موفق چیست؟ (آزاد ۷۹)

- (۱) $\frac{۲۸}{۱۰}$ (۲) $\frac{۱۸}{۱۰}$ (۳) $\frac{۲۹}{۱۰}$ (۴) $\frac{۲۶}{۹}$

۱۸- دو آرایه n تایی A و B حاوی اعداد حقیقی و یک عدد M داده شده‌اند. می‌خواهیم در صورت وجود یک i و یک j پیدا کنیم به طوری که $A[i] + B[j] = M$. بهترین الگوریتم برای حل این مسئله از چه مرتبه‌ای است؟ (علوم کامپیوتر سراسری ۸۱)

- (۱) $O(n \log n)$ (۲) $O(n)$ (۳) $O(n^2)$ (۴) $\Omega(n^2)$

۱۹- تابع روبرو به منظور جستجوی دودویی برای x در یک آرایه صعودی مرتب شده از اعداد صحیح نوشته شده است. بجای آرایه از اشاره گر به ابتدا و انتهای آرایه استفاده شده است.

چه عبارتی باید بجای (A) نوشته شود؟ (علوم کامپیوتر ۸۳)

```

int* bsearch(int* p, int* q, int x) {
    (1)  $(p + q) / 2$ 
    if (p == q) (2)  $(p + q + 1) / 2$ 
        return (*p) == x ? p : NULL; (3)  $p + (q - p) / 2$ 
    int* r = (A); (4)  $p + (q - p) / 2 + 1$ 
    if ((*r) < x)
        return bsearch(r + 1, q, x);
    else
        return bsearch(p, r, x);
}

```

۲۰- اگر $s(n)$ متوسط تعداد مقایسه‌ها برای جستجوی موفق در یک آرایه مرتب با طول n و $u(n)$ متوسط تعداد مقایسه‌ها برای جستجوی ناموفق در این آرایه با استفاده از روش جستجوی دودویی باشد. کدام یک از گزینه‌های زیر نادرست است؟ (۸۳ IT)

(1) $s(n) = \theta(u(n))$ (2) $s(n) = u(n) - 1$ (3) $s(n) = \left(1 + \frac{1}{n}\right)u(n) - 1$ (4) موارد ۱ و ۳ صحیح است.

۲۱- در ابتدای برنامه روبرو می‌خواهیم درایه‌های یک ماتریس را بطور شطرنجی با صفر و یک پر کنیم. چه دستوری باید در خط (۵) نوشته شود؟ (علوم کامپیوتر ۸۳)

```

1 - main() {
    T[J / ۸][J % ۸] = (J + (J / ۸)) % ۲; (1)
2 - int T[۸][۸];
    T[J / ۸][J % ۸] = (J % ۲); (2)
3 - int J;
    T[J % ۸][J / ۸] = (J + (J / ۸)) % ۲; (3)
4 - for (J = ۰; J < ۶۴; ++J)
    T[J % ۸][J / ۸] = (J % ۲); (4)
5 - .....
6 - .....
7 - }

```

۲۲- کار تابع f بر روی رشته s با n کاراکتر چیست؟ تابع $sub(s, i, n)$ تعداد n کاراکتر از موقعیت i در رشته s را برمی گرداند.

$$f(s, n) = \begin{cases} s & n = 1 \\ f(sub(s, 1, n-1), n-1) + sub(s, n, 1) & n > 1 \end{cases}$$

(۱) رشته s را برمی گرداند.

(۲) معکوس رشته s را برمی گرداند.

(۳) یک کاراکتر از انتها به رشته s اضافه می کند.

(۴) یک کاراکتر به ابتدای رشته s اضافه می کند.

۲۳- رشته " $ABCD$ " چند زیر رشته دارد؟ (آزاد ۷۷)

(۱) ۴ (۲) ۵ (۳) ۱۱ (۴) ۱۰

۲۴- یک لیست خطی یکطرفه با دو اشاره گر F و R که به ترتیب به عنصر اول و آخر لیست اشاره می کنند پیاده سازی شده است. هزینه کدامیک از اعمال زیر وابسته به تعداد عناصر لیست است؟ (دوای ۸۰)

(۱) حذف اولین عنصر (۲) حذف آخرین عنصر
(۳) درج یک عنصر در انتهای لیست (۴) درج یک عنصر در ابتدای لیست

۲۵- زیر برنامه g بر روی لیست پیوندی یکطرفه کدام عمل را انجام می دهد؟

(۱) پیمایش لیست
 $Procedure\ g(Var\ Start: Nodeptr);$
 Var (۲) حذف کردن لیست
 $p, q, r: Nodeptr;$ (۳) معکوس کردن لیست
 $begin$ (۴) مرتب کردن لیست
 $p:=start;$
 $q:=Nil;$
 $while\ p<>\ nil\ do$
 $begin$
 $r:=q;\ q:=p;$
 $p:=p^.Link;$
 $q^.Link:=r;$
 $end;$
 $Start:=q;$
 $end;$

۲۶- تابع زیر چه عملی انجام می‌دهد؟ (با فرض اینکه نوع *list* اشاره‌گر است)

```

list x(list L)
(۱) محل دو عنصر در لیست L را جابه‌جا می‌کند.
{   list m,t;
    m=NULL;
    while(L) {
        (۲) لیست پیوندی L را معکوس می‌کند.
        (۳) عنصری را از لیست L جابه‌جا می‌کند.
        (۴) لیست L را مرور می‌کند.
        t=m; m=L;
        L = L → link;
        m → link = t; }
    return m; }

```

۲۷- تابع زیر چه عملی انجام می‌دهد؟ توضیح اینکه *List* نشان دهنده لیستی از اعداد بوده و منظور از تابع *Head* تابعی است که مقدار اولین عنصر در لیست را برمی‌گرداند و تابع *Tail* لیستی حاوی همه عناصر لیست ورودی به استثنای اولین عنصر را برمی‌گرداند. مکان‌های لیست را از مکان ۱ در نظر بگیرید: (دوالتی ۷۷)

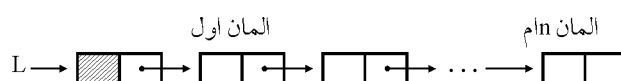
```

function what (L:List): integer;
begin
    if L=nil then
        what := ۰;
    else
        if Tail(L) ≠ nil then
            what := (Head(L) + what (Tail(Tail(L))))
        else
            what := Head(L)
    end;

```

- (۱) تعداد عناصر لیست را برمی‌گرداند.
- (۲) مجموع عناصر در مکان‌های فرد لیست ورودی را برمی‌گرداند.
- (۳) تعداد عناصر در مکان‌های فرد لیست ورودی را برمی‌گرداند.
- (۴) مجموع عناصر لیست ورودی را برمی‌گرداند.

۲۸- لیست یکطرفه و خطی *L* با سر لیست بصورت زیر داده شده است: (دوالتی ۷۱)



بر روی این لیست تکه برنامه زیر را اجرا می‌کنیم:

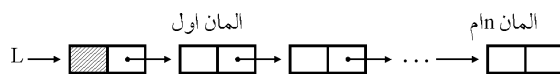
```
P := L ↑ .next;
while P <> nil do begin
  q := P;
  while q <> nil do begin
    q := q ↑ .next;
  writeln ('**');
  end;
  P := P ↑ .next;
end;
```

اگر تعداد المان‌های این لیست n باشد، تعداد دفعاتی که دستور `writeln` اجرا می‌شود نسبتاً برحسب n برابر است با:

$$\frac{n(n+1)}{2} \quad (۱) \qquad \frac{n(n-1)}{2} \quad (۲) \qquad n(n+1) \quad (۳) \qquad n(n-1) \quad (۴)$$

۲۹- لیست خطی یک طرفه و با سر لیست L داده شده است.

(دولتی ۷۱)



فرض کنید:

```
type list = ↑ Celltype;
celltype=record
  element:integer;
  next:List;
end;
```

تابع زیر چه کاری انجام می‌دهد؟

```
function F(L:List): List;
begin
  if (L = nil) OR (L ↑ .next = nil) then F := L
  else begin
    F := F(L ↑ .next);
    L ↑ .next ↑ .next := nil;
    L ↑ .next := nil;
  end
end;
```

- (۱) لیست L را معکوس می‌کند. (۲) در لیست L تغییری نمی‌دهد.
(۳) لیست L را مرتب می‌کند. (۴) هیچکدام

۳۰- می‌خواهیم تغییراتی در یک لیست پیوندی اعمال کنیم که عمل افزودن عنصر ابتدا و یا انتهای لیست با عملیاتی از مرتبه $O(1)$ قابل انجام باشد. لیست پیوندی را (علوم کامپیوتر ۷۹)

(۱) حلقوی می‌کنیم.

(۲) دو طرفه می‌کنیم.

(۳) حلقوی می‌کنیم و آدرس آخرین عنصر را برای دسترسی به لیست ذخیره می‌کنیم.

(۴) معکوس می‌کنیم.

۳۱- الگوریتم زیر چیست؟

procedure print (L: listpointer);

begin

if (L<>nil) then begin

if tag(L)=0 then

write (data(L))

else

print (data(L));

print (Link(L));

end

end;

(۱) پیمایش لیست حلقوی

(۲) پیمایش لیست عمومی

(۳) شمارش تعداد گره‌های لیست عمومی

(۴) حذف گره‌های لیست عمومی

۳۲- اگر سه اشاره‌گر $P1$ و $P2$ و $P3$ روی لیست L حرکت کنند و سه تابع $First$ و $Next$ و end مقدار منطقی متناسب را برگردانند، تعداد دفعات اجرای تابع $first$ بر حسب n (تعداد

دولتی ۸۱)

عناصر لیست) چیست؟

$P1:=First(L);$

while $P1<>end(L)$ *do begin*

$P2:=P1;$

while $P2<>end(L)$ *do begin*

$P2:=Next(P2,L);$

$P3:=First(L);$

while $P3<>P2$ *do* $P3:=Next(P3,L)$

end;

$P1:=Next(P1,L);$

end;

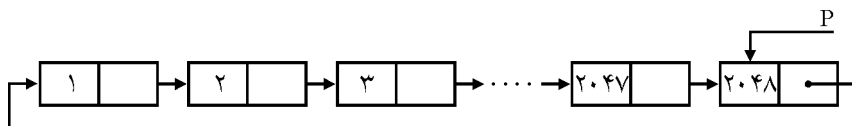
$$(1) \frac{(n-1)^2}{2} + 1$$

$$(2) \frac{(n-1)^2}{2} + 1$$

$$(3) \frac{n}{2}(n-1) + 1$$

$$(4) \frac{n}{2}(n+1)$$

۳۳- با وجود بودن لیست پیوندی حلقوی به شکل زیر، خروجی قطعه کد داده شده چیست؟ فرض کنید هر عنصر لیست پیوندی از دو مؤلفه *CellData* از نوع *Integer* و *Next* از نوع اشاره گر تشکیل شده باشد. (دهانی ۷۶)



while $P \uparrow .next \neq P$ *do*

begin

$P \uparrow .next := P \uparrow .next \uparrow .next;$

$P := P \uparrow .next$

end;

write($P \uparrow .CellData$);

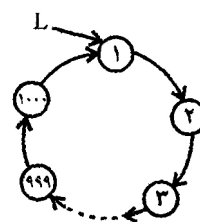
۲۰۴۷ (۱)

۱ (۲)

۲۰۴۸ (۳)

۱۰۲۴ (۴)

۳۴- با فرض اجرای تابع بر روی لیست پیوندی حلقوی شکل زیر، مقدار خروجی چقدر است؟ (۸۸ - IT)



int $S(List * L)$

{

if ($L \rightarrow next == L$) *return* $L \rightarrow data$;

$L \rightarrow next = L \rightarrow next \rightarrow next$;

return $S(L \rightarrow next)$;

}

۱۰۰۰ (۴)

۹۷۷ (۳)

۳۲۷ (۲)

۱ (۱)

۳۵- با توجه به تابع روبه‌رو و لیست حلقوی مذکور به ازای مقادیر n برابر ۷۲۹ و ۲۲۰۰ مقدار خروجی به ترتیب برابر چند خواهد بود؟ (مهندسی کامپیوتر ۸۹)

int $SO(List * L)$ {

while ($L \rightarrow next \neq L$) {

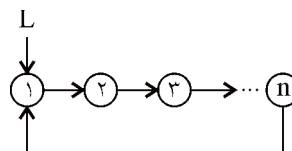
$L \rightarrow next = L \rightarrow next \rightarrow next$

$L = L \rightarrow next$;

}

return $L \rightarrow data$;

}



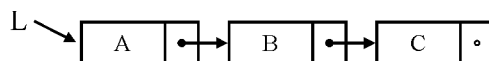
هیچ کدام (۴)

۷۲۹ و ۲۲۰۰ (۳)

۱ و ۱ (۲)

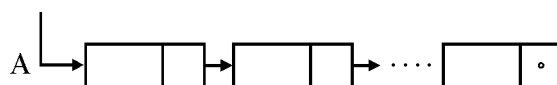
۴۰ و ۱ (۱)

۳۶- خروجی رویه برگشتی *WHAT* برای لیست پیوندی یکطرفه زیر چیست؟ (آ(اد ۸۰)



<i>Procedure WHAT(L)</i>	<i>CCBCCBACCBCCBA</i> (۱)
<i>begin</i>	<i>CBCCBCACBCCBCA</i> (۲)
<i>if L ≠ ∅ then begin</i>	<i>ACBBACBBACBBA</i> (۳)
<i> WHAT(Link(L));</i>	<i>CBCBCCACBCBCCA</i> (۴)
<i> print (Data(L));</i>	
<i> WHAT(Link(L));</i>	
<i> print (Data(L));</i>	
<i>end</i>	
<i>end;</i>	

۳۷- فرض کنید بخواهیم ترتیب اشاره‌گرها را در لیست تک پیوندی *A* وارون نمائیم. (۸۳IT)



به عبارت دیگر، هر اشاره‌گر به جای اشاره به عنصر بعدی، به عنصر قبلی اشاره نماید. برای انجام این کار، کدام یک از گزاره‌های ذیل درست است. (فرض کنید الگوریتم غیربازگشتی باشد).

- (۱) اشاره‌گر به جز نام لیست مورد نیاز است.
- (۲) اشاره‌گر به جز نام لیست مورد نیاز است.
- (۳) اشاره‌گر دیگری غیر از *A*، مورد نیاز است.
- (۴) کافی است از یک استک استفاده کنیم، لذا اشاره‌گر بالای استک و نام لیست، مورد نیاز است.

۳۸- یک پشته خالی با اعداد ۱ تا ۶ در ورودی داده شده است. اعمال زیر بر روی پشته قابل انجام هستند.

PUSH: کوچکترین عدد ورودی را برداشته و وارد پشته می‌کنیم.

POP: عنصر بالای پشته را در خروجی نوشته و سپس آن را حذف می‌کنیم.

کدامیک از گزینه‌های زیر را نمی‌توان با هیچ ترتیبی از اعمال فوق بدست آورد؟ (اعداد از

چپ به راست) (دولتی ۷۴- مشابه ارشد آزاد ۷۱ و ۷۲)

- | | |
|-----------------|-----------------|
| (۱) ۱ ۲ ۳ ۵ ۶ ۴ | (۲) ۳ ۲ ۴ ۶ ۵ ۱ |
| (۳) ۴ ۳ ۲ ۱ ۶ ۵ | (۴) ۲ ۱ ۵ ۳ ۴ ۶ |

۳۹- عبارت $\{x + (y - [a + b] * c - [(d + e)])\} / (j - (K - [L - n]))$ را در نظر بگیرید. می‌خواهیم با استفاده از یک پشته بررسی کنیم که آیا پرانتز، کروشه و آکولادها بدرستی تطبیق می‌شوند یا نه. پشته مورد استفاده حداقل بایستی گنجایش چند عنصر را داشته باشد؟

۴ (۱) ۹ (۲) ۱۸ (۳) ۲۷ (۴)

۴۰- سه پشته S_1 و S_2 و S_3 هر یک حاوی دو عدد به شکل زیر می‌باشند:

۲	۴	۶
۱	۳	۵

دو عملگر $pop(i)$ و $poppush(i, j)$ به صورت زیر تعریف شده‌اند. $Poppush(i, j)$ یک قلم از پشته S_i حذف و به پشته S_j اضافه می‌کند. $pop(i)$ یک قلم از پشته S_i حذف و سپس آن را چاپ می‌کند. برای چاپ اعداد ۱ تا ۶ به صورت ۱ و ۳ و ۵ و ۲ و ۴ و ۶ عملگر $poppush$ بایستی حداقل چند بار مورد استفاده قرار گیرد؟ اولین عددی که چاپ می‌شود عدد ۱، دومین عدد، عدد ۳ و ... می‌باشد. (آزاد ۸۰)

۳ (۱) ۵ (۲) ۶ (۳) ۴ (۴) بار

۴۱- کم هزینه‌ترین (از نظر تخصیص حافظه) راه برای اینکه ترتیب عناصر یک $Stack$ را برعکس کنیم کدام است؟

- (۱) از طریق ۲ پشته ($Stack$) اضافی
- (۲) از طریق یک صف اضافی ($queue$)
- (۳) از طریق یک پشته اضافی و چند متغیر
- (۴) هیچکدام

۴۲- کم هزینه‌ترین (از نظر تخصیص حافظه) راه برای اینکه عناصر یک پشته S_1 را به پشته

دیگر S_2 بدون اینکه ترتیب عناصر تغییر یابد، انتقال دهیم کدام است؟

- (۱) از طریق یک متغیر
- (۲) از طریق یک پشته اضافی
- (۳) از طریق دو پشته اضافی
- (۴) از طریق چند متغیر

۴۳- آنچه در زیر آمده است یک شبه کد اشتباه برای الگوریتمی است که باید متوازن بودن یا نبودن رشته‌ای از پرانتزها را تعیین کند:

(دولتی ۸۰ و ۸۲)

```
declare a character stack
while (more input is available)
{
    read a character
    if (the character is '(')
        then push it on the stack
    else if (the character is a ')' and the stack is not empty)
        then pop a character off the stack
    else print "unbalanced" and exit
}
print "balanced"
```

کدامیک از رشته‌های نامتوازن زیر توسط الگوریتم فوق، متوازن در نظر گرفته می‌شوند؟

(۱) (()) (۲) () () (۳) () () (۴) () () ()

۴۴- بر روی پشته S اعمال زیر تعریف شده‌اند:

$Push(S, x)$: درج x در بالای پشته با هزینه $O(1)$

$Pop(S, x)$: حذف عنصر بالای پشته با هزینه $O(1)$

$MultiPop(S, k)$: حذف k عنصر بالای پشته (با فرض آن که k حداکثر برابر تعداد

عناصر موجود S است) با هزینه $O(k)$.

اگر N عمل از اعمال فوق به ترتیب دلخواه بر روی پشته S که در ابتدا تهی است انجام

می‌شود، مجموع هزینه این N عمل در بدترین حالت چقدر است؟ (دولتی ۷۸)

(۱) $O(N \log N)$ (۲) $O(N^2)$ (۳) $O(NK)$ (۴) $O(N)$

۴۵- می‌خواهیم ساختار داده‌ای مشابه پشته برای حداکثر n عدد طراحی کنیم تا اعمال

$Push$, Pop , $FindMax$ و $FindMin$ را بتواند به صورت کارا انجام دهد. برای این

کار از دو آرایه $A[1..n]$ و $B[1..n]$ استفاده می‌کنیم که با انجام اعمال فوق مقادیرشان

عوض می‌شود. $A[i]$ عنصر i ام پشته است و $B[i]$ اندیس کوچکترین عنصر $A[1]$ تا $A[i]$

با استفاده از این آرایه‌ها، اعمال فوق را با چه هزینه‌ای می‌توان انجام داد؟ (سراسری ۸۳)

(۱) همهٔ اعمال $O(1)$ (۲) همهٔ اعمال $O(\lg n)$

(۳) همهٔ اعمال به جز $FindMax$ $O(1)$ (۴) همهٔ اعمال به جز Pop و $FindMax$ $O(1)$

۴۶- با استفاده از یک پشته (*stack*) و اعمال *push* و *pop* و ترتیب ورودی زیر (به ترتیب از چپ به راست) A, B, C, D, E, F کدام یک از ترتیب‌های زیر در پشته ممکن است؟ (رشد پشته را از چپ به راست فرض کنید)

(۸۵IT)

(۱) BDF (۲) BEC (۳) DBF (۴) $EDCBA$

۴۷- در عبارت محاسباتی $9 + ((8 * (7 + 6)) * 5 + 4 * (3 + 2))$ عملگر + بر * اولویت دارد. این عبارت معادل عبارت پیشوندی زیر است؟

(۸۵IT)

(۱) $9 + 5 * 234 + 4 * 6789 + 23 + 45$ (۲) $9 + 5 * 234 + 4 * 6789 + 23 + 45$
(۳) $9 + 5 * 234 + 4 * 6789 + 23 + 45$ (۴) $9 + 5 * 234 + 4 * 6789 + 23 + 45$

۴۸- معادل *Infix* عبارت *Prefix* روبه‌رو چیست؟

(۸۶IT)

*Prefix: / * A - B / * C - + D E F G - H I*

(۱) $A * B - C * D + E - F / G / H - I$ (۲) $((D + E - F) * C / G - B) * A / H - I$

(۳) $A * (B - C * (D + E - F) / G) / (H - I)$ (۴) $A * (B / (((C * (D + E)) - F) - G)) / (H - I)$

۴۹- عبارت *infix* زیر را به *Polish* وارونه تبدیل کنید.

(آزاد ۷۸)

$A * (B - D) / E - F * (G + H / K)$

(۱) $ABD - * E / FG HK / + * -$ (۲) $A + B * DEF / GH K / - * -$
(۳) $- * + / KH G / FED * B + A$ (۴) $AB * + DEF / G + HK / * -$

۵۰- حاصل *Postfix* عبارت روبه‌رو چیست؟

(مشابه دولتی ۷۷)

$6 + 3 \uparrow 2 * 3 + 2 / 8 \wedge 3 - 3 + 2 \wedge 6$

(۱) ۷ (۲) ۲۵ (۳) ۴۹ (۴) ۵۲

۵۱- مینیمم تعداد متغیرهای میانی در محاسبه عبارت جبری $ab + cd * a +$ که به صورت

(علوم کامپیوتر ۷۹)

Postfix است. برابر است با

(۱) ۱ (۲) ۲ (۳) ۳ (۴) ۴

۵۲- تعداد عناصر (M) در یک صف حلقوی از کدام فرمول بدست می‌آید؟ متغیر F به خانه‌ای که بلافاصله قبل از اول صف قرار دارد، اشاره می‌کند و متغیر R به انتهای صف اشاره می‌کند؟

(دولتی IT ۸۳)

(۱) $M = R - F$

(۲) $M = n - (R - F)$

(۳) $M = \begin{cases} n - (F - R) & \text{if } F > R \\ R - F & \text{if } F < R \end{cases}$

(۴) $M = \begin{cases} n - (R - F) & \text{if } F > R \\ R - F & \text{if } F < R \end{cases}$

۵۳- با توجه به الگوریتم‌های زیر در مورد صف‌ها کدام گزینه رابطه درستی است و دارای خطا نمی‌باشد؟

- صف خالی Q را ایجاد می‌کند. $CREATE(Q)$: الگوریتم ۱
 عنصر i را به صف Q اضافه می‌کند و صف را برمی‌گرداند. $ADD(i, Q)$: الگوریتم ۲
 عنصری را از صف Q حذف می‌کند و صف را برمی‌گرداند. $DELETE(Q)$: الگوریتم ۳
 عنصر ابتدایی از صف Q را می‌دهد. $FRONT(Q)$: الگوریتم ۴
 اگر صف Q تهی باشد مقدار $True$ و اگر خالی نباشد مقدار $False$ برمی‌گرداند. $ISEMPTY(Q)$: الگوریتم ۵

(۱) $DELETE(CREATE(Q))$

(۲) $FRONT(CREATE(Q))$

(۳) $ISEMPTY(Q) THEN FRONT(Q)$

(۴) $IF NOT ISEMPTY(ADD(i, Q) THEN FRONT(Q)$

۵۴- عناصر صف‌های Q_1 و Q_2 از چپ به راست به صورت زیر است (عنصر سمت چپ ابتدای صف است) اگر x و y عناصر صف باشند، پس از اجرای تکه برنامه زیر:

$$Q_1 = 10, 25, 17, 41, 19, 26, 75$$

$$Q_2 = 1, 5, 7, 4, 9, 6$$

$$Makenull(Q_3)$$

$$i = 0$$

$$while(not\ empty(Q_1)\ and\ not\ empty(Q_2))$$

$$i = i + 1$$

$$x = DeleteQ(Q_1)$$

$$y = DeleteQ(Q_2)$$

$$if\ (y = i)\ then\ AddQ(Q_3, x)$$

end while

(دولتی ۷۹)

محتوی صف Q_3 برابر است با:

$$Q_3 = 10, 25, 17 \quad (2)$$

$$Q_3 = 1, 4, 6 \quad (1)$$

$$Q_3 = 1, 5, 7 \quad (4)$$

$$Q_3 = 10, 41, 26 \quad (3)$$

۵۵- برای حذف یک عنصر از اول یک صف حلقوی که عناصر آن درون یک آرایه بنام *queue* از نوع *ELEMENT* ذخیره شده، برنامه زیر ارائه شده است ولی یک سطر از برنامه حذف شده است. کدام گزینه باید به جای سطر حذف شده قرار گیرد تا برنامه کامل شود؟ (*MAXQSIZE* تعداد خانه‌های آرایه *queue* است.) (دو نمره)

*ELEMENT delete (int*front , int rear)*

```
{
    if (*front == rear)
        return empty-q();
    else
        ???
    return queue[*front];
}
```

**front = *fornt - ۱* (۱)

**front = *front + ۱* (۲)

**front = *(front + ۱)* (۳)

**front = (*front + ۱) % MAXQSIZE* (۴)

پاسخنامه سؤال‌های طبقه‌بندی شده فصل سوم

(۱) گزینه (۱).
$$j + m * (i - 1) + 1 = [(i - 1) * m + (j - 1)] + 1 = \text{جواب}$$

(۲) گزینه (۲). اگر آدرس شروع B در حافظه صفر باشد:

$$+ (k - 1) + (J - 1) \times P + (I - 1) \times n \times P$$

(۳) گزینه (۲).
$$1932 = 4 + 0 + [(3 - 0) + (2 - 0) + (4 - 0)(2 - 0) + (1 - 0)(2 - 0)]$$

(۴) گزینه (۲).

$$23280 = 4 + 10000 + [20 + 30 \times 10 + 20 \times 30 + (5 - 0)]$$

(۵) گزینه (۲). با فرض این که اندیس شروع آرایه‌ها ۱ باشد:

$$20000 + 4 + [(11 - 1) \times 30 + (11 - 1) \times 30 + (9 - 1) \times 20 \times 30]$$

$$40480 = 20000 + 4 + [20 + 300 + 4800]$$

(۶) گزینه (۱).
$$A_{3 \times 5} B_{5 \times 89} C_{89 \times 3} D_{3 \times 34}$$

باید ضرب را طوری انجام داد که اعداد خیلی بزرگ فقط یک بار در عمل ضرب شرکت کنند.

مثلاً بهتر است ابتدا $B \times C$ محاسبه شود زیرا عدد ۸۹ یک بار در ضرب شرکت می‌کند:

$$A_{3 \times 5} (B C)_{89 \times 3} D_{3 \times 34}$$

پس از آن بهتر است ماتریس D آخر ضرب شود تا عدد ۳۴ نیز یک بار در ضرب شرکت کند:

$$(A (B C)) D$$

(لازم به ذکر است که این قانون، کلی نیست)

(۷) گزینه (۳). مشابه تست قبل

(۸) گزینه (۱). الگوریتم در متن درس آمده است.

(۹) گزینه (۴). گزینه ۳ مرتبه الگوریتم معمولی و گزینه ۴ مرتبه الگوریتم سریع است.

(۱۰) گزینه (۲). مرتبه الگوریتم معمولی در گزینه‌ها آمده است.

(۱۱) گزینه (۳). با توجه به توضیحات مسئله ماتریس به صورت مقابل است.

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 0 & a_{22} & a_{23} & a_{24} & 0 \\ 0 & a_{32} & a_{33} & a_{34} & 0 \\ 0 & a_{42} & a_{43} & a_{44} & 0 \\ 0 & a_{52} & a_{53} & a_{54} & 0 \end{bmatrix}$$

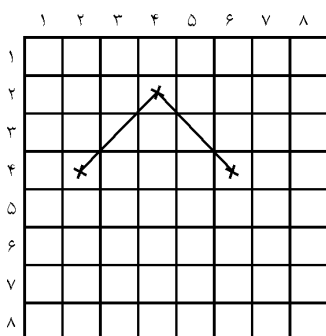
$$a_{33} = \frac{1}{2}(a_{22} + a_{24}) = \frac{1}{2}(1 + 1) = 1$$

$$a_{22} = \frac{1}{2}(a_{11} + a_{13}) = \frac{1}{2}(1 + 1) = 1$$

$$a_{24} = \frac{1}{2}(a_{13} + a_{15}) = \frac{1}{2}(1 + 1) = 1$$

$$a_{32} = \frac{1}{2}(a_{21} + a_{23}) = \frac{1}{2}(0 + 1) = \frac{1}{2}$$

$$a_{23} = \frac{1}{2}(a_{12} + a_{14}) = 1$$



(۱۲) گزینه (۲). چند حالت را با شکل بررسی می‌کنیم:

$$(i, j) \leftrightarrow (k, L)$$

$$(2, 4) \leftrightarrow (4, 6)$$

$$(2, 4) \leftrightarrow (4, 2)$$

به طور کلی شرط هم قطر بودن

$$|i - k| = |j - L| \text{ می‌باشد.}$$

(۱۳) گزینه (۳). در جستجوی ناموفق تعداد مقایسه‌ها برابر $\left\lfloor \log_2^n \right\rfloor + 1$ یا $\left\lceil \log_2^n \right\rceil$ است. پس

همواره برابر نیست.

(۱۴) گزینه (۲).

$$\left\lfloor \log_2^{200} \right\rfloor + 1 = 8$$

(۱۵) گزینه (۱).

$$4 < 512 \Leftarrow mid = \left\lfloor \frac{1 + 1024}{2} \right\rfloor = 512$$

$$4 < 256 \Leftarrow mid = \left\lfloor \frac{1 + 511}{2} \right\rfloor = 256$$

$$4 < 128 \Leftarrow mid = \left\lfloor \frac{1 + 255}{2} \right\rfloor = 128$$

$$4 < 64 \Leftarrow mid = \left\lfloor \frac{1 + 127}{2} \right\rfloor = 64$$

$$\text{پنجمین مقایسه: } 4 < 32 \Leftarrow mid = \left\lfloor \frac{1+63}{2} \right\rfloor = 32$$

$$\text{ششمین مقایسه: } 4 < 16 \Leftarrow mid = \left\lfloor \frac{1+31}{2} \right\rfloor = 16$$

$$\text{هفتمین مقایسه: } 4 < 8 \Leftarrow mid = \left\lfloor \frac{1+15}{2} \right\rfloor = 8$$

$$\text{هشتمین مقایسه: } 4 = 4 \Leftarrow mid = \left\lfloor \frac{1+7}{2} \right\rfloor = 4$$

(۱۶) گزینه (۲).

-۱	۰	۱	۲	۳	۴	۵	۶	۷
----	---	---	---	---	---	---	---	---

$$\text{مقایسه} = \begin{array}{cccccccc} 3 & 2 & 3 & 4 & 1 & 3 & 2 & 3 & 4 \end{array}$$

تعداد مقایسه‌ها برای هر یک از عناصر را محاسبه می‌کنیم. مثلاً عنصر وسط نیاز به یک مقایسه دارد و سایر عناصر نیز هر یک طبق الگوریتم جستجوی دودویی محاسبه می‌شوند. در نتیجه:

$$\text{متوسط مقایسه‌ها} = \frac{3+2+3+4+1+3+2+3+4}{9} = \frac{25}{9}$$

(۱۷) گزینه (۳).

-۶	۰	۲	۳	۷	۹	۵۴	۸۲	۱۰۱	۱۲۰
----	---	---	---	---	---	----	----	-----	-----

$$\text{تعداد مقایسه‌ها} \begin{array}{cccccccc} 3 & 2 & 3 & 4 & 1 & 3 & 4 & 2 & 3 & 4 \end{array}$$

$$\text{متوسط مقایسه‌ها} = \frac{3+2+3+4+1+3+4+2+3+4}{10} = \frac{29}{10}$$

(۱۸) گزینه (۱). ابتدا آرایه A را با زمان $n.lgn$ مرتب می‌کنیم (در فصل مرتب‌سازی خواهیم دید که روش‌های مقایسه‌ای برای مرتب‌سازی نمی‌توانند از $n.lgn$ سریع‌تر شوند) سپس برای هر عنصر $j = 1, 2, \dots, n$ ، مقدار $M - B[j]$ را در A جستجوی دودویی می‌کنیم که این نیز $n.lgn$ زمان می‌خواهد.

(۱۹) گزینه (۳). r به وسط آرایه اشاره داده می‌شود. دقت کنید گزینه ۱ در زبان C خطاست چون اشاره‌گرها را نمی‌توان جمع کرد، ولی تفریق می‌توان کرد.

(۲۰) گزینه (۲). برای بررسی درستی سایر گزینه‌ها به کتاب طراحی الگوریتم پوران پژوهش رجوع کنید.

(۲۱) گزینه (۱). با بررسی می‌توان نشان داد در گزینه ۱

$$T[o][o] = o$$

$$T[o][1] = 1..T[o][7] = 1...T[1][o] = 1$$

(۲۲) گزینه (۱). به صورت $f('ALI', 3)$ فراخوانی کنید نتیجه خود ALI است.

(۲۳) گزینه (۳). زیر رشته‌ها عبارتند از:

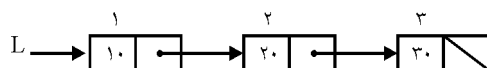
$$\left\{ \begin{array}{l} \text{تهی} \\ A, B, C, D \\ AB, BC, CD \\ ABC, BCD \\ ABCD \end{array} \right.$$

(۲۴) گزینه (۲). برای حذف آخرین عنصر باید لیست پیمایش شود تا به گره ماقبل آخر برسیم و $link$ آن را تهی کنیم. در بقیه گزینه‌ها طول لیست اهمیت ندارد.

(۲۵) گزینه (۳). زیر برنامه را روی یک لیست با دو گره آزمایش کنید. (این رویه را حفظ کنید و خاطرتان باشد برای معکوس کردن لیست به صورت غیربازگشتی ۳ اشاره‌گر کمکی باید تعریف کرد)

(۲۶) گزینه (۲). مشابه تست قبل

(۲۷) گزینه (۲).



$what(1) := Head(1) + what(3) = 10 + 30 = 40$: بار اول

$$what(3) = Head(3) = 30$$

(۲۸) گزینه (۱). ابتدا p به المان اول اشاره می‌کند و حلقه $while$ داخلی، n بار تکرار می‌شود. سپس p به المان دوم اشاره می‌کند و حلقه، $n-1$ بار تکرار می‌شود. به همین ترتیب:

$$writeln \text{ تکرار} = n + (n-1) + \dots + 1 = \frac{n(n+1)}{2}$$

(۲۹) گزینه (۴). این تابع همه اشاره‌گرهای $next$ را برابر nil می‌گذارد. اگر جمله

$L \uparrow next \uparrow next := nil$ به $L \uparrow next \uparrow next := L$ تبدیل شود، لیست معکوس می‌شود.

۳۰) گزینه (۳). در لیست حلقوی اگر بجای ذخیره آدرس اولین گره، آدرس آخرین گره را ذخیره کنیم، افزودن عنصر به آخر یا اول لیست، نیازی به پیمایش ندارد و با $O(1)$ انجام می‌شود.

۳۱) گزینه (۲). رویه به صورت بازگشتی، لیست عمومی را پیمایش می‌کند.

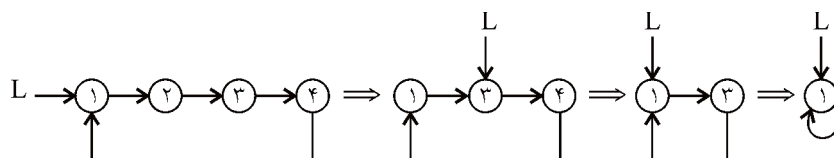
۳۲) گزینه (۳). حلقه $while$ سوم هیچ تاثیری در فراخوانی $first$ ندارد. حلقه $while$ دوم بار اول $n-1$ بار، بار دوم $n-2$ بار و ... اجرا می‌شود: $1 + 2 + 3 + \dots + (n-1) = \frac{n(n-1)}{2}$

چون تابع $first$ یک بار در ابتدای برنامه زیر فراخوانی می‌شود: $1 + \frac{n(n-1)}{2}$

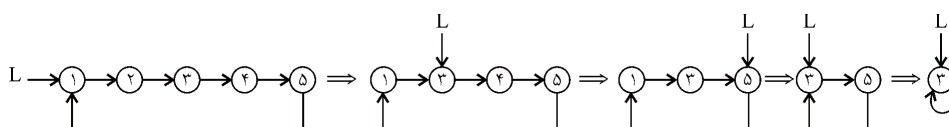
۳۳) گزینه (۳). اولین بار گره‌های با شماره فرد از لیست حذف می‌شوند. بار دوم گره‌های شماره ۲ و ۶ و ۱۰ و ... و ۲۰۴۶ در مرحله بعد گره‌های ۴ و ۱۲ و ۲۰ و ... و ۲۰۴۴ حذف می‌شوند. می‌توان نشان داد در نهایت ۲۰۴۸ می‌ماند.

۳۴) گزینه (۳). عدد ۱۰۰۰ را باینری بنویسید و چرخش به چپ دهید! (مسئله جوزف)

۳۵) گزینه (۴). این مسئله همان مسئله معروف جوزف است برای اطمینان به ازای $n = 4$ بررسی می‌کنیم:



به ازای $n = 5$ بررسی می‌کنیم:



مسئله معروف جوزف به این صورت است که اعداد ۱ تا n را در جهت عقربه ساعت روی محیط یک دایره می‌نویسیم و از شماره ۲ یک در میان حذف می‌کنیم تا در نهایت یک شماره بماند. این تست دقیقاً مسئله جوزف است. در مسئله جوزف اگر عدد n را باینری بنویسید و آن را چرخش به سمت چپ دهید، شماره باقی‌مانده بدست می‌آید! اثبات این مطلب از حوصله این نوشتار خارج است، پس:

$$729 = 512 + 128 + 64 + 16 + 8 + 1 = (1011011001)_2$$

$$\xrightarrow[\text{left}]{\text{rotate}} (011011001)_2 = 435 \text{ شمارنده باقی مانده}$$

$$2200 = 2048 + 128 + 16 + 8 = (100010011000)_2$$

$$\xrightarrow[\text{left}]{\text{rotate}} (000100110001)_2 = 305$$

(۳۶) گزینه (۱). تابع $what$ را با w نشان می‌دهیم:

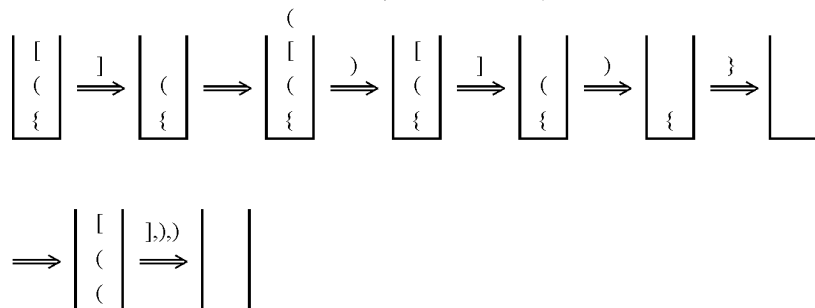
$$W(A) \rightarrow W(B) \rightarrow W(C) \rightarrow W(\circ)$$

بنابراین اولین مقداری که چاپ می‌شود c می‌باشد. سپس مجدداً فراخوانی بازگشتی با صفر انجام می‌شود و دستور $print$ دوم مجدداً c را چاپ می‌کند. یعنی فقط گزینه ۱ می‌تواند درست باشد.

(۳۷) گزینه (۱). برای معکوس کردن لیست، ۳ اشاره گر کمکی لازم است.

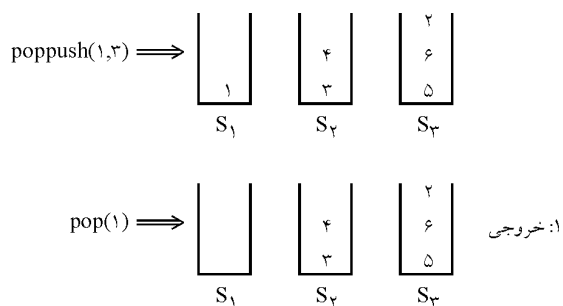
(۳۸) گزینه (۴). در گزینه ۴، سه عدد ۵۳۴ وجود دارند که $3 < 4 < 5$.

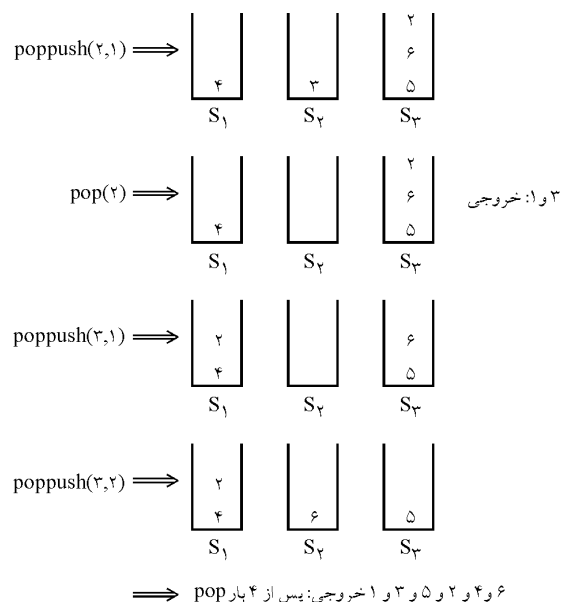
(۳۹) گزینه (۱). باید پرانتز، کروشه و آکولادهای باز را به پشته $push$ کنیم و با رسیدن به یک مورد بسته، نمونه باز آن را از پشته حذف کنیم.



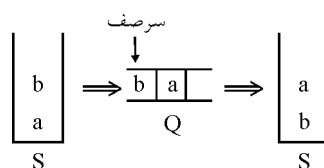
در نتیجه حداقل اندازه پشته باید ۴ باشد.

(۴۰) گزینه (۴). عملیات به صورت زیر است.



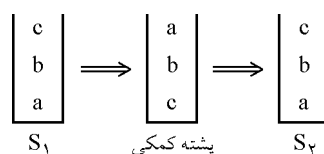


(۴۱) گزینه (۲).



البته با گزینه ۱ نیز می‌توان ولی گزینه ۲ بهتر است.

(۴۲) گزینه (۲).



(۴۳) گزینه (۱). در گزینه ۱ ابتدا سه پرانتز باز $push$ می‌شوند و سپس به خاطر دو پرانتز بسته، دو پرانتز باز pop می‌شوند و حلقه تمام می‌شود و $balanced$ چاپ می‌شود.

(۴۴) گزینه (۴). چون پشته در ابتدا تهی است پس نمی‌توان ابتدا عمل $Multipop$ انجام داد و باید $push$ انجام دهیم. چون باید N بار عملیات را انجام دهیم، مجبوریم مثلاً $\frac{N}{3}$ بار $push$ کنیم و سپس $\frac{N}{3}$ بار pop و یا هر حالت دیگری. به هر حال هر ترتیبی در نظر بگیریم هزینه $O(N)$ خواهد بود.

گزینه (۳). در چنین ساختاری در آرایه A زمان $push$ و pop مانند هر پشته معمولی از مرتبه $O(1)$ خواهد بود. اما برای افزودن اندیس مناسب به آرایه B کفایت هر عنصر $push$ شده را با اندیس $B[i-1]$ مقایسه کنیم. اگر محتویات اندیس مورد بحث از ورودی کوچکتر باشد پس کفایت همان اندیس برای عنصر جدید در آرایه B ذخیره گردد. در غیر این صورت اندیس عنصر جاری در آرایه ذخیره خواهد شد. حذف عنصر یا pop نیز تفاوتی با پشته معمولی ندارد و از زمان $O(1)$ خواهد بود. بدیهی است که با کمک آرایه B یافتن کوچکترین عنصر در زمان ثابت انجام می‌شود. اما برای یافتن بزرگترین عنصر حداکثر مرتبه زمانی غیر از $O(1)$ لازم است.

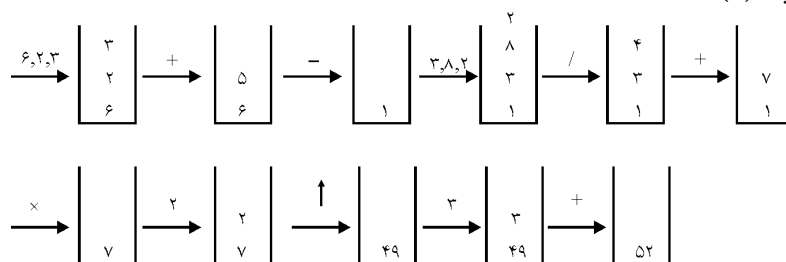
گزینه (۱). اگر ترتیب زیر را از چپ به راست انجام دهیم، گزینه ۱ در پشته مشاهده خواهد شد.
 $Push - pop - push - push - pop - push - push - pop - push$

گزینه (۲). بخاطر $4+5$ حتماً باید $4+5$ دیده شود (گزینه ۲ یا ۴). آخرین عمل + آخرین اولویت است که باید به ابتدای عبارت منتقل شود (گزینه ۲)

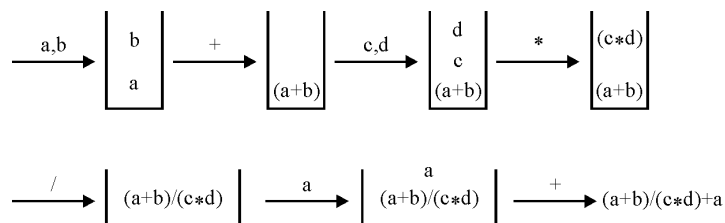
گزینه (۳). ترتیب عملوندها نباید عوض شود پس گزینه ۲ صحیح نیست. عبارت $+DE$ تبدیل به $D+E$ می‌شود پس گزینه ۱ صحیح نیست. عبارت DEF تبدیل به $(D+E)$ می‌شود که گزینه ۳ صحیح است.

گزینه (۱). $Polish$ وارونه یعنی پسوندی، با توجه به اولویت ابتدا BD بدست می‌آید که فقط گزینه ۱ می‌تواند صحیح باشد.

گزینه (۴). (۵۰)



گزینه (۳). حداقل اندازه پشته باید برابر ۳ باشد. (۵۱)



(۵۲) گزینه (۳). اگر $F < R$ آنگاه $R - F$ تعداد عناصر پر موجود در صف است و اگر $F > R$ آنگاه $F - R$ تعداد عناصر خالی است، پس $n - (F - R)$ تعداد عناصر پر موجود در صف است.

(۵۳) گزینه (۴).

گزینه ۱ درست نیست چون صف خالی عنصری ندارد که حذف شود.

گزینه ۲ درست نیست چون صف خالی عنصری ندارد، بنابراین عنصر اول آن بی‌معنی است.

گزینه ۳ درست نیست چون اگر صف تهی باشد، عنصر اول ندارد.

(۵۴) گزینه (۳).

i	۱	۲	۳	۴	۵	۶
x	۱۰	۲۵	۱۷	۴۱	۱۹	۲۶
y	۱	۵	۷	۴	۹	۶
$y=i$	بله	خیر	خیر	بله	خیر	بله

$\Rightarrow Q_3 = ۱۰, ۴۱, ۲۶$

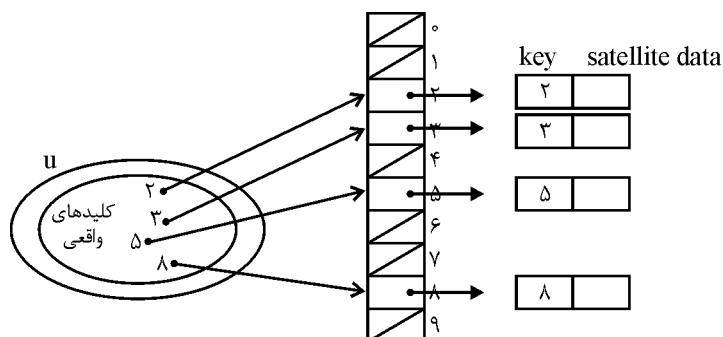
(۵۵) گزینه (۴). در الگوریتم حذف دیدیم که $front = (front + ۱) \bmod n$ که معادله گزینه ۴ است.

فصل ۴

جداول درهم‌سازی (Hash Tables)

جداول با آدرس‌دهی مستقیم

مجموعه دینامیکی که عملیات درج، حذف و جستجو را پشتیبانی می‌کند، دیکشنری گویند. جدول درهم‌سازی ساختمان داده مناسبی برای پیاده‌سازی دیکشنری‌هاست. آدرس‌دهی مستقیم ساده‌ترین تکنیک درهم‌سازی است و وقتی مناسب است که تعداد کلیدها (عناصر) کم باشد. در روش مستقیم فرض کنید کلیدها از مجموعه $u = \{0, 1, \dots, m-1\}$ هستند و m خیلی بزرگ نیست و یک آرایه یا جدول آدرس مستقیم $T[0..m-1]$ داریم که هر مکان این آرایه ($slot$) یا حفره گویند) متناظر با یک کلید است. مثلاً عنصر با کلید ۲ در حفره $T[2]$ قرار می‌گیرد یا عنصر با کلید ۸ در حفره $T[8]$ قرار دارد. حفره‌ها می‌توانند اشاره‌گر باشند. مثلاً فرض کنید عالم کلیدها $u = \{0, 1, \dots, 9\}$ است و کلیدهای واقعی $\{2, 3, 5, 8\}$ هستند شکل زیر نحوه آدرس‌دهی مستقیم را نشان می‌دهد.

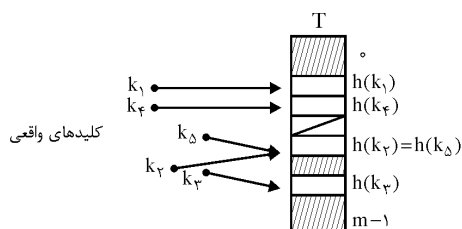


در این شکل key کلید است که براساس آن ذخیره و جستجو انجام می‌شود. $Satellite\ data$ سایر فیلدهای هر عنصر است. مثلاً شماره دانشجویی کلید است و نام و نام‌خانوادگی و ... $satellite\ data$ هستند.

البته هر عنصر می‌تواند مستقیم در حفره مربوطه‌اش ذخیره شود و لزومی ندارد که حتماً حفره‌ها اشاره‌گر باشند. مثلاً عنصر با کلید i در حفره $T[i]$ ذخیره شود. همچنین می‌توان فقط $satellite\ data$ را ذخیره کرد زیرا اندیس، خودش کلید است. واضح است که در روش آدرس‌دهی مستقیم، حذف، درج و جستجو همواره با مرتبه $\theta(1)$ انجام می‌شود.

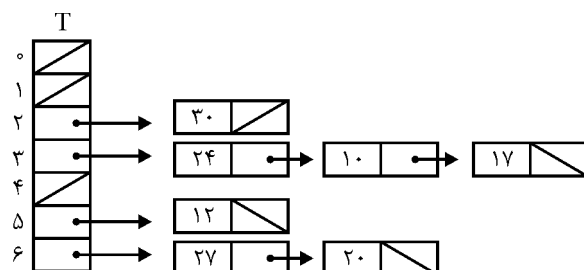
درهم‌سازی

اگر مرجع کلیدها (U) بزرگ باشد، آدرس‌دهی مستقیم قابل استفاده نیست؛ و باید از یک تابع درهم‌ساز استفاده کنیم. یعنی کلید k را به تابع درهم‌ساز h می‌دهیم و این تابع یک آدرس از آرایه T را محاسبه می‌کند و عنصر در $T[h(k)]$ ذخیره می‌شود. یعنی $h: U \rightarrow \{0, 1, \dots, m-1\}$. یکی از مشکلات، تصادف ($Collision$) است یعنی دو کلید متفاوت را به تابع h داده‌ایم ولی تابع h ، یک آدرس محاسبه کرده است. در این شکل $k_p \neq k_\Delta$ ولی $h(k_p) = h(k_\Delta)$ که تصادف پیش آمده است.



یکی از روش‌های حل تصادف، زنجیره‌سازی ($Chaining$) است. (روش دیگر آدرس‌دهی باز است). در زنجیره‌سازی، حفره‌ها اشاره‌گر هستند. $T[j]$ به یک لیست پیوندی اشاره می‌کند و هر کلیدی که توسط تابع h به j نگاشته شود، در لیست $T[j]$ قرار می‌گیرد. مثلاً فرض کنید کلیدها به ترتیب ۱۷ و ۲۰ و ۱۲ و ۱۰ و ۳۰ و ۲۴ و ۲۷ است و تابع درهم‌ساز $h(k) = k \bmod 7$ باشد و $T[0..6]$ است آنگاه:

کلید k	۱۷	۲۰	۱۲	۱۰	۳۰	۲۴	۲۷
$h(k)$ آدرس	۳	۶	۵	۳	۲	۳	۶



دقت کنید عنصر تازه وارد را در ابتدای لیست درج می‌کنیم که در این صورت مرتبه عمل درج $\theta(1)$ می‌شود مثلاً $17 \bmod 7 = 3$ پس ۱۷ در لیست $T[3]$ قرار می‌گیرد. عنصر ۱۰ نیز در لیست $T[3]$ قرار می‌گیرد ولی قبل از ۱۷ درج می‌شود و در نهایت ۲۴ نیز در لیست $T[3]$ و اول لیست درج می‌شود. الگوریتم‌های درج، جستجو و حذف در روش زنجیره‌سازی عبارتند از:

Insert (T, x)

Insert x at the head of list $T[h(x.\text{key})]$

Search (T, k)

search for an element with key k in list $T[h(k)]$

Delete (T, x)

delete x from the list $T[h(x.\text{key})]$

در الگوریتم درج فرض شده عنصر x قبلاً وجود نداشته است. در الگوریتم حذف، عنصر x (و نه کلید) به عنوان پارامتر ورودی دریافت می‌شود و اگر لیست‌ها دو طرفه باشند، با مرتبه $\theta(1)$ می‌توان عنصر x را حذف کرد (دقت کنید نیاز به جستجو نیست چون مکان عنصر x مشخص است). در الگوریتم جستجوی کلید k ، لیست $T[h(k)]$ بررسی می‌شود. این الگوریتم در بدترین حالت از مرتبه $\theta(n)$ است (n تعداد عناصر است و m تعداد حفره‌ها) زیرا در بدترین حالت، همه n عنصر در لیست $T[h(k)]$ درج شده‌اند ولی احتمال وقوع بدترین حالت بسیار ضعیف است. اگر فرض کنیم زمان تابع و محاسبه $h(k)$ از مرتبه $O(1)$ باشد و $\alpha = \frac{n}{m}$ متوسط تعداد عناصر در هر لیست باشد (α را فاکتور لود گویند)، آنگاه زمان اجرای جستجو در حالت متوسط برای جستجوی ناموفق $\theta(1 + \alpha)$ و برای جستجوی موفق $\theta(1 + \alpha) = \theta\left(1 + 1 + \frac{\alpha}{2} - \frac{\alpha}{2n}\right)$ است (اثبات نمی‌کنیم).

آدرس‌دهی باز (open addressing)

در این روش عناصر داخل خود جدول درهم‌ساز ذخیره می‌شوند: یعنی حفره‌ها شامل عناصر هستند یا تهی (NIL) می‌باشند. بنابراین عنصر با کلید k در حفره $h(k)$ ذخیره می‌شود و اگر این حفره اشغال باشد باید حفره دیگری پیدا کنیم که خالی (NIL) باشد. بنابراین باید حفره‌ها را کاوش ($probe$) کنیم تا به حفره خالی برسیم. برای آنکه مشخص شود کدام حفره‌ها واری می‌شوند، به تابع h ، شماره واری که از o شروع می‌شود را به عنوان پارامتر دوم اضافه می‌کنیم، بنابراین برای کلید k دنباله واری‌ها عبارتند از $\langle h(k, 0), h(k, 1), \dots, h(k, m-1) \rangle$ که جایگشتی از $\langle 0, 1, \dots, m-1 \rangle$ است، یعنی در بدترین حالت همه حفره‌ها واری می‌شوند.

Insert (T, k)

```
{
     $i = o$ 
    repeat
         $j = h(k, i)$ 
        if  $T[j] = NIL$ 
             $T[j] = k$ , return  $j$ 
        else  $i = i + 1$ 
    until  $i = m$ 
    error "hash table overflow"
}
```

Search (T, k)

```
{
     $i = o$ 
    repeat
         $j = h(k, i)$ 
        if  $T[j] = k$ 
            return  $j$ 
         $i = i + 1$ 
    until  $T[j] = NIL$  or  $i = m$ 
    return  $NIL$ 
}
```

لازم به ذکر است که آدرس‌دهی باز با حذف عناصر مشکل دارد. هنگام حذف یک عنصر، نباید حفره آن را NIL اعلام کنیم بلکه باید علامتی مثل *deleted* در آن حفره قرار دهیم و الگوریتم *Insert* را تغییر دهیم که حفره‌های *deleted* را نیز به عنوان حفره خالی ببیند. ولی لازم نیست الگوریتم *search* را تغییر دهیم زیرا خانه‌های *deleted* برای *search* پر در نظر گرفته می‌شوند. در آدرس‌دهی باز با ۳ روش می‌توان واری حفره‌ها را انجام داد: واری خطی، مربعی و دابل

کاوش خطی (linear probing)

فرض کنید هنگام درج، کلید k را به تابع h' داده‌ایم ولی $T[h'(k)]$ پر است، این روش $T[h'(k) + 1]$ را واری می‌کند، اگر پر بود $T[h'(k) + 2]$ و به همین ترتیب $T[m-1]$ واری

می‌شود که در صورت پر بودن $T[0]$ و $T[1]$ و $T[h'(k)-1] \dots T[h'(k)]$ واری می‌شوند. یعنی تابع درهم ساز این روش $h(k, i) = (h'(k) + i) \bmod m$ است. این روش پیادسازی ساده‌ای دارد ولی مشکل *Primary clustering* دارد. یعنی حفره‌های پشت سرهم پر می‌شوند که زمان جستجو افزایش می‌یابد. کلاسترها به این خاطر ایجاد می‌شوند که یک حفره خالی بعد از i حفره پر با احتمال $\frac{i+1}{m}$ پر می‌شود.

کاوش مربعی (quadratic probing)

تابع درهم سازی در این روش $h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$ است که c_1 و c_2 مقادیر ثابت هستند و $i = 0, 1, 2, \dots, m-1$. اولین حفره $T[h'(k)]$ بررسی می‌شود و سایر حفره‌های مورد بررسی به c_1 و c_2 بستگی دارند. c_1 و c_2 باید طوری باشند که همه حفره‌ها واری می‌شوند. این روش نیز مشکل کلاسترینگ دارد ولی خفیف‌تر از قبلی است و به آن *secondary clustering* گویند.

مثال: کلیدهای ۱۰ و ۲۲ و ۳۱ و ۴ و ۱۵ و ۲۸ و ۱۷ و ۸۸ و ۵۹ در یک جدول درهم‌ساز با $m = ۱۱$ و آدرس‌دهی باز درج کنید. فرض کنید تابع درهم‌ساز جانبی $h'(k) = k$. یک بار با روش واری خطی درج کنید و یک بار با کاوش مربعی با $c_1 = ۱$ و $c_2 = ۳$.

پاسخ: کاوش خطی:

$$\begin{aligned} h(k, i) &= (h'(k) + i) \bmod ۱۱ \text{ و } h(۱۰, ۰) = (۱۰ + ۰) \bmod ۱۱ = ۱۰ \\ h(۲۲, ۰) &= (۲۲ + ۰) \bmod ۱۱ = ۰ \text{ و } h(۳۱, ۰) = (۳۱ + ۰) \bmod ۱۱ = ۹ \\ h(۴, ۰) &= ۴ \text{ و } h(۱۵, ۰) = ۴ \Rightarrow h(۱۵, ۱) = (۱۵ + ۱) \bmod ۱۱ = ۵ \\ h(۲۸, ۰) &= ۶ \text{ و } h(۱۷, ۰) = ۶ \Rightarrow h(۱۷, ۱) = ۷ \text{ و } h(۸۸, ۰) = ۰ \Rightarrow \text{تصادف} \\ h(۸۸, ۱) &= ۱ \text{ و } h(۵۹, ۰) = ۴ \Rightarrow \text{تصادف} \Rightarrow h(۵۹, ۱) = ۵ \text{ و تصادف} \Rightarrow h(۵۹, ۲) = ۶ \text{ و تصادف} \\ \Rightarrow h(۵۹, ۳) &= ۷ \Rightarrow \text{تصادف} \Rightarrow h(۵۹, ۴) = ۸ \end{aligned}$$

	۰	۱	۲	۳	۴	۵	۶	۷	۸	۹	۱۰
T	۲۲	۸۸			۴	۱۵	۲۸	۱۷	۵۹	۳۱	۱۰

کاوش مربعی:

$$\begin{aligned} h(k, i) &= (k + i + 3i^2) \bmod ۱۱ \\ h(۱۰, ۰) &= ۱۰ \text{ و } h(۲۲, ۰) = ۰ \text{ و } h(۳۱, ۰) = ۹ \text{ و } h(۴, ۰) = ۴ \end{aligned}$$

$$h(۱۵, ۰) = ۴ \text{ تصادف} \Rightarrow h(۱۵, ۱) = (۱۵ + ۱ + ۳) \bmod ۱۱ = ۸$$

$$h(۲۸, ۰) = ۶$$

$$h(۱۷, ۰) = ۶ \text{ تصادف} \Rightarrow h(۱۷, ۱) = ۱۰ \text{ تصادف} \Rightarrow h(۱۷, ۲) = ۹ \text{ تصادف} \Rightarrow h(۱۷, ۳) = ۳$$

$$h(۸۸, ۰) = ۰ \text{ تصادف} \Rightarrow h(۸۸, ۱) = ۴ \text{ تصادف} \Rightarrow h(۸۸, ۲) = ۳ \text{ تصادف} \Rightarrow h(۸۸, ۳) = ۸$$

$$\text{تصادف} \Rightarrow h(۸۸, ۴) = ۸ \text{ تصادف} \Rightarrow h(۸۸, ۵) = ۳ \text{ تصادف} \Rightarrow h(۸۸, ۶) = ۴ \text{ تصادف}$$

$$\Rightarrow h(۸۸, ۷) = ۰ \text{ تصادف} \Rightarrow h(۸۸, ۸) = ۲ \text{ و } h(۵۹, ۰) = ۴ \text{ تصادف} \Rightarrow h(۵۹, ۱) = ۸ \text{ تصادف}$$

$$\Rightarrow h(۵۹, ۲) = ۷$$

	۰	۱	۲	۳	۴	۵	۶	۷	۸	۹	۱۰
T	۲۲		۸۸	۱۷	۴		۲۸	۵۹	۱۵	۳۱	۱۰

البته مقادیر c_1 و c_2 در این سوال مناسب نیستند زیرا همانطور که دیدیم $h(۸۸, ۳)$ و $h(۸۸, ۴)$ هر دو یک آدرس تولید کردند.

توجه: مدلی از کاوش مربعی وجود دارد که از تابع $h(k, i) = (h'(k) \pm i^2) \bmod m$ که

$۰ \leq i \leq \frac{m-1}{2}$ استفاده می‌کند. در این تابع اگر m عدد اولی به فرم $۴j + ۳$ باشد (۳ و ۷ و ۱۱ و ۱۹ و ۲۳ و ۳۱ و ...) آنگاه این تابع همه حفره‌ها را واری می‌کند.

کاوش دابل (double hashing)

تابع درهم‌ساز این روش $h(k, i) = (h_1(k) + ih_2(k)) \bmod m$ می‌باشد که h_1 و h_2 توابع درهم‌ساز جانبی هستند. برای آنکه تمام حفره‌ها بررسی شوند، مقدار $h_2(k)$ باید نسبت به m اول باشد. یک راه برای رسیدن به این منظور آن است که m را توانی از ۲ انتخاب کنیم و h_2 را طوری انتخاب کنیم که عدد فرد تولید کند. راه دیگر این است که m اول باشد و h_2 مقداری کوچکتر از m تولید کند؛ مثلاً می‌توان m را اول انتخاب کرد و $h_2(k) = k \bmod m$ و $h_1(k) = ۱ + (k \bmod m')$ که m' مقدار کوچکتری از m (مثلاً $m - ۱$) می‌باشد.

مثال: فرض کنید $m = ۱۳$ و $h_1(k) = k \bmod ۱۳$ و $h_2(k) = ۱ + (k \bmod ۱۱)$ آنگاه طبق جدول مقابل، کلید ۱۴ در کدام حفره قرار می‌گیرد؟

پاسخ:

	۰	۱	۲	۳	۴	۵	۶	۷	۸	۹	۱۰	۱۱	۱۲
T		۷۹			۶۹	۹۸		۷۲		۹		۵۰	

$$h(۱۴, ۰) = ۱ \text{ تصادف} \Rightarrow h(۱۴, ۱) = (۱ + ۴) \bmod ۱۳ = ۵ \text{ تصادف} \Rightarrow h(۱۴, ۲) = ۹ \text{ تصادف}$$

$$\Rightarrow h(۱۴, ۳) = (۱ + ۱۲) \bmod ۱۳ = ۰ \Rightarrow \text{در حفره } ۰ \text{ قرار می‌گیرد.}$$

قضیه: در جدول درهم‌ساز با آدرس‌دهی باز و فاکتور لود $\alpha = \frac{n}{m} < 1$ ، تعداد مورد انتظار کاوش‌ها در جستجوی ناموفق حداکثر $\frac{1}{1-\alpha}$ است. زیرا ما همیشه اولین کاوش را انجام می‌دهیم و سپس با احتمال α ، اولین کاوش، حفره را پر می‌بیند و باید کاوش دوم انجام شود، حال با احتمال α^2 دو حفره اول پر هستند و باید کاوش سوم انجام شود و ... پس $1 + \alpha + \alpha^2 + \dots$ که برابر $\frac{1}{1-\alpha}$ است.

نتیجه: درج یک عنصر در یک جدول درهم‌ساز با آدرس‌دهی باز حداکثر $\frac{1}{1-\alpha}$ کاوش در حالت متوسط انجام می‌دهد.

قضیه: در جدول درهم‌ساز با آدرس‌دهی باز تعداد مورد انتظار کاوش در جستجوی موفق حداکثر $\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$ است.

اثبات: جستجو برای کلید k دقیقاً همان دنباله کاوش‌هایی را تولید می‌کند که کلید k را می‌خواهیم درج کنیم. بنابراین اگر k ، کلید $i+1$ ام باشد که درج می‌شود، تعداد مورد انتظار کاوش برای جستجوی k حداکثر $\frac{1}{m-i} = \frac{m}{m-i}$ است. با میانگین‌گیری روی همه n کلید جواب بدست می‌آید:

$$\begin{aligned} \frac{1}{n} \sum_{i=0}^{n-1} \frac{m}{m-i} &= \frac{m}{n} \left(\frac{1}{m} + \frac{1}{m-1} + \dots + \frac{1}{m-n+1} \right) \\ &\approx \frac{1}{\alpha} \left[\ln m - \ln(m-n) \right] = \frac{1}{\alpha} \ln \frac{m}{m-n} = \frac{1}{\alpha} \ln \frac{1}{1-\alpha} \end{aligned}$$

توابع درهم‌ساز

یک تابع درهم‌ساز خوب باید تقریباً یکنواخت ساده باشد یعنی هر کلید با احتمال یکسان به هر یک از m حفره نگاشته شود، مستقل از محل قرارگیری سایر کلیدها. ولی متأسفانه راهی وجود ندارد که این خاصیت را چک کنیم. چون لزوماً اطلاعاتی راجع به کلیدها نداریم. ولی اگر اطلاع داشته باشیم می‌توان تابع مناسبی یافت. مثلاً اگر کلیدها به صورت تصادفی در بازه $[0, 1)$ باشند

آنگاه تابع $h(k) = \lfloor km \rfloor$ شرط درهم‌سازی یکنواخت ساده را برآورده می‌کند.

تابع تقسیم نیز معروف است که به صورت $h(k) = k \bmod m$ می‌باشد. در استفاده از تابع تقسیم باید مراقب باشیم و از برخی مقادیر m حذر کنیم. مثلاً m نباید توانی از ۲ باشد زیرا اگر

$m = 2^p$ آنگاه $h(k)$ فقط p بیت سمت راست k است و به همه بیت‌های k وابسته نیست، زیرا اصولاً بهتر است تابع طوری باشد که به همه بیت‌های کلید وابسته باشد. یک عدد اول که خیلی نزدیک به توان‌های ۲ نباشد انتخاب خوبی برای m است. مثلاً فرض کنید $n = 2000$ کلید را با روش زنجیره‌سازی ذخیره می‌کنیم و می‌پذیریم که در جستجوی ناموفق متوسط ۳ عنصر را بررسی کنیم در این صورت $m = 701$ برای اندازه جدول درهم‌ساز انتخاب خوبی است زیرا ۷۰۱ عدد اولی نزدیک $\frac{2000}{3}$ است ولی نزدیک هیچ توانی از ۲ نیست. تابع دیگری به نام تابع ضرب به شکل $h(k) = \lfloor m(kA \bmod 1) \rfloor$ است که $0 < A < 1$ و $kA \bmod 1$ یعنی قسمت اعشاری kA یعنی $kA - \lfloor kA \rfloor$.

تابع دیگری به نام *mid-square* نیز مطرح است که کلید را به توان ۲ می‌رساند و تعدادی از بیت‌های وسط حاصل را انتخاب می‌کند. مثلاً اگر r بیت انتخاب شود، بازه آدرس از $2^r - 1$ تا 2^r است پس بهتر است در این روش اندازه جدول توانی از ۲ انتخاب شود. تابع دیگری به نام *folding* نیز مطرح است. در این روش کلید k به قسمت‌های با اندازه مساوی (احتمالاً به جز آخرین قسمت) تقسیم می‌شود و سپس این قسمت‌ها با هم جمع می‌شوند که آدرس بدست آید. این روش خود دو نوع است: *shift folding* -۱ و *shift folding at the boundaries* -۲. با مثال توضیح می‌دهیم.

مثال: فرض کنید کلید $k = 12320324111220$ باشد و ما این کلید را به بخش‌های ۳ رقمی تقسیم می‌کنیم یعنی $P_1 = 123$ و $P_2 = 203$ و $P_3 = 241$ و $P_4 = 112$ و $P_5 = 20$ حال در روش *shift folding* این بخش‌ها با هم جمع می‌شوند:

$$h(k) = \sum_{i=1}^5 P_i = 123 + 203 + 241 + 112 + 20 = 699$$

در روش *folding at the boundaries*، P_4 و P_5 معکوس می‌شوند و سپس جمع انجام می‌شود.

$$h(k) = 123 + 302 + 241 + 211 + 20 = 897 \text{ یعنی}$$

تمرین

- ۱- در یک جدول آدرس مستقیم T به طول m ، یک مجموعه پویای s قرار دارد. عنصر ماکزیمم را بیابید. بدترین حالت از چه مرتبه‌ای است؟
- ۲- یک بردار بیتی یک آرایه از بیت‌ها $(1, 0)$ است. توضیح دهید چگونه با یک بردار بیتی می‌توان یک مجموعه دینامیک شامل عناصر متمایز بدون *Satellite data* را ذخیره کرد. عملیات دیکشنری باید در زمان $O(1)$ انجام شود.
- ۳- توضیح دهید چگونه می‌توان یک جدول آدرس مستقیم که کلیدها متمایز نیستند و عناصر می‌توانند *satellite data* داشته باشند پیاده‌سازی کرد. هر سه عمل درج حذف جستجو باید در زمان $O(1)$ انجام شود (فراموش نکنید، الگوریتم حذف به عنوان آرگومان اشاره‌گر به عنصر را دریافت می‌کند و نه کلید را).
- ۴- می‌خواهیم یک دیکشنری با استفاده از آدرس‌دهی مستقیم در یک آرایه نامحدود پیاده‌سازی کنیم. خانه‌های آرایه قابل مقداردهی اولیه نیستند و مقادیر نامشخص (زباله) دارند. روشی برای پیاده‌سازی دیکشنری با آدرس‌دهی مستقیم در آرایه نامحدود ارائه دهید. عملیات حذف درج جستجو باید با زمان $O(1)$ انجام شود. و مقداردهی به ساختمان داده نیز باید زمان $O(1)$ باشد. (راهنمایی: یک آرایه اضافی که عمل پشته را انجام دهد و طولش تعداد واقعی کلیدهایی است که در دیکشنری ذخیره می‌شوند استفاده کنید که نشان دهند خانه‌های آرایه نامحدود، معتبرند یا خیر.
- ۵- فرض کنید تابع درهم‌ساز h را برای ذخیره n کلید متمایز در آرایه به طول m استفاده کرده‌ایم. با فرض درهم‌سازی یکنواخت ساده، تعداد تصادف‌های مورد انتظار چندتاست؟ به عبارتی تعداد اعضای $\{(k, l) : k \neq l \text{ and } h(k) = h(l)\}$ چند است؟
- ۶- مجموعه توابع H که مجموعه کلیدهای u را به m حفره نگاشت می‌کنند. جهانی (*universal*) گویند هرگاه برای هر دو کلید متفاوت $x, y \in u$ تعداد توابع $h \in H$ که برای آنها $h(x) = h(y)$ دقیقاً $\frac{|H|}{m}$ باشد. بررسی کنید مجموعه توابع $H = \{h_1, h_2, h_3\}$ که کلیدهای $u = \{A, B, C, D\}$ را به اعداد $\{0, 1, 2\}$ می‌نگارند با توجه به جدول زیر، جهانی است.

x	$h_1(x)$	$h_2(x)$	$h_3(x)$
A	۱	۰	۲
B	۰	۱	۲
C	۰	۰	۰
D	۱	۱	۰

پاسخ. داریم $|H| = ۳$ و $m = ۳$ بنابراین برای هر دو کلید متفاوت از ۴ کلید A و B و C و D دقیقاً یک تابع باید ایجاد تصادف کند. $h(A) = h(B)$ فقط برای h_p که به حفره ۲ نگاشت می‌کند. $h(C) = h(D)$ فقط برای h_p که به حفره ۰ نگاشت می‌کند.

$h(A) = h(C)$ فقط برای h_p که به حفره ۰ نگاشت می‌کند.

$h(A) = h(D)$ فقط برای h_p که به حفره ۱ نگاشت می‌کند.

$h(B) = h(C)$ فقط برای h_p که به حفره ۰ نگاشت می‌کند.

$h(B) = h(D)$ فقط برای h_p که به حفره ۱ نگاشت می‌کند.

سؤال‌های طبقه‌بندی شده فصل چهارم

- ۱- در جدول در هم‌سازی (*hashing*) با واریسی خطی (*linear probing*) اگر تابع درهم‌سازی برای هفت عنصر ورودی به صورت زیر باشد،

<i>key</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>F</i>	<i>G</i>
<i>hash</i>	۳	۵	۳	۴	۵	۶	۳

کدام یک از گزینه‌های زیر نمی‌تواند حاصل درج این عناصر با هر ترتیب دلخواه در آرایه ۷ تایی $H[0..6]$ (که در ابتدا تهی است) باشد؟

- (۱) $H[0..6] = [E \ F \ G \ A \ C \ B \ D]$ (۲) $H[0..6] = [C \ E \ B \ G \ F \ D \ A]$
 (۳) $H[0..6] = [B \ D \ F \ A \ C \ E \ G]$ (۴) $H[0..6] = [C \ G \ B \ A \ D \ E \ F]$

- ۲- جدول درهم‌سازی (*hashing table*) به اندازه‌ی $2n$ را در نظر بگیرید که تصادف به صورت لیست پیوندی حل می‌شود. فرض کنید که به تعداد $\frac{n}{8}$ عنصر در جدول هستند. با فرض آن که تابع درهم‌سازی (*hashing function*) با احتمال برابر هر عنصر را به یک درایه از جدول نگاشت می‌کند، متوسط تعداد عناصر در هر درایه از جدول چند است؟ (دوایه ۸۴)

- (۱) $\frac{1}{8}$ (۲) $\frac{1}{4}$ (۳) $\frac{1}{2}$ (۴) $\frac{1}{16}$

- ۳- اگر آرایه $s[0..7]$ و عناصر A تا E به آدرس‌های جدول زیر با روش کاوش خطی نگاشت شوند، متوسط تعداد مقایسه‌ها برای جستجوی موفق (s) و ناموفق (u) کدام است؟

	$s = 1$	$s = 2$
	$u = 2 / 5$ (۲)	$u = 4$ (۱)
	$s = \frac{1}{5}$	$s = \frac{1}{7}$
	$u = \frac{19}{8}$ (۴)	$u = 2 / 5$ (۳)

- ۴- H یک جدول درهم‌سازی (*Hash table*) است که دارای m خانه است و در آن n عنصر ذخیره می‌شود. برای رفع برخورد، این جدول از روش زنجیر (*collision resolution by chaining*) بهره می‌جوید. متوسط و بدترین زمان جستجو برای یک عنصر چقدر است؟ (II-۸۸)

- (۱) $\theta\left(\frac{n}{m}\right), \theta(n)$ (۲) $\theta\left(\frac{m}{n}\right), \theta(m)$
 (۳) $\theta\left(\frac{n}{m}\right), \theta\left(\frac{n}{m}\right)$ (۴) $\theta\left(\frac{m}{n}\right), \theta\left(\frac{n}{m}\right)$

۵- فرض کنید که n عنصر با کلیدهای مجزا از هم را با استفاده از یک تابع در هم‌سازی ساده و یکنواخت در یک آرایه به اندازه‌ی m درج می‌کنیم. با این تابع احتمال آن که دو عنصر دلخواه و متفاوت به یک درایه نگاشت شوند برابر است. میانگین تعداد برخورد‌های دو عنصر چقدر است؟

$$(1) \Theta\left(\frac{n}{m}\right) \quad (2) \Theta\left(\frac{m}{n}\right) \quad (3) \Theta\left(\frac{n^2}{m}\right) \quad (4) \Theta\left(\frac{n^2}{m^2}\right)$$

۶- فرض کنید که یک جدول در هم‌سازی به اندازه ۸۰ (هشتاد) از روش آدرس‌دهی خطی باز استفاده می‌کند. ابتدا جدول خالی بوده است و تنها عملیات اضافه کردن و جستجو روی جدول انجام شده است. در حال حاضر وضعیت درایه‌های ۴۵ تا ۵۶ جدول به صورت زیر است. اعداد بالای آرایه، اندیس درایه‌ها و اعداد پایین آرایه خروجی تابع در هم‌سازی است. اگر یکبار دیگر از جدول در هم‌سازی خالی شروع کنیم و همان عملیات را به غیر از اضافه کردن کلید e دوباره انجام دهیم. در داریه‌ی ۵۰ چه کلیدی قرار می‌گیرد؟ (۸۹ IT)

$i(1)$	$f(2)$	۵۶	۵۵	۵۴	۵۳	۵۲	۵۱	۵۰	۴۹	۴۸	۴۷	۴۶	۴۵
$h(3)$	$g(4)$	j	i	h	g	f	e	d	c	b	a		
		۴۹	۴۸	۴۶	۴۷	۵۱	۴۶	۴۷	۴۶	۴۶	۴۶	۴۶	۴۵

۷- فرض کنید که در ساخت جدول در هم‌سازی زیر از روش آدرس‌دهی خطی باز استفاده شده است. اگر تابع $Hash$ به صورت باقیمانده تقسیم بر ۵ تعریف شده باشد، کدام گزینه نمی‌تواند ترتیب صحیح درج عناصر در جدول باشد؟ ترتیب از چپ به راست است. (۸۹ IT)

۱۰	۰
۱۱	۱
۶	۲
۷	۳
۱۵	۴

(۱) ۱۱، ۷، ۱۰، ۶، ۱۵

(۲) ۱۰، ۱۱، ۶، ۷، ۱۵

(۳) ۱۱، ۶، ۷، ۱۰، ۱۵

(۴) ۱۱، ۱۰، ۶، ۱۵، ۷

۸- یک جدول در هم‌سازی ($hashing\ table$) ساده را در نظر بگیرید که اندازه‌ی آن در ابتدا ۱ است و آن درایه هم خالی است. در هم‌سازی هم به صورت بسته ($closed$) (در مقابل زنجیره‌ای یا $chaining$) است و با یک تابع در هم‌سازی ساده و واریسی خطی ($linear\ probing$) انجام می‌شود. برای درج یک عنصر x ، اگر جدول جا داشته باشد، x مستقیماً درج می‌شود. وگرنه اندازه جدول را دو برابر می‌کنیم و عناصر فعلی را با هزینه‌ای برابر تعدادشان، به جدول جدید منتقل می‌کنیم و سپس x در جدول جدید درج می‌شود.

اگر ۱۳۸۹ عنصر در این جدول درج شوند، هزینه‌ی کل در مجموع چقدر خواهد بود؟ (توجه کنید که هزینه در اینجا تعداد نوشتن در جدول است و ما بقیه‌ی هزینه‌ها را ثابت فرض می‌کنیم). (مهندسی کامپیوتر ۹۰)

$$(1) 1389 \quad (2) 2048 \quad (3) 3412 \quad (4) 3445$$

[پاسخنامه سؤال‌های طبقه‌بندی شده فصل چهارم]

(۱) گزینه (۲). اگر عناصر به همان ترتیب ($ABCDEFG$) چپ به راست) وارد شوند، گزینه ۱ حاصل می‌شود. اگر ترتیب به صورت ($ACEGBDF$) باشد، گزینه ۳ بدست می‌آید (البته ترتیب‌های دیگر نیز دارد). اگر ورودی به صورت ($ADEFCGB$) باشد گزینه ۴ حاصل می‌شود. در گزینه ۲، ابتدا حتماً باید G وارد شود ولی بعد از آن هر عنصری وارد شود، گزینه ۲ حاصل نمی‌شود.

(۲) گزینه (۴).

$$\frac{\frac{n}{8}}{2n} = \frac{1}{16}$$

(۳) گزینه (۴).

۰	۱	۲	۳	۴	۵	۶	۷
	E		A	C	B	D	

$$s = \frac{1+1+1+1+4}{5} = \frac{8}{5} \quad u = \frac{1+2+1+5+4+3+2+1}{8} = \frac{19}{8}$$

(۴) گزینه (۱). متوسط $\theta(1+\alpha)$ که $\alpha = \frac{n}{m}$ که در این تست از ۱ صرف‌نظر شده و فقط در صورتی درست که $\alpha \geq 1$.

$$(۵) \text{ گزینه (۳). میانگین تعداد برخوردها } \frac{n(n-1)}{2m} \times \frac{1}{m} = \frac{n}{2m} \text{ است که از مرتبه } \theta\left(\frac{n^2}{m}\right)$$

(۶) گزینه (۴). a و b و c و d تغییر نمی‌کنند. f نیز در ۵۱ قرار می‌گیرد. g که با b تصادف می‌کند به ۵۰ وارد می‌شود.

(۷) گزینه (۴). گزینه ۴ را بررسی می‌کنیم:

$$11 \bmod 5 = 1$$

۰	۱	۲	۳	۴
	۱۱			

$$10 \bmod 5 = 0$$

۰	۱	۲	۳	۴
۱۰	۱۱			

$$6 \bmod 5 = 1$$

۰	۱	۲	۳	۴
۱۰	۱۱	۶		

$$۱۵ \bmod ۵ = ۰$$

۰	۱	۲	۳	۴
۱۰	۱۱	۶	۱۵	

$$۷ \bmod ۵ = ۲$$

۰	۱	۲	۳	۴
۱۰	۱۱	۶	۱۵	۷

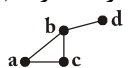
سایر گزینه‌ها را بررسی کنید که صحیح هستند.

۸) این تست حذف شد. فرض کنید هزینه برابر مجموع تعداد درجهایی است که انجام می‌دهیم به علاوه تعداد عملیات کپی که بخاطر دو برابر شدن جدول انجام می‌شود:

$$= ۳۴۳۶ = (۱۰۲۴ + \dots + ۴ + ۲ + ۱) + ۱۳۸۹ = (\text{کپی‌ها}) + \text{درج‌ها} = \text{هزینه}$$

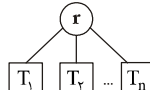
(دقت کنید پس از درج عنصر ۲^i ، جدول ۲ برابر می‌شود و ۲^i عنصر جدول اول به جدول دوم کپی می‌شوند)

درخت ریشه‌دار (Rooted Tree) فصل ۵

می‌دانیم درخت ($tree$) یا درخت آزاد ($free\ tree$) گراف همبند (متصل $connected$) بدون سیکل (دور $cycle$) است. مثلاً  درخت نیست چون دارای سیکل $(a\ b\ c\ a)$ است. و یا  درخت نیست چون ناهمبند است و شامل ۳ مؤلفه است.

ولی گراف  درخت است. در این بخش می‌خواهیم درخت ریشه‌دار را بررسی کنیم.

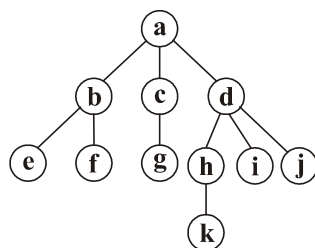
درخت ریشه‌دار، ساختمان داده‌ای است که گره (نود $node$) خاصی به اسم ریشه دارد و سایر نودها به مجموعه‌های جدا از هم تقسیم می‌شوند که به آن‌ها زیردرخت‌های ریشه گویند که این

زیردرخت‌ها، خود درخت ریشه‌دار هستند. شکل کلی درخت ریشه‌دار به صورت  صورت

است که r ریشه است و T_1 و T_2 و $\dots$ و T_n زیر درختان ریشه هستند که خود درخت ریشه‌دار می‌باشند. توجه کنید که درخت ریشه‌دار، گراف همبند بدون سیکل است و علاوه بر این مفاهیم خاصی وجود دارد مثل والد (پدر $parent$)، فرزند ($child$)، گره‌های همزاد (برادر، هم‌نیا $sibling$)، جد ($ancestor$)، نواده ($descendant$)، ارتفاع ($height$)، عمق ($depth$) و... که با یک مثال این مفاهیم را تعریف می‌کنیم.

مثال ۱: در درخت ریشه‌دار شکل ۱، a ریشه است. d, c, b فرزندان ریشه هستند، پس d, c, b با هم برادر یا همزاد هستند. b پدر یا والد f, e است. c پدر g است. d پدر h, i, j می‌باشد و h پدر k است. می‌توان گفت نود c عموی نودهای j, i, h, f, e است. زیرا c برادر نودهای

d, b است. اجداد یک نود، همه نودهایی هستند که در مسیر از آن نود به ریشه قرار دارند.



شکل ۱

نواده‌های یک نود، همه نودهایی هستند که در مسیر از آن نود به سمت پایین قرار دارند. لازم به ذکر است که یک نود هم جد و هم نواده خودش می‌باشد. در درخت شکل ۱، اجداد نود k عبارتند از a, d, h, k و یا اجداد نود d عبارتند از a, d . نوده‌های نود d عبارتند از k, j, i, h, d . نوده‌های a عبارتند از تمام نوده‌های موجود در درخت.

منظور از اجداد محض یک نود، همه اجداد آن نود به جز خود نود است؛ و منظور از نواده‌های محض یک نود، همه نواده‌های آن نود به جز خود نود است. در درخت فوق، اجداد محض k عبارتند از a, d, h و نواده‌های محض d عبارتند از k, j, i, h . درجه ($degree$) یک نود، تعداد فرزندان آن نود است. پس در درخت شکل ۱، درجه a برابر ۳ یعنی $\deg(a) = 3$ می‌باشد. به همین ترتیب $\deg(b) = 2$ و $\deg(c) = 1$ و $\deg(d) = 3$ و درجه نوده‌های j, i, k, g, f, e برابر صفر است.

ماکزیمم درجه را درجه درخت گویند. پس درخت شکل ۱، درخت درجه ۳ یا درخت ۳تایی است. بنابراین منظور از درخت k تایی (k -ary tree) درختی است که هر نود آن حداکثر k فرزند دارد.

نودهایی که درجه صفر هستند را برگ ($leaf$) یا گره خارجی ($external node$) گویند پس j, i, k, g, f, e برگ هستند. سایر نودها یعنی نودهایی که فرزند دارند یعنی درجه‌شان مخالف صفر است را نود داخلی ($internal node$) گویند. پس a, b, c, d, h نود داخلی هستند.

ارتفاع یک نود برابر فاصله آن نود تا دورترین برگ موجود در زیر درخت آن نود است. منظور از فاصله، تعداد یال ($edge$) موجود در مسیر است. مثلاً در درخت شکل ۱، ارتفاع نود d برابر ۲ است زیرا فاصله d تا k ($d \rightarrow h \rightarrow k$) برابر ۲ است. ارتفاع نود b برابر ۱ است (فاصله b تا e یا b تا f برابر ۱ است) ارتفاع برگ‌ها برابر صفر است. ارتفاع a برابر فاصله a تا k یعنی ۳ است. ارتفاع درخت، ارتفاع ریشه درخت است. پس ارتفاع درخت شکل ۱ برابر ۳ است.

توجه کنید این درخت شامل ۴ سطح ($level$) است ولی ارتفاع آن ۳ می‌باشد. پس ارتفاع درخت، یک واحد کمتر از تعداد سطوح است (هرچند برخی منابع ارتفاع درخت را برابر تعداد سطوح تعریف می‌کنند و ارتفاع درخت شکل ۱ را ۴ در نظر می‌گیرند ولی ما طبق تعریف منابع معتبر، ارتفاع درخت را یک واحد کمتر از تعداد سطوح تعریف می‌کنیم). عمق یک نود برابر فاصله

آن نود تا ریشه است. پس در درخت شکل ۱، عمق a (یعنی ریشه) برابر صفر است. عمق گره‌های d, c, b برابر یک است. عمق گره‌های e, f, g, h, i, j برابر ۲ است. عمق گره k برابر ۳ است. حداکثر عمق در این درخت ۳ است (عمق گره k). عمق درخت برابر حداکثر عمق گره‌های آن است. پس عمق این درخت همانند ارتفاع آن برابر ۳ است.

مثال ۲: درخت ۳تایی (3-ary tree) که دارای ۴ سطح است. حداکثر چند تا نود، حداکثر چند تا برگ و حداکثر چند تا نود داخلی دارد؟

پاسخ: حداکثر نود وقتی است که درخت اصطلاحاً پر باشد. سطح اول فقط ریشه است و یک نود وجود دارد. در سطح دوم ۳ نود، در سطح سوم ۹ نود و در سطح چهارم ۲۷ نود، پس جمعاً ۴۰ نود وجود دارد که ۲۷ نود آن برگ و ۱۳ نود آن داخلی است. شکل این درخت به صورت مقابل است.



مثال ۳: درخت k تایی که x تا سطح دارد، حداکثر چند نود، حداکثر چند برگ و حداکثر چند نود داخلی دارد؟

پاسخ: مشابه مثال قبل، درخت باید پر باشد. در سطح اول یک نود، در سطح دوم k نود، در سطح سوم k^2 نود، در سطح چهارم k^3 نود و به همین ترتیب در سطح x ام تعداد k^{x-1} نود وجود دارد. پس حداکثر تعداد نود برابر $\frac{k^x - 1}{k - 1} = 1 + k + k^2 + \dots + k^{x-1}$ است.

در چنین درختی، نودهای سطح آخر برگ و سایر نودها، داخلی هستند. پس حداکثر k^{x-1} برگ و حداکثر $\frac{k^{x-1} - 1}{k - 1} = 1 + k + k^2 + \dots + k^{x-2}$ نود داخلی دارد.

مثال ۴: یک درخت m تایی پر با ارتفاع ۷ (۸ سطح) دارای ۲۷۹۹۳۶ برگ است. تعداد رئوس (نودهای) داخلی این درخت چقدر است؟

پاسخ: طبق مثال ۳، تعداد برگ چنین درختی m^7 است پس $m^7 = 279936$ که $m = 6$ به دست می‌آید پس تعداد نودهای داخلی برابر $\frac{m^7 - 1}{m - 1} = \frac{279935}{5} = 55987$ است.

نتیجه: حداکثر تعداد نود درخت درجه k (درخت k تایی $k\text{-ary tree}$) به ارتفاع h برابر $\frac{k^{h+1} - 1}{k - 1}$ و حداکثر تعداد برگ‌ها k^h و حداکثر تعداد نود داخلی $\frac{k^h - 1}{k - 1}$ است. دقت کنید درخت به ارتفاع h دارای $h + 1$ سطح است.

مثال ۵: درخت ۳تایی با ۱۰۰۰ نود حداقل و حداکثر دارای چند سطح است؟

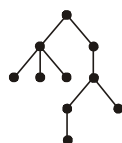
پاسخ: حداکثر تعداد سطوح ۱۰۰۰ تاست و وقتی حاصل می‌شود که در هر سطح فقط یک نود قرار گیرد، یعنی هر نود داخلی فقط یک فرزند داشته باشد. توجه کنید درخت ۳تایی درختی است که هر نود آن حداکثر ۳ فرزند دارد یعنی نودها می‌توانند صفر یا یک یا دو یا سه فرزند داشته باشند. فرض کنید حداقل تعداد سطح برابر x است. حداقل تعداد سطوح وقتی حاصل می‌شود که در هر سطح، ماکزیمم تعداد نود قرار گیرد. پس در سطح اول یک نود، در سطح دوم ۳ نود، در سطح سوم تعداد ۳^۲ نود و به همین ترتیب در سطح x ام تعداد ۳ ^{$x-1$} نود باید قرار گیرد پس:

$$1 + 3 + 3^2 + \dots + 3^{x-1} \geq 1000 \rightarrow \frac{3^x - 1}{3 - 1} \geq 1000 \rightarrow 3^x \geq 2001$$

$$\rightarrow x \geq \log_3 2001$$

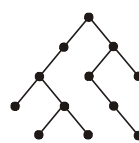
باتوجه به اینکه $3^6 < 2001 < 3^7$ پس $6 < \log_3 2001 < 7$ بنابراین x باید حداقل ۷ باشد.

نکته: فرض کنید n_k برابر تعداد گره‌های k فرزندی، n_{k-1} برابر تعداد گره‌های $k-1$ فرزندی و به همین ترتیب n_0 برابر تعداد گره‌های بی‌فرزند (تعداد برگ‌ها) باشد. آنگاه در هر درخت k تایی همیشه رابطه $n_0 = n_k + (k-2)n_{k-1} + \dots + n_2 + 1$ برقرار است. مثلاً در درخت ۳تایی رابطه $n_0 = 2n_2 + n_1 + 1$ برقرار است و یا در درخت ۲تایی (دودویی) رابطه $n_0 = n_2 + 1$ برقرار است. برای تحقیق درستی این روابط، در شکل ۲ چند درخت ریشه‌دار کشیده شده است:



(a) درخت ۳تایی

$$n_0 = 5, n_2 = 2, n_1 = 1$$



(b) درخت دودویی

$$n_0 = 5, n_2 = 4$$

شکل ۲- در قسمت a رابطه $n_0 = 2n_2 + n_1 + 1$ و در قسمت b رابطه $n_0 = n_2 + 1$ برقرار است.

مثال ۶: رابطه $n_0 = n_2 + 1$ را در درخت دودویی اثبات کنید.

پاسخ: اگر n تعداد نودها و e تعداد یال‌ها باشد، در هر درخت $n = e + 1$ است. از طرفی در درخت دودویی $n = n_0 + n_1 + n_2$ و $e = 2n_2 + n_1$ زیرا از هر نود ۲ فرزندی، ۱ یال و از هر نود یک فرزندی، یک یال خارج می‌شود. پس با ترکیب این روابط داریم:

$$n = e + 1 \rightarrow n_0 + n_1 + n_2 = 2n_2 + n_1 + 1 \rightarrow n_0 = n_2 + 1$$

تمرین: در درخت k تایی رابطه $n_k = (k-1)n_{k-1} + (k-2)n_{k-2} + \dots + n_2 + 1$ را اثبات کنید.
 تست ۱: در یک درخت ۳ تایی تعداد نودها ۲۹ تا نود سه فرزندی و ۳ تا نود ۲ فرزندی دارد. تعداد نودهای تک‌فرزندی کدام است؟

$$۴ \quad (۴) \quad ۶ \quad (۳) \quad ۱۰ \quad (۲) \quad ۸ \quad (۱)$$

پاسخ: $n_3 = ۴$ و $n_2 = ۳$ پس $n_1 = ۱۲ = ۲n_2 + n_3 + 1$. از طرفی اگر n تعداد نودها باشد داریم:

$$n = n_1 + n_2 + n_3 + n_4 \quad \text{پس } n = n_1 + ۳ + ۴ + ۱۲ = ۲۹ \text{ در نتیجه } n_4 = ۱۰.$$

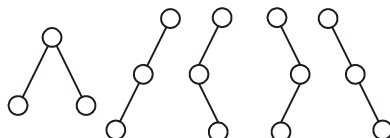
درخت دودویی (binary tree)

درخت دودویی، درخت ریشه‌داری است که هر نود آن حداکثر ۲ فرزند دارد. یعنی نودها یا صفر یا یک یا دو فرزند دارند. در درخت دودویی، فرزند چپ با فرزند راست متفاوت است. مثلاً

درخت دودویی  با درخت دودویی  متفاوت است، زیرا در اولی b فرزند چپ a است و در دومی b فرزند راست a است.

مثال ۷: چه تعداد درخت دودویی شامل ۳ نود، می‌توان ساخت؟

پاسخ: با ۳ نود می‌توان ۵ درخت دودویی ساخت که عبارتند از:



◀ **نکته:** تعداد درخت‌های دودویی با n نود برابر است با عدد n ام کاتالان $c_n = \frac{1}{n+1} \binom{2n}{n}$.

مثال ۸: چه تعداد درخت دودویی شامل ۴ نود، می‌توان ساخت؟

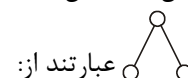
پاسخ: با ۴ نود تعداد $c_4 = ۱۴$ درخت دودویی وجود دارد:

$$c_4 = \frac{1}{5} \binom{8}{4} = \frac{1}{5} \frac{8!}{4!4!} = \frac{1}{5} \frac{8 \times 7 \times 6 \times 5}{4 \times 3 \times 2} = 7 \times 2 = ۱۴$$

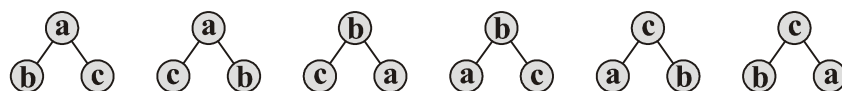
مثال ۹: با ۳ نود چه تعداد درخت دودویی برچسب‌دار می‌توان ساخت؟

پاسخ: منظور از درخت برچسب‌دار، درختی است که در هر نود آن یک کلید (عدد یا حرف) وجود دارد و جایگشت کلیدها، درخت‌های متفاوت ایجاد می‌کند. می‌دانیم n شیء متفاوت دارای

$n!$ جایگشت هستند. پس سه کلید متمایز a, b, c دارای $3! = 6$ جایگشت هستند بنابراین تعداد $3 \times 5 = 6 \times 5 = 30$ درخت دودویی برچسب‌دار با ۳ نود وجود دارد. در واقع هر یک از درخت‌های مثال ۷، تبدیل به ۶ درخت دودویی برچسب‌دار می‌شوند که ۶ درخت برچسب‌دار برای درخت



عبارتند از:



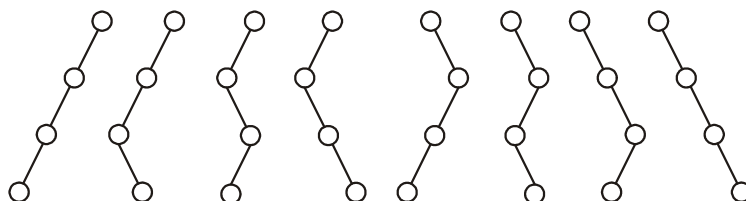
نکته: تعداد درخت دودویی برچسب‌دار شامل n نود برابر است با $n! \times c_n$ که c_n عدد n ام

$$\text{کاتالان یعنی } c_n = \frac{1}{n+1} \binom{2n}{n} \text{ است.}$$

مثال ۱۰: چه تعداد درخت دودویی با ۴ نود و بیشترین ارتفاع می‌توان ساخت؟

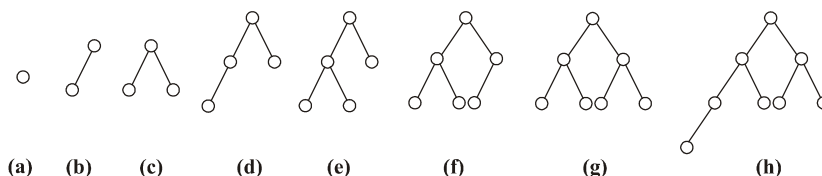
پاسخ: بیشترین ارتفاع وقتی حاصل می‌شود که در هر سطح، یک نود قرار گیرد. با ۴ نود تعداد

$2^3 = 8$ درخت با ارتفاع ۳ (۴ سطح) می‌توان ساخت. زیرا ریشه فقط یک موقعیت دارد ولی سایر نودها هر یک ۲ موقعیت دارند (فرزند چپ یا فرزند راست):



نکته: با n نود تعداد 2^{n-1} درخت دودویی با بیشترین ارتفاع می‌توان ساخت.

درخت‌های دودویی خاص: در درخت دودویی پر (full)، همه نودهای داخلی دارای ۲ فرزند هستند و همه برگ‌ها هم سطح هستند. در درخت دودویی کامل (complete)، همه سطوح ماقبل آخر، پر هستند و در سطح آخر، نودها از سمت چپ قرار می‌گیرند، توجه کنید که سطح آخر نیز می‌تواند پر باشد. یعنی هر درخت پر، درخت کامل نیز هست، البته واضح است که عکس این جمله صحیح نیست. شکل ۳ تعدادی درخت دودویی کامل را نشان می‌دهد که برخی از آن‌ها پر هستند:



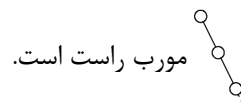
شکل ۳- درخت‌های دودویی کامل ۱ تا ۸ نودی، درخت‌های (a)، (c) و (g) پر هستند.

درخت دودویی مورب (*skewed*)، درخت دودویی است که در آن یا اصلاً فرزند چپ وجود

ندارد (مورب راست) و یا اصلاً فرزند راست وجود ندارد (مورب چپ) پس



مورب چپ و



مورب راست است.

◀ **نکته:** درخت دودویی پر با x سطح (ارتفاع $x-1$)، دارای $2^x - 1$ نود، 2^{x-1} برگ و

$1 - 2^{x-1}$ نود داخلی است.

◀ **نکته:** درخت دودویی کامل با x سطح (ارتفاع $x-1$)، حداقل $2^{x-1} = (2^{x-1} - 1) + 1$ و

حداکثر $2^x - 1$ نود دارد. حداقل وقتی است که $x-1$ سطح اول پر باشند و در سطح آخر فقط یک نود در سمت چپ‌ترین مکان، قرار دهیم. حداکثر وقتی است که همه x سطح پر باشند.

مثال ۱۱: چند تا درخت دودویی کامل با ۴ سطح می‌توان ساخت؟

پاسخ: باید سه سطح اول پر باشند و در سطح چهارم از سمت چپ هر نودی که قرار دهیم، یک

درخت کامل با ۴ سطح ایجاد می‌شود. پس تعداد این درخت‌ها برابر حداکثر تعداد برگ‌های موجود

در سطح چهارم یعنی ۸ تا است:



◀ **نکته:** تعداد درخت دودویی کامل با x سطح برابر 2^{x-1} است.

مثال ۱۲: با ۱۰۰۰ نود چند تا درخت دودویی کامل می‌توان ساخت؟

پاسخ: با یک تعداد نود مشخص فقط یک درخت دودویی کامل می‌توان ساخت. مثلاً با ۵ نود تنها

درخت دودویی کامل است. پس با ۱۰۰۰ نود فقط یک درخت دودویی کامل می‌توان ساخت.



مثال ۱۳: درخت دودویی کامل با ۱۰۰۰ نود دارای چند سطح است؟

پاسخ: فرض کنید تعداد سطوح برابر x باشد. در سطح اول تعداد یک نود، در سطح دوم تعداد ۲ نود، در سطح سوم تعداد $2^2 = 4$ نود و به همین ترتیب در سطح x ام حداکثر تعداد 2^{x-1} نود قرار می‌گیرد که مجموع این تعداد نودها باید بیشتر یا مساوی ۱۰۰۰ باشد پس:

$$1 + 2 + 2^2 + \dots + 2^{x-1} \geq 1000 \rightarrow 2^x - 1 \geq 1000 \rightarrow 2^x \geq 1001 \rightarrow x \geq \log_2 1001$$

پس x برابر ۱۰ می‌باشد.

نکته: تعداد سطوح درخت دودویی کامل n نودی برابر $\lceil \lg(n+1) \rceil = \lfloor \lg n \rfloor + 1$ است. پس ارتفاع درخت دودویی کامل n نودی برابر $\lfloor \lg n \rfloor$ است. لازم به ذکر است که ارتفاع درخت کامل، کمینه است.

مثال ۱۴: در سطح آخر درخت دودویی کامل ۱۰۰۰ نودی، چندتا نود وجود دارد؟

پاسخ: طبق مثال ۱۳، چنین درختی دارای ۱۰ سطح است و تا سطح ۹ پر است. تعداد نودها تا سطح ۹ برابر $2^9 - 1 = 511$ می‌باشد، پس تعداد نودهای سطح ۱۰ برابر $1000 - 511 = 489$ است.

مثال ۱۵: درخت دودویی کامل با ۱۰۰۰ نود، چند تا برگ دارد؟

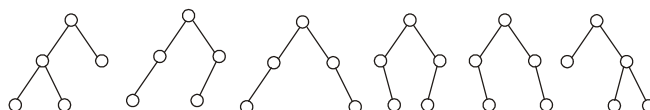
پاسخ: تعداد ۴۸۹ برگ در سطح آخر وجود دارد. در سطح ماقبل آخر (سطح نهم) تعداد $2^8 = 256$ نود قرار دارد که از این تعداد ۲۴۵ $\left\lceil \frac{489}{2} \right\rceil$ نود، دارای فرزند هستند پس $11 = 256 - 245$ نود، برگ هستند. بنابراین تعداد کل برگ‌ها $489 + 11 = 500$ می‌باشد.

نکته: تعداد برگ در درخت دودویی کامل n نودی برابر $\left\lceil \frac{n}{2} \right\rceil$ و تعداد نود داخلی برابر $\left\lfloor \frac{n}{2} \right\rfloor$ است.

مثال ۱۶: چه تعداد درخت دودویی با ۵ نود و ارتفاع کمینه وجود دارد؟

پاسخ: با ۵ نود، ارتفاع کمینه $\lfloor \lg 5 \rfloor = 2$ است یعنی ۳ سطح وجود دارد. سه نود در سطوح اول و دوم مصرف می‌شوند و ۲ نود باقی‌مانده باید در سطح سوم قرار گیرند که $\binom{4}{2} = 6$ حالت

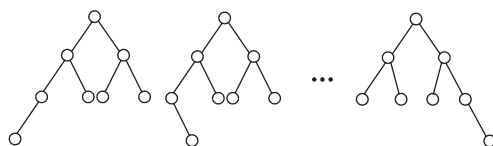
دارد. پس ۶ درخت دودویی با ۵ نود و ارتفاع ۲ وجود دارد که عبارتند از:



مثال ۱۷: چه تعداد درخت دودویی با ۸ نود و ارتفاع کمینه وجود دارد؟

پاسخ: ارتفاع کمینه با $n = 8$ نود برابر $\lceil \lg 8 \rceil = 3$ است یعنی چنین درختی دارای ۴ سطح است. حالات مختلف برای ساخت چنین درختی وجود دارد:

حالت ۱: در سه سطح ابتدایی درخت تعداد ۷ نود قرار دهیم و نود هشتم را به ۸ حالت در سطح چهارم قرار دهیم:

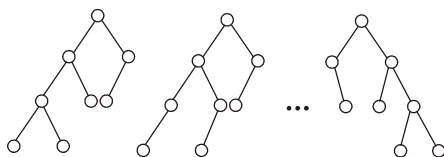


حالت ۲: در سه سطح ابتدایی، تعداد ۶ نود قرار دهیم. یعنی سطح سوم، ۳ نود قرار گیرد. که

خود ۴ حالت دارد (حداکثر تعداد نود در سطح سوم ۴ تا است که به $\binom{4}{1} = 4$ حالت یکی از آن‌ها را

باید حذف کنیم) و سپس ۲ نود باقی‌مانده را به $\binom{6}{2} = 15$ حالت در سطح چهارم قرار دهیم. پس

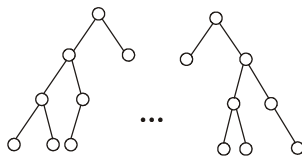
کل حالات این روش $4 \times 15 = 60$ می‌باشد:



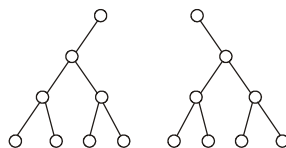
حالت ۳: در سه سطح ابتدایی، تعداد ۵ نود قرار دهیم. یعنی در سطح سوم، ۲ نود قرار گیرد که

$\binom{4}{2} = 6$ حالت دارد و سپس ۳ نود باقی‌مانده را به $\binom{4}{3} = 4$ حالت در سطح چهارم قرار دهیم.

پس کل حالات این روش $6 \times 4 = 24$ می‌باشد:



حالت ۴: دو درخت به شکل مقابل با ۴ سطح و ۸ نود وجود دارد:



بنابراین تعداد درخت‌های دودویی با ۸ نود و ارتفاع کمینه برابر $94 = 8 + 60 + 24 + 2$ است.

تست ۲: با ۲۵ عنصر چند درخت دودویی با ارتفاع کمینه می‌توان ساخت؟

$$(۱) \quad \binom{۱۶}{۱۰} + \binom{۱۶}{۶} + ۵۶$$

$$(۲) \quad \binom{۱۶}{۱۰} + ۸ \binom{۱۴}{۳} + \binom{۸}{۲}$$

$$(۳) \quad \binom{۱۶}{۱۰} + \binom{۸}{۲} \binom{۱۴}{۳} + \binom{۸}{۲}$$

پاسخ: با ۲۵ عنصر، ارتفاع کمینه $\lceil \lg ۲۵ \rceil = ۴$ یعنی تعداد سطوح برابر ۵ است. چنین درختی حالات مختلف دارد:

حالت ۱: سطح ابتدایی پر باشند که ۱۵ نود مصرف می‌شود و ۱۰ نود باقی‌مانده را به

$$\binom{۱۶}{۶} = \binom{۱۶}{۱۰} \text{ حالت در سطح پنجم قرار دهیم.}$$

حالت ۲: سه سطح ابتدایی پر باشند و در سطح چهارم تعداد ۷ نود قرار دهیم که $\binom{۸}{۱} = ۸$

حالت دارد و ۱۱ نود باقی‌مانده را به $\binom{۱۴}{۳} = \binom{۱۴}{۱۱}$ حالت در سطح پنجم قرار دهیم.

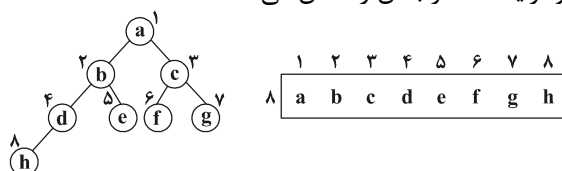
حالت ۳: سه سطح ابتدایی پر باشند و در سطح چهارم تعداد ۶ نود قرار دهیم که $\binom{۸}{۲}$ حالت

دارد و سپس ۱۲ نود باقی‌مانده را به $\binom{۱۲}{۱۲} = ۱$ حالت در سطح پنجم قرار دهیم.

بنابراین کل حالات برابر گزینه ۳ است.

پیاده‌سازی (نمایش) درخت دودویی

برای پیاده‌سازی درخت دودویی یعنی نمایش آن در کامپیوتر، دو روش مطرح است: آرایه و لیست پیوندی. روش آرایه برای درخت دودویی کامل مناسب است و به این صورت است که از ریشه شروع کرده و به ترتیب سطحی از چپ به راست، نودها را در آرایه ذخیره کنیم. یعنی به ریشه شماره ۱ بدهیم به فرزندان ریشه به ترتیب شماره‌های ۲ و ۳ تخصیص دهیم. به نودهای سطح سوم به ترتیب شماره‌های ۴ و ۵ و ۶ و ۷ نسبت دهیم و به همین ترتیب. شکل ۴ درخت دودویی کامل با $n = ۸$ نود و آرایه متناظر با آن را نشان می‌دهد:



شکل ۴- درخت دودویی کامل و آرایه متناظر با آن

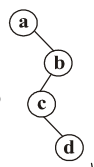
در آرایه، به راحتی می‌توان روابط پدر و فرزندی را بیان کرد.

پدر نود موجود در اندیس i ، $i \neq 1$ ، در اندیس $\lfloor \frac{i}{2} \rfloor$ قرار دارد. مثلاً در شکل ۴، پدر g ($i = 7$) در اندیس $3 = \lfloor \frac{7}{2} \rfloor$ قرار دارد. اگر $i = 1$ یعنی نود i ریشه است و پدر ندارد. فرزند چپ گره i در اندیس $2i$ قرار دارد، البته به شرطی که $2i \leq n$ که تعداد نودهای درخت است. فرزند راست گره i در اندیس $2i + 1$ قرار دارد، البته به شرطی که $2i + 1 \leq n$. در شکل ۴، فرزند چپ اندیس ۳ در اندیس ۶ و فرزند راست آن در اندیس ۷ می‌باشد:

$$\begin{aligned} \text{Parent}(i) &= \left\lfloor \frac{i}{2} \right\rfloor, & i \neq 1 \\ \text{Left}(i) &= 2i, & 2i \leq n \\ \text{right}(i) &= 2i + 1, & 2i + 1 \leq n \end{aligned}$$

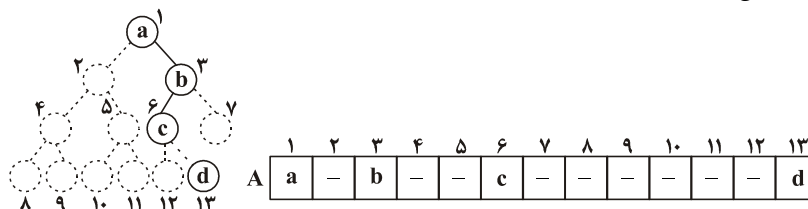
لازم به ذکر است که اگر درخت دودویی کامل را در آرایه $A[1..n]$ ذخیره کنیم آنگاه عناصر موجود در $A[1.. \lfloor \frac{n}{2} \rfloor]$ نودهای داخلی و عناصر موجود در $A[\lfloor \frac{n}{2} \rfloor + 1..n]$ برگ هستند. مثلاً در شکل ۴، که $n = 8$ می‌باشد، $A[1..4]$ داخلی و $A[5..8]$ برگ می‌باشند. با توجه به روابط پدر و فرزندی در آرایه، یافتن پدر و فرزندان گره موجود در اندیس i از مرتبه $\theta(1)$ است.

اگر درخت کامل نباشد و بخواهیم آن را با آرایه نمایش دهیم، ابتدا باید نودهای پوچی به درخت اضافه کنیم که کامل شود و سپس نمایش دهیم.



مثال ۱۸: برای نمایش درخت دودویی با آرایه، اندازه آرایه حداقل باید چند باشد؟

پاسخ: ابتدا درخت باید به درخت دودویی کامل تبدیل شود، که همانطور که مشاهده می‌شود، آرایه باید حداقل ۱۳ خانه داشته باشد.



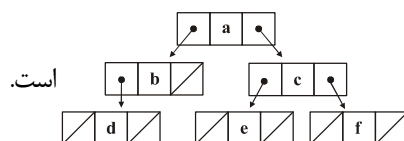
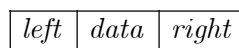
بنابراین روش آرایه برای درخت‌های دودویی غیر کامل، فضای هدر رفته دارد.

◀ **نکته:** برای نمایش درخت مورب چپ n نودی با آرایه، اندازه آرایه حداقل 2^{n-1} می‌باشد.

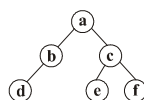
◀ **نکته:** برای نمایش درخت مورب راست n نودی با آرایه، اندازه آرایه حداقل $2^n - 1$ می‌باشد.

روش دیگر برای نمایش درخت دودویی، روش پیوندی است. در این روش هر نود رکورد یا ساختاری است با حداقل سه فیلد $left$ (اشاره‌گر به فرزند چپ)، $Right$ (اشاره‌گر به فرزند راست) و $data$ (داده موردنظر که مثلاً از نوع int است). مثلاً به زبان C ، تعریف نودهای درخت دودویی به شکل مقابل است:

```
struct node{
    node * left;
    int data;
    node * right;
};
```

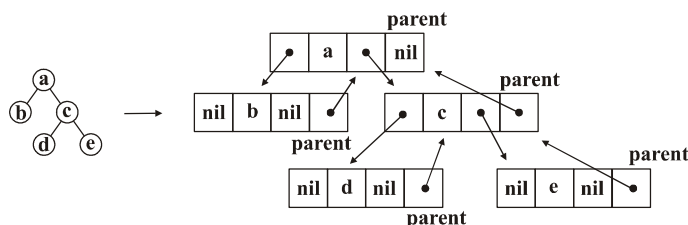


به عنوان مثال نمایش درخت به صورت



لینک‌های تهی با علامت « / » مشخص شده‌اند. در درخت دودویی n نودی با این روش تعداد $2n$ لینک وجود دارد (هر نود دارای دو لینک یا اشاره‌گر است) که از این تعداد، $n-1$ لینک غیرتهی است (زیرا به هر نود به جز ریشه، یک لینک اشاره می‌کند) پس تعداد لینک‌های تهی برابر $2n - (n-1) = n+1$ است. در درخت فوق که $n=6$ نود وجود دارد تعداد ۷ لینک تهی وجود دارد. همانطور که در روش آرایه دیدیم، مرتبه یافتن پدر و فرزندان یک نود، $\theta(1)$ است. در روش پیوندی، مرتبه یافتن فرزندان یک نود، $\theta(1)$ است. ولی مرتبه یافتن پدر یک نود، $O(n)$ است، زیرا یک نود آدرس پدر خود را ندارد و برای یافتن پدر یک نود، باید از ریشه درخت (همیشه آدرس ریشه درخت را داریم)، نودها را بررسی کنیم. اگر بخواهیم پدر هر نود را با مرتبه $\theta(1)$ به دست آوریم، باید به هر نود لینکی اضافه کنیم که به پدر آن نود اشاره کند.

در شکل مقابل، لینک $parent$ ، اشاره‌گر به پدر نود است:



پیمایش درخت دودویی (binary tree traversal)

هدف از پیمایش، ملاقات همه نودهاست مثلاً می‌خواهیم داده‌ی موجود در همه نودها را با نظم خاصی چاپ کنیم. درخت دودویی دو نوع پیمایش دارد: پیمایش سطحی (*level order*) و پیمایش عمقی (*depth order*).

پیمایش سطحی ابتدا ریشه درخت را ملاقات می‌کند سپس سایر نودها را به صورت سطح به سطح و از چپ به راست، ملاقات می‌کند. مثلاً پیمایش سطحی درخت شکل ۴ به صورت (*abcdefgh*) (چپ به راست) می‌باشد.

این پیمایش از یک صف استفاده می‌کند. ابتدا ریشه را وارد صف می‌کند و سپس تا زمانی که صف خالی نشده است، عنصر سرصف را حذف می‌کند و ملاقات می‌کند و فرزندان آن را وارد صف می‌کند.

شبه کد این پیمایش به صورت مقابل است:

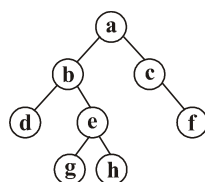
```
Levelorder (root)
{
  Q = empty queue
  addq (Q, root)
  while (not empty (Q))
  {
    node = delq (Q)
    visit (node)
    if Left (node) ≠ nil addq (Q, Left (node))
    if (Right (node) ≠ nil) addq (Q, right (node))
  }
}
```

در این کد، *root* اشاره‌گر به ریشه درخت است. *Q* یک صف است که ابتدا *root* را به آن اضافه می‌کنیم و در حلقه *while* تا وقتی صف *Q* خالی نشده است، عنصر سر صف حذف شده در *node* قرار می‌گیرد و ملاقات می‌شود و سپس فرزند چپ و راست *node* در صورت وجود به صف *Q* اضافه می‌شود. واضح است برای درخت *n* نودی، مرتبه این الگوریتم $\theta(n)$ است، زیرا هر نود دقیقاً یک بار به صف اضافه و یک بار حذف می‌شود و مرتبه *addq* و *delq* نیز $\theta(1)$ است.

پیمایش عمقی درخت دودویی، به ۶ صورت قابل انجام است. این نوع پیمایش‌ها، بازگشتی هستند و با پشته قابل پیاده‌سازی می‌باشند. اگر حرکت به سمت چپ و راست درخت را به ترتیب با *L* و *R* و ملاقات ریشه را با *D* (یا *N* یا *V*) نشان دهیم آنگاه ۶ پیمایش عمقی عبارتند از: *DRL* و *RDL*, *RLD*, *LRD*, *LDR*, *DLR*.

سه پیمایش اول یعنی DLR و LDR و LRD که حرف L قبل از R آمده است، معروف هستند و سه پیمایش دیگر، به ترتیب معکوس این سه پیمایش می‌باشند. DLR یا پیمایش پیش ترتیب ($preorder$)، ابتدا ریشه را ملاقات می‌کند سپس زیردرخت چپ ریشه و سپس زیردرخت راست ریشه را به صورت بازگشتی، پیمایش می‌کند. LDR یا پیمایش میان ترتیب ($inorder$)، ابتدا زیردرخت چپ را پیمایش می‌کند سپس ریشه را ملاقات می‌کند و در نهایت زیردرخت راست را پیمایش می‌کند. LRD یا پیمایش پس‌ترتیب ($postorder$) ابتدا زیردرخت چپ و سپس زیردرخت راست را پیمایش می‌کند و در نهایت ریشه را ملاقات می‌کند.

پیمایش RDL ($reverse preorder$) معکوس پیمایش DLR است. پیمایش RDL ($reverse inorder$) معکوس پیمایش LDR است. و پیمایش DRL ($reverse postorder$) معکوس پیمایش LRD است.



مثال ۱۹: پیمایش سطحی و عمقی درخت دودویی مقابل را انجام

دهید.

پاسخ: پیمایش سطحی این درخت عبارت است از:

$a\ b\ c\ d\ e\ f\ g\ h$

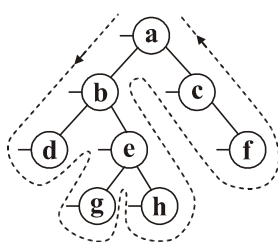
پیمایش پیش‌ترتیب (DLR): ابتدا ریشه ملاقات می‌شود، سپس زیردرخت چپ و در نهایت زیردرخت راست پیمایش می‌شوند، پس ابتدا a ملاقات می‌شود سپس زیردرخت چپ با همین شیوه پیمایش می‌شود. زیردرخت چپ، خود درخت دودویی است که ابتدا b ملاقات می‌شود و سپس زیردرخت چپ b یعنی d ملاقات می‌شود. حال نوبت زیردرخت راست b است که به صورت پیش‌ترتیب یعنی $g\ h\ e$ (چپ به راست) پیمایش می‌شود. حال نوبت زیردرخت راست a است که به صورت $c\ f$ (چپ به راست) پیمایش می‌شود. پس پیمایش پیش‌ترتیب به صورت $a\ b\ d\ e\ g\ h\ c\ f$ (چپ به راست) می‌باشد.

پیمایش میان‌ترتیب (LDR): ابتدا زیردرخت چپ پیمایش می‌شود سپس ریشه ملاقات می‌شود و در نهایت زیردرخت راست پیمایش می‌شود. زیردرخت چپ خود یک درخت دودویی است که باید با همین شیوه پیمایش شود، پس ابتدا d سپس b و سپس زیردرخت راست b به صورت میان‌ترتیب یعنی $g\ e\ h$ (چپ به راست) پیمایش می‌شود. حال ریشه a ملاقات می‌شود و در نهایت زیردرخت راست a به صورت میان‌ترتیب یعنی $c\ f$ (چپ به راست) پیمایش می‌شود. بنابراین پیمایش میان‌ترتیب $d\ b\ g\ e\ h\ a\ c\ f$ (چپ به راست) است.

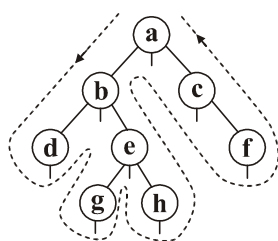
پیمایش پس ترتیب (LRD): ابتدا زیردرخت چپ سپس زیردرخت راست و در نهایت ریشه پیمایش می‌شود. زیردرخت چپ خود یک درخت دودویی است که با همین شیوه پیمایش می‌شود. پس ابتدا d سپس زیردرخت راست b به صورت پس ترتیب یعنی ghe (چپ به راست) پیمایش می‌شود و سپس b ملاقات می‌شود. اکنون که زیردرخت چپ a تمام شد، زیردرخت راست a باید به صورت پس ترتیب یعنی fc (چپ به راست) پیمایش شود و در نهایت ریشه a ملاقات می‌شود. پس پیمایش پس ترتیب این درخت $dghebfca$ (چپ به راست) است.

پیمایش معکوس پیش ترتیب (RLD): به صورت $f c h g e d b a$ (چپ به راست) می‌باشد. پیمایش معکوس میان ترتیب (RDL): به صورت $f c a h e g b d$ (چپ به راست) می‌باشد. پیمایش معکوس پس ترتیب (DRL): به صورت $a c f b e h g d$ (چپ به راست) می‌باشد.

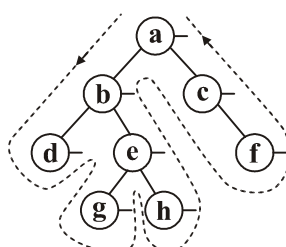
نکته: پیمایش‌های DLR و LDR و LRD را می‌توان به شیوه جالب‌تری نیز یافت.



برای یافتن پیمایش DLR ، همانند شکل مقابل، یک پاره‌خط سمت چپ هر نود رسم کنید و سپس از بالا بین نودهای درخت حرکت کنید و هر پاره‌خطی که در مسیر حرکت مشاهده کردید، نود نظیر آن را ملاقات کنید. پس ابتدا a سپس b بعد d و بعد e سپس g و سپس h و بعد c و در نهایت f ملاقات می‌شوند.



برای یافتن پیمایش LDR ، همانند شکل مقابل یک پاره‌خط زیر هر نود رسم کنید و سپس از بالا مطابق شکل بین نودها حرکت کنید و هر نودی که پاره‌خط رسم شده آن، مشاهده شد را چاپ کنید. بنابراین ترتیب ملاقات $d b g e h a c f$ (چپ به راست) است.



برای یافتن پیمایش LRD ، یک پاره‌خط سمت راست هر نود کشیده و سپس عمل قبل را انجام دهید. بنابراین ترتیب ملاقات مطابق شکل به صورت $dghebfca$ (چپ به راست) می‌باشد.

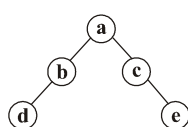
```

preorder(root)
{
  if (root ≠ nil)
  {
    ۱) visit(root)
    ۲) preorder(Left(root))
    ۳) preorder(Right(root))
  }
}

```

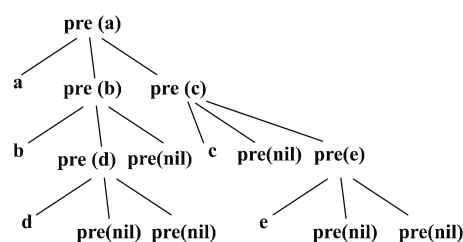
شبه کد پیمایش پیش‌ترتیب به صورت بازگشتی به شکل مقابل است:

اگر خطوط ۱ و ۲ و ۳ با هم جابه‌جا شوند، سایر پیمایش‌های عمقی حاصل می‌شود.



مثال ۲۰: با توجه به الگوریتم *preorder*، چه تعداد فراخوانی برای درخت مقابل انجام می‌شود؟

پاسخ: الگوریتم *preorder* را باید با ریشه درخت یعنی *a* فراخوانی کنیم. درخت فراخوانی‌ها به شکل زیر است (نام الگوریتم را *pre* گذاشته‌ایم).



باتوجه به درخت فراخوانی‌ها، خروجی *a b d c e* (چپ به راست) است و تعداد فراخوانی‌ها برابر ۱۱ تاست. به طور کلی پیمایش عمقی برای درخت شامل n نود، تعداد $2n + 1$ فراخوانی دارند.

نکات مربوط به پیمایش

۱- ترتیب ملاقات برگ‌ها در پیمایش‌های *DLR* و *LDR* و *LRD* یکسان و از چپ به راست است.

به همین ترتیب، ترتیب ملاقات برگ‌ها در پیمایش‌های *RLD* و *RDL* و *DRL* یکسان و از راست به چپ است.

تست ۳- اگر پیمایش *DLR* یک درخت دودویی *a e k c b a h g f d* (چپ به راست) باشد و نودهای *h*، *c*، *b*، *e* برگ باشند، کدام گزینه می‌تواند پیمایش *LRD* باشد؟ (چپ به راست)

۲) *d h g e a k c b f*

۱) *h g b d c k e a f*

۴) *h b g d e c k a f*

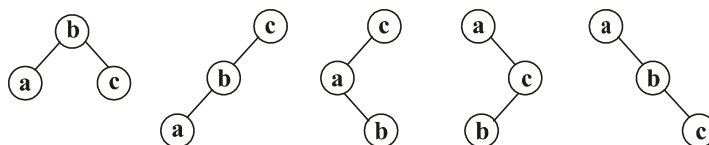
۳) *h g b d a c e k f*

پاسخ: گزینه (۴). ترتیب برگ‌ها در DLR به صورت $\langle h, b, e, c \rangle$ (چپ به راست) است و تنها گزینه‌ای که این ترتیب در آن رعایت شده است، گزینه ۴ می‌باشد. ■

۲- با داشتن یک پیمایش نمی‌توان درخت را به صورت منحصر به فرد مشخص کرد.

مثال ۱۱: چند تا درخت دودویی وجود دارد که پیمایش میان‌ترتیب آن به صورت a, b, c (چپ به راست) باشد؟

پاسخ: تعداد درخت‌های دودویی شامل ۳ نود از لحاظ شکلی (توپولوژی) برابر ۵ تا است و می‌توان برچسب‌های a, b, c را طوری در نودها قرار داد که پیمایش میان‌ترتیب تمام این ۵ درخت برابر a, b, c شود. پس جواب سوال ۵ می‌باشد:



نتیجه: با داشتن یک پیمایش شامل n برچسب، می‌توان تمام شکل‌های درخت دودویی را

کشید یعنی به تعداد عدد n ام کاتالان $c_n = \frac{1}{n+1} \binom{2n}{n}$ ، درخت دودویی قابل رسم است.

۳- با داشتن پیمایش‌های میان‌ترتیب و پیش‌ترتیب یا میان‌ترتیب و پس‌ترتیب یا میان‌ترتیب و سطح‌ترتیب، درخت دودویی منحصر به فرد است. (به طور مشابه می‌توان برای پیمایش‌های معکوس نیز چنین گفت).

تست ۴- پیمایش‌های DLR و LDR یک درخت دودویی به صورت مقابل هستند. پیمایش LRD این درخت کدام است؟

$DLR: f a e c k d g h b$

$LDR: e a k c f g d h b$

$e k c a g b h d f$ (۲)

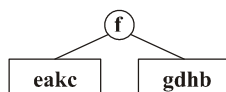
$e h g a c b k d f$ (۱)

$e c k a h b g d f$ (۴)

$h b g d e c k a f$ (۳)

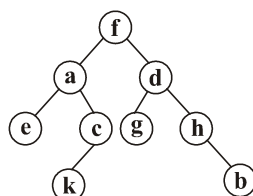
پاسخ: گزینه (۲). اولین عنصر در پیمایش پیش‌ترتیب، ریشه است. پس f ریشه درخت است.

حال با توجه به پیمایش میان‌ترتیب، $e a k c$ زیردرخت چپ و $g d h b$ زیردرخت راست هستند:



در زیردرخت چپ $(e a k c)$ ، اولین عنصر در پیمایش پیش‌ترتیب ریشه است، بنابراین a ریشه زیردرخت چپ است پس e سمت چپ a و $k c$ سمت راست a قرار می‌گیرد. در زیردرخت شامل $k c$ نیز ریشه است (c قبل از k در پیمایش پیش‌ترتیب آمده است) و k سمت چپ c قرار دارد.

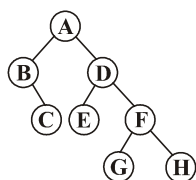
در بین عناصر $b h d g$ عنصر d ریشه است (در پیمایش پیش‌ترتیب، d اولین عنصر در بین این عناصر است) پس g سمت چپ d و $h b$ سمت راست d است. در بین $h b$ ، عنصر h ریشه و b سمت راست h است:



مثال ۲۲: اگر پیمایش‌های پس‌ترتیب و میان‌ترتیب یک درخت دودویی به شکل زیر باشد، درخت را مشخص کنید.

$$LRD : C B E G H F D A \quad , \quad LDR : B C A E D G F H$$

پاسخ: آخرین عنصر در پیمایش پس‌ترتیب، ریشه است پس A ریشه است. حال با توجه به پیمایش میان‌ترتیب، BC چپ A قرار دارد و $E D G F H$ راست A می‌باشد. حال با همین شیوه در زیردرخت چپ و راست با توجه به پیمایش پس‌ترتیب می‌توان ریشه را یافت. در واقع در هر زیردرخت، سمت راست‌ترین عضو در پیمایش پس‌ترتیب ریشه است. پس بین BC عنصر B ریشه است و بین $E D G F H$ عنصر D ریشه است و $G F H$ سمت راست D قرار دارد که F ریشه این سه عنصر است:



۴- با داشتن پیمایش‌های پیش‌ترتیب و پس‌ترتیب و با فرض وجود k گره تک‌فرزندی، تعداد ۲^k درخت دودویی متفاوت وجود دارد. زیرا تک‌فرزند چه چپ پدرش باشد و چه راست،

پیمایش‌های پیش‌ترتیب و پس‌ترتیب عوض نمی‌شوند. مثلاً هر دو درخت دودویی



دارای پیمایش پیش‌ترتیب ab و پیمایش پس‌ترتیب ba هستند.



مثال ۱۳: با داشتن پیمایش‌های $LRD: dijgf e b h c a$ و $DLR: a b d e f g i j c h$

چه تعداد درخت دودویی می‌توان رسم کرد؟

پاسخ: باید تعداد نودهای تک‌فرزندی را بیابیم. اگر در پیمایش پیش‌ترتیب $(...xy...)$ و در پیمایش پس‌ترتیب $(...yx...)$ مشاهده شد، به این معنی است که x تک‌فرزندی است و y فرزند x است.

پس با توجه به دو پیمایش داده شده e و f و c گره‌های تک‌فرزندی هستند. زیرا ef و fg و ch در پیمایش پیش‌ترتیب و fe و gf و hc در پیمایش پس‌ترتیب ظاهر شده‌اند. بنابراین ۳ تا گره تک‌فرزندی وجود دارد بنابراین $2^3 = 8$ درخت دودویی می‌توان کشید.

نتیجه: با داشتن پیمایش‌های پیش‌ترتیب و پس‌ترتیب اگر گره تک‌فرزندی وجود نداشته باشد آنگاه درخت دودویی منحصر به فرد است.

۵- با داشتن پیمایش پیش‌ترتیب یا پیمایش پس‌ترتیب، اگر گره تک‌فرزندی وجود نداشته باشد و برگ‌ها مشخص باشند آنگاه درخت منحصر به فرد است.

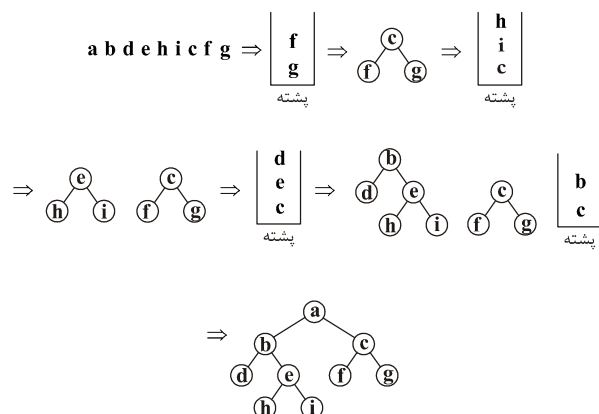
مثال ۱۴: پیمایش پیش‌ترتیب یک درخت دودویی که گره تک‌فرزندی ندارد به صورت زیر

است:

$preorder: a b d e h i c f g$

و گره‌های d و h و i و f و g برگ هستند. درخت را مشخص کنید.

پاسخ: از سمت راست عبارت را پیمایش می‌کنیم و گره‌های برگ را به یک پشته $push$ می‌کنیم و با رسیدن به گره غیربرگ، دو مقدار بالای پشته را pop کرده و آن‌ها را فرزندان گره غیربرگ قرار می‌دهیم و گره غیربرگ را به پشته $push$ می‌کنیم:




 $DLR = LDR = abc$
 مورب راست
 
 $LRD = LDR = cba$
 مورب چپ

نمایش عبارت ریاضی با درخت دودویی

می‌توان با یک درخت دودویی نمایش داد. مثلاً عبارت $a + b$ را به شکل



نمایش

می‌دهیم. یعنی عملگر را ریشه قرار می‌دهیم و عملوندها را به ترتیب سمت چپ و راست ریشه قرار می‌دهیم. برای تشکیل درخت دودویی یک عبارت ریاضی، ابتدا عبارت را اولویت‌بندی کنید و سپس عملگر با کمترین اولویت را ریشه درخت قرار دهید و هر چه سمت چپ آن عملگر باشد را در زیردرخت چپ و هر چه سمت راست آن عملگر باشد را در زیر درخت راست قرار دهید و سپس با زیردرخت چپ و راست، همین عمل را انجام دهید.

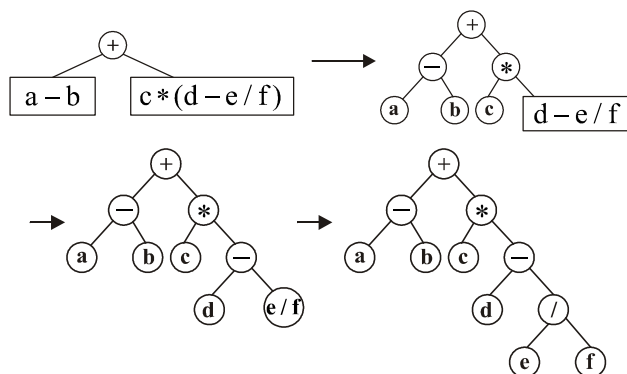
با توجه به درخت حاصل، فرم پیشوندی و فرم پسوندی عبارت را بیابید.

```

graph TD
    5((5)) --- 4((4))
    5 --- 3((3))
    4 --- a[a]
    4 --- 2((2))
    2 --- minus[-]
    2 --- 1((1))
    1 --- b[b]
    3 --- plus[+]
    3 --- 6((6))
    6 --- c[c]
    6 --- 7((7))
    7 --- LP("(")
    7 --- 8((8))
    7 --- RP(")")
    8 --- d[d]
    8 --- 9((9))
    9 --- minus2[-]
    9 --- 10((10))
    10 --- e[e]
    10 --- 11((11))
    11 --- slash[/]
    11 --- 12((12))
    12 --- f[f]
  
```

پرانتر بیشترین اولویت را دارند و می‌دانیم عملگرهای هم اولویت، از چپ به راست بررسی می‌شوند (البته به جز عملگرهای یکانی و به جز توان) بنابراین اولویت عملگرها به شکل مقابل است:

عملگر + دارای کمترین اولویت است پس ریشه درخت است. در زیر درخت چپ، عبارت $a - b$ و در زیر درخت راست، عبارت $c * (d - e / f)$ قرار می‌گیرد و به همین ترتیب، درخت حاصل می‌شود:



اگر درخت عبارت را به صورت پیش‌ترتیب پیمایش کنید، فرم پیشوندی عبارت حاصل می‌شود و اگر درخت را به صورت پس‌ترتیب پیمایش کنید، فرم پسوندی عبارت به دست می‌آید. بنابراین با توجه به درخت فوق داریم:

$preorder = prefix = + - ab * c - d / ef$

$postorder = postfix = ab - cdef / - * +$

توجه: درخت دودویی عبارتی که عملگر یکانی (تک عملوندی) دارد، دارای گره تک‌فرزندی

است مثلاً درخت عبارت $A + (-B)$ به شکل می‌باشد.

تست ۵- درخت دودویی یک عبارت ریاضی شامل ۳۰ نود است، کدام گزینه راجع به این عبارت صحیح است؟

- (۱) این عبارت شامل تعدادی زوج عملگر دوتایی است.
- (۲) این عبارت شامل تعداد فرد عملگر دوتایی است.
- (۳) این عبارت نمی‌تواند عملگر یکانی داشته باشد.
- (۴) این عبارت حداقل یک عملگر یکانی دارد.

پاسخ: گزینه (۴). با توجه به اینکه تعداد گره‌ها زوج است پس حتماً گره تک‌فرزندی در درخت وجود دارد پس حتماً حداقل یک عملگر یکانی در عبارت هست. توجه کنید اگر گره تک‌فرزندی در درخت وجود نداشته باشد آنگاه تعداد گره‌ها عددی فرد است. زیرا اگر تعداد گره‌های ۲ فرزندگی برابر x باشد آنگاه تعداد برگ‌ها برابر $x + 1$ است پس مجموع تعداد گره‌های ۲ فرزندگی و برگ‌ها برابر $2x + 1$ است که عددی فرد است.

برخی الگوریتم‌های مهم درخت دودویی

۱- شمارش تعداد برگ‌ها

```

function count(t:tree): integer;
begin
    if t=nil then count:=0
    else if t^.Lchild=nil and t^.rchild=nil then count:=1
    else
        count:=count (t^.Lchild)+count (t^.rchild);
    end;

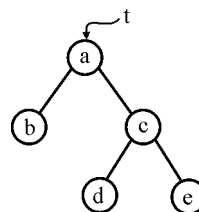
```

اگر این تابع با ریشه درخت فراخوانی شود، آنگاه این تابع سراغ تک‌تک گره‌ها می‌رود، و اگر گره‌ای نه فرزند چپ و نه فرزند راست داشته باشد (برگ) یک واحد به تابع اضافه می‌شود. به ازای سایر گره‌ها، فراخوانی مجدد انجام می‌شود و فقط به ازای برگ‌ها، یک واحد اضافه می‌شود:

$$\text{count}(a) = \text{count}(b) + \text{count}(c) = 1 + 2 = 3$$

$$\text{count}(b) = 1$$

$$\text{count}(c) = \text{count}(d) + \text{count}(e) = 1 + 1 = 2$$

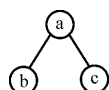


۲- شمارش تمام گره‌ها

```

function count(t: tree): integer;
begin
    if t= nil then count:= 0;
    if t ≠ nil then
        if rchild(t)=nil and Lchild (t)=nil then
            count := 1
        else
            count := count (Rchild(t)) + count(Lchild(t)) + 1;
        end;
    end;

```



$$\rightarrow \text{count}(a) = \text{count}(b) + \text{count}(c) + 1 = 1 + 1 + 1 = 3$$

۳- محاسبه تعداد سطوح درخت

```

function count(t: tree): integer;
begin
    if t=nil then count:=0
    else
        count:=1+max(count(Lchild(t)) , count(rchild(t)));
    end;

```

۴- الگوریتم کپی کردن یک درخت دودویی

```

Tree * Copy (Tree * cn)
{ Tree * temp ;
  if (cn != Null)
  {
      Temp = new (Tree);
      temp → data = cn → data;
      temp → lchild = Copy (cn → lchild);
      temp → rchild = Copy (cn → rchild);
      return temp;
  }
  else
      return null;
}

```

۵- الگوریتم برابری دو درخت دودویی

```

bool equal (Tree * a , Tree * b)
{
    if (a == Null && b == Null)
        return true ;
    if (a != Null && b != Null && (a → data == b → data) &&
        equal(a → lchild , b → lchild) && equal(a → rchild , b → rchild))
        return true;
    return false;
}

```

۶- الگوریتم معکوس کردن یک درخت دودویی

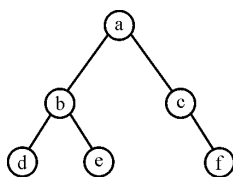
این الگوریتم جای زیر درختان چپ و راست دو گره در درخت را جابجا می‌کند.

```
Tree * reverse (Tree * cn)
{
    Tree * temp;
    if (cn != Null)
    {
        Temp = new (Tree);
        temp → lchild = reverse (cn → rchild);
        temp → rchild = reverse (cn → lchild);
        temp → data = cn → data;
        return temp ;
    }
    return Null;
}
```

درخت نخ‌ی دودویی Threaded binary Tree

اگر به نحوه نمایش درخت دودویی با n گره توجه کنید (با استفاده از لیست پیوندی) ملاحظه می‌کنید که از $2n$ اشاره‌گر موجود در درخت دودویی، تعداد $n+1$ اشاره‌گر، تهی (Nil یا $NULL$) می‌باشد. در درخت نخ‌کشی شده، این اشاره‌گرهای تهی را به عناصر قبلی یا بعدی گره‌ها در یک پیمایش خاص اشاره می‌دهند تا بدین ترتیب آن پیمایش سریعتر انجام شود.

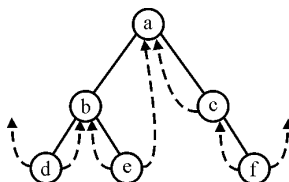
به عنوان مثال برای نخ‌کشی *Inorder* درخت مقابل:



ابتدا باید پیمایش *Inorder* را بدست آوریم سپس اشاره‌گر تهی چپ هر گره را به گره قبلی و اشاره‌گر تهی راست هر گره را به گره بعدی آن، اشاره دهیم:

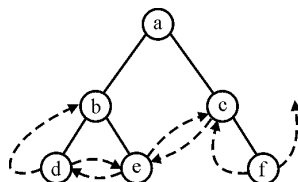
Inorder : d b e a c f

در نتیجه درخت نخ‌ی به صورت زیر است:

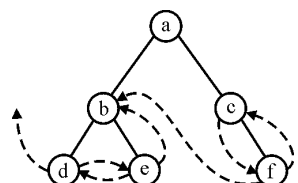


مثال ۱۶: درخت قبل را به صورت *preorder* و *postorder* نخ‌کشی کنید:

Preorder : *abdecf* $\Rightarrow$



Postorder : *debfca* $\Rightarrow$



توجه: نخ‌کشی *Inorder* رایج‌تر از نخ‌کشی‌های دیگر است.

در درخت نخ‌ی برای اینکه اشاره‌گرهای عادی و نخ‌ی از یکدیگر مجزا باشند، ساختار گره‌ها در درخت به صورت زیر تعریف می‌شود:

```
type Ttree = ^node;
node = record
    Lthread: Boolean;
    Lchild: Ttree;
    Data: items;
    Rchild: Ttree;
    Rthread: Boolean;
end;
```

<i>Lthread</i>	<i>Lchild</i>	<i>Data</i>	<i>Rchild</i>	<i>Rthread</i>

اگر برای گره‌ای مقدار *Lthread*=*true* باشد آنگاه اشاره‌گر *Lchild* نخ‌ی است و اگر *Lthread*=*False* یعنی اشاره‌گر *Lchild*، معمولی است. به همین ترتیب می‌توان در مورد *Rchild* و *Rthread* صحبت کرد.

پیمایش *inorder* درخت نخ‌ی دودویی

تابع *insucc*، عنصر بعد از گره مفروض *t* را در پیمایش *inorder* یک درخت نخ‌ی دودویی می‌دهد. در این تابع، بررسی می‌شود که اگر برای گره *t*، *Rthread*(*t*)=*true* آنگاه *Rchild*(*t*)

همان گره بعد از t است. ولی اگر $Rthread(t)=False$ ، گره بعدی t با پایین رفتن روی مسیر فرزندان چپ $Rchild(t)$ تا وقتی که به گره‌ای با وضعیت $Lthread=true$ برسیم تعیین می‌گردد. یعنی اگر $Rthread(t)$ برابر $true$ نباشد آنگاه گره بعدی t ، سمت چپ‌ترین گره در زیر درخت راست t است:

```
function insucc(t: Ttree): Ttree;
Var
    temp: Ttree;
begin
    temp:=Rchild(t);
    if (Rthread(t) <> TRUE) then
        while (Lthread (temp) <> TRUE) do
            temp:=Lchild(temp);
        insucc:=temp;
    end;
```

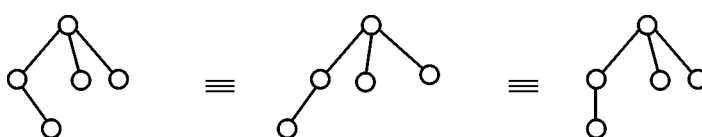
حال برای پیمایش *inorder* درخت نخی می‌توانیم با فراخوانی‌های مکرر *insucc* تمام گره‌ها را بازیابی کنیم که به عنوان تمرین بررسی کنید.

لازم به ذکر است که برای پیمایش *inorder* درخت نخی، کافی است فقط لینک‌های تهی راست نخ‌کشی شوند (بجز لینک راست آخرین نود) در این صورت پیمایش *inorder* را می‌توان به شکل زیر نوشت: (p اشاره‌گر به ریشه درخت است)

```
while (P ≠ NIL)
{
    while (Lchild(p) ≠ NIL)
        P = Lchild (p)
    write (data (p))
    while (Rchild (p) is threaded)
    {
        P = Rchild (p)
        write (data (p))
    }
    P=Rchild (p)
}
```

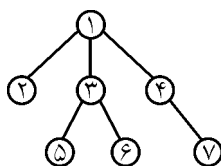
درخت عمومی (general tree)

درختی که درجه آن بیش از ۲ باشد، درخت عمومی (یا درخت) است. درخت عمومی دو تفاوت با درخت دودویی دارد: ۱- درخت دودویی می‌تواند تهی باشد، ولی درخت عمومی تهی نیست و حداقل یک گره دارد. ۲- در درخت عمومی فرزند چپ و راست مفهومی ندارد، پس درخت‌های زیر با هم معادلند:



نمایش درخت عمومی

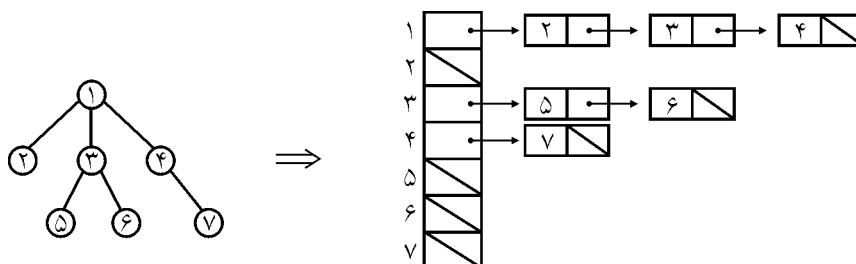
۱- آرایه: در این روش به تعداد گره‌های درخت، آرایه عنصر دارد و محتویات هر عنصر آرایه، معرف پدر آن گره است:



۱	۲	۳	۴	۵	۶	۷
۰	۱	۱	۱	۳	۳	۴

این روش کاربرد ندارد.

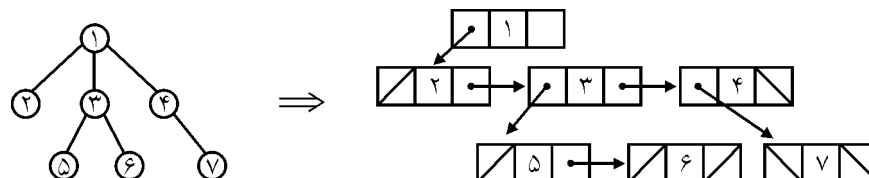
۲- استفاده از لیست فرزندان: آرایه‌ای به تعداد گره‌های درخت در نظر گرفته می‌شود که هر عنصر آن معرف یکی از عناصر درخت است. محتویات هر عنصر آرایه، اشاره‌گری به ابتدای یک لیست پیوندی است که عناصر آن، لیست فرزندان آن گره هستند:



این نحوه نمایش امکان دسترسی به فرزندان یک گره را به راحتی میسر می‌سازد. ولی دسترسی به پدر یک گره نیازمند اعمال الگوریتم زمان‌بری می‌باشد.

۳- ساختار مشابه روش پیوندی درخت دودویی (فرزند چپ - برادر راست *left child - right sibling*):

از همان ساختار کلی یک گره در درخت دودویی استفاده می‌شود با این تفاوت که از فرزند راست برای گره *Sible* (همزاد) استفاده می‌شود، اشاره‌گر سمت چپ هر گره به اولین فرزند (از چپ) اشاره می‌کند:

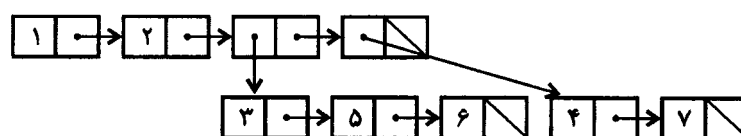


همانطور که ملاحظه می‌شود اشاره‌گر چپ (*Lchild*) هر گره به چپ‌ترین فرزند آن اشاره می‌کند و اگر گره‌ای فرزند نداشته باشد (برگ)، اشاره‌گر چپ آن تهی است. اشاره‌گر راست هر گره نیز به همزاد راست آن اشاره می‌کند و اگر گره‌ای همزاد راست نداشته باشد اشاره‌گر راست آن گره تهی است.

۴- ساختاری مشابه لیست عمومی: درخت فوق را می‌توان به فرم پرانتزی نشان داد:

$1(2, 3(5, 6), 4(7))$

و این معادل لیست عمومی زیر است:



۵- اگر درخت درجه k باشد، نودها را به صورت

<i>data</i>	<i>child1</i>	<i>child2</i>	...	<i>childk</i>
-------------	---------------	---------------	-----	---------------

تعریف می‌کنیم که *child 1* به اولین فرزند و ... و *childk* به فرزند k ام اشاره کند. روش مناسبی نیست و می‌توان نشان داد که از nk لینک موجود، $1 + n(k-1)$ لینک تهی است.

تبدیل درخت عمومی به درخت دودویی (الگوریتم فرزند چپ برادر راست)

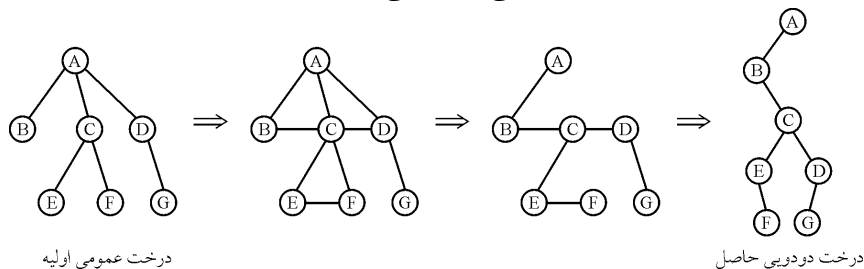
به دلیل اهمیت درخت دودویی، درخت عمومی را با الگوریتم زیر به درخت دودویی تبدیل می‌کنیم: (با توجه به روش ۳ نمایش درخت)

۱- ابتدا کلیه برادرها را به هم متصل می‌کنیم.

۲- ارتباط کلیه فرزندان به پدر را به جز اتصال سمت چپ‌ترین فرزند، قطع کنید.

۳- گره‌های متصل به هم در هر سطح افقی را ۴۵ درجه در جهت حرکت عقربه‌های ساعت بچرخانید.

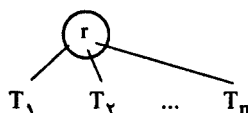
مثال ۲۷: مراحل تبدیل درخت عمومی به دودویی:



پیمایش‌های درخت عمومی

همانند درخت دودویی، پیمایش‌های پیش‌ترتیب، میان‌ترتیب و پس‌ترتیب برای درخت

عمومی نیز تعریف می‌شود:



پیمایش پیش‌ترتیب: ابتدا ریشه را ملاقات می‌کنیم سپس زیردرخت‌ها را از چپ به راست

پیمایش می‌کنیم: $r, T_1, T_2, \dots, T_n$

پیمایش پس‌ترتیب: ابتدا زیردرخت T_1 سپس زیردرخت T_2 و ... در نهایت ریشه درخت

ملاقات می‌شود. $T_1, T_2, \dots, T_n, r$

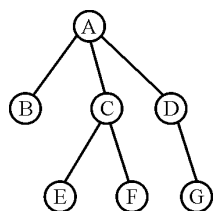
پیمایش میان‌ترتیب: ابتدا زیردرخت T_1 ، سپس ریشه و سپس زیردرخت T_2 و ... و T_n

ملاقات می‌شوند: $T_1, r, T_2, \dots, T_n$

توجه: در بعضی کتاب‌ها، پیمایش میان‌ترتیب، برای درخت عمومی به شکل گفته شده تعریف

نمی‌شود و ابتدا درخت عمومی، به دودویی تبدیل می‌شود و سپس پیمایش میان‌ترتیب روی

درخت دودویی حاصل، اجرا می‌شود.



مثال ۲۸: پیمایش‌های گفته شده را روی درخت مقابل اجرا کنید.

Preorder: A, B, C, E, F, D, G

postorder: B, E, F, C, G, D, A

Inorder: B, A, E, C, F, G, D (بدون تبدیل به دودویی)

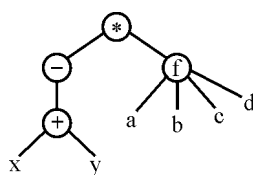
B, E, F, C, G, D, A (با تبدیل به درخت دودویی)

نکته: پیمایش *preorder* درخت عمومی با پیمایش *preorder* درخت دودویی معادل آن

یکسان است.

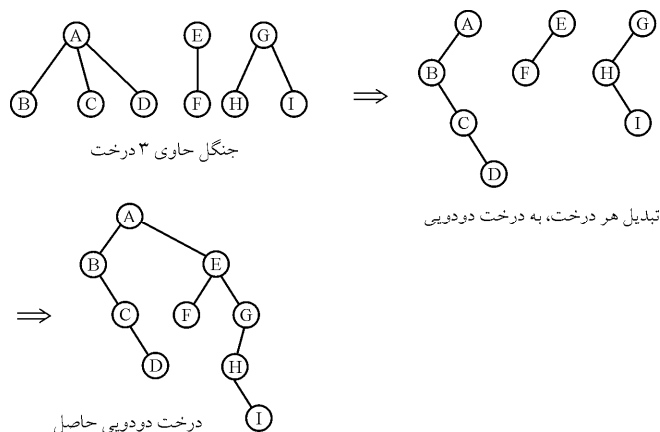
نکته: پیمایش $postorder$ درخت عمومی با پیمایش $inorder$ درخت دودویی معادل آن یکسان است.

نکته: یکی از کاربردهای درخت عمومی، نمایش عباراتی است که عملگرهای آن می‌توانند هر تعداد عملوند داشته باشند. مثلاً رجوع به تابع مثل $f(a,b,c,d)$ به صورت عملگری در نظر گرفته می‌شود به نام f که عملوندهای آن d و c و b و a هستند. مثلاً درخت عمومی معادل عبارت ریاضی: $f(a,b,c,d) * -(x+y)$ به صورت مقابل است:



پیمایش $preorder$ و $postorder$ درخت عمومی فوق، به ترتیب فرم $prefix$ و $postfix$ عبارت ریاضی را می‌دهد.

جنگل: جنگل مجموعه‌ای از $n \geq 0$ درخت مجزاست. اگر ریشه یک درخت حذف شود، جنگل حاصل می‌شود. یکی از مسائل مهم در جنگل، تبدیل جنگل به یک درخت دودویی است. برای این منظور، ابتدا تک‌تک درخت‌ها را به درخت دودویی تبدیل می‌کنیم (طبق الگوریتم فرزند چپ - برادر راست) سپس از چپ به راست، ریشه هر درخت را به عنوان فرزند راست ریشه درخت سمت چپی در نظر می‌گیریم به عنوان مثال:



نکته: برای پیمایش جنگل، درخت‌های آن را از چپ به راست پیمایش می‌کنیم.

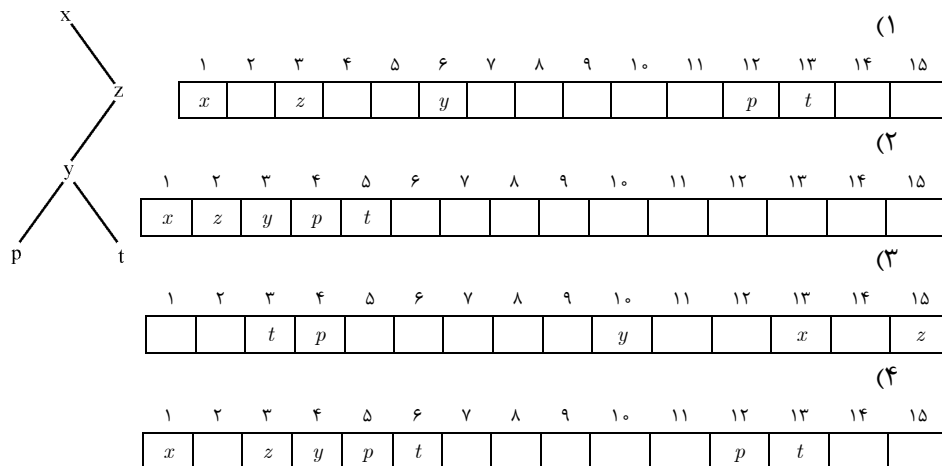
نکته: پیمایش $preorder$ یک جنگل همان پیمایش $preorder$ درخت دودویی متناظر آن می‌باشد.

نکته: پیمایش $postorder$ جنگل، با پیمایش $Inorder$ درخت دودویی حاصل، یکسان است.

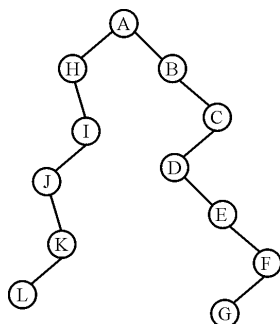
[سؤال‌های طبقه‌بندی شده فصل پنجم]

- ۱- حداکثر تعداد گره‌های یک درخت دودویی با ۴ سطح کدام است؟ (آزاد ۷۱)
- (۱) ۱۵ گره (۲) ۱۶ گره (۳) ۸ گره (۴) ۷ گره
- ۲- ماکزیمم تعداد $NODE$ ها در یک درخت دودویی با ارتفاع h برابر است با: (آزاد ۷۲)
- (۱) 2^{h+1} (۲) $2^{h+1} + 1$ (۳) $2^{h+1} - 1$ (۴) $2^h + 1$
- ۳- یک درخت دوتایی کامل به ارتفاع ۷ می‌بایست حداقل دارای این تعداد $NODE$ باشد: (آزاد ۷۳)
- (۱) ۶۴ (۲) ۶۵ (۳) ۳۳ (۴) هیچکدام
- ۴- درخت T با n راس مفروض است. اگر این درخت دارای ۵ راس ($node$) از درجه چهار ($degree$) و شش راس از درجه سه و پنج راس از درجه دو باشد. تعداد برگ‌های آن چقدر است؟ (دولتی ۷۱)
- (۱) $n-1$ (۲) $2n-2$ (۳) ۱۸ (۴) هیچکدام
- ۵- با ۵ گره چند درخت دودویی متفاوت وجود دارد؟ (علوم کامپیوتر ۸۱)
- (۱) ۴۲ (۲) ۱۵ (۳) ۷۰ (۴) ۲۸
- ۶- تعداد درخت‌های دودویی با n عنصر به ارتفاع $n-1$ برابر است با: (علوم کامپیوتر ۸۲)
- (۱) ۱ (۲) ۲ (۳) 2^{n-1} (۴) 2^n
- ۷- کدام گزینه نادرست است؟
- (۱) تنها یک درخت دودویی کامل با n گره می‌توان رسم کرد.
- (۲) ارتفاع درخت کامل دودویی که n برگ دارد برابر است با $\lceil \log_2 n \rceil$
- (۳) اگر یک درخت کلی (n برگ داشته باشد) تعداد کل گره‌های آن $2n-1$ است.
- (۴) اگر یک درخت دودویی پر n گره غیربرگ داشته باشد، تعداد کل گره‌های آن $2n+1$ است.
- ۸- اگر در یک درخت با حداکثر درجه ۲، تعداد کل گره‌ها ۱۷ و تعداد گره‌ها با درجه ۲، ۶ باشد، تعداد گره‌ها با درجه ۱ برابر است با: (آزاد ۸۲)
- (۱) ۲ (۲) ۳ (۳) ۴ (۴) ۵
- ۹- در یک درخت T با n عنصر، همه عناصر غیربرگ دارای دقیقاً ۲ فرزند هستند. $E(T)$ و $I(T)$ را به ترتیب مجموع عمق برگ‌ها و مجموع عمق عناصر غیربرگ تعریف می‌کنیم. اگر $T(n) = E(T) - I(T)$ باشد، داریم: (دولتی ۸۲)
- (۱) $T(n) = T(n-2) + 2$ (۲) $T(n) = T(n-1) + 1$
- (۳) $T(n) = T(n-1) + n - 1$ (۴) $T(n) = T(n-2) + n - 2$

۱۰- پیاده‌سازی درخت زیر با آرایه به کدام صورت صحیح است؟



۱۱- پیمایش LVR درخت دودویی مقابل کدام است؟



AHIJKLBCDEFG (۱)

HJLKIABDEGFC (۲)

HJLKIADCECFG (۳)

LKJIHAGFEDCB (۴)

۱۲- در تست ۱۱ پیمایش Preorder کدام است؟

AHIJKLBCDEGF (۲)

AHIJKLBCDEFG (۱)

LKJIHAGFEDCB (۴)

LHJKIADCECFG (۳)

۱۳- در تست ۱۱ پیمایش Postorder کدام است؟

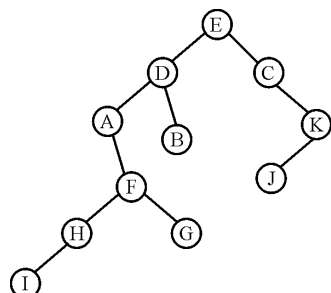
LKJIHGFEDCBA (۲)

HIJKLBCDEFGA (۱)

LKJIHAGFEDCB (۴)

LHJKIADCECFG (۳)

۱۴- کدام گزینه پیمایش RLN (زیر درخت راست، زیر درخت چپ و سپس ریشه) درخت زیر را نشان می‌دهد؟



KJCBGIHFADE (۱)

KJCBDFGHIAE (۲)

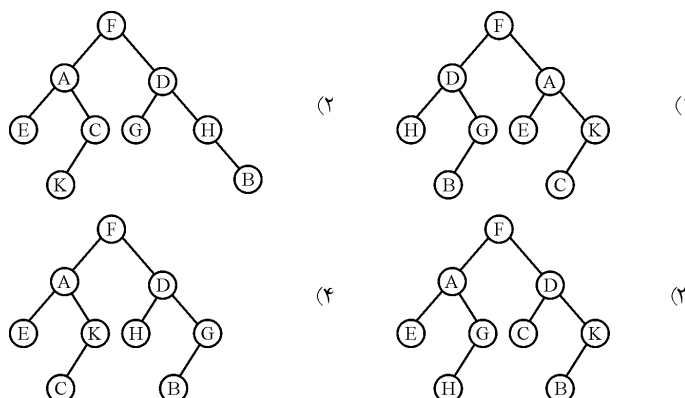
JKCBGIHFADE (۳)

JKCGIHFBADE (۴)

۱۵- در کدام پیمایش می‌توان با استفاده از دستور *Dispose* به جای دستور *Writeln*، تمام گره‌های یک درخت دودویی را حذف کرد؟

LRV (۱) LVR (۲) VLR (۳) هر سه (۴)

۱۶- درخت دودویی که پیمایش‌های *LVR* و *VLR* آن به صورت زیر است کدام است؟
LVR : EACKFHDBG *VLR : FAEKCDHGB*



۱۷- پیمایش یک درخت دودویی به شکل *Preorder* و *inorder* داده شده است. پیمایش *Postorder* آن کدام است؟ (دوای ۸۰)

Preorder: GBQACPDER

Inorder: QBCAGPEDR

CERQADBPG (۲) QCABERDPG (۱)
 RDEPGABQ (۴) GBPQADCER (۳)

۱۸- پیمایش پس‌ترتیب یک درخت دودویی به صورت *DEBFCA* می‌باشد کدامیک از گزینه‌های زیر، پیمایش پیش‌ترتیبی را نشان می‌دهد؟ (آزاد ۷۱)

DBEACF (۱) DABCEF (۲) ABDECF (۳) ACEDBF (۴)

۱۹- روش پیمایش درخت دودویی زیر را که به صورت بازگشتی تعریف می‌شود در نظر بگیرید:
 ۱- ریشه را ملاقات می‌کنیم.
 ۲- درخت فرعی سمت راست را پیمایش می‌کنیم.
 ۳- درخت فرعی سمت چپ را پیمایش می‌کنیم.

بین گره‌های ملاقات شده توسط روش فوق‌الذکر و *Preorder* و *Inorder* و *Postorder* چه رابطه‌ای وجود دارد؟ (دوای ۷۱)

- (۱) گره‌های ملاقات شده معکوس گره‌های ملاقات شده توسط *Preorder* است.
 (۲) گره‌های ملاقات شده معکوس گره‌های ملاقات شده توسط *Inorder* است.
 (۳) گره‌های ملاقات شده معکوس گره‌های ملاقات شده توسط *Postorder* است.
 (۴) هیچ ربطی بین گره‌های ملاقات شده و گره‌های ملاقات شده توسط روش‌های فوق نیست.

۲۰- لیست مربوط به گره‌های درخت دودویی t را در نظر بگیرید. $root$ (ریشه درخت)، $left$ (ریشه چپ) و $right$ (ریشه راست) است. پیمایش *inorder* درخت t عبارت است از:

	INFO	left	right	
root → ۱	A	۲	۳	DBGEHACOF (۱)
۲	B	۴	۵	ABDEGHCF (۲)
۳	C	°	۶	DBEGHACFO (۳)
۴	D	°	°	OFCADGHEB (۴)
۵	E	۷	۸	
۶	F	۹	°	
۷	G	°	°	
۸	H	°	°	
۹	O	°	°	

۲۱- اگر *acdfbeg* پیمایش *Preorder* یک درخت دودویی باشد، کدام یک از دنباله‌های زیر نمی‌تواند پیمایش *inorder* آن درخت باشد؟

(علوم کامپیوتر ۸۱)

- (۱) cdbfage (۲) cabfged (۳) fdecbag (۴) fdbcage

۲۲- عمق درخت دودویی معادل با عبارت محاسباتی $(-a) * b * c - d / e * g + h$ برابر است با:

(علوم کامپیوتر ۷۹)

- (۱) ۴ (۲) ۵ (۳) ۶ (۴) ۷

۲۳- تعداد گره‌های یک درخت دودویی که یک عبارت ریاضی را نمایش می‌دهد ۱۴ می‌باشد. عملگرهای این عبارت، دودویی یا یکتایی (*unary*) می‌باشند. کدام یک از گزینه‌های زیر صحیح است؟

(دولتی ۷۹)

- (۱) این عبارت حتماً تعداد فردی عملگر دودویی دارد.
 (۲) این عبارت حتماً تعداد زوجی عملگر یکتایی دارد.
 (۳) این عبارت نمی‌تواند عملگر دودویی داشته باشد.
 (۴) حداقل یک عملگر یکتایی در این عبارت وجود دارد.

۲۴- در یک درخت باینری دلخواه پیچیدگی زمان سه پیمایش $Preorder$ و $Postorder$ و

$Inorder$ به ترتیب برابر است با: (علوم کامپیوتر ۸۲)

$$(۱) \quad O(n), O(\log n), O(n) \quad (۲) \quad O(n^2), O(n), O(n^2)$$

$$(۳) \quad O(n), O(n), O(n) \quad (۴) \quad O(n), O(\log n), O(n^2)$$

۲۵- روش‌های پیمایش پیش‌ترتیب، میان‌ترتیب و پس‌ترتیب را در درخت‌های دودویی در نظر

بگیرید. کدام گزاره صحیح است؟ (علوم کامپیوتر ۸۲)

(۱) ترتیب ملاقات برگ‌ها در روش‌های پیش‌ترتیب و میان‌ترتیب یکسان است ولی در پیش

ترتیب و پس‌ترتیب متفاوت می‌باشند.

(۲) تنها حالت‌های خاصی ممکن است برگ‌های درخت در سه روش به یک ترتیب ملاقات شوند.

(۳) برگ‌های درخت با ترتیب‌های متفاوتی ملاقات می‌شوند.

(۴) برگ‌های درخت در هر سه روش همیشه به یک ترتیب ملاقات می‌شوند.

۲۶- پیش‌ترتیب ($preorder$) یک درخت دودویی کامل با برجسب‌های متفاوت داده شده

است. همچنین برجسب‌های برگ‌های آن درخت مشخص شده‌اند. می‌خواهیم درخت را

بسازیم (درخت دودویی کامل یک درخت دودویی است که در آن گره‌ها یا بدون فرزند

باشند یا دارای دو فرزند باشند) (دولتی ۷۵)

(۱) تنها در صورتی که درخت متوازن ($Balanced$) هم باشد آن را می‌توان ساخت.

(۲) درخت را می‌توان ساخت ولی حاصل واحد نیست.

(۳) اطلاعات فوق کافی نیست لذا درخت را نمی‌توان ایجاد کرد.

(۴) درخت را می‌توان ساخت و حاصل درخت واحد است.

۲۷- تابع زیر برای درخت دودویی T چه عملی را انجام می‌دهد؟ (دولتی ۷۵ – آزاد ۷۷)

$function \ n(T: tree): integer;$

$begin$

$if \ T = nil \ then \ n := 0;$

$if \ T \neq nil \ then$

$if \ Rchild(t)=nil \ and \ Lchild(t)=nil \ then$

$n:=1$

$else \ n := n(Rchild(t)) + n(Lchild(t)) + 1;$

$end;$

(۱) تعداد برگ‌های T را می‌شمارد.

(۲) تعداد گره‌های T را می‌شمارد.

(۳) تعداد برگ‌های دو فرزندی T را می‌شمارد.

(۴) تعداد گره‌های غیربرگ را می‌شمارد.

۲۸- رویه زیر را برای یک درخت دودوئی در نظر بگیرید: (دوئل ۷۳)

```
function Count(tree: pointer): integer;
begin
    if tree = nil then Count := ۰
    else if tree^.left = nil and tree^.right = nil then Count := ۱
    else
        Count := Count(tree^.left) + Count(tree^.right)
    end;
end;
```

این رویه:

- ۱) تعداد گره‌های یک درخت دودوئی را محاسبه می‌کند.
- ۲) تعداد برگ‌های یک درخت دودوئی را محاسبه می‌کند.
- ۳) تعداد گره‌هایی که دارای دو فرزند می‌باشند را محاسبه می‌کند.
- ۴) تعداد گره‌هایی که دارای یک فرزند می‌باشند را محاسبه می‌کند.

۲۹- اگر T یک درخت دودوئی باشد، روال زیر آن را به کدام روش پیمایش می‌کند؟ (دوئل ۷۴)

<i>Procedure Traversal(T);</i>	<i>Preorder</i> (۱)
<i>begin</i>	<i>Inorder</i> (۲)
<i>push(S , T);</i>	<i>Postorder</i> (۳)
<i>while (Stack S is not empty) do</i>	<i>Levelorder</i> (۴)
<i>begin</i>	
<i>T:=pop(S)</i>	
<i>while (T ≠ nil) do</i>	
<i>begin</i>	
<i>push (S, rightChild(T));</i>	
<i>visit(T);</i>	
<i>T:=leftchild(T)</i>	
<i>end</i>	
<i>end</i>	
<i>end;</i>	

۳۰- رویه زیر برای درخت دودویی T تعریف شده است؟ (دوای ۷۴)

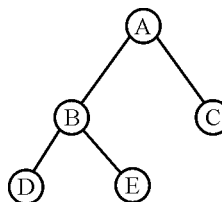
Procedure Traversal (T); این رویه:

```
begin
  while  $T \neq \varphi$  do
    begin
      Traversal(LChild(T));
      VISIT(T);
       $T := rchild(T)$ 
    end
  end;
```

- (۱) پیمایش پیش ترتیب را روی درخت T انجام می‌دهد.
- (۲) پیمایش بین ترتیب را روی درخت T انجام می‌دهد.
- (۳) پیمایش پس ترتیب را روی درخت T انجام می‌دهد.
- (۴) هیچکدام

۳۱- رویه زیر را در نظر بگیرید:

```
Procedure xtraversal(T)
begin
  if  $T \neq \circ$  then
    begin
      print (data(T));
      xtraversal (rchild(T));
      print (Data(T));
      xtraversal (Lchild(T));
    end
  end;
```



(آاد ۷۹) اگر این رویه بر روی درخت فوق اجرا شود چه خروجی تولید می‌کند؟

- | | |
|----------------|----------------|
| DCAACEBBED (۲) | ACACEBEBDD (۱) |
| ACCABEEBDD (۴) | ACABCEBDBD (۳) |

۳۲- کار تابع h بر روی یک درخت دودویی چیست؟

Procedure h(root: Treeptr; Var i:integer);

Var

x,y:integer;

begin

if root=nil then

i:=0

else begin

h(root^.left, x);

h(root^.right, y);

if x>y then

i := x + ۱

else

i:=y+1

end

end;

(۱) شمارش تعداد گره‌های درخت

(۲) شمارش تعداد گره‌های سطح آخر

(۳) شمارش تعداد سطوح درخت

(۴) شمارش تعداد شاخه‌های درخت

(آزاد ۷۷)

۳۳- الگوریتم زیر:

Procedure x(Left, right, root, count)

if Root=NULL then count=0

else

call x(Left,right,Left[Root], count_l)

call x(Left,right,Right[Root], count_r)

if count_l >= count_r then

count := count_l + ۱

else

count := count_r + ۱

endif

return count

end x

(۱) تعداد نودهای یک درخت را محاسبه می‌کند.

(۲) تعداد نودهای یک درخت را که بزرگتر از مقدار معین است به صورت بازگشتی محاسبه می‌کند.

(۳) عمق یک درخت باینری را محاسبه می‌کند.

(۴) هیچکدام

۳۴- تابع *count* برای درخت دودویی نوشته شده است. مقدار این تابع برای درخت دودویی زیر چیست؟ (آزاد ۸۰)

function count(T);

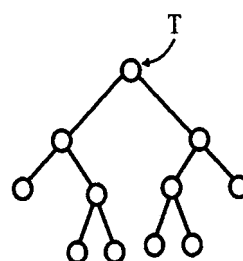
begin

if T=0 then count:=0;

if T ≠ 0 then

count := ۲ count(Left Child(right Child(T))) +
count(right Child(left Child(T))) + ۱

end;



۸ (۴)

۶ (۳)

۵ (۲)

۴ (۱)

۳۵- روال زیر کدام ویژگی از درخت دودویی *T* را محاسبه می‌کند؟ (دولتی ۷۸)

function test(T)

if T = Null then return ۰

return ۱ + max (test (T.LeftChild), test (T.RightChild))

(۲) تعداد زیردرخت‌ها

(۱) تعداد عناصر

(۴) ارتفاع

(۳) تعداد برگ‌ها

۳۶- یک درخت دودویی در سه آرایه موازی بنام‌های *info* و *left* و *right* پیاده‌سازی شده، به

طوری که *info* به اطلاعات هر گره و *right* و *left* اشاره‌گر به فرزندان راست و چپ آن

می‌باشند و *root* به ریشه درخت اشاره می‌کند. زیر برنامه بازگشتی زیر چه کاری روی

درخت فوق انجام می‌دهد؟

A(Left, right, root, S)

if root=null then S:=0 & return

call A(Left, right, Left[root], S_l)

call A(Left, right, right[root], S_r)

S := S_l + S_r + ۱

return

- (۱) تعداد برگ‌های درخت را پیدا می‌کند. (۲) عمق درخت را پیدا می‌کند.
 (۳) سطح درخت را پیدا می‌کند. (۴) تعداد گره‌های درخت را پیدا می‌کند.

۳۷- یک درخت دودویی (*2-ary tree*) درختی است که هر گره آن صفر یا ۲ فرزند دارد. اگر $n(h)$ و $N(h)$ به ترتیب حداقل و حداکثر تعداد گره‌های یک درخت دودویی به ارتفاع h باشد، برای $h > 0$ کدام یک از گزینه‌های زیر درست است؟

$$(۱) \quad n(h) = 2h + 1 \text{ و } N(h) = 2^{h+1} - 1$$

$$(۲) \quad n(h) = h + 1 \text{ و } N(h) = 2^{h-1}$$

$$(۳) \quad n(h) = h + 1 \text{ و } N(h) = 2^{h+1}$$

$$(۴) \quad n(h) = 2h + 1 \text{ و } N(h) = 2^h - 1$$

۳۸- اگر T یک درخت دودویی غیرتهی با n گره باشد و n_0 تعداد گره‌های نهایی (برگ) و n_1 تعداد گره‌هایی با درجه ۱ و n_2 تعداد گره‌های با درجه ۲ و K عمق درخت باشد، کدام یک از روابط زیر همیشه صحیح است؟ (آزاد ۸۱)

$$(الف) \quad K = \left\lceil \log_2 n \right\rceil + 1 \quad (ب) \quad n_0 = n_2 + 1 \quad (ج) \quad n = 2^K - 1$$

$$(۱) \quad \text{الف و ج} \quad (۲) \quad \text{ب} \quad (۳) \quad \text{الف و ب} \quad (۴) \quad \text{الف و ج و ب}$$

۳۹- الگوریتم زیر را در نظر بگیرید:

```
P(x) {
    if !x return (0)
    else return (1 + P(Left(x)) + P(right(x)))
}
```

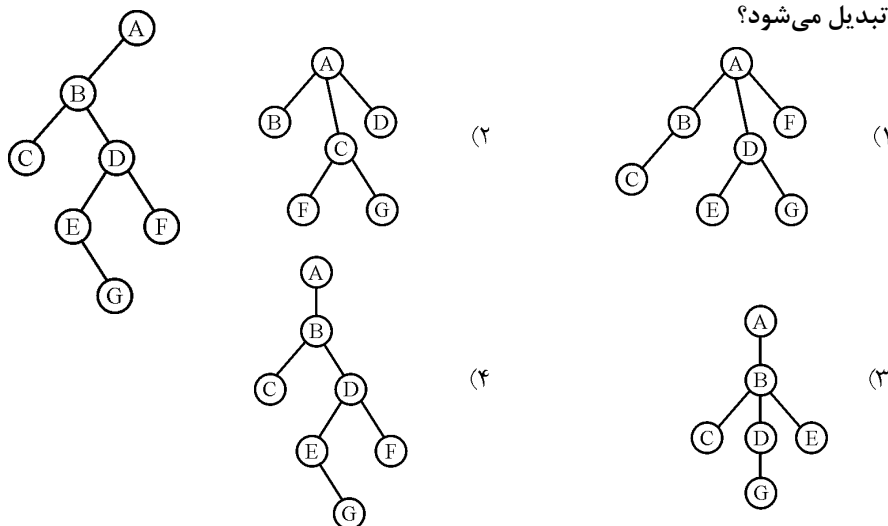
پیچیدگی الگوریتم برای درخت باینری x با n گره برابر است با: (علوم کامپیوتر ۸۶)

$$(۱) \quad O(n^2) \quad (۲) \quad O(\log n)$$

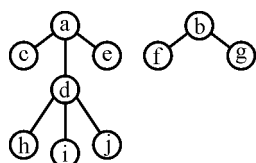
$$(۳) \quad O(n \log n) \quad (۴) \quad O(n)$$

۴۰- کدام درخت با استفاده از الگوریتم فرزندان چپ - برادر راست به درخت دودویی شکل روبرو

تبدیل می‌شود؟



۴۱- پیمایش *inorder* جنگل (غیر دودویی) زیر برابر است با (علوم کامپیوتر ۸۰)



- (۱) *cahdijefbg* (۲) *acdhijsbfg*
(۳) *chijdeafgb* (۴) *abcdefghijkl*

۴۲- کدام یک از جملات زیر در مورد درخت‌ها درست می‌باشد. (آزاد ۸۱ و ۸۲ و ۸۳)

- (۱) پیمایش *Preorder* جنگل و *Preorder* درخت دودویی متناظر با آن یکسان است.
(۲) پیمایش *Postorder* جنگل و *preorder* درخت دودویی متناظر با آن یکسان است.
(۳) پیمایش *inorder* جنگل و *inorder* درخت دودویی متناظر با آن یکسان است.
(۴) پیمایش به ترتیب سطح جنگل و *inorder* درخت دودویی متناظر با آن نتیجه یکسان ندارد.

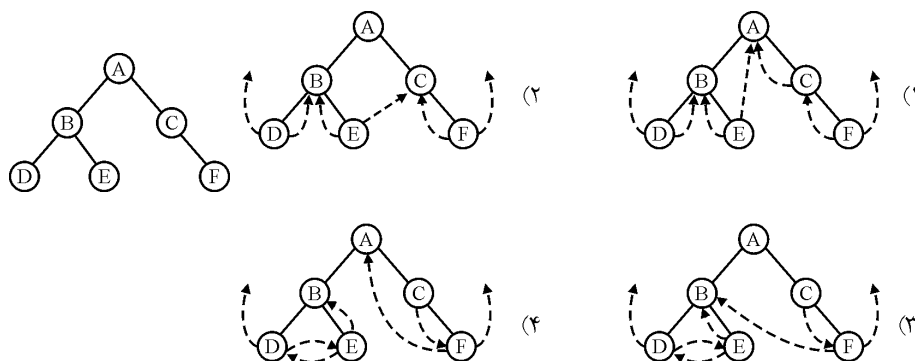
۴۳- حداکثر تعداد گره‌ها برای یک درخت از درجه ۳ که دارای ارتفاع h می‌باشد، چقدر است؟

(ارتفاع ریشه یک فرض شود)

(آزاد ۸۲)

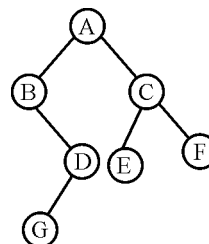
- (۱) $3^h - 1$ (۲) 3^h (۳) $\frac{3^h - 1}{2}$ (۴) $\frac{3^h}{2}$

۴۴- درخت زیر را در نظر بگیرید، می‌خواهیم این درخت را برای پیمایش *inorder* نخ‌ی (*threaded*) کنیم. گزینه صحیح را انتخاب کنید. (علوم کامپیوتر ۷۹)



۴۵- الگوریتم زیر را در نظر بگیرید:

```
Void traverse (ptr P) {
    stack S; ptr q;
    push (P, S);
    while (! Empty(S)) {
        q=pop (S);
        If (q != NULL) {push (q → left, S);
                        print(q → Label);
                        push(q → right, S);
                        }
    }
}
```



اگر این الگوریتم برای پیمایش درخت بالا استفاده شود، کدام یک از ترتیب‌های زیر بدست می‌آید؟ (علوم کامپیوتر ۸۳)

A B D G C E F (۲)

A C F E B D G (۱)

A B C D E F G (۴)

G D B E F C A (۳)

۴۶- چه تعداد درخت دودویی برچسب‌دار متفاوت با n گره و با برچسب‌های ۱ تا n که دارای ترتیب‌های یکسان در دو روش پس ترتیب و بین ترتیب می‌باشند، وجود دارند؟ (علوم کامپیوتر ۸۳)

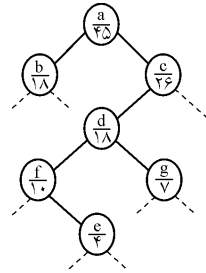
$n!$ (۴)

n (۳)

۱ (۲)

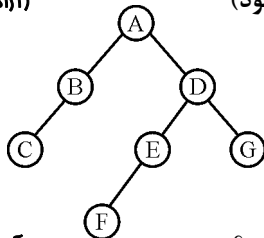
۰ (۱)

۴۷- در شکل زیر قسمتی از یک درخت دودویی نشان داده شده است. در زیر برجسب هر گره عددی نوشته شده است که تعداد کل گره‌های موجود در زیر درخت آن گره را نشان می‌دهد (با احتساب خود آن گره). اگر درخت مفروض را بطور *inorder* پیمایش کنیم f چندمین خروجی خواهد بود؟



- (۱) ۲۵ امین
- (۲) ۲۶ امین
- (۳) ۲۹ امین
- (۴) ۳۰ امین

۴۸- اگر از الگوریتم *Postorder* غیربازگشتی برای درخت زیر استفاده کنیم. ترتیب *pop* برای دو گره F و G عبارت است از: (شماره‌ها از یک شروع می‌شود)



- (۱) ۵ و ۴
- (۲) ۵ و ۳
- (۳) ۷ و ۵
- (۴) ۷ و ۴

۴۹- الگوریتم زیر به چه صورتی درخت دودویی را پیمایش می‌کند؟

(آزاد ۸۳)

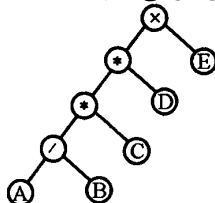
```

xorder(tree * root)
{
    tree * t=root ;
    queue *q;
    while (r!=Null)
    {
        visit (r -> data);
        if (r -> Lchild != Null)    q.Insert (r -> Lchild);
        if (r -> rchild != Null)    q.Insert (r -> rchild);
        r=q.delete( );
    }
}

```

- (۱) پیمایش به صورت عمقی
- (۲) پیمایش به صورت *inorder*
- (۳) پیمایش به صورت *Preorder*
- (۴) پیمایش به صورت سطحی

۵۰- اگر رویه *inorder* بازگشتی را برای درخت زیر اجرا کنیم، تعداد فراخوانی‌ها چند تا است؟



- (۱) ۹
- (۲) ۱۰
- (۳) ۱۸
- (۴) ۱۹

۵۱- دو پیمایش *preorder* و *postorder* از یک درخت دودویی با n گره در دسترس است.

کدام گزینه صحیح است؟ (دوگزینه‌ای)

(۱) برگ‌ها و گره‌های تک فرزندی و ریشه را می‌توان تعیین کرد و تعداد درخت‌های دودویی به تعداد کل گره‌های پیمایش شده بستگی دارد.

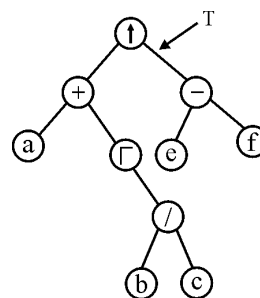
(۲) نمی‌توان درختی از روی این دو پیمایش ساخت و فقط ریشه را می‌توان تعیین کرد.

(۳) برگ‌ها و گره‌های تک فرزندی و ریشه را می‌توان تعیین کرد و تعداد درخت‌های دودویی که می‌توان ساخت به تعداد گره‌های تک فرزندی بستگی دارد.

(۴) فقط ریشه درخت را می‌توان تعیین کرد و نمی‌توان برگ‌ها و گره‌های تک فرزندی را تعیین کرد.

۵۲- درخت عبارت زیر داده شده است. ($\uparrow$ به معنی توان و Γ به معنی تغییر علامت است) فرض کنید

```
type TREE =  $\uparrow$  nodetype;
nodetype = record
    element: char;
    left, right: TREE;
end;
```



(دوگزینه‌ای)

procedure printtree (T: TREE);

begin

if T=nil then return;

if T $\uparrow$.left <> nil then write('(');

printtree(T $\uparrow$.left);

if T $\uparrow$.left <> nil then write(')');

write(T $\uparrow$.element);

if T $\uparrow$.right <> nil then write('(');

Printtree(T $\uparrow$.Right);

if T $\uparrow$.right <> nil then write(')');

end;

خروجی پردازش printtree چیست؟

(۱) $((a) + (\Gamma((b)/(c)))) \uparrow ((e) - (f))$

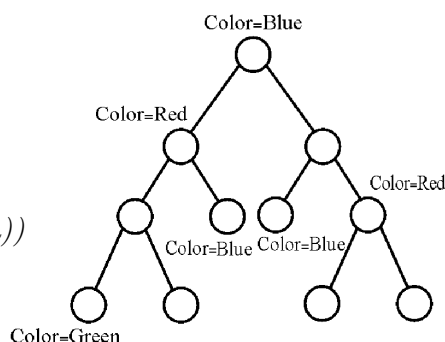
(۲) $((a) + (\Gamma((b)/(c)))) \uparrow ((e) - (f))$

(۳) $(a + (\Gamma(b/c))) \uparrow (e - f)$

(۴) $((a + \Gamma(b/c)) \uparrow (e - f))$

۵۳- در درخت زیر، برای تعدادی از گره‌ها مشخصه $Color$ تعریف شده است که رنگ آن گره‌ها را نشان می‌دهد. برای مشخص کردن رنگ یک گره n ، برنامه زیر را اجراء می‌کنیم: (دوای ۷۷)

```
Function FindColor(n): Color;
begin
  if n has Color attribute then
    FindColor := Color(n);
  else
    FindColor := FindColor (parent(n))
end;
```



اگر $FindColor(n)$ برای هر یک از ۱۱ گره‌های درخت فوق اجراء شود، در مجموع چه تعداد رنگ قرمز (Red) باز می‌گردد؟

۵ (۴) ۴ (۳) ۶ (۲) ۲ (۱)

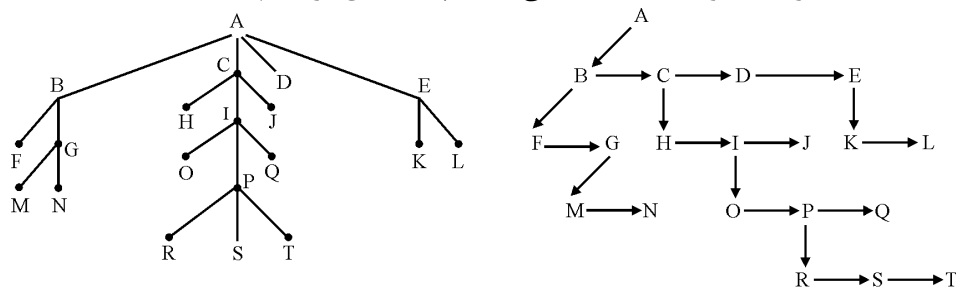
۵۴- یک عبارت E حاوی عملگرهای دودویی و یا یکتائی است و عملگرهای آن همگی یک حرفی ($Character$) هستند. برای یک عبارت E ، $T(E)$ درخت عبارت E ، $prefix(E)$ فرم پیشوندی، $postfix(E)$ فرم پسوندی و $Infix(E)$ فرم پرانتزی کامل عبارت E است. منظور از فرم پرانتزی کامل E ، عبارت میانوندی است که اولویت عملگرها توسط پرانتزها کاملاً مشخص شده باشد. (بدیهی است که $postfix(E)$ و $Prefix(E)$ و $T(E)$ حاوی پرانتز نیستند) کدام یک از گزینه‌های زیر درست است؟ (دوای ۷۴)

(۱) از $T(E)$ می‌توان $prefix(E)$ و $postfix(E)$ و $Infix(E)$ را به دست آورد.
(۲) از $prefix(E)$ می‌توان $T(E)$ را به دست آورد، به شرط آن که اولویت عملگرها را داشته باشیم.

(۳) از $Infix(E)$ می‌توان $postfix(E)$ و $prefix(E)$ را تولید کرد.

(۴) کلیه موارد فوق.

۵۵- درخت زیر را به صورت درخت دودویی معادل پیاده‌سازی کرده‌ایم: (دویتی ۷۳)



پیاده‌سازی فوق به صورت زیر است:

```
type nodeptr = ^ node;
node = record
    element: char;
    down, right: nodeptr;
end;
```

تابع زیر نوشته شده است: (کلا $return(z)$ اجرای $find$ را قطع می‌کند و حاصل آن z خواهد شد.)

```
function find (x, t: nodeptr): nodeptr;
var q, r: nodeptr;
begin
    q := t^.down;
    while q <> nil do
        if q = x then return(t)
        else begin
            r := find(x, q);
            if r <> nil then return(r)
            else q := q^.right
        end;
    return (nil)
end;
```

اگر y اشاره‌گری به المان P و t اشاره‌گری به ریشه این درخت باشد، حاصل تابع $find(y, t)$ چیست؟

(۲) اشاره‌گری به المان H
(۴) nil

(۱) اشاره‌گری به المان B
(۳) اشاره‌گری به المان I

۵۶- کدام یک از جملات زیر درست نیستند؟ (آزاد ۸۳)

- (۱) پیمایش به ترتیب سطح جنگل و درخت دودویی متناظر با آن نتیجه یکسان دارد.
- (۲) پیمایش *postorder* جنگل و *postorder* درخت دودویی متناظر با آن یکسان نیست.
- (۳) پیمایش *inorder* جنگل و *inorder* درخت دودویی متناظر با آن یکسان است.
- (۴) پیمایش *preorder* جنگل و *preorder* درخت دودویی متناظر با آن یکسان است.

۵۷- الگوریتم پیمایش درخت در زمان $O(d)$ اجرا می‌شود. d چه می‌باشد.

- (۱) تعداد برگ‌های درخت
- (۲) عمق درخت
- (۳) درجه درخت
- (۴) تعداد گره‌های درخت

۵۸- یک درخت دودویی کامل با ارتفاع h چند گره دارد؟ (دوئل ۸۴)

- (۱) 2^h
- (۲) 2^{h+1} گره
- (۳) بین 2^h و 2^{h+1} گره
- (۴) بین 2^{h-1} و 2^h گره

۵۹- در یک درخت باینری T با n گره در چه حالتی اولین گره در پیمایش *postorder* مشابه

آخرین گره در پیمایش *inorder* است؟ (علوم کامپیوتر ۸۵)

- (۱) زیر درخت چپ T خالی باشد.
- (۲) T دارای ارتفاع $n-1$ باشد.
- (۳) زیر درخت راست T خالی باشد.
- (۴) T حداکثر ۳ گره داشته باشد.

۶۰- فرض کنید در یک درخت، درجه تمام عناصر داخلی (*Internal*) k باشد، فرض کنید n

تعداد عناصر خارجی (*External*) باشد، در این صورت، کدام یک از گزاره‌های زیر، برقرار

است؟ (دوئل ۸۵)

- (۱) $n \bmod k = 1$
- (۲) $(n+1) \bmod k = 1$
- (۳) $n \bmod (k-1) = 1$
- (۴) $(n-1) \bmod (k-1) = 1$

۶۱- الگوریتم زیر بر روی یک گره r از درخت دودویی اجرا می‌شود.

Traverse (r , *inc*)

if r is not a leaf

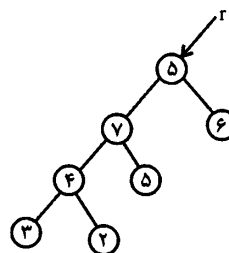
Then {*Traverse* (*left*(r), *inc* + ۱)

Traverse (*right*(r), *inc* + ۱)

If *key* (*left*(r)) > *key* (*right*(r))

Then *key*(r) $\leftarrow$ *key*(r) + *inc*

}



اگر *Traverse*(r , o) بر روی درخت با ریشه r و کلیدهای شکل فوق اجرا شود بیشترین

key عناصر درخت در انتها چه خواهد بود؟ (دوئل ۸۵)

- (۱) ۱۰
- (۲) ۹
- (۳) ۸
- (۴) ۷

۶۲- اگر عبارت $SBDHXEJKTFG$ نتیجه پیمایش پیشوندی یک درخت دودویی کامل باشد کدام یک از گزینه‌های زیر صحیح است؟ (دو نمره)

(۱) $HXDBJKEFGTS$ نتیجه نمایش پسوندی و $HDXBEJKSTFG$ نتیجه نمایش میانوندی همان درخت هستند.

(۲) $HXDJKEBFGTS$ نتیجه نمایش پسوندی و $HDXBEJKSTFG$ نتیجه نمایش میانوندی همان درخت هستند.

(۳) $HXDBJKEFGTS$ نتیجه نمایش پسوندی و $HDXBJEKSFTG$ نتیجه نمایش میانوندی همان درخت هستند.

(۴) $HXDJKEBFGTS$ نتیجه پیمایش پسوندی و $HDXBJEKSFTG$ نتیجه نمایش میانوندی همان درخت هستند.

۶۳- کدام یک از گزاره‌های زیر در مورد یک درخت ۵ تایی کامل با ۹۵ گره صحیح است؟ با فرض این که ریشه گره اول باشد و در هر عمق گره‌ها به ترتیب از چپ به راست در نظر گرفته شوند؟ (دو نمره)

- (۱) این درخت ۷۶ برگ دارد و گره پانزدهم آن پدر گره هفتاد و دوم آن است.
- (۲) این درخت ۷۶ برگ دارد و گره چهاردهم آن پدر گره هفتاد و دوم آن است.
- (۳) این درخت ۷۷ برگ دارد و گره چهاردهم آن پدر گره هفتاد و دوم آن است.
- (۴) این درخت ۷۷ برگ دارد و گره پانزدهم آن پدر گره هفتاد و دوم آن است.

۶۴- یک درخت عبارت E فقط با عملگرهای دودویی و گونه کاملاً پرانتزی آن را در نظر بگیرید. اگر درخت فقط یک برگ با برجسب a باشد، گونه کاملاً پرانتزی آن (a) و گره $(E_r E_r)$ است، که در این حالت r عملگر ریشه و E_r و E_r گونه‌های کاملاً پرانتزی زیر درخت‌های ناتمامی E_r و E_r است. مثلاً $((a)-((b)*(c)))/((d)/(e))$ گونه کاملاً پرانتزی عبارت $(a-b*c)/(d/e)$ است. گونه «میانوندی ساده شده» یک عبارت میانوندی کاملاً پرانتزی همان رشته است که از آن برجسب برگ‌ها و نیز همه " "ها حذف شده باشند. مثلاً $((+/(/(/($ گونه میانوندی ساده شده E است.

فقط با داشتن گونه میانوندی ساده شده یک عبارت ریاضی E با عملگرهای دودویی (خود E را نداریم)، کدام یک از گزینه‌های زیر صحیح است؟ (بهترین جواب را انتخاب کنید). (مهلدس ۸۷)

- (۱) درخت عبارت E را می‌توان با $O(n^2)$ و به صورت تک ساخت.
- (۲) درخت عبارت E را می‌توان با $O(n)$ و به صورت تک ساخت.
- (۳) درخت عبارت E را می‌توان با $O(n^2)$ ساخت، ولی جواب لزوماً تک نیست.
- (۴) درخت عبارت E را می‌توان با $O(n)$ ساخت، ولی جواب لزوماً تک نیست.

۶۵- در یک درخت n -ary هر گره حداکثر n فرزند می‌تواند داشته باشد. در درخت n -ary با k گره و ارتفاع h کدام یک از روابط زیر حد بالایی برای تعداد برگ‌های درخت می‌باشد؟
(مهندس ۸۷)

$$(1) h^n \quad (2) n^h \quad (3) \log_h^k \quad (4) \frac{k}{\log_h^k}$$

۶۶- یک درخت دودویی به نام T با n گره داریم. اگر اولین گره در پیمایش $Postorder$ درخت با آخرین گره از پیمایش $Preorder$ درخت یکسان باشد، کدام گزینه زیر را می‌توان نتیجه گرفت؟
(۸۷-IT)

- (۱) زیردرخت چپ T خالی است. (۲) درخت T حداکثر ۳ گره دارد.
(۳) زیردرخت راست T خالی است. (۴) ارتفاع درخت T برابر n است.

۶۷- تعداد درخت‌های دودویی که پیمایش‌های پیش‌ترتیب ($Preorder$) و پس‌ترتیب ($Postorder$) آنها برابر باشند، چقدر است؟
(۸۸-IT)

$Preorder: a b d e f g c h i j k l$

$Postorder: e d g f b i h k l j c a$

- (۱) ۱ درخت (۲) ۴ درخت (۳) ۸ درخت (۴) بی‌نهایت درخت

۶۸- می‌دانیم که در یک درخت دودویی، سطح (یا عمق) یک گره برابر طول مسیر از آن گره تا ریشه است. ارتفاع درخت هم بزرگ‌ترین سطح گره‌ها در آن درخت است. «پهنای» یک درخت دودویی T را برابر بیش‌ترین تعداد گره‌های هم سطح در T تعریف می‌کنیم. آیا درخت دودویی با n گره و ارتفاع و پهنای زیر وجود دارد؟

I . ارتفاع $\Theta(n)$ و پهنای ۱ II . ارتفاع $\Theta(\log n)$ و پهنای $\Theta(n)$

III . ارتفاع $\Theta(n)$ و پهنای $\Theta(n)$ IV . ارتفاع $\Theta(\log n)$ و پهنای $\Theta(\sqrt{n})$

جواب چند تا از موارد فوق درست است؟ (مهندی کامپیوتر ۸۹)

- (۱) ۲ (۲) ۱ (۳) ۳ (۴) ۴

۶۹- درخت‌های دودویی که $Preorder$ و $Postorder$ آنها در زیر ذکر شده است، چه تعداد است؟
(۸۹ IT)

$Pre: a b d e f g c h i j$

$Post: d g f e b i j h c a$ (۱) ۸ (۲) ۱

(۳) ۴ (۴) هیچ درختی را نمی‌توان پیدا کرد.

۷۰- در یک درخت دودویی خاص T گره‌های داخلی هر کدام یک عنصر دارند اما برگ b واقع در عمق d ، به جای یک گره، یک آرایه‌ی نامرتب A_b به طول حداکثر 2^d است (عمق ریشه صفر فرض می‌شود)

در بین همه‌ی درخت‌های با n عنصر، درختی با بیش‌ترین ارتفاع را در نظر بگیرید. اگر ارتفاع این درخت h باشد، چه رابطه‌ای برقرار است؟ (بهترین گزینه را انتخاب کنید)
(مهندسی کامپیوتر ۹۰)

$$n \leq h + 2^h \quad (۲) \qquad n \leq 2^{h+1} \quad (۱)$$

$$n \leq h + 2^{h+1} - 1 \quad (۴) \qquad n \leq 2^{h+1} - 1 \quad (۳)$$

۷۱- با n گره، چند درخت دودویی متمایز (از لحاظ توپولوژی) به ارتفاع n می‌توان ساخت؟
(ارتفاع یک درخت دودویی با یک گره را یک در نظر بگیرید).
(۹۰ IT)

$$2^{n-1} \quad (۱) \qquad 2 \times (n-1) \quad (۲) \qquad 2 \times n - 1 \quad (۳) \qquad 2^n - 1 \quad (۴)$$

۷۲- نمایش *Preorder* یک درخت دودویی به صورت (از چپ به راست) $XYZABFOD$ است. این درخت گره تک فرزندی ندارد. همچنین گره‌های $DOVBZ$ مجموعه برگ‌های درخت را تشکیل می‌دهند. نمایش *Post order* درخت کدام گزینه است؟
(۹۰ IT)

$$ZBVA YODFX \quad (۱) \qquad ZAYVODFBX \quad (۲)$$

$$ZAYOFVDBX \quad (۳) \qquad BVZAYOFDX \quad (۴)$$

[پاسخنامه سؤال‌های طبقه‌بندی شده فصل پنجم]

۱- گزینه (۱). $2^4 - 1 = 15$

۲- گزینه (۳). درخت دودویی ارتفاع h ، دارای $h + 1$ سطح است و حداکثر 2_{-1}^{h+1} نود دارد.

۳- گزینه (۴). درخت دودویی به ارتفاع ۷ دارای ۸ سطح است و به خاطر کامل بودن ۷ سطح اول پر است که $2^7 - 1 = 127$ نود دارد و اگر یک نود در سطح هشتم اضافه شود، درخت کامل با ۸ سطح و حداقل نود ایجاد می‌شود که ۱۲۸ نود دارد.

۴- گزینه (۴). اگر غیر از راس‌های گفته شده در سوال راس دیگری نباشد آنگاه:

$$n_o = 3n_e + 2n_p + n_r + 1$$

$$n_o = 3 \times 5 + 2 \times 6 + 5 + 1 = 33$$

۵- گزینه (۱).

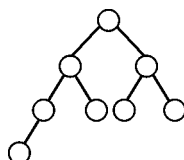
$$\frac{1}{6} \binom{10}{5} = 42$$

۶- گزینه (۳). ارتفاع $n - 1$ یعنی ماکزیمم ارتفاع؛ ریشه یک موقعیت دارد ولی $n - 1$ نود

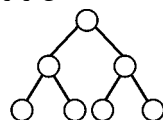
دیگر هر کدام ۲ حالت دارند (چپ، راست) پس 2^{n-1} درخت وجود دارد.

۷- گزینه (۲). با n گره فقط یک درخت کامل می‌توان رسم کرد (گزینه ۱)

در مورد گزینه ۲ به درخت‌های کامل زیر توجه کنید:



۳ = ارتفاع ، ۴ = تعداد برگ



۲ = ارتفاع ، ۴ = تعداد برگ

که گزینه ۲ نقض می‌شود.

گزینه ۳: درخت پر که n برگ دارد $n - 1$ نود داخلی دارد پس جمعاً $2n - 1$ نود دارد.

گزینه ۴: درخت پر که n گره داخلی دارد، $n + 1$ برگ دارد پس جمعاً $2n + 1$ نود دارد.

۸- گزینه (۳).

$$n_o = n_p + 1 = 7, \quad n = n_o + n_i + n_p \rightarrow n_i = 17 - (7 + 6) = 4$$

۹- گزینه (۱). اگر درخت را در نظر بگیرید (عمق ریشه صفر فرض می‌شود) در این صورت $E(T) = 1 + 1 = 2$ و $I(T) = 0$ بنابراین $T(3) = 2 - 0 = 2$. و می‌توان نشان داد که گزینه ۱ صحیح است.

نکته: به طور کلی در درخت دودویی که گره درجه ۱ ندارد.

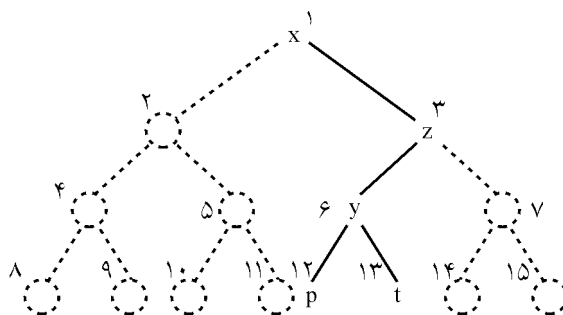
$$E(T) = I(T) + 2n_p$$

بنابراین با توجه به اینکه در این مسئله $n = n_o + n_p$ است و داریم $n_o = n_p + 1$

آنگاه: $n_p = \frac{n-1}{2}$ پس:

$$E(T) = I(T) + 2\left(\frac{n-1}{2}\right) \Rightarrow E(T) - I(T) = n - 1 \Rightarrow T(n) = n - 1$$

۱۰- گزینه (۱).



۱۱- گزینه (۲). منظور پیمایش *inorder* است.

۱۲- گزینه (۱).

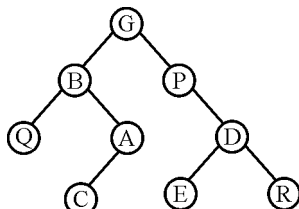
۱۳- گزینه (۲).

۱۴- گزینه (۳). *RLN* معکوس پیمایش *preorder* است.

۱۵- گزینه (۱). در پیمایش *postorder* ابتدا فراخوانی‌های بازگشتی کامل انجام می‌شود، پس از اتمام فراخوانی‌ها، دستور *write* برای تک‌تک گره‌ها انجام می‌شود، حال اگر بجای *write*، *dispose* قرار گیرد، همه گره‌ها حذف می‌شوند. در سایر فراخوانی‌ها این اتفاق نمی‌افتد.

۱۶- گزینه (۴). می‌توانید از الگوریتم گفته شده در درس استفاده کنید و یا اینکه گزینه‌ها را آزمایش کنید.

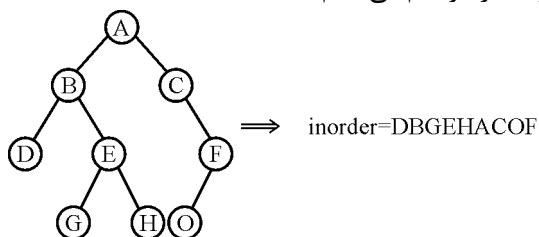
۱۷- گزینه (۱). با استفاده از الگوریتم گفته شده در درس، درخت به صورت زیر است:



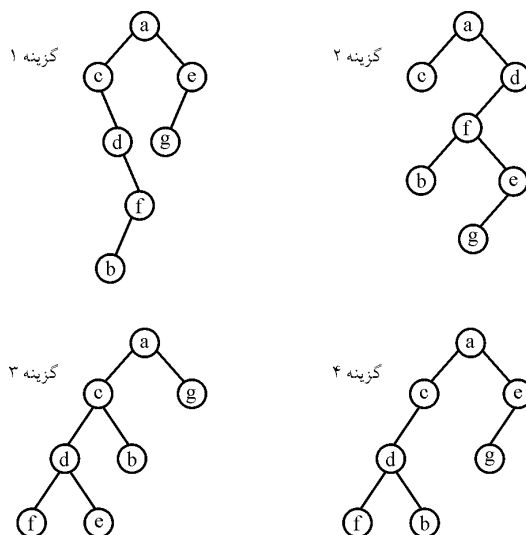
۱۸- گزینه (۳). با توجه به پیمایش پس ترتیب، A ریشه است و در پیمایش پیش ترتیب A باید اول باشد، پس گزینه‌های ۱ و ۲ نادرست هستند. با بررسی می‌توان نشان داد که گزینه ۴ نمی‌تواند درست باشد. در واقع درخت مورد نظر در این تست یک درخت کامل با ۶ گره است.

۱۹- گزینه (۳). پیمایش گفته شده VRL است که عکس LRV است.

۲۰- گزینه (۱). ابتدا درخت را ترسیم می‌کنیم.

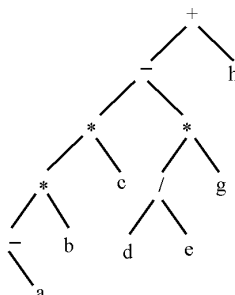


۲۱- گزینه (۳).



همانطور که ملاحظه می‌شود گزینه ۳ نادرست است.

۲۲- گزینه (۲). درخت ۶ سطح دارد و عمق آن (ارتفاع آن) ۵ است.



۲۳- گزینه (۴).

$$\left. \begin{aligned} n &= n_o + n_l + n_r = 14 \\ n_o &= n_r + 1 \end{aligned} \right\} \Rightarrow 2n_r + n_l = 13$$

با توجه به رابطه بدست آمده از آنجایی که سمت راست تساوی فرد است و $2n_r$ زوج است بنابراین n_l باید فرد باشد و نمی‌تواند صفر باشد.

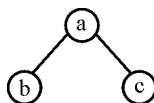
۲۴- گزینه (۳). زمان پیمایش بر روی ساختار درخت دودویی وابسته به تعداد گره‌هاست و چون هر گره دقیقاً یک بار ملاقات می‌شود، زمان $O(n)$ است.

۲۵- گزینه (۴). ترتیب ملاقات برگ‌ها در ۳ پیمایش گفته شده یکسان است.

۲۶- گزینه (۴). توجه کنید در این تست تعریف درخت کامل متفاوت با متن کتاب است. در این تست درخت کامل درختی است که نود تک فرزندی ندارد. الگوریتمی در متن کتاب برای رسم درخت گفته شده است.

۲۷- گزینه (۲). به ازای هر گره فراخوانی انجام می‌دهد و با هر فراخوانی، یک واحد به مقدار تابع اضافه می‌کند. مثلاً:

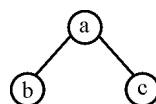
$$n(a) = \underbrace{n(b)}_1 + \underbrace{n(c)}_1 + 1 = 3$$



۲۸- گزینه (۲). به ازای هر گره، فراخوانی انجام می‌دهد و گره‌هایی که چپ و راست آنها تهی است را می‌شمارد.

۲۹- گزینه (۱). الگوریتم، درخت را به صورت *preorder* غیربازگشتی پیمایش می‌کند.

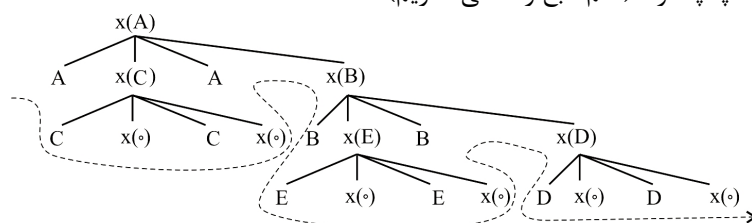
می‌توانید برای درخت الگوریتم را دنبال کنید که خروجی abc می‌شود.



و چون از پشته استفاده می‌کند *levelorder* نیست.

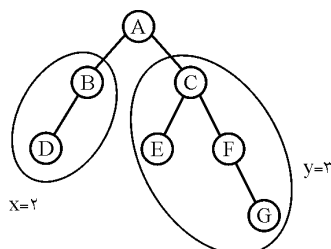
۳۰- گزینه (۲). الگوریتم، مسلماً نمی‌تواند *preorder* یا *postorder* باشد (به دلیل نحوه ملاقات گره‌های و حرکت اولیه به زیر درخت سمت چپ ملاقات ریشه پس از ملاقات زیر درخت چپ و قبل از ملاقات زیر درخت راست انجام می‌گیرد. هر چند ملاقات زیر درخت راست به صورت مشخص بازگشتی نیست ولی در این زیر درخت نیز بخش چپ آن به صورت بازگشتی ملاقات می‌شود. یعنی پس از ملاقات ریشه اشاره‌گر به عنصر سمت راست اشاره می‌کند و از این به بعد برنامه در درخت راست ادامه می‌یابد.

۳۱- گزینه (۴). می‌توان درخت فراخوانی‌های بازگشتی را کشید و سپس برگ‌ها را از چپ به راست چاپ کرد. (اسم تابع را x می‌گذاریم)



خروجی : ACCABEEBDD

۳۲- گزینه (۳). این الگوریتم تعداد سطوح سمت راست درخت را در y و سمت چپ را در x قرار می‌دهد. به شکل توجه کنید:



پس از محاسبه x و y ، هر کدام که بزرگتر بود به آن یک واحد اضافه می‌شود یعنی تعداد سطوح درخت محاسبه می‌شود.

۳۳- گزینه (۳). مشابه تست قبل

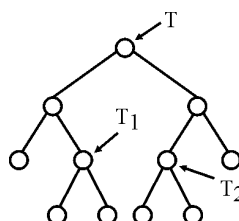
۳۴- گزینه (۱).

$$\text{count}(T) = 2\text{count}(T_l) + \text{count}(T_r) + 1$$

$$\text{count}(T_l) = 2\text{count}(\text{تهی}) + \text{count}(\text{تهی}) + 1 = 1$$

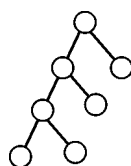
$$\text{count}(T_r) = 1 \text{ به همین ترتیب}$$

$$\Rightarrow \text{count}(T) = 2 \times 1 + 1 + 1 = 4$$



۳۵- گزینه (۴). در این تست تعداد سطوح را با ارتفاع یکسان فرض کرده ایم.

۳۶- گزینه (۴). جمله $s = s_l + s_r + 1$ تعداد گره‌های زیر درخت چپ و راست را با یک (ریشه) جمع می‌کند.



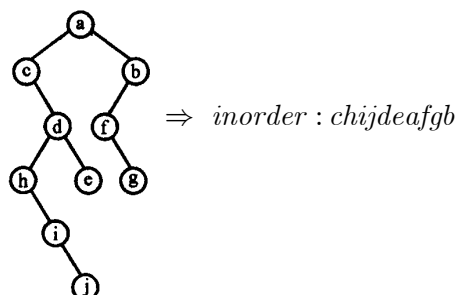
۳۷- گزینه (۱). حداکثر نود یعنی درخت پر، که درخت پر به ارتفاع h (شامل $h+1$ سطح) دارای $2^{h+1} - 1$ نود است. حداقل نود وقتی است که در هر سطح فقط ۲ نود قرار گیرد. (بجز اولین سطح که فقط یک نود است) پس حداقل تعداد نود برابر $2h + 1$ است مثلاً درخت مقابل درختی است با ارتفاع $h = 3$ و حداقل تعداد نود:

۳۸- گزینه (۲). در هر درخت دودویی تعداد گره‌های برگ از ۲ فرزند یک واحد بیشتر است. الف و ج لزوماً صحیح نیستند.

۳۹- گزینه (۴). این الگوریتم تمام گره‌های درخت را شمارش می‌کند یعنی زمان اجرای آن به تعداد گره‌ها وابسته است یعنی $O(n)$.

۴۰- گزینه (۱). طبق الگوریتم گفته شده در درس.

۴۱- گزینه (۳). برای پیمایش *inorder* ابتدا درخت دودویی معادل را می‌یابیم.



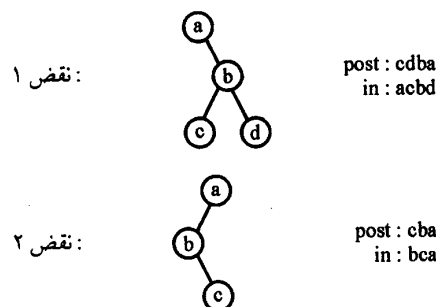
۴۲- گزینه (۱). پیمایش *pre* جنگل با *pre* درخت دودویی معادل یکسان است. پیمایش *post* جنگل با پیمایش *inorder* درخت دودویی معادل، یکسان است.

۴۳- گزینه (۳). حداکثر تعداد نود درخت درجه k با ارتفاع h برابر $\frac{k^h - 1}{k - 1}$ است (ارتفاع ریشه یک یعنی درخت دارای h سطح است)

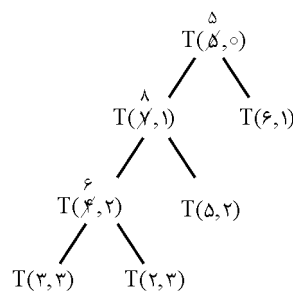
۴۴- گزینه (۱). درخت را به صورت *inorder* پیمایش می‌کنیم و سپس طبق روش گفته شده در درس عمل می‌کنیم.

- ۴۵- گزینه (۱). الگوریتم پیمایش VRL می‌باشد. ابتدا ریشه در پشته $push$ می‌شود سپس داخل حلقه، ریشه $pop(A)$ شده و در q قرار می‌گیرد. سپس زیر درخت چپ A یعنی B ، $push$ می‌شود و بعد A چاپ می‌شود و پس از آن زیر درخت راست A یعنی C ، $push$ می‌شود. حلقه مجدداً تکرار می‌شود این بار C از پشته pop شده، چپ آن (E) در پشته $push$ می‌شود و C چاپ می‌شود و F به پشته $push$ می‌شود و ...
- ۴۶- گزینه (۴). درخت مورب چپ است که عناصر داخل نودها، $n!$ جایگشت دارند.
- ۴۷- گزینه (۱). برای ملاقات f ابتدا باید زیر درخت سمت چپ a ملاقات شود که دارای ۱۸ گره است. سپس خود a ملاقات می‌شود و بعد از آن زیر درخت چپ f ملاقات می‌شود. تعداد گره‌های زیر درخت چپ f برابر ۵ تاست زیرا در سمت راست f ، ۴ گره وجود دارد و خود f نیز یک گره است، پس ۵ گره سمت چپ f است. پس از سمت چپ f خود f ملاقات می‌شود، پس: $۱۸+۱+۵=۲۴$ یعنی قبل از ملاقات f ، ۲۴ گره ملاقات می‌شوند یعنی f گره ۲۵ام است.
- ۴۸- گزینه (۲). توالی pop عناصر، به صورت $A D G E F B C$ (از چپ به راست) می‌باشد یعنی F در مکان ۳ و G در مکان ۵ قرار دارد. توالی pop در واقع همان ترتیب ملاقات کردن نودهاست.
- ۴۹- گزینه (۴). از صف استفاده می‌کند.
- ۵۰- گزینه (۴). پیمایش‌های عمقی درخت، به ازای هر نود و به ازای هر لینک تهی یک فراخوانی انجام می‌دهند. در این درخت ۹ نود و ۱۰ لینک تهی (تعداد لینک تهی از تعداد نود، یکی بیشتر است) وجود دارد پس ۱۹ فراخوانی انجام می‌شود.
- ۵۱- گزینه (۳). در پیمایش پیشوندی ریشه اولین گره و در پسوندی ریشه آخرین گره‌ای است که ملاقات می‌شود. پس از آن با در نظر گرفتن همین قاعده زیر درخت‌های چپ و راست و محل گره‌های دو فرزندی و فرزندان آنها قابل تشخیص می‌باشند. ولی در مورد گره‌های تک فرزندی، چپ یا راست بودن فرزند را نمی‌توان تشخیص داد بنابراین اگر n گره تک فرزندی داشته باشیم ۲^n درخت متمایز می‌توان ساخت. در ضمن برگ‌ها، تک فرزندی‌ها و ۲ فرزندی‌ها را می‌توان تشخیص داد.
- ۵۲- گزینه (۲). به صورت $inorder$ پیمایش می‌کند و دور عبارت حاصل از شاخه‌های چپ و راست بطور جداگانه پرانتز می‌گذارد. اما دور عبارت پیمایش شده اصلی پرانتز نمی‌گذارد پس گزینه ۱ درست نیست. از طرفی باید دور عبارت پیمایش شده داخلی، پرانتز باشد یعنی ۳ و ۴ نیز نادرست هستند. به راحتی می‌توان فراخوانی کرد.
- ۵۳- گزینه (۲). گره‌هایی که رنگ دارند، رنگ خودشان و آنهایی که رنگ ندارند رنگ پدرشان یا جدشان بر می‌گردد پس مقدار قرمزها برابر ۶ می‌باشد. (دو گره خود قرمز هستند و ۴ گره بی‌رنگ، پدرشان یا جدشان قرمز است)

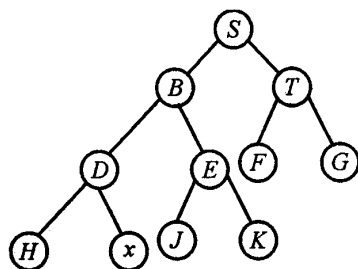
- ۵۴- گزینه (۴).
- ۵۵- گزینه (۳). اشاره‌گر به پدر یک عنصر را باز می‌گرداند و در صورتی که عنصر داده شده ریشه باشد، nil برمی‌گردد.
- ۵۶- گزینه (۱). البته منظور گزینه ۳ این است که برای یافتن $inorder$ یک جنگل، $inorder$ درخت دودویی معادل آن را بیابیم.
- ۵۷- گزینه (۴). زمان پیمایش‌های درخت، به تعداد نودهای درخت وابسته است.
- ۵۸- گزینه (۳ و ۴). اگر عمق ریشه را صفر فرض کنیم، حداقل تعداد نود 2^h و حداکثر $2^{h+1} - 1$ است و اگر عمق ریشه را یک فرض کنیم، حداقل نود 2^{h-1} و حداکثر $2^h - 1$ است.
- ۵۹- گزینه (۰). هیچکدام
- برای همه گزینه‌ها، مثال نقض وجود دارد.



- و به همین ترتیب سایر گزینه‌ها، نادرست هستند.
- ۶۰- گزینه (۴). این تست از کنکور حذف شد. برای همه گزینه‌ها می‌توان مثال نقض آورد. ولی اگر n تعداد نودها باشد (و نه برگ‌ها) آنگاه گزینه ۱ قابل اثبات است.
- ۶۱- گزینه (۳). تابع را T می‌نامیم:

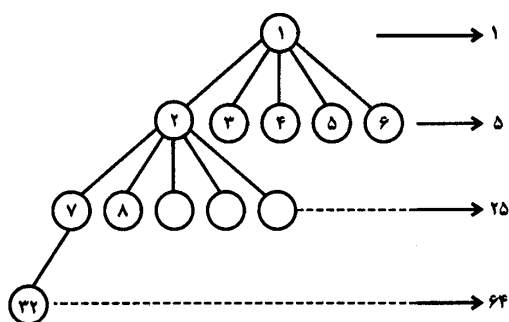


۶۲- گزینه (۴). درخت دودویی کامل دارای ۱۱ نود به شکل روبرو است:



که گزینه ۴ صحیح است.

۶۳- گزینه (۱).



تا سطح سطوم ۳۱ گره وجود دارد پس تعداد گره‌های سطح چهارم $95 - 31 = 64$ می‌باشد. بنابراین در سطح سوم از چپ به راست، ۱۲ نود ۵ فرزند و یک نود ۴ فرزند دارد و ۱۲ نود فرزند ندارند پس تعداد برگ‌ها برابر $76 = 64 + 12$ می‌باشد. گره هفتاد و دوم در سطح چهارم، چهل و یکمین گره است بنابراین پدر آن در سطح سوم از چپ به راست نهمین گره است و شماره آن $15 = 9 + 6$ می‌باشد.

نکته: در درخت m تایی کامل n نودی، پدر گره i در $1 + \left\lfloor \frac{i-1}{m} \right\rfloor$ است پس پدر گره ۷۲ در این تست $15 = 1 + \left\lfloor \frac{72-1}{5} \right\rfloor$ است. و فرزند j ام گره i در $1 + m(i-1) + j$ قرار دارد.

۶۴- گزینه (۲). به نمونه مقابل توجه کنید:

$$x = (a - b * c) \Rightarrow ((a) - ((b) * (c))) \Rightarrow y = (((-((*$$

این سوال می‌خواهد از عبارت $y = (((-((*$ به عبارت $x = (a - b * c)$ برسیم.

با داشتن y و با کمک پشته و یک پیمایش از چپ و سپس بررسی محتویات پشته از بالا، می‌توان درخت عبارت را به صورت یکتا رسم کرد که مرتبه این عملیات $O(n)$ است.

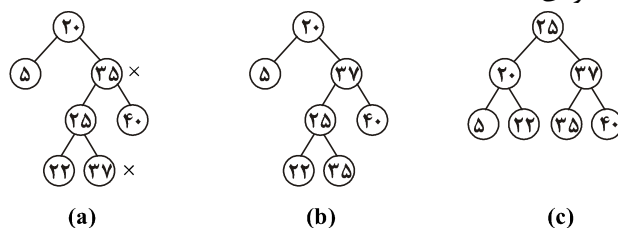
- ۶۵- گزینه (۲). حداکثر تعداد برگ n^h است.
- ۶۶- گزینه (۴). درخت با ارتفاع ماکزیمم دارای این خاصیت است.
- ۶۷- گزینه (۳). اگر در پیمایش pre ، دو مقدار پشت سر هم به صورت xy و در پیمایش $post$ به صورت yx باشند آنگاه x تک فرزندی است پس d و f و h تک فرزندی هستند پس $2^3 = 8$ درخت می‌توان کشید.
- ۶۸- گزینه (۳). I . درخت مورب، II . درخت کامل
- III . ظاهراً چنین درختی وجود ندارد ولی اثبات می‌کنیم که وجود دارد! کفایت با $\frac{n}{4}$ نود درخت تقریباً بر بسازیم که در این صورت پهنای این درخت حدوداً $\frac{n}{4}$ است که $\theta(n)$ است. حال $\frac{n}{4}$ نود باقیمانده را به صورت مورب زیر سطح آخر بچینیم که ارتفاع نیز $\theta(n)$ می‌شود.
- IV . چنین درختی وجود ندارد! درختی که ارتفاعش $\theta(\lg n)$ باشد پهنایش $\theta(n)$ است و درختی که پهنایش $\sqrt{n}$ است ارتفاعش حداقل $\sqrt{n}$ است.
- ۶۹- گزینه (۱). تعداد درخت‌های دودویی 2^k است که k تعداد گره‌های تک فرزندی است. اگر xy در pre و yx در $post$ دیده شود یعنی x تک فرزندی است و برعکس. پس e و f و c تک فرزندی هستند پس $2^3 = 8$ درخت وجود دارد.
- ۷۰- گزینه (۲). درخت با بیشترین ارتفاع مورب است که h تا گره داخلی هر کدام با یک عنصر و یک برگ شامل 2^h عنصر پس بیشترین تعداد عناصر $h + 2^h$ است. کلید طراح گزینه ۴ است.
- ۷۱- گزینه (۱). ریشه یک حالت دارد ولی سایر نودها می‌توانند چپ یا راست پدرشان باشند یعنی ۲ حالت دارند پس 2^{n-1} درخت می‌توان رسم کرد.
- ۷۲- گزینه (۱) البته گره V به اشتباه در پیمایش pre نیامده است. می‌دانیم ترتیب برگ‌ها در پیمایش‌های pre و $post$ و in یکسان است که فقط گزینه ۱ چنین است.

درخت‌های ویژه فصل ۶

درخت جستجوی دودویی (Binary search tree: BST)

درخت جستجوی دودویی (د.ج.د) یک درخت دودویی است که در هر نود آن یک کلید وجود دارد (البته اگر درخت تهی نباشد)، کلیدها متمایز هستند (البته در برخی منابع گفته شده که کلیدها می‌توانند تکراری باشند). کلید هر نود از کلید همه نودهای موجود در زیردرخت چپ (راست) آن نود بزرگتر (کوچکتر) است.

با توجه به این خاصیت، اگر یک د.ج.د را به صورت میان ترتیب پیمایش کنیم، کلیدها به صورت صعودی مرتب می‌شوند. شکل $a-1$ یک د.ج.د نیست زیرا ۳۷ از ۳۵ بزرگتر است ولی در زیردرخت چپ ۳۵ واقع است. اگر ۳۷ و ۳۵ با هم تعویض شوند، د.ج.د شکل $b-1$ حاصل می‌شود. پیمایش میان ترتیب د.ج.د شکل $b-1$ و $c-1$ به صورت $\langle 5, 20, 22, 25, 35, 37, 40 \rangle$ می‌باشد که مرتب صعودی است.

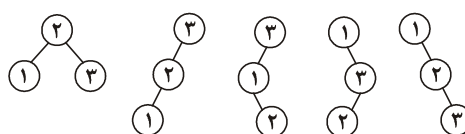


شکل $a-1$ یک د.ج.د نیست و دارای پیمایش میان ترتیب $\langle 5, 20, 22, 25, 37, 35, 40 \rangle$ (چپ به راست) است که مرتب صعودی نیست. b, c د.ج.د هستند و پیمایش میان ترتیب هر دو به صورت $\langle 5, 20, 22, 25, 35, 37, 40 \rangle$ (چپ به راست) است که مرتب صعودی است.

مثال ۱: با سه کلید متفاوت ۱ و ۲ و ۳ چند د.ج.د متفاوت می‌توان ساخت؟

پاسخ: با سه نود تعداد ۵ درخت دودویی وجود دارد که می‌توان کلیدهای ۱ و ۲ و ۳ را طوری در نودها قرار داد که همگی خاصیت د.ج.د داشته باشند یعنی پیمایش میان‌ترتیب همگی، مرتب صعودی باشد.

شکل ۲ این ۵ درخت را نشان می‌دهد:



شکل ۲

نکته: با n کلید متفاوت، می‌توان $C_n = \frac{1}{n+1} \binom{2n}{n}$ درخت جستجوی دودویی متفاوت

ساخت. برای اثبات این مطلب، فرض کنید با کلیدهای ۱ تا n ، د.ج.د ساخته‌ایم و تعداد آن‌ها C_n است. ریشه درخت را کلید k فرض کنید پس کلیدهای ۱ تا $k-1$ در زیردرخت چپ ریشه قرار دارند که خود باید د.ج.د باشند پس تعداد آن‌ها C_{k-1} است و کلیدهای $k+1$ تا n در زیردرخت راست ریشه قرار دارند که خود باید د.ج.د باشند که تعداد آن‌ها C_{n-k} است. پس تعداد کل درخت‌های جستجوی دودویی با ریشه k برابر $C_{k-1} \cdot C_{n-k}$ است. بنابراین تعداد درخت‌های

جستجوی دودویی با کلیدهای ۱ تا n برابر $C_n = \sum_{k=1}^n C_{k-1} \cdot C_{n-k}$ می‌باشد زیرا ریشه (k) می‌تواند از ۱ تا n متغیر باشد. حل این رابطه بازگشتی را می‌توانید در کتاب ساختمان گسسته

بیابید که با تابع مولد قابل حل است و $C_n = \frac{1}{n+1} \binom{2n}{n}$ می‌شود.

نکته: با n کلید متمایز تعداد سطوح درخت جستجوی دودویی حداکثر n (ارتفاع حداکثر

$n-1$) است زیرا در هر سطح ممکن است فقط یک نود وجود داشته باشد. حداقل تعداد سطوح برابر $\lceil \lg n \rceil + 1$ (حداقل ارتفاع برابر $\lfloor \lg n \rfloor$) می‌باشد. پس می‌توان گفت ارتفاع د.ج.د در بدترین حالت از مرتبه $\theta(n)$ و در حالت متوسط و بهترین حالت از مرتبه $\theta(\lg n)$ است. البته اثبات حالت متوسط بسیار دشوار است.

P		
$left$	key	$right$

توجه: برای پیاده‌سازی د.ج.د، از روش پیوندی استفاده می‌کنیم و شکل نودها به صورت مقابل است که p اشاره‌گر به پدر نود می‌باشد. در ادامه برخی الگوریتم‌ها را برای د.ج.د بررسی می‌کنیم.

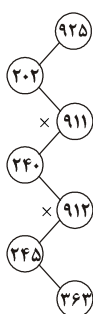
جستجو: برای جستجوی کلید k ، ابتدا k را با کلید ریشه مقایسه می‌کنیم، اگر k با کلید ریشه مساوی باشد، کار تمام است و جستجو خاتمه می‌یابد. اگر k از کلید ریشه کوچکتر باشد زیردرخت چپ و در غیر این صورت، زیردرخت راست با همین شیوه، جستجو می‌شوند. بنابراین زمان اجرای جستجوی کلید k ، به ارتفاع درخت وابسته است یعنی از مرتبه $O(h)$ است که در بدترین حالت $\theta(n)$ و در حالت متوسط $\theta(\lg n)$ می‌باشد. مثلاً در درخت شکل $1-b$ برای جستجوی $k=22$ ، ابتدا $k=22$ با 20 مقایسه می‌شود سپس با 37 و بعد با 25 و در نهایت با 22 مقایسه می‌شود و جستجو موفق است. برای جستجوی $k=32$ کلیدهای $20 \leftarrow 37 \leftarrow 25 \leftarrow 35$ بررسی می‌شوند و جستجو ناموفق است.

تست ۱: فرض کنید اعداد ۱ تا ۱۰۰۰ در یک درخت دودویی جستجو ذخیره شده‌اند و ما می‌خواهیم عدد ۳۶۳ را پیدا کنیم. کدام یک از ترتیب‌های زیر (از چپ به راست) نمی‌تواند بیانگر ترتیب دسترسی به عناصر درخت در این جستجو باشد؟

(دولتی ۸۰)

- (۱) ۳۶۳ و ۲۴۵ و ۹۱۲ و ۲۴۰ و ۹۱۱ و ۲۰۲ و ۹۲۵
- (۲) ۳۶۳ و ۳۶۲ و ۲۵۸ و ۸۹۸ و ۲۴۴ و ۹۱۱ و ۲۲۰ و ۹۲۴
- (۳) ۳۶۳ و ۳۹۷ و ۳۴۴ و ۳۳۰ و ۳۹۸ و ۴۰۱ و ۲۵۲ و ۲
- (۴) ۳۶۳ و ۲۷۸ و ۳۸۱ و ۳۸۲ و ۲۶۶ و ۲۱۹ و ۳۸۷ و ۳۹۹ و ۲

پاسخ: در گزینه ۱، اولین عدد ۹۲۵ است پس ۹۲۵ ریشه است. عدد ۲۰۲ سمت چپ ریشه است. ۹۱۱ از ۹۲۵ کمتر و از ۲۰۲ بزرگتر است پس فرزند راست ۲۰۲ است. به همین ترتیب، این دنباله جستجو به شکل مقابل است. همانطور که مشخص است، ۹۱۲ از ۹۱۱ بزرگتر است ولی در زیردرخت چپ ۹۱۱ قرار دارد که اشتباه است. به طور کلی شرط لازم و کافی برای آنکه یک دنباله از اعداد بتواند دنباله جستجو در یک د.ج.د باشد آن است که هر عدد مثل x در دنباله، از همه اعداد بعد از x کوچکتر یا از همه اعداد بعد از x بزرگتر باشد. بنابراین گزینه ۱ نمی‌تواند دنباله درستی باشد زیرا بعد از ۹۱۱ اعدادی قرار دارند که برخی از ۹۱۱ بزرگتر و برخی از ۹۱۱ کوچکترند. ولی گزینه‌های دیگر، می‌توانند دنباله جستجو باشند.



تست ۲: می‌خواهیم الگوریتمی ارائه دهیم که مشخص کند آیا دنباله‌ای از n کلید، می‌تواند دنباله‌ی جستجوی کلید k در یک د.ج.د باشد یا خیر. الگوریتم کارا برای این مسئله از چه مرتبه‌ای است؟

$$O(n) \quad (۱) \quad O(n^2 \lg n) \quad (۲) \quad O(n^2) \quad (۳) \quad O(\lg n) \quad (۴)$$

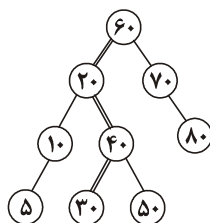
پاسخ: کافی است از انتهای دنباله حرکت کنیم. عنصر کمینه $\min$ و عنصر بیشینه $\max$ را ذخیره کنیم و به هر عدد مثل x که رسیدیم اگر $\min < x < \max$ آنگاه اعلام کنیم که دنباله نادرست است، در غیر این صورت مقادیر $\min$ و $\max$ را به روز کنیم. مثلاً برای دنباله گزینه ۱ تست قبل، از انتهای دنباله حرکت کنید تا قبل از ۹۱۱، کمینه $\min = ۲۴۰$ و بیشینه $\max = ۹۱۲$ است. حال که به $x = ۹۱۱$ می‌رسیم و چون $۲۴۰ < ۹۱۱ < ۹۱۲$ پس این دنباله نادرست است. بدیهی است که مرتبه این الگوریتم $O(n)$ است.

تست ۳: در یک د.ج.د برای جستجوی یک کلید، یک مسیر از ریشه تا یک برگ را پیموده‌ایم. مجموعه عناصری که در سمت چپ این مسیر واقع هستند را A ، عناصر روی مسیر جستجو را مجموعه B و عناصر سمت راست این مسیر را مجموعه C می‌نامیم. برای هر $a \in A, b \in B, c \in C$ کدام گزینه همواره برقرار است؟

$$a \leq b, b \geq c \quad (۲) \quad a \leq b \leq c \quad (۱)$$

$$a \geq b, a \leq c \quad (۳) \quad (۴) \text{ نمی‌توان اظهارنظر کرد.}$$

پاسخ: فرض کنید در درخت مقابل کلید ۳۰ را جستجو می‌کنیم، مسیر



۶۰ ← ۲۰ ← ۴۰ ← ۳۰ را دنبال می‌کنیم. پس طبق صورت سوال، عناصر سمت چپ این مسیر $A = \{۵, ۱۰\}$ و عناصر روی این مسیر $B = \{۶۰, ۲۰, ۴۰, ۳۰\}$ و عناصر سمت راست این مسیر $C = \{۵۰, ۷۰, ۸۰\}$ می‌باشند. اگر $a = ۱۰$ و $b = ۶۰$ و $c = ۵۰$ آنگاه گزینه‌های ۱ و ۳ نادرست می‌شوند. اگر $a = ۵$ و $b = ۲۰$ و $c = ۵۰$ آنگاه گزینه ۲ نادرست می‌شود، پس گزینه ۴ صحیح است. ■

الگوریتم جستجوی کلید k در د.ج.د که x اشاره‌گر به ریشه است به صورت بازگشتی و غیربازگشتی در زیر آمده است. اگر k در درخت باشد، خروجی الگوریتم، اشاره‌گر به نود شامل k است و در غیر این صورت، خروجی تهی است.

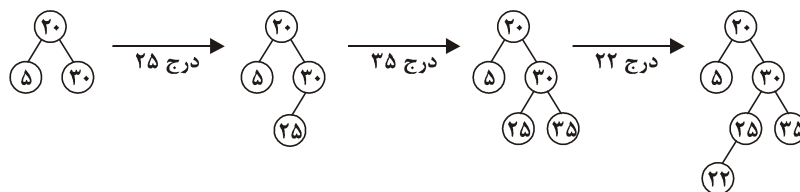
***rec_search* (*x*, *k*)**

```
if (x == NIL or k == key(x))
    return x
if (k < key(x))
    return rec_search(left(x), k)
else
    return rec_search(right(x), k)
```

***iterative_search* (*x*, *k*)**

```
while (x ≠ NIL and k ≠ key(x))
    if (k < key(x)) x = left(x)
    else x = right(x)
return x
```

درج: برای درج عنصر با کلید k در یک د.ج.د، باید k را جستجو کنیم و اگر جستجو ناموفق بود یعنی k در درخت وجود نداشت آنگاه عمل درج را انجام دهیم. یعنی یک نود بسازیم و k را در آن قرار دهیم و این نود را در همان نقطه‌ای که جستجو تمام شده است، وارد کنیم. توجه کنید که ما فرض کرده‌ایم، در درخت جستجوی دودویی کلید تکراری وجود ندارد. در شکل ۳، عمل درج چند عنصر نشان داده شده است.



شکل ۳

عمل درج عنصر با کلید k ، در د.ج.د از مرتبه $O(h)$ است که در بدترین حالت $\theta(n)$ و در حالت متوسط $\theta(\lg n)$ می‌باشد.

در حالت کلی درج عناصر جابجاپذیر نیست یعنی ترتیب درج مهم است. مثلاً در شکل ۳، اگر ترتیب درج ۲۵ و ۲۲ عوض شود یعنی ابتدا ۲۲ و سپس ۲۵ درج شود، شکل درخت نهایی، تغییر می‌کند.

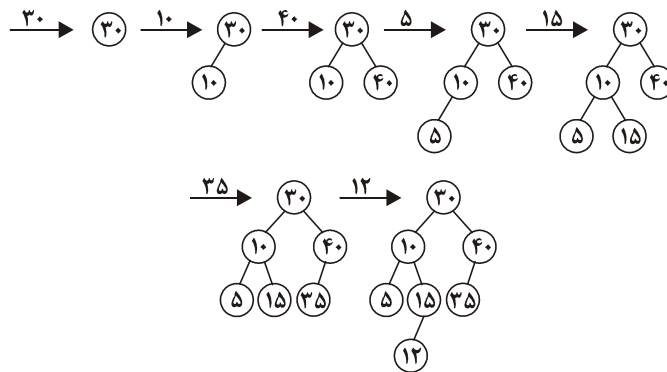
ساخت د.ج.د: فرض کنید می‌خواهیم با n کلید، د.ج.د بسازیم. دو روش برای این منظور وجود دارد.

روش ۱: به ترتیب داده شده، کلیدها را درج کنیم. یعنی اولین کلید ورودی را ریشه قرار دهیم. کلید دوم را با ریشه مقایسه کنیم و چپ یا راست ریشه درج کنیم و به همین ترتیب.

مثال ۲: کلیدهای زیر از چپ به راست وارد شده‌اند. یک د.ج.د بسازید.

۳۰، ۱۰، ۴۰، ۵، ۱۵، ۳۵، ۱۲

پاسخ:



واضح است که این روش از مرتبه $O(n.h)$ (n بار درج) است که در بدترین حالت $\theta(n^2)$ و در حالت متوسط $\theta(n \lg n)$ در واقع اگر ورودی مرتب باشد، درخت حاصل از این روش مورب می‌شود و مرتبه ساخت $\theta(n^2)$ می‌شود.

روش ۲: اگر همه n کلید در اختیار باشند. یعنی قرار نباشد در آینده وارد شوند، می‌توان ابتدا کلیدها را به صورت صعودی با مرتبه $O(n \lg n)$ مرتب کرد سپس کلید وسط (میانه) را با مرتبه $\theta(1)$ ریشه قرار داد و سپس با زیردرخت چپ و راست ریشه، همین عمل را به صورت بازگشتی انجام داد.

اگر $T(n)$ زمان اجرای ساخت درخت با n کلید مرتب باشد آنگاه:

$$T(n) = 2T\left(\frac{n}{2}\right) + \theta(1) = \theta(n)$$

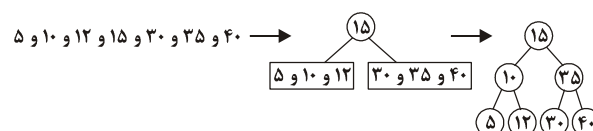
پس مرتبه این روش $O(n \lg n)$ است (به خاطر مرتب‌سازی). در ضمن درخت حاصل از این روش بسیار متوازن است و ارتفاع آن $\theta(\lg n)$ می‌باشد. ولی همانطور که گفته شد این روش فقط در صورتی قابل اجراست که همه n کلید را در اختیار باشند.

مثال ۳: کلیدهای زیر داده شده‌اند با روش ۲ با این کلیدها د.ج.د بسازید.

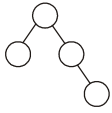
۳۰, ۱۰, ۴۰, ۵, ۱۵, ۳۵, ۱۲

پاسخ: ابتدا کلیدها را به صورت صعودی مرتب می‌کنیم و سپس کلید وسط را ریشه قرار

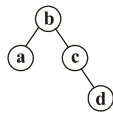
می‌دهیم و همین عمل را با زیردرخت چپ و راست انجام می‌دهیم:



مثال ۴: فرض کنید می‌خواهیم با ۴ کلید $a < b < c < d$ یک د.ج.د با روش درج بسازیم.

چند ترتیب ورود مختلف برای این ۴ کلید وجود دارد که شکل درخت حاصل  باشد؟

پاسخ: با توجه به اینکه پیمایش میان‌ترتیب، لیست صعودی تولید می‌کند پس کلیدها فقط به

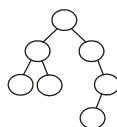
صورت  باید در درخت قرار گیرند. ولی تعداد حالات ورود این کلیدها پرسیده شده است. b

اولین کلیدی است که باید درج شود. و همیشه کلید c باید قبل از کلید d درج شود. ولی کلید a می‌تواند بلافاصله بعد از b یا بلافاصله بعد از c یا بلافاصله بعد از d وارد شود پس سه ترتیب ورود برای این ۴ کلید وجود دارد:

$bacd, bcad, bcda$

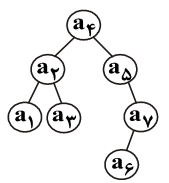
مثال ۵: فرض کنید می‌خواهیم با ۷ کلید $a_1 < a_2 < \dots < a_7$ یک د.ج.د با روش درج بسازیم.

چند ترتیب ورود مختلف وجود دارد که شکل درخت مقابل به دست آید؟



پاسخ: با توجه به شکل درخت، نحوه قرارگیری a_i ها در این درخت فقط به یک شکل

امکان‌پذیر است:



برای ساخت چنین درختی، اولین کلید ورودی a_4 است. a_2 باید قبل از a_1 و a_3 وارد شود.

a_1 و a_3 نسبت به هم می‌توانند جابه‌جا شوند. a_5 باید قبل از a_7 و a_6 قبل از a_6 وارد شوند.

a_1, a_3, a_4 می‌توانند بین a_5, a_6, a_7 وارد شوند. برای شمارش جایگشت‌های مجاز به یکی از

این جایگشت‌ها یعنی $a_4 a_2 a_1 a_3 a_5 a_7 a_6$ (چپ به راست) توجه کنید. a_4 همیشه اولی است، ۶

عنصر دیگر! جایگشت دارند که $a_5 a_7 a_6$ نباید با هم جابه‌جا شوند و $a_1 a_3 a_2$ نیز به همین

ترتیب ولی a_1, a_3 می‌توانند جابه‌جا شوند پس تعداد حالات مجاز $2 \times \frac{6!}{3!3!}$ است.

یافتن عنصر کمینه و بیشینه در د.ج.د: اگر به ساختار درخت جستجوی دودویی توجه کنید، برای یافتن عنصر کمینه، باید از ریشه، لینک‌های چپ را دنبال کنیم و اولین نودی که لینک چپ آن تهی باشد، محل قرارگیری عنصر کمینه است. برای یافتن عنصر بیشینه، باید از ریشه، لینک‌های راست را دنبال کنیم و اولین نودی که لینک راست آن تهی باشد، عنصر بیشینه است. مثلاً در شکل ۴، d عنصر کمینه و c عنصر بیشینه است.

```

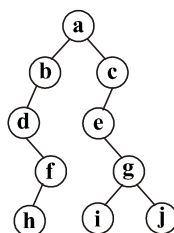
MINIMUM(x)
while (left[x] ≠ NIL)
    x = left[x]
return x

```

```

MAXIMUM(x)
while (right[x] ≠ NIL)
    x = right[x]
return x

```



شکل ۴- d کمینه و c بیشینه است.

یافتن عنصر بعدی ($successor$) و عنصر قبلی ($predecessor$)

عنصر بعدی ($succ(x)$) یعنی کوچکترین عدد بزرگتر از x . عنصر قبلی ($pred(x)$) یعنی بزرگترین عدد کوچکتر از x . در واقع اگر یک د.ج.د به صورت میان ترتیب پیمایش شود، کلیدها به صورت صعودی مرتب می‌شوند و عنصری که بعد از x (قبل از x) در لیست حاصل، ظاهر می‌شود، $succ(x)$ ($pred(x)$) است. پس یک روش برای یافتن $succ(x)$ و $pred(x)$ ، پیمایش میان ترتیب است. ولی پیمایش یک درخت n نودی از مرتبه $\theta(n)$ است، حال آنکه می‌توان با مرتبه $O(h)$ ، عنصر بعدی و قبلی x را یافت. به درخت شکل ۴ توجه کنید. پیمایش میان ترتیب این درخت $d h f b a e i g j c$ (چپ به راست) است. با توجه به این پیمایش می‌توان گفت:

$$succ(d) = h, succ(h) = f, succ(f) = b, pred(e) = a, pred(c) = j, \dots$$

برای یافتن $succ(x)$ الگوریتم کارا به این صورت است: اگر x فرزند راست داشته باشد آنگاه مینیمم زیردرخت راست x ، کوچکترین مقدار بزرگتر از x است پس $succ(x)$ است. اگر x فرزند راست نداشته باشد یعنی زیردرخت راست x تهی باشد آنگاه اولین جد x که x در زیردرخت چپ آن واقع است، کوچکترین مقدار بزرگتر از x یعنی $succ(x)$ است.

الگوریتم یافتن عنصر قبلی $pred(x)$ به این صورت است: اگر x فرزند چپ داشته باشد آنگاه ماکزیمم زیردرخت چپ x ، بزرگترین عنصر کوچکتر از x یعنی $pred(x)$ است. اگر x فرزند چپ نداشته باشد یعنی زیردرخت چپ x تهی باشد آنگاه اولین جد x که x در زیردرخت راست آن واقع است، بزرگترین عنصر کوچکتر از x یعنی $pred(x)$ است.

شبه کد یافتن عنصر بعدی x در ادامه آمد است:
به راحتی می‌توان شبه کد $pred(x)$ را نیز نوشت.

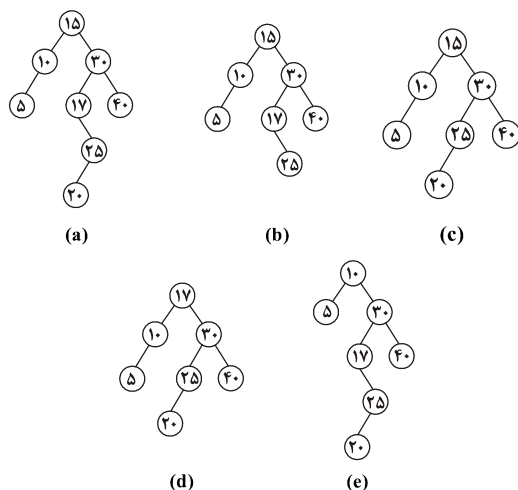
```

succ(x)
if (right(x) ≠ NIL)
    return MINIMUM(right(x))
y = p(x)
while (y ≠ NIL and x == right(y))
    {x = y
    y = p[y]}
return y

```

حذف: فرض کنید می‌خواهیم نود z را از یک د.ج.د حذف کنیم. سه حالت وجود دارد:

- ۱- اگر نود z ، فرزند نداشته باشد، به راحتی نود z را حذف می‌کنیم و اشاره‌گر به z از جانب پدر z را تهی می‌کنیم.
- ۲- اگر نود z ، یک فرزند داشته باشد، نود z را حذف می‌کنیم و فرزند z را به جای z قرار می‌دهیم. یعنی پدر z را به فرزند z اشاره می‌دهیم.
- ۳- اگر نود z ، دو فرزند داشته باشد، در این صورت می‌توان کوچکترین کلید زیردرخت راست z ($succ(z)$) یا بزرگترین کلید زیردرخت چپ z ($pred(z)$) را به جای z قرار داد. در شکل ۵- چند عمل حذف نشان داده شده است.



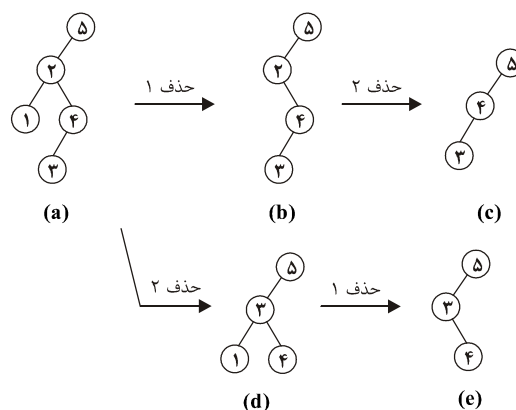
شکل ۵- (a) درخت اولیه (b) حذف ۲۰ از درخت (a)

(c) حذف ۱۷ از درخت (a) (d) حذف ۱۵ از درخت (a) و قرار دادن $succ(15) = 17$ به جای ۱۵

(e) حذف ۱۵ از درخت (a) و قرار دادن $pred(15) = 10$ به جای ۱۵

لازم به ذکر است که اگر z دارای ۲ فرزند باشد، آنگاه $succ(z)$ فرزند چپ ندارد و $pred(z)$ فرزند راست ندارد.

❖ **نکته:** عمل حذف همانند عمل درج، جابه‌جاپذیر نیست. یعنی ترتیب حذف مهم است. در شکل ۶ کلیدهای ۱ و ۲ را با دو ترتیب مختلف حذف کرده‌ایم. شکل نهایی متفاوت شده است:



شکل ۶- حذف کلیدهای ۱ و ۲ از درخت شکل (a) با ترتیب‌های مختلف. در شکل (d) برای حذف کلید ۲، عنصر بعدی ۲ ($succ(2) = 3$) را به جای ۲ قرار داده‌ایم.

❖ **نکته:** عملیات جستجو، درج، یافتن کوچکترین و بزرگترین کلید، یافتن عنصر بعدی و قبلی یک عنصر و حذف یک عنصر در درخت جستجوی دودویی، همگی از مرتبه $O(h)$ هستند که در بدترین حالت $\theta(n)$ و در حالت متوسط $\theta(\lg n)$ می‌باشد.

❖ **نکته:** درخت جستجوی دودویی می‌تواند کلید تکراری نیز داشته باشد. در این صورت هنگام درج کلید k اگر کلید تکراری باشد، می‌توان به زیردرخت چپ یا راست (اختیاری)، کلید k را درج کرد، و یا می‌توان در هر نود یک فیلد قرار داد که تعداد تکرار هر کلید را بشمارد.

دوران (چرخش rotate)

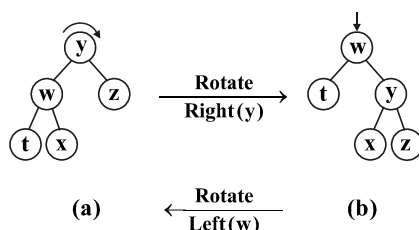
دوران عملی است با مرتبه $\theta(1)$ که می‌توان روی نودی از درخت دودویی انجام داد. کاربرد دوران را در ادامه خواهیم دید. دو نوع دوران وجود دارد: دوران چپ و دوران راست. دوران چپ را می‌توان روی نودی که فرزند راست دارد، انجام داد، و دوران راست روی نودی که فرزند چپ دارد، قابل انجام است. در دوران به چپ روی نود x ، فرزند راست x ، پدر x می‌شود و خود x در جایگاه فرزند چپ قرار می‌گیرد. مثلاً دوران به چپ روی نود x در درخت



موجب می‌شود y پدر x شود

و x فرزند چپ y شود یعنی  حاصل می‌شود. دوران راست روی نود x در درخت  موجب تولید  می‌شود.

اگر عمل دوران روی نودهای درخت جستجوی دودویی انجام شود، پیمایش میان‌ترتیب درخت عوض نمی‌شود یعنی د.ج.د بعد از دوران همچنان د.ج.د است. در هر عمل دوران سه تا اشاره‌گر تغییر می‌کنند. در شکل ۷ چند دوران نشان داده شده است.



شکل ۷

در شکل $a - y$ ، بعد از دوران راست روی y شکل $b - y$ حاصل شده است. این دوران موجب تغییر سه اشاره‌گر شده است که عبارتند از:

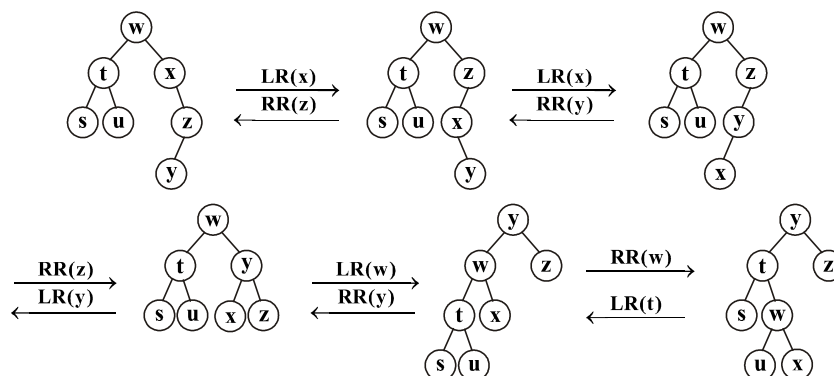
۱- پدر y بعد از دوران، پدر w شده است.

۲- اشاره‌گر چپ y به w اشاره می‌کرده که بعد از دوران به x اشاره می‌کند.

۳- اشاره‌گر راست w به x اشاره می‌کرده که بعد از دوران به y اشاره می‌کند.

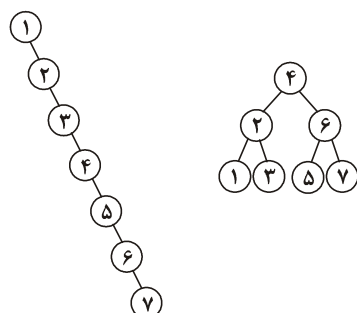
دوران به چپ روی w در شکل $b - w$ ، شکل $a - w$ را تولید کرده است. دوران به چپ روی w موجب تغییر سه اشاره‌گر شده است که عبارتند از: ۱- پدر w بعد از دوران پدر y شده است. ۲- اشاره‌گر راست w به y اشاره می‌کرده که بعد از دوران به x اشاره می‌کند. ۳- اشاره‌گر چپ y به x اشاره می‌کرده که بعد از دوران به w اشاره می‌کند.

در شکل ۷ هر دو درخت دارای پیمایش میان‌ترتیب $z y x w t$ (چپ به راست) هستند. یعنی همانطور که گفته شد عمل دوران، پیمایش میان‌ترتیب را تغییر نمی‌دهد. در شکل ۸ چندین عمل دوران نشان داده شده است. منظور از RR دوران راست ($Right Rotate$) و LR دوران چپ ($left Rotate$) می‌باشد:



شکل ۸- در تمام درخت‌ها، پیمایش میان ترتیب $(s \ t \ u \ w \ x \ y \ z)$ (چپ به راست) می‌باشد.

تست ۴: اگر منظور از L_x ، دوران به چپ روی نود x و منظور از R_x ، دوران به راست روی نود x باشد، چه ترتیبی از دوران‌ها (از چپ به راست) درخت سمت چپ شکل مقابل را به درخت سمت راست تبدیل می‌کند.



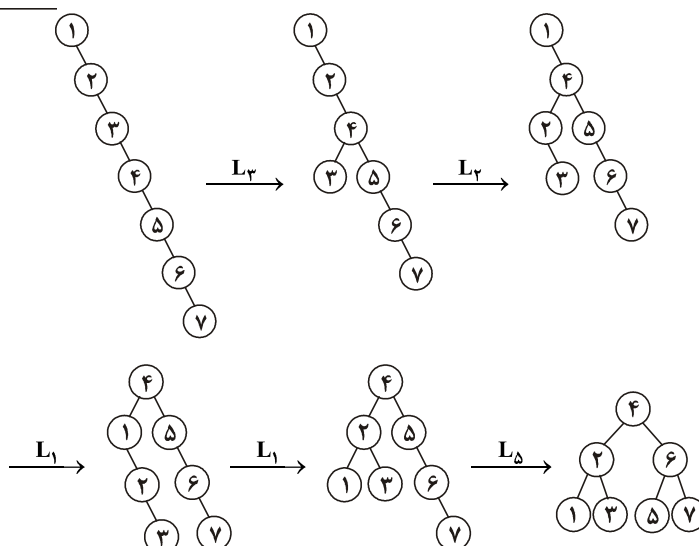
$$L_\Delta \ L_\gamma \ L_\epsilon \ L_\epsilon \ L_\gamma \ (1)$$

$$L_\epsilon \ L_\gamma \ L_\gamma \ L_\epsilon \ L_1 \ (2)$$

$$L_\gamma \ L_\gamma \ L_1 \ L_1 \ L_\Delta \ (3)$$

$$L_\epsilon \ L_1 \ L_\gamma \ L_\gamma \ L_\Delta \ (4)$$

پاسخ: گزینه (۳):



تست ۵: فرض کنید می‌خواهیم با عمل دوران، یک درخت مورب راست شامل $n = 2^k - 1$ عنصر را به درخت پر تبدیل کنیم، اگر $T(n)$ حداقل تعداد عمل دوران برای این منظور باشد آنگاه:

$$T(n) = \left\lceil \frac{n}{2} \right\rceil + 2T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) \quad (۲) \quad T(n) = \left\lceil \frac{n}{2} \right\rceil + 2T\left(\left\lceil \frac{n}{2} \right\rceil\right) \quad (۱)$$

$$T(n) = 3T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) \quad (۴) \quad T(n) = \left\lfloor \frac{n}{2} \right\rfloor + 2T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) \quad (۳)$$

پاسخ: گزینه (۳). مشابه تست قبل می‌توان عمل کرد و یا می‌توان ابتدا $\left\lfloor \frac{n}{2} \right\rfloor$ دوران چپ روی ریشه بدهیم. حال درختی داریم که $\left\lfloor \frac{n}{2} \right\rfloor$ نود در زیردرخت چپ به صورت مورب چپ و $\left\lfloor \frac{n}{2} \right\rfloor$ نود در زیردرخت راست به صورت مورب راست وجود دارد که هر زیردرختی نیاز به $T\left(\left\lfloor \frac{n}{2} \right\rfloor\right)$ دوران دارد پس $T(n) = \left\lfloor \frac{n}{2} \right\rfloor + 2T\left(\left\lfloor \frac{n}{2} \right\rfloor\right)$.

تست ۶: کدام دوران‌ها ارتفاع درخت مقابل را از ۵ به ۳ کاهش می‌دهد؟

(LR یعنی دوران چپ و RR یعنی دوران راست)

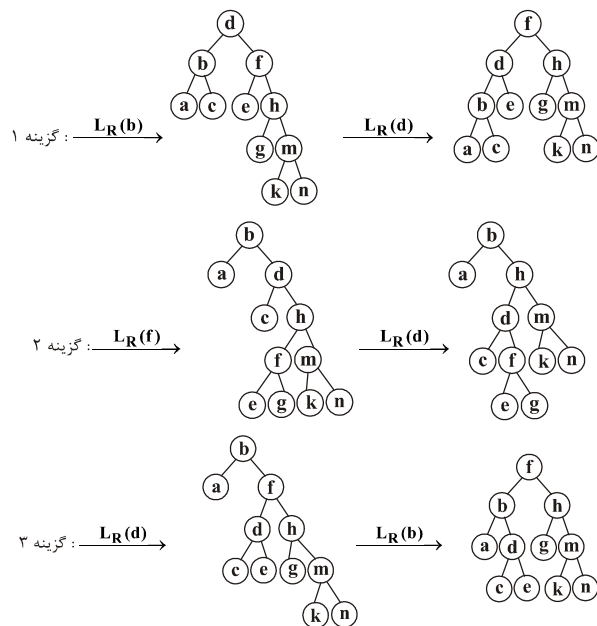
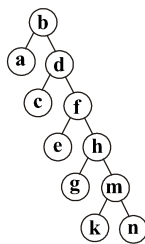
(۱) ابتدا $LR(b)$ و بعد $LR(d)$

(۲) ابتدا $LR(f)$ و بعد $LR(d)$

(۳) ابتدا $LR(d)$ و بعد $LR(b)$

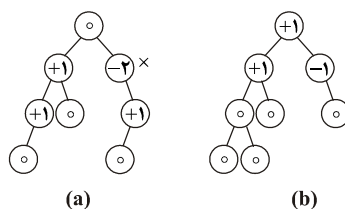
(۴) گزینه‌های ۱ و ۳

پاسخ: گزینه (۴).



نام AVL ، برگرفته از نام دو نفری است که این درخت را ابداع کردند. AVL درخت جستجوی دودویی متوازن است.

درخت دودویی متوازن (*balanced*) درخت دودویی است که تعداد سطوح زیردرخت چپ و منهای تعداد سطوح زیردرخت راست هر نودی، یکی از اعداد $+1$ یا -1 یا 0 باشد. در شکل ۹ تعدادی درخت دودویی نشان داده شده‌اند که داخل هر نود، فاکتور توازن آن نود نوشته شده است و نودهایی که فاکتورشان ± 2 است، توازن را به هم زده‌اند.



شکل ۹- a متوازن نیست b متوازن هست

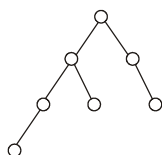
نکته: اگر $N(h)$ حداقل تعداد نود لازم برای ساخت AVL به ارتفاع h باشد آنگاه $N(0) = 1$ زیرا برای ساخت AVL به ارتفاع صفر فقط یک نود لازم است. $N(1) = 2$ زیرا برای

ساخت AVL به ارتفاع ۱، حداقل دو نود لازم است:  یا $N(2) = 4$. زیرا برای ساخت

AVL به ارتفاع ۲ حداقل ۴ نود لازم است:  به طور کلی، برای ساخت AVL به ارتفاع h

و با کمترین تعداد نود، کافی است سمت چپ ریشه تعداد $N(h-1)$ و سمت راست ریشه تعداد $N(h-2)$ نود قرار دهیم. پس $N(h) = N(h-1) + N(h-2) + 1$. مثلاً

لازم است:



با توجه به رابطه $N(h) = N(h-1) + N(h-2) + 1$ می‌توان نتیجه گرفت $N(h) = \theta\left(\left(\frac{1+\sqrt{5}}{2}\right)^h\right)$ (طبق حل رابطه بازگشتی فیبوناچی). پس $h = \theta\left(\log_{\left(\frac{1+\sqrt{5}}{2}\right)} N(h)\right)$ یعنی ارتفاع درخت AVL از مرتبه $\theta(\lg n)$ است. بنابراین تمام الگوریتم‌های پایه مثل جستجو، درج، حذف، یافتن کمینه، یافتن بیشینه، یافتن عنصر بعدی $(succ(x))x$ و یافتن عنصر قبلی $(pred(x))x$ در AVL از مرتبه $O(\lg n)$ هستند.

تست ۷: فرض کنید F_n جمله n ام فیبوناچی است. می‌دانیم $F_0 = 0$ و $F_1 = 1$ و $F_n = F_{n-1} + F_{n-2}$. حداقل تعداد گره‌های یک درخت AVL با ارتفاع h کدام است؟

$$F_{h+3} - 1 \quad (۴) \quad 2F_{h+2} - 2h \quad (۳) \quad F_{h+2} - 1 \quad (۲) \quad F_{h+3} \quad (۱)$$

پاسخ: گزینه (۴). حداقل تعداد گره‌های AVL با ارتفاع h از رابطه $N(h) = N(h-1) + N(h-2) + 1$ که $N(0) = 1$ و $N(1) = 2$ است به دست می‌آید. با مقایسه جملات فیبوناچی با جملات $N(h)$ می‌توان رابطه $N(h) = F_{h+3} - 1$ را نتیجه گرفت:

$$N(0) = 1, N(1) = 2, N(2) = 4, N(3) = 7, N(4) = 12, N(5) = 20, \dots$$

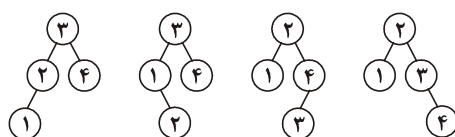
$$F_0 = 0, F_1 = 1, F_2 = 1, F_3 = 2, F_4 = 3, F_5 = 5, F_6 = 8, F_7 = 13, F_8 = 21, \dots$$

$$\text{مثلاً } N(5) = F_8 - 1 \text{ و یا } N(4) = F_7 - 1 \text{ و به همین ترتیب.}$$

تست ۸: با کلیدهای ۱ و ۲ و ۳ و ۴ چه تعداد AVL شامل ۴ گره با کلیدهای متمایز می‌توان ساخت؟

$$5 \quad (۴) \quad 4 \quad (۳) \quad 3 \quad (۲) \quad 2 \quad (۱)$$

پاسخ: گزینه (۳).

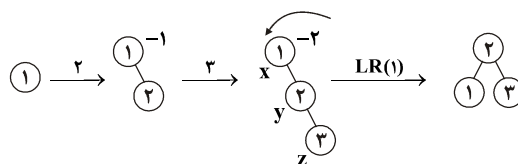


ساخت AVL

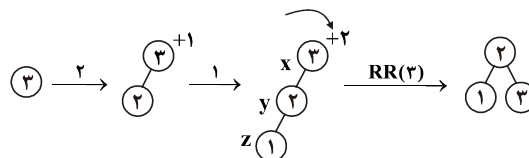
برای ساخت AVL با تعدادی کلید، همانند ساخت درخت جستجوی دودویی، کلیدها را به ترتیب درج می‌کنیم. با هر عمل درج باید فاکتور توازن اجداد نود جدید را بررسی کنیم و اگر فاکتور توازن نودی $+2$ یا -2 شده باشد باید با عمل دوران، درخت را متوازن کنیم. با درج عنصر جدید z ممکن است تعدادی از اجداد نود z ، دارای فاکتور توازن ± 2 شوند که در این صورت باید به پایین‌ترین جد نود z ، توجه کنیم. چهار حالت ممکن است پیش آید که با یک مثال نشان

می‌دهیم و سپس حالات را بیان می‌کنیم. فرض کنید می‌خواهیم با کلیدهای ۱ و ۲ و ۳ ترتیب‌های مختلف، AVL بسازیم. به ۴ حالت زیر توجه کنید:

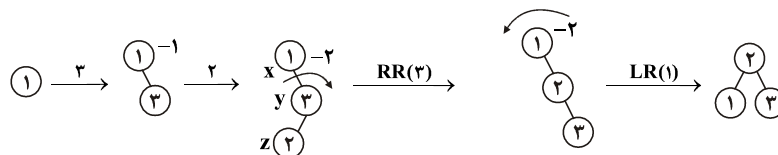
حالت ۱: به ترتیب با ۱ و ۲ و ۳ (راست به چپ) AVL می‌سازیم:



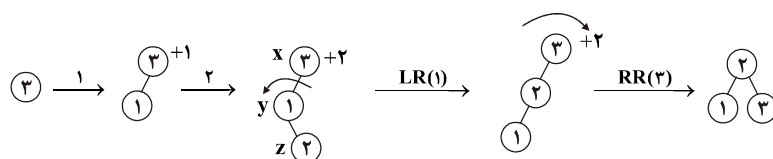
حالت ۲: به ترتیب با ۳ و ۲ و ۱ (راست به چپ) AVL می‌سازیم:



حالت ۳: به ترتیب با ۱ و ۳ و ۲ (راست به چپ) AVL می‌سازیم:



حالت ۴: به ترتیب با ۳ و ۱ و ۲ (راست به چپ) AVL می‌سازیم:



حال به بررسی ۴ حالت می‌پردازیم. منظور از z نود جدید است. x اولین جد z است که فاکتور توازنش $+2$ یا -2 شده است و y فرزند x است که در مسیر درج واقع است.

حالت ۱: y فرزند راست x است و z به زیردرخت راست y درج شده است و فاکتور توازن x برابر -2 شده است. در این حالت کافی است x را دوران به چپ (LR) بدهیم.

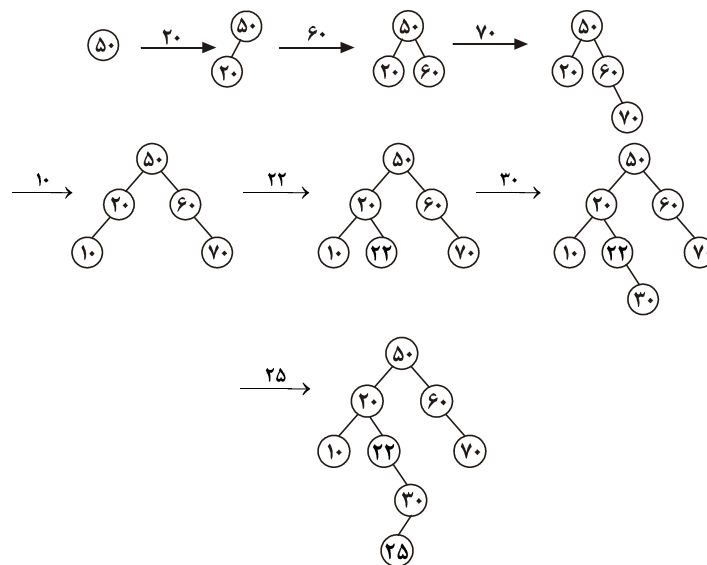
حالت ۲: y فرزند چپ x است و z به زیردرخت چپ y درج شده است و فاکتور توازن x برابر $+2$ شده است. در این حالت کافی است x را دوران به راست (RR) می‌دهیم.

حالت ۳: y فرزند راست x است و z در زیردرخت چپ y درج شده است و فاکتور توازن x برابر -2 شده است. در این حالت ابتدا y را به راست و سپس x را به چپ دوران می‌دهیم.

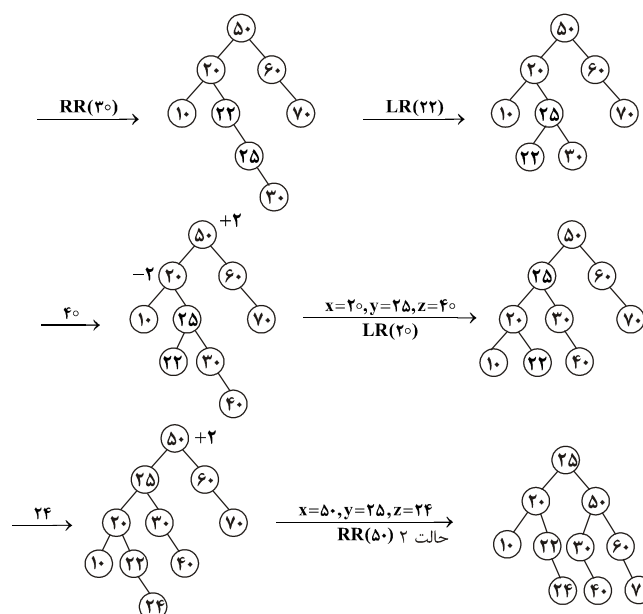
حالت ۴: y فرزند چپ x است و z در زیردرخت راست y درج شده است و فاکتور توازن x برابر $+2$ شده است. در این حالت ابتدا y را به چپ و سپس x را به راست دوران می‌دهیم.

مثال ۶: با کلیدهای داده شده (AVL از چپ به راست) بسازید.
 $50, 20, 60, 70, 10, 22, 30, 25, 40, 24$

پاسخ:



با درج ۲۵ فاکتور توازن ۲۲ برابر -2 ، فاکتور توازن ۲۰ برابر -2 و فاکتور توازن ۵۰ برابر $+2$ می‌شود. یعنی سه تا از اجداد نود $z = 25$ دارای فاکتور ± 2 شده‌اند. ما باید به پایین‌ترین جد یعنی به $x = 22$ توجه کنیم. پس $x = 22$ و $y = 30$ و $z = 25$ می‌باشد و حالت ۳ اتفاق افتاده است. ابتدا $y = 30$ را به راست و سپس $x = 22$ را به چپ دوران می‌دهیم:



توجه کنید پس از درج ۲۴، فاکتور ۵۰ برابر ۲ می‌شود پس $x = 50$ است، فرزند x که در مسیر درج واقع است $y = 25$ می‌باشد و نود جدید $z = 24$ است. و چون $y = 25$ فرزند چپ $x = 50$ است و $z = 24$ به زیردرخت چپ $y = 25$ درج شده است پس حالت ۲ می‌باشد.

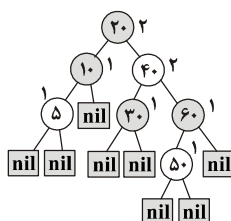
درخت قرمز سیاه (Red-Black:RB)

همان‌طور که دیدیم، در درخت جستجوی دودویی عملیات پایه یعنی جستجو و درج و حذف و یافتن کمینه و بیشینه و بعدی و قبلی یک عنصر از مرتبه $O(h)$ هستند، یعنی زمان اجرای این عملیات به ارتفاع درخت وابسته است که در بدترین حالت از مرتبه $\theta(n)$ می‌باشد؛ زیرا در د.ج.د ممکن است در هر سطح فقط یک نود قرار گیرد و ارتفاع درخت $n - 1$ شود. درخت AVL این مشکل را برطرف کرد که مرتبه عملیات گفته شده در AVL از مرتبه $O(\lg n)$ است. ولی AVL خیلی متداول نیست ارتفاع درخت قرمز سیاه، همانند AVL از مرتبه $\theta(\lg n)$ است بنابراین عملیات پایه گفته شده در درخت قرمز سیاه از مرتبه $O(\lg n)$ هستند. درخت قرمز سیاه نوعی د.ج.د است که هر نود دو حالت دارد، یا قرمز است و یا سیاه. در واقع در هر نود، فیلدی وجود دارد که رنگ نود را ذخیره می‌کند.

در درخت قرمز سیاه، خواصی وجود دارد که تضمین می‌کند طول هر مسیری از هر نودی مثل x به سمت برگ‌های موجود در زیر درختان x از دو برابر مسیرهای دیگر از x به برگ‌ها، بیشتر نیست. در درخت قرمز سیاه برگ‌ها متفاوت از سایر درخت‌ها تعریف می‌شوند. به لینک‌های تهی (اتصالات تهی) برگ گفته می‌شود. در واقع در این درخت، در برگ‌ها هیچ اطلاعاتی ذخیره نمی‌شود و فقط نودهای داخلی دارای کلید و اطلاعات هستند. خواص درخت قرمز سیاه عبارتند از:

۱- هر نود، قرمز یا سیاه است. ۲- رنگ ریشه سیاه است. ۳- برگ‌ها (یعنی nil ها یا اتصالات تهی) سیاه هستند. ۴- اگر نودی قرمز باشد آنگاه فرزندان آن سیاه هستند. ۵- برای هر نود، تمام مسیرهای ساده از آن نود به هر برگ موجود در زیردرختان آن نود، دارای تعداد یکسانی نود سیاه است. به تعداد نودهای سیاه موجود در مسیرهای از نود x به سمت برگ‌های موجود در زیر درختان x ، سیاه ارتفاع $(black\ height : bh(x))x$ گویند.

شکل ۱۰- یک درخت قرمز سیاه را نشان می‌دهد. نودهای سیاه کمی تیره نشان داده شده‌اند و اتصالات تهی به شکل مستطیل نمایش داده شده‌اند.



شکل ۱۰- درخت قرمز سیاه حاوی ۷ نود داخلی، کنار هر نود، سیاه ارتفاع (bh) آن نوشته شده است.

سیاه ارتفاع درخت، برابر سیاه ارتفاع ریشه است. پس سیاه ارتفاع درخت شکل ۱۰ برابر $bh = ۲$ می‌باشد. در شکل ۱۰ دقت کنید، ریشه سیاه است. برگ‌ها (nil ها) سیاه هستند. از ریشه به سمت هر برگ حرکت کنید، ۲ تا نود سیاه (خود ریشه را بشمارید) مشاهده می‌کنید پس سیاه ارتفاع ریشه برابر ۲ است. از نود با کلید ۴۰ به سمت برگ‌های واقع در زیردرختان ۴۰ حرکت کنید، ۲ تا نود سیاه مشاهده می‌کنید پس سیاه ارتفاع این نود نیز برابر ۲ است. به همین ترتیب سیاه ارتفاع نودهای داخلی، کنار آن‌ها نوشته شده است. لازم به ذکر است که ارتفاع درخت شکل ۱۰ برابر $h = ۴$ می‌باشد زیرا فاصله ریشه تا دورترین برگ برابر ۴ می‌باشد.

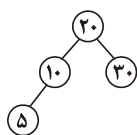
ممکن است برای درخت قرمز سیاه، برگ‌ها یعنی نودهای مستطیلی شکل کشیده نشوند ولی بدانیم که این نودها با رنگ سیاه وجود دارند.

مثال ۷: دو د.ج.د مقابل را به چند حالت می‌توان به درخت

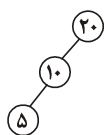
قرمز سیاه تبدیل کرد؟

یعنی به چند حالت می‌توان به نودها، رنگ قرمز یا سیاه

نسبت داد؟

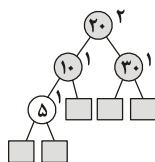


(a)



(b)

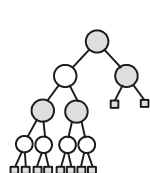
پاسخ: a ریشه (۲۰) باید سیاه باشد. برگ‌ها (nil) که نشان داده نشده‌اند، سیاه هستند. حداقل یکی از نودهای ۱۰ و ۵ باید سیاه شود زیرا فرزند یک نود قرمز نمی‌تواند قرمز باشد. پس حتماً ۳۰ باید سیاه شود چون مسیرهای موجود از ۲۰ به برگ‌ها باید تعداد نود سیاه یکسانی ببینند. همچنین ۱۰ باید سیاه باشد و ۵ باید قرمز بماند که مشکلی برای تعداد نودهای سیاه در مسیرها به برگ‌ها، ایجاد نشود. پس این درخت فقط به یک حالت قرمز سیاه می‌شود.



کنار هر نود داخلی bh آن نوشته شده است.

b ریشه باید سیاه باشد. حداقل یکی از نودهای ۱۰ و ۵ باید سیاه شوند ولی هر کدام که سیاه شوند، آنگاه مسیر چپ ۲۰ دوتا سیاه (با احتساب برگ nil) و مسیر راست ۲۰ شامل یک سیاه می‌باشد. پس این درخت را نمی‌توان قرمز سیاه کرد.

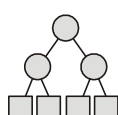
نکته: در درخت قرمزسیاه، تعداد سطوح زیردرخت چپ هر نود مثل x حداکثر ۲ برابر (یا



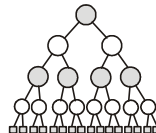
حداقل نصف) تعداد سطوح زیردرخت راست نود x است. زیرا می‌توان در زیردرخت چپ x ، به صورت یک درمیان، نودهای قرمز و سیاه قرار داد و در زیردرخت راست x همه نودها سیاه باشند. مثلاً شکل مقابل یک درخت قرمز سیاه با $bh = ۲$ را نشان می‌دهد که زیردرخت چپ ریشه دارای ۴ سطح و زیردرخت راست ریشه دارای ۲ سطح است.

مثال ۸: درخت قرمز سیاه که سیاه ارتفاع آن $bh = k$ است، حداقل و حداکثر چند نود داخلی دارد؟

پاسخ: به عنوان مثال فرض کنید $bh = ۲$ ، حداقل تعداد نود داخلی ۳ و حداکثر تعداد نود داخلی برابر ۱۵ می‌باشد:



حداقل نود داخلی با $bh=۲$



حداکثر نود داخلی با $bh = ۲$

در واقع حداقل تعداد نود وقتی است که درخت پر باشد و همه نودها سیاه باشند و حداکثر تعداد نود وقتی است که درخت پر باشد و نودها یک سطح در میان سیاه - قرمز باشند. پس حداقل تعداد نود برابر $۲^k - ۱$ و حداکثر تعداد نود برابر $۲^{2k} - ۱$ می‌باشد.

مثال ۹: ارتفاع درخت قرمز سیاه با n نود داخلی، حداکثر چقدر است؟

پاسخ: همانطور که در مثال قبل دیدیم حداقل تعداد نود داخلی با سیاه ارتفاع k برابر $2^k - 1$ است پس $n \geq 2^k - 1$ (زیرا $2^k - 1$ حداقل تعداد نود داخلی ممکن است پس n که تعداد نود داخلی است از $2^k - 1$ بیشتر یا مساوی است) در نتیجه $k \leq \lg(n+1)$. از طرفی ارتفاع درخت حداکثر دو برابر سیاه ارتفاع درخت است پس $h \leq 2k \rightarrow h \leq 2\lg(n+1)$ نتیجه: با n نود داخلی، ارتفاع درخت قرمز سیاه حداکثر $2\lg(n+1)$ است و می‌توان نشان داد، حداقل ارتفاع $\lg(n+1)$ می‌باشد پس همیشه $h \in \theta(\lg n)$.

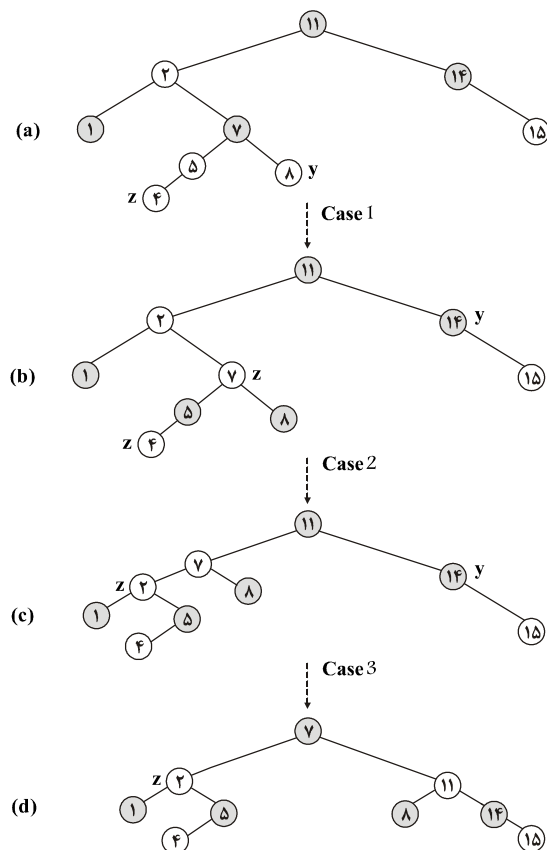
درج در درخت قرمز سیاه: برای درج نود z در درخت قرمز سیاه، همانند درج در د.ج.د، جستجو می‌کنیم تا محل درج پیدا شود و سپس z را با رنگ قرمز درج می‌کنیم. اگر پدر z ، سیاه باشد که کار تمام است ولی اگر پدر z ، قرمز باشد آنگاه یکی از خواص درخت قرمز سیاه نقض شده است و باید درخت تنظیم شود که برای این منظور دو عمل انجام می‌شود: تغییر رنگ و عمل دوران که سه حالت پیش می‌آید. کد مقابل، بعد از درج z در درخت T ، عمل بازسازی و تنظیم را انجام می‌دهد.

```

BB - INSERT - FIXUP( $T, z$ )
۱. while ( $color[p[z]] == RED$ )
    {
۲.   if ( $P[z] == Left[p[p[z]]]$ )
        {
۳.        $y = right[p[p[z]]]$ 
۴.       if ( $color[y] == RED$ )
            {
۵.          $color[p[z]] = BLACK$  // case ۱
۶.          $color[y] = BLACK$  // case ۱
۷.          $color[p[p[z]]] = RED$  // case ۱
۸.          $z = p[p[z]]$  // case ۱
            }
۹.       else if ( $z == right[p[p[z]]]$ )
            {
۱۰.         $z = p[z]$  // case ۲
۱۱.         $LEFT - ROTATE(T, z)$  // case ۲
            }
۱۲.         $color[p[z]] = BLACK$  // case ۳
۱۳.         $color[p[p[z]]] = RED$  // case ۳
۱۴.         $RIGHT - ROTATE(T, p[p[z]])$  // case ۳
            }
۱۵.       else (same as then clause
                with "right" and "left" exchanged)
        }
۱۶.  $color[root[T]] = BLACK$ 

```

در این کد منظور از $p[z]$ پدر z است. $p[p[z]]$ پدر بزرگ z است. $color[p[z]]$ یعنی رنگ پدر z و $right[p[p[z]]]$ یعنی فرزند راست پدر بزرگ z . شکل ۱۱ پس از درج z ، سه حالت پیش آمده را نشان می‌دهد.



شکل ۱۱

همانطور که گفته شده z را با رنگ قرمز درج می‌کنیم. اگر z ریشه درخت باشد که باید آن را سیاه کنیم و کار تمام است. اگر پدر z سیاه باشد که هیچ خاصیت از درخت قرمز سیاه نقض نشده است ولی اگر پدر z قرمز باشد آنگاه باید درخت را تنظیم کنیم و همانطور که در کد و در شکل ۱۱ می‌بینیم سه حالت وجود دارد. همانطور که در خط ۳ الگوریتم دیدیم، y به عموی z اشاره می‌کند و خط ۴، رنگ y را بررسی می‌کند. اگر رنگ y قرمز باشد، حالت ۱ ($case\ 1$) اجرا می‌شود. در غیر این صورت، حالات ۲ و ۳ اجرا می‌شوند. در تمامی این حالات، پدر بزرگ z ، سیاه است زیرا پدر z قرمز است.

حالت ۱: این حالت وقتی پیش می‌آید که پدر z ($p[z]$) و y (عموی z) هر دو قرمز هستند. در این حالت کافی است پدر z و عموی z را سیاه کنیم و پدر بزرگ z را قرمز کنیم و سپس z را به پدر بزرگ z اشاره دهیم ($z = p[p[z]]$) (خطوط ۵ تا ۸ الگوریتم)

حالت ۲: رنگ y (عموی z) سیاه است و z فرزند راست پدرش است.

حالت ۳: رنگ y (عموی z) سیاه است و z فرزند چپ پدرش است.

در حالت ۲، نود z فرزند راست پدرش است. در این حالت، z را به پدر z اشاره می‌دهیم و سپس z جدید را دوران به چپ می‌دهیم و بدین ترتیب به حالت ۳ وارد می‌شویم.

در حالت ۳، عموی z یعنی y سیاه است و z فرزند چپ پدرش است و همچنین z و پدرش هر دو قرمز هستند. در این حالت، رنگ پدر z را سیاه می‌کنیم و پدر بزرگ z را قرمز می‌کنیم و روی پدر بزرگ z عمل دوران راست انجام می‌دهیم و کار تمام است و در واقع حلقه *while* پایان می‌یابد.

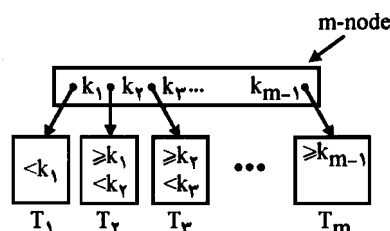
تمامی این ۳ حالت وقتی است که پدر z فرزند چپ پدر بزرگ z باشد. اگر پدر z فرزند راست پدر بزرگ z باشد، تمامی این ۳ حالت پیش می‌آید فقط در همه بحث‌ها *left* و *right* باید با هم تعویض شوند.

نتیجه: در عمل درج در درخت قرمز سیاه، حداکثر ۲ عمل دوران انجام می‌شود و مرتبه درج $O(\lg n)$ است.

◀ **نکته:** عمل حذف در درخت قرمز سیاه از مرتبه $O(\lg n)$ است.

تعریف: یک m -node در یک درخت جستجو گره‌ای است که $m-1$ داده به نام‌های $k_1, k_2, \dots, k_{m-1}$ بطوریکه $k_1 < k_2 < \dots < k_{m-1}$ را ذخیره می‌کند و دارای لینک‌هایی به m زیر درخت T_1 و T_2 و ... و T_m است، بطوری که:

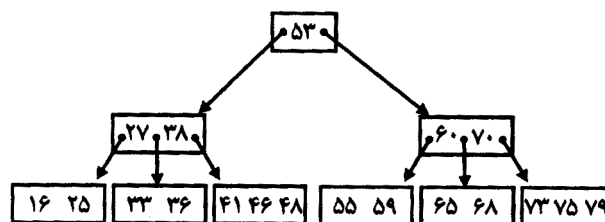
داده‌های موجود در $k_i \leq T_{i+1}$ و داده‌های موجود در $T_i < k_i$



با این تعریف می‌توان گفت در یک BST ، گره‌ها 2 -node هستند.

درخت ۲-۳-۴

درختی که هر نود آن دارای حداکثر ۳ مقدار است و همه نودهای داخلی آن ۲-node، 3-node یا 4-node هستند و همه برگ‌ها هم سطح هستند.



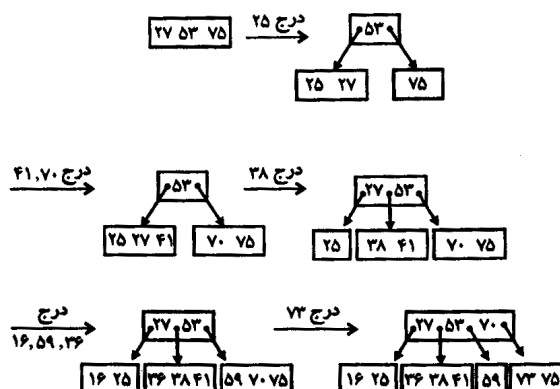
برای جستجوی $x=36$ در این درخت، x را با ریشه مقایسه می‌کنیم و چون $36 < 53$ به زیر درخت چپ می‌رویم و چون $27 < 36 < 38$ می‌باشد زیر درخت وسطی یعنی $33 \quad 36$ را بررسی می‌کنیم.

برای ساخت درخت ۲-۳-۴ با یک نود شروع می‌کنیم و مقادیر را به آن وارد می‌کنیم تا پر شود، یعنی ۳ مقدار در آن قرار گیرد. در این صورت وقتی یک داده جدید وارد می‌شود، نود دو قسمت می‌شود در یک قسمت مقدار کمتر از عدد وسط و در نود دیگر مقدار بیشتر از عدد وسط قرار می‌گیرد. خود عدد وسط در نود پدر دو نود دیگر قرار می‌گیرد. و مقدار جدید به یکی از برگ‌ها وارد می‌شود.

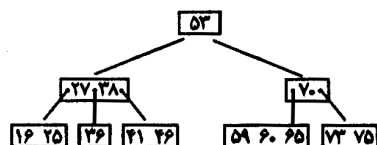
مثال ۱۰: اعداد زیر به ترتیب از چپ به راست در یک درخت ۲-۳-۴ درج می‌شود:

۵۳ ۲۷ ۷۵ ۲۵ ۷۰ ۴۱ ۳۸ ۱۶ ۵۹ ۳۶ ۷۳ ۶۵ ۶۰ ۴۶ ۵۵

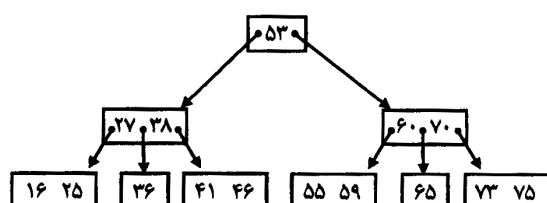
پاسخ: اعداد ۵۳ و ۲۷ و ۷۵ در یک گره درج می‌شوند.



درج ۶۵ و ۶۰ بدون مشکل انجام می‌شود ولی درج ۴۶ برگ دوم از سمت چپ را دو قسمت می‌کند:



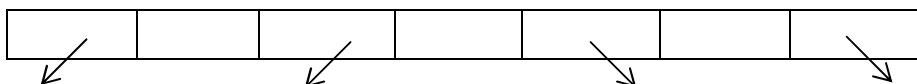
اکنون درج ۵۵ برگ دوم از سمت راست را دو قسمت می‌کند.



نمودار: برای درج یک نود از ریشه جستجو می‌کنیم و بهتر است در مسیر جستجو هر نودی که پر بود را بشکنیم تا به یک برگ برسیم و درج را در آن برگ انجام دهیم.

برای پیاده‌سازی درخت ۲-۳-۴ می‌توان نودها را به شکل مقابل تعریف کرد:

$child[1]$ $data[1]$ $child[2]$ $data[2]$ $child[3]$ $data[3]$ $child[4]$



ولی این روش بهینه نیست و اتلاف حافظه زیادی دارد.

B-tree

یک درخت جستجو است که هر نود دارای n کلید غیرنزولی و $n + 1$ اشاره‌گر است. برگ‌ها هم سطح هستند و اشاره‌گرهای آنها تعریف نشده است. برای B -tree، درجه $t \geq 2$ مطرح می‌شود، هر نود بجز ریشه باید حداقل $t - 1$ کلید داشته باشد (ریشه می‌تواند حداقل یک کلید داشته باشد)، هر نود باید حداکثر $2t - 1$ کلید داشته باشد. ساده‌ترین B -tree همان درخت ۲-۳-۴ است که $t = 2$ می‌باشد.

قضیه: اگر تعداد کل کلیدها n و ارتفاع h و درجه t باشد آنگاه $h \leq \log_t \frac{n+1}{2}$

اثبات: تعداد نودها وقتی مینیمم است که ریشه یک کلید و سایر نودها دارای $t - 1$ کلید باشند، در

این صورت در عمق صفر یک نود، در عمق یک ۲ نود، در عمق ۲ تعداد $2t$ نود، در عمق ۳ تعداد $2t^2$

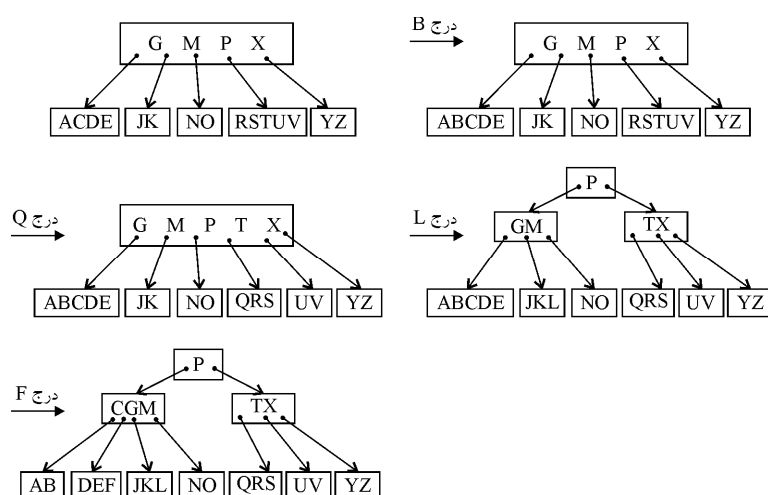
نود و ... در عمق h تعداد $2t^{h-1}$ نود وجود دارد پس برای تعداد کلیدها یعنی n داریم:

$$n \geq 1 + (t-1) \sum_{i=1}^h 2t^{i-1} = 2t^h - 1 \Rightarrow h \leq \log_t \frac{n+1}{2}$$

❖ **نکته:** جستجو، درج و حذف در B -tree از مرتبه $O(t \log_t^n) = O(t \cdot h)$ است.

❖ **نکته:** برای درج در B -tree، از ریشه جستجو می‌کنیم و در مسیر جستجو هر نودی که پر بود (شامل $t - 1$ کلید بود) آن را دو قسمت می‌کنیم و کلید وسط آن نود را به نود پدرش منتقل می‌کنیم، البته ممکن است ارتفاع درخت یک واحد افزایش یابد. در نهایت کلید جدید در یک برگ درج می‌شود:

مثال ۱۱: فرض کنید در B -tree مقابل $t = 3$ یعنی هر نود حداکثر ۵ و حداقل ۲ کلید می‌تواند داشته باشد. چند عمل درج نشان داده شده است:



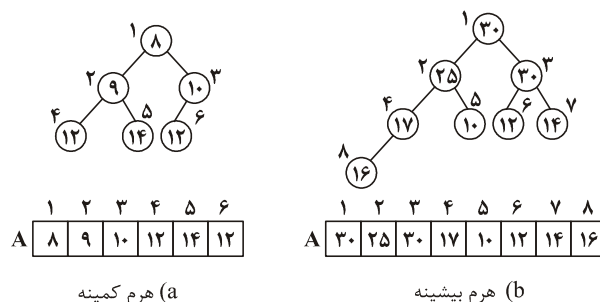
مثلاً برای درج L چون ریشه پر بود آن را $Split$ کردیم و یا برای درج F چون F باید در نود $ABCDE$ که پر است درج شود، این نود $Split$ شد.

هرم دودویی (binary heap)

هرم دودویی برای ساخت صف اولویت و همچنین مرتب‌سازی کاربرد دارد. دو نوع هرم دودویی وجود دارد:

هرم دودویی کمینه ($binary\ minheap$) و هرم دودویی بیشینه ($binary\ maxheap$) از این به بعد هرم دودویی کمینه را هرم کمینه و هرم دودویی بیشینه را هرم بیشینه خواهیم گفت. هرم کمینه ($minheap$) یک درخت دودویی کامل است که در هر نود آن یک کلید (عدد) وجود دارد و کلید هر نود از کلید فرزندان آن نود، کوچک‌تر یا مساوی است. هرم بیشینه ($maxheap$) یک درخت دودویی کامل است که در هر نود آن یک کلید وجود دارد و کلید هر نود از کلید فرزندان

آن نود، بزرگتر یا مساوی است. یادآوری می‌کنیم که درخت دودویی کامل همه سطوح ماقبل آخرش پر می‌باشد و نودها در سطح آخر آن از چپ قرار می‌گیرند. درخت دودویی کامل شامل n گره را می‌توان با آرایه $A[1..n]$ نشان داد که $A[\lfloor \frac{n}{2} \rfloor + 1..n]$ گره‌های داخلی و $A[\lfloor \frac{n}{2} \rfloor + 1..n]$ گره‌های خارجی (برگ) هستند. در شکل ۱۲ قسمت a یک هرم کمینه و قسمت b یک هرم بیشینه نمایش داده شده‌اند.



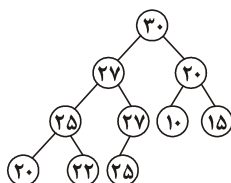
شکل ۱۲

یادآوری می‌کنیم در درخت دودویی کامل، روابط پدر و فرزندی به این شکل است:

$$\begin{aligned} \text{parent}(i) &= \left\lfloor \frac{i}{2} \right\rfloor \quad (i \geq 1) \\ \text{Left}(i) &= 2i \quad (2i \leq n) \\ \text{Right}(i) &= 2i + 1 \quad (2i + 1 \leq n) \end{aligned}$$

مثال ۱۲: آیا آرایه A نشان‌دهنده یک هرم بیشینه است؟

پاسخ: هر عنصر داخلی باید از فرزندانش بزرگتر یا مساوی باشد. $A[1] = 30$ که از فرزندانش یعنی از $A[2] = 27$ و $A[3] = 20$ بزرگتر است. $A[2] = 27$ از فرزندانش یعنی از $A[4] = 25$ و $A[5] = 27$ بزرگتر یا مساوی است. $A[3] = 20$ از $A[6] = 10$ و $A[7] = 15$ بزرگتر است. $A[4] = 25$ از $A[8] = 20$ و $A[9] = 22$ بزرگتر است. $A[5] = 27$ از تنها فرزندش یعنی $A[10] = 25$ بزرگتر است. پس آرایه A یک هرم بیشینه است و البته می‌توان شکل درخت کامل A را رسم کرد:



نتیجه: تشخیص اینکه آرایه $A[1..n]$ یک هرم کمینه یا یک هرم بیشینه است از مرتبه $O(n)$

```

for (i = 1 to ⌊ $\frac{n}{2}$ ⌋)
    if ((A[i] < A[2i]) or
        (2i + 1 ≤ n and A[i] < A[2i + 1]))
        return false;
return true;

```

است. زیرا باید عناصر داخلی $A[1]$ تا $A[\lfloor \frac{n}{2} \rfloor]$ را با فرزندانشان مقایسه کنیم. مثلاً شبه کد تشخیص هرم بیشینه به شکل مقابل است:

در این کد، شرط $2i + 1 \leq n$ به این علت ذکر شده است که مطمئن شویم $A[i]$ حتماً فرزند راست دارد. دقت کنید اگر n زوج باشد آنگاه آخرین گره داخلی $(A[\lfloor \frac{n}{2} \rfloor])$ فقط فرزند چپ دارد و فرزند راست ندارد. ■

با توجه به ساختار هرم کمینه و بیشینه به راحتی می‌توان نکات زیر را استنباط کرد:

۱- در هرم بیشینه، بزرگترین کلید در ریشه است و کوچکترین کلید در یکی از برگ‌هاست. پس یافتن ماکزیمم در هرم بیشینه از مرتبه $\theta(1)$ است و یافتن مینیمم از مرتبه $\theta(n)$ می‌باشد زیرا برای یافتن مینیمم باید تمام $\lfloor \frac{n}{2} \rfloor$ برگ، بررسی شوند.

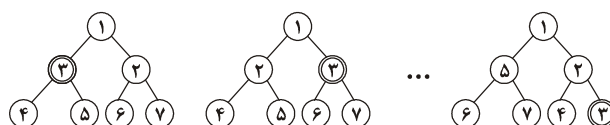
۲- در هرم کمینه، کوچکترین کلید در ریشه است و بزرگترین کلید در یکی از برگ‌هاست. پس یافتن مینیمم در هرم کمینه از مرتبه $\theta(1)$ و یافتن ماکزیمم از مرتبه $\theta(n)$ می‌باشد.

۳- از آنجایی که هرم درخت کامل است پس تعداد سطوح هرم با n گره برابر $\lfloor \lg n \rfloor + 1$ است. یعنی ارتفاع هرم برابر $\lfloor \lg n \rfloor$ می‌باشد.

مثال ۱۳: در هرم کمینه $A[1..n]$ شامل n کلید، دومین کوچکترین، سومین کوچکترین و به طور کلی k امین کوچکترین مقدار، در کدام اندیس می‌توانند باشند؟

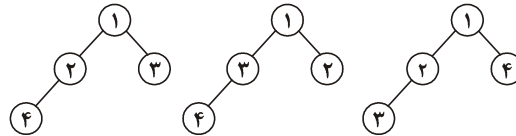
فرض کنید n به اندازه کافی بزرگ است و همچنین کلیدها را متمایز در نظر بگیرد.

پاسخ: اولین کوچکترین کلید که همیشه در $A[1]$ است. دومین کوچکترین کلید در سطح دوم یعنی در $A[2]$ یا $A[3]$ است. سومین کوچکترین مقدار در سطح دوم یا سوم می‌تواند باشد پس در $A[2..7]$ می‌تواند قرار گیرد. به طور کلی k امین کوچکترین مقدار ($k \neq 1$) در $A[2^{k-1}..2^k - 1]$ می‌تواند قرار گیرد. مثلاً فرض کنید با کلیدهای ۱ تا ۷ هرم کمینه ساختیم، عدد ۳ سومین کوچکترین است که می‌تواند در $A[2]$ یا $A[3]$ یا ... یا $A[7]$ قرار گیرد:



مثال ۱۴: با کلیدهای ۱ تا ۴ چند تا هرم کمینه شامل ۴ گره و با کلیدهای متمایز می‌توان ساخت؟

پاسخ: می‌توان ۳ تا هرم کمینه ساخت که عبارتند از:



مثال ۱۵: با کلیدهای ۱ تا ۷ چند تا هرم کمینه شامل ۷ گره و با کلیدهای متمایز می‌توان

ساخت؟

پاسخ: مینیمم کلید ریشه است. از بین ۶ کلید دیگر باید ۳ کلید برای زیر درخت چپ ریشه

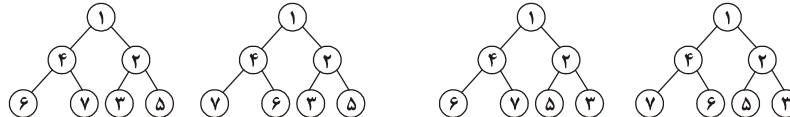
انتخاب کنیم که $\binom{6}{3}$ حالت دارد و سه کلید دیگر به زیردرخت راست ریشه وارد می‌شوند. حال ۳

کلید در زیردرخت چپ ریشه، ۲ حالت دارند (کوچکترین این سه کلید ریشه و دو کلید دیگر به دو

حالت می‌توانند قرار گیرند). سه کلید در زیردرخت راست ریشه نیز ۲ حالت دارند پس جواب برابر

۷ و ۶ و ۴ کلیدهای ۴ و ۶ و ۷ را برای زیردرخت چپ ریشه انتخاب کرده‌ایم، در این صورت تعداد ۴ هرم

کمینه حاصل می‌شود که عبارتند از:



مثال ۱۶: با کلیدهای ۱ تا ۱۰ چند هرم کمینه با کلیدهای متمایز شامل ۱۰ گره می‌توان

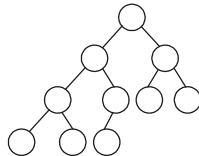
ساخت؟

پاسخ: کوچکترین کلید، ریشه است. از بین ۹ کلید باقی‌مانده باید ۶ کلید برای زیردرخت چپ

انتخاب شود. مینیمم این ۶ کلید ریشه است و از بین ۵ کلید باید ۳ کلید برای زیردرخت چپ

انتخاب شود. این ۳ کلید ۲ حالت دارند. ۳ کلید موجود در زیردرخت راست ریشه نیز ۲ حالت

دارند پس جواب برابر است با:



$$\binom{9}{6} \times \binom{5}{3} \times 2 \times 2 = \frac{9!}{6!3!} \times \frac{5!}{3!2!} \times 2 \times 2 = 336.$$

تست ۹: فرض کنید $T(n)$ تعداد هرم‌های کمینه (یا بیشینه) با n گره ($n = 2^k - 1$) شامل n کلید متمایز باشد آنگاه:

$$\begin{aligned} T(n) &= \binom{n-1}{\lceil \frac{n}{2} \rceil} \cdot T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) \quad (۲) & T(n) &= \binom{n-1}{\lfloor \frac{n}{2} \rfloor} \cdot T\left(\left\lceil \frac{n}{2} \right\rceil\right) \quad (۱) \\ T(n) &= 2 \binom{n-1}{\lceil \frac{n}{2} \rceil} \cdot T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) \quad (۴) & T(n) &= 2 \binom{n-1}{\lfloor \frac{n}{2} \rfloor} \cdot T\left(\left\lceil \frac{n}{2} \right\rceil\right) \quad (۳) \end{aligned}$$

پاسخ: با $n = 2^k - 1$ گره، هرم حاصل یک درخت دودویی پر می‌باشد. فرض کنید می‌خواهیم هرم کمینه بسازیم. کوچکترین کلید را ریشه قرار می‌دهیم و از $n - 1$ کلید باقی‌مانده تعداد $\lfloor \frac{n}{2} \rfloor$ کلید برای زیردرخت چپ به $\binom{n-1}{\lfloor \frac{n}{2} \rfloor}$ حالت انتخاب می‌کنیم. حال همین عمل را با $\lfloor \frac{n}{2} \rfloor$ کلید زیردرخت چپ و با $\lceil \frac{n}{2} \rceil$ کلید زیردرخت راست، به صورت بازگشتی انجام می‌دهیم، که در هر زیردرخت $T(\lfloor \frac{n}{2} \rfloor)$ هرم کمینه می‌توان ساخت. پس تعداد هرم کمینه (یا بیشینه) برابر $T(n) = \binom{n-1}{\lfloor \frac{n}{2} \rfloor} \cdot T(\lfloor \frac{n}{2} \rfloor)$ می‌باشد. پس گزینه ۱ صحیح است.

الگوریتم‌های هرم بیشینه

الگوریتم‌های مربوط به هرم بیشینه را ارائه می‌دهیم. برای هرم کمینه نیز به طور مشابه می‌توانید الگوریتم ارائه دهید.

درج در هرم بیشینه

می‌خواهیم کلید x را در هرم بیشینه درج کنیم. از آنجایی که هرم، درخت دودویی کامل است، کلید x را انتهای هرم یعنی در اولین مکان خالی قرار می‌دهیم و سپس x را با اجدادش مقایسه می‌کنیم و تا زمانی که x از پدرش بزرگتر است، x را با پدرش تعویض می‌کنیم.

مثال ۱۷: ابتدا کلید ۳۰ و سپس کلید ۴۰ را در هرم بیشینه

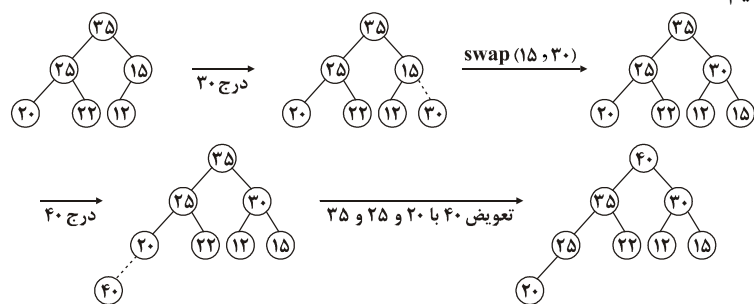
	۱	۲	۳	۴	۵	۶	۷	۸
۸	۳۵	۲۵	۱۵	۲۰	۲۲	۱۲		

درج کنید.

پاسخ: طول آرایه A ($Length(A)$) برابر ۸ است که ثابت است. تعداد عناصر موجود در هرم $heapsize(A)$ برابر ۶ است. با هر عمل درج، $heapsize$ یک واحد اضافه می‌شود. برای درج

کلید ۳۰، این کلید را در $A[7]$ قرار می‌دهیم ($heapsize$ برابر ۷ می‌شود) سپس ۳۰ را با $A[\lfloor \frac{7}{2} \rfloor] = A[3] = 15$ مقایسه می‌کنیم و چون $30 > 15$ ، باید ۳۰ با ۱۵ تعویض شود. حال ۳۰ را با $A[\lfloor \frac{3}{2} \rfloor] = A[1] = 35$ مقایسه می‌کنیم که نیازی به تعویض نیست. پس نتیجه

سپس با اجدادش یعنی با $A[4], A[2], A[1]$ مقایسه می‌کنیم و چون ۴۰ از همه اجدادش بزرگتر است، تعویض می‌کنیم و نتیجه A است. برای درک بهتر شکل درخت این مثال می‌کشیم:



پس جمعاً با ۴ تعویض، کلیدهای ۳۰ و ۴۰ درج شدند.

زمان اجرای درج یک کلید، به ارتفاع درخت وابسته است و چون ارتفاع درخت کامل شامل n گره، برابر $\lceil \lg n \rceil$ است، پس عمل درج یک کلید در هرم از مرتبه $O(\lg n)$ است. شبه کد الگوریتم درج کلید key در هرم بیشینه A به شکل مقابل است:

در این کد، تا زمانی که key از اجدادش بزرگتر است، عنصر $A[\lfloor \frac{i}{2} \rfloor]$ به $A[i]$ منتقل می‌شود.

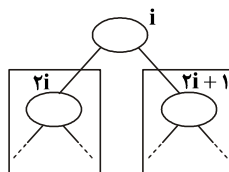
```

MaxHeapInsert ( $A, key$ )
 $heapsize(A) = heapsize(A) + 1$ 
 $i = heapsize(A)$ 
while ( $i > 1$  and  $key > A[\lfloor \frac{i}{2} \rfloor]$ )
{
     $A[i] = A[\lfloor \frac{i}{2} \rfloor]$ 
     $i = \lfloor \frac{i}{2} \rfloor$ 
}
 $A[i] = key$ 

```


الگوریتم $maxheapify$ (بازسازی هرم)

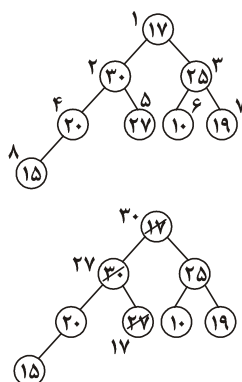
این الگوریتم اندیس i را دریافت می‌کند. هنگام فراخوانی این الگوریتم با اندیس i ، باید زیردرختان چپ و راست اندیس i (یعنی زیردرختان با ریشه $A[2i]$ و $A[2i+1]$) خود هرم بیشینه باشند و احتمالاً عنصر $A[i]$ ممکن است از فرزندانش کوچکتر باشد. این الگوریتم عنصر $A[i]$ را با فرزندانش ($A[2i]$ و $A[2i+1]$) مقایسه می‌کند و اگر خاصیت هرم بیشینه نقض شده باشد یعنی اگر $A[i]$ از فرزندانش (یا از یک فرزندش) کوچکتر باشد، $A[i]$ را با بزرگترین فرزندش تعویض می‌کند و سپس همین عمل را روی فرزند $A[i]$ ، انجام می‌دهد، شکل ۱۳ نشانگر این الگوریتم است.



شکل ۱۳- الگوریتم بازسازی هرم اگر روی i انجام شود، عنصر $A[i]$ را در صورت لزوم با بزرگترین فرزندش تعویض می‌کند و بازگشتی ادامه می‌دهد.

مثال ۱۸: الگوریتم $maxheapify$ را روی اندیس ۱ در آرایه A اجرا کنید. $(maxheapify(A, 1))$

پاسخ: آرایه A را به شکل درخت دودویی کامل نمایش می‌دهیم. همانطور که مشاهده می‌کنید، زیر درختان با ریشه $A[2]$ و $A[3]$ خود هرم بیشینه هستند و فقط $A[1]$ خاصیت هرم بیشینه را نقض کرده است. الگوریتم $maxheapify$ ، $A[1]$ را با $A[2]$ و $A[3]$ مقایسه می‌کند و با $A[2]$ تعویض می‌کند. حال $A[2]$ برابر ۱۷ می‌باشد که باید با فرزند بزرگش یعنی $A[5] = ۲۷$ تعویض شود. پس با دو تعویض خاصیت هرم بیشینه برقرار می‌شود:



شبه کد الگوریتم *maxheapify* به صورت مقابل است:

```

Maxheapify (A, i)
  if ( $\text{2}i \leq \text{heapsize}(A)$  and  $A[\text{2}i] > A[i]$ )
    Largest =  $\text{2}i$  else Largest = i
  if ( $\text{2}i + 1 \leq \text{heapsize}(A)$  and  $A[\text{2}i + 1] > A[\text{Largest}]$ )
    Largest =  $\text{2}i + 1$ 
  if (Largest  $\neq i$ )
    {
      swap( $A[i]$ ,  $A[\text{Largest}]$ )
      maxheapify(A, Largest)
    }

```

عنصر $A[i]$ با $A[\text{2}i]$ و $A[\text{2}i + 1]$ مقایسه می‌شود و اندیس بزرگترین عنصر بین این سه عنصر، در *Largest* قرار می‌گیرد و اگر *Largest* برابر $\text{2}i$ یا $\text{2}i + 1$ باشد آنگاه $A[i]$ با $A[\text{Largest}]$ تعویض می‌شود و الگوریتم با اندیس *Largest* فراخوانی بازگشتی می‌شود.

مرتبه این الگوریتم روی هرم n عنصری، $O(\lg n)$ است زیرا زمان این الگوریتم به ارتفاع درخت وابسته است. البته اگر بخواهیم دقیق‌تر بحث کنیم، زمان این الگوریتم به ارتفاع $A[i]$ بستگی دارد پس بهتر است بگوییم زمان این الگوریتم از مرتبه $O(\lg \frac{n}{i})$ است.

ساخت هرم بیشینه (*Buildmaxheap*)

می‌خواهیم با n عنصر موجود در $A[1..n]$ یک هرم بیشینه بسازیم. دو روش برای این منظور وجود دارد.

روش ۱: n بار عمل *maxheapinsert* را اجرا کنیم. یعنی به ترتیب $A[1]$ سپس $A[2]$ سپس $A[3]$ به همین ترتیب تا $A[n]$ را درج کنیم.

```

Buildmaxheap ( $A[1..n]$ )
  heapsize(A) = 1
  for (i = 2 to n)
    maxheapinsert(A,  $A[i]$ )

```

البته می‌توان عمل درج را از $A[2]$ تا $A[n]$ انجام داد. واضح است که مرتبه این روش $O(n \lg n)$ است.

روش ۲: از الگوریتم $maxheapify$ کمک بگیریم. به این صورت که از آخرین عنصر داخلی یعنی از $A[\lfloor \frac{n}{2} \rfloor]$ تا عنصر $A[1]$ عمل $maxheapify$ را فراخوانی کنیم:

Buildmaxheap ($A[1..n]$)

$heapsize(A) = n$

for ($i = \lfloor \frac{n}{2} \rfloor$ downto ۱)
 $maxheapify(A, i)$

ثابت خواهیم کرد که این روش از مرتبه $\theta(n)$ است.

مثال ۱۹: با آرایه A هرم بیشینه بسازید.

۱	۲	۳	۴	۵	۶	۷	۸
۳۰	۴۰	۱۰	۱۵	۱۲	۴۵	۳۵	۲۲

پاسخ: روش ۱: ابتدا $heapsize(A) = ۱$ می‌باشد پس فقط $A[۱] = ۳۰$ در هرم می‌باشد.

سپس در حلقه for عنصر $A[۲] = ۴۰$ درج می‌شود که باید با $A[۱]$ تعویض شود پس تا کنون هرم

به صورت $\begin{bmatrix} 1 & 2 \\ 40 & 30 \end{bmatrix}$ است. در مرحله بعدی $A[۳] = ۱۰$ درج می‌شود و چون از پدرش یعنی $A[۱]$

کوچکتر است، نیاز به تعویض نیست، پس هرم به صورت $\begin{bmatrix} 1 & 2 & 3 \\ 40 & 30 & 10 \end{bmatrix}$ می‌باشد. حال $A[۴] = ۱۵$

درج می‌شود و چون از پدرش یعنی $A[۲] = ۳۰$ کوچکتر است نیاز به تعویض نیست پس هرم به

شکل $\begin{bmatrix} 1 & 2 & 3 & 4 \\ 40 & 30 & 10 & 15 \end{bmatrix}$ می‌باشد. اکنون $A[۵] = ۱۲$ درج می‌شود که از پدرش یعنی $A[۲] = ۳۰$

کوچکتر است و نیاز به تعویض نیست پس هرم به شکل $\begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 40 & 30 & 10 & 15 & 12 \end{bmatrix}$ می‌باشد. اکنون عنصر

$A[۶] = ۴۵$ را درج می‌کنیم و چون ۴۵ از اجدادش یعنی از $A[۳]$ و $A[۱]$ بزرگتر است باید

تعویض شود و هرم به شکل $\begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 \\ 45 & 30 & 40 & 15 & 12 & 10 \end{bmatrix}$ می‌باشد. حال $A[۷] = ۳۵$ درج می‌شود که

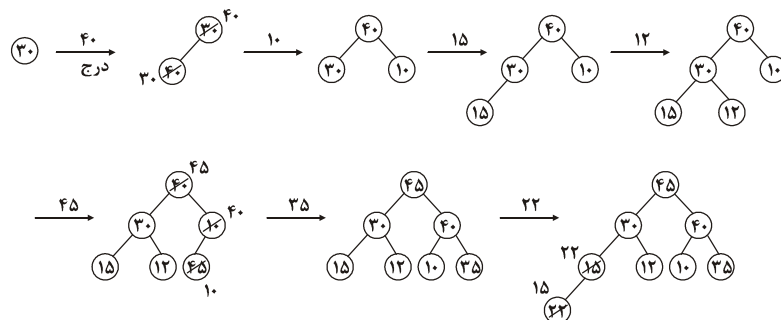
چون از پدرش یعنی از $A[۳] = ۴۰$ کوچکتر است نیاز به تعویض نیست پس هرم به شکل

$\begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ 45 & 30 & 40 & 15 & 12 & 10 & 35 \end{bmatrix}$ می‌باشد. در نهایت عنصر $A[۸] = ۲۲$ درج می‌شود که از $A[۴]$ بزرگتر

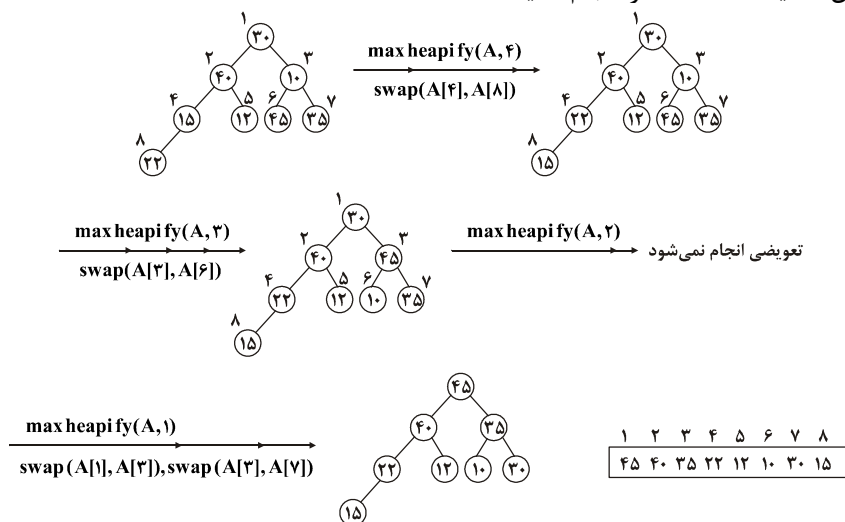
است ولی از $A[۲]$ بزرگتر نیست پس $A[۸]$ با $A[۴]$ تعویض می‌شود و هرم نهایی به شکل

می‌باشد. عملیات فوق را به شکل درخت نیز نشان می‌دهیم:

۱	۲	۳	۴	۵	۶	۷	۸
۴۵	۳۰	۴۰	۲۲	۱۲	۱۰	۳۵	۱۵



روش ۲: الگوریتم *maxheapify* را به ترتیب روی $A[4]$ سپس $A[3]$ و $A[2]$ و $A[1]$ انجام می‌دهیم. برای اجرای این روش، ابتدا همه عناصر آرایه را به شکل درخت دودویی کامل رسم کنید و سپس عملیات گفته شده را انجام دهید:



در این دو مثال هر دو روش جمعاً ۴ تعویض انجام دادند ولی لزوماً تعداد تعویض این دو روش یکسان نیست. همچنین هرم حاصل از دو روش نیز لزوماً یکسان نیست.

نکته: روش دوم از روش اول سریع‌تر است. همان‌طور که گفته شد مرتبه روش اول $O(n \lg n)$ و مرتبه روش دوم $\theta(n)$ است. ولی روش دوم را فقط وقتی می‌توان استفاده کرد که همه عناصر در اختیار باشند. یعنی اگر قرار باشد عناصر بعداً وارد شوند باید از روش اول استفاده شود و با ورود هر عنصر عمل درج انجام شود.

مثال ۲۰: ثابت کنید ساخت هرم با روش ۲ از مرتبه $\theta(n)$ است.

پاسخ: *maxheapify* روی نودی با ارتفاع h از مرتبه $O(h)$ است. همچنین ثابت می‌شود در

درخت کامل با n گره، در ارتفاع h حداکثر $\left\lceil \frac{n}{2^{h+1}} \right\rceil$ گره وجود دارد. پس زمان اجرای روش ۲ برای

ساخت هرم، حداکثر $O(h) \sum_{h=0}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil$ است زیرا ارتفاع هرم برابر $\lfloor \lg n \rfloor$ است پس داریم:

$$\sum_{h=0}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil O(h) = O\left(n \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h}\right) = O\left(n \sum_{h=0}^{\infty} \frac{h}{2^h}\right) = O(n)$$

(یادآوری می‌کنیم سری هندسی $\sum_{k=0}^{\infty} x^k = \frac{1}{1-x}$ به ازای $|x| < 1$ می‌باشد. حال اگر از طرفین

این رابطه مشتق بگیریم و سپس در x ضرب کنید رابطه $\sum_{k=0}^{\infty} kx^k = \frac{x}{(1-x)^2}$ به دست می‌آید.

حال اگر به جای k قرار دهید h و به جای x قرار دهید $\frac{1}{2}$ رابطه $\sum_{h=0}^{\infty} \frac{h}{2^h} = \frac{\frac{1}{2}}{(1-\frac{1}{2})^2} = 2$

حاصل می‌شود).

مرتب‌سازی هرمی (heapsort)

فرض کنید می‌خواهیم آرایه $A[1..n]$ را به صورت صعودی مرتب کنیم. روش مرتب‌سازی هرمی به این صورت است که ابتدا با آرایه A یک هرم بیشینه می‌سازد و سپس $n-1$ بار عمل حذف ریشه (*extractmax*) را انجام می‌دهد. عمل حذف ریشه به این صورت است که کلید ریشه را با کلید آخرین عنصر تعویض می‌کنیم و *heapsize* را یک واحد کم می‌کنیم و در نهایت روی ریشه عمل *maxheapify* را انجام می‌دهیم با این عمل در واقع عنصر ماکزیمم در انتهای آرایه قرار می‌گیرد و از هرم خارج می‌شود و سپس هرم بازسازی می‌شود.

```

Heapsort ( $A[1..n]$ )
۱. Buildmaxheap ( $A[1..n]$ )
۲. for ( $i = n$  downto ۲)
    {
        swap ( $A[1]$ ,  $A[i]$ )
        heapsize( $A$ ) = heapsize( $A$ ) - ۱
        maxheapify( $A$ , ۱)
    }
```

مرتب‌سازی مرحله ۱ الگوریتم $O(n)$ و مرتبه مرحله ۲ الگوریتم $O(n \lg n)$ است. پس مرتبه مرتب‌سازی هرمی $O(n \lg n)$ است. توجه کنید هر بار اجرای *maxheapify* از مرتبه $O(\lg n)$ است و در مرحله ۲، $n-1$ بار این الگوریتم فراخوانی می‌شود که در نتیجه، همانطور که گفته شد، مرحله ۲ الگوریتم فوق از مرتبه $O(n \lg n)$ است.

مثال ۱۱: آرایه A را با روش هرمی، مرتب کنید.

پاسخ: مرحله ۱: با این آرایه باید هرم بیشینه بسازیم که طبق مثال ۱۹ با روش ۲، هرم

حاصل می‌شود.

۱	۲	۳	۴	۵	۶	۷	۸
۴۵	۴۰	۳۵	۲۲	۱۲	۱۰	۳۰	۱۵

مرحله ۲: باید ۷ بار عمل حذف ریشه انجام دهیم. طبق الگوریتم *Heapsort*، ابتدا $i = 8$ است و $A[1] = 45$ و $A[8] = 15$ با هم تعویض می‌شوند و $heapsize = 7$ می‌شود و سپس روی $A[1]$ عمل *maxheapify* انجام می‌شود:

۱	۲	۳	۴	۵	۶	۷	$\xrightarrow{\text{max heapify}(A,1)}$	۱	۲	۳	۴	۵	۶	۷
۱۵	۴۰	۳۵	۲۲	۱۲	۱۰	۳۰		۴۰	۲۲	۳۵	۱۵	۱۲	۱۰	۳۰

سپس $i = 7$ می‌شود و $A[1] = 40$ و $A[7] = 30$ تعویض می‌شوند $heapsize = 6$ می‌شود و هرم بازسازی می‌شود:

۱	۲	۳	۴	۵	۶	$\xrightarrow{\text{max heapify}(A,1)}$	۱	۲	۳	۴	۵	۶
۳۰	۲۲	۳۵	۱۵	۱۲	۱۰		۳۵	۲۲	۳۰	۱۵	۱۲	۱۰

سپس $i = 6$ می‌شود و $A[1] = 35$ و $A[6] = 10$ تعویض می‌شوند و $heapsize = 5$ و هرم بازسازی می‌شود:

۱	۲	۳	۴	۵	$\xrightarrow{\text{max heapify}(A,1)}$	۱	۲	۳	۴	۵
۱۰	۲۲	۳۰	۱۵	۱۲		۳۰	۲۲	۱۰	۱۵	۱۲

سپس $i = 5$ می‌شود و $A[1] = 30$ و $A[5] = 12$ تعویض می‌شوند و $heapsize = 4$ می‌شود و هرم بازسازی می‌شود:

۱	۲	۳	۴	$\xrightarrow{\text{max heapify}(A,1)}$	۱	۲	۳	۴
۱۲	۲۲	۱۰	۱۵		۲۲	۱۵	۱۰	۱۲

سپس $i = 4$ می‌شود و $A[1] = 22$ و $A[4] = 12$ تعویض می‌شوند و $heapsize = 3$ می‌شود و هرم بازسازی می‌شود:

۱	۲	۳	$\xrightarrow{\text{max heapify}(A,1)}$	۱	۲	۳
۱۲	۱۵	۱۰		۱۵	۱۲	۱۰

سپس $i = 3$ می‌شود و $A[1] = 15$ و $A[3] = 10$ تعویض می‌شوند و $heapsize = 2$ می‌شود و هرم بازسازی می‌شود:

۱	۲	$\xrightarrow{\text{max heapify}(A,1)}$	۱	۲
۱۰	۱۲		۱۲	۱۰

و در نهایت $i = 2$ می‌شود و $A[1] = 12$ و $A[2] = 10$ تعویض می‌شوند و آرایه مرتب می‌شود:

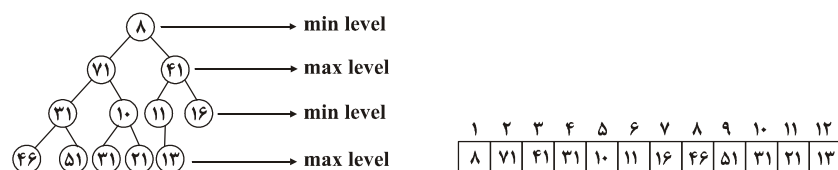
۱۰	۱۲	۱۵	۲۲	۳۰	۳۵	۴۰	۴۵
----	----	----	----	----	----	----	----

❖ **نکته:** یکی از کاربردهای هرم، ساخت صف اولویت (*priority queue*) است. در صف اولویت هر عنصر دارای یک اولویت است. و همیشه عنصری که اولویت بیشتری دارد، زودتر سرویس می‌گیرد و از صف خارج می‌شود. پس اگر هرم را براساس اولویت عناصر بسازیم یعنی معیار مقایسه و ساخت هرم، اولویت عناصر باشد، آنگاه ریشه هرم عنصری است با بیشترین اولویت بنابراین حذف ریشه معادل است با حذف عنصر با بیشترین اولویت. در نتیجه حذف و درج در صف اولویتی که با هرم ساخته می‌شود از مرتبه $O(\lg n)$ است.

❖ **نکته:** حذف عنصر مشخص، افزایش کلید، کاهش کلید، درج عنصر با کلید دلخواه، در هرم از مرتبه $O(\lg n)$ هستند.

هرم کمینه بیشینه (*minmaxheap*)

هرم کمینه بیشینه، یک درخت دودویی کامل است که اگر تهی نباشد آنگاه سطوح آن یک سطح در میان، کمینه و بیشینه هستند. ریشه در سطح کمینه (*min*) است. سطح دوم بیشینه (*max*) است، سطح سوم کمینه (*min*) است و به همین ترتیب در واقع در هر گره یک عنصر به نام کلید وجود دارد، کلید گره‌ای که در سطح کمینه است، از کلید تمام نواده‌های آن گره، کوچکتر یا مساوی است. و کلید گره‌ای که در سطح بیشینه است، از کلید تمام نواده‌های آن گره، بزرگتر یا مساوی است. در شکل ۱۴ یک هرم کمینه بیشینه به همراه نمایش آرایه آن نشان داده شده است:



شکل ۱۴ هرم کمینه بیشینه. ریشه یعنی $A[1]$ کمینه است در $A[2]$ یا $A[3]$ عنصر بیشینه است.

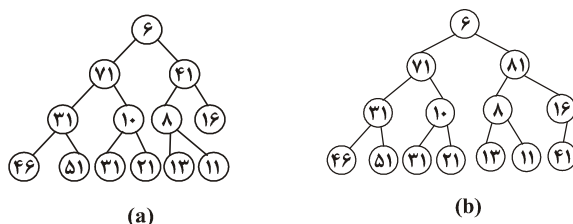
بنابراین یافتن عنصر کمینه و بیشینه در این نوع هرم از مرتبه $\theta(1)$ است، زیرا عنصر کمینه همیشه در ریشه یعنی $A[1]$ است و عنصر بیشینه در $A[2]$ یا $A[3]$ قرار دارد.

در این نوع هرم، درج یک عنصر با کلید دلخواه، حذف عنصر کمینه و حذف عنصر بیشینه همگی از مرتبه $O(\lg n)$ هستند.

درج: برای درج عنصر با کلید key ، این عنصر را در انتهای هرم (درخت کامل) قرار دهید و سپس با کلید پدرش مقایسه کنید، اگر از کلید پدرش کوچکتر (بزرگتر) بود، آنگاه از کلید همه اجدادش که در سطح max (min) هستند نیز کوچکتر (بزرگتر) است. پس فقط کافی است key را با اجدادی که در سطح min (max) هستند مقایسه کنید و در صورتی که key از جدش در سطح min (max) کوچکتر (بزرگتر) بود، عمل تعویض را انجام دهید.

مثال ۳۲: کلیدهای ۶ و ۸۱ را به ترتیب در هرم کمینه بیشینه شکل ۱۴ درج کنید.

پاسخ: ۶ را فرزند راست ۱۱ قرار می‌دهیم و چون ۶ از ۱۱ کوچکتر است پس ۶ را فقط باید با اجداد سطح min یعنی با ۱۱ و ۸ مقایسه کنیم و چون ۶ از ۱۱ و ۸ کوچکتر است، تعویض انجام می‌شود و ۶ به ریشه درخت منتقل می‌شود و شکل $a - ۱۵$ حاصل می‌شود. برای درج ۸۱ در شکل $a - ۱۵$ ، ۸۱ را فرزند چپ ۱۶ قرار می‌دهیم و چون ۸۱ از ۱۶ بزرگتر است پس ۸۱ را فقط باید با اجداد سطح max یعنی با ۴۱ مقایسه کنیم که چون ۸۱ از ۴۱ بزرگتر است تعویض می‌کنیم و شکل $b - ۱۵$ حاصل می‌شود:



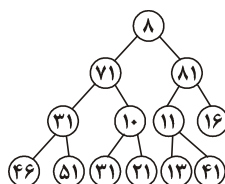
شکل ۱۵- هرم کمینه بیشینه حاصل پس از درج ۶ و ۸۱ در هرم کمینه بیشینه شکل ۱۴

حذف عنصر کمینه از هرم کمینه بیشینه: عنصر کمینه در ریشه یعنی $A[1]$ است. برای حذف این عنصر، عنصر آخر هرم که x می‌نامیم نیز باید حذف شود و برای x طبق روالی که گفته می‌شود، یک محل جدید پیدا شود. پس کلید ریشه را حذف می‌کنیم یعنی اکنون داخل ریشه هیچ عنصری نیست. عنصر آخر هیپ یعنی x را نیز حذف می‌کنیم. اگر ریشه فرزند نداشته باشد کافی است x را در ریشه قرار دهیم و کار تمام است. ولی اگر ریشه حداقل یک فرزند دارد، عنصر کمینه را در بین دو فرزند و ۴ نوه ریشه یعنی در بین عناصر $A[2]$ تا $A[7]$ می‌یابیم فرض کنید عنصر کمینه $y = A[m]$ باشد. حال اگر $x \leq y$ ، کافی است x را در ریشه قرار دهیم و کار تمام است. اگر $x > y$ و y فرزند ریشه است، از آنجایی که y در سطح max قرار دارد پس هیچ نواده‌ای ندارد و کافی است y را در ریشه و x را به جای y قرار دهیم. اگر $x > y$ و y نوه ریشه است، پدر y را p می‌نامیم، اکنون y را به ریشه منتقل کنید و x

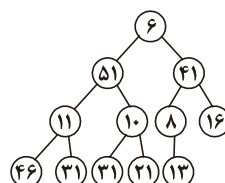
را به محل قدیم $y(A[m])$ حال اگر $x > p$ ، باید p و x را تعویض کنید. سپس تمام عملیات فوق را روی $A[m]$ انجام دهید.

مثال ۲۳: عمل حذف عنصر کمینه را روی هرم شکل $b - 15$ انجام دهید.

پاسخ: ریشه یعنی ۶ حذف می‌شود و ۴۱ به جای آن قرار می‌گیرد. عنصر کمینه در بین فرزندان و نوه‌های ریشه برابر $A[6] = 8$ است و چون $8 < 41$ باید ۸ و ۴۱ با هم تعویض شوند و چون $81 < 41$ نیاز به جابجایی ۸۱ و ۴۱ نیست. حال باید روی $A[6]$ (که اکنون ۴۱ است) همین عمل را انجام دهیم. عنصر کمینه در بین فرزندان $A[6]$ ، برابر ۱۱ است که باید با $A[6]$ تعویض شود و شکل نهایی هرم به صورت زیر است:



حذف عنصر بیشینه از هرم کمینه بیشینه: عنصر بیشینه هرم $A[1..n]$ یا در $A[2]$ یا در $A[3]$ قرار دارد. برای حذف این عنصر، آخرین عنصر هرم را به جای آن قرار دهید و مشابه روال گفته شده برای حذف عنصر کمینه، هرم را بازسازی کنید. مثلاً در هرم شکل $b - 15$ ، عنصر بیشینه ۸۱ است که با حذف آن، ۴۱ در $A[3]$ قرار می‌گیرد و چون ۴۱ در سطح max است و از همه نواده‌هایش بزرگتر است، کار تمام است. فرض کنید می‌خواهیم عنصر بیشینه را در هرم شکل $a - 15$ حذف کنیم. عنصر بیشینه $A[2] = 71$ است که با حذف آن باید ۱۱ را در $A[2]$ قرار دهیم، حال در بین فرزندان و نوه‌های $A[2]$ ، عنصر بیشینه را بیابیم که ۵۱ می‌باشد و ۱۱ و ۵۱ را تعویض کنیم حال چون ۱۱ در سطح max قرار گرفته است و از پدرش یعنی ۳۱ کوچکتر است باید با ۳۱ تعویض شود که شکل مقابل حاصل می‌شود:

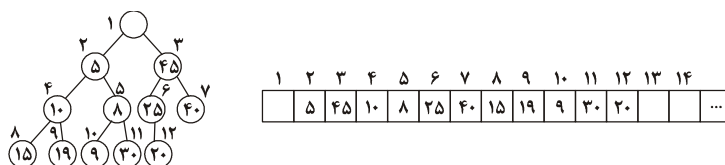


نکته: ساخت هرم کمینه بیشینه، روالی مشابه ساخت هرم کمینه یا هرم بیشینه دارد و با مرتبه $\theta(n)$ می‌توان با n عدد، یک هرم کمینه بیشینه ساخت.

◀ **نکته:** کاربرد اصلی هرم کمینه بیشینه، ساخت صف اولویت دو طرفه (*double ended priority queue*) است. در این نوع صف، باید بتوان عملیات درج یک عنصر با کلید دلخواه، حذف عنصر با کمترین کلید و حذف عنصر با بیشترین کلید را با سرعت خوبی انجام داد، که تمام این عملیات در هرم کمینه بیشینه با مرتبه $O(\lg n)$ قابل انجام است. همچنین یافتن عنصر کمینه و عنصر بیشینه در این نوع هرم از مرتبه $O(1)$ است.

: (Double ended heap) Deap

یک درخت دودویی کامل است که در ریشه آن هیچ کلیدی (مقداری) وجود ندارد. زیر درخت چپ ریشه یک هرم کمینه است. زیردرخت راست ریشه یک هرم بیشینه است و کلید هر نود در زیردرخت چپ از کلید نود نظیرش در زیردرخت راست، کوچکتر یا مساوی است. البته ممکن است برخی نودها در زیر درخت چپ، نظیری در زیردرخت راست نداشته باشند (نودهای سطح آخر ممکن است چنین باشند چون درخت لزوماً پر نیست)، اینگونه نودها باید از کلید نود نظیر پدرشان، کوچکتر یا مساوی باشند. شکل ۱۶ یک *Deap* را به همراه نمایش آرایه‌ای آن، نشان داده است:



شکل ۱۶- زیردرخت با ریشه $A[2]$ ، هرم کمینه و زیردرخت با ریشه $A[3]$ هرم بیشینه است و داریم:
 $A[2] < A[3], A[4] < A[6], A[5] < A[7], A[8] < A[12], A[9] < A[6], A[10] < A[7], A[11] < A[7]$

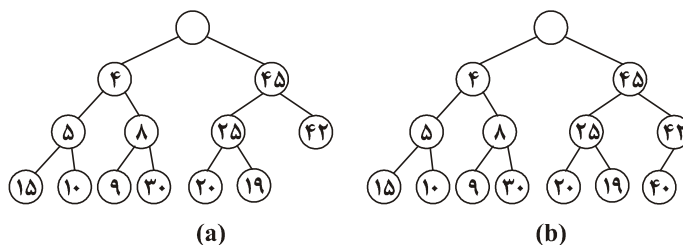
در شکل ۱۶، $A[9]$ و $A[10]$ و $A[11]$ که در زیردرخت چپ واقع هستند، متناظرشان در زیردرخت راست وجود ندارد پس کلید این نودها باید با کلید نود نظیر پدر این نودها یعنی به ترتیب با $A[6]$ و $A[7]$ و $A[7]$ مقایسه شوند.

مثال ۲۴: اگر i اندیس نودی در زیردرخت چپ (هرم کمینه) و j اندیس نود متناظر با i در زیردرخت راست (هرم بیشینه) باشد، چه رابطه‌ای بین i و j برقرار است؟

پاسخ: در شکل ۱۶ مشاهده کردیم که مثلاً نود با اندیس ۴ در زیردرخت چپ متناظر نود با اندیس ۶ در زیردرخت راست است. نود با اندیس i در عمق $\lfloor \lg i \rfloor$ قرار دارد. مثلاً نود با اندیس‌های ۳ و ۲ در عمق ۱ هستند ($\lfloor \lg 2 \rfloor = \lfloor \lg 3 \rfloor = 1$) و یا نودهای شماره ۴ تا ۷ در عمق ۲ قرار دارند ($\lfloor \lg 4 \rfloor = \lfloor \lg 7 \rfloor = 2$). در عمق $\lfloor \lg i \rfloor$ ، $\lfloor \lg i \rfloor$ نود وجود دارد. در هر عمق، برای یافتن شماره

نظیر i باید نصف نودهای آن عمق را به i اضافه کنیم پس $j = i + 2^{\lfloor \lg i \rfloor - 1}$. البته اگر $j > n$ یعنی i دارای نظیر نباشد باید $j = \left\lfloor \frac{j}{2} \right\rfloor$ قرار دهیم.

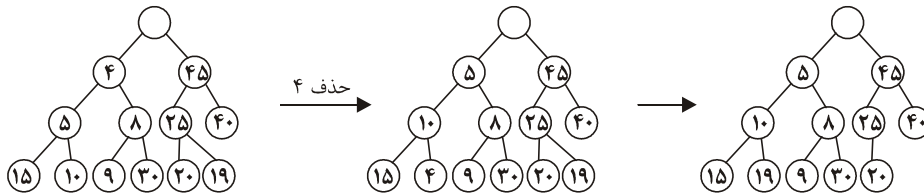
درج در Deap: برای درج عنصر با کلید x را در انتهای $Deap$ درج می‌کنیم. دو حالت وجود دارد، یا x در زیردرخت چپ درج شده است یا در زیردرخت راست. اگر x در زیردرخت چپ (راست) درج شده است، x را با کلید نود نظیرش در زیردرخت راست (چپ) مقایسه کنید و اگر x از کلید نود نظیرش بزرگتر (کوچکتر) بود، x را با نود نظیرش تعویض کنید و سپس در زیردرخت راست (چپ) همانند هرم معمولی x را با اجدادش مقایسه کنید و در صورت لزوم x را به بالا حرکت دهید. فرض کنید می‌خواهیم کلیدهای ۴ و ۴۲ را به ترتیب در دیپ شکل ۱۶ درج کنیم. $x = 4$ را در $A[13] = 25$ یعنی فرزند راست $A[6] = 25$ قرار دهیم و چون ۴ از کلید نود نظیرش یعنی $A[9] = 19$ کوچکتر است، ۴ و ۱۹ را با هم تعویض می‌کنیم و در زیر درخت چپ، ۴ را با اجدادش یعنی با ۱۰ و ۵ مقایسه و تعویض می‌کنیم و شکل $17 - a$ حاصل می‌شود. برای درج $x = 42$ ، باید ۴۲ را در $A[14] = 40$ یعنی فرزند چپ 40 قرار دهیم و چون ۴۲ از نظیرش یعنی از $A[10] = 9$ بزرگتر است پس نیازی به تعویض نیست. حال باید ۴۲ را با اجدادش مقایسه کنیم که چون $42 > 40$ پس ۴۲ و 40 تعویض می‌شوند و چون $42 > 45$ تعویض ۴۲ و 45 صورت نمی‌گیرد و شکل $17 - b$ حاصل می‌شود.



شکل ۱۷ - (a) دیپ حاصل پس از درج ۴ در دیپ شکل ۱۶ (b) دیپ حاصل پس از درج ۴۲ در دیپ شکل ۱۷ - a.

حذف عنصر کمینه و بیشینه در دیپ: عنصر کمینه در $A[2]$ و عنصر بیشینه در $A[3]$ قرار دارد. فرض کنید می‌خواهیم از دیپ شکل $17 - a$ ، عنصر کمینه یعنی ۴ را حذف کنیم. نحوه حذف به این صورت است که ۴ را با فرزندانش مقایسه کنید و مینیمم بین این دو را به جای ۴ در $A[2]$ قرار دهید و حال ۴ را با ۱۵ و ۱۰ مقایسه کنید و ۱۰ را در $A[4]$ قرار دهید. یعنی ۴ به یکی از برگ‌های درخت منتقل می‌شود حال آخرین کلید در درخت یعنی ۱۹ را به جای ۴ در $A[9]$ قرار دهید و عمل شبیه درج را برای ۱۹ انجام دهید. یعنی ۱۹ را با نظیرش (۲۵) مقایسه کنید و چون $19 < 25$ نیاز به تعویض

نیست. و چون ۱۹ از پدرش یعنی ۱۰ نیز کوچکتر است، کار تمام است. در شکل ۱۸ این عملیات نشان داده شده‌اند:



شکل ۱۸

پس نحوه حذف عنصر کمینه یا بیشینه به این صورت است که این عنصر را با مقایسه با فرزندانش به پایین منتقل کنید و سپس آخرین برگ درخت را به جای آن قرار دهید و عمل شبیه درج را انجام دهید.

◀ **نکته:** درج یک عنصر با کلید دلخواه، حذف عنصر کمینه، حذف عنصر بیشینه در دیپ از مرتبه $O(\lg n)$ هستند.

◀ **نکته:** یافتن عنصر کمینه و بیشینه در دیپ از مرتبه $\theta(1)$ است.

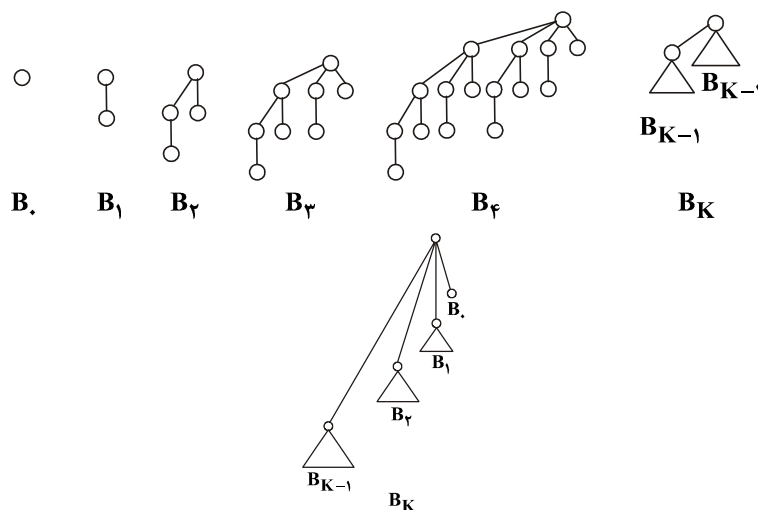
◀ **نکته:** ساخت دیپ با n عنصر داده شده، از مرتبه $\theta(n)$ است.

◀ **نکته:** کاربرد اصلی دیپ ساخت صف اولویت دوطرفه است.

◀ **نکته:** دیپ و هرم کمینه بیشینه کاربرد مشابه دارند و مرتبه عملیات مختلف روی هر دو یکسان است ولی دیپ ضرایب بهتری دارد و از هرم کمینه بیشینه، کمی سریع‌تر است.

درخت دوجمله‌ای (binomial tree)

درخت دوجمله‌ای را با B_k نشان می‌دهیم و به آن درخت دوجمله‌ای درجه k گوییم. B_0 یعنی درخت دوجمله‌ای درجه صفر فقط یک نود دارد. B_1 از اتصال دو B_0 حاصل می‌شود، در واقع اگر B_0 را فرزند B_1 دیگری قرار دهیم، B_1 حاصل می‌شود. حال اگر ریشه یک B_1 را فرزند ریشه B_1 دیگری قرار دهیم، B_2 حاصل می‌شود. به طور کلی اگر ریشه یک B_{k-1} را فرزند ریشه B_{k-1} دیگری قرار دهیم B_k حاصل می‌شود. شکل ۱۹ تعدادی درخت دوجمله‌ای را نشان می‌دهد. البته می‌توان B_k را طور دیگری نیز تعریف کرد. مثلاً به B_2 توجه کنید، زیردرخت اول ریشه‌ی آن B_1 و زیردرخت دوم ریشه‌ی آن B_0 است. یا در B_3 ، زیردرختان ریشه به ترتیب B_2 و B_1 و B_0 هستند. پس در B_k ، زیردرختان ریشه $B_{k-1}, B_{k-2}, \dots, B_0$ هستند.

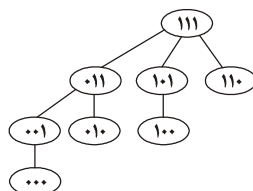


شکل ۱۹

درخت دوجمله‌ای B_k شامل 2^k نود است، ارتفاع این درخت برابر k است و در عمق i تعداد $\binom{k}{i}$ نود وجود دارد و درجه ریشه برابر k است و درجه سایر نودها کمتر از k است. مثلاً به B_4 توجه کنید، ارتفاع B_4 برابر ۴ است (۵ سطح دارد). در عمق ۰ و ۱ و ۲ و ۳ و ۴ به ترتیب $\binom{4}{0} = 1, \binom{4}{1} = 4, \binom{4}{2} = 6, \binom{4}{3} = 4, \binom{4}{4} = 1$ نود دارد. درجه ریشه در B_4 برابر ۴ است و درجه سایر نودها از ۴ کمتر است.

نکته: اگر نودهای B_k را به صورت پس‌ترتیب پیمایش کنیم و هر نود را با k بیت و با شروع از صفر، شماره‌گذاری کنیم آنگاه نودی که در عمق صفر (ریشه) است شامل k تا بیت ۱ می‌باشد، نودهای عمق ۱ هر کدام شامل $k-1$ بیت ۱ هستند و به طور کلی نودهای عمق d شامل $k-d$ بیت یک هستند. مثلاً در شکل ۲۰، B_3 را به صورت پس‌ترتیب پیمایش کرده‌ایم و نودها را با ۳ بیت شماره‌گذاری کرده‌ایم. ریشه ۱۱۱ است که سه بیت ۱ دارد. نودهای عمق یک هر کدام ۲ بیت $(3-1)$ یک دارند و به همین ترتیب در ضمن تعداد یک‌های سمت راست هر نود تا اولین صفر (در صورت وجود صفر)، درجه نود را نشان می‌دهد. مثلاً درجه ریشه در B_3 برابر ۳ است که برچسب آن ۱۱۱ می‌باشد.

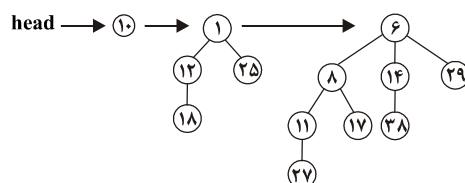
و یا نود با برچسب ۰۱۱ درجه ۲ است و به همین ترتیب



شکل ۲۰

هیپ دوجمله‌ای کمینه (minbinomial heap)

هیپ دوجمله‌ای از تعدادی درخت دوجمله‌ای تشکیل شده است. در هر نود یک کلید وجود دارد. هر درخت دوجمله‌ای در هیپ دوجمله‌ای دارای خاصیت کمینه است یعنی کلید هر نود از کلید فرزنداناش کوچکتر یا مساوی است. شکل ۲۱ یک هیپ دوجمله‌ای شامل سه درخت دوجمله‌ای (B_0, B_1, B_2) را نشان می‌دهد.



شکل ۲۱

در هیپ دوجمله‌ای، ریشه درخت‌ها به هم لینک می‌شوند و می‌توان یک اشاره‌گر به ریشه درختی که شامل عنصر کمینه است در نظر گرفت که در این صورت یافتن عنصر کمینه از مرتبه $O(1)$ می‌باشد. در هیپ دوجمله‌ای، درخت‌های دوجمله‌ای $B_0, B_1, B_2, \dots$ یا وجود ندارند یا فقط یکبار ظاهر شده‌اند. یعنی مثلاً نمی‌توان هیپ دوجمله‌ای تعریف کرد که شامل دوتا B_1 باشد. هیپ دوجمله‌ای شکل ۲۱ شامل ۱۳ نود می‌باشد و اگر ۱۳ را باینری بنویسید، متوجه می‌شوید که این هیپ شامل چه درخت‌هایی است. با توجه به اینکه $13 = (1101)_2$ پس این هیپ شامل B_0 و B_1 و B_2 است. در واقع اگر بیت موقعیت 2^k در صورت دودویی، ۱ باشد یعنی هیپ دوجمله‌ای شامل درخت B_k است.

مثال ۲۵: هیپ دوجمله‌ای که شامل ۴۲ نود است، از چه درخت‌های دوجمله‌ای، تشکیل شده

است؟

پاسخ: باتوجه به اینکه $42 = (101010)_2$ پس این هیپ شامل B_1 و B_3 و B_5 می‌باشد.

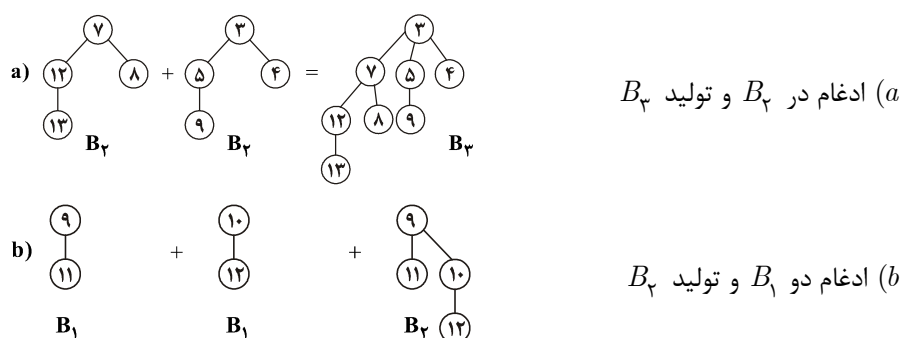
مثال ۲۶: هیپ دوجمله‌ای ۶۳ نودی شامل چه درخت‌های دوجمله‌ای است؟

پاسخ: با توجه به اینکه $۶۳ = (۱۱۱۱۱۱)_۲$ پس این هیپ شامل $B_۵, B_۴, B_۳, B_۲, B_۱, B_۰$ است یعنی شامل ۶ درخت دوجمله‌ای است.

نتیجه: هیپ دوجمله‌ای n نودی شامل حداکثر $۱ + \lfloor \lg n \rfloor$ درخت دوجمله‌ای است.

نکته: مزیت هیپ دوجمله‌ای نسبت به هیپ دودویی این است که عمل ادغام دو هیپ دوجمله‌ای با مجموع n نود را می‌توان با مرتبه $O(\lg n)$ انجام داد. ولی ادغام دو هیپ دودویی با مجموع n نود از مرتبه $O(n)$ است.

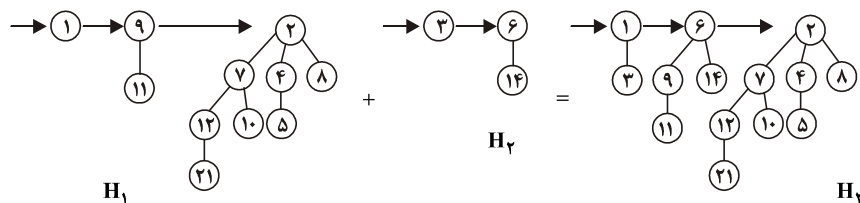
ادغام (merge): ادغام دو درخت دوجمله‌ای هم درجه، عمل بسیار ساده‌ای است. برای ادغام دو تا B_k ، ریشه آن‌ها را مقایسه کنید و ریشه‌ای که کلید بزرگتری دارد فرزند ریشه دیگری قرار دهید و بدین ترتیب B_{k+1} تولید می‌شود. به چند نمونه ادغام درخت‌های دوجمله‌ای در شکل ۲۲ توجه کنید:



شکل ۲۲

برای ادغام دو هیپ دوجمله‌ای، درخت‌های دوجمله‌ای موجود در دو هیپ را به ترتیب بررسی می‌کنیم، و اگر دو درخت هم درجه (یعنی دو تا B_k) مشاهده کردیم، باید آن‌ها را ادغام کنیم و B_{k+1} تولید کنیم. توجه کنید در هیپ دوجمله‌ای، درخت‌های دوجمله‌ای به ترتیب درجه‌شان قرار گرفته‌اند، یعنی اولین درخت $B_۰$ (در صورت وجود) درخت بعدی $B_۱$ (در صورت وجود) و به همین ترتیب می‌باشد. پس برای ادغام دو هیپ دوجمله‌ای، اگر هر دو شامل $B_۰$ باشند، دو تا $B_۰$ را ادغام کرده $B_۱$ تولید می‌کنیم. حال ممکن است حداکثر ۳ تا $B_۱$ وجود داشته باشد (ممکن است خود هیپ‌ها شامل $B_۱$ باشند و یک $B_۱$ نیز ما تولید کرده‌ایم) که دوتای آن‌ها را ادغام می‌کنیم و $B_۲$ تولید می‌شود. و به همین ترتیب عمل ادغام درخت‌ها را ادامه می‌دهیم. با توجه به اینکه در هر هیپ

حداکثر $1 + \lceil \lg n \rceil$ درخت دوجمله‌ای وجود دارد، پس مرتبه ادغام دو هیپ دوجمله‌ای $O(\lg n)$ است. شکل ۲۳ ادغام دو هیپ دوجمله‌ای را نشان می‌دهد.



شکل ۲۳- ادغام H_1 و H_2 و تولید H_3

ادغام دو هیپ دوجمله‌ای شبیه جمع دو عدد دودویی است. مثلاً H_1 و H_2 به ترتیب ۱۱ و ۳ نود دارند که $11 = (1011)_2$, $3 = (0011)_2$ ، به جمع این دو عدد توجه کنید:

$$\begin{array}{r} 11 \\ + 1011 \\ \hline 0011 \\ \hline 1110 \end{array}$$

بیت سمت راست هر دو عدد ۱ است پس در هر دو هیپ، B_0 وجود دارد.

با جمع این دو بیت، حاصل جمع صفر می‌شود پس در هیپ حاصل B_1 وجود ندارد. یک رقم نقلی تولید شده است که باید با بیت دوم از سمت راست جمع شود و چون این بیت‌ها یک هستند یعنی ۳ تا B_1 وجود دارد و به همین ترتیب حاصل جمع $(1110)_2$ است که یعنی هیپ حاصل B_1 و B_2 وجود دارد.

درج در هیپ دوجمله‌ای: برای درج کلید x ، کافی است کلید x را یک هیپ دوجمله‌ای شامل یک نود در نظر بگیریم و با هیپ دوجمله‌ای اصلی، ادغام کنیم. پس مرتبه درج در بدترین حالت $O(\lg n)$ است ولی اگر n عمل درج پشت سر هم انجام دهیم، آنگاه هزینه استهلاکی عمل درج $O(1)$ است (هزینه استهلاکی یعنی هزینه n عمل درج را جمع کنیم و تقسیم به n کنیم).

یافتن عنصر کمینه (مینیمم) در هیپ دوجمله‌ای: مینیمم در ریشه یکی از درخت‌های دوجمله‌ای موجود در هیپ است. اگر اشاره‌گری به مینیمم داشته باشیم آنگاه یافتن مینیمم از مرتبه $O(1)$ است ولی اگر آدرس مینیمم را نداشته باشیم، یافتن مینیمم از مرتبه $O(\lg n)$ است زیرا در هیپ دوجمله‌ای n نودی، حداکثر $\lceil \lg n \rceil$ درخت وجود دارد پس حداکثر $\lceil \lg n \rceil$ ریشه باید بررسی شوند.

حذف مینیمم از هیپ دوجمله‌ای: ابتدا مینیمم را پیدا می‌کنیم که در ریشه یکی از درخت‌هاست (اگر اشاره‌گر به مینیمم را داشته باشیم، محل مینیمم مشخص است) سپس مینیمم را یعنی ریشه یکی از درخت‌ها را حذف می‌کنیم، سپس ریشه زیردرخت‌هایی که با حذف مینیمم به دست می‌آیند را با هم لینک می‌کنیم و یک هیپ دوجمله‌ای می‌سازیم و این هیپ را با هیپ اصلی ادغام می‌کنیم پس مرتبه این عمل $O(\lg n)$ است.

نکته: می‌توان هر نود را با ساختاری به شکل مقابل نشان داد که p اشاره‌گر به پدر، $child$ اشاره‌گر به چپ‌ترین فرزند نود، $sibling$ اشاره‌گر به برادر سمت راست، key کلید موردنظر و $degree$ درجه نود را نشان می‌دهد.

P
key
degree
child
sibling

هیپ فیبوناچی (مینیمم)

برای عملیاتی مثل درج، ادغام و کاهش یک کلید، این هیپ از هیپ کمینه دودویی عملکرد بهتری دارد. ولی عمل حذف یک کلید یا حذف مینیمم در هر دو هیپ، از یک مرتبه است. از ذکر ساختار این هیپ خودداری می‌کنیم ولی جدول زیر مرتبه برخی عملیات را برای برخی ساختار داده‌ها نشان داده است. برای هیپ فیبوناچی، مرتبه هر عمل به طور استهلاکی آمده است. در واقع این هیپ وقتی خوب است که عملیات به تعداد بار زیادی انجام شوند. مثلاً در الگوریتم پریم و دایجسترا با کمک این نوع هیپ به جای هیپ دودویی می‌توان مرتبه را از $O(e \cdot \lg n)$ به $O(e + n \cdot \lg n)$ کاهش داد.

<i>operation</i>	<i>linked list</i>	<i>binray min heap</i>	<i>binomial min heap</i>	<i>fibonacci min heap</i>
<i>is - empty</i>	۱	۱	۱	۱
<i>insert</i>	۱	$\lg n$	$\lg n$	۱
<i>find min</i>	n	۱	$\lg n$	۱
<i>del - min</i>	n	$\lg n$	$\lg n$	$\lg n$
<i>decreasekey</i>	n	$\lg n$	$\lg n$	۱
<i>union(merge)</i>	۱	n	$\lg n$	۱

Treaps

اگر n عدد به BST درج کنیم ممکن است ارتفاع درخت خیلی زیاد شود و مثلاً درخت، مورب شود. اگر همه اعداد را همزمان در اختیار داشته باشیم می‌توانیم آنها را به صورت تصادفی جابجا کنیم و سپس BST بسازیم که در این صورت احتمال آنکه درخت حاصل مورب یا شبیه مورب شود خیلی ناچیز است. ولی اگر اعداد یکی یکی وارد شوند دیگر نمی‌توان آنها را جابجا کرد و به محض ورود هر عدد باید آن را در BST درج کنیم، برای آنکه احتمال مورب شدن درخت را کم کنیم یک پیشنهاد این است که به هر کلید که وارد می‌شود، یک عددی به نام اولویت به صورت تصادفی نسبت دهیم و درخت را طوری بسازیم که روی کلیدها خاصیت BST داشته باشد و روی اولویت‌ها خاصیت $Mintree$ (یعنی اولویت هر نود از اولویت فرزندانش کمتر یا مساوی باشد) داشته باشد. یعنی اگر کلید و اولویت عنصر x را $x \cdot key$ و $x \cdot priority$ بنامیم داشته باشیم:

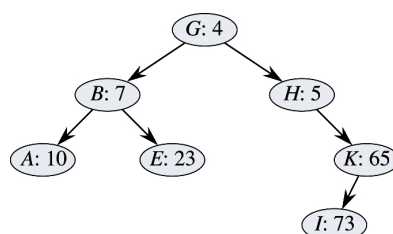
- اگر v فرزند چپ u است آنگاه $u \cdot key < v \cdot key$

- اگر v فرزند راست u است آنگاه $u \cdot key > v \cdot key$

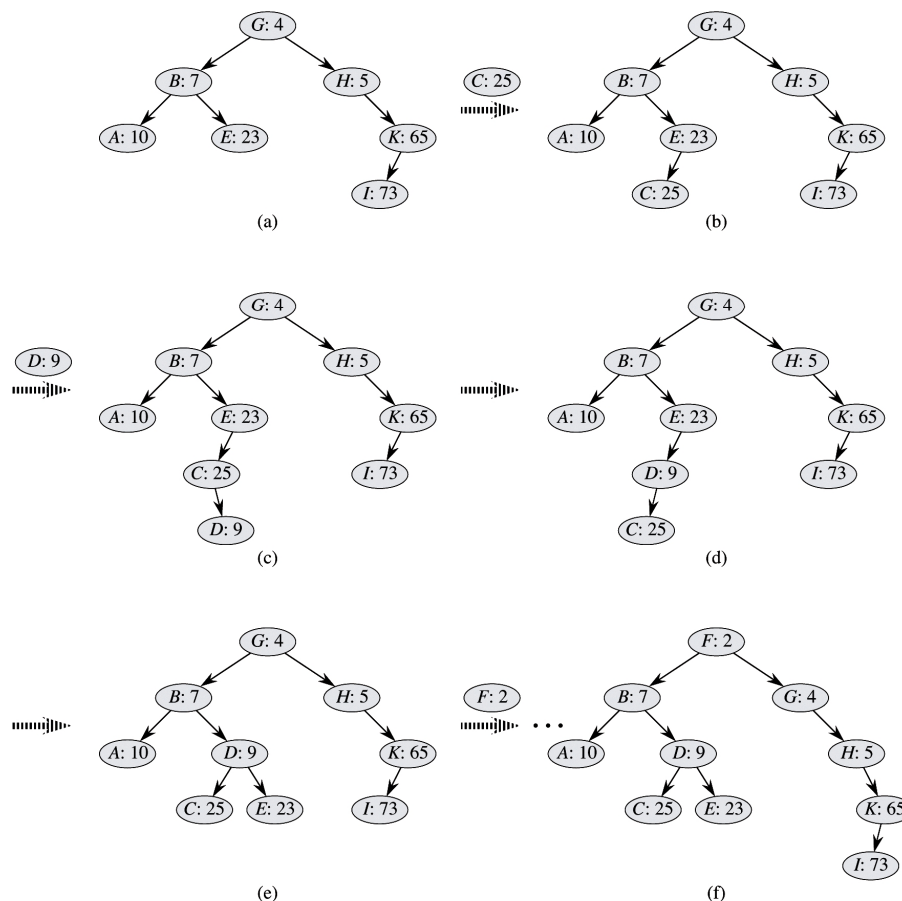
- اگر v فرزند u است آنگاه $u \cdot priority > v \cdot priority$

شکل زیر یک $Treap$ را نشان می‌دهد که کلیدها حروف الفبا هستند و اولویت‌ها اعداد

هستند:



برای درج در $Treap$ ابتدا براساس کلید طبق جستجوی BST ، محل عنصر را می‌یابیم و درج می‌کنیم، حال اگر اولویت عنصر جدید با پدرش مشکل داشت، پدر عنصر جدید را دوران می‌دهیم و این عمل را تا زمانی که عنصر به محل درست وارد شود ادامه می‌دهیم. شکل‌های زیر چند عمل درج را نشان می‌دهند.



البته اگر کلیدها متمایز باشند و اولویت‌ها نیز متمایز باشند، *Treap* منحصر به فرد است. همچنین ارتفاع مورد انتظار برای *Treap* برابر $\theta(Lgn)$ است پس عملیات پایه‌ای روی *Treap* در حالت متوسط از مرتبه $\theta(Lgn)$ هستند. همچنین ثابت می‌شود که می‌توان با روشی تعداد دوران را هنگام درج یک عنصر کاهش داد و در حالت متوسط کمتر از ۲ دوران برای هر درجی کافی است.

کد هافمن

اگر بخواهیم متنی شامل n کاراکتر را به صورت باینری کد کنیم، $\lceil \log_2^n \rceil$ بیت به ازای هر کاراکتر نیاز است مثلاً برای کدگذاری حروف الفبای انگلیسی (۲۶ حرف)، ۵ بیت نیاز است.

در این حالت هر کاراکتر دارای طول بیتی یکسان خواهد بود. مثلاً $a = ۰۰۰۰۰$ و $b = ۰۰۰۰۱$ و به راحتی می‌توان چنین متنی را از حالت کد خارج کرد (*decode*) زیرا هر ۵ بیت نشان دهنده یک کاراکتر است.

کد هافمن یک کد با طول متغیر است. به این معنی که به هر کاراکتر رشته بیتی با طول متفاوت نسبت داده می‌شود. مثلاً $a = ۰۰$ و $b = ۱۰۱$ و $c = ۱۰۰۰$ و

این تخصیص بیت باید دارای خاصیت پیشوند آزاد (*prefix-free*) باشد. یعنی هیچ کاراکتری نباید پیشوند هیچ کاراکتر دیگری باشد. مثلاً $a = ۰۰$ و $b = ۰۰۱$ نادرست است. هافمن با استفاده از درخت دودویی کدی پیشوند آزاد بهینه تولید می‌کند. هافمن براساس تعداد تکرار کاراکترها، به آنها بیت نسبت می‌دهد.

کاراکترهایی که فراوانی آنها زیاد است، رشته بیتی کوچک دریافت می‌کنند. با یک مثال روش هافمن را توضیح می‌دهیم.

مثال ۲۷: فرض کنید متنی شامل حروف a و b و c و d و e و f می‌باشد و فراوانی این حروف در جدول زیر آمده است، می‌خواهیم کدی بهینه برای این حروف تولید کنیم.

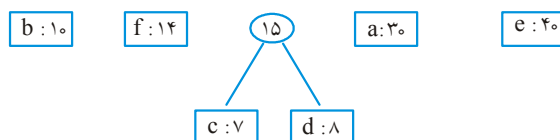
کاراکتر	a	b	c	d	e	f
فراوانی	۳۰	۱۰	۷	۸	۴۰	۱۴

پاسخ: ابتدا دو کاراکتر با کمترین فراوانی انتخاب می‌شوند و با آنها یک درخت ساخته می‌شود و جمع فراوانی آنها در ریشه درخت نوشته می‌شود و آن دو فراوانی از جدول حذف می‌شوند و جمع فراوانی آنها به جدول اضافه می‌شود. سپس دو فراوانی مینیمم دیگر انتخاب می‌شوند و به همین ترتیب. مراحل اجرا در ادامه آمده است.

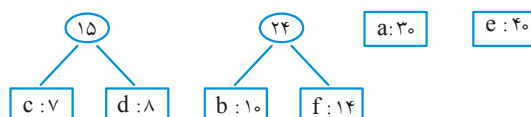
(الف) فراوانی‌ها را به ترتیب صعودی مرتب می‌کنیم:

c: ۷	d: ۸	b: ۱۰	f: ۱۴	a: ۳۰	e: ۴۰
------	------	-------	-------	-------	-------

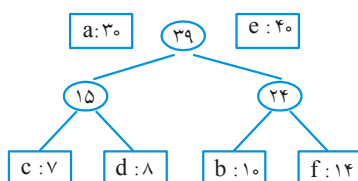
ب) c و d انتخاب می‌شوند و مجموع فراوانی آنها یعنی ۱۵ بعد از ۱۰ قرار می‌گیرد:



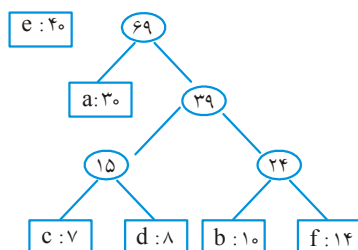
ج) حال b و f انتخاب می‌شوند و با آنها درخت ساخته می‌شود:



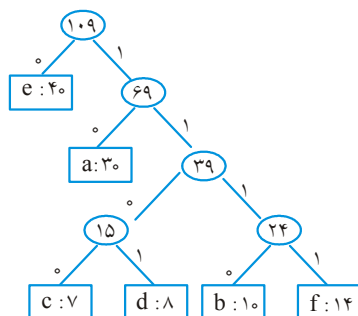
د) با ۱۵ و ۲۴ درخت ساخته می‌شود:



ه) با ۳۰ و ۳۹ درخت ساخته می‌شود:



و) در نهایت با ۴۰ و ۶۹ درخت می‌سازیم و بعد از ساخت درخت، روی یال‌های چپ و روی یال‌های راست ۱ می‌نویسیم. بدین ترتیب کد هر کاراکتر بدست می‌آید:



کد حروف مختلف عبارتند از: $e = 0$ و $a = 10$ و $c = 1100$ و $d = 1101$ و $b = 1110$ و $f = 1111$

الگوریتم هافمن برای عناصر با فراوانی مینیمم از *minheap* استفاده می‌کند. به این صورت که با فراوانی‌ها یک *minheap* (صف اولویت) می‌سازد و با عمل حذف ریشه، مینیمم را انتخاب می‌کند. کد این الگوریتم به شکل زیر است:

HUFFMAN(C)

```
{
   $n = |C|$  // مجموعه الفباست که  $n$  نوع کاراکتر دارد
   $Q = C$  // صف اولویت است که با minheap و با مرتبه  $O(n)$  ساخته می‌شود
  for ( $i = 1$  to  $n - 1$ )
  {
    allocate a new node  $z$ 
    نود  $z$  ساخته می‌شود و دو تا مینیمم  $x$  و  $y$  چپ و راست  $z$  قرار می‌گیرند.
     $Left(z) = x = extract\_min(Q)$ 
     $right(z) = y = extract\_min(Q)$ 
     $f(z) = f(x) + f(y)$  //  $f(x)$  فراوانی  $x$  است.
    insert ( $Q, z$ )
  }
}
```

مرتبه این الگوریتم $O(n \lg n)$ است. زیرا ساخت هیپ $O(n)$ است و سپس $n - 1$ بار ۲ عمل حذف و ۱ عمل درج در هیپ انجام می‌شود که مرتبه هر حذف و درج $O(\log n)$ است.

🔑 **نکته:** در درخت هافمن، نود یک فرزندی وجود ندارد، چون در غیر این صورت کد بهینه تولید نمی‌شود.

تمرین: الگوریتم هافمن را به کلمه کد مبنای ۳ تعمیم دهید (یعنی از سمبول‌های ۰ و ۱ و ۲ استفاده کنید)

تمرین: فرض کنید یک فایل داده‌ای شامل ۲۵۶ کاراکتر است که با روش عادی هر یک با ۸ بیت نمایش داده می‌شوند. نشان دهید اگر تعداد بیشترین تکرار از کمترین تکرار، کمتر از ۲ برابر باشد، کد هافمن مزیتی به کد عادی ثابت ۸ بیتی ندارد.

تمرین

۱- یک هیپ d تایی (d -ary heap) شبیه یک هیپ باینری است با این تفاوت که غیربرگ‌ها به جای ۲ فرزند، d فرزند دارند.

(a) چگونه یک هیپ d تایی را در یک آرایه ذخیره کنیم؟

(b) ارتفاع یک هیپ d تایی با n عنصر چند است؟

(c) تابع $EXTRACT-MAX$ را روی یک ماکزیمم هیپ d تایی پیاده‌سازی کنید و مرتبه آن را بیابید.

(d) مرتبه $INSERT$ در یک هیپ d تایی چیست؟

(e) تابع $INCREASE-KEY(A, i, K)$ ابتدا $A[i] \leftarrow \max(A[i], k)$ را انجام می‌دهد و سپس

ساختار هیپ ماکزیمم d تایی را تنظیم می‌کند. مرتبه این تابع چیست؟

۲- پس از حذف می‌نیمم از $MinMax$ heap مقابل، درخت حاصل را بیابید.

۱	۲	۳	۴	۵	۶	۷	۸	۹	۱۰	۱۱	۱۲
۵	۶۰	۱۷	۲۵	۱۸	۱۰	۱۴	۵۰	۴۰	۲۰	۳۰	۱۵

۳- در $Deap$ مقابل پس از حذف می‌نیمم، $Deap$ حاصل را بیابید.

۱	۲	۳	۴	۵	۶	۷	۸	۹	۱۰	۱۱	۱۲	۱۳
	۴	۴۵	۵	۸	۲۵	۴۰	۱۵	۱۰	۹	۳۰	۲۰	۱۹

پاسخ:

۱- a ریشه $A[۱]$ ، فرزندانش $A[۲]$ تا $A[d+۱]$. فرزندان اینها $A[d+۲]$ تا $A[d^۲+d+۱]$ و به همین ترتیب داریم:

$$parent(i) : \left\lfloor \frac{i-۲}{d} \right\rfloor + ۱$$

(فرزند j ام نود i) $child(i, j) : d(i-۱) + j + ۱$ ($۱ \leq j \leq d$)

$$(b) \theta(\log_d^n)$$

(c) مرتبه بدترین حالت $\theta(d \cdot \log_d^n)$

(d) مرتبه بدترین حالت $\theta(\log_d^n)$

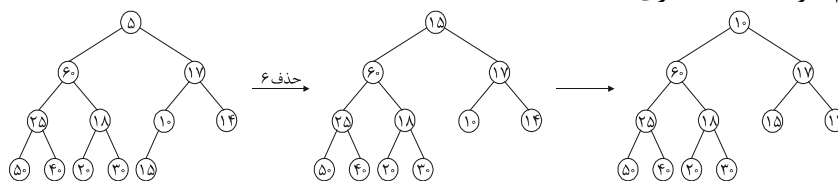
(e) بدترین حالت $\theta(\log_d^n)$ است:

$$A[i] = \max(A[i], k)$$

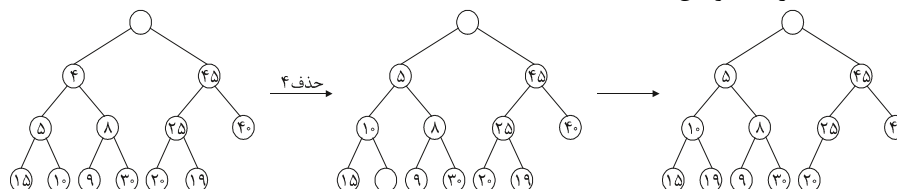
while($i > ۱$ and $A[PARENT(i)] < A[i]$)

$$\{swap(A[i], A[PARENT(i)]), i = PARENT(i)\}$$

۲- می‌نیمم ریشه (۵) است که پس از حذف آن، آخرین عدد یعنی ۱۵ به جای آن قرار می‌گیرد. حال باید در بین ۲ فرزند و ۴ نوه ریشه، می‌نیمم را بیابیم که ۱۰ است و چون $۱۰ < ۱۵$ ، ۱۰ و ۱۵ را با هم تعویض می‌کنیم. حال پدر ۱۵ که ۱۷ است و $۱۵ < ۱۷$ ، پس مشکلی نیست (در غیر این صورت تعویض انجام می‌شد). حال باید عملی که روی ریشه درخت انجام شد روی ۱۵ انجام شود که اینجا نیازی نیست:



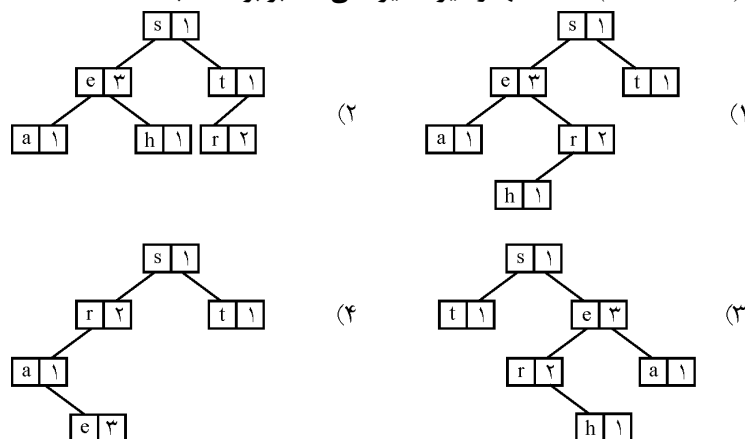
۳- برای حذف می‌نیمم ($A[2] = 4$) ابتدا ۴ را با ۵ و سپس با ۱۰ تعویض می‌کنیم (یعنی می‌نیمم را به یک برگ منتقل می‌کنیم) سپس آخرین عنصر یعنی ۱۹ را به جای آن در $A[9]$ قرار می‌دهیم حال برای ۱۹ عمل شبیه درج انجام می‌دهیم یعنی ۱۹ را با نظیرش (۲۵) مقایسه می‌کنیم و چون $۱۹ < ۲۵$ نیاز به تعویض نیست. حال ۱۹ را با پدرش مقایسه می‌کنیم و چون $۱۹ < ۱۰$ نیاز به تعویض نیست.



سؤال‌های طبقه‌بندی شده فصل ششم

- ۱- در یک درخت جستجوی باینری کدام عمل در $O(1)$ انجام می‌شود؟
 - (۱) بررسی تهی بودن درخت
 - (۲) پیدا کردن کمترین مقدار
 - (۳) حذف یک عنصر
 - (۴) درج کردن یک عنصر جدید
- ۲- چنانچه بخواهیم داده‌های تکراری از لیستی را حذف کنیم، از کدام ساختار داده‌ای برای لیست مزبور، استفاده می‌کنیم؟
 - (۱) درخت جستجوی دودویی
 - (۲) درخت $heap$
 - (۳) پشته
 - (۴) صف
- ۳- یک درخت جستجوی دودویی اگر دارای n عضو و عمق K باشد، کدام رابطه زیر برای دستیابی به یک عضو این درخت مصداق دارد؟
 - (۱) کوچکتر خواهد بود از $\log_K^{N+1}$
 - (۲) حداکثر برابر خواهد بود با $\log_K^N$
 - (۳) حداکثر برابر خواهد بود با K
 - (۴) هیچکدام
- ۴- یک مجموعه A از اعداد صحیح دو به دو نامساوی با n عنصر داده شده است. می‌خواهیم ساختمان داده‌ای برای A طراحی کنیم تا اعمال زیر را بتوان همواره در $O(\log_p^n)$ انجام داد:
 - $Insert(x): A$ درج عنصر x در
 - $Delete(x): A$ حذف عنصر x از
 - $Search(x): A$ جستجو برای پیدا کردن x در
 - $Next(x): A$ پیدا کردن عدد بعدی x (کوچکترین عدد بزرگتر از x) در
 کدام یک از ساختمان‌های داده زیر جواب است؟ (دولتی ۷۴)
 - (۱) درخت دودویی جستجو
 - (۲) درخت دودویی جستجوی متوازن
 - (۳) لیست مرتب
 - (۴) درخت نیمه مرتب ($heap$)
- ۵- کدام یک از گزینه‌های زیر در ارتباط با هر درخت دودویی جستجو با n عنصر غلط است؟ (دولتی ۷۴)
 - (۱) در حذف تعدادی عناصر از درخت، ترتیب حذف تأثیری در درخت حاصل ندارد.
 - (۲) متوسط ارتفاع کلیه درخت‌های دودویی جستجو با n المان متناسب است با $\log_p^n$
 - (۳) اگر n عنصر را از قبل داشته باشیم می‌توان یک درخت دودویی با ارتفاع متناسب با $\log_p^n$ ایجاد کرد.
 - (۴) می‌توان کوچکترین عنصر را در این درخت با مرتبه $O(\log_p^n)$ حذف کرد.

۶- درخت جستجوی دودویی حاصل از لیست $(s, e, a, r, e, h, t, r, e)$ که علاوه بر نویسه‌ها (Characters) تعداد آنها را نیز ذخیره می‌کند برابر است با: (علوم کامپیوتر ۸۰)



۷- گره‌های ۷ و ۵ و ۲ و ۹ و ۶ و ۴ و ۱ و ۳ (از چپ به راست) را در یک درخت جستجوی دودویی خالی به نام T درج می‌کنیم. پیمایش پس ترتیب (Postorder) کدام است؟ (از چپ به راست) (دولتی ۸۱)

- (۱) ۳ و ۴ و ۶ و ۹ و ۷ و ۵ و ۱ و ۲ (۲) ۹ و ۷ و ۶ و ۵ و ۴ و ۳ و ۱ و ۲
(۳) ۹ و ۷ و ۵ و ۶ و ۴ و ۳ و ۱ و ۲ (۴) ۹ و ۷ و ۵ و ۶ و ۴ و ۳ و ۲ و ۱

۸- اگر اعداد ۱۰ و ۷ و ۶ و ۴ و ۱۲ و ۹ و ۱ و ۲ و ۸ و ۵ و ۳ از چپ به راست در یک درخت دودویی جستجوی تهی درج شوند، ارتفاع درخت حاصل چند است؟ (علوم کامپیوتر ۸۱)

- (۱) ۳ (۲) ۴ (۳) ۶ (۴) ۵

۹- n عنصر با کلیدهای مختلف را می‌توان با استفاده از یک درخت دودویی جستجو T که در ابتدا تهی است به صورت زیر مرتب کرد:

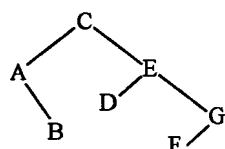
(۱) عناصر را به ترتیب در T درج کن

(۲) T را به روش «بین ترتیب» (inorder) پیمایش کن و عناصر را به همین ترتیب در خروجی بنویس. مرتبه زمان اجرای این الگوریتم به ترتیب در بهترین حالت، بدترین حالت و حالت متوسط (از راست به چپ) کدام است؟ (دولتی ۷۸)

- (۱) $O(n \log n)$, $O(n^2)$, $O(n)$ (۲) $O(n)$, $O(n \log n)$, $O(n^2)$

- (۳) $O(n \log n)$, $O(n^2)$, $O(n \log n)$ (۴) $O(n)$, $O(n^2)$, $O(n \log n)$

- ۱۰- کدام یک از دنباله‌های زیر که هر یک نشان دهنده ترتیب درج عناصر از چپ به راست در یک درخت دودویی جستجوی تهی است، درخت زیر را تولید نمی‌کند؟ (سراسری ۸۳)



(۱) $C A E G D B F$

(۲) $C A B E G D F$

(۳) $C E G A F D B$

(۴) $C E B G A D F$

- ۱۱- فرض کنید اعداد ۱ تا ۱۰۰۰ در یک درخت دودویی جستجو ذخیره شده‌اند و ما می‌خواهیم عدد ۳۶۳ را پیدا کنیم. کدامیک از ترتیب‌های زیر (از چپ به راست) نمی‌تواند بیانگر ترتیب دسترسی به عناصر درخت در این جستجو باشد؟ (دوئل ۸۰)

(۱) ۳۶۳ و ۲۴۵ و ۹۱۲ و ۲۴۰ و ۹۱۱ و ۲۰۲ و ۹۲۵

(۲) ۳۶۳ و ۳۶۲ و ۲۵۸ و ۸۹۸ و ۲۴۴ و ۹۱۱ و ۲۲۰ و ۹۲۴

(۳) ۳۶۳ و ۳۹۷ و ۳۴۴ و ۳۳۰ و ۳۹۸ و ۴۰۱ و ۲۵۲ و ۲

(۴) ۳۶۳ و ۲۷۸ و ۳۸۱ و ۳۸۲ و ۲۶۶ و ۲۱۹ و ۳۸۷ و ۳۹۹ و ۲

- ۱۲- در یک درخت دودویی جستجوی T ، اگر x برگ و y پدر x باشد، کدام گزینه در مورد مقادیر $a = Key[x]$ و $b = Key[y]$ درست است؟ (دوئل ۸۰)

(۱) b بزرگترین کلید در T است که کوچکتر از a باشد.

(۲) a کوچکترین کلید در T است که بزرگتر از b باشد.

(۳) b کوچکترین کلید در T است که بزرگتر از a باشد.

(۴) هیچکدام از گزینه‌های فوق همواره درست نیست.

- ۱۳- ترتیب $(level-order)$ یک درخت، عناصر درخت را به ترتیب سطح (عمق) آن عناصر و در هر سطح از چپ به راست بررسی می‌کند. (سراسری ۸۳)

فرض کنید که دنباله‌های سطح - ترتیب و میان ترتیب $(inorder)$ یک درخت دودویی جستجو داده شده‌اند. کدام یک از گزینه‌های زیر درست است؟

(۱) تنها با دنباله سطح - ترتیب می‌توان درخت را ساخت، ولی درخت یکتا نیست.

(۲) تنها با دنباله سطح - ترتیب می‌توان درخت را به صورت یکتا ساخت.

(۳) با سطح - ترتیب و میان ترتیب می‌توان درخت را ساخت، ولی درخت یکتا نیست.

(۴) با سطح - ترتیب و میان ترتیب می‌توان درخت را ساخت و درخت یکتا است.

۱۴- کدام یک از گزاره‌های زیر در مورد درخت‌های دودویی جستجو غلط است: (دوای ۷۳)

(۱) n عدد را می‌توان با استفاده از درخت دودویی جستجو با الگوریتم از مرتبه $O(n \log n)$ مرتب کرد.

(۲) ترتیب حذف دو المان مختلف از یک درخت دودویی جستجو مهم نیست و درخت حاصل برای هر دو ترتیب یکسان خواهد بود.

(۳) ارتفاع یک درخت جستجو با n المان می‌تواند $n-1$ باشد.

(۴) اگر درخت دودویی جستجوی T شامل عناصر $a_1 < a_2 < \dots < a_n$ باشد و اگر a_i در T هر دو فرزند چپ و راست را داشته باشد در آن صورت a_{i+1} نمی‌تواند فرزند چپ و a_{i-1} نمی‌تواند فرزند راست داشته باشد.

۱۵- T یک درخت جستجوی دودویی است. هدف پیدا کردن آدرس یکی از برگ‌های درخت T می‌باشد که اگر محتوای آن به جای ریشه درخت T بنشیند، درخت T یک درخت جستجوی دودویی باقی بماند. کدام یک از توابع زیر آدرس این برگ را پیدا می‌کند؟ (آاد ۸۰)

$function\ findleaf(T);$	$function\ findleaf(T);$
$begin$	$begin$
$P := rightchild(T);$	$P := leftchild(T)$
$while\ (leftchild(P) \neq \circ)\ do\ (۲)$	$while\ (rightchild(P) \neq \circ)\ (۱)$
$P := rightchild(P);$	$P := rightchild(P);$
$findleaf := P$	$findleaf := P$
$end;$	$end;$
$function\ findleaf(T);$	$function\ findleaf(T);$
$begin$	$begin$
$P := leftchild;$	$P := leftchild(T);$
$while\ (leftchild(P) \neq \circ)\ do\ (۴)$	$while\ (rightchild(P) \neq \circ)\ do\ (۳)$
$P := leftchild(P);$	$P := leftchild(P);$
$findleaf := P$	$findleaf := P$
$end;$	$end;$

۱۶- با n عنصر متفاوت، چند درخت دودویی جستجوی متفاوت به ارتفاع $n-1$ وجود دارد؟

(دوای ۸۱)

(۱) ۱ (۲) ۲ (۳) $n!$ (۴) 2^{n-1}

۱۷- ساختمان داده‌ای را بر روی مجموعه A از اعداد صحیح در نظر بگیرید که اعمال درج ($Insert$)، حذف ($Delete$) و پیدا کردن نزدیکترین ($Find\ closest$) را فراهم می‌آورد. منظور از $Find\ closest(x)$ پیدا کردن $y \in A$ است که $|x-y|$ نسبت به بقیه y ها کمینه باشد. فرض کنید که T بیشترین زمان اجرای اعمال فوق در بدترین حالت باشد. در این صورت کدام ساختمان داده کمترین مقدار T را خواهد داشت. (دوالتی ۸۱)

(۱) $Heap$ (۲) لیست نامرتب

(۳) لیست مرتب (۴) درخت دودویی جستجوی متوازن

۱۸- یک درخت جستجوی باینری با کلمه "SEARCH" (از چپ به راست) بسازید. تعداد متوسط مقایسه برای پیدا کردن یک کاراکتر در این درخت برابر است با: (علوم کامپیوتر ۸۳)

(۱) $2/83$ (۲) $2/4$ (۳) $2/6$ (۴) 6

۱۹- در یک درخت جستجوی باینری با n گره، تعداد مقایسه برای جستجوی یک عنصر حداکثر برابر است با: (علوم کامپیوتر ۸۴)

(۱) $O\left(\frac{n}{2}\right)$ (۲) $O(n)$ (۳) $O(\sqrt{n})$ (۴) $O(\lg n)$

۲۰- برای یک درخت جستجوی باینری که در گره‌های آن تعدادی عدد ذخیره شده است، با داشتن پیمایش $Preorder$ آن کدام جمله صحیح است؟ (علوم کامپیوتر ۸۴)

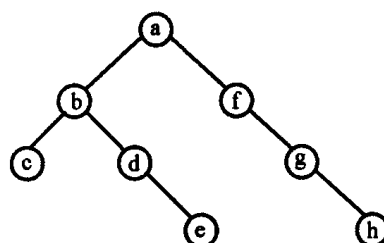
(۱) فقط با یک پیمایش نمی‌توان درخت را ساخت.

(۲) می‌توان درخت را در $O(n^2)$ ساخت.

(۳) می‌توان درخت را در $O(n \log n)$ ساخت.

(۴) می‌توان درخت را در $O(n)$ ساخت.

۲۱- در درخت جستجوی باینری زیر با حذف ریشه کدام یک از عناصر زیر جایگزین می‌شود؟ (علوم کامپیوتر ۸۴)



(۱) f یا b

(۲) h یا c

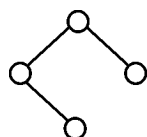
(۳) f یا e

(۴) e یا h

۲۲- چند حالت عناصر با کلیدهای $a < b < c < d$ را می‌توان وارد یک درخت دودویی جست و

جوی تهی کرد تا درختی به شکل زیر ایجاد شود؟

(دوای ۸۴)



۱ (۴)

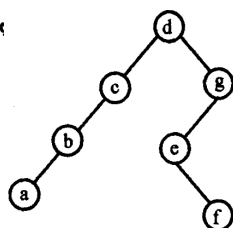
۲ (۳)

۳ (۲)

۴ (۱)

۲۳- با چند عدد دوران می‌توان درخت دودویی جستجوی سمت چپ را به درخت سمت راست

(۸۴ ای)



تبدیل کرد؟

۲ (۱)

۳ (۲)

۴ (۳)

(۴) نمی‌شود.

۲۴- بهترین جواب را علامت بزنید. ساختن یک درخت دودویی جستجو برای n عنصر داده شده

(۸۴ IT)

حتماً از مرتبه‌ی زیر است؟

$\Omega(n \log n)$ (۴)

$\Theta(n \log n)$ (۳)

$\Omega(n^2)$ (۲)

$\Theta(n^2)$ (۱)

۲۵- کدام گزینه نادرست است؟

(۸۴ IT)

(۱) با داشتن پیمایش *Inorder* یک درخت جستجوی دودویی (*BST*) همواره می‌توان

درخت را منحصر به فرد رسم کرد.

(۲) با داشتن پیمایش *Preorder* یک درخت جستجوی دودویی (*BST*) همواره می‌توان

درخت را منحصر به فرد رسم کرد.

(۳) با داشتن پیمایش‌های *Inorder* و *Preorder* یک درخت دودویی همواره می‌توان

درخت را منحصر به فرد رسم کرد.

(۴) با داشتن پیمایش *Preorder* یک درخت دودویی کامل (*Complete*) همواره می‌توان

درخت را منحصر به فرد رسم کرد.

۲۶- اگر عناصر یک درخت جستجوی باینری را به صورت *inorder* پیمایش کنیم و در داخل

یک استک قرار دهیم و سپس عناصر استک را خارج کنیم و یک درخت جستجوی باینری

(علوم کامپیوتر ۸۵)

بسازیم درخت حاصل چه درختی خواهد بود؟

(۱) درخت باینری تغییر نمی‌کند.

(۲) درخت باینری مورب به چپ

(۳) درخت باینری مورب به راست

(۴) زیر درخت‌های چپ و راست درخت باینری تعویض می‌شود.

۲۷- با داده‌های زیر یک درخت جستجوی باینری می‌سازیم:

۱۵۰۰ و ۱۲۰۰ و ۱۰۵۰ و ۱۰۰۰ و ۱۱۰۰ و ۱۰۲۵ و ۱۴۰۰ و ۱۳۰۰

اگر عدد ۱۰۲۵ را از درخت حذف کنیم در آن صورت پیمایش *Preorder* درخت پس از حذف برابر است با:

(علوم کامپیوتر ۸۵)

(۱) ۱۵۰۰ و ۱۲۰۰ و ۱۴۰۰ و ۱۱۰۰ و ۱۰۰۰ و ۱۰۵۰ و ۱۳۰۰

(۲) ۱۴۰۰ و ۱۵۰۰ و ۱۲۰۰ و ۱۰۵۰ و ۱۱۰۰ و ۱۰۰۰ و ۱۳۰۰

(۳) ۱۵۰۰ و ۱۴۰۰ و ۱۲۰۰ و ۱۱۰۰ و ۱۰۵۰ و ۱۰۰۰ و ۱۳۰۰

(۴) ۱۵۰۰ و ۱۴۰۰ و ۱۲۰۰ و ۱۱۰۰ و ۱۰۰۰ و ۱۰۵۰ و ۱۳۰۰

۲۸- یک درخت جستجوی دودویی ۱۵۰ گره دارد. کدام یک از موارد زیر صحیح است؟ (IT ۸۵)

(۱) این درخت حداقل ۶ سطح و حداکثر ۸ سطح دارد.

(۲) این درخت حداقل ۶ سطح و حداکثر ۷ سطح دارد.

(۳) این درخت حداقل ۷ سطح و حداکثر ۸ سطح دارد.

(۴) این درخت حداقل ۷ سطح و حداکثر ۱۵۰ سطح دارد.

۲۹- در یک درخت جستجوی دودویی T با n گره، فرض کنید هر گره x دارای کلید x ($Key(x)$) و نیز $size(x)$ است که تعداد عناصر زیردرخت به ریشه x (شامل خود x) را نشان می‌دهد.

فرض کنید $left[x]$ ، $right[x]$ و $p[x]$ به ترتیب فرزند چپ، راست و پدر x باشد. می‌خواهیم با

دریافت x مرتبه‌ی $key(x)$ را به دست آوریم. مرتبه یک عدد، تعداد عددهای کوچکتر یا مساوی آن عدد است. کدام یک از الگوریتم‌های زیر درست است؟

$r := size[left[x]] + 1$

$r := \phi$

$y := x$

$y := x$

while $y \neq root[T]$ do

while $y \neq root[T]$ do

if $y = right[p[y]]$

if $y = right[p[y]]$

then $r := r + size[left[p[y]]] + 1$ (۲)

then $r := r + size[left[p[y]]] \times 1$ (۱)

$y := p[y]$

$y := p[y]$

end

end

return r

return r

$r := \phi$

$r := size[left[x]]$

$y := x$

$y := x$

while $y \neq root[T]$ do

while $y \neq root[T]$ do

if $y = right[p[y]]$

if $y = right[p[y]]$

then $r := r + size[left[p[y]]]$ (۴)

then $r := r + size[left[y]]$ (۳)

$y := p[y]$

$y := p[y]$

end

end

return r

return r

۳۰- کدام گزینه پیمایش *Preorder* یک درخت جستجوی دودویی *BST* با پیمایش *Postorder* به صورت زیر است؟

(۸۶ IT)

Postorder: ۵ و ۶ و ۱۵ و ۱۰ و ۲۳ و ۲۴ و ۲۲ و ۲۶ و ۲۰

(۱) ۲۰ و ۲۴ و ۲۶ و ۱۰ و ۶ و ۵ و ۱۵ و ۲۳ و ۲۰

(۲) ۲۶ و ۲۳ و ۲۴ و ۲۲ و ۲۰ و ۱۵ و ۱۰ و ۵ و ۶

(۳) ۲۲ و ۲۳ و ۲۴ و ۲۶ و ۱۵ و ۵ و ۶ و ۱۰ و ۲۰

(۴) ۲۳ و ۲۴ و ۲۲ و ۲۶ و ۱۵ و ۵ و ۶ و ۱۰ و ۲۰

۳۱- در یک درخت *AVL* به ارتفاع ۴ لااقل چند گره وجود دارد؟

(دولتی ۷۴)

(۱) ۷ (۲) ۸ (۳) ۱۰ (۴) ۱۲

۳۲- درخت *ALV* یک درخت دودویی جستجو است که اختلاف ارتفاع دو زیر درخت هر عنصر در آن حداکثر ۱ باشد. با عناصر ۱، ۲، ۳، ۴ و ۵ حداکثر چند تا درخت *AVL* می‌توان ساخت؟ (ارتفاع درخت تهی ۱- فرض می‌شود)

(دولتی ۸۵)

(۱) ۵ (۲) ۶ (۳) ۷ (۴) ۸

۳۳- درخت *AVL* یک درخت دودویی است که ارتفاع دو زیر درخت هر گره آن حداکثر یک واحد با هم اختلاف دارد. (ارتفاع یک درخت تهی را ۱- فرض کنید). اگر $T(h)$ کمترین تعداد گره برای یک درخت *AVL* به ارتفاع h باشد، کدام یک از روابط بازگشتی زیر برای $T(h)$ درست است؟

(دولتی ۷۹)

$T(0) = 1$ و $T(1) = 1$

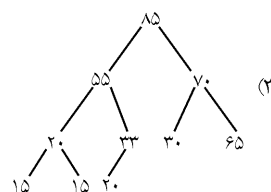
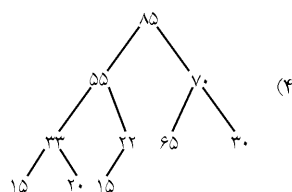
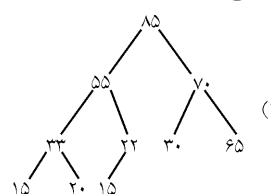
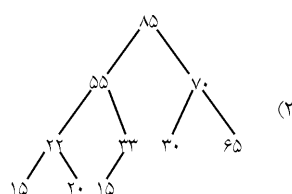
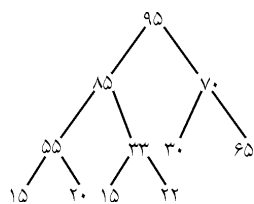
(۲) $T(h) = T(h-1) + T(h-2) + 1$

(۱) $T(h) = 2T(h-2) + 1$

(۴) $T(h) = T\left(\left\lfloor \frac{h}{2} \right\rfloor\right) + T\left(\left\lfloor \frac{h}{2} \right\rfloor\right) + 1$

(۳) $T(h) = 2T(h-1) + 1$

۳۴- با توجه به درخت (*Maxheap*) زیر چنانچه ریشه آن (۹۵) حذف شود شکل جدیدش کدام است؟

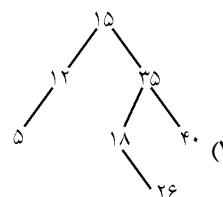
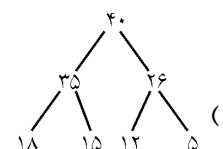
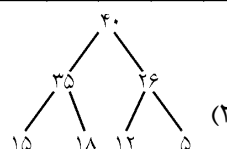


۳۵- گزینه صحیح را انتخاب کنید. (علوم کامپیوتر ۸۰)

- (۱) حداکثر عمق درخت جستجوی باینری برابر با $O(\log n)$ است.
- (۲) حداکثر عمق درخت باینری $heap$ برابر $O(n)$ است.
- (۳) حداکثر عمق درخت جستجوی باینری برابر با $O(n)$ است.
- (۴) حداکثر عمق درخت باینری $heap$ و نصف جستجوی باینری برابر با $O(\log n)$ است.

۳۶- فرض کنید آرایه زیر را با روش $heapsort$ می‌خواهیم مرتب کنیم پس از اجرای مرحله اول الگوریتم، درخت $heap$ حاصل کدام است؟

۱۵	۳۵	۱۲	۱۸	۴۰	۲۶	۵
----	----	----	----	----	----	---



(۴) گزینه ۱ و ۲

۳۷- یک $Max\text{-}heap$ با n عنصر به صورت آرایه پیاده‌سازی شده است. (عنصر ماکزیمم در ریشه است). مناسبترین گزینه برای پیدا کردن عنصر مینیمم در این ساختمان داده کدام است؟

(دوگزینه ۸۱)

- (۱) این کار را همواره می‌توان با $O(\log n)$ مقایسه بین عناصر $Heap$ انجام داد.
- (۲) این کار به حداکثر $\frac{n}{2}$ مقایسه بین عناصر $heap$ نیاز دارد.
- (۳) این کار ممکن است به $n-1$ مقایسه بین عناصر $heap$ نیاز داشته باشد.
- (۴) تنها در صورتی که $heap$ عناصر تکراری نداشته باشد، می‌توان این کار را با $O(\log n)$ مقایسه بین عناصر $heap$ انجام داد.

۳۸- آرایه n خانه‌ای A را در نظر می‌گیریم که برای ذخیره‌سازی عناصر یک درخت دودویی کامل مورد استفاده قرار گرفته فرض کنید الگوریتمی کارا را برای بررسی اینکه این درخت دودویی یک $heap$ است یا خیر در اختیار داریم. پیچیدگی این الگوریتم در بدترین حالت چقدر است؟

(دوگزینه ۸۱)

- (۱) $O(n)$ (۲) $O(n^2)$ (۳) $O(\log n)$ (۴) $O(n \log n)$

۳۹- صف اولویت (*Priority queue*) با استفاده از *Heap* پیاده‌سازی شده است. کدام یک از

عبارات زیر صحیح است؟ (n تعداد ارقام در صف اولویت می‌باشد.) (دوای ۷۵)

(۱) عملگر «پیدا کردن ماکزیمم و حذف آن» از صف اولویت دارای پیچیدگی زمانی $O(1)$ می‌باشد.

(۲) عملگر «اضافه کردن» یک قلم جدید به صف اولویت دارای پیچیدگی زمانی

$O(n \log_p^n)$ می‌باشد.

(۳) عملگر «پیدا کردن ماکزیمم» دارای پیچیدگی $O(\log_p^n)$ می‌باشد.

(۴) عملگر «اضافه کردن» یک قلم جدید به صف اولویت دارای پیچیدگی زمانی $O(\log_p^n)$ می‌باشد.

۴۰- فرض کنید آرایه A از اندیس ۱ تا $n-1$ به صورت *heap* است. (ریشه *heap* بزرگترین عنصر

است) الگوریتم زیر برای درج $A(n)$ به این *heap* پیشنهاد شده است. کدام یک از گزینه‌های

زیر را باید به جای قرار دهیم تا الگوریتم همواره درست کار کند؟ (دوای ۷۹)

$Procedure\ insert(A, n) \quad i > 1\ and\ A(i) > temp \quad (1)$

$integer\ i, j, n; \quad i > 0\ and\ A(i) < temp \quad (2)$

$j \leftarrow n; \quad i > 0\ and\ A(i) \neq temp \quad (3)$

$i \leftarrow \left\lfloor \frac{n}{2} \right\rfloor; \quad i > 1\ and\ A(i) < A(j) \quad (4)$

$temp \leftarrow A(n);$

while ... do

$A(j) \leftarrow A(i)$

$j \leftarrow i;$

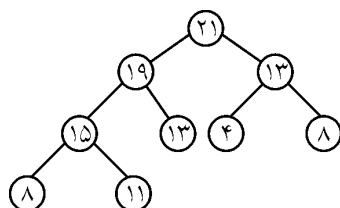
$i \leftarrow \left\lfloor \frac{i}{2} \right\rfloor;$

endwhile

$A(j) \leftarrow temp;$

end

۴۱- اگر یک گره از درخت *max-Heap* زیر حذف شود، داده آخرین سطح درخت چیست؟



۱۱ (۱)

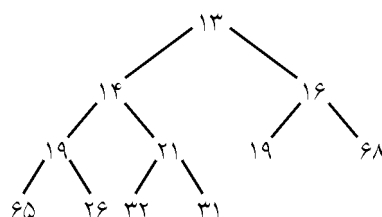
۱۵ (۲)

۸ (۳)

۲۱ (۴)

۴۲- درخت نیمه مرتب (*Heap*) زیر را به صورت آرایه پیاده‌سازی کرده‌ایم.

۱۳	۱۴	۱۶	۱۹	۲۱	۱۹	۶۸	۶۵	۲۶	۳۲	۳۱
----	----	----	----	----	----	----	----	----	----	----



اگر D را عمل *Deletemin* (حذف کوچکترین المان) $I(x)$ را عمل درج عنصر x در یک درخت نیمه مرتب بنامیم آرایه حاصل از اعمال زیر را که به ترتیب از چپ به راست بر روی این درخت انجام می‌شوند کدامیک از گزینه‌های زیر است: $D, I(17), I(15), D, D$

(۱) ۳۲ و ۲۶ و ۶۵ و ۲۱ و ۶۸ و ۳۱ و ۱۹ و ۱۹ و ۱۷ و ۱۶

(۲) ۳۲ و ۲۶ و ۶۵ و ۶۸ و ۲۱ و ۳۱ و ۱۹ و ۱۷ و ۱۹ و ۱۶

(۳) ۳۲ و ۳۱ و ۶۵ و ۶۸ و ۲۱ و ۱۹ و ۲۶ و ۱۹ و ۱۷ و ۱۶

(۴) ۲۱ و ۲۶ و ۶۵ و ۶۸ و ۳۲ و ۱۹ و ۳۱ و ۱۷ و ۱۹ و ۱۶

۴۳- آرایه زیر یک *Heap* است. برای درج عدد ۹۵ در آرایه به گونه‌ای که آرایه نهائی نیز وضعیت *Heap* داشته باشد، چند عمل *exchange* (تعویض دو کمیت) لازم است؟ (دوای ۸۰)

۱۰۰
۹۰
۸۲
۸۵
۷۴
۷۵
۷۳
۶۸
۷۰

(۱) دو
(۲) چهار
(۳) شش
(۴) هشت

۴۴- پس از حذف (یا اضافه کردن) یک عنصر، می‌توان هرم (*Heap*) را مجدداً را

(علوم کامپیوتر سراسری ۸۱)

(۱) در زمان n^2 بازسازی کرد. (۲) در زمان $n \log n$ بازسازی کرد.

(۳) در زمان $\log n$ بازسازی کرد. (۴) هیچکدام

۴۵- عمق یک گره درایه i در یک *heap* با n گره که با آرایه پیاده‌سازی شده است چقدر است؟

(علوم کامپیوتر سراسری ۸۱ و IT ۸۳)

(۱) $\left\lceil \frac{i}{2} \right\rceil$ (۲) $\left\lceil \frac{i}{2} \right\rceil$ (۳) $i-1$ (۴) $\left\lceil \log_2^i \right\rceil$

۴۶- یک $max\text{-heap}$ با N عنصر متمایز را در نظر بگیرید که با آرایه پیاده‌سازی شده (بزرگترین عنصر در درایه‌ی اول قرار دارد) چهارمین بزرگترین عنصر در کدام یک از درایه‌های زیر می‌تواند قرار گیرد؟

(مهندسی کامپیوتر سراسری ۸۲)

- (۱) ۲ یا ۳ (۲) ۸ تا ۱۵ (۳) ۴ و ۵ و ۶ یا ۷ (۴) همه موارد فوق

۴۷- یک ماکزیمم - هیپ حاوی ۶۴ عنصر با کلیدهای متفاوت ۱ تا ۶۴ است. بزرگترین عددی که می‌تواند در برگ واقع در آخرین سطح این هیپ قرار گیرد، کدام یک از اعداد زیر می‌تواند باشد؟ (در ماکزیمم هیپ هر عنصر از فرزندانش کوچک‌تر نیست).

(سراسری ۸۳)

- (۱) ۵۹ (۲) ۵۸ (۳) ۵۷ (۴) ۵۶

۴۸- برای ادغام لیست‌های مرتب شده $L_1, L_2, \dots, L_k$ کدام یک از ساختار داده‌های زیر (علاوه بر لیست‌های ورودی) بهترین است؟

(سراسری ۸۳)

- (۱) درخت $Min\text{-heap}$ (۲) لیست دو طرفه خطی
(۳) آرایه خطی (۴) درخت دودویی جستجو

۴۹- تابع $aproc$ بصورت زیر تعریف شده است، $swap(o, o)$ تابعی است که مقدار عملوندهایش را جابجا می‌کند.

(علوم کامپیوتر ۸۳)

```
void aproc(int *A, int n) {
    if (n > 1)
        if (A[n] < A[n / 2]) {
            swap(A[n], A[n / 2]);
            aproc(A, n / 2);
        }
}
```

با اجرای حلقه

```
for (i = 1; i <= n; ++i)
    Aproc(A, i);
```

چه مقداری در $A[1]$ خواهد بود؟

- (۱) مقدار میانه عنصر A
(۲) کمترین مقدار در بین عناصر A
(۳) بیشترین مقدار در بین عناصر A
(۴) $A[1]$ تغییر نمی‌کند و مقدار آن همان مقدار قبل از اجرای برنامه است.

۵۰- اگر آرایه

(علوم کامپیوتر ۸۳)

۱۰	۱۵	۲۲	۴	۱۱	۲۳	۱۹	۱۴
----	----	----	---	----	----	----	----

نمایش یک درخت باینری باشد، و آن را تبدیل به یک *Min-Heap* نمائیم در آن صورت محتوای آرایه برابر خواهد بود با:

- (۱) ۱۴ و ۲۲ و ۲۳ و ۱۱ و ۱۹ و ۱۵ و ۱۰ و ۴
 (۲) ۲۳ و ۲۲ و ۱۹ و ۱۱ و ۱۵ و ۱۴ و ۱۰ و ۴
 (۳) ۲۳ و ۲۲ و ۱۹ و ۱۵ و ۱۴ و ۱۱ و ۱۰ و ۴
 (۴) ۱۵ و ۲۲ و ۲۳ و ۱۱ و ۱۴ و ۱۹ و ۱۰ و ۴

۵۱- فرض کنید که ماکزیمم - هیپ حاوی اعداد متمایز ۱ تا ۱۰۲۳ است. حداکثر چند تا از اعداد

بیش‌تر از ۱۰۰۰ می‌تواند در پایین‌ترین سطح درخت قرار گیرند؟ (علوم کامپیوتر ۸۳)

- (۱) ۱۰ (۲) ۱۲ (۳) ۱۳ (۴) ۱۴

۵۲- فرض کنید که یک ماکزیمم - هیپ دودویی حاوی N عدد متمایز است (بزرگترین عنصر در ریشه و در درایهٔ اول قرار دارد). چهارمین عنصر این عناصر در کدام یک از درایه‌ها

نمی‌تواند قرار بگیرد. (۸۳IT)

- (۱) ۲ یا ۳ (۲) ۸ تا ۱۵ (۳) ۱۶ تا ۳۱ (۴) ۴، ۵، ۶ یا ۷

۵۳- ماکزیمم تعداد مقایسه برای *min-heap* کردن یک *max-heap* با n گره برابر است با:

(علوم کامپیوتر ۸۳)

- (۱) $O(n + \log n)$ (۲) $O(\log n)$ (۳) $O(n \log n)$ (۴) $O(2n)$

۵۴- به یک *Min-Heap* خالی به ترتیب گره‌هایی با کلیدهای (از راست به چپ) ۷۰ و ۴۵ و ۵۰ و۴۲ و ۴۵ و ۵۵ و ۴۰ و ۷۵ اضافه شده است. سپس سه عمل حذف (*Delete*) بر روی *Min-Heap* انجام شده است. *Min-Heap* حاصل مطابق کدام گزینه خواهد بود. (۸۴IT)

- (۱) [۴۵, ۵۰, ۵۵, ۷۰, ۷۵] (۲) [۴۵, ۵۰, ۵۵, ۷۵, ۷۰]
 (۳) [۴۵, ۵۵, ۵۰, ۷۰, ۷۵] (۴) [۴۵, ۵۵, ۵۰, ۷۵, ۷۰]

۵۵- اگر در یک *min-heap* دلخواه جای زیر درخت‌های چپ و راست تعویض شود در آن صورت

کدام گزینه صحیح است؟ (علوم کامپیوتر ۸۵)

(۱) همیشه ساختار *min-heap* از بین می‌رود.(۲) ساختار *min-heap* تبدیل به *max-heap* می‌شود.(۳) در برخی از شرایط ساختار *min-heap* از بین می‌رود.(۴) در هیچ شرایطی ساختار *min-heap* از بین نمی‌رود.

۵۶- اگر بخواهیم دو $min\text{-heap}$ به تعداد m و n را با هم $merge$ کنیم پیچیدگی زمانی آن برابر است با:

$$\begin{aligned} (۱) \quad m \log mn & \quad (۲) \quad m \log n + n \log m \\ (۳) \quad n \log n \cdot \lg m & \quad (۴) \quad m \cdot \log n \cdot \log m \end{aligned}$$

۵۷- در یک $heap$ با n گره که در یک آرایه نمایش داده شده باشد گره i ام در زیر درخت چپ درخت، متناظر با گره j ام در زیر درخت راست درخت می‌باشد به طوری که: (علوم کامپیوتر ۸۵)

$$\begin{aligned} (۱) \quad j &= i - 2^{\lfloor \log_2^i \rfloor + 1} & (۲) \quad j &= i + 2^{\lfloor \log_2^i \rfloor - 1} \\ \text{if } j > n \quad j &= j / 2; & \text{if } j > n \quad j &= j / 2; \\ (۳) \quad j &= i + 2^{\lfloor \log_2^i \rfloor - 1} & (۴) \quad j &= i + 2^{\lfloor \log_2^i \rfloor + 1} \\ \text{if } j > n \quad j &= \frac{j+1}{2}; & \text{if } j > n \quad j &= \frac{j+1}{2}; \end{aligned}$$

۵۸- آرایه T که در آن تعدادی از خانه‌ها هنوز مقداردهی نشده‌اند را در نظر می‌گیریم.

	۱	۲	۳	۴	۵	۶	۷	۸	۹	۱۰
T	۷۵		۳۰		۱۷	۲۸	۲۰	۷		

با قرار دادن مقادیر کدام یک از موارد زیر این آرایه به یک هیپ تبدیل خواهد شد؟

(دوای ۸۵)

$$\begin{aligned} (۱) \quad T[۲] &= ۳۰; T[۴] = ۹; T[۹] = ۱۰; T[۱۰] = ۶ \\ (۲) \quad T[۲] &= ۱۵; T[۴] = ۱۵; T[۹] = ۱۷; T[۱۰] = ۱۰ \\ (۳) \quad T[۲] &= ۳۵; T[۴] = ۳۰; T[۹] = ۳۲; T[۱۰] = ۱۶ \\ (۴) \quad T[۲] &= ۲۱; T[۴] = ۱۴; T[۹] = ۰; T[۱۰] = ۱۶ \end{aligned}$$

۵۹- کدام یک از ترتیب‌های زیر ساختمان داده‌های همه منظوره برای ذخیره مرتب اشیاء

(براساس مقدار کلید) را به ترتیب از کندترین به سریع‌ترین لیست کرده است؟ (۸۵ IT)

- (۱) جدول درهم سازی ($hash$)، آرایه‌های مرتب، لیست‌های زنجیره‌ای مرتب
- (۲) آرایه‌های مرتب، درخت‌های جستجوی دودویی، لیست‌های زنجیره‌ای، $heap$
- (۳) آرایه‌های مرتب، لیست‌های زنجیره‌ای مرتب، درخت‌های جستجوی دودویی
- (۴) آرایه‌های نامرتب، لیست‌های زنجیره‌ای نامرتب، درخت‌های جستجوی دودویی

۶۰- کدام یک از عبارتهای زیر همیشه صحیح است؟ (۸۵ IT)

- (۱) کوچکترین گره در یک درخت جستجوی دودویی همیشه برگ است.
- (۲) درج در یک درخت دودویی جستجو می‌تواند در زمان حداکثر $O(1)$ انجام شود.
- (۳) پیمایش میان ترتیب یک $max\ heap$ عناصر موجود در آن را به صورت مرتب طی می‌کند.
- (۴) پیمایش بین ترتیب یک درخت دودویی جستجو عناصر موجود در آن را به صورت مرتب طی می‌کند.

۶۱- کدام یک از گزینه‌های زیر نمی‌تواند یک بازنمایی از یک $Max\ Heap$ با آرایه باشد (ترتیب

عناصر از چپ به راست است). (۸۵ IT)

- (۱) ۱ ۴ ۳ ۶ ۱۰ ۵ ۲۰
- (۲) ۱ ۲ ۴ ۳ ۱۷ ۱۲ ۲۰
- (۳) ۱ ۲ ۴ ۳ ۱۰ ۵ ۲۰
- (۴) ۲ ۴ ۳ ۱۸ ۱۰ ۲۰

۶۲- به یک $Min\ heap$ خالی گره‌هایی با کلیدهای (به ترتیب از راست به چپ) ۴۵ و ۵۵ و ۴۰ و

۲۲ و ۷۵ و ۷۰ و ۴۵ و ۵۰ و ۲ اضافه شده است. $Min\ heap$ حاصل کدام گزینه است؟ (۸۶ IT)

- (۱) ۵۰ و ۵۵ و ۴۵ و ۷۰ و ۷۵ و ۴۰ و ۴۵ و ۲۲ و ۲
- (۲) ۵۵ و ۷۰ و ۴۰ و ۴۵ و ۷۵ و ۵۰ و ۲۲ و ۴۵ و ۲
- (۳) ۵۵ و ۵۰ و ۴۵ و ۷۰ و ۷۵ و ۴۵ و ۴۰ و ۲۲ و ۲
- (۴) ۵۵ و ۵۰ و ۴۵ و ۷۰ و ۷۵ و ۴۰ و ۴۵ و ۲۲ و ۲

۶۳- سومین کوچکترین کلید در یک $Min\ Heap$ با کلیدهای متمایز در درایه‌هایی با چه

اندیس‌هایی می‌تواند باشد؟ (۸۶ IT)

- (۱) ۱ و ۲ و ۳
- (۲) ۲ و ۷ و ۶ و ۵ و ۴ و ۳
- (۳) ۷ و ۶ و ۵ و ۴ و ۳ و ۲ و ۱
- (۴) ۷ و ۶ و ۵ و ۴ و ۳ و ۲ و ۱

۶۴- متنی شامل ۷۰۰۰ حرف از حروف a و b و c و d و e و f با تعداد تکرار $a = ۱۰۰۰$ و

$b = ۱۲۰۰$ و $c = ۸۰۰$ و $d = ۱۵۰۰$ و $e = ۱۸۰۰$ و $f = ۷۰۰$ موجود است. چنانچه

کدی بهینه برای حروف بالا انتخاب نماییم، تعداد کل بیت‌های لازم برای تبدیل متن مذکور

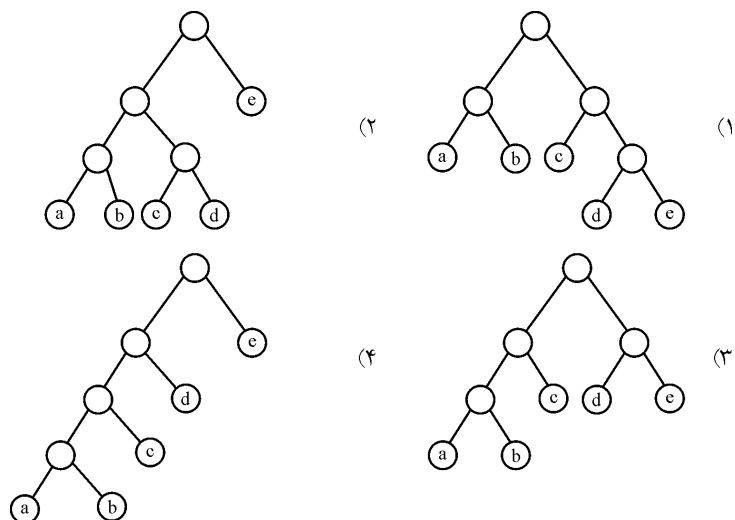
به مجموعه‌ای از بیت‌ها چقدر است؟ (دهلی ۸۱)

- (۱) ۱۴۶۰۰
- (۲) ۱۷۷۰۰
- (۳) ۲۴۳۰۰
- (۴) ۳۵۲۰۰

۶۵- حروف e و d و c و b و a با جدول فراوانی زیر داده شده است:

درخت هافمن وابسته به این حروف به قرار است.

حروف	a	b	c	d	e
فراوانی	۰/۰۵	۰/۱	۰/۲۵	۰/۲۸	۰/۳۲



۶۶- در فشرده‌سازی با استفاده از کد هافمن اگر x و y دو کاراکتری باشند که کمترین تکرار را داشته

باشند در آن صورت کد بهینه برای این دو نویسه (Character) (علوم کامپیوتر ۸۰)

(۱) دارای پیشوند یکسان می‌باشند، فقط در آخرین لیست آنها متفاوت است.

(۲) دارای پسوند یکسان می‌باشند، فقط در اولین لیست آنها متفاوت است.

(۳) دارای پیشوند یکسان نمی‌باشند و فقط کد آخر آنها برابر است.

(۴) دارای پیشوند یکسان نمی‌باشند و فقط کد اول آنها برابر است.

۶۷- اگر رشته $abcabbaccaabdffe$ را با روش کدینگ هافمن کد کنیم، طول کد چند بیت

خواهد شد؟ (۸۵ IT)

(۱) ۳۸ (۲) ۳۶ (۳) ۳۴ (۴) هیچکدام

۶۸- فرض کنید که پیمایش پس‌وندی (Postfix) یک درخت دودویی جست‌جوی T با n عنصر

در آرایه‌ای به نام $postfix$ و به طول n ذخیره شده است، که $postfix[i]$ کلید i امین گره در

این پیمایش باشد. فرض کنید که برای $1 < i < n$ داریم $postfix[i] = x$ ، $postfix[i-1] = a$

و شرط $postfix[i+1] = b$ لازم و کافی برای آنکه x برگ درخت T باشد کدام یک از

موارد زیر است؟ (مهندسی ۸۷)

(۱) $a < x$ (۲) $x > b$

(۳) $a < b$ (۴) هر سه مورد صحیح است.

۶۹- تعداد درخت‌های دودویی جست و جویی که می‌توان با ۳۶ کلید داده شده مجزا از هم ساخت به طوری که اختلاف عمق برگ‌های آن درخت حداکثر ۱ باشد، چند تاست؟ (مهندس ۸۷)

$$\begin{pmatrix} 36 \\ 32 \end{pmatrix} (4) \quad \begin{pmatrix} 32 \\ 5 \end{pmatrix} (3) \quad \begin{pmatrix} 36 \\ 5 \end{pmatrix} (2) \quad \begin{pmatrix} 16 \\ 5 \end{pmatrix} (1)$$

۷۰- نمی‌توان ساختمان داده‌ای برای n عنصر طراحی کرد که (مهندس ۸۷)

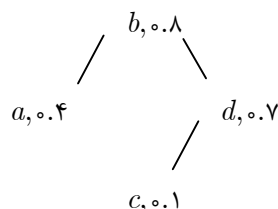
(۱) ساخت آن $O(n \lg n)$ و حذف بزرگ‌ترین عنصر آن، درج و حذف کلید یک عنصر دلخواه در آن $O(\lg n)$ باشد.

(۲) ساخت آن $O(n)$ و حذف بزرگ‌ترین عنصر آن، درج حذف و افزایش و کاهش کلید یک عنصر دلخواه در آن $O(\lg n)$ باشد.

(۳) ساخت آن $O(n)$ و حذف بزرگ‌ترین عنصر آن، حذف و افزایش کلید یک عنصر دلخواه در آن $O(\lg n)$ باشد.

(۴) ساخت آن $O(n)$ و حذف بزرگ‌ترین عنصر آن، $O(1)$ و درج و حذف یک عنصر دلخواه در آن $O(\lg n)$ باشد.

۷۱- داده ساختار *treap* درخت دودویی است که بر روی n عنصر با کلیدهای متمایز ساخته می‌شود. هر عنصر علاوه بر کلید یک مولفه‌ی «امتیاز» دارد که عدد حقیقی بین ۰ و ۱ می‌باشد. *Treap* طوری طراحی می‌شود که از نظر کلیدها دودویی جست و جو و از نظر امتیازها یک هرم بیشینه (*max-heap*) است. مثلاً عناصر a, b, c, d با امتیازهای به ترتیب برابر ۰/۴، ۰/۸، ۰/۱، ۰/۷ در یک *treap* زیر قرار می‌گیرند. (مهندس ۸۷)



توجه کنید که فقط خاصیت هرم (*heap*) بین امتیاز عناصر مهم است (یعنی امتیاز هر عنصر از پدریش بیش‌تر نیست). کدام یک از گزینه‌های زیر در مورد *treap* درست است؟

(۱) اگر کلیدها و امتیاز عناصر متمایز باشند، فقط یک *treap* برای آن وجود دارد.
 (۲) اگر کلیدها و امتیاز عناصر متمایز باشند، ممکن است *treap* برای آن وجود نداشته باشد.
 (۳) اگر کلیدها متمایز ولی امتیازها در حالت کلی باشند، ممکن است *treap* برای آن وجود داشته باشد.

(۴) اگر کلیدها و امتیاز عناصر متمایز باشند، *treap* برای آن وجود دارد ولی ممکن است جواب تک نباشد.

۷۲- فرض کنید می‌خواهیم مرتب‌سازی *Heap Sort* را روی آرایه‌ای که شامل ۱۴ عنصر زیر است. انجام دهید.

(۸۷ IT)

$$A[1] \dots A[14] = 16, 50, 23, 20, 21, 40, 41, 30, 3, 32, 33, 34, 35, 36$$

برای تبدیل آرایه یک *Max Heap* به صورت درجا () چند جابجایی بین گره با فرزند انجام می‌شود؟

$$4 \ (4) \qquad 5 \ (3) \qquad 6 \ (2) \qquad 7 \ (1)$$

۷۳- چند *Min-heap* متفاوت می‌توان با ۷ گره با کلیدهای ۱ تا ۷ ساخت؟

(۹۰۹۸۸ IT)

$$20 \ (4) \qquad 40 \ (3) \qquad 80 \ (2) \qquad 160 \ (1)$$

۷۴- هیپ زیر داده شده است:

$$A[1 \dots 18] = 20, 15, 18, 7, 9, 14, 16, 3, 6, 8, 4, 13, 10, 12, 11, 1, 2, 5$$

عمل *Change (i, k)* کلید $A[i]$ را به k تغییر دهد و با انجام تعداد جابجایی کاری می‌کند که آرایه مجدداً به صورت هیپ در آید. ما این ۲ عمل را به ترتیب انجام می‌دهیم:

$$Change(1, 16) \ Change(2, 4)$$

مجموع تعداد جابجایی (*swap*)‌ها چند تاست؟

(مهندسی ۸۸)

$$6 \ (4) \qquad 5 \ (3) \qquad 4 \ (2) \qquad 3 \ (1)$$

۷۵- کدام یک از اعمال زیر را نمی‌توان در یک *max-heap* با n عنصر در مرتبه‌ی $O(\lg n)$ انجام داد؟

(مهندسی ۸۸)

(۱) یافتن یک عنصر با کلید مشخص (۲) حذف یک عنصر داده شده

(۳) کاهش مقدار کلید یک عنصر داده شده (۴) افزایش مقدار کلید یک عنصر داده شده

۷۶- می‌خواهیم k فایل مرتب f_1 تا f_k را در هم ادغام کنیم و یک فایل مرتب بسازیم. فایل f_i

به اندازه‌ی n_i رکورد دارد و فایل خروجی هم به اندازه‌ی $n = \sum_{i=1}^k n_i$ رکود خواهد داشت. با هر بار خواندن از یک فایل و نوشتن در انتهای یک فایل می‌توانیم بلوکی به

اندازه‌ی r رکود از یک فایل را بخوانیم یا در فایل خروجی بنویسیم. فایل‌ها همه ترتیبی هستند، یعنی هر چه که باشیم فقط بلوک بعدی را می‌توانیم بخوانیم یا در انتهای فایل خروجی بنویسیم.

تعداد کل خواندن بلوک‌ها و تعداد کل نوشتن بلوک‌ها چند تاست؟ در صورتی که حافظه اصلی برای $k+1$ بلوک ظرفیت داشته باشد.

(مهندسی ۸۸)

(۱) خواندن و نوشتن هر دو برابر $\left\lfloor \frac{n}{r} \right\rfloor$

(۲) خواندن $\sum_{i=1}^k \left\lfloor \frac{n_i}{r} \right\rfloor$ و نوشتن $\left\lfloor \frac{n}{r} \right\rfloor$

(۳) خواندن و نوشتن هر دو برابر $\sum_{i=1}^k \left\lfloor \frac{n_i}{r} \right\rfloor$

(۴) خواندن و نوشتن هر دو برابر $\sum_{i=1}^k \left(1 + \left\lfloor \frac{n_i}{r} \right\rfloor\right)$

۷۷- اگر b_n تعداد درخت‌های دودوئی باشد که با n گره ساخته می‌شوند کدام یک نادرست است؟ (مهندسی ۸۸)

$$b_n \in \Omega(2^n) \quad (۱) \quad b_n = \frac{1}{n+1} \binom{2n}{n} \quad (۲)$$

$$\binom{2}{n} \prod_{i=1}^n b_n \quad (۳) \quad b_n = \sum_{k=1}^n b_{k-1} b_{n-k} \quad (۴)$$

۷۸- A یک آرایه n عضوی با اعضاء غیر تکراری و B یک آرایه m عضوی با اعضاء غیر تکراری هستند و $m < n$ است می‌خواهیم آرایه c را طوری بیابیم که اعضاء A و B در آن وارد شده باشد و عضو تکراری هم نداشته باشد بهترین الگوریتم برای این کار چه زمانی نیاز دارد؟ (مهندسی ۸۸)

(۱) زمان $m \log m + n \log n + m + n$

(۲) زمان $(m+n) \log(m+n)$

(۳) زمان $(m+n) \log n$

(۴) زمان $(m+n) \log m$

۷۹- n عدد در آرایه T ذخیره شده اند. برای T الگوریتم ذکر شده را اجرا می‌کنیم. اگر t تعداد دفعات اجرای حلقه $repeat$ باشد کدام یک از گزاره‌های زیر صحیح است با فرض اینکه $K = \lceil \lg n \rceil$. (دولتی ۸۶)

```

procedure M( $T[1 \dots n]$ )
  for  $i \leftarrow \left\lfloor \frac{n}{2} \right\rfloor$  down to 1 do
     $R \leftarrow i$ 
    repeat
       $j \leftarrow R$ 
      if  $2j \leq n$  and  $T[2j] > T[R]$  then
         $R \leftarrow 2j$ 
      if  $2j + 1 \leq n$  and  $T[2j + 1] > T[R]$  then
         $R \leftarrow 2j + 1$ 
    swap  $T[j], T[R]$ 
  until  $j = R$ 

```

$$t \leq 2 \times 2^{k-2} + 3 \times 2^{k-3} + \dots + k \quad (1)$$

$$t \leq 3 \times 2^{k-2} + 4 \times 2^{k-3} + \dots + k \quad (2)$$

$$t \leq 3 \times 2^{k-2} + 4 \times 2^{k-3} + \dots + (k+1) \quad (3)$$

$$t \leq 2 \times 2^{k-1} + 3 \times 2^{k-2} + \dots + (k+1) \quad (4)$$

۸۰- حداکثر تعداد کلید در یک B -Tree با مینیمم درجه نود t و ارتفاع h چقدر است؟ (IT۸۸)

$$(1) \quad (2t)^{h+1} - 1 \quad (2) \quad (2t)^{h+1} - 1 \quad (3) \quad 2t^h - 1 \quad (4) \quad 2t^{h+1} - 1$$

۸۱- کدام یک از موارد زیر صحت ندارد؟ (آیاد۷۲)

(۱) درخت‌های نوع AVL عمدتاً برای دستیابی بهتر و تغییر محتوای دینامیکی در حافظه اصلی به کار می‌روند.

(۲) ساختار B -Tree ساختارهای خوبی جهت ذخیره فایل‌های دینامیکی بسیار بزرگ روی حافظه جانبی هستند.

(۳) درخت‌های نوع AVL ساختار خوبی برای ساخت درخت‌های بهینه ($Optimal$) می‌باشند.

(۴) درخت‌های AVL معمولاً برای ذخیره‌سازی فایل‌های دینامیکی روی حافظه جانبی به کار می‌روند.

۸۲- رشته متنی (*abaabacadcade*) را در نظر بگیرید. کد هافمن حاصل برای هر یک از نویسه‌ها برابر است با

(علوم کامپیوتر ۸۰)

$a = 00$	$a = 000$	$a = 0$	$a = 1$
$b = 01$	$b = 001$	$b = 100$	$b = 01$
$c = 10$ (۴)	$c = 010$ (۳)	$c = 101$ (۲)	$c = 001$ (۱)
$d = 11$	$d = 011$	$d = 110$	$d = 0001$
$e = 100$	$e = 100$	$e = 111$	$e = 0000$

۸۳- یک لیست ۱۲ عنصری حاوی کلیدهای یک تا دوازده به صورت صعودی مرتب است. اگر این لیست به صورت درجا تبدیل به یک *Max Heap* شود، عنصر پنجم لیست کدام گزینه است؟

(۸۹ IT)

۱۱ (۱) ۹ (۲) ۱۰ (۳) ۸ (۴)

۸۴- هفتمین کلید در پیمایش *preoder* یک درخت جستجوی دودویی (*BST*) که پیمایش *postorder* آن به شکل زیر است (از چپ به راست)، کدام گزینه است؟

(۸۹ IT)

Postorder: 5, 6, 15, 10, 23, 24, 22, 26, 20

۲۶ (۱) ۲۲ (۲) ۲۴ (۳) ۱۵ (۴)

۸۵- فرض کنید صد کلید یک تا صد را به یک درخت جستجوی دودویی اضافه کرده‌ایم. ترتیب اضافه شدن کلیدها مشخص نیست اما احتمال تمام حالات ترتیب‌های اضافه شدن کلیدها (تمام *Permutation* های ممکن یک تا صد) با هم برابر است. با چه احتمالی در روند اضافه شدن کلیدها به درخت کلید ۴ و کلید ۹ با هم مقایسه می‌شوند، در صورتی که اولین کلید اضافه شده کلید ۴۳ باشد؟

(۸۹ IT)

$\frac{1}{2}$ (۱) $\frac{1}{4}$ (۲) $\frac{1}{3}$ (۳) $\frac{43}{100} \times \frac{1}{4}$ (۴)

۸۶- فرض کنید که در *Max-heap* استفاده شده در ساختمان داده‌ی *X*، هر عنصر یک زوج مرتب به صورت $\langle Key, Value \rangle$ است که معیار مقایسه‌ی عناصر مقدار *key* آنها می‌باشد.

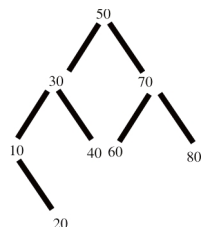
ساختمان داده *X* یک پیاده‌سازی از کدام گزینه است؟

(۸۹ IT)

$X :: A(x)\{$	<i>Stack</i> (۱)
$Count ++;$	<i>Max-Heap</i> (۲)
$Max - heap - insert(H, \langle count, x \rangle);$	<i>Queue</i> (۳)
$\}$	<i>Min Heap</i> (۴)
$X :: B(x)\{$	
$P = Heap - Extract - \max(H);$	
$Return - Value(p);$	
$\}$	

۸۷- اگر به درخت AVL زیر کلید ۱۵ اضافه شود، کدام گزینه بخشی از پیمایش $Preorder$

(۹۰IT)



درخت حاصل خواهد بود؟

(۱) ۳۰ و ۴۰ و ۱۵ و ۱۰ و ۲۰

(۲) ۴۰ و ۱۵ و ۲۰ و ۱۰ و ۳۰

(۳) ۴۰ و ۲۰ و ۱۰ و ۱۵ و ۳۰

(۴) ۴۰ و ۱۵ و ۱۰ و ۲۰ و ۳۰

۸۸- یک $Max\ Heap$ با n عنصر را که در آرایه $A[1..n]$ قرار دارد، در نظر بگیرید. مرتبه زمانی

الگوریتم حذف عنصر i ام ($1 \leq i \leq n$) از این $Max\ Heap$ به گونه‌ای که ساختار Max

(۹۰IT)

$Heap$ را حفظ کند، چیست؟

(۴) n

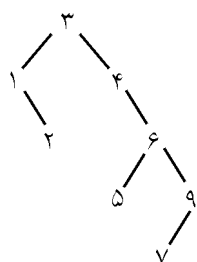
(۳) $\log n$

(۲) $n \log n$

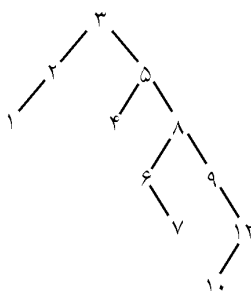
(۱) ۱

پاسخنامه سؤال‌های طبقه‌بندی شده فصل ششم

- (۱) گزینه (۱). بقیه گزینه‌ها $O(h)$ هستند.
- (۲) گزینه (۱). BST برای حذف عناصر تکراری مناسب است.
- (۳) گزینه (۳). در BST یافتن یک عنصر از مرتبه ارتفاع درخت است.
- (۴) گزینه (۲). در BST الگوریتم‌های گفته شده از مرتبه $O(h)$ هستند و برای آنکه h از مرتبه $\log_2^n$ باشد، درخت باید متوازن باشد.
- (۵) گزینه (۱). حذف جابجایی‌پذیر نیست. گزینه ۴ در حالت متوسط صحیح است.
- (۶) گزینه (۱). S ریشه است. e از s کوچکتر است (حروف الفبا) و سمت چپ s قرار می‌گیرد و به همین ترتیب، در ضمن در هر نود، تعداد کاراکتر نیز ذخیره می‌شود.
- (۷) گزینه (۱). درخت را رسم می‌کنیم.



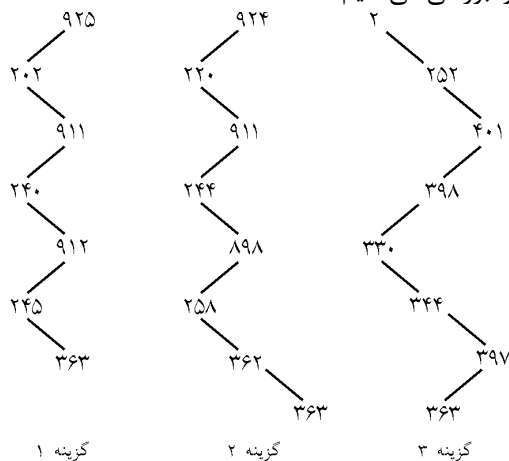
Postorder: ۲ و ۱ و ۵ و ۷ و ۹ و ۶ و ۴ و ۳



(۸) گزینه (۴).

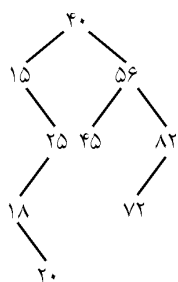
- (۹) گزینه (۳). بهترین و متوسط از مرتبه $n \cdot \lg n$ و بدترین n^2 است.
- (۱۰) گزینه (۴). بدیهی است گزینه ۴ اشتباه است. چون B قبل از A در دنباله ورودی قرار دارد، اما در درخت رسم شده ترتیب ورود باید برعکس باشد.

(۱۱) گزینه (۱). گزینه‌ها را بررسی می‌کنیم:



گزینه ۱ نمی‌تواند *BST* باشد. زیرا ۹۱۲ در زیر درخت چپ ۹۱۱ است ولی از ۹۱۱ بزرگتر است. گزینه ۴ را خودتان بررسی کنید.

(۱۲) گزینه (۴). درخت زیر را در نظر بگیرید:



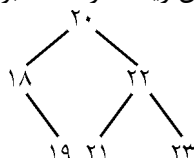
اگر $a=72$ و $b=82$ را در نظر بگیرید، گزینه‌های ۱ و ۲ نادرست هستند.

اگر $a=20$ و $b=18$ را در نظر بگیرید، گزینه ۳ نادرست است.

(۱۳) گزینه (۲). با داشتن دنباله سطح ترتیب از یک درخت دودوئی امکان ساخت درخت وجود ندارد. اما توجه کنید که در سؤال درخت جستجوی دودوئی مورد توجه قرار گرفته. با استفاده از خاصیت درخت جستجوی دودوئی نه تنها با این دنباله می‌توان درخت اولیه را تولید نمود بلکه درخت حاصل یکتا نیز خواهد بود. دقت کنید اولین گره در دنباله سطح ترتیب ریشه درخت دودوئی است و چون در این تست درخت جستجوی دودوئی مفروض است با مقایسه دیگر عناصر موجود در دنباله (از نظر مقدار) با ریشه به راحتی می‌توان گره‌های متعلق به زیر درخت چپ و راست آن را مشخص نمود. بدیهی است کلیه مقادیر کوچکتر از ریشه در سمت چپ و کلیه مقادیر بزرگتر از ریشه در سمت راست قرار دارند. به همین ترتیب می‌توان افزاز گره‌های سطوح بعدی را نیز انجام داد.

(۱۴) گزینه (۲). گزینه ۱ در حالت متوسط صحیح است. گزینه ۳ و ۴ نیز صحیح هستند.

(۱۵) گزینه (۱). باید گره بلافاصله بعدی (مینیمم زیر درخت راست) یا گره بلافاصله قبلی (ماکزیمم زیر درخت چپ) را جایگزین ریشه کرد. مثلاً برای درخت



هم می‌توان ۱۹ (گره بلافاصله قبلی) و هم ۲۱ (گره بلافاصله بعدی) را جایگزین ریشه کرد. گزینه ۱، ۱۹ را جایگزین ریشه می‌کند.

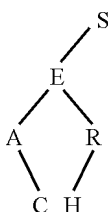
(۱۶) گزینه (۴). در این تست سطح ریشه صفر است. تعداد درخت‌های با ارتفاع $n-1$ (ارتفاع ماکزیمم) برابر 2^{n-1} است زیرا موقعیت ریشه ثابت است ولی $n-1$ نود دیگر می‌توانند فرزند چپ یا راست پدرشان باشند یعنی ۲ حالت دارند.

(۱۷) گزینه (۴). جدول زیر برای چهار ساختمان داده، بدترین حالت زمان اجرا را نشان می‌دهد.

	لیست نامرتب	لیست مرتب	heap	درخت جستجوی دودویی متوازن
درج	$O(1)$	$O(n)$	$O(\log n)$	$O(\log n)$
حذف	$O(n)$	$O(n)$	$O(\log n)$	$O(\log n)$
Findclosest	$O(n)$	$O(1)$	$O(n)$	$O(\log n)$

در واقع مزیت درخت جستجوی متوازن بر heap عمل findclosest است. زیرا در heap برای یافتن عنصر نزدیک x باید همه عناصر بررسی شوند ولی در BST متوازن succ و pred را می‌یابیم.

(۱۸) گزینه (۱). درخت به صورت مقابل است.



تعداد مقایسه‌ها	کاراکتر
۱	S
۲	E
۳	A
۳	R
۴	C
۴	H

$\Rightarrow$ تعداد کل مقایسه‌ها $= 1+2+3+3+4+4=17$

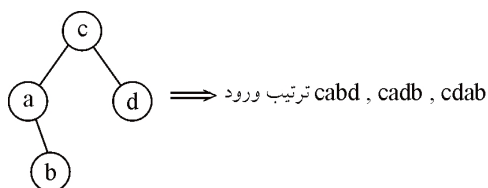
$$\text{متوسط مقایسه‌ها} = \frac{17}{6} = 2 \frac{5}{6}$$

۱۹) گزینه (۲). حداکثر، درخت شبیه مورب است.

۲۰) گزینه (۳). با مرتبه $O(n \cdot \lg n)$ می‌توان درخت ساخت.

۲۱) گزینه (۳). مینیمم زیر درخت راست (f) و یا ماکزیمم زیر درخت چپ (e).

۲۲) گزینه (۲).



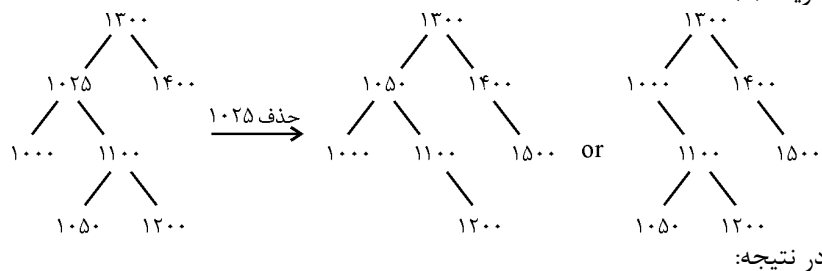
۲۳) گزینه (۲). یک دوران راست c ، یک دوران چپ e ، یک دوران راست g .

۲۴) گزینه (۴). مرتبه ساخت BST با n عنصر، از $n \lg n$ بیشتر یا مساوی است.

۲۵) گزینه (۱). پیمایش $inorder$ یک BST همیشه در دسترس است. برای رسم درخت، اطلاعات دیگری نیز لازم است.

۲۶) گزینه (۲). درخت مورب چپ می‌شود.

۲۷) گزینه (۴).



$preorder = 1300, 1050, 1000, 1100, 1200, 1400, 1500$

or

$preorder = 1300, 1000, 1100, 1050, 1200, 1400, 1500$

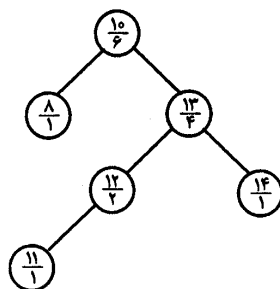
۲۸) گزینه (۴). البته حداقل تعداد سطوح $1 + \lceil \log_2^{150} \rceil$ یعنی ۸ است!

۲۹) گزینه (۲). نودهای کوچکتر x عبارتند از:

۱- تمام نودهای سمت چپ x

۲- اگر x فرزند راست پدرش باشد، تمام نودهای سمت چپ پدر x و خود پدر x مثلاً در

BST زیر، نودها به شکل $\frac{key}{size}$ می‌باشند.



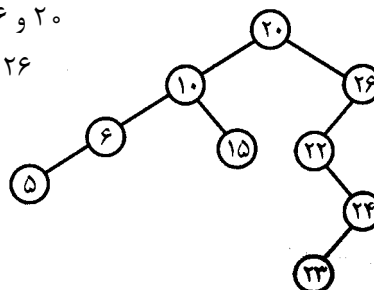
نود با کلید ۱۳ را در نظر بگیرید.

نودهای با کلید کمتر از ۱۳ عبارتند از ۱۲ و ۱۱ (سمت چپ ۱۳) و ۱۰ و ۸ (سمت چپ پدر ۱۳ و خود پدر ۱۳)

۳۰ گزینه (۴). می‌دانیم پیمایش $inorder$ یک درخت BST ، صعودی است، و می‌دانیم با داشتن دو پیمایش $inorder$ و $Preorder$ درخت قابل ترسیم است:

$Post: ۵$ و ۶ و ۱۵ و ۱۰ و ۲۳ و ۲۴ و ۲۲ و ۲۶ و ۲۰

$in: ۵$ و ۶ و ۱۰ و ۱۵ و ۲۰ و ۲۲ و ۲۳ و ۲۴ و ۲۶



پیمایش $preorder$ درخت حاصل گزینه ۴ می‌باشد.

۳۱ گزینه (۴). اگر ریشه AVL صفر فرض شود طبق فرمول گفته شده در متن کتاب:

$$N(h) = N(h-1) + N(h-2) + 1$$

$$N(0) = 1$$

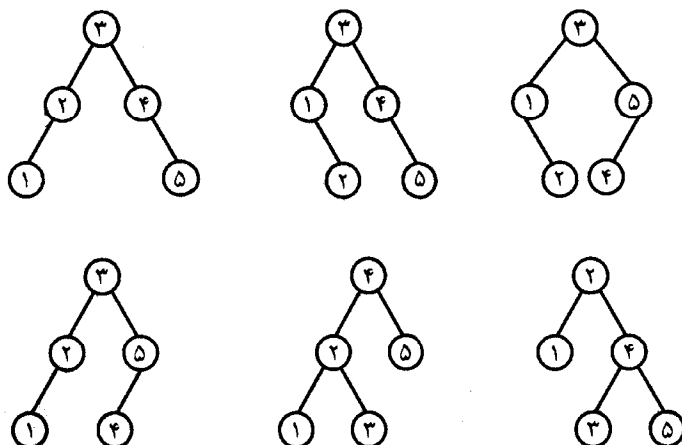
$$N(1) = 2$$

$$N(2) = N(1) + N(0) + 1 = 4$$

$$N(3) = N(2) + N(1) + 1 = 7$$

$$N(4) = N(3) + N(2) + 1 = 12$$

(۳۲) گزینه (۲).



(۳۳) گزینه (۲). برای ساخت AVL به ارتفاع h با کمترین تعداد نود، کفایت یک نود بسازیم، چپ آن نود AVL به ارتفاع $h-1$ و راست آن نود AVL به ارتفاع $h-2$ قرار دهیم پس

$$T(h) = T(h-1) + T(h-2) + 1$$

(۳۴) گزینه (۲). پس از حذف ۹۵، ۲۲ به جای آن قرار می‌گیرد و سپس روی ۲۲ عمل $heapify$ انجام می‌شود یعنی ۲۲ با ۸۵ سپس با ۵۵ تعویض می‌شود.

(۳۵) گزینه (۳). عمق درخت $heap$ همیشه $O(\log_2^n)$ است.

(۳۶) گزینه (۴). مرحله اول یعنی ساخت $heap$ ، گزینه ۱ با $heapify$ و گزینه ۲ با درج‌های متوالی $heap$ ساخته است. البته اگر اعداد یکی یکی وارد شوند با روش درج، در غیر این صورت با روش $heapify$ ، هیپ بسازیم.

(۳۷) گزینه (۲). مینیمم همواره در یکی از برگ‌ها است و برگ‌ها نیز از اندیس $\left\lfloor \frac{n}{2} \right\rfloor + 1$ تا اندیس

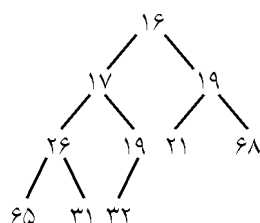
n هستند بنابراین $\left\lfloor \frac{n}{2} \right\rfloor$ نود باید بررسی شوند.

(۳۸) گزینه (۱). برای هر گره داخلی باید بررسی کنیم که فرزندانش از آن کوچکتر یا مساوی هستند یا خیر (برای $maxheap$). پس مرتبه $O(n)$ می‌شود. (یعنی از اندیس ۱ تا $\left\lfloor \frac{n}{2} \right\rfloor$ باید هر عنصر را با فرزندانش مقایسه کنیم)

۳۹ گزینه (۴). اگر $maxheap$ باشد، ریشه ماکزیمم است پس یافتن ماکزیمم از مرتبه $O(1)$ است ولی حذف آن نیاز به تنظیم دارد که از مرتبه $O(\log_p^n)$ است. درج عنصر نیز از مرتبه $O(\log_p^n)$ است.

۴۰ گزینه (۲). عنصر جدید باید در پائین‌ترین سطح، آخرین مکان قرار گیرد یعنی باید در اندیس n قرار گیرد ($A(n)$) سپس با پدر خود مقایسه شود اگر بیشتر باشد، باید با پدر تعویض شود به همین ترتیب تا جای صحیح آن مشخص شود. این الگوریتم نیز عنصر جدید را در $temp$ قرار می‌دهد و اندیس پدر آن را در i می‌گذارد. و تا زمانی که $temp$ از $A[i]$ بزرگتر است باید جابجایی صورت گیرد.

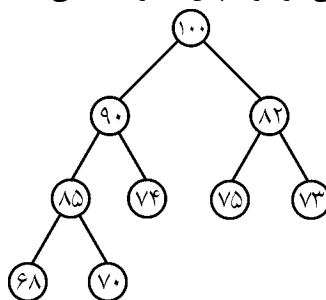
۴۱ گزینه (۳). ریشه حذف می‌شود، ۱۱ بجای ریشه قرار می‌گیرد سپس ابتدا با ۱۹ سپس با ۱۵ تعویض می‌شود.



۴۲ گزینه (۳). پس از انجام عملیات حذف و اضافه درخت به صورت مقابل تبدیل می‌شود. (اولین حذف، ۱۳ را حذف می‌کند و ۳۱ را بجای آن قرار می‌دهد و با ۳ جابجایی ۳۱ جای ۲۶ قرار می‌گیرد و به همین ترتیب)

۴۳ گزینه (۱). شکل درخت را می‌کشیم:

۹۵ به سمت چپ ۷۴ اضافه می‌شود و سپس با دو جابجایی، جای ۹۰ قرار خواهد گرفت.



۴۴ گزینه (۳). الگوریتم حذف یک عنصر از $Heap$

۴۵ گزینه (۴). عمق یعنی فاصله نود تا ریشه، عمق ریشه ($i = 1$) صفر است. عمق نودهای ۲ و ۳ برابر ۱ است. عمق نودهای ۴ تا ۷ برابر ۲ است و ...

۴۶ گزینه (۴). چهارمین بزرگترین می‌تواند در اندیس ۲ تا ۱۵ باشد (بررسی کنید). مثلاً در هیپ

۱	۲	۳	۴	۵	۶
۲۰	۵	۱۵	۳	۲	۱۰

عنصر با اندیس ۲ (عدد ۵) چهارمین بزرگترین است و یا در هیپ مقابل، عنصر با اندیس ۱۵، چهارمین بزرگترین است:

۱	۲	۳	۴	۵	۶	۷	۸	۹	۱۰	۱۱	۱۲	۱۳	۱۴	۱۵
۷۰	۵۰	۶۵	۴۵	۴۰	۳۰	۶۰	۳۵	۲۵	۲۰	۱۵	۱۰	۵	۱	۵۵

(۴۷) گزینه (۲). این درخت دارای ۷ سطح است، اعداد ۶۴ تا ۵۹ می‌توانند در سطوح ۱ تا ۶ و به صورت پدر و فرزند قرار گیرند و عدد ۵۸ بزرگترین عددی است که می‌تواند در سطح ۷ باشد.

(۴۸) گزینه (۱). الگوریتم ادغام بهینه لیست‌های مرتب مربوط به مبحث روش‌های حریصانه از درس طراحی الگوریتم‌هاست. الگوریتم آن به قرار زیر است:

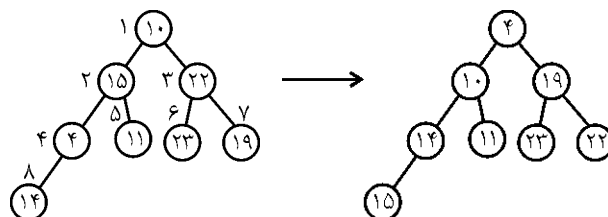
$Tree(n)$

```
{
    for (i=1; i<= n-1 ; i++)
    {
        Pt=new(treenode);
        Pt->Lchild=Least(list);
        Pt->rchild=Least(list);
        Pt->weight=pt-> Lchild ->weight+pt->rchild->weight;
        Insert(list, pt);
    }
}
```

در این الگوریتم، لیست در ابتدا شامل n گره منفرد و ورودی است که هر گره مبین یک لیست مرتب است. تابع $Least$ کوتاه‌ترین لیست را از مجموعه لیست‌ها حذف نموده و تابع $insert$ زیر درخت تشکیل شده در هر چرخه از الگوریتم را به مجموعه لیست اضافه می‌کند. بهترین روش ذخیره‌سازی این ساختار لیست استفاده از $min\text{-}heap$ است. چون عمل درج ($Insert$) و حذف کوچکترین عنصر ($Least$) از آن در زمان $\log n$ صورت می‌گیرد. یادآوری می‌گردد که هافمن نیز از ایده همین الگوریتم استفاده می‌کند.

گزینه (۲). الگوریتم ساخت *Minheap* با روش درج‌های متوالی است.

گزینه (۴). ابتدا روی عنصر ۴ عمل *Minheapify* انجام می‌دهیم که تغییری نمی‌کند
سپس روی عنصر ۳ (۲۲)، ۲۲ با ۱۹ تعویض می‌شود. سپس روی عنصر ۲ (۱۵) که ۱۵ با ۴
و سپس با ۱۴ تعویض می‌شود و در نهایت روی عنصر ۱ (۱۰) که ۱۰ با ۴ تعویض می‌شود و
نتیجه:

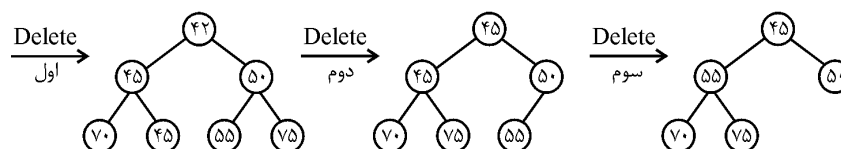
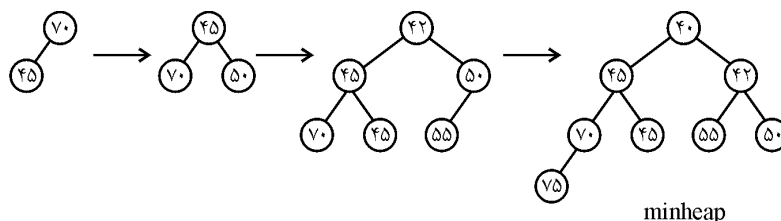


گزینه (۴). تعداد سطوح درخت ۱۰ تاست و حداقل ۹ عدد بیشتر از ۱۰۰۰ در سطوح ۱ تا ۹
هستند. پس $23 - 9 = 14$ جواب است. البته این ۱۴ عدد همزمان در پایین‌ترین سطح نیستند
بلکه هر یک می‌توانند در پایین‌ترین سطح باشند.

گزینه (۳). در سطح ۴ به بعد نمی‌تواند باشد. یعنی چهارمین بزرگترین می‌تواند در خانه ۲ تا
۱۵ قرار گیرد.

گزینه (۴). یک روش آن است که با n عنصر، یک *Minheap* بسازیم که ساخت هیپ با
بهترین روش از مرتبه n است. البته گزینه ۱ نیز صحیح است.

گزینه (۳).



گزینه (۳). ممکن است با عمل تعویض زیر درخت‌های چپ و راست، درخت حاصل دیگر
کامل نباشد پس هیپ نخواهد بود.

(۵۶) گزینه (۱). می‌دانیم ساخت هیپ با $n + m$ عنصر از مرتبه $\theta(n + m)$ است که در گزینه‌ها نیست. در این تست ریشه هیپ m عنصری را در زمان $\text{Log} m$ حذف و با زمان $\text{Log} n$ در هیپ n عنصری درج می‌کنیم پس:

$$m\text{Log} m + m\text{Log} n = m(\text{Log} m + \text{Log} n) = m\text{Log} mn$$

(۵۷) گزینه (۲). گره ۱ در سطح صفر است، گره‌های ۲ و ۳ در سطح ۱ هستند و به طور کلی گره شماره i در سطح $\lfloor \text{Lgi} \rfloor$ است. در سطح شماره x تعداد نود 2^x است (احتمالاً بجز سطح آخر). پس در سطحی که گره شماره i هست، تعداد نود حداکثر $2^{\lfloor \text{Lgi} \rfloor}$ است. برای یافتن نظیر گره i باید نصف گره‌های سطح گره i را به i اضافه کنیم و اگر حاصل از n بیشتر شد، آن را نصف کنیم. پس متناظر گره i گره $i + 2^{\lfloor \text{Lgi} \rfloor - 1} = i + \frac{2^{\lfloor \text{Lgi} \rfloor}}{2}$ است و اگر

$$j > n \quad j = \left\lfloor \frac{j}{2} \right\rfloor$$

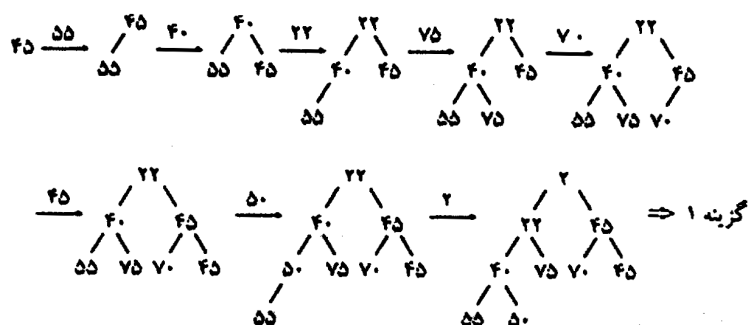
(۵۸) گزینه (۴). باید $T[i]$ از $T[2i]$ و $T[2i+1]$ بیشتر یا مساوی باشد.

(۵۹) گزینه (۳). درج در BST از آرایه و لیست پیوندی سریع‌تر است. درج در لیست پیوندی از آرایه سریع‌تر است.

(۶۰) گزینه (۴). سایر گزینه‌ها غلط هستند.

(۶۱) گزینه (۱). روابط پدر و فرزندی را بررسی کنید. در گزینه ۱، عدد ۵ پدر ۶ است ولی از ۶ کوچکتر است.

(۶۲) گزینه (۱). به ترتیب ورود عمل درج را انجام می‌دهیم:



۶۳ گزینه (۲). سومین کوچکترین می‌تواند در سطح دوم یا سوم باشد یعنی از اندیس ۲ تا اندیس ۷.

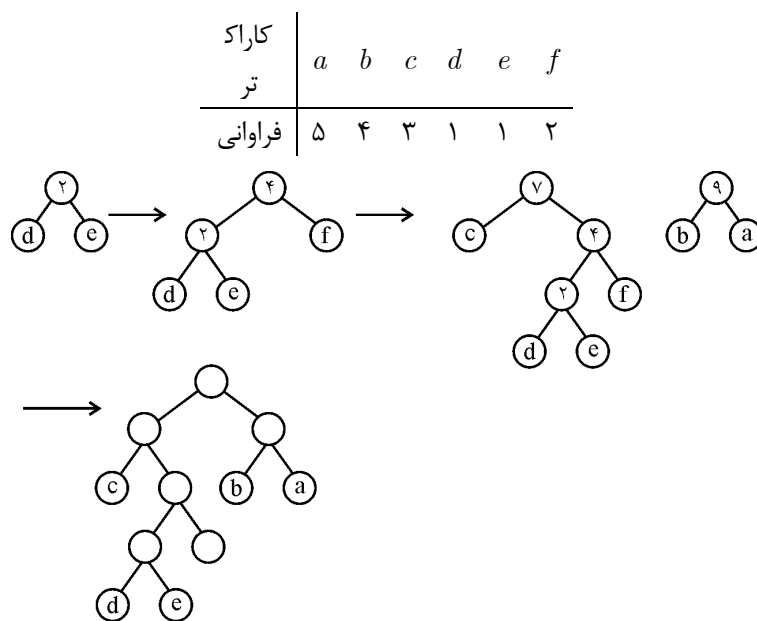
۶۴ گزینه (۲). اگر درخت هافمن را تشکیل دهید خواهید دید که e و d ، ۲ بیتی و بقیه ۳ بیتی می‌شود. پس:

$$1000 \times 3 + 1200 \times 3 + 800 \times 3 + 1500 \times 2 + 1800 \times 2 + 700 \times 3 = 17700$$

۶۵ گزینه (۳). به راحتی می‌توان به جواب رسید.

۶۶ گزینه (۱). می‌توانید کدهای کاراکترهای تست قبل، مثلاً a و b را با هم مقایسه کنید، که $a = 000$ و $b = 001$ است.

۶۷ گزینه (۱).



پس a و b و c ، ۲ بیتی هستند. f سه بیتی و d و e ۴ بیتی هستند بنابراین:

$$\text{طول متن} = (5 + 4 + 3) \times 2 + 2 \times 3 + (1 + 1) \times 4 = 38$$

۶۸ گزینه صحیح وجود ندارد. کلید اولیه سنجش گزینه ۴ است. اگر x برگ سمت چپ پدرش باشد و پدر x دو فرزندی باشد، $x > b$ صادق نیست. این تست حذف شد.

۶۹ گزینه (۳). منظور درختی است که تا سطح یکی مانده به آخرش پر است که البته در بیان سوال اشکال وجود دارد! این تست حذف شد.

(۷۰) گزینه (۴). گزینه ۱ توصیف AVL است. گزینه ۲ و ۳ توصیف $Maxheap$ هستند.

(۷۱) گزینه (۱). اگر همه داده‌ها متمایز باشند فقط یک درخت می‌توان ساخت.

(۷۲) گزینه (۱). با توجه به صورت سوال، با روش $heapify$ و از پایین به بالا هیپ می‌سازیم و تعویض‌ها عبارتند از:

$(21, 33), (20, 30), (23, 41), (23, 36), (16, 50), (16, 23), (16, 32)$.

(۷۳) گزینه (۲). عدد ۱ ریشه است. از ۶ عدد باقیمانده ۳ عدد برای زیر درخت چپ به $\begin{pmatrix} 6 \\ 3 \end{pmatrix}$ حالت

باید انتخاب کرد که خود ۲ حالت دارند. سه عددی که به زیر درخت راست می‌روند نیز ۲

$$\text{حالت دارند پس } \begin{pmatrix} 6 \\ 3 \end{pmatrix} \times 2 \times 2 = 80.$$

(۷۴) گزینه (۲).

	۱	۲	۳	۴	۵	۶	۷	۸	۹	۱۰	۱۱	۱۲	۱۳	۱۴	۱۵	۱۶	۱۷	۱۸
chang (۱۱ و ۱۶)	۲۰	۱۵	۱۸	۷	۹	۱۴	۱۶	۳	۶	۸	۱۶	۱۳	۱۰	۱۲	۱۱	۱	۲	۵

دو تعویض لازم است: $A[۱۱]$ با $A[۵]$ و سپس با $A[۲]$ باید تعویض شود:

	۱	۲	۳	۴	۵	۶	۷	۸	۹	۱۰	۱۱	۱۲	۱۳	۱۴	۱۵	۱۶	۱۷	۱۸
	۲۰	۱۶	۱۸	۷	۱۵	۱۴	۱۶	۳	۶	۸	۹	۱۳	۱۰	۱۲	۱۱	۱	۲	۵

	۱	۲	۳	۴	۵	۶	۷	۸	۹	۱۰	۱۱	۱۲	۱۳	۱۴	۱۵	۱۶	۱۷	۱۸
chang (۲ و ۴)	۲۰	۴	۱۸	۷	۱۵	۱۴	۱۶	۳	۶	۸	۹	۱۳	۱۰	۱۲	۱۱	۱	۲	۵

دو تعویض لازم است: $A[۲]$ با $A[۵]$ و سپس با $A[۱۱]$.

(۷۵) گزینه (۱). برای یافتن یک عنصر باید در بدترین حالت کل آرایه بررسی شود.

(۷۶) گزینه (۲). فایل شماره i دارای $\left\lfloor \frac{n_i}{r} \right\rfloor$ بلوک است پس مجموعاً $\sum_{i=1}^k \left\lfloor \frac{n_i}{r} \right\rfloor$ بلوک وجود دارد که

باید خوانده شود و چون n رکورد در نهایت وجود دارد پس $\left\lfloor \frac{n}{r} \right\rfloor$ بلاک وجود خواهد داشت که در فایل خروجی باید نوشته شوند.

(۷۷) گزینه (۳). b_n همان عدد کاتالان است که گزینه‌های ۱ و ۲ و ۴ صحیح هستند.

(۷۸) گزینه (۴). آرایه B را با مرتبه $mlgm$ مرتب می‌کنیم و عناصر A را در B جستجوی دودویی می‌کنیم که $nlgm$ زمان می‌خواهد پس زمان کل $m \lg m + n \lg m$ است.

۷۹) گزینه (۴). الگوریتم ساخت هیپ به صورت درجا و از پایین به بالاست. گره‌های سطح آخر درخت که همگی برگ هستند، *repeat* را اجرا نمی‌کنند، گره‌های سطح ماقبل آخر که تعدادشان 2^{k-1} است هر کدام حداکثر ۲ بار *repeat* را اجرا می‌کنند. گره‌های دو سطح قبل از آخر که تعدادشان 2^{k-2} است هر کدام حداکثر ۳ بار *repeat* را اجرا می‌کنند و به همین ترتیب تا در نهایت به ریشه برسیم که $k+1$ بار *repeat* را اجرا می‌کند. برای درک سوال یک درخت کامل کوچک بکشید و بررسی کنید.

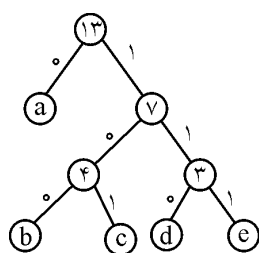
۸۰) گزینه (۲). هر نود حداکثر $2t-1$ کلید و در نتیجه $2t$ فرزند دارد پس حداکثر تعداد کلید برابر است با:

$$(2t-1) + 2t(2t-1) + (2t)^2(2t-1) + \dots + (2t)^h(2t-1) \\ = (2t-1)[1 + 2t + (2t)^2 + \dots + (2t)^h] = (2t)^{h+1} - 1$$

۸۱) گزینه (۴). *AVL* اغلب برای حافظه اصلی و *B-Tree* اغلب برای حافظه جانبی استفاده می‌شود.

۸۲) گزینه (۲). جدول مقابل فراوانی هر کاراکتر را نشان می‌دهد.

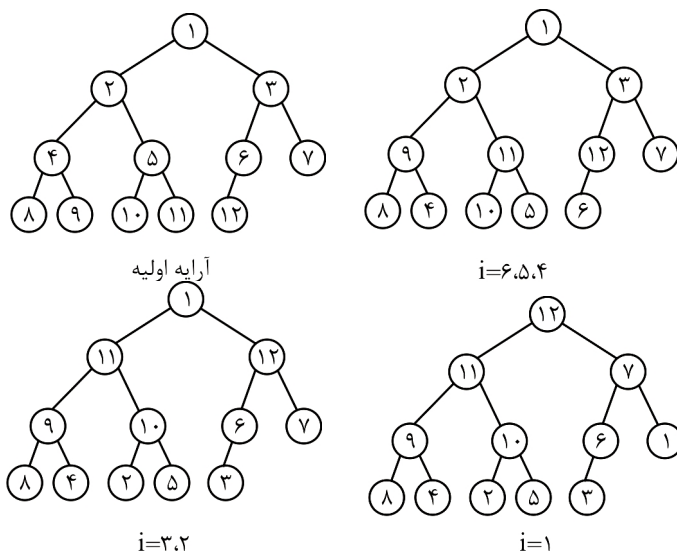
حرف	a	b	c	d	e
فراوانی	۶	۲	۲	۲	۱



حال دو کاراکتر با کمترین فراوانی را انتخاب می‌کنیم، و به همین ترتیب طبق روش گفته شده در متن، درخت را می‌سازیم:

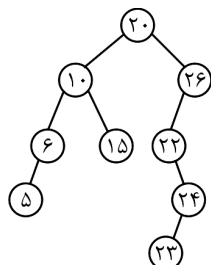
توجه: ممکن است شما درختی غیر از شکل بسازید و کدهای متفاوتی بدست آورید. مهم تعداد بیت‌های هر کاراکتر است یعنی *a* حتماً یک بیتی و بقیه ۳ بیتی باید باشند.

۸۳) گزینه (۳). از $i=6$ تا $i=1$ عمل *heapify* را انجام می‌دهیم: (یعنی هر نود را با فرزندانش مقایسه می‌کنیم و در صورت لزوم تعویض می‌کنیم. دقت کنید ساخت هیپ دو روش دارد، یک روش درج‌های متوالی است که در این تست منظور نیست و روش دوم به صورت درجا یعنی از آخرین نود داخلی به سمت ریشه حرکت کنیم و عمل *heapify* انجام دهیم).



همانطور که ملاحظه می‌شود، در پایان در خانه ۵ عدد ۱۰ قرار دارد. اگر این تست را با روش درج حل کنید جواب اشتباه خواهد شد.

گزینه (۲). پیمایش *inorder* یک *BST* لیست مرتب صعودی است و با داشتن پیمایش‌های *inorder* و *postorder* درخت به صورت منحصر به فرد مشخص می‌شود:

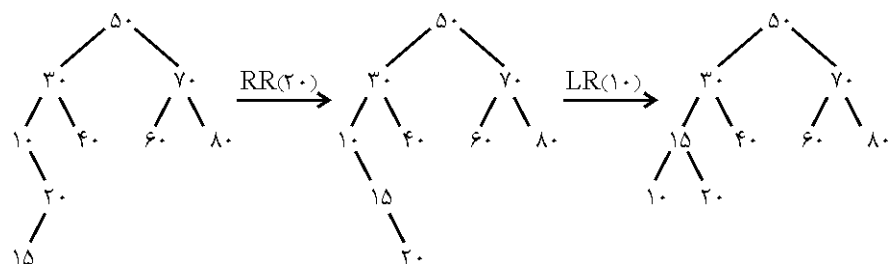


Preorder : ۲۰, ۱۰, ۶, ۵, ۱۵, ۲۶, ۲۲, ۲۴, ۲۳

گزینه (۳). اعداد ۴ و ۹ در صورتی با هم مقایسه نمی‌شوند که یکی از اعداد ۸ و ۷ و ۶ و ۵ قبل از آنها به درخت اضافه شوند. پس اگر ۴ یا ۹ زودتر از ۵ و ۶ و ۷ و ۸ به درخت اضافه شوند، با هم مقایسه می‌شوند. احتمال اینکه ۴ یا ۹ زودتر از ۵ و ۶ و ۷ و ۸ به درخت اضافه شود برابر $\frac{2}{6}$ است (دو به خاطر ۴ و ۹، شش به خاطر تعداد اعداد ۴ تا ۹).

گزینه (۱). هر عنصری که درج می‌شود، مقدار *count* یک واحد اضافه می‌شود و کلید عنصر قرار می‌گیرد، و عنصر جدید به ریشه وارد می‌شود (*Max heap*). حذف نیز همیشه از ریشه انجام می‌شود. یعنی آخرین عنصری که درج شده، اولین عنصری است که حذف می‌شود (*LIFO*).

۸۷) گزینه (۳). کلید ۱۵ فرزند چپ ۲۰ می‌شود سپس باید ۲۰ را به راست و سپس ۱۰ را به چپ دوران دهیم:



پیمایش pre درخت حاصل عبارت است از:

$pre : 50, 30, 15, 10, 20, 40, 70, 60, 80$

۸۸) گزینه (۳). حذف هر عنصری از هیپ از مرتبه $O(\lg n)$ است.

گراف فصل ۷

گراف

گراف G شامل دو مجموعه V (رئوس $Vertices$ جمع $Vertex$) و E (یال‌ها، لبه‌ها $Edges$) می‌باشد. E مجموعه‌ای از زوج‌ها است که اگر زوج‌ها مرتب باشند، گراف جهت‌دار ($digraph$) است و اگر زوج‌ها غیرمرتب باشند، گراف غیرجهت‌دار ($undirected graph$) است. در برخی منابع یال‌های جهت‌دار را به صورت $\langle a, b \rangle$ یا (a, b) نمایش می‌دهند و یال‌های غیرجهت‌دار را به صورت (a, b) یا $\{a, b\}$ یا ab نشان می‌دهند.

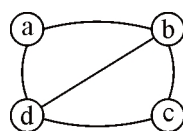
در این کتاب یال جهت‌دار را به صورت $\langle a, b \rangle$ (که a را $tail$ و b را $head$ گویند) و یال غیرجهت‌دار را به صورت (a, b) نمایش می‌دهیم.

مثال ۱: گراف $G_1 = (V_1, E_1)$ که $V_1 = \{a, b, c, d\}$ و

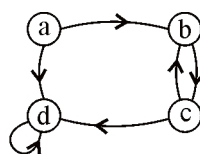
$E_1 = \{(a, b), (b, c), (c, d), (a, d), (b, d)\}$ غیرجهت‌دار و گراف $G_2 = (V_2, E_2)$ که

$$V_2 = \{a, b, c, d\}$$

و $E_2 = \{\langle a, b \rangle, \langle b, c \rangle, \langle c, b \rangle, \langle c, d \rangle, \langle a, d \rangle, \langle d, d \rangle\}$ جهت‌دار است:



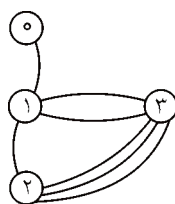
a) G_1



b) G_2

شکل ۱

یال $\langle d, d \rangle$ در G_p یک خود حلقه (*selfLoop* یا *selfedge*) است. در برخی منابع، وجود خود حلقه در گراف مجاز نیست. اگر در گرافی بین دو راس چندین یال باشد (*multiedge*)، گراف را چندگانه (*multigraph*) گویند:



شکل ۲

در برخی منابع گفته شده که *multigraph* نمی‌تواند *selfloop* داشته باشد. درجه (*degree*) یک راس در گراف غیرجهت‌دار برابر تعداد یال‌هایی است که از آن راس عبور کرده یا برخورد کرده، مثلاً در گراف G_p شکل ۱، $\deg(c) = \deg(a) = 2$ و $\deg(b) = \deg(d) = 3$ و در شکل ۲، $\deg(3) = 5$. در گراف جهت‌دار، درجه ورودی (*indegree*) و درجه خروجی (*outdegree*) تعریف می‌شود.

در گراف G_p شکل ۱:

$$\text{in deg}(a) = 0, \quad \text{out deg}(a) = 2$$

$$\text{in deg}(d) = 3, \quad \text{out deg}(d) = 1$$

درجه یک راس در گراف جهت‌دار برابر مجموع *indeg* و *outdeg* است مثلاً در شکل ۱ گراف G_p ، درجه راس d برابر ۴ است.

در گراف غیرجهت‌دار، جمع درجات همه رئوس دو برابر تعداد یال‌هاست و در گراف جهت‌دار، جمع درجات ورودی رئوس با جمع درجات خروجی رئوس با تعداد یال‌ها مساوی است.

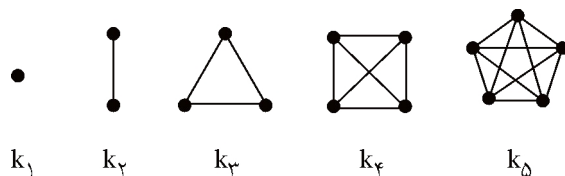
معمولاً گرافی که *selfloop* و *multiedge* ندارد گراف ساده (*simple*) گویند.

در اکثر منابع منظور از گراف، گراف ساده است، در این کتاب نیز چنین است.

گراف کامل (*complete*) گرافی است که ماکزیمم تعداد یال را دارد. پس گراف کامل غیر

جهت‌دار n راسی دارای $\binom{n}{2} = \frac{n(n-1)}{2}$ یال و جهت‌دار کامل دارای $n(n-1)$ یال است. در

شکل ۳ گراف‌های کامل ۱ تا ۵ راسی بدون جهت کشیده شده‌اند:



شکل ۳

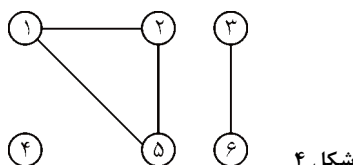
یک مسیر (*path*) از راس u به راس v ، دنباله‌ای است از رئوس و یال‌ها که از u شروع و به v ختم می‌شود. مثلاً در گراف G_7 شکل ۱، دنباله $P_1 = [a, b, c, d]$ یک مسیر از a به d به طول ۳ و $P_7 = [a, b, c, b]$ یک مسیر از a به b به طول ۳ است (طول مسیر برابر تعداد یال‌های مسیر است). مسیر را ساده گویند (*simple*) اگر همه رئوس آن (احتمالاً بجز راس شروع و پایان) متمایز باشند پس P_1 مسیر ساده هست ولی P_7 خیر.

مسیر ساده‌ای که راس شروع و پایان آن یکسان باشد، سیکل (*Cycle*، دور) گویند.

مثلاً در شکل ۱ گراف G_1 ، دنباله $P_7 = [a, b, d, a]$ یک سیکل به طول ۳ است.

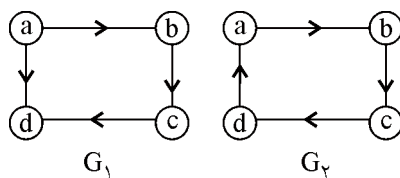
به گرافی که سیکل ندارد *acyclic* گویند.

گراف غیرجهت‌دار را متصل (همبند *connected*) گویند اگر بین هر دو راس آن حداقل یک مسیر وجود داشته باشد مثلاً گراف شکل ۴ ناهمبند است و دارای ۳ مولفه همبندی $\{1$ و 2 و $5\}$ و $\{4\}$ و $\{3$ و $6\}$ است:



شکل ۴

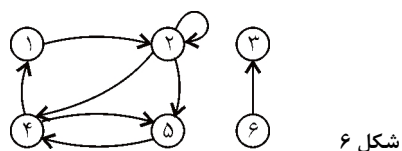
یک گراف جهت‌دار، را قویاً همبند (*strongly connected*) گویند اگر به ازای هر دو راسی بتوان از هر کدام به دیگری رسید یعنی به ازای هر دو راس a و b هم از a به b و هم از b به a مسیری وجود داشته باشد مثلاً در شکل ۵، G_1 همبند قوی نیست و G_2 متصل قوی هست:



شکل ۵

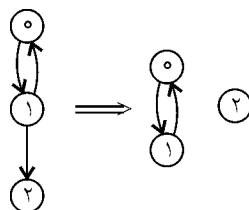
مولفه‌های همبند قوی یک گراف جهت‌دار، یک افراز (*partition*) برای رئوس گراف هستند که رئوس هر مولفه‌ای همبند قوی هستند مثلاً گراف شکل ۶ دارای ۳ مولفه همبند قوی $\{1$ و 2 و $5\}$ و $\{4\}$ و $\{3$ و $6\}$ است:

$\{1\}$ و $\{6\}$ و $\{3\}$ است زیرا هر جفت راس در $\{5\}$ و $\{4\}$ و $\{2\}$ و $\{1\}$ با هم متصل قوی هستند و 3 و 6 با هم متصل قوی نیستند:



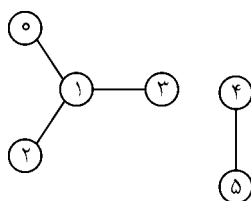
شکل ۶

و یا گراف شکل ۷ دارای دو مولفه همبند قوی $\{1\}$ و $\{0\}$ است:

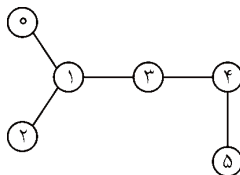


شکل ۷

یک گراف غیرجهت‌دار بدون سیکل را جنگل (*forest*) گویند:



و یک گراف غیر جهت‌دار بدون سیکل همبند را درخت (*free tree, tree*) گویند:



در هر درخت $n = e + 1$ (n تعداد رئوس و e تعداد یال‌ها) و بین هر دو راس فقط و فقط یک

مسیر ساده وجود دارد، همچنین درخت گرافی است همبند با کمترین تعداد یال.

یک گراف جهت‌دار بدون سیکل را *dag* (*directed acyclic graph*) گویند.

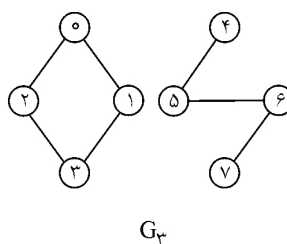
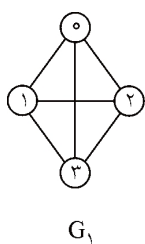
روش‌های نمایش گراف

اگر چه روش‌های مختلفی برای نمایش گراف امکان‌پذیر است، سه روش متداول را بررسی می‌کنیم:

ماتریس مجاورتی، لیست مجاورتی، لیست چندگانه مجاورتی.

ماتریس مجاورتی

فرض کنید G یک گراف n راسی است، ماتریس مجاورتی یک آرایه $n \times n$ مثل A است که درایه‌های آن صفر و یک است. اگر راس i و j مجاور باشند $A[i][j] = 1$ و اگر مجاور نباشند $A[i][j] = 0$.



$$A_1 = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

$$A_2 = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

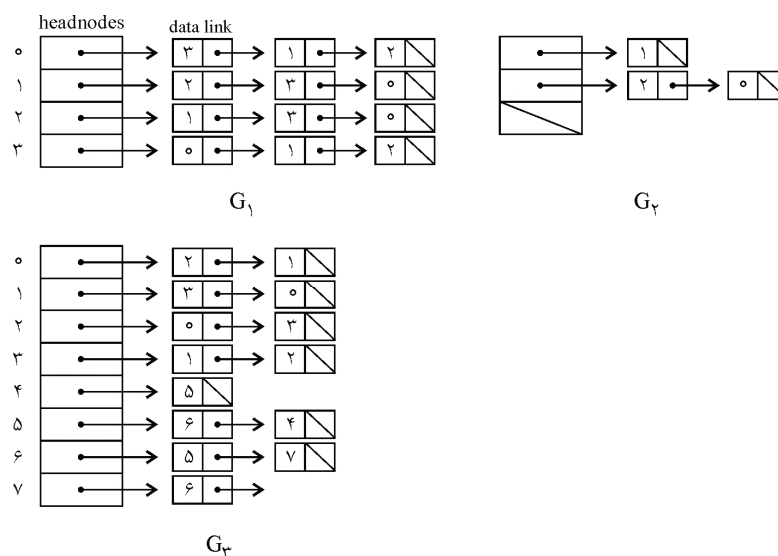
$$A_3 = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

ماتریس مجاورتی گراف غیر جهت‌دار متقارن است پس می‌توان فقط مثلث زیر قطر یا بالای قطر اصلی را ذخیره کرد یعنی $\frac{n(n-1)}{2}$ بیت نیاز است. ولی ماتریس گراف جهت‌دار لزوماً متقارن نیست. جمع درایه‌های سطر یا ستون i در گراف غیر جهت‌دار برابر درجه راس i است. در گراف جهت‌دار، جمع درایه‌های هر سطر درجه خروجی و ستون درجه ورودی راس نظیرش است.

پاسخ سوالات متداولی از قبیل، چند یال در گراف G وجود دارد؟ یا آیا G همبند است؟ با ماتریس مجاورتی حداقل $O(n^2)$ زمان لازم است. ولی اگر گراف اسپارس باشد یعنی اکثر درایه‌های ماتریس آن صفر باشد $\left(e \ll \frac{n^2}{2}\right)$ می‌توان با لیست مجاورتی به این سوالات در زمان $O(n+e)$ پاسخ داد.

لیست مجاورتی

n تا لیست وجود دارد، به ازای هر رأس یک لیست. نودهای لیست i رئوسی هستند که رأس i با آنها مجاور است. اگر گراف جهت‌دار باشد، رئوس لیست i رئوسی هستند که از رأس i خارج می‌شوند:



برای گراف غیرجهت‌دار، n تا $head\ node$ و $2e$ تا $list\ node$ لازم است و هر لیست نود، دارای ۲ فیلد است.

برای صرفه‌جویی در حافظه می‌توان روشی پیشنهاد کرد که $link$ ها ذخیره نشوند. یک آرایه $node[n + 2e + 1]$ می‌توان استفاده کرد که $node[i]$ نقطه شروع لیست برای رأس i را نشان می‌دهد و $node[n]$ به $n + 2e + 1$ مقداردهی می‌شود پس رئوسی که با رأس i مجاور هستند در $node[i] - 1$ تا $node[i + 1]$ ذخیره شده‌اند.

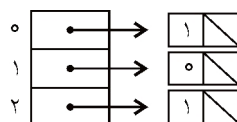
مثلاً نمایش G_3 به شکل زیر است:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
9	11	13	15	17	18	20	22	23	2	1	3	0	0	3	1	2	5	6	4	5	7	6

درجه هر رأس در گراف غیر جهت‌دار به راحتی با شمارش تعداد نودهای لیستش بدست می‌آید پس تعداد یال‌های گراف را با زمان $O(n+e)$ می‌توان تعیین کرد.

برای گراف جهت‌دار تعداد لیست نودها برابر e است. درجه خروجی هر رأس با شمارش تعداد نودهای لیستش حاصل می‌شود، پس تعیین تعداد یال‌ها در زمان $O(n+e)$ امکان‌پذیر است.

تعیین درجه ورودی رئوس دشوار است و اگر مرتباً نیاز به این عمل باشد بهتر است لیست مجاورتی معکوس را نیز ساخت. لیست مجاورتی معکوس G_p عبارت است از:



inverse adjacency Lists for G_p

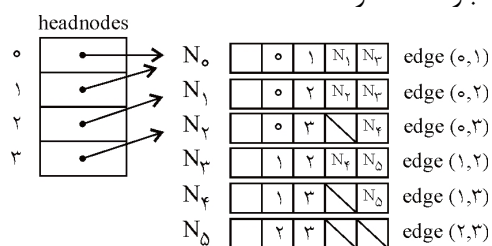
لیست چندگانه مجاورتی (Adjacency Multilists)

در لیست مجاورتی یک گراف غیر جهت دار، هر یال (u, v) دو بار در نودلیست‌ها ظاهر می‌شود: هم در لیست u و هم در لیست v . در لیست چندگانه، هر یال فقط یک بار ظاهر می‌شود در واقع یک یال می‌تواند مشترک در دو لیست باشد. شکل نودهای لیست چندگانه به صورت زیر است:

m	$vertex_1$	$vertex_p$	$list_1$	$list_p$
-----	------------	------------	----------	----------

فیلد m بولین است و مشخص می‌کند آیا یال بررسی شده یا خیر (در برخی کاربردها لازم است)

لیست چندگانه G_1 عبارت است از:



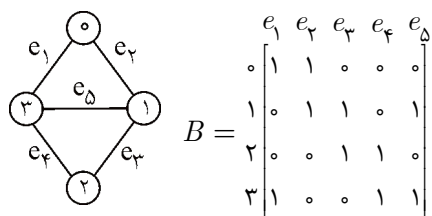
the lists are. $vertex_0 : N_0 \rightarrow N_1 \rightarrow N_2$

$vertex_1 : N_0 \rightarrow N_3 \rightarrow N_4$

$vertex_2 : N_1 \rightarrow N_3 \rightarrow N_5$

$vertex_3 : N_2 \rightarrow N_4 \rightarrow N_5$

نکته: روش دیگری برای نمایش گراف موسوم به ماتریس برخورد (incidence) وجود دارد که $n \times e$ است. به عنوان مثال:



واضح است که جمع اعداد هر ستون برابر ۲ می‌باشد (گراف بدون *self Loop*) و جمع اعداد هر سطر، درجه راس نظیر را نشان می‌دهد.

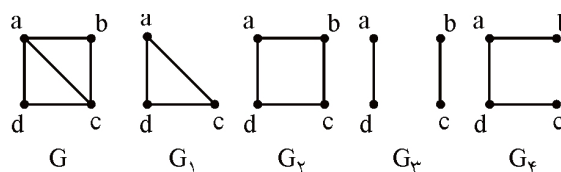
تعریف زیرگراف و انواع آن

۱- اگر $G=(V,E)$ یک گراف باشد آنگاه $G'=(V',E')$ که $V' \subseteq V$ و $E' \subseteq E$ را یک زیرگراف G می‌گویند.

۲- در زیرگراف، اگر $V' = V$ یعنی زیرگراف شامل همه رئوس گراف باشد، آن را زیرگراف پوشا می‌گویند.

۳- اگر زیرگراف پوشا، خود درخت باشد (همبند باشد و دور نداشته باشد) به آن درخت پوشا (*Spanning Tree*) می‌گویند.

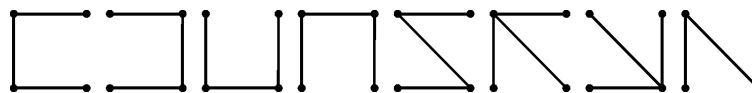
مثال ۲: با توجه به گراف G ، G_1 زیرگراف G هست ولی پوشا نیست چون راس b را ندارد. G_2 زیرگراف پوشاست ولی درخت پوشا نیست چون دور دارد. G_3 زیرگراف پوشاست ولی درخت پوشا نیست چون همبند نیست. G_4 درخت پوشاست.



مثال ۳: تمام درخت‌های پوشای گراف را رسم کنید.

$G=(V, E)$

پاسخ: این گراف دارای ۸ درخت پوشاست. همانطور که ملاحظه می‌شود برای ساخت درخت پوشا در این مثال، ۲ یال گراف حذف می‌شود:



مثال ۴: از گراف کامل k_1 چند یال حذف شود تا درخت پوشا حاصل شود؟

پاسخ: k_1 دارای $\frac{1 \times 9}{2} = 45$ یال است و درخت پوشای آن باید ۹ یال داشته باشد چون

تعداد یال‌های درخت، از رئوس، یک واحد کمتر است. پس:

$$45 - 9 = 36 = \text{تعداد یال‌های حذفی}$$

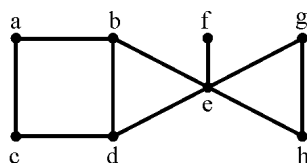
الگوریتم‌های پیمایش گراف

هدف ملاقات کلیه رئوس گراف G هر کدام فقط یک بار است. دو الگوریتم مشهور در این زمینه وجود دارد: ۱- جستجوی عمقی ۲- جستجوی سطحی.

۱- جستجوی عمقی (Depth First Search: DFS)

ابتدا راس v را ملاقات می‌کنیم سپس راسی مانند w که مجاور v هست و قبلاً ملاقات نشده را ملاقات می‌کنیم. این عمل را با w انجام می‌دهیم یعنی یکی از رئوس مجاور w را انتخاب کرده، ملاقات می‌کنیم. این عمل را تا زمانی انجام می‌دهیم که تمام رئوس گراف ملاقات شوند. اگر جایی به گره‌ای رسیدیم که امکان ملاقات کردن هیچ راس دیگری وجود نداشت، یک مرحله به عقب برمی‌گردیم.

مثال ۵: اگر از راس a شروع کنیم و به رئوس اولویت دهیم و اولویت به ترتیب حروف الفبای انگلیسی باشد (یعنی a بعد b و ...) پیمایش dfs گراف مقابل را بیابید.

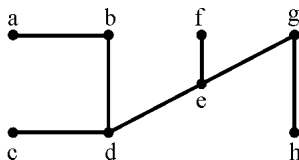


پاسخ: ابتدا a را ملاقات می‌کنیم از a هم می‌توان b و هم c را ملاقات کرد ولی چون صورت مسئله اولویت b را بیشتر فرض کرده، b را ملاقات می‌کنیم از b راس d و از d راس c را ملاقات می‌کنیم. چون از c نمی‌توان راس جدیدی را ملاقات کرد یک مرحله به عقب برمی‌گردیم، یعنی به راس d باز می‌گردیم و از d راس e و از e راس f را ملاقات می‌کنیم. از f به عقب برمی‌گردیم. از e راس g را و از g راس h را ملاقات می‌کنیم:

ترتیب ملاقات: $a \ b \ d \ c \ e \ f \ g \ h$

نکته: اگر ترتیب ملاقات‌ها را رسم کنیم، یک درخت پوشا حاصل می‌شود که به آن درخت

پوشای dfs می‌گویند.



نکته: الگوریتم این روش بازگشتی است و از پشته استفاده می‌کند:

```

Procedure dfs( $V: \text{Vertex}$ )
begin
    writeln(data( $V$ ));
    Visited[ $V$ ]:=true;
    for (each vertex  $W$  adjacent to  $V$ ) do
        if (not visited[ $W$ ]) then dfs( $W$ );
end;
```

لازم به ذکر است در الگوریتم *dfs*، آرایه *Visited* از نوع بولین است که مقادیر اولیه آن *False* می‌باشد و هر راس که ملاقات شود، عنصر مربوط به آن در این آرایه، *true* می‌شود.

انواع یالها

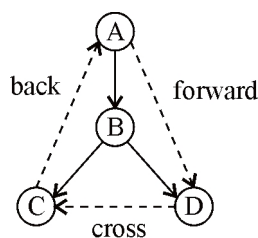
اگر درخت حاصل از پیمایش *DFS* گراف *G* را *T* بنامیم آنگاه با توجه به آن ۴ نوع یال می‌توان تعریف کرد:

۱- یال درختی (*tree edge*): یالهایی که در درخت *T* وجود دارند. (سه نوع دیگر یالها در گراف *G* هستند ولی در درخت *T* نیستند)

۲- یال جلو (Forward edge): از یک نود به یک نود در زیر درخت آن نود. (از یک نود به نواده‌اش)

۳- یال پشتی (*back edge*): از یک نود به جدش

۴- یال عبوری (*cross edge*): هیچکدام از یالهای فوق. مثلاً اگر درخت حاصل از *DFS* به شکل مقابل باشد آنگاه:



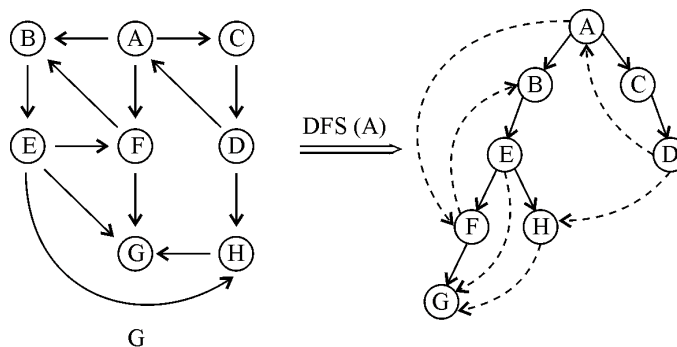
AB, BC, CD : tree edge

AD : forward edge

CA : back edge

DC : cross edge

مثال ۴: پیمایش *DFS(A)* گراف *G* را با فرض اولویت حروف الفبا بیابید و انواع یالها را مشخص کنید.



$AB, BE, EF, FG, EH, AC, CD$: tree edge

AF, EG : forward edge

FB, DA : back edge

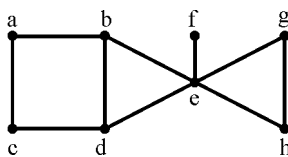
DH, HG : cross edge

نکته: یک گراف جهت‌دار دارای سیکل است، اگر و فقط اگر DFS آن، یال پشتی داشته باشد.

۲- جستجوی سطحی (breadth first search: bfs)

از گره v شروع می‌کنیم، سپس کلیه گره‌های مجاور این گره را ملاقات می‌کنیم. برای این منظور از یک صف استفاده می‌کنیم و هر راس که ملاقات می‌شود رئوس مجاور آن را به صف وارد می‌کنیم. سپس از سر صف یک عنصر را حذف کرده، ملاقات می‌کنیم رئوس مجاور این راس را به صف وارد می‌کنیم. این عمل را تا زمانیکه صف خالی نشده، ادامه می‌دهیم.

مثال ۷: اگر از a شروع کنیم و به رئوس اولویت دهیم. پیمایش bfs گراف مقابل را بیابید.



پاسخ: ابتدا a را ملاقات می‌کنیم و b و c وارد صف می‌کنیم. b را از صف برداشته، ملاقات می‌کنیم و رئوس مجاور آن یعنی d و e را وارد صف می‌کنیم. سپس c را از صف برداشته ملاقات می‌کنیم ولی چیزی وارد صف نمی‌کنیم. و به همین ترتیب تا زمانیکه صف خالی شود و همه رئوس ملاقات شوند ادامه می‌دهیم:

ترتیب ملاقات: $a \ b \ c \ d \ e \ f \ g \ h$

نکته: اگر ترتیب ملاقات‌ها را رسم کنیم یک درخت پوشا حاصل می‌شود که به آن درخت پوشای bfs می‌گویند.

Procedure bfs ($V:Vertex$);

begin

Visited[V]:=true;

Addq(q, V);

while not emptyqueue(q) do

begin

delq(q, V);

for all nodes W adjacent to V do begin

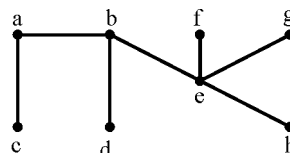
addq(q, W);

Visited[W]:=true;

end

end

end;



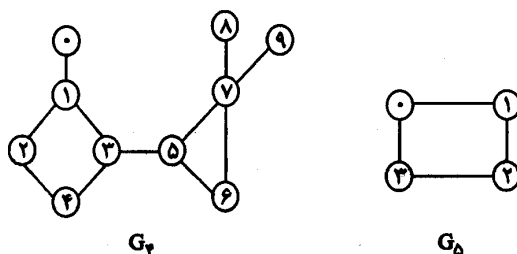
نکته: اگر گراف همبند با لیست مجاورتی نمایش داده شود آنگاه مرتبه دو روش bfs و dfs ,

$O(e)$ خواهد بود و اگر گراف با ماتریس مجاورتی نمایش داده شود مرتبه این دو روش $O(n^2)$ می‌باشد. (e تعداد یال‌ها و n تعداد رئوس)

نکته: با استفاده از الگوریتم‌های bfs و dfs می‌توان همبند یا ناهمبند بودن یک گراف را تشخیص داد. به این ترتیب که اگر پس از اتمام الگوریتم تمام رئوس ملاقات شدند، گراف همبند است. در غیر این صورت ناهمبند است. همچنین می‌توان مولفه‌های همبند گراف را نیز بدست آورد که با لیست مرتبه‌اش $O(n+e)$ و با ماتریس $O(n^2)$ است.

◀ **نکته:** BFS طول کوتاه‌ترین مسیرها (برحسب تعداد یال) را از راس مبدا به سایر رئوس می‌دهد.

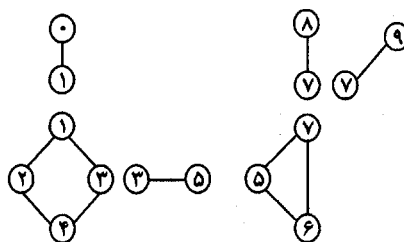
تعریف: راس a را نقطه انفصال (*articulation point*) گویند اگر و فقط اگر حذف a و همه یال‌های مجاورش، مولفه‌های گراف را بیشتر کند (یعنی اگر گراف همبند است، آن را ناهمبند کند). مثلاً در گراف G ، رئوس ۱ و ۳ و ۵ و ۷ رئوس انفصال هستند:



ولی گراف G_D نقطه انفصال ندارد.

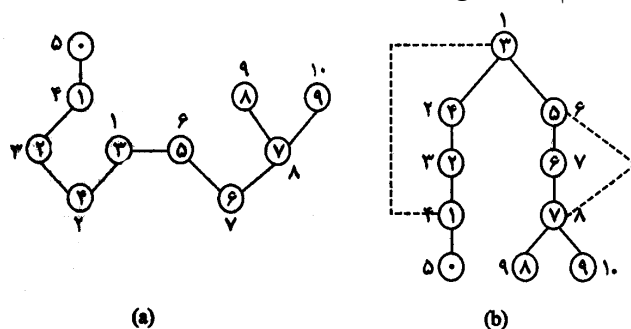
تعریف: گرافی که نقطه انفصال ندارد، گراف *biconnected* (همبند دو طرفه) گویند. پس G_D ، *biconnected* است ولی G_F نیست.

تعریف: بزرگترین زیر گراف‌های دو طرف همبند گراف را مولفه‌های *biconnected* (دو طرف همبند) گویند. مولفه‌های دو طرف همبند G_F عبارتند از:



گراف G_D فقط یک مولفه دو طرفه همبند دارد، خودش. دقت کنید که مولفه‌ها حداکثر یک رأس مشترک دارند و یال مشترک ندارند پس یال‌ها را افزاز می‌کنند. یافتن مولفه‌های دو طرف همبند گراف از مرتبه $O(n + e)$ است.

پیمایش *dfs* گراف G_F با شروع از رأس ۳ عبارت است از (a):



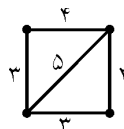
کنار هر نود یک عدد نوشته شده که ترتیب ملاقات را در پیمایش *dfs* نشان می‌دهد. به این اعداد *dfn* (*depth-first number*) گویند مثلاً $dfn(0) = 5$ یعنی رأس ۰، پنجمین رأس ملاقات شده است. در شکل (b) فوق، درخت پوشا را به فرم درختی آن رسم کرده‌ایم. یال‌هایی که

با خط چین مشخص شده‌اند، یال‌های غیردرختی هستند (یال‌هایی که در گراف G_f بودند ولی در درخت پوشا نیستند) یال غیردرختی (u, v) را یال پشتی (*back edge*) گویند اگر در درخت پوشای آن یا u جد v باشد یا v جد u . بنابراین یال‌های (۱ و ۳) و (۷ و ۵) یال پشتی هستند. یال غیردرختی که یال پشتی نیست، یال عبوری (*cross edge*) گویند. می‌توان نشان داد که در پیمایش *dfs* گراف غیرجهت‌دار یال عبوری و یال پیش رو وجود ندارد.

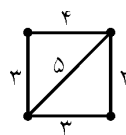
❖ **نکته:** کاربردهای *BFS* عبارتند از: ۱- تشخیص دو بخشی بدون گراف ۲- یافتن کوتاه‌ترین مسیرهای هم مبدأ در گراف غیروزن‌دار.

کاربردهای *DFS* عبارتند از: ۱- یافتن نقاط انفصال گراف ۲- یافتن مولفه‌های همبند قوی ۳- ترتیب توپولوژیکی

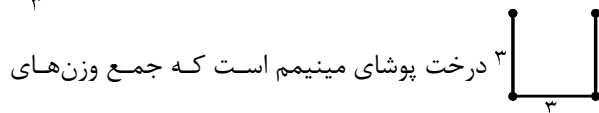
گراف وزن‌دار (Network): گرافی است که به هر یال آن یک عدد مثبت، نسبت داده شود:



درخت پوشای مینیمم



درخت پوشایی از یک گراف که جمع وزن‌های یال‌های آن کمترین باشد. مثلاً گراف



دارای ۸ درخت پوشاست که فقط ۳ درخت پوشای مینیمم است که جمع وزن‌های

آن $2+3+3=8$ می‌باشد، و از سایر درخت‌های پوشا، وزن کمتری دارد.

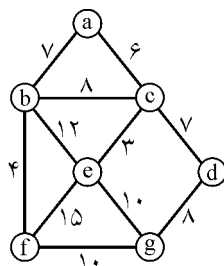
۳ روش برای یافتن درخت پوشای مینیمم ارائه می‌دهیم: ۱- الگوریتم کراسکال (*Kruskal*)

۲- الگوریتم پریم (*Prim*) ۳- الگوریتم سولین (*Sollin*)

۱- الگوریتم کراسکال (راشال)

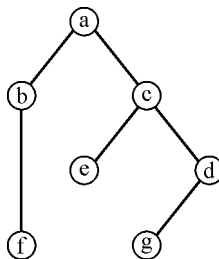
ابتدا یال‌ها را به ترتیب صعودی وزن‌ها، مرتب می‌کنیم. سپس یال‌ها را به ترتیب انتخاب می‌کنیم. این عمل را تا زمانی انجام می‌دهیم که درخت پوشا حاصل شود. فقط باید توجه داشت که دور تشکیل نشود.

مثال ۸: درخت پوشای مینیمم گراف مقابل را با الگوریتم کراسکال بیابید.



پاسخ: ابتدا یال‌ها را به صورت صعودی مرتب می‌کنیم:

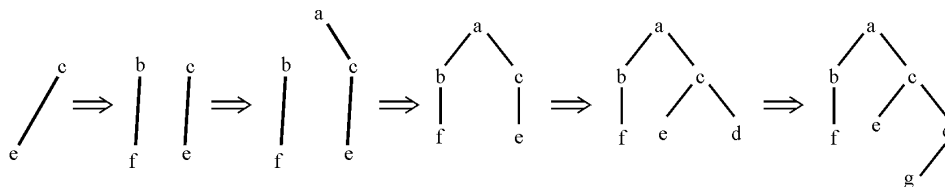
یال	وزن
ce	۳
bf	۴
ac	۶
ab	۷
cd	۷
bc	۸
dg	۸
eg	۱۰
fg	۱۰
be	۱۲
ef	۱۵



وزن درخت حاصل $= 3 + 4 + 6 + 7 + 7 + 8 = 35$

حال به ترتیب، مینیمم یال‌ها را انتخاب می‌کنیم و به هم وصل می‌کنیم و هر یال که انتخاب می‌شود علامت می‌زنیم.

توجه کنید که مراحل انتخاب یال‌ها به صورت زیر است:



همانطور که ملاحظه می‌شود در الگوریتم کراسکال در مراحل میانی، گراف ناهمبند می‌شود ولی در نهایت درخت پوشا، حاصل می‌شود.

یک الگوریتم سطح بالا برای کراسکال عبارت است از: T مجموعه یال‌های انتخاب شده، E مجموعه یال‌های گراف، n تعداد رئوس)

```

۱  $T = \varphi$ ;
۲ while  $((T \text{ Contains Less than } n - \text{edges}) \ \& \ (E \text{ not empty}))$ 
  {
۳ Choose an edge  $(v, w)$  from  $E$  of lowest cost  $t$ ;
۴ delete  $(v, w)$  from  $E$ ;
۵ if  $((v, w)$  does not create a cycle in  $T$ ) add  $(v, w)$  to  $T$ ;
۶ else discard  $(v, w)$ ;
  }
۷ if  $(T \text{ contains fewer than } n - \text{edges})$  cout  $<< \text{"no spanning Tree"}$ ;

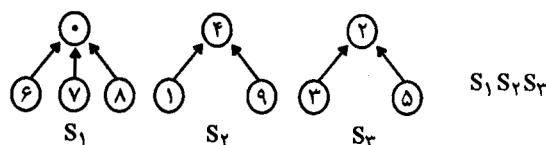
```

برای پیاده‌سازی خط ۵ الگوریتم می‌توان از اجتماع مجموعه‌های مجزا استفاده کرد. به این صورت که ابتدا هر عضو یک مجموعه تک عضوی تشکیل می‌دهد. سپس هر یال که انتخاب می‌شود، مجموعه‌های آنها اجتماع گرفته می‌شوند. اگر یال دو رأس در یک مجموعه بود، انتخاب نمی‌شود. در مثال ۸:

$$\begin{aligned}
 & \{a\} \ \{b\} \ \{c\} \ \{d\} \ \{e\} \ \{f\} \ \{g\} \xrightarrow{(c,e)} \{a\} \ \{b\} \ \{c,e\} \ \{d\} \ \{f\} \ \{g\} \\
 & \xrightarrow{(b,f)} \{a\} \ \{b,f\} \ \{c,e\} \ \{d\} \ \{g\} \xrightarrow{(a,c)} \{a,c,e\} \ \{b,f\} \ \{d\} \ \{g\} \\
 & \xrightarrow{(a,b)} \{a,b,c,e,f\} \ \{d\} \ \{g\} \xrightarrow{(c,d)} \{a,b,c,d,e,f\} \ \{g\} \\
 & \xrightarrow{(b,c)} \text{انتخاب نمی‌شود چون } b \text{ و } c \text{ در یک مجموعه هستند} \xrightarrow{(d,g)} \{a,b,c,d,e,f,g\}
 \end{aligned}$$

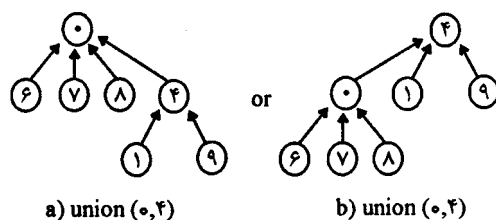
نمایش مجموعه‌ها

یکی از کاربردهای درخت، نمایش مجموعه است. فرض می‌کنیم اعضای مجموعه $\{0, 1, 2, 3, \dots, n-1\}$ است و مجموعه‌ها دو به دو مجزا هستند. مثلاً $n=10$ و $S_1 = \{0, 6, 7, 8\}$ و $S_2 = \{1, 4, 9\}$ و $S_3 = \{2, 3, 5\}$. یکی از نمایش‌های این مجموعه‌ها عبارت است از:



در این درخت‌ها، برخلاف معمول، جهت اشاره‌گرها از فرزندان به ریشه است. در شکل فوق، مجموعه S_1 را مجموعه ۰، مجموعه S_2 را مجموعه ۴ و مجموعه S_3 را مجموعه ۲ می‌نویسیم (مقدار ریشه را نام مجموعه می‌گذاریم). در عمل دو عمل $union$ و $find$ روی مجموعه‌ها انجام می‌شود.

(۱) $union(i, j)$: اجتماع مجموعه‌های i و j را می‌یابد و هر دو مجموعه i و j با مجموعه جدید جایگزین می‌شوند. در شکل فوق اجتماع S_1 و S_2 یکی از شکل‌های زیر است:



برای یافتن اجتماع، کافیست، ریشه یکی از درخت‌ها را، پدر ریشه درخت دیگر قرار دهیم. فقط قانونی هست به نام قانون وزنی (*weighting rule*) که با توجه به این قانون، ریشه درختی که نودهای بیشتری دارد پدر ریشه درختی که نودهای کمتری دارد می‌شود. بنابراین در شکل فوق شکل (a) حاصل می‌شود.

(۲) $Find(i)$: اسم مجموعه‌ای که i عضو آن است را برمی‌گرداند. مثلاً در شکل فوق قسمت a، داریم $Find(6) = 4$ ، $Find(7) = Find(8) = 0$ و یا در قسمت b، $Find(6) = 4$.

برای نمایش مجموعه‌ها می‌توان از آرایه‌ای به اسم مثلاً $Parent$ استفاده کرد. مثلاً شکل آرایه‌ای مجموعه‌های S_1 و S_2 و S_3 (شکل $S_1 S_2 S_3$) به صورت زیر است:

	[۰]	[۱]	[۲]	[۳]	[۴]	[۵]	[۶]	[۷]	[۸]	[۹]
<i>Parent</i>	-۴	۴	-۳	۲	-۳	۲	۰	۰	۰	۴

اگر i ریشه باشد $Parent[i] = -j$ که j تعداد نودهای درخت i است. مثلاً $Parent[0] = -۴$ یعنی درخت با ریشه ۰ (درخت ۰) دارای ۴ نود است. اگر i ریشه نباشد، $Parent[i] = j$ که j پدر i است. مثلاً $Parent[۱] = ۴$ یعنی پدر ۱ گره ۴ است.

بنابراین عمل $union(i, j)$ به صورت زیر است:

```
int temp = parent [i]+parent[j];
if (parent[i]>parent[j])//i has fewer nodes
{parent[i]=j; parent[j]=temp;}
else {parent[j]=i; parent[i]=temp;}
```

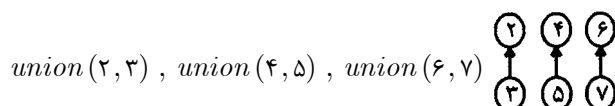
مثال ۹: فرض کنید

	[۰]	۱	۲	۳	۴	۵	۶	۷
Parent:	-۱	-۱	-۱	-۱	-۱	-۱	-۱	-۱

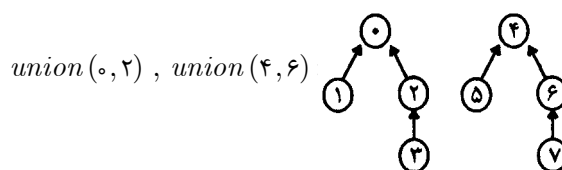
یعنی ۸ مجموعه مجزای تک عنصری داریم. عملیات زیر را انجام دهید:
 $union(0,1), union(2,3), union(4,5), union(6,7), union(0,2), union(4,6), union(0,4)$

پاسخ:

	۰	۱	۲	۳	۴	۵	۶	۷
$union(0,1):$	-۲	۰	-۱	-۱	-۱	-۱	-۱	-۱



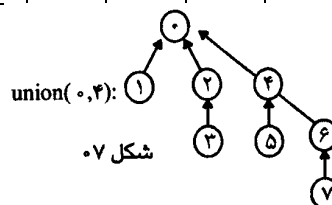
	۰	۱	۲	۳	۴	۵	۶	۷
	-۲	۰	-۲	۲	-۲	۴	-۲	۶



	۰	۱	۲	۳	۴	۵	۶	۷
	-۴	۰	۰	۲	-۴	۴	۴	۶

$union(0,4):$

	۰	۱	۲	۳	۴	۵	۶	۷
	-۸	۰	۰	۲	۰	۴	۴	۶



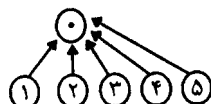
مثال: فرض کنید

	۰	۱	۲	۳	۴	۵
Parent:	-۱	-۱	-۱	-۱	-۱	-۱

زیر را انجام دهید:

$union(0,1), union(0,2), union(0,3), union(0,4), union(0,5)$

پاسخ: بررسی کنید که نتیجه نهایی به شکل زیر است:



توجه: عمل $find(i)$ به شکل زیر است:

```
while (parent[i] >= 0) i=parent[i];
return i;
```

مثلاً: در درخت حاصل (۰۷)، عمل $Find(۷)$ ، ۳ بررسی یال باید انجام دهد ($Find(۷) = ۶$) و $Find(۶) = ۴$ و $Find(۴) = ۰$ و $Find(۰) < ۰$.

نکته: ارتفاع درخت حاصل از $union$ برای n عدد، حداکثر $1 + \lceil \log_2 n \rceil$ است.

نکته: مرتبه عمل $union$ هر بار $O(1)$ و عمل $Find$ ، $O(\log n)$ است.

نکته: مجموعه‌ها در الگوریتم کراسکال کاربرد دارند.

عمل $Find$ دیگری وجود دارد به اسم $Collapsing Find(i)$ به شکل زیر:

```
for (int r=i; parent[r] >= 0; r=parent[r]); // find root
while (i!=r) {
    int S=Parent[i];
    Parent[i]=r;
    i=S;
}
return r;
```

در این الگوریتم، تمام نودهایی که در مسیر i تا ریشه قرار دارند، $Parent$ شان برابر ریشه می‌شود. مزیت این الگوریتم این است که اگر چندین $Find$ بخواهیم انجام دهیم، تعداد کل عملیات برای $Find$ های دوم به بعد کم می‌شود.

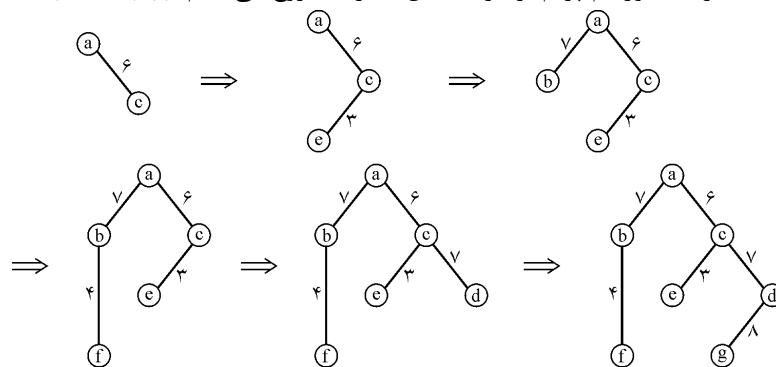
مثلاً اگر بخواهیم ۸ عمل $Find(۷)$ را $(Find(۷), \dots, Find(۷))$ را روی درخت مثال ۹ (شکل ۰۷) انجام دهیم، با $Simple Find$ هر عمل ۳ لینک را باید چک کند و جمعاً ۲۴ حرکت می‌شود ولی با $Collapsing Find$ ، عمل $Find$ اول ۳ لینک ولی بقیه هر کدام فقط با یک بررسی لینک به نتیجه می‌رسند. در ضمن عمل $Find$ اول، ۳ لینک $Parent$ را نیز $reset$ می‌کند ($Parent[۴], Parent[۶], Parent[۷]$) پس تعداد کل عملیات ۱۳ است.

توجه: برای نمایش مجموعه، روش‌های دیگری نیز وجود دارد که به کتاب طراحی الگوریتم رجوع کنید.

۲- الگوریتم پریم

از یک رأس شروع می‌کنیم، یالی که با این رأس مجاور است و کمترین وزن را دارد انتخاب می‌کنیم. سپس یالی را انتخاب می‌کنیم که با حداقل یکی از دو رأس فعلی مجاور است و کمترین وزن را دارد. این روند را تا زمانی که درخت پوشا حاصل شود ادامه می‌دهیم، یعنی در هر مرحله یک رأس ملاقات نشده، ملاقات می‌شود.

مثال ۱۰: اجرای الگوریتم پریم بر گراف مثال ۸. از a شروع می‌کنیم ($Prim(a)$)



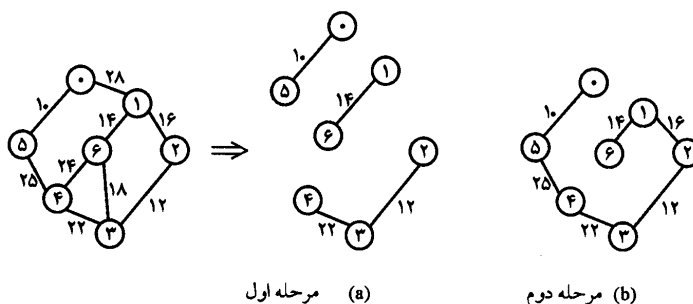
توجه: الگوریتم پریم در مراحل میانی هیچگاه ناهمبند نمی‌شود.

۳- الگوریتم سولین (Sollin)

الگوریتم سولین چندین لبه را در هر مرحله انتخاب می‌کند. در شروع یک مرحله، لبه‌های انتخابی همراه با تمامی n گره گراف تشکیل جنگل پوشا (*spanning forest*) را می‌دهند. در طی یک مرحله یک لبه برای هر درخت این جنگل انتخاب می‌شود. این لبه، لبه با حداقل هزینه بوده که دقیقاً یک گره در این درخت را شامل می‌شود. توجه کنید که احتمال دارد ۲ درخت جنگل لبه یکسان را انتخاب کنند. بنابراین چندین کپی از یک لبه وجود خواهد آمد که باید حذف شوند. در شروع اولین مرحله مجموعه لبه‌های انتخابی خالی است. الگوریتم زمانی خاتمه می‌یابد که تنها یک درخت در پایان باقی مانده باشد و یا اینکه لبه‌ای باقی نمانده باشد. نکته مهم آنست که اگر دو یا چند لبه وزن یکسان داشته باشند، امکان دارد ۲ درخت دو لبه متفاوت را برای اتصال بهم انتخاب کنند که تنها یکی باید باقی بماند.

در شکل زیر ضمن رسم یک گراف بعنوان نمونه الگوریتم سولین روی آن اعمال شده و مراحل ایجاد درخت پوشا با حداقل هزینه مشخص شده است. برای الگوریتم سولین ابتدا در مرحله اول

برای رئوس $0, 1, \dots, 6$ لبه‌های $(0, 5)$ ، $(0, 6)$ ، $(1, 3)$ و $(2, 3)$ و $(2, 3)$ و $(3, 2)$ ، $(3, 4)$ ، $(4, 5)$ ، $(5, 0)$ و $(6, 1)$ انتخاب می‌شوند که لبه‌های $(0, 5)$ ، $(0, 6)$ ، $(1, 3)$ و $(2, 3)$ باقی مانده و بقیه لبه‌ها به علت تکراری بودن حذف می‌شوند. در مرحله دوم، ۳ درخت وجود دارد و برای هر درخت یک یال انتخاب می‌کنیم که یال‌های $(4, 5)$ و $(2, 3)$ و $(1, 2)$ و $(1, 1)$ انتخاب می‌شوند که $(1, 2)$ و $(1, 1)$ یکسان هستند و یکی حذف می‌شود:



نکات

- ۱- کراسکال و سولین در مراحل میانی ممکن است جنگل بوجود آورند و در نهایت به درخت پوشا برسند. ولی پریم از ابتدا تا انتها همبند است.
- ۲- کراسکال و پریم و سولین برای گراف همبند، درخت پوشای مینیمم را می‌یابند.
- ۳- اگر وزن همه یال‌ها متمایز باشد، درخت حاصل از ۳ روش یکسان است. ولی اگر در گراف وزن‌های مساوی وجود داشته باشد، ممکن است شکل درخت حاصل از این روش‌ها یکسان نباشد ولی جمع وزن‌هایشان همیشه یکسان است.
- ۴- مرتبه الگوریتم کراسکال $O(e \log e)$ و پریم $O(n^2)$ است. بنابراین اگر گراف خلوت (یال کم) باشد، آنگاه $e = O(n)$ و مرتبه کراسکال $O(n \log n)$ می‌شود که از پریم بهتر است ولی اگر تعداد یال‌ها زیاد (گراف متراکم) باشد آنگاه $e = O(n^2)$ و مرتبه کراسکال $O(n^2 \log n)$ می‌شود که از پریم بدتر است.
- ۵- الگوریتم‌های DFS و BFS و کراسکال و پریم و سولین، از یک گراف غیرجهت‌دار همبند، یک درخت پوشا بدست می‌آورند پس دقیقاً $n-1$ یال را ملاقات می‌کنند.

مسئله کوتاهترین مسیرها از مبدأ واحد

گرافی بصورت $G=(V, E)$ موجود است که در آن هر لبه وزنی ($weight$) غیرمنفی دارد و یک گره بعنوان مبدأ معرفی شده است. مسئله تعیین هزینه کوتاهترین مسیر از مبدأ به دیگر گره‌های V می‌باشد. هزینه یک مسیر از جمع هزینه‌های لبه‌های آن تشکیل یافته است.

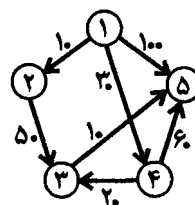
برای حل این مسئله الگوریتمی حریصانه برای اولین بار توسط دایجسترا (*Dijkstra*) مطرح شده است. در این الگوریتم از مجموعه‌ای تحت عنوان S استفاده شده است که در آن کلیه گره‌هایی که فاصله حداقل آن‌ها از مبدأ پیدا شده است ذخیره می‌شوند. در ابتدا S تنها شامل گره مبدأ خواهد بود. در هر مرحله گره‌ای که کمترین فاصله تا مبدأ را دارد به مجموعه S اضافه خواهد شد. با فرض غیرمنفی بودن وزن لبه‌ها این الگوریتم همواره به جواب مطلوب خواهد رسید. آرایه D برای ثبت طول کوتاهترین مسیر تا لحظه کنونی مورد استفاده قرار می‌گیرد. زمانیکه تمامی گره‌ها به S افزوده شوند، آرایه D طول کوتاهترین مسیر را برای تمامی گره‌ها نشان می‌دهد. الگوریتم به صورت زیر است، در این الگوریتم می‌خواهیم کوتاهترین مسیر از راس ۱ به سایر رئوس را بیابیم:

Dijkstra ()

```
{
    S = {۱};
    for (i = ۲ to n)
        D[i] = C[۱, i];
    for (i = ۱ to n - ۱)
    {
        choose a vertex w in V-S such that D[w] is minimum ;
        add w to S ;
        for (each vertex v in V-S)
            D[v] = min(D[v] , D[w] + C[w , v]) ;
    }
```

ماتریس C در این الگوریتم ماتریس هزینه نام دارد و هر خانه آن معروف وزنه لبه بین گره‌های i و j است. اگر بین دو گره لبه‌ای وجود نداشته باشد این وزن بینهایت در نظر گرفته خواهد شد. منظور از بینهایت وزنی است که بیشتر از هر وزن ممکن برای لبه‌های گراف باشد. شکل زیر مثالی از یک گراف است که با فرض گره مبدأ یک، راه حل آن توسط الگوریتم دایجسترا مشخص شده است.

$$C = \begin{matrix} & \begin{matrix} ۱ & ۲ & ۳ & ۴ & ۵ \end{matrix} \\ \begin{matrix} ۱ \\ ۲ \\ ۳ \\ ۴ \\ ۵ \end{matrix} & \begin{bmatrix} ۰ & ۱۰ & \infty & ۳۰ & ۱۰۰ \\ \infty & ۰ & ۵۰ & \infty & \infty \\ \infty & \infty & ۰ & \infty & ۱۰ \\ \infty & \infty & ۲۰ & ۰ & ۶۰ \\ \infty & \infty & \infty & \infty & ۰ \end{bmatrix} \end{matrix}$$



مثالی از یک دایگراف

Iteration	S	w	$D[2]$	$D[3]$	$D[4]$	$D[5]$
initial	$\{1\}$	-	۱۰	∞	۳۰	۱۰۰
۱	$\{1, 2\}$	۲	۱۰	۶۰	۳۰	۱۰۰
۲	$\{1, 2, 4\}$	۴	۱۰	۵۰	۳۰	۹۰
۳	$\{1, 2, 4, 3\}$	۳	۱۰	۵۰	۳۰	۶۰
۴	$\{1, 2, 4, 3, 5\}$	۵	۱۰	۵۰	۳۰	۶۰

محاسبات الگوریتم دایجسترا

اگر بخواهیم علاوه بر حصول اندازه کوتاهترین مسیر، خود مسیر را نیز مشخص کنیم باید از آرایه دیگری بنام P کمک گرفته شود. $P[v]$ در این آرایه معرف پدر بلافضلی است که در کوتاهترین مسیر قبل از v خواهد آمد. آرایه P ابتدا به یک مقداردهی خواهد شد و آخرین حلقه *for* باید به صورت زیر نوشته شود:

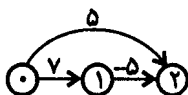
for (each vertex v in $V-S$)

if ($D[w] + C[w, v] < D[v]$)

$\{D[v] = D[w] + C[w, v];$

$P[v] = w;\}$

در این حالت پس از توقف الگوریتم دایجسترا می‌توان براساس آرایه P مسیر را نیز پیدا نمود. برای مثال مذکور $P[2]=1$, $P[3]=4$, $P[4]=1$ و $P[5]=3$ خواهد بود. حال مثلاً برای یافتن مسیر از گره یک به پنج باید در جهت عکس از گره پنج کار را آغاز نمود و این عمل را تا رسیدن به گره مبدأ (گره ۱) ادامه داد. مثلاً $P[5]=3$ یعنی پدر ۵ رأس است و $P[3]=4$ یعنی پدر ۳ رأس ۴ است و $P[4]=1$ پس کوتاهترین مسیر از ۱ به ۵ عبارت است از: $1 \rightarrow 4 \rightarrow 3 \rightarrow 5$ بسادگی می‌توان الگوریتمی خود بازگشتی برای این پیمایش طرح نمود. الگوریتم دایجسترا برای گراف با یال منفی لزوماً درست کار نمی‌کند مثلاً گراف زیر را در نظر بگیرید.



با الگوریتم دایجسترا، کوتاهترین مسیر از رأس ۰ به رؤوس ۱ و ۲ به ترتیب ۷ و ۵ محاسبه می‌شود که غلط است چون کوتاهترین مسیر از رأس ۰ به ۲ طول ۲ دارد.

مرتبه الگوریتم $O(n^2)$ است. البته با استفاده از هیپ فیبوناچی و لیست مجاورتی مرتبه الگوریتم $O(n \log n + e)$ می‌شود که برای گراف‌های خلوت از $O(n^2)$ بهتر است.

نکته: اگر گراف وزن منفی داشته باشد (به شرطی که سیکل منفی مثل نداشته باشد) در این صورت با الگوریتمی موسوم به بلمن و فورد می‌توان از یک راس به سایر رئوس کوتاهترین مسیرها را یافت. اگر گراف با ماتریس مجاورت نمایش داده شود، پیچیدگی این الگوریتم $O(n^3)$ و اگر با لیست مجاورتی نمایش داده شود، پیچیدگی $O(ne)$ است. البته اگر گراف سیکل منفی که از راس شروع قابل دسترسی است داشته باشد، بلمن فورد آن را اعلام می‌کند.

مسئله کوتاهترین مسیرها بین تمامی زوج گره‌ها

هدف یافتن کوتاهترین مسیر بین هر زوج گره ممکن در یک گراف جهت‌دار است. برای بیان ریاضی مسئله گراف $G=(V,E)$ به همراه ماتریس C که ماتریس هزینه ($cost$) نامیده می‌شود مفروض است. هدف ایجاد ماتریسی مثل A بعنوان خروجی الگوریتم است که اگر n گره داشته باشیم، ابعاد آن $n \times n$ بوده و هر خانه آن معرف طول کوتاهترین مسیر بین دو گره گراف است. راه حل اولیه این مسئله توسعه الگوریتم دایجسترا بصورتی است که n بار این الگوریتم اعمال شده و هر بار یکی از گره‌های گراف مبدأ فرض شود. بدین نحو هر آرایه D در هر چرخه یک سطر از ماتریس A را می‌سازد. اما راه حلی دیگر نیز توسط فلویید ($Floyd$) مطرح شده که آن را الگوریتم فلویید می‌نامند. در ابتدا ماتریس A با ماتریس C مقداردهی اولیه می‌شود. حلقه اصلی برنامه n بار بر روی ماتریس A اعمال می‌گردد. در k امین تکرار عنصر $A[i,j]$ معرف کوتاهترین مقدار مسیر بین i و j است که از گره‌های بزرگتر از k عبور نکرده باشد. یعنی i و j که ابتدا و انتهای مسیر هستند هر گره‌ای می‌توانند باشند ولی گره‌های بینابینی مسیر نمی‌توانند بزرگتر از k باشند.

در k امین تکرار رابطه زیر برای محاسبه A برقرار است.

$$A_k[i, j] = \min(A_{k-1}[i, j], A_{k-1}[i, k] + A_{k-1}[k, j])$$

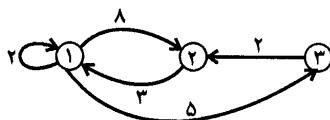
یعنی آیا رأس k را واسطه i و j قرار دهیم مسیر کوتاهتر می‌شود یا همان مسیر قبلی بدون واسطه k کوتاهتر است. (A_0 همان C است، A_1 از روی A_0 محاسبه می‌شود سپس A_2 از روی A_1 و به همین ترتیب) اندیس k معرف مرتبه تکرار است و نیازی به ایجاد n ماتریس A وجود ندارد. الگوریتم زیر، براساس رابطه بازگشتی فوق، الگوریتم را بیان نموده است.

Floyd ()

```
{
  for (i = ۱ to n)
    for (j = ۱ to n)
      A[i,j] = C[i,j] ;
  for (k = ۱ to n)
    for (i = ۱ to n)
      for (j = ۱ to n)
        if (A[i,k] + A[k,j] < A[i,j])
          A[i,j] = A[i,k] + A[k,j] ;
}
```

الگوریتم فلوید

بسادگی می‌توان دریافت که مرتبه الگوریتم فلوید $O(n^3)$ است. برای آنکه بتوان علاوه بر مقدار مسیر، خود مسیر را نیز بدست آورد مشابه با الگوریتم دایجسترا باید از ماتریسی همانند P استفاده نمود. این ماتریس ابتدا به صفر مقداردهی شده و سپس در شرط انتهایی الگوریتم کافیست جمله $P[i,j]=k$ افزوده گردد. شکل زیر مثالی از یک گراف جهت‌دار وزن‌دار است که حل آن توسط الگوریتم فلوید مشخص شده است.



مثالی از یک دایگراف وزن‌دار

	۱	۲	۳
۱	۰	۸	۵
۲	۳	۰	∞
۳	∞	۲	۰
	$A_0[i,j]$		

	۱	۲	۳
۱	۰	۸	۵
۲	۳	۰	۸
۳	∞	۲	۰
	$A_1[i,j]$		

	۱	۲	۳
۱	۰	۸	۵
۲	۳	۰	۸
۳	۵	۲	۰
	$A_2[i,j]$		

	۱	۲	۳
۱	۰	۷	۵
۲	۳	۰	۸
۳	۵	۲	۰
	$A_3[i,j]$		

مراحل تشکیل ماتریس A

A_0 همان ماتریس هزینه‌ها است. A_1 یعنی کوتاهترین مسیرها به شرطی که راس ۱ بتواند واسط باشد و ... (دقت کنید *self Loop* ها را نادیده می‌گیریم)

نکته: فلویید برای گراف جهت‌دار با وزن‌های منفی (بدون سیکل منفی) جواب را می‌یابد.

الگوریتم وارشل (Warshall)

اگر هدف تنها تشخیص این نکته باشد که در گراف مسیری بین هر زوج گره وجود دارد یا خیر آنگاه می‌توان الگوریتمی ساده‌تر از الگوریتم فلویید طرح نمود. در این حالت ماتریس C تنها شامل مقادیر یک (وجود لبه مستقیم) و یا صفر (عدم وجود لبه مستقیم) است. ماتریس A نیز تنها شامل همین دو مقدار خواهد بود. الگوریتم زیر که الگوریتم وارشل نامیده می‌شود فرم تعدیل یافته الگوریتم فلویید می‌باشد.

```
Warshall ( )
{
    for (i = ۱ to n)
        for (j = ۱ to n)
            A[i,j] = C[i,j] ;
    for (k = ۱ to n)
        for (i = ۱ to n)
            for (j = ۱ to n)
                if (A[i,j] = false)
                    A[i,j] = A[i,k] AND A[k,j] ;
}
```

الگوریتم وارشل

تعریف: در ماتریس بستار بعدی (*transitive closure*) که با A^+ نشان می‌دهیم،

$$A^+[i][j] = ۱ \text{ اگر از } i \text{ به } j \text{ مسیر با طول حداقل } ۱ \text{ باشد در غیر این صورت } ۰$$

تعریف: در ماتریس بستار تعدی بازتاب (*reflexive transitive closure*) که با A^* نشان

می‌دهیم، $A^*[i][j] = ۱$ اگر از i به j مسیر با طول حداقل ۰ باشد و در غیر این صورت

$$A^*[i][j] = ۰$$

مثال:

$$A = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix} \end{matrix}$$

ماتریس مجاورتی (b)



(a) گراف جهت دار

$$A^* = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{bmatrix}$$

(d)

$$A^+ = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{bmatrix}$$

(c)

همانطور که ملاحظه می‌شود تنها تفاوت A^+ و A^* این است که قطر اصلی A^* همیشه یک است. الگوریتم وارشل A^+ را با مرتبه $O(n^3)$ بدست می‌آورد. البته اگر گراف غیرجهت‌دار باشد می‌توان مولفه‌های همبند گراف را با مرتبه $O(n^2)$ بدست آورد و اگر i و j در یک مولفه باشند آنگاه $A^+[i][j] = 1$ که از وارشل سریعتر است.

مرتب‌سازی توپولوژیک (Topological Sort)

فرض کنید می‌خواهید برنامه‌ای طرح کنید که در دانشگاه کنترل کند که آیا دانشجویان رعایت پیش نیازها را در انتخاب واحد نموده‌اند یا خیر؟ برای اینکار ابتدا باید به دنبال ساختار داده مناسبی برای بیان مدل مسئله باشیم که گراف جهت‌دار برای اینکار مناسب است. هر درس بوسیله یک گره نمایش داده می‌شود که در محتویات گره ذکر شده است. هر لبه جهت‌دار در این گراف معرف رعایت پیش‌نیازها است. حال اگر بخواهیم تشخیص دهیم برای گرفتن یک درس به چه درس‌هایی نیاز داریم کافیهست که گره‌هایی که درس مورد نظر مجاور آنهاست را مشخص کنیم. این کار در روش پیاده‌سازی بصورت ماتریس همجواری بسادگی از طریق کنترل ستون مربوطه میسر است. اما هر کدام از این پیش‌نیازها نیز ممکن است پیش‌نیازهای دیگری داشته باشند و در نهایت به درس‌هایی خواهیم رسید که پیش‌نیازی ندارند. هدف از مرتب‌سازی توپولوژیک تعیین ترتیب خطی برای گره‌های چنین گرافی است.

برای طرح چنین الگوریتمی دو روش مورد استفاده قرار می‌گیرد. در روش اول کار را از گره‌هایی شروع می‌کنیم که پیش نیاز ندارند، یعنی در روش پیاده‌سازی ماتریس همجواری ستون‌هایی که فقط صفر هستند. پس از پیمایش این گره‌ها کافیهست که برای گره‌های مجاور این گره‌ها از عناصر مجاور آنها یکی کم کنیم. بدین ترتیب گره‌هایی ایجاد می‌شود که قابل پیمایش هستند. این عمل را تا زمانی انجام می‌دهیم که به گره‌های انتهایی برسیم.

در روش دوم برعکس روش اول عمل می‌کنیم. بدین نحوه که از گره‌هایی که مجاور بیشترین گره‌ها هستند شروع می‌کنیم و بصورت خود بازگشتی به عقب برگشته تا به گره‌هایی برسیم که هیچ مجاوری ندارند. در این شرایط پس از بازگشت از تابع خودبازگشتی در انتهای الگوریتم اقدام به چاپ گره می‌گردد.

لازم به ذکر است با DFS می‌توان ترتیب توپولوژیکی یافت که در کتاب طراحی الگوریتم آمده است. (تست ۳۴ را بررسی کنید).

مسائل گراف

۱- مربع گراف جهت‌دار $G(V, E)$ عبارت است از $G^2 = (V, E^2)$ که:

$$(u, w) \in E^2 \leftrightarrow \exists v \in V : (u, v) \in E, (v, w) \in E$$

یعنی G^2 مسیرهای به طول ۲ را شامل است. الگوریتمی ارائه دهید که G^2 را بدست آورد.

پاسخ: اگر A ماتریس مجاورتی باشد، به راحتی می‌توان A^2 را محاسبه کرد فقط بجای جمع دایره‌ها، باید عمل OR انجام شود و با روش ضرب استراسن (به کتاب طراحی الگوریتم رجوع کنید) مرتبه این عمل $O(n^{2/3})$ است. اگر گراف با لیست مجاورتی نشان داده شود، به راحتی می‌توان لیست‌ها را به هم $append$ کرد و $duplicate$ ها را حذف کرده که مرتبه این عمل $O((e+n)n)$ است.

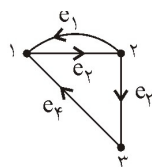
۲- با ماتریس مجاورتی اکثر الگوریتم‌های گراف، زمان $\Omega(n^2)$ صرف می‌کنند، ولی می‌توان الگوریتم یافتن $universal\ sink$ (راسی با درجه ورودی $n-1$ و درجه خروجی صفر) در گراف جهت‌دار G را با مرتبه $O(n)$ بدست آورد. این الگوریتم را بیابید.

۳- ماتریس برخورد گراف جهت‌دار G ، $B = [b_{ij}]_{n \times e}$ است که:

$$b_{ij} = \begin{cases} -1 & \text{if edge } j \text{ Leaves vertex } i \\ 1 & \text{if edge } j \text{ enters vertex } i \\ 0 & \text{else} \end{cases}$$

درایه‌های $B.B^T$ نشان‌دهنده چیست؟

پاسخ:



$$B.B^T(i, j) = \begin{cases} \text{degree of vertex } i = \text{indeg} + \text{outdeg} & , \text{ if } i = j \\ -(number \text{ of edges connecting } i \text{ and } j) & , \text{ if } i \neq j \end{cases}$$

$$B = \begin{matrix} & \begin{matrix} e_1 & e_2 & e_3 & e_4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 1 & -1 & 0 & 1 \\ -1 & 1 & -1 & 0 \\ 0 & 0 & 1 & -1 \end{bmatrix} \end{matrix} \Rightarrow B.B^T = \begin{bmatrix} 1 & -1 & 0 & 1 \\ -1 & 1 & -1 & 0 \\ 0 & 0 & 1 & -1 \end{bmatrix} \begin{bmatrix} 1 & -1 & 0 \\ -1 & 1 & 0 \\ 0 & -1 & 1 \\ 1 & 0 & -1 \end{bmatrix} = \begin{matrix} & \begin{matrix} 1 & 2 & 3 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 1 & 3 & -2 & -1 \\ -2 & 3 & -1 \\ 3 & -1 & 2 \end{bmatrix} \end{matrix}$$

سؤال‌های طبقه‌بندی شده فصل هفتم

۱- اگر G یک گراف ساده از درجه n باشد، در صورتی که گراف G دارای ۵۶ یال باشد و مجموع درجات رئوس مکمل گراف G برابر ۱۶۰ باشد، مقدار n کدام است؟

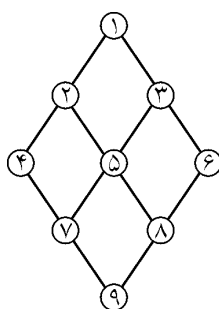
- (۱) ۱۶ (۲) ۱۶ (۳) ۱۷ (۴) ۱۷

۲- اگر a و b دو نود در یک گراف بدون جهت G باشند و اگر دو مسیر P_1 و P_2 از a به b وجود داشته باشد آنگاه:

(آ) (۷۶)

- (۱) a و b مجاورند. (۲) G نمی‌تواند یک گراف باشد.

- (۳) G دارای چرخه است. (۴) احتیاج به جهت مسیر داریم.



۳- پیمایش عمقی (dfs) گراف زیر کدام می‌تواند باشد؟

- (۱) ۱ و ۲ و ۳ و ۴ و ۵ و ۶ و ۷ و ۸ و ۹
(۲) ۱ و ۲ و ۳ و ۴ و ۵ و ۶ و ۷ و ۸ و ۹
(۳) ۱ و ۲ و ۳ و ۴ و ۵ و ۶ و ۷ و ۸ و ۹
(۴) ۱ و ۲ و ۳ و ۴ و ۵ و ۶ و ۷ و ۸ و ۹

۴- در تست قبل کدام گزینه پیمایش bfs را نشان می‌دهد؟

- (۱) ۱ و ۲ و ۳ و ۴ و ۵ و ۶ و ۷ و ۸ و ۹
(۲) ۱ و ۲ و ۳ و ۴ و ۵ و ۶ و ۷ و ۸ و ۹
(۳) ۱ و ۲ و ۳ و ۴ و ۵ و ۶ و ۷ و ۸ و ۹
(۴) ۱ و ۲ و ۳ و ۴ و ۵ و ۶ و ۷ و ۸ و ۹

۵- کدام یک از مجموعه‌های زیر نمایش دهنده یک درخت است؟

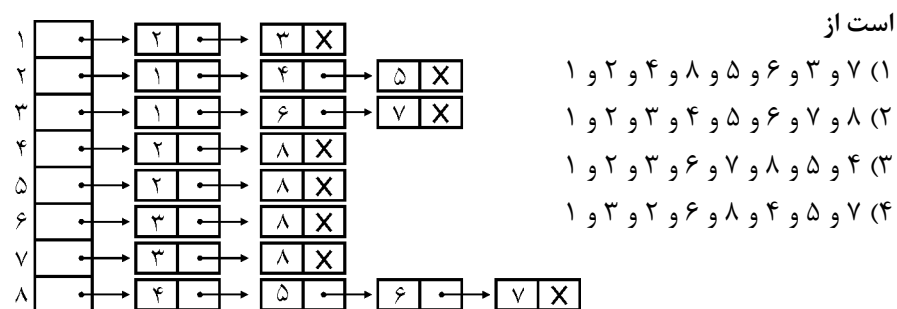
$$(۱) A = \{(0,1), (0,3), (3,2), (3,3), (4,2)\}$$

$$(۲) A = \{(0,2), (0,3), (2,1), (2,3)\}$$

$$(۳) A = \{(0,2), (0,3), (3,1), (3,4)\}$$

$$(۴) A = \{(0,1), (0,3), (0,4), (4,1)\}$$

۶- لیست مجاورتی گراف غیرجهت دار زیر را در نظر بگیرید. جستجوی عمقی (*dfs*) عبارت



۷- در تست قبل پیمایش (*bfs*) کدام است؟

- (۱) ۱ و ۲ و ۳ و ۴ و ۵ و ۶ و ۷ و ۸
(۲) ۱ و ۲ و ۳ و ۴ و ۵ و ۶ و ۷ و ۸
(۳) ۱ و ۲ و ۳ و ۴ و ۵ و ۶ و ۷ و ۸
(۴) ۱ و ۲ و ۳ و ۴ و ۵ و ۶ و ۷ و ۸

۸- خروجی کدام الگوریتم درخت پوشا برای گراف نیست؟

- (۱) *Prim* (۲) *Dijkstra* (۳) *Kruskal* (۴) *BFS*

۹- لیست مجاورت مربوط به گراف G را در نظر بگیرید. پیمایش عمقی این گراف عبارت است

از:

$A: B, C, D$

(۱) $A B D C F E$

$B: A, D, E$

(۲) $A B E F C A$

$C: A, D, F$

(۳) $A C F D E B$

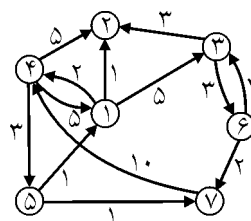
$D: A, B, C$

(۴) $A D B E F$

$E: B, F$

$F: C, E$

۱۰- در گراف زیر کمترین هزینه از گره ۱ به گره ۷ عبارت است از:



(۱) ۸

(۲) ۱۰

(۳) ۲۰

(۴) ۶

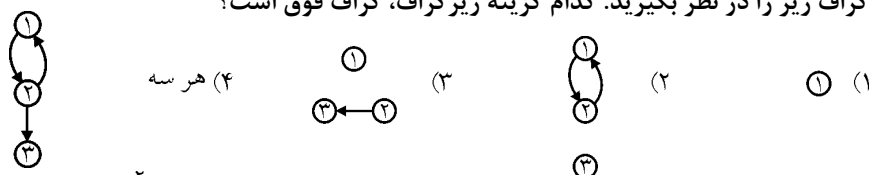
۱۱- حداکثر تعداد لبه‌های یک گراف جهت دار شامل n گره برابر است با

- (۱) $2n-1$ (۲) $n^2 - n$ (۳) $n^2 - 1$ (۴) $2n - n$

۱۲- گراف غیر جهت دار G یک جنگل است و از ۵ مولفه همبند تشکیل شده است. جمع درجات هریک از این مولفه‌ها ۱۰ و ۲۰ و ۳۰ و ۴۰ و ۵۰ می‌باشد. این گراف چند راس دارد؟

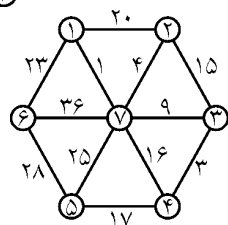
- (۱) ۷۰ (۲) ۷۵ (۳) ۸۰ (۴) ۸۵

۱۳- گراف زیر را در نظر بگیرید. کدام گزینه زیرگراف، گراف فوق است؟

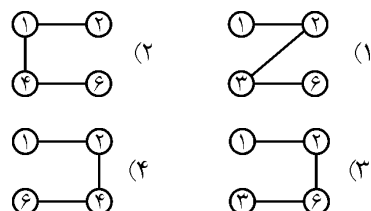
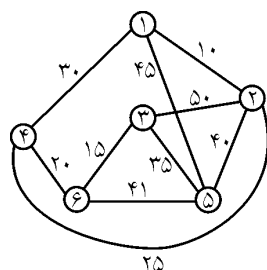


۱۴- هزینه درخت پوشای مینیمم گراف زیر چیست؟

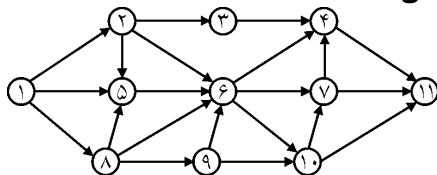
- (۱) ۶۸ (۲) ۴۱
(۳) ۸۱ (۴) ۵۷



۱۵- اگر برای پیدا کردن درخت پوشای مینیمم زیر از الگوریتم پریم استفاده کنیم کدام گزینه، درخت حاصله را در انتهای مرحله سوم الگوریتم نشان می‌دهد؟ (آزاد ۷۸)



۱۶- در جستجوی عمق اول (DFS) گراف جهت دار زیر، فرض کنید که گره ۱، گره شروع باشد و گره‌های مجاور یک گره به ترتیب مقدار عددی‌شان ملاقات می‌شوند. کدام گزینه (از چپ به راست) ترتیب گره‌های ملاقات شده را نشان می‌دهد؟



- (۱) ۹ و ۸ و ۱۰ و ۷ و ۶ و ۵ و ۱۱ و ۴ و ۳ و ۲ و ۱
(۲) ۷ و ۱۰ و ۱۱ و ۴ و ۶ و ۹ و ۳ و ۵ و ۸ و ۲ و ۱
(۳) ۱۱ و ۷ و ۱۰ و ۴ و ۶ و ۹ و ۳ و ۸ و ۵ و ۲ و ۱
(۴) ۱۱ و ۱۰ و ۹ و ۸ و ۷ و ۶ و ۵ و ۴ و ۳ و ۲ و ۱

۱۷- در تست قبل با توجه به اولویت گفته شده، نتیجه پیمایش *bfs* کدام است؟

- (۱) ۹ و ۸ و ۱۰ و ۷ و ۶ و ۵ و ۱۱ و ۴ و ۳ و ۲ و ۱
 (۲) ۷ و ۱۰ و ۱۱ و ۴ و ۶ و ۹ و ۳ و ۵ و ۸ و ۲ و ۱
 (۳) ۱۱ و ۱۰ و ۷ و ۴ و ۹ و ۶ و ۳ و ۸ و ۵ و ۲ و ۱
 (۴) ۱۱ و ۱۰ و ۹ و ۸ و ۷ و ۶ و ۵ و ۴ و ۳ و ۲ و ۱

۱۸- اگر A ماتریس مجاورت یک گراف جهت‌دار از مرتبه n بوده و V یک آرایه n عنصری که تمام عناصر آن در ابتدا *True* هستند، باشد. تابع ذیل با دریافت دو راس i و j چه کاری انجام می‌دهد؟

```
Function F(i, j:byte): boolean;
Var
    T:boolean;
begin
    V[i]:=False;
    if A[i, j] = ۱ then F := true
    else begin
        T:=False;
        for K := ۱ to n do
            if (A[i, K] = ۱) and V[K] then
                T := T or F(i, k);
        F:=T;
    end
end;
```

- (۱) تشخیص می‌دهد آیا از راس i تا راس j مسیر وجود دارد یا خیر.
 (۲) تشخیص می‌دهد آیا دوری با شروع از راس i و با گذر از راس j در گراف وجود دارد یا خیر.
 (۳) تشخیص می‌دهد آیا یالی از راس i به راس j در گراف می‌باشد یا خیر.
 (۴) تشخیص می‌دهد آیا مسیری به طول n از i به j وجود دارد یا خیر.

۱۹- فضای مورد نیاز برای پیمایش یک گراف $G(V, E)$ به روش لیست همسایگی (*Adjacency List*) کدام است؟ (دو نمره)

- (۱) $O(|E| + |V|)$ (۲) $O(|E|)$ (۳) $O(|V|)$ (۴) $O(|E| \cdot |V|)$

۲۰- الگوریتم زیر:

(آزاد ۷۸)

1. initialize all nodes to the ready state ($STATUS = ۱$)
2. push the starting node A onto $STACK$ and change its status to the waiting state ($STATUS = ۲$)
3. Repeat steps ۴ and ۵ until $STACK$ is empty
4. Pop the top node N of $STACK$. Process N and change its status to the processed state ($STATUS = ۳$)
5. Push onto $STACK$ all the neighbors of N that are still in the ready state ($STATUS = ۱$) and change their status to the waiting state ($STATUS = ۲$).
- [End of step ۳ loop]
6. Exit

(۱) الگوریتم جستجوی $depth$ -first بر روی یک گراف است.(۲) الگوریتم مرتب‌سازی $topological$ است.(۳) الگوریتم جستجوی $breadth$ first است.

(۴) الگوریتم تبدیل یک گراف به پشته است.

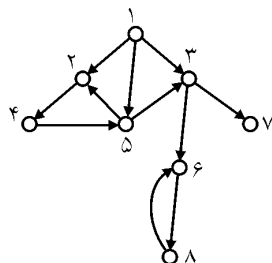
۲۱- الگوریتم زیر یک گراف G را به صورت عمق اول ($depth$ -First) جستجو می‌کند و بهرئوس شماره‌های (DFN) ($Depth$ First Number) می‌دهد. (دولتی ۷۵)*Procedure* dfs ($V:Vertex$); $W:Vertex$;

begin

mark V as "Visted"; for all vertices W adjacent to V do if W is not "Visited" then dfs(W); *Count* := *Count* + ۱; *DFN*[V] := *Count*;

end;

می‌خواهیم گراف زیر را با استفاده از الگوریتم فوق جستجو کنیم. فرض کنید که با $dfs(۱)$ آغاز می‌کنیم و رئوس مجاور یک راس، به ترتیب شماره‌های آنها مورد بررسی قرار می‌گیرند. همچنین مقدار اولیه $Count$ را صفر فرض کنید. $DFN[4]$ برابر چه عددی است؟



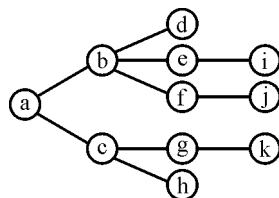
۴ (۱)
۳ (۲)
۶ (۳)
۵ (۴)

۲۲- فرض کنید وزن تمام یال‌ها در یک گراف بدون جهت برابر با یک است. با اجرای یک الگوریتم، کوتاهترین مسیر در این گراف، فاصله هر راس تا راس مبدا (a) در جدول زیر بدست آمده است. درجه راس f چند است؟ (علوم کامپیوتر ۸۳)

a	b	c	d	e	f	g	h	i	j
۰	۲	۱	۵	۲	۴	۱	۲	۳	۵

۵ (۴) ۴ (۳) ۳ (۲) ۲ (۱)

۲۳- با اجرای یک پیمایش DFS بر روی یک گراف بدون جهت همبند مثل G درخت DFS بصورت شکل زیر بدست آمده است. با فرض اینکه a راس مبدا پیمایش است و دانستن اینکه درجه a برابر ۴ است، کدام گزاره درست است؟ (علوم کامپیوتر ۸۳)

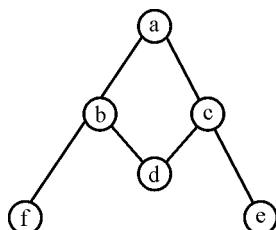


- (۱) $G-a$ سه مولفه همبندی دارد.
- (۲) G دارای دور نمی‌باشد.
- (۳) $G-a$ همبند است.
- (۴) $G-b$ نمی‌تواند همبند باشد.

۲۴- فرض کنید یک نوع الگوریتم پیمایش گراف بر روی دو گراف ساده همبند G_1 و G_2 اجرا شده است و $T_1 = a_1..a_n$ و $T_2 = b_1..b_m$ نشان دهنده دنباله راس‌های ملاقات شده به ترتیب در G_1 و G_2 هستند کدام گزاره نادرست است؟ (علوم کامپیوتر ۸۳)

- (۱) اگر T_1 و T_2 از پیمایش BFS بدست آمده باشند، آنگاه $T = T_1T_2$ نشان دهنده یک پیمایش BFS در گرافی است که از اتصال راس‌های a_n و b_1 بدست آمده است.
- (۲) اگر T_1 و T_2 از پیمایش BFS بدست آمده باشند، آنگاه $T = T_1T_2$ نشان دهنده یک پیمایش BFS در گرافی است که از اتصال راس‌های a_1 و b_1 بدست آمده است.
- (۳) اگر T_1 و T_2 از پیمایش DFS بدست آمده باشند، آنگاه $T = T_1T_2$ نشان دهنده یک پیمایش DFS در گرافی است که از اتصال راس‌های a_1 و b_1 بدست آمده است.
- (۴) اگر T_1 و T_2 از پیمایش DFS بدست آمده باشند، آنگاه $T = T_1T_2$ نشان دهنده یک پیمایش DFS در گرافی است که از اتصال راس‌های a_n و b_1 بدست آمده است.

۲۵- کدام گزینه نمی‌تواند خروجی پیمایش اول عمق (*dfs*) گراف زیر باشد؟ (علوم کامپیوتر ۸۰)



(۱) *abdcef*

(۲) *abdcfe*

(۳) *acdbfe*

(۴) *abfdce*

۲۶- وجود دور در یک گراف بدون جهت را در بهترین حالت با چه مرتبه‌ای می‌توان به دست آورد؟

(علوم کامپیوتر ۸۲)

(۱) $O(|E| + |V|)$ (۲) $O(|V|)$ (۳) $O(|V| |E|)$ (۴) $O(|E| \log |V|)$

۲۷- اگر تعداد گره‌های یک گراف $|V|$ و تعداد لبه‌های آن $|E|$ باشد و بر روی آن الگوریتم *DFS* را اعمال کنیم، مرتبه این الگوریتم به ترتیب از راست به چپ برای پیاده‌سازی گراف با ماتریس همجواری و لیست همجواری چه خواهد بود؟ (آزاد ۸۱)

(۱) $O(|V|)$ و $O(|E|)$ (۲) $O(|V|^2)$ و $O(|E|^2)$

(۳) $O(|E|)$ و $O(|V|^2)$ (۴) $O(|E|)$ و $O(|V|)$

۲۸- فرض کنیم گراف $G=(V,E)$ یک گراف بی‌جهت و M یک درخت فراگیر مینیمم برای G باشد، کدام عبارت درست است؟ (علوم کامپیوتر ۸۲)

(۱) مسیر M بین هر جفت راس V_1 و V_2 کوتاهترین مسیر است.

(۲) مسیر M بین هر جفت راس V_1 و V_2 کوتاهترین مسیر نیست.

(۳) تمام مسیرهای موجود در M کوتاهترین مسیر می‌باشند.

(۴) بین تمام رئوس در M مسیری وجود ندارد.

۲۹- گراف بدون جهت $G=(V,E)$ را در نظر بگیرید. اگر B ماتریس برخورد (*incidence*)

گراف G و B^T ماتریس ترانزپوز آن باشد، مفهوم ماتریس BB^T عبارت است از: (آزاد ۸۲)

(۱) مفهوم خاصی را بیان نمی‌کند.

(۲) مجموع مقادیر این ماتریس $|V|$ است.

(۳) مجموع مقادیر این ماتریس برابر $|E|$ است.

(۴) مقادیر این ماتریس، همجواری بین رئوس را بیان می‌کند.

۳۰- گراف همبندی بدون جهت با n گره و $n-2$ لبه داریم. کدام یک از الگوریتم‌های زیر برای تولید درخت پوشا با حداقل هزینه بر روی گراف مناسب‌تر است؟ (آزاد ۸۲)

(۱) *Prim* (۲) *Kruskal* (۳) *Sollin* (۴) هیچکدام

۳۱- کدام عبارت در مورد الگوریتم دایکسترا بر روی گراف وزن‌دار $G=(V,E)$ نادرست است؟ (گراف $G=(V,E)$ وزن‌دار است). (دولتی ۸۱)

(۱) پیچیدگی الگوریتم می‌تواند $O(|E| \log |V|)$ باشد.

(۲) اگر وزن هیچ یالی منفی نباشد الگوریتم درست کار می‌کند.

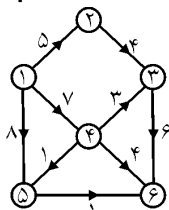
(۳) تنها اگر گراف دارای دور با وزن منفی نباشد الگوریتم درست کار می‌کند.

(۴) هم برای گراف‌های بدون جهت درست کار می‌کند و هم برای گراف‌های جهت‌دار.

۳۲- برای یافتن درخت پوشای حداقل *(minimum Spanning Tree)* یک گراف خلوت کدامیک از الگوریتم‌های زیر مناسب‌تر است؟ (آزاد ۸۰)

(۱) *Floyd* (۲) *Prim* (۳) *Kruskal* (۴) *Dijkstra*

۳۳- ترتیب رئوس حاصل از اجرای الگوریتم دایکسترا برای تعیین کوتاهترین مسیر در گراف جهت‌دار زیر به قرار است؟ (علوم کامپیوتر ۷۹)



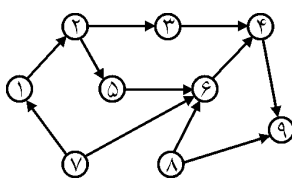
(۱) ۱ و ۲ و ۳ و ۴ و ۵ و ۶ و ۷

(۲) ۱ و ۲ و ۴ و ۶ و ۳ و ۵ و ۷

(۳) ۱ و ۲ و ۳ و ۶ و ۴ و ۵ و ۷

(۴) ۱ و ۲ و ۴ و ۵ و ۶ و ۳ و ۷

۳۴- یک *topological sort* مربوط به گراف زیر را انتخاب کنید. (علوم کامپیوتر ۸۱)



(۱) ۱ و ۲ و ۳ و ۴ و ۵ و ۶ و ۷ و ۸ و ۹

(۲) ۱ و ۲ و ۳ و ۸ و ۶ و ۴ و ۵ و ۹ و ۷

(۳) ۱ و ۲ و ۳ و ۴ و ۵ و ۶ و ۷ و ۸ و ۹

(۴) ۱ و ۲ و ۳ و ۵ و ۴ و ۶ و ۷ و ۸ و ۹

۳۵- اگر رویه زیر قرار باشد جستجوی سطح اول (*Breadth First search*) روی یک گراف

انجام دهد، عباراتی که با شماره‌های ۱ و ۲ و ۳ مشخص شده‌اند چه بایستی باشند؟ (آزاد ۷۸)

procedure BFS(v)

begin

visit v;

1.

while queue is not empty do

```

    begin
2. 
    for all vertices  $w$  adjacent to  $v$  do
        if  $w$  is not visited then
            begin
3. 
                Visit  $w$ 
            end;
        end;
    end;
end;

۱: AddQueue(Queue, v);          مورد نیاز نیست
۲: DeleteQueue(Queue, v) (۲)   ۲: AddQueue(Queue, v) (۱)
۳: AddQueue(Queue, w)          ۳: DeleteQueue(Queue, w)

۱: AddQueue(Queue, v);          ۱: AddQueue(Queue, v);
۲: DeleteQueue(Queue, v) (۴)   ۲: DeleteQueue(Queue, v) (۳)
۳: DeleteQueue(Queue, w)       ۳: DeleteQueue(Queue, v)

```

۳۶- کدام یک از گزاره‌های ذیل، در مورد دو روش تعیین درخت پوشای کمینه برای هر گراف، نادرست است؟

(۱) در روش *Kruskal*، برای تعیین درخت پوشای کمینه، در پایان کار، یک درخت به وجود می‌آید.

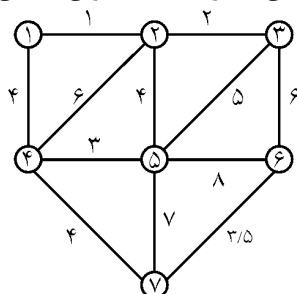
(۲) در روش *Prim* برای تعیین درخت پوشای کمینه، در هر مرحله، یک درخت به وجود می‌آید.

(۳) هم در روش *Prim* و هم در روش *Kruskal*، در هر مرحله از کار، یک درخت به وجود می‌آید.

(۴) در روش *Prim*، از هر رأس گراف می‌توان شروع کرد، اما در پایان، درخت پوشای کمین به دست می‌آید.

۳۷- الگوریتم *Kruskal* در مرحله چهارم خود، کدام کمان از گراف را به عنوان کمان درخت

(۸۳ IT)

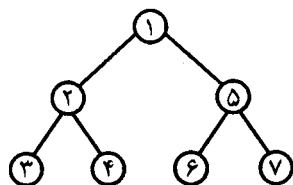


پوشای با حداقل هزینه در نظر می‌گیرد؟

- (۱) (۱ و ۴)
- (۲) (۴ و ۵)
- (۳) (۶ و ۷)
- (۴) هیچکدام

۳۸- اگر در پیمایش *DFS* در گراف غیر جهت‌دار $G=(V,E)$ درخت فراگیر آن برابر با درخت

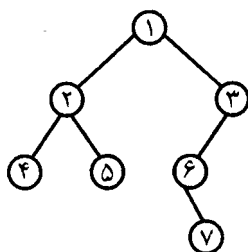
زیر باشد در آن صورت کدام گزینه صحیح است؟ در گراف $G=(V,E)$: (علوم کامپیوتر ۸۴)



- (۱) بین دو رأس ۲ و ۵ می‌تواند یال وجود داشته باشد.
- (۲) بین دو رأس ۲ و ۶ می‌تواند یال وجود داشته باشد.
- (۳) بین دو رأس ۱ و ۴ می‌تواند یال وجود داشته باشد.
- (۴) بین دو رأس ۶ و ۷ می‌تواند یال وجود داشته باشد.

۳۹- اگر در پیمایش *BFS* یک گراف غیر جهت‌دار $G=(V,E)$ درخت فراگیر حاصل برابر با درخت

زیر باشد در آن صورت در گراف $G=(V,E)$ کدام گزینه صحیح است؟ (علوم کامپیوتر ۸۴)



- (۱) بین دو رأس ۴ و ۵ می‌تواند یال وجود داشته باشد.
- (۲) بین دو رأس ۵ و ۱ می‌تواند یال وجود داشته باشد.
- (۳) بین دو رأس ۳ و ۷ می‌تواند یال وجود داشته باشد.
- (۴) بین دو رأس ۱ و ۷ می‌تواند یال وجود داشته باشد.

۴۰- یک گراف جهت‌دار (*Digraph*) را در نظر بگیرید. فرض کنید این گراف را، با ماتریس هم

جواری نشان می‌دهیم، کدام یک از گزاره‌های ذیل درست است؟ (۸۴ IT)

- (۱) شرط کافی برای آنکه این ماتریس از نوع بالا مثلثی باشد، آنست که *Acyclic* نباشد.
- (۲) شرط لازم برای آنکه این ماتریس از نوع پایین مثلثی باشد، آنست که دو نقطه انفصالی داشته باشد.
- (۳) شرط لازم و کافی برای آنکه این ماتریس از نوع پایین مثلثی باشد، آنست که گراف فوق از نوع *Acyclic* باشد.
- (۴) شرط لازم برای آنکه این ماتریس از نوع بالا مثلثی باشد، آنست که گراف فوق از نوع *Acyclic* باشد.

۴۱- کدام یک از گزاره‌های ذیل، در مورد محاسبه درخت پوشای کمین برای یک گراف همبند (*Connected*)، درست است؟ (طراحی الگوریتم IT ۸۴)

(۱) در روش *Prim* در هر مرحله، همبندی درخت رعایت می‌شود و زمان مصرفی الگوریتم *Prim* و الگوریتم *Kruskal*، یکسان است.

(۲) در روش *Prim* در هر مرحله، همبندی درخت رعایت می‌شود اما در روش *Kruskal* در مرحله پایانی، این امر اتفاق می‌افتد، اما زمان مصرفی روش *Prim* بهتر است.

(۳) در روش *Kruskal*، در هر مرحله، همبندی درخت رعایت می‌شود اما در روش *Prim* در مرحله پایانی، این امر اتفاق می‌افتد، اما زمان مصرفی روش *Prim* بهتر است.

(۴) در روش *Kruskal*، در هر مرحله، همبندی درخت رعایت نمی‌شود اما در پایان الگوریتم، درخت همبند است، اما زمان مصرفی آن از روش *Prim* بهتر است.

۴۲- در یک گراف کامل بی‌جهت با n رأس تعداد یال‌های *visit* نشده در پیمایش *DFS* برابر است با: (علوم کامپیوتر ۸۴)

$$(۱) \quad n-1 \quad (۲) \quad \frac{1}{2}(n-1)(n-2)$$

$$(۳) \quad \frac{1}{2}n(n-1) \quad (۴) \quad \frac{1}{2}n(n-3)$$

۴۳- فرض کنید G یک گراف غیر جهت‌دار بدون وزن باشد. کدام یک از روش‌های جستجو زیر در گراف G ، گره‌های G را به صورتی ملاقات می‌کند که گره‌هایی که فاصله آنها از گره شروع کمتر است زودتر ملاقات شوند؟ (علوم کامپیوتر ۸۵)

(۱) روش *BFS*

(۲) روش *DFS*

(۳) روش‌های *BFS* و *DFS*

(۴) با پیمایش‌های *BFS* و *DFS* نمی‌توان نتیجه‌ای به دست آورد.

۴۴- فرض کنید G یک گراف جهت‌دار باشد، کدام یک از شرایط زیر کافی است تا تضمین کند در جستجوی *DFS*، برای هر رأس V و W در G اگر یک مسیر از V به W وجود داشته باشد $DFS(W)$ سریع‌تر از $DFS(V)$ پایان می‌یابد؟ (علوم کامپیوتر ۸۵)

(۱) همبند نباشد. (۲) همبند باشد.

(۳) G دارای یک *Cycle* جهت‌دار باشد. (۴) G دارای هیچ *Cycle* جهت‌دار نباشد.

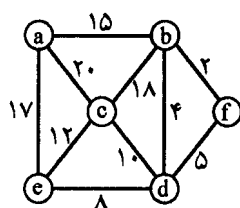
۴۵- T درخت فراگیر کمینه برای یک گراف $G=(V,E)$ است. وزن یک یال $e \in E$ را کاهش می‌دهیم. با چه مرتبه‌ای می‌توان درخت فراگیر کمینه گراف جدید را بدست آورد.

(طراحی الگوریتم دهلی ۸۵)

$$O(|V|) \quad (۱) \quad O(|V|+|E|) \quad (۲) \quad O(|E| \log |V|) \quad (۳) \quad O(|V| \log |E|) \quad (۴)$$

۴۶- ترتیب انتخاب یال‌ها در الگوریتم $Kruskal$ بر روی گراف زیر کدام مورد است (از چپ به راست)؟

(۸۵ IT)



$$(۱) \quad (a,b), (b,d), (e,d), (c,d), (b,f)$$

$$(۲) \quad (a,b), (b,f), (b,d), (f,d), (c,d)$$

$$(۳) \quad (b,f), (b,d), (e,d), (e,c), (a,e)$$

(۴) هیچکدام

۴۷- در تست قبل ترتیب انتخاب یال‌ها در الگوریتم $Prim$ بر روی گراف با شروع از راس a کدام مورد است؟

(۸۵ IT)

$$(۱) \quad (a,b), (c,d), (e,d), (b,d), (b,f)$$

$$(۲) \quad (e,c), (d,e), (f,d), (b,f), (a,b)$$

$$(۳) \quad (d,c), (d,e), (b,d), (b,f), (a,b)$$

(۴) هیچکدام

۴۸- اگر گراف $G=(V,E)$ به صورت لیست مجاورت ($Adjacency\ list$) پیاده‌سازی شده

باشد، پیچیدگی پیمایش $Breadth-first$ چیست؟

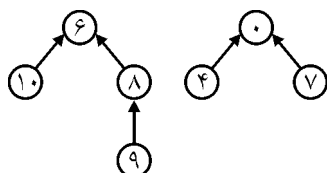
$$(۱) \quad O(|V|^2) \quad (۲) \quad O(|V| \cdot |E|)$$

$$(۳) \quad O(|V|^2 \cdot |E|) \quad (۴) \quad O(|V|+|E|)$$

۴۹- اگر دو مجموعه (۹ و ۸ و ۱۰ و ۶) و (۷ و ۴ و ۰) را به صورت درختی نمایش دهیم:

اجتماع (بهینه) این دو مجموعه، درختی است که پیمایش BFS آن بدون در نظر گرفتن

جهت برابر است با:



$$(۱) \quad ۹ \text{ و } ۰ \text{ و } ۸ \text{ و } ۱۰ \text{ و } ۷ \text{ و } ۴ \text{ و } ۶$$

$$(۲) \quad ۹ \text{ و } ۸ \text{ و } ۱۰ \text{ و } ۶ \text{ و } ۷ \text{ و } ۴ \text{ و } ۰$$

$$(۳) \quad ۷ \text{ و } ۴ \text{ و } ۰ \text{ و } ۹ \text{ و } ۸ \text{ و } ۱۰ \text{ و } ۶$$

$$(۴) \quad ۱۰ \text{ و } ۹ \text{ و } ۸ \text{ و } ۷ \text{ و } ۴ \text{ و } ۰ \text{ و } ۶$$

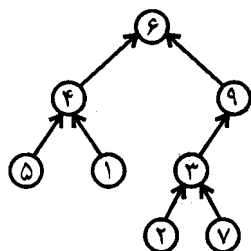
۵۰- اگر دنباله درجه یک گراف بدون جهت ۲ و ۲ و ۳ و ۳ و ۳ باشد در آن صورت تعداد

یال‌های پشتی ($backedge$) این گراف در جستجوی DFS برابر است با: (علوه کامپیوتر ۸۴)

$$(۱) \quad ۲ \quad (۲) \quad ۳ \quad (۳) \quad ۴ \quad (۴) \quad ۵$$

۵۱- اگر درخت زیر را به صورت آرایه نمایش دهیم کدام آرایه بهترین ساختار برای انجام عملیات بر روی درخت می‌باشد؟

(علوم کامپیوتر ۸۴)



(۱)

۱	۲	۳	۴	۵	۶	۷	۸	۹
۶	۴	۵	۱	۹	۳	۲	۷	-۱

(۲)

۱	۲	۳	۴	۵	۶	۷	۸	۹
۶	۴	۹	۵	۱	۳	-۱	۲	۷

(۳)

۱	۲	۳	۴	۵	۶	۷	۸	۹
۶	۴	۹	۵	۱	۳	۲	۷	-۱

(۴)

۱	۲	۳	۴	۵	۶	۷	۸	۹
۴	۳	۹	۶	۴	-۱	۳	-۱	۶

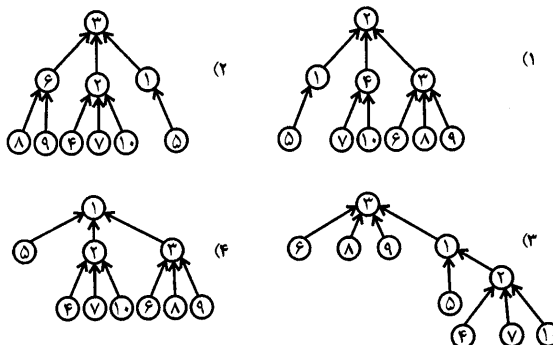
۵۲- فرض می‌کنیم n مجموعه تک عنصری متمایز از هم داریم. اگر u عمل $union$ انجام دهیم حداقل تعداد مجموعه تک عنصر که باقی می‌ماند برابر است با:

(علوم کامپیوتر ۸۴)

$$(۱) \quad n - 2u \quad (۲) \quad \max\{n - 2u, 0\} \quad (۳) \quad \max\{2u, n\} \quad (۴) \quad \min\left\{u, \frac{n}{2}\right\}$$

۵۳- درخت حاصل از اجتماع وزن دار سه مجموعه (دو به دو از چپ به راست) (۳ و ۶ و ۹ و ۱۰ و ۷ و ۴ و ۲) و (۵ و ۱) کدام درخت می‌تواند باشد؟

(علوم کامپیوتر ۸۴)



۵۴- اگر A ماتریس مجاورت گراف چندگانه G باشد و A^T به شکل مقابل باشد، از رأس b به

$$A^T = b \begin{bmatrix} a & 5 & 3 & 6 \\ 3 & 13 & 2 \\ 6 & 2 & 10 \end{bmatrix}$$

رأس c چند مسیر به طول ۴ وجود دارد؟

$$(۱) \quad 2 \quad (۲) \quad 54 \quad (۳) \quad 46 \quad (۴) \quad 64$$

[پاسخنامه سؤال‌های طبقه‌بندی شده فصل هفتم]

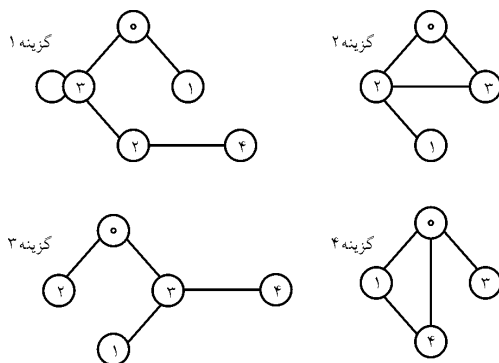
(۱) گزینه (۲). از آنجایی که مجموع درجات رئوس مکمل گراف G برابر ۱۶۰ است پس مکمل گراف دارای ۸۰ یال است و از آنجایی که خود گراف ۵۶ یال دارد بنابراین تعداد یال‌های گراف کامل K_p ، $۱۳۶ = ۸۰ + ۵۶$ می‌باشد. بنابراین تعداد رئوس از رابطه زیر بدست می‌آید:

$$\frac{p(p-1)}{2} = ۱۳۶ \Rightarrow p = ۱۷$$

پس تعداد رئوس ۱۷ است. گراف درجه n یعنی گرافی که حداکثر درجه رئوس آن n باشد و گرافی که ۱۷ راس دارد درجه آن حداکثر ۱۶ است.

- (۲) گزینه (۳). وجود بیش از یک مسیر بین دو راس یعنی وجود دور در گراف.
- (۳) گزینه (۲). گزینه‌های ۱ و ۳ بعد از رأس ۲، رأس را ملاقات کرده اند که غلط است. گزینه ۴ بعد از رأس ۶ رأس ۵ را ملاقات کرده که غلط است.
- (۴) گزینه (۱). ابتدا ۱ را ملاقات می‌کنیم. سپس ۲ و ۳، پس از آن با کمک ۲، رئوس ۴ و ۵ را ملاقات می‌کنیم و با کمک ۳، ۶ را ملاقات می‌کنیم و پس از آن ۷ و سپس ۸ و بعد ۹ را ملاقات می‌کنیم.

(۵) گزینه (۳).



(۶) گزینه (۱). از راس ۱ راس ۲ را ملاقات می‌کنیم. با کمک راس ۲، راس ۴ و با کمک ۴ راس ۸ و با راس ۸، راس ۵ را ملاقات می‌کنیم. از راس ۵ نمی‌توان راس جدیدی را ملاقات کرد پس یک مرحله عقب برمی‌گردیم و با راس ۸، راس ۶ و با راس ۶ راس ۳ و با راس ۳ راس ۷ را ملاقات می‌کنیم:

۷ و ۳ و ۶ و ۵ و ۸ و ۴ و ۲ و ۱

(۷) گزینه (۲).

$$\left. \begin{array}{l} ۱ \text{ راس} \Rightarrow ۲ \text{ و } ۳ \\ ۲ \text{ راس} \Rightarrow ۴ \text{ و } ۵ \\ ۳ \text{ راس} \Rightarrow ۶ \text{ و } ۷ \\ ۴ \text{ راس} \Rightarrow ۸ \end{array} \right\} \Rightarrow ۱ \text{ و } ۲ \text{ و } ۳ \text{ و } ۴ \text{ و } ۵ \text{ و } ۶ \text{ و } ۷ \text{ و } ۸$$

(۸) گزینه (۲). الگوریتم دایجسترا، برای یافتن کوتاهترین مسیر از یک راس به سایر رئوس است.

(۹) گزینه (۱).

$$\left. \begin{array}{l} A \Rightarrow B \\ B \Rightarrow D \\ D \Rightarrow C \\ C \Rightarrow F \\ F \Rightarrow E \end{array} \right\} \Rightarrow A, B, D, C, F, E$$

(۱۰) گزینه (۴). الگوریتمی که برای این منظور وجود دارد، الگوریتم دایجسترا است ولی می‌توان با بررسی گراف کوتاهترین مسیر را یافت: $[۱, ۴, ۵, ۷]$

(۱۱) گزینه (۲). بین هر دو راس دو یال وجود دارد.

(۱۲) گزینه (۳). هر مؤلفه یک درخت است پس: $\text{تعداد رئوس مؤلفه } ۱ = \frac{۱}{۲} + ۱$

$$\text{تعداد رئوس مؤلفه } ۲ = \frac{۲}{۲} + ۱$$

$$\text{تعداد رئوس مؤلفه } ۳ = \frac{۳}{۲} + ۱$$

$$\text{تعداد رئوس مؤلفه } ۴ = \frac{۴}{۲} + ۱ \Rightarrow \text{تعداد کل رئوس} = ۸۰$$

$$\text{تعداد رئوس مؤلفه } ۵ = \frac{۵}{۲} + ۱$$

نکته: به طور کلی در یک جنگل که دارای k مؤلفه است، تعداد رئوس از تعداد یال‌ها، k

$$n = e + k \text{ واحد بیشتر است}$$

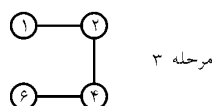
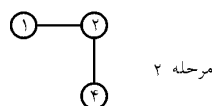
(۱۳) گزینه (۴).

۱۴) گزینه (۴).

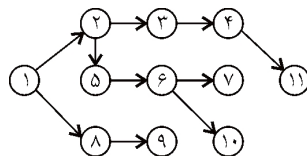
انتخاب	هزینه	یال
+	۱	(۱ و ۷)
+	۳	(۳ و ۴)
+	۴	(۲ و ۷)
+	۹	(۳ و ۷)
-	۱۵	(۲ و ۳)
-	۱۶	(۴ و ۷)
+	۱۷	(۴ و ۵)
-	۲۰	(۱ و ۲)
+	۲۳	(۱ و ۶)
-	۲۵	(۵ و ۷)
-	۲۸	(۵ و ۶)
-	۳۶	(۶ و ۷)

$\Rightarrow$ هزینه $= ۱ + ۳ + ۴ + ۹ + ۱۷ + ۲۳ = ۵۷$

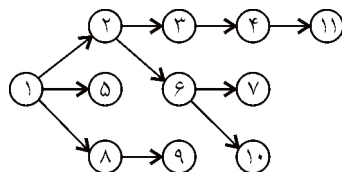
۱۵) گزینه (۴).



۱۶) گزینه (۱).



۱۷) گزینه (۳).



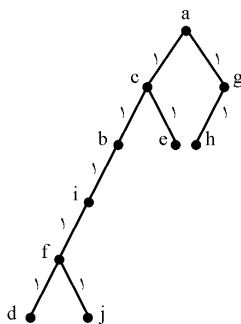
با مثال پر رسی، مے، کنیم۔

پس خروجی تابع *True* است. اگر (۱ و ۳) را فراخوانی کنید باز هم خروجی *True* می‌شود ولی (۱ و ۲)، *False* است.

(۲۱) گزینه (۳).

پس از پایان فراخوانی‌ها، از آخرین فراخوانی، جملات باقیمانده را برای همه فراخوانی‌ها اجرا می‌کنیم. هر یک *count* را یک واحد می‌افزاید. پس $dfs(4)$ برابر ۶ می‌شود.

(۲۲) گزینه (۲). راس‌هایی که عدد ۱ دارند با a مجاورند. راس‌هایی که عدد ۲ دارند. از a به آنها مسیری به طول ۲ وجود دارد. می‌توان گراف را به صورت زیر رسم کرد: (البته منحصر به فرد نیست)



(۲۴) گزینہ (۲). اگر فرض کنیم دو بیماریش انجام شده بر روی دو گراف BFS هستند.

باید آخرین گره یکی از دو دنباله را مجاور اولین گره از دنباله دیگر قرار دهیم. (پس گزینه ۱ و ۴ درست هستند.) در مورد dfs نیز کفایت دو گره آغازین را مجاور هم قرار دهیم. گزینه ۲ نمی‌تواند درست باشد چون در صورت مجاورت دو گره آغازین دنباله، حاصل ترکیبی از دو دنباله قبلی خواهد شد و حاصل اتصال این دو دنباله نخواهد بود.

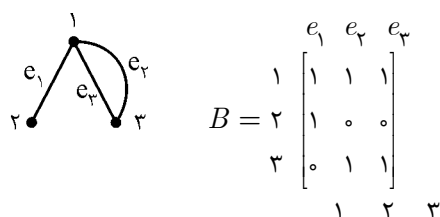
(۲۵) گزینه (۲). در گزینه ۲ بعد از c گره f ملاقات شده که غلط است.

(۲۶) گزینه (۲). الگوریتم یافتن دور در گراف، با تغییراتی بر روی الگوریتم dfs قابل پیاده‌سازی است. در بهترین حالت اگر فرض کنیم درجه هر گره در گراف ۲ باشد، مرتبه الگوریتم $O(|V|)$ خواهد شد.

(۲۷) گزینه (۴).

(۲۸) گزینه (۲). هیچ ارتباط مستقیمی مابین مفهوم کوتاهترین مسیر در گراف و درخت پوشای مینیمم وجود ندارد.

(۲۹) گزینه (۴).



$$BB^T = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ 1 & 0 & 1 \end{bmatrix} = \begin{matrix} & \begin{matrix} 1 & 2 & 3 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 3 & 1 & 2 \\ 1 & 1 & 0 \\ 2 & 0 & 2 \end{bmatrix} \end{matrix}$$

عناصر قطر اصلی، درجات رئوس هستند و درایه $i \cdot j$ ($i \neq j$) تعداد یال بین راس i و j را نشان می‌دهد. گزینه های ۱ و ۲ و ۳ حتماً نادرست هستند. گزینه ۴ نیز خیلی با دقت بیان نشده است.

(۳۰) گزینه (۴). گرافی با n راس و $n-2$ یال اصلاً نمی‌تواند همبند باشد.

(۳۱) گزینه (۳). الگوریتم دایجسترا زمانی که لبه‌هایی با وزن منفی باشد، درست عمل نخواهد کرد. در ضمن جهت‌دار یا بی‌جهت بودن گراف تاثیری بر الگوریتم ندارد (گزینه ۴). گزینه ۳ نادرست است، چرا که اگر گراف فاقد دور منفی باشد، دلیلی وجود ندارد که فاقد لبه منفی نیز باشد، پس دلیلی بر درستی عملکرد آن وجود ندارد. گزینه ۱ نیز در صورت پیاده‌سازی گراف با استفاده از هیپ و لیست همجواری، امکان‌پذیر است.

۳۲) گزینه (۳). منظور از گراف خلوت، گرافی است که تعداد لبه‌های آن نسبت به گره‌های آن اندک باشد، در چنین شرایطی الگوریتم کراسکال که بر پایه لبه‌ها، درخت را تولید می‌کند، زودتر به جواب خواهد رسید.

۳۳) گزینه (۴). از رأس ۱ به سایر رئوس، ابتدا ۲ بعد ۴ سپس ۵ و سپس ۶ و بعد ۳.

۳۴) گزینه (۴). ترتیب توپولوژیکی به این معنی است که ابتدا راسی را ملاقات کنیم که پیش نیاز ندارد یعنی راسی که درجه ورودی آن صفر است یعنی راس ۷ یا راس ۸. پس از آن باید راسی ملاقات شود که همه رئوس پیش‌نیاز آن ملاقات شده باشند.

گزینه ۲ نادرست است زیرا راس ۶ قبل از راس ۵ ملاقات شده. در گزینه ۳ نیز راس ۴ قبل از ۶ ملاقات شده است.

۳۵) گزینه (۲). در خط ۱ باید رأس شروع یعنی v را به صف اضافه کنیم سپس تا زمانی که صف خالی نشده، در خط ۲ رأس سر صف را حذف کرده در v بگذاریم و تمام رئوس مجاور v را در صورتی که هنوز ملاقات نشده اند در صف قرار دهیم پس در خط ۳ باید w که مجاور v است را به صف اضافه کنیم.

۳۶) گزینه (۳). در روش کراسکال در مراحل میانی ممکن است گراف ناهمبند بدست آید.

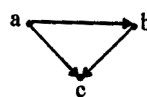
۳۷) گزینه (۳). ابتدا یال (۱ و ۲) سپس (۳ و ۲) و بعد (۴ و ۵) و سپس (۶ و ۷) انتخاب می‌شوند.

۳۸) گزینه (۳). در پیمایش DFS گراف غیرجهت دار، یال $cross$ وجود ندارد. پس یال‌های (۵ و ۲) (۶ و ۲) (۷ و ۶) نمی‌توانند وجود داشته باشند ولی یال (۴ و ۱) یال $back$ است.

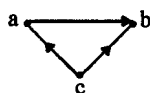
۳۹) گزینه (۱). در پیمایش BFS گراف غیرجهت‌دار یال $back$ و $forward$ وجود ندارد. پس یال‌های (۵ و ۱) (۷ و ۳) (۴ و ۱) (۷ و ۱) نمی‌توانند وجود داشته باشند. ولی یال (۵ و ۴) یک یال $cross$ است.

۴۰) گزینه (۴). اگر ماتریس همجواری یک گراف جهت‌دار مثلثی باشد آنگاه گراف دور ندارد ($Acyclic$ است):

$$A = \begin{bmatrix} a & b & c \\ \circ & 1 & 1 \\ \circ & \circ & 1 \\ c & \circ & \circ \end{bmatrix} \Rightarrow$$



یعنی دور نداشتن شرط لازم برای مثلثی بودن است.
اگر گراف دور نداشته باشد، لزوماً ماتریس مثلثی نیست:



$$A = \begin{matrix} & \begin{matrix} a & b & c \end{matrix} \\ \begin{matrix} a \\ b \\ c \end{matrix} & \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 0 \\ 1 & 1 & 0 \end{bmatrix} \end{matrix}$$

(۴۱) گزینه (۲). برای گراف‌های کامل، *prim* بهتر است. البته در صورت سوال نوع گراف گفته نشده که سوال نقص دارد. ولی پیاده‌سازی‌هایی برای پریم وجود دارد که از کراسکال سریع‌تر است.

(۴۲) گزینه (۲).

$$\frac{n(n-1)}{2} - (n-1) = \frac{(n-1)(n-2)}{2}$$

(۴۳) گزینه (۱). *BFS* می‌تواند کوتاه‌ترین مسیرها از رأس شروع را در گراف غیروزن‌دار بیابد.

(۴۴) گزینه (۴). مثلاً برای گراف $v \rightarrow w \rightarrow y$, $DFS(w)$ از $DFS(v)$ زودتر تمام می‌شود ولی اگر از y به v یال باشد چنین نیست.

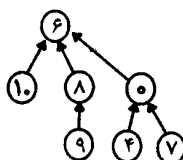
(۴۵) گزینه (۱). اگر $e \in T$ باشد آنگاه خود T مینیمم است. در غیر این صورت اگر $e = \{x, y\}$ باید مسیر x به y را بررسی کرده و یال با ماکزیمم وزن را حذف کنیم که این عمل از مرتبه $O(|V|)$ است.

(۴۶) گزینه (۱). ابتدا یال (b, f) که کمترین وزن را دارد انتخاب می‌شود. سپس یال (b, d) . ولی یال (d, f) انتخاب نمی‌شود چون سیکل تشکیل می‌شود. پس از آن یال (d, e) و سپس یال (c, d) و در نهایت (a, b) انتخاب می‌شوند.

(۴۷) گزینه (۳).

(۴۸) گزینه (۴). اگر گراف همبند باشد، آنگاه مرتبه *BFS* و *DFS*، $O(|E|)$ می‌شود. ولی در حالت کلی $O(|V| + |E|)$ می‌باشد.

(۴۹) گزینه (۳). مجموعه کوچک را زیرمجموعه، مجموعه بزرگ قرار می‌دهیم:
BFS: ۶, ۱۰, ۸, ۰, ۹, ۴, ۷



۵۰) گزینه (۲). این گراف ۸ یال و ۶ راس دارد که ۳ یال باید حذف شوند و این ۳ یال *back* هستند.

۵۱) گزینه (۴). پدر ۱ گره ۴ است که فقط گزینه ۴ می‌تواند صحیح باشد.

۵۲) گزینه (۲). مثلاً فرض کنید $n = 5$ یعنی:

$$\{1\}, \{2\}, \{3\}, \{4\}, \{5\}$$

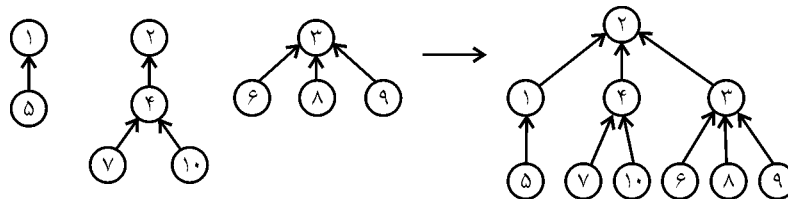
$$u = 1 \Rightarrow \{1, 2\}, \{3\}, \{4\}, \{5\} \Rightarrow \text{جواب} = 3$$

$$u = 2 \Rightarrow \{1, 2\}, \{3, 4\}, \{5\} \Rightarrow \text{جواب} = 1$$

$$u = 3 \Rightarrow \{1, 2\}, \{3, 4, 5\} \Rightarrow \text{جواب} = 0$$

که فقط گزینه ۲ می‌توان جواب باشد.

۵۳) گزینه (۱). درخت کوچکتر، زیردرختِ درخت بزرگتر می‌شود. البته سوال شکل درخت‌های اولیه را مشخص نکرده که از این نظر اشکال دارد ولی شکل‌ها به صورت زیر فرض شده است.



۵۴) گزینه (۴). درایه سطر ۲ ستون ۳ در A^4 تعداد مسیرهای به طول ۴ از b به c را می‌دهد.

$$3 \times 6 + 13 \times 2 + 2 \times 10 = 64$$

مرتب‌سازی (Sorting) فصل ۸

- روش‌های مرتب‌سازی، یا داخلی (*internal*) هستند یا خارجی (*external*). در مرتب‌سازی داخلی، عناصر ورودی همگی در حافظه اصلی قرار دارند و نتیجه نیز در حافظه اصلی قرار می‌گیرد. (مثل مرتب‌سازی آرایه) تمام روش‌های مرتب‌سازی در این بخش، از نوع داخلی می‌باشند. در مرتب‌سازی خارجی، عناصر همگی در حافظه اصلی نیستند و قسمتی از عناصر در حافظه جانبی است (مثل مرتب‌سازی فایل). در این نوع الگوریتم‌ها، دسترسی به عناصر تعیین‌کننده زمان اجراست. این نوع روش‌ها معمولاً در درس «ذخیره و بازیابی اطلاعات» بررسی می‌شوند.

- روش‌های مرتب‌سازی می‌توانند پایدار (متعادل *Stable*) باشند و یا ناپایدار (*unstable*). الگوریتم مرتب‌سازی پایدار، الگوریتمی است که ترتیب عناصر با کلید مساوی را حفظ می‌کند. مثلاً اگر کلید عناصر ورودی به صورت ۲' و ۳ و ۱' و ۲ و ۱ (چپ به راست) باشند آنگاه مرتب شده این عناصر به صورت صعودی می‌تواند هر یک از حالات زیر باشد:

الف) ۳ و ۲' و ۲ و ۱' و ۱ و ۲' و ۱' (ب)

ج) ۳ و ۲ و ۲' و ۱' و ۱ و ۳ و ۲' و ۱' (د)

ولی از بین حالات فوق فقط حالت الف پایدار است ولی حالات ب و ج و د ناپایدار هستند، زیرا ترتیب عناصر با کلید مساوی، عوض شده است.

- روش‌های مرتب‌سازی یا درجا (*inplace*) هستند یا برون از جا (*outplace*). اگر در روش مرتب‌سازی، از فضای کمکی به طول ثابت (غیر وابسته به تعداد عناصر ورودی) استفاده شده باشد، روش مرتب‌سازی در جا، و در غیر این صورت برون از جا است.

روش‌های مرتب‌سازی که بررسی می‌کنیم عبارتند از:

- ۱- مرتب‌سازی انتخابی *selection sort*
- ۲- مرتب‌سازی حبابی *bubble sort*
- ۳- مرتب‌سازی درجی *insertion sort*
- ۴- مرتب‌سازی ادغامی *merge sort*
- ۵- مرتب‌سازی سریع *quick sort*
- ۶- مرتب‌سازی کومه‌ای (درخت نیمه مرتب) *heap sort*
- ۷- مرتب‌سازی درختی *Tree sort*
- ۸- مرتب‌سازی شمارشی *Counting sort*
- ۹- مرتب‌سازی پایه‌ای (شعاعی) *Radix sort*
- ۱۰- مرتب‌سازی باکتی *bucket sort*

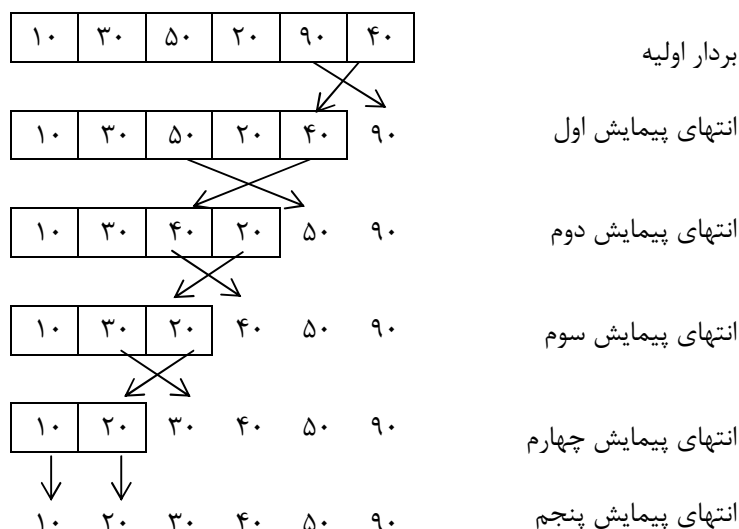
توجه: در تمام روش‌ها، فرض می‌شود عناصر به صورت صعودی مرتب می‌شوند. آرایه ورودی را $A[1..n]$ فرض می‌کنیم.

۱- مرتب‌سازی انتخابی *Selection sort*

در این روش، آرایه n عنصری، $n-1$ بار پیمایش می‌شود. در هر پیمایش، بزرگترین عنصر پیدا می‌شود و در محل درست خود قرار می‌گیرد (در محل آخر) سپس طول آرایه یک واحد کم می‌شود.

$A[1]$	$A[2]$	$A[3]$	$A[4]$	$A[5]$	$A[6]$
--------	--------	--------	--------	--------	--------

مثال:



در پیمایش اول، بزرگترین عنصر یعنی $A[5]$ پیدا شده و با یک جابجایی با $A[6]$ تعویض می‌شود. در پیمایش دوم، فرض می‌شود طول آرایه ۵ می‌باشد و عنصر ماکزیمم یعنی $A[3]$ پیدا می‌شود و با $A[5]$ تعویض می‌شود. به همین ترتیب پس از ۵ پیمایش، آرایه مرتب می‌شود. با توجه به مثال، الگوریتم به صورت زیر است:

```
for  $i \leftarrow n$  downto 2
  do { $max \leftarrow A[1]$  ,  $index \leftarrow 1$ 
    for  $j \leftarrow 2$  to  $i$ 
      do if  $A[j] > max$ 
        then { $max \leftarrow A[j]$  ,  $index \leftarrow j$ }
     $A[index] \leftarrow A[i]$  ,  $A[i] \leftarrow max$ }
```

الگوریتم ۱- ویرایش اول مرتب‌سازی انتخابی

نکته: با توجه به الگوریتم نوشته شده، روش انتخابی، درجا است ولی متعادل نیست.

مثال: بررسی کنید اگر آرایه ورودی

۱	۲	۳
۵	۴	۵

 باشد، خروجی الگوریتم

۱	۲	۳
۴	۵	۵

 است یعنی ترتیب عناصر مساوی ۵ حفظ نمی‌شود.

سوال: اگر در الگوریتم ۱ بجای $A[j] > max$ بنویسیم $A[j] \geq max$ ، آیا متعادل می‌شود؟

پاسخ: خیر. مثلاً اگر آرایه ورودی

۱	۲	۳
۵	۴	۴

 باشد، خروجی

۱	۲	۳
۴	۴	۵

 خواهد بود.

توجه: مرتب‌سازی انتخابی را می‌توان طوری تغییر داد که در هر پیمایش کوچکترین عنصر پیدا شود و با عنصر اول جابجا شود. الگوریتم ۲ مرتب‌سازی انتخابی را با روش دوم، نشان می‌دهد.

الگوریتم ۲- ویرایش دوم مرتب‌سازی انتخابی، در هر پیمایش کوچکترین عنصر پیدا می‌شود و در جای صحیح خود قرار می‌گیرد.

```
for  $i \leftarrow 1$  to  $n-1$ 
  do { $smallest \leftarrow i$ 
    for  $j \leftarrow i+1$  to  $n$ 
      do if  $A[j] < A[smallest]$ 
        then  $smallest \leftarrow j$ 
    exchange  $A[i] \leftrightarrow A[smallest]$ }
```

نکته: با توجه به الگوریتم‌های ۱ و ۲ تعداد مقایسه‌ها در روش انتخابی برابر است با:

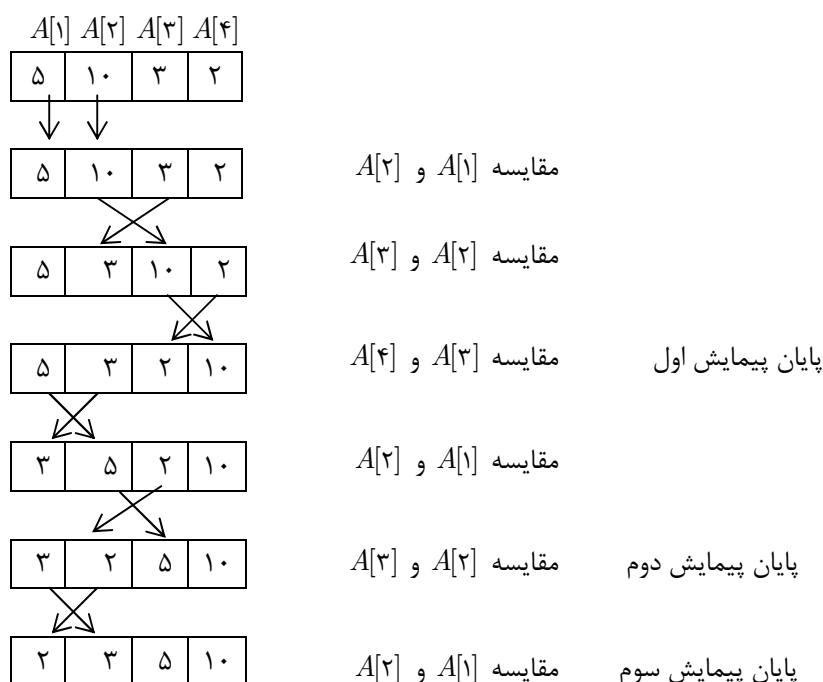
$$\sum_{i=1}^{n-1} \sum_{j=i+1}^n = \sum_{i=1}^{n-1} (n-i) = \frac{n(n-1)}{2}$$

نکته: با توجه به الگوریتم نوشته شده، تعداد تعویض‌ها همیشه $n-1$ است.

۲- مرتب‌سازی حبابی *bubble Sort*

آرایه n عنصری، $n-1$ بار پیمایش می‌شود. در هر پیمایش دو عنصر متوالی با هم مقایسه می‌شوند و اگر عنصر $A[i]$ از $A[i+1]$ بزرگتر بود، تعویض می‌شوند. در هر پیمایش نسبت به پیمایش قبل، طول آرایه یک واحد کم می‌شود. با یک مثال توضیح می‌دهیم.

مثال:



در هر پیمایش عنصر بزرگ از پایین آرایه به سمت بالای آرایه حرکت می‌کند و در جای خود قرار می‌گیرد (مثل حباب که از کف به سمت بالا حرکت می‌کند)

الگوریتم ۳- مرتب‌سازی حبابی - ویرایش اول - حرکت عناصر بزرگ به سوی انتهای آرایه

```
for pass ← 1 to n-1
  do for i ← 1 to n-pass
    do if A[i] > A[i+1]
      then exchange A[i] ↔ A[i+1]
```

توجه: می‌توان مرتب‌سازی حبابی را طوری نوشت که عناصر کوچک به سمت ابتدای آرایه حرکت کنند. نتیجه، همانند قبل است و آرایه صعودی مرتب می‌شود:

الگوریتم ۴- مرتب‌سازی حبابی - ویرایش دوم - حرکت عناصر کوچک به سوی ابتدای آرایه

```
for i ← 1 to n-1
  do for j ← n down to i+1
    do if A[j] < A[j-1]
      then exchange A[j] ↔ A[j-1]
```

با توجه به الگوریتم‌های ۳ و ۴ تعداد مقایسه‌ها در الگوریتم حبابی برابر $O(n^2)$ است و تعداد جابجایی در بهترین حالت برابر صفر ($O(1)$) می‌باشد و در بدترین حالت برابر $O(n^2)$ است. بنابراین در حالت متوسط، روش از مرتبه $O(n^2)$ است.

در الگوریتم‌های ۳ و ۴، اگر در یک پیمایش، هیچ تعویضی صورت نگیرد، آرایه در همان جا مرتب شده است و نیازی به اجرای پیمایش‌های بعدی نیست. بنابراین برای بهبود الگوریتم‌های ۳ و ۴ می‌توان از یک متغیر کمکی بولین (به نام *flag*) کمک گرفت. هرگاه در یک پیمایش تعویضی صورت گیرد، این متغیر، *True* می‌شود و حلقه بیرونی این متغیر را بررسی می‌کند؛ اگر *true* بود حلقه اجرا می‌شود و گرنه حلقه پایان می‌یابد. بنابراین الگوریتم مرتب‌سازی حبابی را می‌توان به صورت بهینه زیر نوشت:

الگوریتم ۵- مرتب‌سازی حبابی بهینه، تعداد مقایسه‌ها در بهترین حالت $n-1$ می‌باشد، وقتی آرایه صعودی باشد.

```
flag ← true, pass ← 1
while pass < n and flag = true
  do {flag ← false
    for i ← 1 to n-pass
      do if A[i] > A[i+1]
        then {flag ← true, exchange A[i] ↔ A[i+1]}
    pass ← pass + 1}
```

نکته: مرتب‌سازی حبابی، متعادل و درجا است.

نکته: الگوریتم ۵ در حالت متوسط و بدترین حالت از مرتب $\theta(n^2)$ و در بهترین حالت $\theta(n)$ می‌شود.

۳- مرتب‌سازی درجی

برای درج یک عنصر در یک لیست مرتب، از آخر لیست مقایسه می‌کنیم و در صورت لزوم،

عنصر لیست را به جلو حرکت می‌دهیم. مثلاً برای درج عدد ۲۰ در لیست

۱۰	۱۵	۲۵	۳۰	
----	----	----	----	--

، ابتدا ۲۰ با ۳۰ مقایسه می‌شود و چون $۲۰ < ۳۰$ ، ۲۰ را به خانه ۵ منتقل می‌کنیم و سپس چون $۲۵ < ۲۰$ ، ۲۵ را به خانه ۴ منتقل می‌کنیم و حال چون $۱۵ > ۲۰$ پس ۲۰ در خانه ۳ درج می‌شود

۱۰	۱۵	۲۰	۲۵	۳۰
----	----	----	----	----

.

مرتب‌سازی درجی شامل n مرحله است که از عنصر ۱ تا n را به ترتیب درج می‌کند، پس در مرحله i ام، لیست $A[1..i-1]$ مرتب است و $A[i]$ در مکان درست درج می‌شود و در مرحله n ام لیست $A[1..n]$ مرتب است و $A[n]$ در مکان صحیح درج می‌شود و آرایه مرتب می‌شود.

مثال: با توجه به آرایه A ، در مرحله ۱، ۵۰ درج می‌شود که البته این مرحله نیاز به انجام نیست. در مرحله ۲، ۶۰ درج می‌شود و به همین ترتیب در مرحله ۵، ۲۰ درج می‌شود که در این مرحله، ۲۰ به ترتیب با ۷۰ و ۶۰ و ۵۰ و ۴۰ مقایسه می‌شود و در خانه ۱ درج می‌شود.

$A[1]$	$A[2]$	$A[3]$	$A[4]$	$A[5]$
۵۰	۶۰	۴۰	۷۰	۲۰

۵۰ مرحله ۱ (درج ۵۰)

۶۰ مرحله ۲ (درج ۶۰)

۴۰ ۵۰ ۶۰ مرحله ۳ (درج ۴۰)

۴۰ ۵۰ ۶۰ ۷۰ مرحله ۴ (درج ۷۰)

۲۰ ۴۰ ۵۰ ۶۰ ۷۰ مرحله ۵ (درج ۲۰)

الگوریتم ۶- مرتب‌سازی درجی

for $i \leftarrow 2$ to n

do $\{y \leftarrow A[i], j \leftarrow i-1$

while $j > 0$ and $y < A[j]$

do $\{A[j+1] \leftarrow A[j], j \leftarrow j-1\}$

$A[j+1] \leftarrow y\}$

در حلقه *while* عبارت $j > 0$ برای آن است که مقایسه عناصر تا $A[1]$ انجام شود. اگر این شرط حذف شود، حلقه *while* ممکن است بی‌نهایت شود. روش بهتر این است که تعریف کنیم $A[0] = -\infty$ و شرط $j > 0$ را از حلقه حذف کنیم ($-\infty$ عددی است که از همه اعداد آرایه کوچکتر است)

- حداکثر تعداد مقایسه‌ها در الگوریتم ۶ برابر است با:

$$1 + 2 + \dots + (n-1) = \frac{n(n-1)}{2} = \theta(n^2)$$

- حداقل تعداد مقایسه‌ها برابر $\theta(n) = n-1$ می‌باشد.

- حداکثر تعداد جابجایی (تعویض) برابر $\theta(n^2) = \frac{n(n-1)}{2}$ است.

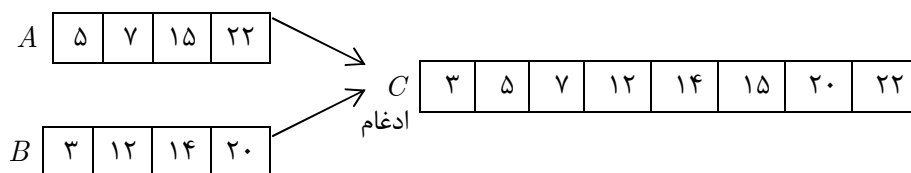
- حداقل تعداد جابجایی (تعویض) برابر $\theta(1) = 0$ صفر، می‌باشد.

نکته: این الگوریتم متعادل و درجا است. برای آرایه مرتب، بهترین عملکرد را دارد.

نکته: برای n ‌های کوچک این روش بسیار مناسب است. در واقع برای $n \leq 20$ این روش سریعترین است.

۴- مرتب‌سازی ادغامی (merge sort)

ایده کلی این روش بر مبنای ایده تقسیم و حل (*Divide & conquer*) بنا نهاده شده است. برای توضیح این روش، ابتدا روش ترکیب (*merge*) دو لیست مرتب را بررسی می‌کنیم. برای ترکیب دو لیست مرتب، عناصر ابتدایی لیست با هم مقایسه می‌شوند و کوچکترین آنها انتخاب می‌شود، این عمل تا زمانی که حداقل یک لیست به پایان برسد، ادامه می‌یابد:



ابتدا ۳ و ۵ مقایسه می‌شوند و ۳ انتخاب می‌شود. سپس ۵ و ۱۲ مقایسه می‌شوند و ۵ انتخاب می‌شود. سپس ۷ و ۱۲ مقایسه می‌شوند و ۷ انتخاب می‌شود و به همین ترتیب با ۷ مقایسه آرایه مرتب C حاصل می‌شود. می‌توان زیر برنامه *merge* را به صورت زیر نوشت:

الگوریتم ۷- ادغام آرایه‌های $A[L..m]$ و $A[m+1, u]$ و ایجاد آرایه مرتب $A[L..u]$

$MERGE(L, m, u)$

```

create Array  $B[L..u]$ 
 $h \leftarrow L, i \leftarrow L, j \leftarrow m+1$ 
while  $h \leq m$  and  $j \leq u$ 
do {if  $A[h] \leq A[j]$ 
    then  $B[i] \leftarrow A[h], h \leftarrow h+1$ 
    else  $B[i] \leftarrow A[j], j \leftarrow j+1$ 
     $i \leftarrow i+1$ }
if  $h > m$ 
then for  $k \leftarrow j$  to  $u$ 
do  $B[i] \leftarrow A[k], i \leftarrow i+1$ 
else for  $k \leftarrow h$  to  $m$ 
do  $B[i] \leftarrow A[k], i \leftarrow i+1$ 
for  $k \leftarrow L$  to  $u$ 
do  $A[k] \leftarrow B[k]$ 

```

با توجه به الگوریتم ۷ حداکثر تعداد مقایسه‌ها $u - L + 1$ و حداقل تعداد مقایسه‌ها $\min(m - L + 1, u - m)$ است.

مثلاً ادغام دو لیست $\boxed{10 \ 20 \ 30}$ و $\boxed{40 \ 50 \ 60 \ 70}$ با ۳ مقایسه (به تعداد عناصر لیست کوچکتر) انجام می‌شود به طور کلی مرتبه الگوریتم $merge$ ، $\theta(n)$ است که n تعداد عناصر آرایه A است. در مرتب‌سازی ادغامی، لیست n عنصری به لیست‌های یک عنصری تبدیل می‌شود ($divide$) و سپس این لیست‌های یک عنصری با هم ترکیب می‌شوند، سپس لیست‌های دو عنصری ترکیب می‌شوند، تا جایی که لیست n عنصری مرتب حاصل می‌شود. رویه $mergesort$ به صورت زیر است:

الگوریتم ۸- مرتب‌سازی ادغامی. با فراخوانی‌های بازگشتی، آرایه به لیست‌های یک عنصری تبدیل می‌شود و سپس لیست‌ها با هم ترکیب می‌شوند.

$MERGESORT(L, U)$

```

if  $L < u$ 
then  $\{m = \left\lfloor \frac{L + u}{2} \right\rfloor$ 
     $MERGESORT(L, m)$ 
     $MERGESORT(m+1, u)$ 
     $MERGE(L, m, u)\}$ 

```

فراخوانی این الگوریتم برای مرتب‌سازی آرایه $A[1..n]$ به صورت $MERGESORT(1, n)$ است. با توجه به رویه $mergesort$ ، اگر $T(n)$ زمان اجرای الگوریتم باشد، رابطه بازگشتی زیر بدست می‌آید:

$$T(n) = \begin{cases} 1 & n \leq 1 \\ T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + T\left(\left\lceil \frac{n}{2} \right\rceil\right) + Cn & n > 1 \end{cases}$$

با حل این رابطه بازگشتی مرتبه مرتب‌سازی ادغامی $\theta(n \lg n)$ است.

❖ **نکته:** می‌توان الگوریتم $mergesort$ را با ایده برنامه‌نویسی پویا نوشت (غیربازگشتی) ولی مزیتی ندارد.

❖ **نکته:** مرتب‌سازی ادغامی متعادل است. این روش درجا نیست، زیرا به فضای کمکی به طول n نیاز دارد (آرایه B در الگوریتم $merge$).

۵- مرتب‌سازی سریع (Quick Sort)

در این روش یک عنصر به عنوان محور یا لولا ($Pivot$) انتخاب می‌شود، سپس عناصر آرایه دو بخش می‌شوند، بخش اول عناصر کمتر یا مساوی لولا و بخش دوم عناصر بیشتر یا مساوی لولا. این عمل برای هر یک از دو بخش به صورت بازگشتی انجام می‌شود تا به عناصر تک عنصری برسیم و آرایه مرتب شود. الگوریتم زیر، این روش را نشان می‌دهد. عنصر اول به عنوان محور انتخاب شده است.

الگوریتم ۹- الگوریتم QuickSort. برای مرتب‌سازی آرایه $A[1..n]$. باید فراخوانی به صورت $Quicksort(A, 1, n)$ صورت گیرد.

Procedure Quicksort (Var A:Arraylist; left, right: integer);

Var

i, j, Pivot: integer;

begin

if left < right then

begin

i := left; j := right + 1; pivot := A[left];

repeat

repeat

i := i + 1;

until A[i] >= Pivot;

repeat

j := j - 1;

until A[j] <= Pivot;

if i < j then swap (A[i], A[j]);


```

until  $i >= j$ ;
swap ( $A[left]$ ,  $A[j]$ );
Quicksort ( $A$ ,  $left$ ,  $j - 1$ )
Quicksort ( $A$ ,  $j + 1$ ,  $right$ );
end
end;
```

مثال: فرض کنید آرایه اولیه به صورت زیر باشد. یک مرحله از الگوریتم ۹ نشان داده شده است.

$A[1]$	$A[2]$	$A[3]$	$A[4]$	$A[5]$	$A[6]$	$A[7]$	$A[8]$	$A[9]$	$A[10]$
۲۶	۵	۳۷	۱	۶۱	۱۱	۵۹	۱۵	۴۸	۱۹

$\uparrow$ $i = 1$
 $\uparrow$ $j = 11$

شروع: $left = 1, right = 10, i = 1, j = 11, Pivot = 26$

i زیاد می‌شود تا به اولین عنصر بزرگتر از محور یعنی ۳۷ برسد ($A[3]$), j کم می‌شود تا به اولین عنصر کوچکتر از محور برسد ($A[10]$). سپس $A[3]$ و $A[10]$ با هم عوض می‌شوند و آرایه به صورت زیر در می‌آید:

۱	۲	۳	۴	۵	۶	۷	۸	۹	۱۰
۲۶	۵	۱۹	۱	۶۱	۱۱	۵۹	۱۵	۴۸	۳۷

$\uparrow$ i
 $\uparrow$ j

مجدداً i اضافه می‌شود تا به ۶۱ برسد و j کم می‌شود تا به ۱۵ برسد و جای این دو با هم عوض می‌شود:

۲۶	۵	۱۹	۱	۱۵	۱۱	۵۹	۶۱	۴۸	۳۷
----	---	----	---	----	----	----	----	----	----

$\uparrow$ i
 $\uparrow$ j

مجدداً i اضافه می‌شود و به ۵۹ می‌رسد و j به ۱۱ می‌رسد و چون شرط $i >= j$ درست می‌باشد حلقه‌های *repeat* تمام می‌شوند و سپس محور با $A[j]$ یعنی ۱۱ عوض می‌شود:

۱۱	۵	۱۹	۱	۱۵	۲۶	۵۹	۶۱	۴۸	۳۷
----	---	----	---	----	----	----	----	----	----

$\uparrow$ j

با توجه به مثال، با یک فراخوانی، آرایه دو قسمت می‌شود. حال برای هر یک از قسمت‌ها نیز، این رویه تکرار می‌شود و با فراخوانی‌های بازگشتی، آرایه مرتب می‌شود. در جدول زیر مراحل کامل مرتب‌سازی آرایه نشان داده شده است.

پس از بار اول فراخوانی	[۱۱	۵	۱۹	۱	۱۵]	۲۶	[۵۹	۶۱	۴۸	۳۷]
بار دوم فراخوانی با قسمت اول	[۱	۵]	۱۱	[۱۹	۱۵]	۲۶	[۵۹	۶۱	۴۸	۳۷]
بار سوم	۱	۵	۱۱	[۱۹	۱۵]	۲۶	[۵۹	۶۱	۴۸	۳۷]
بار چهارم	۱	۵	۱۱	۱۵	۱۹	۲۶	[۵۹	۶۱	۴۸	۳۷]
بار پنجم	۱	۵	۱۱	۱۵	۱۹	۲۶	[۴۸	۳۷]	۵۹	[۶۱]
بار ششم	۱	۵	۱۱	۱۵	۱۹	۲۶	۳۷	۴۸	۵۹	[۶۱]
بار هفتم	۱	۵	۱۱	۱۵	۱۹	۲۶	۳۷	۴۸	۵۹	۶۱

نکته: با توجه به الگوریتم ۹، اگر آرایه از قبل مرتب باشد، n بار فراخوانی انجام می‌شود. در واقع با فراخوانی اول یک قسمت آرایه شامل صفر عنصر و قسمت دیگر شامل $n-1$ عنصر خواهد بود. یعنی با هر فراخوانی یک عنصر کم می‌شود. در ضمن هر فراخوانی از مرتبه $O(n)$ است پس در بدترین حالت (وقتی آرایه مرتب است) مرتبه الگوریتم $O(n^2)$ خواهد بود. در حالت متوسط و بهترین حالت، مرتب‌سازی سریع از مرتبه $\theta(n \lg n)$ می‌باشد.

نکته: اگر محور (*Pivot*) عنصری غیر از عنصر اول انتخاب شود (مثلاً به صورت تصادفی انتخاب شود) در این صورت بدترین حالت دیگر وقتی که آرایه مرتب باشد اتفاق نمی‌افتد.

نکته: مرتب‌سازی سریع، درجا و نامتعادل است و در حالت متوسط بهترین عملکرد را دارد.

۶- مرتب‌سازی *heapsort*

با توجه به آنچه در فصل ۶ گفته شد در این مرتب‌سازی، ابتدا با لیست اعداد ورودی یک *heap* ساخته می‌شود و سپس عناصر *heap* به ترتیب حذف می‌شوند. این روش درجا و نامتعادل است و مرتبه آن در حالت متوسط و بدترین $\theta(n \lg n)$ است.

۷- مرتب‌سازی درختی *TreeSort*

با توجه به مطالب فصل ۶، در این مرتب‌سازی، ابتدا با لیست اعداد ورودی یک *BST* ساخته می‌شود و سپس *BST* حاصل به صورت *inorder* پیمایش می‌شود. مرتبه این الگوریتم در بدترین حالت $O(n^2)$ و در سایر حالات $\theta(n \lg n)$ است. این الگوریتم غیر درجا است. در ضمن چون در *BST* عنصر تکراری وجود ندارد مساله متعادل بودن برای این الگوریتم مطرح نیست. ولی می‌توان *BST* را طوری تغییر داد که عنصر تکراری بپذیرد که در این صورت، این روش نامتعادل است. تاکنون الگوریتم‌های مرتب‌سازی مختلفی را مطرح کردیم و مشاهده شد که بهترین آنها در حالت متوسط قادر به مرتب‌سازی آرایه n عنصری در زمان $\theta(n \lg n)$ هستند. در تمامی این

روش‌های مرتب‌سازی، عمل مرتب‌سازی تنها بر پایه مقایسه بین عناصر ورودی انجام گرفت و اپراتور دیگری جز مقایسه بین عناصر دنباله ورودی استفاده نشد.

می‌توان ثابت نمود، اگر الگوریتم مرتب‌سازی بر روی RAM ماشین و بر پایه مقایسه بنا نهاده شود زمان مرتب‌سازی از طریق مقایسه در حالت متوسط نمی‌تواند سریعتر از $\Omega(n \log n)$ صورت گیرد. اما اگر اپراتور مرتب‌سازی را به غیر از مقایسه انتخاب کنیم، نتیجه روش‌های مرتب‌سازی در زمان سریعتری همانند روش‌های زیر خواهد شد.

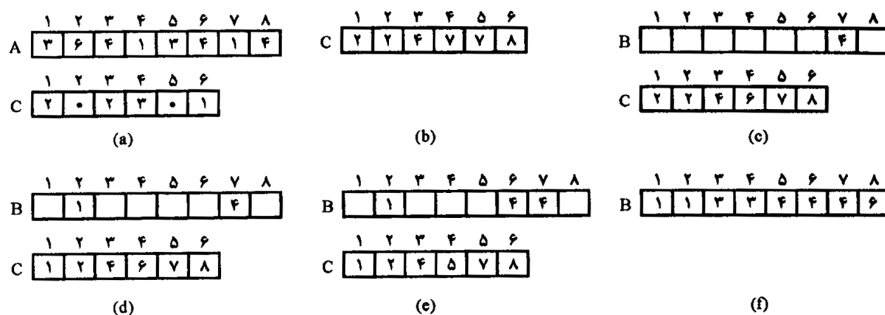
۸- مرتب‌سازی شمارشی (*Counting Sort*)

در این روش مرتب‌سازی فرض می‌کنیم تمامی عناصر ورودی اعداد صحیح و در بازه یک تا k قرار دارند. ایده اصلی این روش مرتب‌سازی، تعیین تعداد عناصر کوچکتر از هر عنصر ورودی مانند x است. این اطلاعات برای جایگزینی عنصر x در موقعیت مناسب آن در آرایه خروجی، مورد استفاده قرار خواهد گرفت. برای مثال، اگر ۱۷ عنصر کوچکتر از x وجود دارند، x متعلق به خانه ۱۸ خروجی خواهد بود. برای پیاده‌سازی این الگوریتم آرایه ورودی $A[1..n]$ و نیاز به دو آرایه $B[1..n]$ برای ذخیره خروجی مرتب شده و $C[1..k]$ برای محاسبات موقت نیاز داریم. الگوریتم به قرار زیر است:

Counting Sort (A, B, K)

```
{
  for ( $i = 1$  to  $k$ )  $c[i] = 0$ ;
  for ( $i = 1$  to  $n$ )  $c[A[i]]++$ ; // با این حلقه  $c[i]$  شامل تعداد  $i$  ها در  $A$  است.
  for ( $i = 2$  to  $k$ )  $c[i] = c[i] + c[i-1]$ ; // با این حلقه  $c[i]$  شامل تعداد اعداد کمتر یا مساوی  $i$  است.
  for ( $i = n$  downto  $1$ ) {  $B[c[A[i]]] = A[i]$ ;  $c[A[i]]--$ ; }
}
```

براحتی می‌توان ثابت نمود مرتبه این الگوریتم $\theta(k + n)$ است و اگر k را حداکثر به اندازه n فرض کنیم الگوریتم $\theta(n)$ خواهد بود. دقت کنید این روش مرتب‌سازی عمل مقایسه بین عناصر ورودی را انجام نمی‌دهد، بلکه از مقادیر واقعی عناصر برای اندیس‌سازی آرایه استفاده می‌کند. نکته قابل توجه در مورد این روش مرتب‌سازی پایدار (*Stable*) بودن آنست. در شکل زیر، نحوه عملکرد الگوریتم را مشاهده می‌کنید:



- تأثیر *counting - sort* روی آرایه $A[1..n]$ که عناصر آرایه، اعداد صحیح ۱ تا ۶ هستند.

(a) آرایه A و آرایه کمکی C بعد از اجرای حلقه دوم

(b) آرایه C بعد از اجرای حلقه سوم

(c)-(e) آرایه خروجی B و آرایه کمکی C بعد از یک، دو و سه بار اجرای حلقه چهارم

(f) آرایه خروجی مرتب شده نهایی B

نکته: اگر حلقه *for* آخر را به صورت *for(i = ۱ to n)* بنویسیم، الگوریتم غیر پایدار خواهد شد.

۹- مرتب‌سازی پایه‌ای (*radix*)

ورودی اعداد صحیح حداکثر d رقمی هستند، ابتدا اعداد براساس یکان سپس دهگان سپس صدگان و ... مرتب می‌شوند.

ورودی

۳۲۹	۷۲۰	۷۲۰	۳۲۹
۴۵۷	۳۵۵	۳۲۹	۳۵۵
۶۵۷	۴۳۶	۴۳۶	۴۳۶
۸۳۹	یکان → ۴۵۷	دهگان → ۸۳۹	صدگان → ۴۵۷
۴۳۶	۶۵۷	۳۵۵	۶۵۷
۷۲۰	۳۲۹	۴۵۷	۷۲۰
۳۵۵	۸۳۹	۶۵۷	۸۳۹

بدیهی است روش مرتب‌سازی برای هر ستون باید پایدار باشد، چون در غیر اینصورت توالی عناصر در ستون ماقبل بهم خواهد خورد. شبه کد این الگوریتم به قرار زیر است:

Radix-Sort (A, d)

```
{
  for( $i = ۱$  to  $d$ )
    Sort array A on digit  $i$  by a stable sort ;
}
```

اگر اعداد ورودی در مبنای k باشند آنگاه هر رقم $\circ$ تا $k - 1$ است و اگر k خیلی بزرگ نباشد می‌توان برای هر رقم از مرتب‌سازی شمارشی استفاده کرد و بنابراین مرتبه این الگوریتم $\theta(d(n + k))$ خواهد بود. و اگر d ثابت باشد و $k = O(n)$ آنگاه مرتبه مرتب‌سازی $\theta(n)$ یعنی خطی خواهد بود.

۱۰- مرتب‌سازی باکتی (bucket)

این روش مرتب‌سازی در زمان خطی قادر به مرتب‌سازی است اگر ورودی دارای توزیع یکنواخت باشد. مشابه مرتب‌سازی شمارشی، بدلیل فرضیاتی در مورد ورودی سرعت خوبی خواهد داشت. برخلاف روش مرتب‌سازی شمارشی که ورودی اعداد صحیح و در بازه کوچکی بود، در اینجا بازه بین $[0, 1)$ بوده و اعداد ورودی فرض شده که بصورت تصادفی و یکنواخت تولید می‌شوند. ایده این روش مرتب‌سازی شامل تقسیم بازه $[0, 1)$ به n زیر ناحیه با اندازه مساوی یا باکت است. از آنجا که ورودی در این بازه به صورت یکنواخت توزیع شده، منطقاً تعداد عناصر هر باکت با دیگر باکت‌ها تقریباً یکسان خواهد بود. برای تولید خروجی کافیس، ابتدا هر باکت مرتب شده و سپس باکت‌ها به ترتیب به خروجی منتقل شوند. الگوریتم این روش مرتب‌سازی به قرار زیر است:

Bucket - Sort ($A[1..n]$)

{

۱. *for* ($i = 1$ *to* n)

۲. *insert* $A[i]$ *into* *list* $B[\lfloor nA[i] \rfloor]$;

۳. *for* ($i = 0$ *to* $n - 1$)

۴. *sort* *list* $B[i]$ *with* *insertion sort* ;

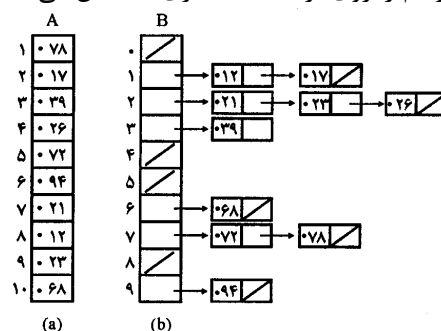
۵. *Concatenate the lists* $B[0], B[1], \dots, B[n - 1]$ *together in order* ;

در این الگوریتم، A آرایه n عنصری ورودی است به طوریکه $0 \leq A[i] < 1$ و آرایه کمکی

$B[0..n - 1]$ آرایه‌ای است از لیست‌های پیوندی یا باکت‌ها.

مرتبه این الگوریتم از $\theta(n)$ خواهد بود.

شکل مقابل اجرای الگوریتم را روی آرایه ۱۰ عنصری A نشان می‌دهد.



(a) آرایه ورودی $A[1 \dots n]$

(b) آرایه $B[0 \dots n-1]$ از لیست‌های مرتب (باکت‌ها) بعد از اجرای خط ۴ الگوریتم، باکت i

شامل مقادیر بازه $\left[\frac{i}{n}, \frac{(i+1)}{n}\right)$ است. خروجی مرتب شده، شامل اتصال $B[0]$ و $B[1]$ و $\dots B[n-1]$ است.

خلاصه روش‌های مرتب‌سازی

روش‌های مبتنی بر مقایسه در جدول زیر آمده‌اند:

بهترین حالت	حالت متوسط	بدترین حالت	درجا	متعادل	
n^2	n^2	n^2	✓	×	selection sort
n	n^2	n^2	✓	✓	bubble sort
n	n^2	n^2	✓	✓	insertion sort
$n \lg n$	$n \lg n$	n^2	✓	×	quick sort
$n \lg n$	$n \lg n$	$n \lg n$	×	✓	merge sort
n	$n \lg n$	$n \lg n$	✓	×	heap sort
$n \lg n$	$n \lg n$	n^2	×	×	tree sort

توجه: روش bubble بهینه در این جدول آمده است.

توجه: روش heapsort در حالتی که همه عناصر آرایه یکسان باشند از مرتبه $\theta(n)$ می‌شود. زیرا ساخت هیپ $\theta(n)$ و n بار حذف ریشه نیز $\theta(n)$ می‌باشد. اگر عناصر آرایه متمایز باشند heapsort همیشه $\theta(n \lg n)$ است.

❖ **نکته:** اگر تعداد عناصر کم باشد (حدود ۲۰ عنصر) یا عناصر مرتب باشند، بهترین روش درجی است.

❖ **نکته:** در حالت متوسط سریع‌ترین روش quicksort است.

❖ **نکته:** تعداد مقایسه در روش‌های حبابی، درجی، سریع حداکثر $\frac{n(n-1)}{2}$ و در روش انتخابی

دقیقاً $\frac{n(n-1)}{2}$ است.

❖ **نکته:** هیچ روش مقایسه‌ای در حالت متوسط نمی‌تواند از $n \lg n$ سریع‌تر باشد.

نکته: روش‌های شمارشی، مبنایی و باکتي، غیرمقایسه‌ای هستند که در حالت متوسط از مرتبه خطی (n) هستند و متعادل و غیردرجا هستند.

تمرین

۱- یک درخت تصمیم‌گیری، درخت دودویی است که یک روش مرتب‌سازی براساس مقایسه را نشان می‌دهد. شکل زیر چنین درختی را برای ۳ عنصر نشان می‌دهد. هر گره داخلی این درخت یک مقایسه بین دو عنصر را نشان می‌دهد. هر برگ یک نتیجه مرتب شده را نشان می‌دهد. یک انشعاب چپ نشان می‌دهد که عنصر اول کوچکتر از دوم است و یک انشعاب راست نشان می‌دهد عنصر اول بزرگتر است.

(فرض می‌کنیم تمام عناصر منحصر به فرد هستند). به عنوان مثال در شکل سه عنصر $A[1]$ و $A[2]$ و $A[3]$ با هم مقایسه شده‌اند: $A[1]$ با $A[2]$ مقایسه می‌شود اگر $A[1] < A[2]$ آنگاه $A[2]$ با $A[3]$ مقایسه می‌شود و اگر $A[2] < A[3]$ آرایه مرتب شده:

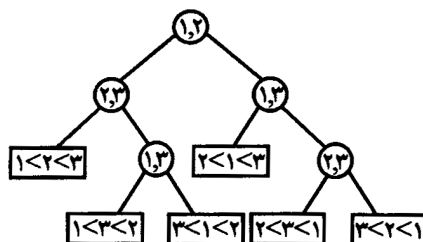
خواهد بود و به همین ترتیب.

$A[1]$	$A[2]$	$A[3]$
--------	--------	--------

(a) نشان دهید چنین درختی به شرطی که اصلاً مقایسه اضافی انجام ندهد حداکثر $n!$ برگ دارد.
 (b) نشان دهید عمق چنین درختی حداقل $\log_2(n!)$ است. (سطح ریشه صفر فرض می‌شود و عمق درخت ماکزیمم شماره سطوح می‌باشد)

(c) نشان دهید $n! \geq \left(\frac{n}{4}\right)^2$ بنابراین عمق درخت از مرتبه $O(n \log n)$ است.

(d) نتیجه بگیرید که روش‌های مرتب‌سازی مبتنی بر مقایسه حداقل $O(n \log n)$ مقایسه نیاز دارند.



۲- روش مرتب‌سازی *shell* (یا *diminishing increment sort*) از مرتب‌سازی درجی استفاده می‌کند. می‌دانیم مرتب‌سازی درجی در دو حالت (تعداد عناصر کم باشد، آرایه مرتب باشد) از سایر

روش‌ها بهتر است. روش *shell* برای مرتب‌سازی $A[1..n]$ ابتدا $\frac{n}{4}$ جفت $\left(A[i], A\left[\frac{n}{4} + i\right]\right)$

را در اولین مرحله مرتب می‌کند. $1 \leq i \leq \frac{n}{4}$

در مرحله دوم $\frac{n}{4}$ تایی‌های $\left(A[i], A\left[\frac{n}{4} + i\right], A\left[\frac{n}{2} + i\right], A\left[\frac{3n}{4} + i\right]\right)$ ، $1 \leq i \leq \frac{n}{4}$ مرتب می‌شوند در مرحله سوم، $\frac{n}{8}$ تا عنصر با هم مرتب می‌شوند و به همین ترتیب. در هر مرحله با روش درجی مرتب‌سازی انجام می‌شود:

```
incr = ⌊ $\frac{n}{2}$ ⌋;
while (incr > ۰)
{
    for (i = incr + ۱ to n)
    {
        j = i - incr;
        while (j > ۰)
        if (A[j] > A[j + incr])
        {
            swap (A[j], A[j + incr]);
            j = j - incr;
        }
        else
            j = ۰ / break
    }
    incr = ⌊ $\frac{incr}{2}$ ⌋
}
```

الف) آرایه مقابل را با این روش مرتب کنید.

A	۲۲	۳۶	۶	۷۹	۲۶	۴۵	۷۵	۱۳	۳۱	۶۲	۲۷	۷۶	۳۳	۱۶	۶۲	۴۷
---	----	----	---	----	----	----	----	----	----	----	----	----	----	----	----	----

ب) فاصله (گپ) بین عناصری که با هم در یک مرحله مقایسه و تعویض می‌شوند، به ترتیب $1, 2, \dots, \frac{n}{4}, \frac{n}{4}$ است. نشان دهید این فاصله هر دنباله‌ای باشد (فقط به شرطی که در نهایت ۱ شود) الگوریتم به درستی کار می‌کند.

ج) نشان دهید زمان *shellsort* برابر $O(n^{1/5})$ است.

۳- جدول یانگ $m \times n$ یک ماتریس $m \times n$ است که هر سطر و هر ستون آن صعودی مرتب هستند. بعضی عناصر ∞ هستند یعنی مقدار ندارند (موجود نیستند). پس تعداد عناصر $r \leq mn$ است.

(a) یک جدول یانگ 4×4 با عناصر $\{9, 16, 3, 2, 4, 8, 5, 14, 12\}$ بکشید.
(b) بررسی کنید اگر $Y[1, 1] = \infty$ آنگاه جدول یانگ Y خالی است و اگر $Y[m, n] < \infty$ جدول پر است.

(c) الگوریتم $EXTRACT-MIN$ عنصر می‌نیم را حذف می‌کند. با مرتبه $O(m + n)$ این الگوریتم را پیاده کنید. الگوریتم بازگشتی است و برای حذف می‌نیم در جدول $m \times n$ ابتدا زیر مسائل با اندازه $(m - 1) \times n$ یا $m \times (n - 1)$ را حل می‌کند.

(d) نشان دهید چگونه یک عنصر جدید به جدول یانگ $m \times n$ با مرتبه $O(m + n)$ درج کنیم.
(e) بدون استفاده از روش‌های مرتب‌سازی، و با استفاده از $EXTRACT-MIN$ عناصر جدول یانگ $n \times n$ را با مرتبه $O(n^3)$ در یک آرایه یک بعدی مرتب کنید.

(f) الگوریتمی با مرتبه $O(m + n)$ برای جستجوی یک عنصر در جدول یانگ $m \times n$ ارائه دهید.
۴- فرض کنید A یک آرایه به طول n است که حداکثر k کلید متفاوت دارند و $k < \sqrt{n}$ می‌خواهیم A را مرتب صعودی کنیم. برای این منظور دو مرحله انجام می‌دهیم.
مرحله ۱: ساخت آرایه مرتب B که شامل k کلید متفاوت موجود در A است.
مرحله ۲: مرتب‌سازی آرایه A با کمک آرایه B
توجه کنید که n عنصر موجود در A علاوه بر کلید، فیلد دیتا (*satellite data*) نیز دارند.

مثال:

$$A = \left[5, \pi, \frac{128}{279}, \pi, \frac{128}{279}, \pi \right]$$

$$n = 6, k = 3$$

$$\text{مرحله ۱: } B = \left[\frac{128}{279}, \pi, 5 \right]$$

$$\text{مرحله ۲: } \left[\frac{128}{279}, \frac{128}{279}, \pi, \pi, \pi, 5 \right]$$

مرتبه این الگوریتم را بیابید.

پاسخ:

مرحله ۱: برای $i = ۱, ۲, \dots, n$ ، عنصر $A[i]$ را در B جستجوی دودویی می‌کنیم. اگر $A[i]$ در B وجود داشت، آن را درج نمی‌کنیم. در غیر این صورت محل درج $A[i]$ در B مشخص شده است که باید عناصر بعد از آن در B یک واحد شیفت راست داده شوند که عنصر $A[i]$ در B درج شود. جستجوی دودویی n عنصر در B از مرتبه $O(n \lg k)$ است (B ، حداکثر k عنصر دارد) علاوه بر این حداکثر k عمل درج در B انجام می‌شود که مرتبه آنها $O(۱ + ۲ + \dots + k) = O(k^۲)$ است پس این مرحله $O(n \lg nk + k^۲) = O(n \lg k)$ است ($k < \sqrt{n}$) مرحله ۲: الگوریتم مرتب‌سازی شمارشی را کمی تغییر می‌دهیم.

Modifiedcountingsort(A)

```
{
  /* use  c[۱..k] , D[۱..k], out[۱..n] */
  for(i = ۱ to k) c[i] = ۰; /* مقداردهی اولیه */
  for(i = ۱ to n) /* شمارش تعداد عناصر */
  {
    Loc = binarysearch(B, A[i]);
    C[Loc] = C[Loc] + ۱;
  }
  D[۱] = C[۱]
  for(j = ۲ to k) /* محاسبه فراوانی تجمعی */
    D[j] = D[j - ۱] + C[j];
  for(i = n down to ۱) /* محاسبه لیست مرتب out */
  {
    Loc = binarysearch(B, A[i]);
    outLoc = D[Loc];
    D[Loc] = D[Loc] - ۱;
    out[outLoc] = A[i];
  }
  output(out)
```

حلقه اول و سوم از مرتبه $O(k)$ است. حلقه دوم و چهارم از مرتبه $O(n \lg k)$ است. دقت کنید $binarysearch(B, A[i])$ عنصر $A[i]$ را در B جستجوی دودویی می‌کند و از مرتبه $O(\lg k)$ است پس کل الگوریتم از مرتبه $O(n \lg k)$ است.

۵- الگوریتم زیر نوعی مرتب‌سازی شمارشی یا لانه کبوتری است. زمان اجرا از چه مرتبه‌ای است؟ متعادل است یا خیر؟

```

T[۱..n];
u[۱..m];
for(k = ۱ to m) u[k] = ۰;
for(i = ۱ to n) u[T[i]] ++;
i = ۰;
for(k = ۱ to m)
    while (u[k] ≠ ۰)
    {
        i ++;
        T[i] = k;
        u[k] --;
    }

```

پاسخ:

n عدد صحیح در بازه ۱ تا m با این الگوریتم به صورت نامتعادل و غیردرجا با مرتبه $\theta(m + n)$ مرتب می‌شوند. تعداد اجرای حلقه for سوم:

$$\sum_{k=1}^m (u[k] + 1) = \sum_{k=1}^m u[k] + \sum_{k=1}^m 1 = n + m$$

۶- آیا می‌توان روش مرتب‌سازی نامتعادل را بدون آنکه مرتبه‌اش زیاد شود متعادل کرد؟

پاسخ:

کافیست کنار هر عنصر اندیشش را نیز قرار دهیم و حال اگر $A[i] == A[j]$ سپس i و j را مقایسه کنیم. این عمل زمان اجرا را حداکثر ۲ برابر می‌کند ولی مرتبه را زیاد نمی‌کند.

[سؤال‌های طبقه‌بندی شده فصل هشتم]

- ۱- کدام الگوریتم مرتب‌سازی، یک آرایه تقریباً مرتب شده را سریعتر مرتب می‌نماید؟
 (۱) Selection sort (۲) Quick sort
 (۳) Bubble sort (۴) Insertion sort
- ۲- لیست S که از $n=6$ حرف تشکیل شده به صورت F و E و D و C و B و A می‌باشد، تعداد مقایسه‌های لازم برای مرتب کردن S با الگوریتم *Quick Sort* عبارت است از:
 (۱) لیست مرتب است و مقایسه‌ای نداریم. (۲) ۳۶
 (۳) ۱۵ (۴) ۱۰
- ۳- فرض کنید بخواهیم روش *Quick Sort* را در مورد آرایه زیر بکار ببریم، در اولین مرحله از کار، شکل آرایه، مطابق کدام یک از موارد ذیل خواهد شد؟ (با توجه به روش گفته شده در این کتاب)
 (۱) ۱۶ و ۲۱ و ۹ و ۲۵ و ۳۲ و ۴۱ (۲) ۴۱ و ۳۲ و ۲۵ و ۲۱ و ۱۶ و ۹
 (۳) ۴۱ و ۳۲ و ۲۵ و ۱۶ و ۲۱ و ۹ (۴) ۴۱ و ۳۲ و ۲۵ و ۹ و ۱۶ و ۲۱
- ۴- کدام یک از روش‌های مرتب‌سازی زیر، ترتیب اولیه رکوردهای هم کلید (منظور برابری کلید مرتب‌سازی در این رکوردهاست) را حفظ می‌کند؟
 (دوگزینه‌ای) (۱) Selection Sort (۲) Quick Sort
 (۳) Insertion Sort (۴) Heap Sort
- ۵- برای مرتب کردن آرایه‌ای به طول $n \geq 1000$ کدام یک از الگوریتم‌های زیر پایین‌ترین زمان اجرا را در بدترین حالت دارد و مقدار حافظه مصرفی کمکی آن مستقل از n می‌باشد؟
 (دوگزینه‌ای) (۱) Heap Sort (۲) Insertion - Sort
 (۳) Quick Sort (۴) Merge-Sort
- ۶- آرایه A شامل n عنصر مرتب (از اندیس ۱ تا n) و k عنصر نامرتب (از اندیس $n+1$ تا $n+k$) است. کدام یک از الگوریتم‌های زیر برای مرتب‌سازی A کمترین تعداد مقایسه را دارد؟
 فرض کنید k مستقل از n و بسیار کمتر از آن است. (دوگزینه‌ای)
 (۱) Insertion Sort (۲) Quick Sort
 (۳) Heap Sort (۴) Merge Sort

- ۷- الگوریتم زیر بیان کننده چه مرتب‌سازی می‌باشد (x - عدد بسیار کوچکی است)
۱. $Set A[0] := -x$ (۱) انتخابی (۲) درجی
 ۲. Repeat steps ۳ to ۵ for $k = ۲, ۳, \dots, N$ (۳) ادغامی (۴) دودویی
 ۳. $Set temp := A[k]$ and $ptr := k - ۱$
 ۴. Repeat while $temp < A[PTR]$
 - $Set A[PTR + ۱] := A[PTR]$
 - $Set PTR := PTR - ۱$
 - $\{end\ of\ Loop\}$
 ۵. $Set A[PTR + ۱] := temp$
 - $\{end\ of\ step\ 2\ Loop\}$
 ۶. return

- ۸- تعداد مقایسه‌های انجام شده در الگوریتم زیر چه خواهد بود؟ (آیاد ۷۵)

```

for item:=n downto 2 do
begin
    Large:=List[1];
    Index:=1;
    for i:=2 to item do
        if List[i]>Large then
            begin
                Large:=List[i];
                Index:=i;
            end;
    List[Index]:=List[item];
    List[item]:=Large;
end;
{1}
end;

```

(۱) $O(n^۳)$

(۲) $O(۲^n)$

(۳) $O(n^۲)$

(۴) $O(n)$

۹- در تست قبل کدامیک از جملات زیر در نقطه $\{1\}$ صحت خواهد داشت؟ (آزاد ۷۵)

(۱) $List[j] < List[j+1]$ for all j such that $item \leq j < n$

(۲) $List[j] \leq List[j+1]$ for all j such that $item \leq j < n$

(۳) $List[j] < List[j-1]$ for all j such that $1 < j \leq item$

(۴) $List[j] < List[j-1]$ for all j such that $item \leq j < n$

۱۰- رویه زیر را در نظر بگیرید:

```

Procedure MergeSort (low, high, A)
// A [low .. high] is an array Containing Values which
// represents the elements to be sorted //
begin
  if Low < high then
    begin
       $mid \leftarrow \lfloor (low + high) / 2 \rfloor$ 
      Mergesort (Low, mid, A)
      Mergesort (mid+1, high, A)
      Merge(Low, mid, high, A)
    end
  end
end

```

رویه Merge در رویه فوق دو لیست مرتب شده $A[low..mid]$ و $A[mid+1..high]$ را ادغام می‌کند. تعداد صداهای انجام گرفته به رویه فوق برای مرتب کردن لیست زیر چیست؟

(آزاد ۷۸)

۲۰ (۴)

۱۹ (۳)

۲۸ (۲)

۲۳ (۱)

(آزاد ۸۰)

۱۱- رویه زیر را در نظر بگیرید:

```

Procedure sort(A , n):
begin
  for i:=1 to n do begin
    j:=i;
    for k:=j+1 to n do
      if  $a[k] < a[j]$  then  $j:=k$ ;
     $t:=a[i]$ ;  $a[i]:=a[j]$ ;  $a[j]:=t$ ;
  end
end

```

end
end;

این رویه لیست n قلمی A را به کدام روش مرتب می‌کند؟

۱) *Insetion Sort*

۲) *Selection Sort*

۳) *Bubble Sort*

۴) این رویه اصولاً برای مرتب کردن یک لیست نوشته نشده است.

۱۲- اگر در الگوریتم مرتب‌سازی سریع (*Quick Sort*) به ترتیب صعودی، عنصر لولا (*Pivot*) را همان عنصر اول لیست بگیریم و با استفاده از آن یک بار لیست مرتب نزولی و یکبار دیگر لیست مرتب صعودی را مرتب کنیم گزینه صحیح برای مرتبه تعداد عملیات اصلی (مقایسه و جابه‌جایی) را در این دو حالت انتخاب کنید. (علوم کامپیوتر ۷۹)

۱) هر دو حالت از $O(n \log n)$

۲) هر دو حالت از $O(n^2)$

۳) برای لیست صعودی $O(n)$ و برای لیست نزولی $O(n^2)$

۴) برای لیست صعودی $O(n \log n)$ و برای لیست نزولی $O(n^2)$

۱۳- یک ماتریس $N \times M$ از اعداد صحیح دلخواه داده شده است. الگوریتم زیر را بر روی این ماتریس انجام می‌دهیم:

الف) هر کدام از سطرها را ماتریس را مستقلاً از چپ به راست به صورت صعودی مرتب می‌کنیم.
ب) هر کدام از ستون‌های ماتریس را مستقلاً از بالا به پایین به صورت صعودی مرتب می‌کنیم.

کدام یک از گزینه‌های زیر درست است؟

۱) سطرها لزوماً مرتب نیستند ولی اگر یک بار سطرها را مرتب کنیم هم سطرها و هم ستون‌ها مرتب می‌شوند.

۲) سطرها لزوماً مرتب نیستند، ولی اگر یک بار دیگر هم سطرها را مرتب کنیم، ستون‌ها لزوماً مرتب نمی‌شوند.

۳) سطرها و ستون‌ها هر کدام به صورت صعودی مرتب می‌شوند.

۴) تکرار این الگوریتم لزوماً هم سطرها و هم ستون‌ها را مرتب نمی‌کند.

۱۴- اگر آرایه مرتب A دارای r عنصر و آرایه مرتب B دارای p عنصر باشد، حداکثر تعداد مقایسه برای ترکیب (*Merge*) دو آرایه کدام است؟

$$\frac{r+p}{2} \quad (۱) \quad r+p \quad (۲) \quad \text{Max}(r,p) \quad (۳) \quad r+p-1 \quad (۴)$$

۱۵- N گلوله با وزن‌های مختلف را می‌خواهیم با یک ترازوی دو کفه‌ای بدون وزنه و با توزین‌های متوالی مرتب کنیم (یک توزین عبارت است از قرار دادن دو گلوله در دو کفه ترازو و مقایسه وزن‌های آنها). کدام یک از گزینه‌های زیر درست است؟ (دو نمره ۷۴)

(۱) ۳ گلوله را می‌توان حداکثر با ۲ بار توزین مرتب کرد.

(۲) ۴ گلوله را می‌توان حداکثر با ۴ بار توزین مرتب کرد.

(۳) ۴ گلوله را می‌توان در مواردی با ۳ بار توزین مرتب کرد.

(۴) هیچ‌کدام

۱۶- اگر بخواهیم یک لیست متصل (*linked list*) که آدرس اول آن *first* می‌باشد را با کمترین تعداد عملیات مرتب نماییم جای علامت؟ در الگوریتم زیر چه مقادیری به ترتیب قرار دهیم: (علوم کامپیوتر ۸۰)

for ($p=first; ?; p=p \rightarrow link$)

for ($q = ?; q=q \rightarrow link$)

if ($p \rightarrow Data > q \rightarrow Data$)

Swap ($\&p \rightarrow Data, \&q \rightarrow Data$);

$$\begin{array}{ccccccc} P & & P \rightarrow link & & P & & P \\ p \rightarrow link & (۴) & P & (۳) & P & (۲) & P \rightarrow link & (۱) \end{array}$$

۱۷- رویه *partition* در الگوریتم *Quick Sort* به صورت زیر است: (دو نمره ۸۱)

function partition($p, r:integer$):integer;

var x,i,j : integer;

begin

$x := A[p]; i := p - 1; j := r + 1;$

while true do begin

repeat $j := j - 1$ until $A[j] \leq x$;

repeat $i := i + 1$ until $A[i] \geq x$;

if $i < j$ then swap ($A[i], A[j]$) else return (j)

end

end;

اگر تمام درایه‌های $A[p..r]$ دارای مقدار یکسانی باشند، مقداری که رویه فوق برمی‌گرداند چقدر است؟

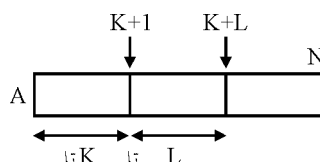
$$\left\lfloor \frac{p+r}{2} \right\rfloor \quad (۴) \quad \left\lceil \frac{p+r}{2} \right\rceil \quad (۳) \quad r \quad (۲) \quad p \quad (۱)$$

۱۸- الگوریتم زیر برای مرتب‌سازی صعودی آرایه A با N عنصر (با اندیس‌های از ۱ تا N) پیشنهاد شده است. در این الگوریتم، فرض کنید $Sort(A, i, j)$ عناصر i تا j از آرایه A را با یکی از الگوریتم‌های شناخته شده به صورت صعودی مرتب می‌کند. (دولتی ۷۸)

$Sort(A, k+1, N)$

$Sort(A, 1, K+L)$

$Sort(A, K+1, N)$



کدام یک از گزینه‌های زیر برای هر N درست است؟

- (۱) تنها اگر $K \leq L$ باشد این الگوریتم درست است.
- (۲) تنها اگر $K > L$ باشد این الگوریتم درست است.
- (۳) تنها اگر $N = 3K = 3L$ باشد الگوریتم درست است.
- (۴) تنها اگر $K = L$ باشد الگوریتم درست است.

۱۹- یک الگوریتم مرتب کننده را بر روی آرایه‌ای از ۹ عدد دنبال کرده‌ایم. بطوری که پس از هر تعویض ($swap$) کل آرایه نوشته شده است. نتیجه به صورت جدول زیر درآمده است. با توجه به خروجی زیر الگوریتم مرتب کننده:

(دولتی ۷۱)

$bubble\ sort$ است.

$insertion\ sort$ است.

$Quick\ sort$ است.

$heap\ sort$ است.

۱	۲	۳	۴	۵	۶	۷	۸	۹	$swap(i,j)$
۷	۵۹	۱۱	۲۶	۵	۷۷	۱	۶۱	۵۹	مقادیر اولیه
۷	۵۹	۱۱	۶۱	۵	۷۷	۱	۲۶	۵۹	$swap(۸ و ۴)$
۷	۵۹	۷۷	۶۱	۵	۱۱	۱	۲۶	۵۹	$swap(۶ و ۳)$
۷	۶۱	۷۷	۵۹	۵	۱۱	۱	۲۶	۵۹	$swap(۴ و ۲)$
۷۷	۶۱	۷	۵۹	۵	۱۱	۱	۲۶	۵۹	$swap(۳ و ۱)$
۷۷	۶۱	۱۱	۵۹	۵	۷	۱	۲۶	۵۹	$swap(۶ و ۳)$
۵۹	۶۱	۱۱	۵۹	۵	۷	۱	۲۶	۷۷	$swap(۹ و ۱)$
۶۱	۵۹	۱۱	۵۹	۵	۷	۱	۲۶	۷۷	$swap(۲ و ۱)$
۲۶	۵۹	۱۱	۵۹	۵	۷	۱	۶۱	۷۷	$swap(۸ و ۱)$
۵۹	۲۶	۱۱	۵۹	۵	۷	۱	۶۱	۷۷	$swap(۲ و ۱)$
۵۹	۵۹	۱۱	۲۶	۵	۷	۱	۶۱	۷۷	$swap(۴ و ۲)$
۱	۵۹	۱۱	۲۶	۵	۷	۵۹	۶۱	۷۷	$swap(۷ و ۱)$
۵۹	۱	۱۱	۲۶	۵	۷	۵۹	۶۱	۷۷	$swap(۲ و ۱)$
۵۹	۲۶	۱۱	۱	۵	۷	۵۹	۶۱	۷۷	$swap(۴ و ۲)$
۷	۲۶	۱۱	۱	۵	۵۹	۵۹	۶۱	۷۷	$swap(۶ و ۱)$
۲۶	۷	۱۱	۱	۵	۵۹	۵۹	۶۱	۷۷	$swap(۲ و ۱)$
۵	۷	۱۱	۱	۲۶	۵۹	۵۹	۶۱	۷۷	$swap(۵ و ۱)$
۱۱	۷	۵	۱	۲۶	۵۹	۵۹	۶۱	۷۷	$swap(۳ و ۱)$
۱	۷	۵	۱۱	۲۶	۵۹	۵۹	۶۱	۷۷	$swap(۴ و ۱)$
۷	۱	۵	۱۱	۲۶	۵۹	۵۹	۶۱	۷۷	$swap(۲ و ۱)$
۵	۱	۷	۱۱	۲۶	۵۹	۵۹	۶۱	۷۷	$swap(۳ و ۱)$
۱	۵	۷	۱۱	۲۶	۵۹	۵۹	۶۱	۷۷	$swap(۲ و ۱)$

۲۰- اگر برای تمام مقادیر $1 \leq i \leq n$ و $A[i] \geq 1$ و $A[0] = 0$ باشد، روال زیر، آرایه

$A[0..n]$ را به چه روشی مرتب می‌کند؟ (دولتی ۷۶)

<i>for i := ۲ to n do</i>	<i>Insertionsort</i> (۱)
<i>begin</i>	<i>selectionsort</i> (۲)
<i> j := i;</i>	<i>shellsort</i> (۳)
<i> while A[j] < A[j - ۱] do</i>	<i>Bubblesort</i> (۴)
<i> begin</i>	
<i> T := A[j - ۱]</i>	
<i> A[j - ۱] := A[j];</i>	
<i> A[j] := T;</i>	
<i> j := j - ۱;</i>	
<i> end</i>	
<i>end;</i>	

۲۱- کدام جمله صحیح است؟ (آزاد ۷۶)

(۱) الگوریتم *Quick Sort* زمانی که داده‌های مرتب باشد بهترین عملکرد را دارد.

(۲) الگوریتم *Quick Sort* زمانی که $n > ۲۳$ باشد بهتر از *Insertion Sort* عمل می‌کند.

(۳) بهترین الگوریتم مرتب‌سازی *Quick Sort* است.

(۴) هیچکدام

۲۲- کدامیک از گزینه‌های زیر در مورد الگوریتم *Quick Sort* درست است؟ فرض کنید که

عنصر اول همواره به عنوان محور (*pivot*) انتخاب می‌شود. n تعداد عناصر آرایه ورودی

است. اگر آرایه ورودی از قبل مرتب باشد، (دولتی ۸۰)

(۱) زمان اجرای الگوریتم $O(n)$ است. (۲) زمان اجرای الگوریتم $O(n^2)$ است.

(۳) زمان اجرای الگوریتم $O(n \log n)$ است. (۴) متوسط زمان اجراء $O(n \log n)$ است.

۲۳- لیست زیر را در نظر بگیرید. اگر عنصر اول لیست یعنی عدد ۹ را به عنوان لولا (*pivot*)

اختیار کنیم کدام یک از گزینه‌های زیر می‌توانند خروجی مرحله اول الگوریتم مرتب‌سازی

سریع (*Quick Sort*) باشد؟ (۳ و ۱۵ و ۶ و ۷ و ۸ و ۱۰ و ۹) (علوم کامپیوتر ۸۰)

(۱) (۱۵ و ۶ و ۳ و ۱۰ و ۹ و ۸ و ۷) (۲) (۱۵ و ۱۰ و ۶ و ۳ و ۹ و ۸ و ۷)

(۳) (۱۰ و ۱۵ و ۹ و ۷ و ۸ و ۳ و ۶) (۴) (۱۵ و ۱۰ و ۳ و ۹ و ۸ و ۷ و ۶)

۲۴- یک الگوریتم مرتب‌سازی برای مرتب کردن n عدد حداقل نیاز به چه تعداد مقایسه دارد؟
(ارشد ۸۳)

$$\log n \quad (1) \quad \log n! \quad (2) \quad n \quad (3) \quad n^2 \quad (4)$$

۲۵- آرایه n عنصری $A[1:n]$ را در نظر بگیرید، کدام یک از گزاره‌های ذیل، درست یا درست‌تر است؟
(۸۳ IT)

(۱) الگوریتم *INSERTION SORT* در بدترین حالت و بطور متوسط، زمان مصرفی مرتبه n^2 دارد.

(۲) الگوریتم *QUICK SORT*، در بدترین حالت و بطور متوسط زمانی در گروه $n \cdot \log n$ دارد.

(۳) الگوریتم *QUICK SORT*، همواره از الگوریتم *INSERTION SORT*، از نظر زمان مصرفی، بهتر است.

(۴) الگوریتم *MERGE SORT*، همواره از *INSERTION SORT* بهتر است، اما *SELECTION SORT* از هر دو، از نظر زمان مصرفی بهتر است.

۲۶- یک الگوریتم مرتب‌سازی صعودی در زمان اجرا در چهارمین تکرار خود دارای ارزش‌های زیر می‌باشد. این الگوریتم چه نوع مرتب‌سازی می‌باشد؟
(علوم کامپیوتر ۸۴)

۱۵	۱۸	۳۰	۳۴	۴۱	۵۰	۹۱	۳۲	۴۹
----	----	----	----	----	----	----	----	----

(۱) *Bubble* (۲) *Selection* (۳) *Insertion* (۴) *Quicksort*

۲۷- الگوریتم *radixsort* می‌تواند n داده را که در محدوده ۰ تا $n^2 - 1$ هستند را در زمان $O(n)$ مرتب کند. چرا این با قضیه‌ای که حد پائین برای مرتب کردن $O(n \log n)$ است تناقض ندارد؟
(علوم کامپیوتر ۸۴)

(۱) حد پایین فقط برای الگوریتم‌هایی که روی داده‌هایی با تعداد کمتر از ۱۰۰ کار می‌کنند.

(۲) حد پایین فقط برای الگوریتم‌هایی که براساس درخت تصمیم‌گیری عمل می‌کنند صادق است.

(۳) حد پایین فقط برای الگوریتم‌هایی که عناصر کنار هم را با یکدیگر عوض می‌کنند صادق است.

(۴) حد پایین فقط برای الگوریتم‌هایی که روی داده‌های اعداد حقیقی کار می‌کنند صادق است.

۲۸- کدام یک از گزینه‌های زیر زمان اجرای الگوریتم *Radix Sort* در مبنای r و برای n عدد d رقمی است؟
(۸۴ IT)

$$\theta(d(n+r)) \quad (1) \quad \Omega(d(n+r)) \quad (2)$$

$$O(d(n+r)) \quad (3) \quad \text{هیچکدام} \quad (4)$$

۲۹- کدام گزینه نادرست است؟ (علوم کامپیوتر ۸۵)

- (۱) الگوریتم‌های *sort* که پیچیدگی زمان آن خطی می‌باشند از حافظه کمکی استفاده می‌کنند.
- (۲) الگوریتم‌های *sort* که از مقایسه استفاده می‌کنند دارای حداقل پیچیدگی زمان $O(n \log n)$ می‌باشند.
- (۳) الگوریتم‌های *sort* که به صورت *Divide-conquer* حل می‌شوند دارای $O(n \log n)$ می‌باشند.
- (۴) الگوریتم‌های *sort* که از مقایسه استفاده نمی‌کنند می‌توانند دارای پیچیدگی زمان $O(n)$ می‌باشند.

۳۰- در الگوریتم *Merge sort* اگر به جای آنکه هر بار لیست به دو قسمت مساوی تقسیم شود به چهار قسمت مساوی تقسیم گردد و در مرحله ترکیب این چهار لیست در یکدیگر ادغام شوند پیچیدگی زمانی الگوریتم چه خواهد شد؟ (آزاد ۸۰)

- (۱) $\theta(n)$
- (۲) $\theta(n \log_p n)$
- (۳) $\theta(n^2)$
- (۴) $\theta(n^2 \log_p n)$

۳۱- آرایه n عنصری A را در نظر بگیرید، فرض کنید $n = 2^k$. الگوریتم *Mergesort* بر روی A . (مهندسی کامپیوتر ۸۶)

- (۱) در بدترین حالت $n \cdot \log_p n - n + 1$ مقایسه میان عناصر آرایه انجام می‌دهد.
- (۲) در بدترین حالت $n \cdot \log_p n + n - 1$ مقایسه میان عناصر آرایه انجام می‌دهد.
- (۳) در بدترین حالت $n \cdot \log_p n - 2n - 1$ مقایسه میان عناصر آرایه انجام می‌دهد.
- (۴) در بدترین حالت $n \cdot \log_p n - n - 1$ مقایسه میان عناصر آرایه انجام می‌دهد.

۳۲- الگوریتم زیر به کدام روش عمل *sort* (مرتب‌سازی) را انجام می‌دهد؟ (آزاد ۸۳)

```

radix (۱)
Xsort ( )
{
    shell (۲)
    for i=2 to n do
        {
            Insertion (۳)
            x=A [i]
            j=i-1;
            while (j>0 and A[j]>x)
                do
                    {
                        Selection (۴)
                        A [j+1]=A [j];
                        j=j-1;
                    }
            A [j+1]=x;
        }
    }
} // end Xsort

```

۳۳- جواب تابع بازگشتی زیر پیچیدگی زمانی کدام یک از الگوریتم‌های زیر را نشان می‌دهد؟

(علوم کامپیوتر ۸۵)

$$t(1) = 0$$

$$t(n) = t(n-1) + \frac{2}{n}$$

Insertion sort (۲)

Binary search (۱)

Sequential search (۴)

Quick sort (۳)

۳۴- فرض کنید $S_1, S_2, \dots, S_n$ آرایه‌هایی باشند که تعداد عنصر هر کدام از آنها عدد صحیحی

بین ۱ تا k است (یعنی S_i ها آرایه می‌باشند که $1 \leq |S[i]| \leq k$ و داریم: $\sum_{i=1}^n |S[i]| = k$)

این بدین معنی است که مجموع تعداد عناصر تمام آنها k است. بهترین الگوریتم

مرتب‌سازی کل این آرایه‌ها دارای چه مرتبه زمانی است؟ (علوم کامپیوتر ۸۵)

$O(k \log k)$ (۴)

$O(k \log n)$ (۳)

$O(n)$ (۲)

$O(k)$ (۱)

۳۵- براساس قطعه کد زیر، کدامیک از گزینه‌ها قطعاً در محل *** درست است؟ (۸۵ IT)

(۱) برای تمام j هایی که $1 < j \leq item$ ، $a[j] < a[j-1]$ (for (item=n; item>1; item--))

(۲) برای تمام j هایی که $item < j \leq n$ ، $a[j] < a[j+1]$ {

(۳) برای تمام j هایی که $item \leq j < n$ ، $a[j] < a[j+1]$; int large = a[1];

(۴) برای تمام j هایی که $item \leq j < n$ ، $a[j] < a[j+1]$; int index = 1;

for (i=2 ; i<item; i++)

if (a[i]>large)

{

large = a[i];

index=i;

}

۳۶- اگر در الگوریتم Merge Sort برای مرتب‌سازی لیست‌های زیر ۲۰ عنصر از الگوریتم

Insertion Sot استفاده شود. پیچیدگی زمانی الگوریتم چه خواهد شد؟ (۸۵ IT)

$\theta(n \log n)$ (۴)

$\theta(n^2 \log n)$ (۳)

$\theta(n)$ (۲)

$\theta(n^2)$ (۱)

۳۷- مرتبه زمانی مرتب‌سازی یک آرایه با n عنصر با روش Merge sort در بدترین حالت

(Worst case) برابر است با: (۸۶ IT)

$n^2 \log n$ (۴)

$n \log n$ (۳)

n^2 (۲)

n (۱)

۳۸- فرض کنید یک درخت تصمیم‌گیری دودویی برای ادغام دو لیست مرتب (*Sorted*) زیر داریم:

$$list\ 1: x_1, x_2$$

$$list\ 2: y_1, y_2, y_3, y_4, y_5$$

در درخت، گره‌های میانی نشان دهنده مقایسه و گره‌های برگ نشان دهنده ترتیب لیست

مرتب نهایی است. حد پایین تعداد گره‌های برگ این درخت چند است؟ (۸۸ IT)

$$(1) \ 32 \quad (2) \ 21 \quad (3) \ 15 \quad (4) \ 10$$

۳۹- الگوریتم *Radix Sort* را روی n عدد در فاصله $[1, n^2]$ اجرا می‌کنیم. پایه استفاده

شده در *Radix Sort* برابر n است. متوسط زمان اجرای این الگوریتم چقدر است؟ (۸۸ IT)

$$(1) \ \theta(n^2) \quad (2) \ \theta(n) \quad (3) \ \theta(n \log n) \quad (4) \ \theta^2(n \log n)$$

۴۰- کدام گزینه تعداد برگ‌های یک درخت تصمیم‌گیری (*Decision tree*) برای مرتب‌سازی

پنج (۵) عنصر را نشان می‌دهد؟ (۸۷ IT)

$$(1) \ 64 \quad (2) \ 120 \quad (3) \ 128 \quad (4) \ 720$$

۴۱- می‌دانیم که هزینه الگوریتم مرتب‌سازی درجی (*Insertion Sort*) برای مرتب‌سازی

یک آرایه A با n عنصر متناسب با تعداد «وارونگی» (*inversion*) های عناصر آن آرایه

است. زوج (i, j) را یک عدد وارونگی می‌گوییم اگر $i < j$ و $A[i] > A[j]$ با فرض

احتمال این که یک زوج اندیس دلخواه از A یک وارونگی باشد برابر $\frac{1}{4}$ است. میانگین

تعداد وارونگی‌های یک آرایه A یا عناصر متمایز چقدر است؟ (مهندسی کامپیوتر ۸۹)

$$(1) \ \frac{n^2 - n}{2} \quad (2) \ \frac{n^2}{2} \quad (3) \ \frac{n^2}{4} \quad (4) \ \frac{n^2 - n}{4}$$

۴۲- در الگوریتم مرتب‌سازی آرایه‌ای A با n عنصر فرض کنید $b > 1$ یک عدد ثابت است.

همچنین فرض کنید که هزینه مقایسه‌ی دو عنصر $A[i]$ و $A[j]$ ، یا تعویض آنها، اگر

$|j - i| \leq b$ برابر صفر (خیلی کم) و در غیر این صورت برابر ۱ (خیلی زیاد) است. توجه

کنید که با این فرض، هزینه مرتب‌سازی درجی، حبابی (*Bubble sort*) برابر $O(1)$

می‌شود. چون فقط عناصر مجاور را مقایسه و تعویض می‌کنند. با این فرض هزینه

مرتب‌سازی ادغامی (*Merge sort*) A در بدترین حالت چقدر است؟ (بهترین جواب را

انتخاب کنید) (بدیهی است که اگر $T(n)$ زمان اجرا باشد داریم: $n < b$ و $T(n) = 1$).

(مهندسی کامپیوتر ۸۹)

$$(1) \ O(n \lg(n/b)) \quad (2) \ O(n/b \lg(n/b))$$

$$(3) \ O(n \lg n) \quad (4) \ O(n \lg n)$$

۴۳- فرض کنید می‌خواهیم یک لیست حاوی عناصر (از چپ به راست) ۲ و ۴ و ۵ و ۳ و ۱ را به لیست با عناصر (از چپ به راست) ۱ و ۲ و ۳ و ۴ و ۵ تبدیل کنیم. تنها عملیات مجاز جابجا کردن درایه‌های متوالی (پشت سرهم) با هم یکدیگر است. حداقل چند جابجایی لازم است؟

(۸۹ IT)

(۱) ۵ (۲) ۶ (۳) ۴ (۴) ۷

۴۴- n عدد صحیح بین ۱ تا $\sqrt{n}$ داده شده که ممکن است یک عدد بیش‌تر یک بار آمده باشد و فرقی بین آنها نیست. داده ساختاری می‌خواهیم تا با مصرف حافظه‌ی کم، اعمال درج، حذف و جستجو را در زمان کمی انجام دهد. بهترین داده ساختار برای این مسئله کدام یک از ویژگی‌های زیر را دارد؟ (بهترین جواب ممکن را علامت بزنید) (مهندسی کامپیوتر ۹۰)

(۱) درج، حذف و جستجو در $O(1)$ و میزان حافظه $\sqrt{n}$

(۲) درج، حذف و جستجو در $O(1)$ و میزان حافظه n

(۳) درج، حذف و جستجو در $O(\lg n)$ و میزان حافظه $\sqrt{n}$

(۴) درج و حذف از $O(\lg n)$ و جستجو در $O(1)$ میزان حافظه $\sqrt{n}$

پاسخنامه سؤال‌های طبقه‌بندی شده فصل هشتم

- (۱) گزینه (۴). مرتب‌سازی درجی برای آرایه مرتب بهترین است.
- (۲) گزینه (۳). می‌توان نشان داد مقایسه‌ها حداکثر $\frac{n(n-1)}{2}$ یعنی ۱۵ است.
- (۳) گزینه (۳). عدد ۲۵ محور می‌باشد. ۴۱ با ۲۱ جابجا می‌شود و سپس عدد ۳۲ با ۱۶ سپس ۲۵ با ۹ عوض می‌شود.
- (۴) گزینه (۳). درجی، متعادل است.
- (۵) گزینه (۱). روش‌های *heapsort* و *mergesort* در بدترین حالت از مرتبه $n \log_2^n$ هستند ولی *merge* حافظه کمکی نیاز دارد.
- (۶) گزینه (۱). آرایه، تقریباً مرتب است و برای آرایه مرتب، روش درجی بهتر از بقیه است.
- (۷) گزینه (۲). جمله $A[\circ] := -x$ نشان می‌دهد که روش درجی است. این جمله باعث می‌شود، مقایسه از اول آرایه عبور نکند.
- (۸) گزینه (۳). الگوریتم، مرتب‌سازی انتخابی است.
- (۹) گزینه (۲). در حلقه داخلی *for* ماکزیمم عدد را پیدا می‌کند و در خانه *List [item]* می‌گذارد، و *item* از آخر به اول کم می‌شود. پس آرایه به صورت صعودی مرتب می‌شود، یعنی گزینه ۱ یا ۲ صحیح است و از آنجا که *list [j]* می‌تواند با *List [j+1]* برابر باشد گزینه ۲ صحیح است.

(۱۰) گزینه (۳).

$$\underbrace{310, 285, 179, 625, 351, 423, 861, 254, 450, 520}_{\text{بار ۱}} \Rightarrow$$

$$\underbrace{310, 285, 179, 625, 351, 423, 861, 254, 450, 520}_{\text{بار ۲}} \Rightarrow$$

$$\underbrace{310, 285, 179, 625, 351, 423, 861, 254, 450, 520}_{\text{بار ۴}} \Rightarrow$$

$$\underbrace{310, 285, 179, 625, 351, 423, 861, 254, 450, 520}_{\text{بار ۲}} \Rightarrow$$

جمعاً ۹ بار صدا زده شد تا آرایه به آرایه‌های تک عضوی تبدیل شود. حال برای هر یک از این تکی‌ها نیز یک بار صدا زده می‌شود پس $9 + 10 = 19$ تعداد فراخوانی است.

(۱۱) گزینه (۲). این الگوریتم در هر مرحله مینیمم را پیدا می‌کند و در جای خود قرار می‌دهد.

(۱۲) گزینه (۲). هر دو حالت، بدترین حالت است که از مرتبه n^2 است.

(۱۳) گزینه (۳). با مثال بررسی می‌کنیم:

$$\begin{bmatrix} 3 & 2 & 10 \\ 9 & 1 & 6 \\ 7 & 12 & 9 \end{bmatrix} \xrightarrow[\text{سطرها}]{\text{مرتب‌سازی}} \begin{bmatrix} 2 & 3 & 10 \\ 1 & 6 & 9 \\ 7 & 9 & 12 \end{bmatrix} \xrightarrow[\text{ستون‌ها}]{\text{مرتب‌سازی}} \begin{bmatrix} 1 & 3 & 9 \\ 2 & 6 & 10 \\ 7 & 9 & 12 \end{bmatrix}$$

(۱۴) گزینه (۴). حداقل تعداد مقایسه $\min(r, p)$ و حداکثر $r + p - 1$ است.

(۱۵) گزینه (۳). در بهترین حالت اگر همه گلوله‌ها مرتب باشند و به ترتیب برداشته شوند، n گلوله را می‌توان با $n-1$ بار وزن کردن، مرتب کرد ولی در بدترین حالت دو به دوی گلوله‌ها باید مقایسه شوند.

(۱۶) گزینه (۴). برای آرایه‌ها می‌توان به صورت زیر نوشت:

$sort(A, n)$

{

for ($i=1$; $i \leq n-1$; $i++$)

for ($j=i+1$; $j \leq n$; $j++$)

if ($A[i] > A[j]$)

swap ($A[i]$, $A[j]$)

این الگوریتم در هر مرحله کوچکترین عنصر را پیدا می‌کند (مشابه مرتب‌سازی انتخابی)

(۱۷) گزینه (۴). به مثال زیر توجه کنید.

	۱	۲	۳	۴	۵	۶
A	۹	۹	۹	۹	۹	۹

$p=1, r=6, x=9, i=0, j=7$

پس از پایان حلقه‌های $repeat$ مقادیر $i=4$ و $j=3$ می‌شود. پس مقدار $j=3$

یعنی $\left\lfloor \frac{1+6}{2} \right\rfloor$ برگردانده می‌شود.

(۱۸) گزینه (۱).

	۱	۲	۳	۴	۵	۶	۷	۸
A	۱۰	۷	۹	۶	۳	۲	۱	۴

$N=۸, k=۴, L=۲$

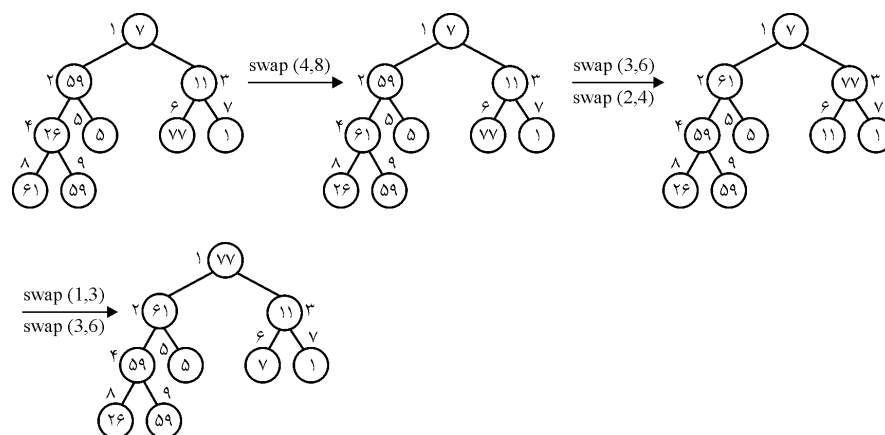
sort(A, ۵, ۸):	۱	۲	۳	۴	۵	۶	۷	۸
	۱۰	۷	۹	۶	۱	۲	۳	۴

sort(A, ۱, ۶):	۱	۲	۳	۴	۵	۶	۷	۸
	۱	۲	۶	۷	۹	۱۰	۳	۴

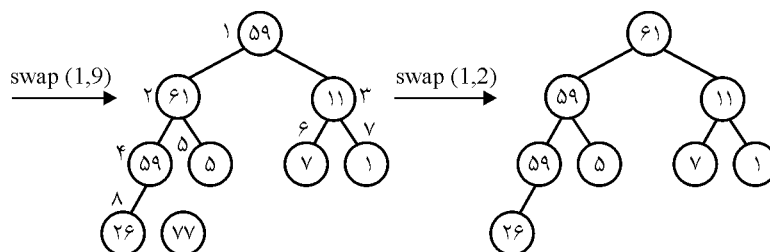
sort(A, ۵, ۸):	۱	۲	۳	۴	۵	۶	۷	۸
	۱	۲	۶	۷	۳	۴	۹	۱۰

همانطور که دیدیم $K > L$ و نتیجه مرتب نیست. این الگوریتم فقط وقتی $K \leq L$ می تواند مرتب کند.

(۱۹) گزینه (۴). در روش *heapsort* ابتدا با آرایه داده شده هیپ ساخته می شود و سپس n بار حذف ریشه انجام می شود؛ برای ساخت هیپ در این مثال از عنصر ۴ شروع به *heapify* می کنیم.



حال باید عملیات حذف ریشه انجام شود که ابتدا ۷۷ با ۵۹ تعویض می شود و *heapsize* یک واحد کم شده و روی ریشه عمل *heapify* انجام می شود.



و به همین ترتیب عملیات حذف ریشه ادامه می‌یابد.

گزینه (۱). هر بار عنصر $n/2$ ام در بین عناصر قبل از خود که قبلاً مرتب شده‌اند، در محل مناسب درج می‌شود. عبارت $A[0] = 0$ نیز نشان می‌دهد که مرتب‌سازی درجی است.

گزینه (۲).

گزینه (۲). زمان مرتب‌سازی سریع از n^2 کمتر یا مساوی است پس می‌توان گفت $O(n^2)$ است.

گزینه (۳). ابتدا ۱۰ با ۳ عوض می‌شود یعنی ۱۰ آخر قرار می‌گیرد، بنابراین فقط گزینه ۳ می‌تواند صحیح باشد.

گزینه (۲). طبق درخت تصمیم در حالت متوسط تعداد مقایسه نمی‌تواند از $Lg(n!)$ کمتر شود.

گزینه (۱).

گزینه (۳). عناصر ابتدایی آرایه مرتب هستند و حال می‌خواهیم عناصر بعدی $(0, 91, 32, 49)$ را درج کنیم.

گزینه (۲). روش *radix* روشی مبتنی بر مقایسه نیست.

گزینه (۱). به متن درس رجوع کنید.

گزینه (۳). روش‌های شمارشی، مبنایی و باکتری که در حالت متوسط خطی هستند همگی از حافظه کمکی استفاده می‌کنند.

گزینه (۲). در این حالت خواهیم داشت:

$$T(n) = \begin{cases} a & n \leq 1 \\ 4T\left(\frac{n}{4}\right) + bn & n > 1 \end{cases} \Rightarrow T(n) = \theta(n \lg n)$$

(۳۱) گزینه (۱).

$$\begin{cases} T(n) = 2T(n/2) + n/2 & \text{تعداد مقایسه‌ها در بهترین حالت} \\ T(n) = 2T(n/2) + (n-1) & \text{تعداد مقایسه‌ها در بدترین حالت} \end{cases}$$

مقایسه بین عناصر آرایه در الگوریتم *merge* صورت می‌گیرد. تعداد دفعات مقایسه این الگوریتم در بهترین حالت $n/2$ است یعنی زمانی که یک زیر لیست از تمامی عناصر زیر لیست دیگر کوچکتر باشد و بدترین شرایط زمانی است که پس از اتمام حلقه *while* در الگوریتم *merge* تقریباً تمامی عناصر (به جز یک عنصر) به آرایه *B* منتقل شده باشند ($n-1$ مقایسه) اما در بهترین حالت تعداد مقایسه $T(n) = n \log n / 2$ و در بدترین حالت $T(n) = n \log n - n + 1$ خواهد بود.

(۳۲) گزینه (۳).

(۳۳) گزینه (۱).

$$t(1) = 0$$

$$t(2) = 0 + \frac{2}{2}$$

$$t(3) = 0 + \frac{2}{2} + \frac{2}{3}$$

$$t(4) = 0 + \frac{2}{2} + \frac{2}{3} + \frac{2}{4}$$

⋮

$$t(n) = 0 + \frac{2}{2} + \frac{2}{3} + \frac{2}{4} + \dots + \frac{2}{n} \approx 2Lnn = \theta(Lgn)$$

(۳۴) گزینه (۱). بهترین الگوریتم مرتب‌سازی می‌تواند شمارشی باشد که مرتبه آن برابر تعداد کل عناصر است.

(۳۵) گزینه (۳). مرتب‌سازی انتخابی صعودی است. چون عنصر ماکزیمم به آخر آرایه منتقل می‌شود. بنابراین از خانه *item* تا خانه *n*ام آرایه مرتب شده صعودی است یعنی $a[j] \leq a[j+1]$ توجه کنید که *item* از خانه آخر به سمت اول حرکت می‌کند و گزینه ۱ غلط است چون *j* به *n* بستگی دارد. گزینه ۲ هم غلط است زیرا اگر $j=n$ باشد شرط $a[j] < a[j+1]$ بی‌معنی است.

۳۶ گزینه (۴). مرتبه زمان اجرا تغییر نمی کند.

۳۷ گزینه (۳).

۳۸ گزینه (۲). دو حالت امکان دارد یا x_1 و x_6 کنار هم قرار می گیرند که ۶ حالت دارد (بین

y_i ها ۶ فضا هست) یا x_1 و x_6 از هم جدا می شوند که $\binom{6}{2}$ حالت دارد پس جواب

$$21 = \binom{6}{2} + 6 \text{ می باشد.}$$

۳۹ گزینه (۲). در مبنای n هر رقم ۰ تا $n-1$ می باشد پس اعداد بازه $[0, n^2-1]$ در مبنای

n دو رقمی می شوند زیرا حداکثر عدد ۲ رقمی مبنای n برابر n^2-1 است پس ۲ بار باید Countingsort اجرا شود که مرتبه $O(n)$ می شود.

۴۰ گزینه (۲). $5! = 120$.

۴۱ گزینه (۴). حداقل تعداد وارونگی برابر صفر است که در صورتی که آرایه صعودی باشد.

حداکثر تعداد وارونگی برابر $\frac{n(n-1)}{2}$ است که برای آرایه n عنصری نزولی اتفاق می افتد چون

هر دو عنصر با یکدیگر وارونه هستند پس $\binom{n}{2}$ وارونگی وجود دارد. متوسط تعداد وارونگی

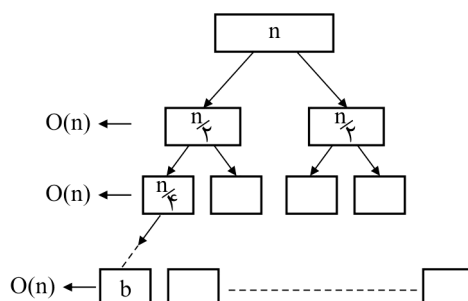
$$\frac{1}{2} \frac{n(n-1)}{2} = \frac{n(n-1)}{4} \text{ می باشد.}$$

۴۲ گزینه (۱). با توجه به صورت سوال اگر b را مثلاً n فرض کنیم آنگاه هزینه همه

مرتب‌سازی‌ها صفر می شود که در نتیجه گزینه ۱ یا ۲ می تواند صحیح باشد. در مرتب‌سازی ادغامی، آرایه نصف می شود و سپس هر نیمه مرتب می شود (با نصف کردن مجدد) و دو نیمه با هم ادغام می شوند.

در این تست وقتی اندازه هر قسمت به b برسد، از آن به بعد دیگر هزینه‌ای وجود ندارد. ولی تا قبل از آن در هر سطح هزینه $O(n)$ است. تعداد سطوح تا وقتی اندازه هر قسمت به b

برسد برابر $O\left(\lg \frac{n}{b}\right)$ است پس هزینه کل $O\left(n \cdot \lg \frac{n}{b}\right)$ است.



۴۳ گزینه (۲).

	۵	۴	۳	۲	۱
A	۱	۳	۵	۴	۲

وارونگی‌ها $(A[۵], A[۴]), (A[۵], A[۳]), (A[۵], A[۲]), (A[۵], A[۱]), (A[۴], A[۳]), (A[۴], A[۲])$:

حداقل تعداد جابجایی به تعداد وارونگی‌هاست. (وارونگی یعنی $i > j$ و $A[i] < A[j]$)

۴۴ گزینه (۱). شبیه *counting sort* یک آرایه C به طول $\sqrt{n}$ تعریف می‌کنیم که در $c[i]$

تعداد i ها ذخیره می‌شوند. درج عنصر i یعنی $C[i]++$ ، حذف عنصر i یعنی $C[i]--$ ،

جستجوی عنصر i یعنی اگر $c[i]$ مخالف صفر باشد i وجود دارد، که تمام این عملیات با

زمان $\theta(۱)$ انجام می‌شوند.

تست‌های تکمیلی

ضمیمه ۱

۱- کدام گزینه نادرست است؟

- (۱) در یک BST برای یافتن یک عنصر یک مسیر از ریشه تا یک برگ را پیموده‌ایم. عناصری که در سمت چپ این مسیر هستند A ، آنهایی که روی مسیر هستند B و عناصری که سمت راست مسیر هستند C می‌نامیم. برای هر $a \in A$ ، $b \in B$ و $c \in C$ می‌توان مطمئن بود که $a \leq b \leq c$
- (۲) درخت قرمز سیاه با ۱۲۸ گره حداقل یک گره قرمز دارد.
- (۳) طول بزرگترین مسیر از یک گره درخت قرمز سیاه تا برگ، حداکثر ۲ برابر طول کوتاه‌ترین مسیر است.
- (۴) با داشتن پیمایش پیش ترتیب یا پس ترتیب یا سطح ترتیب، BST منحصر به فرد است.

۲- چه تعداد از ۷! حالت درج عناصر A تا F در یک BST تهی، همان درختی را تولید می‌کند که اگر این عناصر به ترتیب $CEAGBDF$ (چپ به راست) در آن درج شوند؟

- (۱) $4! \times 3!$ (۲) $4! + 3!$ (۳) ۱۵ (۴) ۴۵

۳- با ۲۵ عنصر چند درخت دودویی با ارتفاع کمینه می‌توان ساخت؟

$$(1) \begin{pmatrix} 16 \\ 10 \end{pmatrix} \quad (2) 56 + \begin{pmatrix} 14 \\ 4 \end{pmatrix} + \begin{pmatrix} 16 \\ 6 \end{pmatrix} \quad (3) \begin{pmatrix} 8 \\ 2 \end{pmatrix} + 8 \begin{pmatrix} 14 \\ 3 \end{pmatrix} + \begin{pmatrix} 16 \\ 6 \end{pmatrix} \quad (4) \begin{pmatrix} 8 \\ 2 \end{pmatrix} + \begin{pmatrix} 8 \\ 2 \end{pmatrix} \begin{pmatrix} 14 \\ 3 \end{pmatrix} + \begin{pmatrix} 16 \\ 10 \end{pmatrix}$$

۴- عمل $Enum(x, a, b)$ بر روی زیر درخت BST به ریشه x ، کلیه کلیدهای $a \leq k \leq b$ را پیدا می‌کند (چاپ می‌کند). اگر h ارتفاع درخت و m تعداد جواب باشد (تعداد اعداد بازه $[a, b]$) یک الگوریتم کارا برای این کار از چه زمانی است؟

(۱) $O(m)$ (۲) $O(h)$ (۳) $O(mh)$ (۴) $O(m + h)$

۵- هزینه ساخت یک BST با n عنصر داده شده مرتب کدام است؟ (می‌دانیم که ورودی مرتب است)

(۱) در بدترین و میانگین حالت $O(n)$
 (۲) در بدترین $O(n \lg n)$ و میانگین $O(n)$
 (۳) در بدترین و میانگین $O(n \lg n)$
 (۴) در بدترین $O(n^2)$ و میانگین $O(n \lg n)$

۶- n کلید داده شده‌اند. کدام گزینه در هزینه ساخت یک BST با این n عنصر درست است؟
 (۱) همیشه می‌توان در $O(n \lg n)$ درخت را ساخت.

(۲) می‌توان در شرایطی با $O(n)$ ساخت.

(۳) در بدترین حالت $O(n^2)$

(۴) همیشه $O(n^2)$

۷- اگر $T(n)$ تعداد عمل دوران برای تبدیل یک درخت مورب چپ n نودی $(n = 2^k - 1)$ به یک درخت پر باشد آنگاه:

$$T(n) = \left\lfloor \frac{n}{2} \right\rfloor + 2T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) \quad (۲) \quad T(n) = \left\lfloor \frac{n}{2} \right\rfloor + 2T\left(\left\lceil \frac{n}{2} \right\rceil\right) \quad (۱)$$

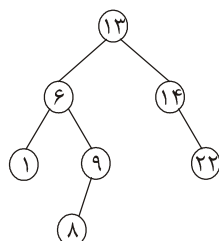
$$T(n) = 2T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) \quad (۴) \quad T(n) = 2T\left(\left\lceil \frac{n}{2} \right\rceil\right) \quad (۳)$$

۸- فرض کنید T_1 یک درخت قرمز سیاه با سیاه ارتفاع $(black \ height)$ برابر h باشد که کمینه تعداد عناصر داخلی را دارد و T_2 یک درخت قرمز سیاه با سیاه ارتفاع h است که تعداد عناصر داخلی آن بیشینه است. اگر $n_b(T)$ تعداد گره‌های داخلی سیاه درخت باشد آنگاه:

$$n_b(T_2) = n_b(T_1) \quad (۱) \quad n_b(T_2) = (n_b(T_1) + 1)^2 - 1 \quad (۲)$$

$$n_b(T_2) = 2n_b(T_1) - 1 \quad (۴) \quad n_b(T_2) = \frac{1}{3} \left[(n_b(T_1) + 1)^2 - 1 \right] \quad (۳)$$

۹- به چند حالت می‌توان گره‌های درخت مقابل را رنگ کرد که درخت قرمز سیاه شود؟



۰ (۱)

۱ (۲)

۲ (۳)

۳ (۴)

۱۰- مقادیر (چپ به راست) $AFEDCBGHI$ به یک درخت قرمز سیاه وارد می‌شوند. چند دوران انجام می‌شود؟

۶ (۴)

۵ (۳)

۴ (۲)

۳ (۱)

۱۱- حداکثر تعداد دوران در درج یک عنصر در یک درخت قرمز سیاه با n عنصر برابر است با:

۱ (۱) ۲ (۲) ۳ (۳) نزدیک به Lgn ۴ (۴) نزدیک به $\sqrt{2}Lgn$

۱۲- کمینه تعداد گره‌های یک درخت AVL به ارتفاع h کدام است؟ F_n جمله n ام فیبوناچی است.

۱ (۱) $F_{n+2} - 1$ (۲) $F_{n+2} - n$ (۳) $F_{n+3} - 1$ (۴)

۱۳- با حداکثر چه تعداد دوران می‌توان مطمئن بود که هر دو درخت BST ، n رأسی به هم تبدیل می‌شوند؟

۲ (۲) $n + 1$

۱ (۱) $2n - 2$

۳ (۳) $nLgn$ ۴ (۴) برای برخی BST ها امکان‌پذیر نیست.

۱۴- کدام گزینه صحیح است؟

۱ (۱) k هرم که اندازه هر کدام n است داده شده‌اند. هرم‌ها به صورت درختی (با اشاره‌گر) داده

شده و به همین ساختار هم در خروجی مورد نیاز است. می‌توان در زمان $O(k \lg n)$ ، با

عناصر این هرم‌ها، یک هرم جدید به اندازه nk ساخت.

۲ (۲) در هر درخت دودویی همیشه یک گره وجود دارد که ارتفاع و عمق آن برابر باشد.

۳ (۳) در یک درخت دودویی با n گره تعداد زیر درخت‌ها $O\left(\frac{n(n-1)}{2}\right)$ است.

۴ (۴) حداکثر تعداد گره‌های یک درخت دودویی با b برگ $2b - 1$ است.

۱۵- در یک هرم بیشینه n تایی، Lgn مین بزرگ‌ترین عنصر را با چه مرتبه‌ای به صورت کارا می‌توان یافت؟

$$\begin{array}{ll} O(\lg^2 n) & (۲) \\ O(\lg n) & (۱) \\ O(n) & (۴) \\ O(\lg n \cdot \lg \lg n) & (۳) \end{array}$$

۱۶- برای یک آرایه n عضوی که به صورت هرم بیشینه ذخیره شده است، جمع ۱۰ بزرگ‌ترین عدد را با چه مرتبه‌ای به صورت کارا می‌توان حساب کرد؟

$$\begin{array}{llll} O(nLgn) & (۴) & O(n) & (۳) \\ O(1 \circ Lgn) & (۲) & O(1) & (۱) \end{array}$$

۱۷- اگر $T(n)$ تعداد هرم‌های کمینه متفاوتی باشد که با $n = 2^k - 1$ عنصر مجزا بتوان ساخت آنگاه:

$$\begin{array}{ll} T(n) = T^2\left(\left\lfloor \frac{n}{2} \right\rfloor\right) & (۱) \\ T(n) = \left\lfloor \frac{n-1}{2} \right\rfloor T^2\left(\left\lfloor \frac{n}{2} \right\rfloor\right) & (۲) \\ T(n) = \left\lfloor \frac{n-1}{2} \right\rfloor T^2\left(\left\lfloor \frac{n}{2} \right\rfloor\right) & (۴) \\ T(n) = \left\lfloor \frac{n-1}{2} \right\rfloor T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) & (۳) \end{array}$$

۱۸- با n عنصر چند درخت دودویی متوازن با ارتفاع $h = \lfloor Lgn \rfloor$ می‌توان ساخت؟ (در این تست متوازن یعنی همه سطوح بجز احتمالاً سطح آخر پر هستند)

$$\begin{array}{llll} \binom{n}{n-2^{h-1}} & (۴) & \binom{n}{n-2^h} & (۳) \\ \binom{2^h}{n-2^h+1} & (۲) & 1 & (۱) \end{array}$$

۱۹- رشته‌های ۱۱، ۱۰، ۰۱۱، ۰۰۰، ۰۰، ۰۱، ۱۰۱ و ۱۰۰ را در یک ترای ($Trie$) دودویی که در ابتدا تهی است درج کرده‌ایم. تعداد گره‌های این درخت چند تاست؟

$$\begin{array}{llll} 8 & (۱) & 10 & (۲) \\ 11 & (۳) & 12 & (۴) \end{array}$$

۲۰- فرض کنید T یک درخت ۲ کامل (تعداد فرزندان هر گره صفر یا ۲ است) هم‌چنین فرض کنید: $n(T)$ تعداد گره‌های غیربرگ T ؛ $h(T)$ ارتفاع T ؛ T_L و T_R به ترتیب زیردرخت‌های راست و چپ T باشند. تابع F به شکل زیر تعریف شده:

$$F(T) = \begin{cases} 0 & \text{if } T \text{ is a leaf} \\ F(T_L) + F(T_R) + \min\{h(T_L), h(T_R)\} & \text{otherwise} \end{cases}$$

آنگاه، $F(T)$ برابر است با:

$$(۱) \quad n(T) + h(T) \quad (۲) \quad n(T) + h(T) + ۱$$

$$(۳) \quad n(T) - h(T) \quad (۴) \quad n(T) - h(T) - ۱$$

۲۱- فرض کنید یک جدول درهم‌سازی با اندازه m حاوی تنها یک عنصر با کلید k است. همچنین فرض کنید با استفاده از یک تابع درهم‌سازی یکنوا کلیدهای غیر از k را r بار جستجو می‌کنیم. با چه احتمالی دست کم یکی از این جستجوها به همان درایه‌ای می‌رود که تنها عنصر آرایه یعنی k در آن قرار دارد؟

$$(۱) \quad \frac{r}{m} \quad (۲) \quad 1 - \left(\frac{1}{m}\right)^r \quad (۳) \quad 1 - \left(1 - \frac{1}{m}\right)^r \quad (۴) \quad \left(1 - \frac{1}{m}\right)^r$$

۲۲- اعداد $۱^۲, ۲^۲, ۳^۲, \dots, ۱۰۰^۲$ را با تابع $h(x) = x \bmod ۷$ در جدول درهم‌سازی $H[۰ \dots ۶]$ به صورت زنجیره‌ای وارد می‌کنیم. تعداد عناصر موجود در $H[۴]$ برابر است با:

$$(۱) \quad ۱۴ \quad (۲) \quad ۲۳ \quad (۳) \quad ۲۸ \quad (۴) \quad ۲۹$$

۲۳- چند تا درخت دودویی ۱۵ نودی وجود دارد که در سطح i ام آن تعداد i نود وجود داشته باشد (سطح اول یک نود، سطح دوم ۲ نود و ...)

$$(۱) \quad ۲۵۵۰ \quad (۲) \quad ۳۳۶۰ \quad (۳) \quad ۷۹۲ \quad (۴) \quad ۱۵۷۰$$

۲۴- چند درخت دودویی ۹ نودی بدون نود تک فرزندی وجود دارد؟

$$(۱) \quad ۱۴ \quad (۲) \quad ۴۲ \quad (۳) \quad ۱۳۲ \quad (۴) \quad ۵۴$$

۲۵- یک لیست پیوندی یک طرفه بزرگ داریم. می‌خواهیم بررسی کنیم آیا آخرین اشاره‌گر این لیست تهی است یا اینکه به یکی دیگر از عناصر لیست اشاره می‌کند. دقت کنید فقط یکی از این دو حالت ممکن است و ما از طول لیست اطلاع نداریم. تعداد عناصر لیست n است و ما فقط آدرس عنصر اول لیست را داریم. الگوریتمی با زمان و حافظه اضافی $O(۱)$ برای تشخیص این امر وجود دارد؟

$$(۱) \quad O(Lgn) \quad (۲) \quad O(n) \quad (۳) \quad O(nLgn) \quad (۴) \quad O(n^۲)$$

۲۶- تعداد کل مقایسه‌های انجام شده برای مرتب‌سازی عناصر $(۴, ۸, ۲۰, ۱۷, ۷, ۲۵, ۲, ۱۳, ۵, <$ (چپ به راست) توسط مرتب‌سازی درجی دودویی چند تاست؟ (برای یافتن محل درج از

جستجوی دودویی کمک می‌گیریم و اندیس وسط $[l \dots u]$ را $\left\lfloor \frac{l+u}{۲} \right\rfloor$ تعریف می‌کنیم)

$$(۱) \quad ۱۷ \quad (۲) \quad ۱۸ \quad (۳) \quad ۲۰ \quad (۴) \quad ۲۱$$

۲۷- n آرایه هر کدام با ۳ عنصر و مقادیر بین ۱ تا ۲۰ داده شده‌اند. الگوریتمی کارا که تشخیص دهد آیا دو آرایه از این n آرایه به طور کامل یکسان هستند یا خیر از چه مرتبه‌ای است؟

$$(۱) O(n) \quad (۲) O(n \lg n) \quad (۳) O(n^2) \quad (۴) O(n^3)$$

۲۸- چند تا از الگوریتم‌های زیر یک آرایه با عناصر یکسان را در $O(n)$ مرتب می‌کند؟
مرتب‌سازی هرمی، مرتب‌سازی سریع، مرتب‌سازی درجی، مرتب‌سازی ادغامی، مرتب‌سازی حبابی (بهبوده)

$$(۱) ۱ \quad (۲) ۲ \quad (۳) ۳ \quad (۴) ۴$$

۲۹- می‌خواهیم ۴ گلوله با وزن‌های مختلف را با یک ترازوی دو کفه‌ای بدون وزنه مرتب کنیم. حداقل چند توزین در بدترین حالت لازم است؟

$$(۱) ۳ \quad (۲) ۴ \quad (۳) ۵ \quad (۴) ۶$$

۳۰- n عدد طبیعی کوچک‌تر از k داده شده‌اند. با انجام پیش‌پردازش از زمان $O(n+k)$ می‌خواهیم کاری کنیم که بتوان هر مسئله زیر را در زمان کوتاهی انجام داد: «چه تعداد از عناصر در بازه $[a, b]$ قرار دارند؟» زمان پاسخ‌گویی به این مسئله‌ها

$$(۱) O(1) \quad (۲) O(\lg k) \quad (۳) O(\lg n) \quad (۴) O(k)$$

۳۱- می‌دانیم در آرایه n عضوی A ، فقط عناصر مجاور نسبت به هم نامرتب هستند. یعنی اگر $j < i$ و $A[i] > A[j]$ آنگاه $j = i \pm 1$. کدام روش مرتب‌سازی بهتر است؟

$$(۱) \text{درجی} \quad (۲) \text{سریع} \quad (۳) \text{هرمی} \quad (۴) \text{ادغامی}$$

۳۲- می‌خواهیم در یک آرایه به طول n ، عنصری که k بار تکرار شده بیابیم. K در ورودی داده شده و مستقل از n است. زمان سریع‌ترین الگوریتم:

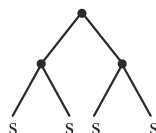
$$(۱) O(n+k) \quad (۲) O(nk) \quad (۳) O(n \lg n) \quad (۴) O(n \lg k)$$

۳۳- یک S -Term ترتیبی از تعدادی نویسه S و پرانتز است که به صورت زیر تعریف می‌شود:
 S یک S -term است.

اگر M و N دو S -term باشند (MN) هم S -term است.

برای نمونه $((ss)(ss))$ یک S -term است که به شکل درخت مقابل قابل نمایش است:

تعداد S -term با $n+1$ عدد s کدام است؟



$$(۲) \frac{1}{n} \binom{2n-2}{n-1}$$

$$(۱) \frac{1}{n+1} \binom{2n}{n}$$

$$(۴) \text{هیچ کدام}$$

$$(۳) \frac{1}{n} \binom{n}{\frac{n}{2}}$$

۳۴- شرط لازم و کافی برای اینکه یک دنباله از کلیدها یک دنباله جستجوی درست در یک درخت با کلیدهای متمایز باشد چیست؟

- (۱) برای هر عنصر این دنباله با کلید x ، همه عناصر بعدی یا از x کمترند یا بیشتر.
- (۲) برای هر عنصر این دنباله با کلید x ، همه عناصر قبلی یا از x کمترند یا بیشتر
- (۳) عنصر قبلی هر عنصر از آن کمتر و عنصر بعدی آن از آن بیشتر است.
- (۴) هیچ کدام

۳۵- فرض کنید لیست پیوندی دو طرفه را با کمک نودهایی که فقط یک لینک دارند ساخته‌ایم. در لینک یک نود، آدرس نود قبل و بعد از آن را xor کرده و قرار داده‌ایم. الگوریتم زیر چه عملی انجام می‌دهد؟

$$Link(p) = Link(p) \oplus q \oplus c$$

$$Link(q) = Link(q) \oplus p \oplus c$$

$$Link(c) = p \oplus q$$

$$q = c$$

(۱) لیست دو طرفه را نمی‌توان با نودهای حاوی یک لینک ساخت.

(۲) نود p را بین نودهای q و c درج می‌کند.

(۳) نود c را بین نودهای p و q درج می‌کند.

(۴) نود q را بین نودهای p و c درج می‌کند.

۳۶- روشی برای جستجو در آرایه موسوم به جستجوی درون‌یابی وجود دارد که در آرایه‌های مرتب قابل استفاده است ولی برخلاف روش دودویی، اندیس mid را وسط آرایه انتخاب نمی‌کند بلکه mid را از رابطه زیر محاسبه می‌کند:

$$mid = L + \left\lfloor \frac{x - A[L]}{A[u] - A[L]} (u - L) \right\rfloor$$

فرض کرده‌ایم آرایه $A[L \cdot u]$ صعودی مرتب است و x را جستجو می‌کنیم. در آرایه زیر برای

جستجوی $x = 25$ چند مقایسه انجام می‌شود؟

	۱	۲	۳	۴	۵	۶	۷	۸	۹	۱۰
A	۴	۲۰	۲۵	۲۷	۴۰	۵۰	۶۰	۷۰	۸۰	۹۷

۱ (۴) ۲ (۳) ۳ (۲) ۴ (۱)

۳۷- یک آرایه A با طول بی‌نهایت فرض کنید که n خانه اول آن شامل n عدد صحیح مرتب است و سایر خانه‌ها مقدار ∞ دارند ولی مقدار n مشخص نیست. الگوریتمی که مشخص کند عدد x در آرایه هست از چه مرتبه‌ای است؟

$$O(\lg^2 n) \quad (۴) \quad O(\sqrt{n}) \quad (۳) \quad O(n) \quad (۲) \quad O(\lg n) \quad (۱)$$

۳۸- با توجه به تابع مقابل، $A(3, 4)$ کدام است؟

$$A(i, j) = \begin{cases} 2^j & , i = 0 \\ 2 & , j = 1 \\ A(i-1, A(i, j-1)) & , \text{else} \end{cases}$$

(۱) ۱۶ بار ۲ به توان ۲ به توان ۲ و ... (۲) ۲۵۶ بار ۲ به توان ۲ به توان ۲ و ...

(۳) ۶۵۵۳۶ بار ۲ به توان ۲ به توان ۲ و ... (۴) ۳۲ بار ۲ به توان ۲ به توان ۲ و ...

۳۹- با توجه به تابع مقابل $A(2, j)$ کدام است؟

$$A(i, j) = \begin{cases} 2^j & , i = 0 \\ 2 & , j = 1 \\ A(i-1, A(i, j-1)) & , \text{else} \end{cases}$$

(۱) 2^j (۲) 2^{2^j} (۳) 2^{2^j} (۴) 2^j

۴۰- درخت دودویی پر به این صورت تعریف می‌شود: درخت دودویی پر به ارتفاع صفر، شامل یک نود است که ریشه است. درخت دودویی پر به ارتفاع $h+1$ شامل دو درخت دودویی پر به ارتفاع h است که ریشه‌هایشان به یک ریشه جدید متصل شده است. فرض کنید T یک درخت دودویی پر به ارتفاع h است. مجموع ارتفاع همه نودهای درخت T برابر است با:

(۱) $2^h + h - 1$ (۲) $2^{h+1} - h - 2$ (۳) $2^{h+1} - h + 1$ (۴) $2^h - h + 1$

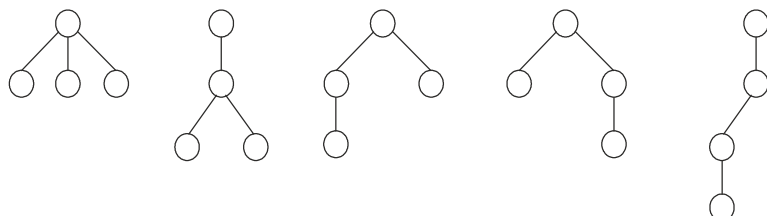
۴۱- پیمایش‌های *preorder* و *postorder* یک درخت دودویی که نود تک فرزندی ندارد، به شکل مقابل است. در پیمایش *inorder* نود هشتم کدام است؟

pre : abdehiefgjk

post : dhiebfjkgea

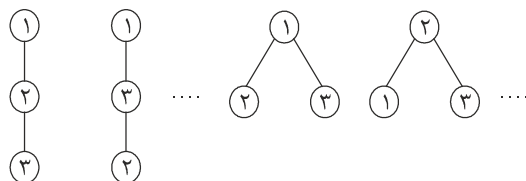
(۱) j (۲) a (۳) f (۴) c

۴۲- چند تا درخت عمومی ریشه‌دار مرتب (*general rooted ordered*) با n نود وجود دارد. مثلاً با ۴ نود، جواب ۵ است.



$$\begin{aligned} (2) & (n-1)! - 1 \\ (4) & n^{n-2} - (n-1)^2 - 2 \\ (3) & \frac{1}{n} \binom{2n-2}{n-1} 2^{n-1} - n + 1 \end{aligned}$$

۴۳- چند تا درخت ریشه‌دار مرتب برچسب‌دار (*ordered rooted with lable*) با n نود وجود دارد، مثلاً با ۳ نود جواب ۱۲ است که تعدادی از آنها عبارتند از:



$$(1) \quad \frac{n!}{n+1} \binom{2n}{n}$$

$$(2) \quad \frac{(2n-2)!}{(n-1)!}$$

$$(3) \quad \frac{(n+1)!}{(n-1)!}$$

$$(4) \quad 2^{n-1} \times n$$

۴۴- چند تا درخت ریشه‌دار غیرمرتب برچسب‌دار (*unordered rooted with lable*) با n نود وجود دارد؟ مثلاً با ۳ نود جواب ۹ است که عبارتند از:

$$(4) \quad 2^n + 1$$

$$(3) \quad n^{n-1}$$

$$(2) \quad n! + n$$

$$(1) \quad \frac{n!}{n+1} \binom{2n}{n}$$

۴۵- فرض کنید پیمایش پس ترتیب درخت عمومی T را در آرایه $post[1..n]$ که n تعداد نودهاست ذخیره کرده‌ایم. فرض کنید $d(x)$ تعداد عناصر زیر درخت نود x است (به جز خود x)، آنگاه عناصر موجود در زیر درخت x (با خود x) در چه اندیسی از آرایه $post$ هستند؟ (راست به چپ)

$$(2) \quad post[x] \dots post[x] - d(x)$$

$$(1) \quad d(x) \dots post[x] + d(x)$$

$$(4) \quad post[x] - d[x] \dots post[x] + d(x)$$

$$(3) \quad post[x] \dots post[x] + d(x)$$

[پاسخ کلیدی تست‌های تکمیلی]

(۱) گزینه (۱)	(۱) گزینه (۱)
(۲) گزینه (۴)	(۲) گزینه (۴)
(۳) گزینه (۳)	(۳) گزینه (۳)
(۴) گزینه (۴)	(۴) گزینه (۴)
(۵) گزینه (۱)	(۵) گزینه (۱)
(۶) گزینه (۱)	(۶) گزینه (۱)
(۷) گزینه (۲)	(۷) گزینه (۲)
(۸) گزینه (۳)	(۸) گزینه (۳)
(۹) گزینه (۲)	(۹) گزینه (۲)
(۱۰) گزینه (۳)	(۱۰) گزینه (۳)
(۱۱) گزینه (۲)	(۱۱) گزینه (۲)
(۱۲) گزینه (۴)	(۱۲) گزینه (۴)
(۱۳) گزینه (۱)	(۱۳) گزینه (۱)
(۱۴) گزینه (۱)	(۱۴) گزینه (۱)
(۱۵) گزینه (۳)	(۱۵) گزینه (۳)
(۱۶) گزینه (۱)	(۱۶) گزینه (۱)
(۱۷) گزینه (۴)	(۱۷) گزینه (۴)
(۱۸) گزینه (۲)	(۱۸) گزینه (۲)
(۱۹) گزینه (۴)	(۱۹) گزینه (۴)
(۲۰) گزینه (۳)	(۲۰) گزینه (۳)
(۲۱) گزینه (۳)	(۲۱) گزینه (۳)
(۲۲) گزینه (۴)	(۲۲) گزینه (۴)
(۲۳) گزینه (۲)	(۲۳) گزینه (۲)
(۲۴) گزینه (۱)	
(۲۵) گزینه (۲)	
(۲۶) گزینه (۲)	
(۲۷) گزینه (۱)	
(۲۸) گزینه (۲)	
(۲۹) گزینه (۳)	
(۳۰) گزینه (۱)	
(۳۱) گزینه (۱)	
(۳۲) گزینه (۳)	
(۳۳) گزینه (۱)	
(۳۴) گزینه (۱)	
(۳۵) گزینه (۳)	
(۳۶) گزینه (۴)	
(۳۷) گزینه (۱)	
(۳۸) گزینه (۳)	
(۳۹) گزینه (۲)	
(۴۰) گزینه (۲)	
(۴۱) گزینه (۴)	
(۴۲) گزینه (۳)	
(۴۳) گزینه (۲)	
(۴۴) گزینه (۳)	
(۴۵) گزینه (۲)	

سؤال‌های آزمون سراسری ۹۴-۹۱

ضمیمه ۲

[سؤال‌های مهندسی کامپیوتر سال ۹۱]

۱- در رابطه‌ی بازگشتی $T(n) = 9T(n/3) + f(n)$ ، به ازای چند تا از عبارت‌های $g(n)$ زیر، دست کم یک تابع برای $f(n)$ وجود دارد به طوری که $T(n) = \theta(g(n))$ ؟

$$n \lg n \parallel n^2 \parallel n^2 \lg^2 n \parallel n^3$$

۴ (۴)

۳ (۳)

۲ (۲)

۱ (۱)

۲- فرض کنید که $LEAFC(T)$ تعداد برگ‌های درخت دودویی T و اگر T تهی باشد صفر را برمی‌گرداند. همچنین، فرض کنید تابع $IsLEAF(T)$ اگر T برگ باشد مقدار ۱ و گرنه مقدار صفر را برمی‌گرداند. کدام یک از رابطه‌های بازگشتی زیر درست است؟

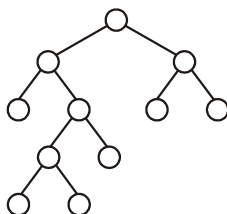
$$LEAFC(T) = LEAFC(Left | T |) + LEAFC(Right | T |) \quad (۱)$$

$$LEAFC(T) = LEAFC(left[T]) + LEAFC(right[T]) + ۱ \quad (۲)$$

$$LEAFC(T) = LEAFC(left[T]) + LEAFC(right[T]) + IsLEAF(T) + ۱ \quad (۳)$$

$$LEAFC(T) = LEAFC(left[T]) + LEAFC(right[T]) + IsLEAF(T) \quad (۴)$$

۳- به چند طریق می‌توان اعداد ۱ تا ۱۱ را در گره‌های درخت روبه‌رو برچسب‌گذاری کرد تا عدد هر گره از اعداد فرزندان آن بزرگ‌تر باشد؟ دقت کنید که از هر ۱۱ عدد باید استفاده شود و تکرار مجاز نیست.



۹۶ (۱)

۲۰۴۸ (۲)

۱۱۵۲۰ (۳)

۳۶۲۸۸۰ (۴)

۴- تابع روبه‌رو را در نظر بگیرید. فرض کنید که $T(n)$ نشان دهنده تعداد عملیات $++$ (هر سه) در تابع فوق باشد. اگر تابع فوق با پارامتر n فراخوانده شود، کدام رابطه‌ی بازگشتی زیر درست است؟

```
int test (int n) {
    int i, j, count = 0;
    for (i = 0; i < n; i++)
        for (j = 0; j < i; j++)
            count++;
    return count;
}
```

$$\begin{array}{ll} T(n) = nT(n-1) + n - 1 & (۲) \quad T(n) = T(n-1) + 2n + 1 & (۱) \\ T(n) = nT(n-1) + n + 1 & (۴) \quad T(n) = T(n-1) + 2n - 1 & (۳) \end{array}$$

۵- داده ساختار D شامل مجموعه‌های S_0, S_1, S_p و .. است به طوری که مجموعه‌ی S_i یا صفر یا 2^i عنصر دارد. درج یک عنصر x در D به این صورت است: ابتدا کوچک‌ترین i که S_i صفر عنصر دارد را پیدا می‌کنیم. سپس تمام عناصر موجود در مجموعه‌های $S_0, S_1, \dots, S_{i-1}$ را به S_i منتقل و در انتها x را در S_i درج می‌کنیم. دقت کنید که پس از انتقال مجموعه‌های $S_0, S_1, \dots, S_{i-1}$ تا S_i خالی می‌شوند و هزینه‌ی انتقال برابر مجموع تعداد این عناصر است. اگر در ابتدا مجموعه‌ها خالی باشند، مجموع هزینه‌ی درج n عنصر در D از چه مرتبه‌ای است؟ بهترین جواب را انتخاب کنید.

$$\begin{array}{llll} O(n) & (۱) & O(n \lg n) & (۲) \\ O(n \lg^2 n) & (۳) & O(n^2) & (۴) \end{array}$$

۶- یک هرم بیشینه ($max\text{-}heap$) حاوی ۶۴ عنصر با کلیدهای ۱ تا ۶۴ است. بزرگ‌ترین عددی که می‌تواند در آخرین سطح این هرم قرار گیرد، کدام یک از اعداد زیر می‌تواند باشد؟

$$\begin{array}{llll} ۵۶ & (۱) & ۵۷ & (۲) \\ ۵۸ & (۳) & ۵۹ & (۴) \end{array}$$

[سؤال‌های سراسری IT سال ۹۱]

۷- فرض کنید در یک جدول درهم‌سازی به اندازه ۸۰ از روش آدرس‌دهی باز به صورت خطی استفاده شده است. در ابتدا جدول خالی بوده و فقط عملیات درج با ترتیبی نامشخص و جستجو انجام شده است. درایه‌های ۴۵ الی ۵۶ در شکل زیر آمده است. شماره بالای درایه، اندیس درایه و حروف داخل درایه، کلیدهای درج شده و اعداد پایین درایه مقدار تابع درهم‌سازی به ازای آن کلید است. درایه خالی یعنی کلیدی در آنجا درج نشده است. حال اگر به همان ترتیبی که کلیدها در جدول درج شده‌اند، یکبار دیگر از ابتدا در جدول درهم‌سازی خالی درج بشوند با این تفاوت که کلید e در دنباله کلیدهای درج شده نباشد، در درایه ۵۰ چه کلیدی قرار می‌گیرد؟

۴۵	۴۶	۴۷	۴۸	۴۹	۵۰	۵۱	۵۲	۵۳	۵۴	۵۵	۵۶	
...	a	b	c	d	e	f	g	h	i	j	...	
	۴۶	۴۶	۴۶	۴۷	۴۶	۵۱	۴۷	۴۶	۴۸	۴۹		

(۱) i (۲) f (۳) g (۴) h

۸- مرتبه زمانی تابع بازگشتی زیر چیست؟

$$T(n) = 3T\left(\left\lfloor \frac{n}{3} \right\rfloor\right) + n^2$$

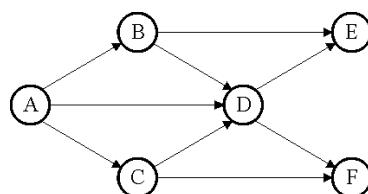
$$\Omega(n^2 \log n) \quad (۴)$$

$$\Omega(n \log n) \quad (۳)$$

$$\Omega(n^2) \quad (۲)$$

$$\Omega(\log n) \quad (۱)$$

۹- گراف شکل زیر را در نظر بگیرید.



کدام گزینه پیمایش درست جستجوی سطح اول (Breadth - First - Search) گراف به کمک ساختمان داده صف نیست؟

ADBCFE (۴)

ADBCEF (۳)

ABCDEF (۲)

ABCD FE (۱)

۱۰- تابع زیر تمام داده‌های بزرگتر از i یا مساوی i و کوچکتر از j را در یک درخت جستجوی دودویی به صورت صعودی چاپ می‌کند. کدام گزینه محل‌های A, B, C و D را بدرستی مقداردهی می‌کند؟

```
Void BST - Find - Range (BSTNode *b, int i, int j)
{
    if (b == Null) return;
    int v = b -> key;
    if (v < i) BST - Find - Range(A, i, j);
    if (v > j) BST - Find - Range(B, i, k);
    else
    {
        cout << v;
        BST - Find - Range(C, i, j);
        BST - Find - Range(D, i, j);
    }
}
```

۱) $A = b \rightarrow \text{Leftchild}$ $B = b \rightarrow \text{Rightchild}$ $C = b \rightarrow \text{Right child}$, $D = b \rightarrow \text{Leftchild}$

۲) $A = b \rightarrow \text{Leftchild}$ $B = b \rightarrow \text{Rightchild}$ $C = b \rightarrow \text{Leftchild}$, $D = b \rightarrow \text{Rightchild}$

۳) $A = b \rightarrow \text{Rightchild}$ $B = b \rightarrow \text{Leftchild}$ $C = b \rightarrow \text{Rightchild}$, $D = b \rightarrow \text{Leftchild}$

۴) $A = b \rightarrow \text{Rightchild}$ $B = b \rightarrow \text{Leftchild}$ $C = b \rightarrow \text{Leftchild}$, $D = b \rightarrow \text{Rightchild}$

۱۱- کدام گزینه درست نیست؟

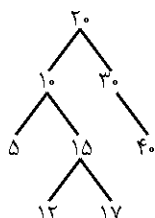
۱) هزینه زمانی درج یک عنصر به درخت AVL حاوی n عنصر، در بدترین حالت $O(\log n)$ است.

۲) هزینه زمانی هر یک از چرخش‌ها در درخت AVL حاوی n عنصر، در بدترین حالت $O(l)$ است.

۳) هزینه زمانی درج یک عنصر به جدول درهم‌سازی با آدرس‌دهی باز ($Open Addressing$) حاوی n عنصر، در بدترین حالت $O(n)$ است.

۴) هزینه زمانی پیمایش اول عمق ($Depth First Search$) گراف $G = (V, E)$ که به صورت ماتریس مجاورت ($Adjacency Matrix$) بیان شده است، در بدترین حالت $O(|V| + |E|)$ است.

۱۲- اگر عنصر با کلید ۱۴ به درخت AVL زیر اضافه شود، پیمایش پیش ترتیب ($preorder$)



درخت حاصل کدام گزینه است؟

- (۱) ۴۰ و ۲۰ و ۳۰ و ۱۵ و ۱۲ و ۵ و ۱۴ و ۱۷
- (۲) ۴۰ و ۲۰ و ۳۰ و ۱۵ و ۱۲ و ۵ و ۱۴ و ۱۷
- (۳) ۴۰ و ۳۰ و ۱۷ و ۱۴ و ۱۵ و ۵ و ۱۰ و ۱۲ و ۲۰
- (۴) ۴۰ و ۳۰ و ۱۵ و ۱۷ و ۱۲ و ۵ و ۱۰ و ۱۴ و ۲۰

[سؤال‌های مهندسی کامپیوتر سال ۹۲]

۱۳- برای ساخت یک صف Q از دو پشته‌ی S_1 و S_2 استفاده می‌کنیم. برای درج x در انتهای Q ، عمل $Push(S_1, x)$ را انجام می‌دهیم. برای حذف یک عنصر از ابتدای Q ، اگر S_2 خالی نباشد، عمل $Pop(S_2)$ را انجام می‌دهیم. در غیر این صورت، تمامی عناصر S_1 را به ترتیب Pop کرده و در S_2 $Push$ می‌کنیم. اکنون عمل Pop بر روی S_2 عنصر ابتدایی Q را برمی‌گرداند.

اگر بر روی Q که در ابتدا خالی است، ۱۰۰ عمل صورت گیرد (درج در انتها، حذف از ابتدا یا هر ترتیب دل‌خواهی از آن‌ها) حداکثر هزینه چه مقدار خواهد بود؟ فرض کنید هر $Push$ و هر Pop بر روی هر یک از این دو پشته ۱ واحد هزینه دارد.

- (۱) ۱۵۰
- (۲) ۱۵۱
- (۳) ۱۹۹
- (۴) ۲۰۰

۱۴- کدام یک از گزینه‌های زیر حل رابطه‌ی بازگشتی زیر $(T(n, k))$ است؟

$$T(n, k) = T(n/2, k) + T(n, k/4) + kn$$

$$(۱) \Theta(nk) \quad (۲) \Theta(n \log_4(nk))$$

$$(۳) \Theta(n + \log_4(nk)) \quad (۴) \Theta(nk \log_4(nk))$$

۱۵- برای n عنصر با کلیدهای مجزا کدام یک از داده ساختارهای زیر را می‌توان ایجاد کرد؟

(الف) درج و حذف از مرتبه‌ی $O(\lg n)$ و یافتن تعداد عناصر بزرگ‌تر از a و کوچک‌تر از b (به ازای هر a و b) در $O(\lg n)$.

(ب) ساخت آن از مرتبه‌ی $O(n)$ ، درج یک عنصر در $O(\lg n)$ و حذف کوچک‌ترین عنصر در $O(\lg n)$ (ای کوچک)

(۱) الف: می‌توان، ب: می‌توان

(۲) الف: می‌توان، ب: نمی‌توان

(۳) الف: نمی‌توان، ب: می‌توان

(۴) الف: نمی‌توان، ب: نمی‌توان

۱۶- n عدد صحیح غیر تکراری در بازه‌ی $[0, n^k - 1]$ را با چه مرتبه‌ی زمانی می‌توان مرتب کرد؟ بهترین گزینه را انتخاب کنید. منظور از هزینه تعداد دفعات خواندن و نوشتن این اعداد از (در) حافظه است؟

$$\Theta(n) \quad (۱) \quad \Theta(n+k) \quad (۲) \quad \Theta(nk) \quad (۳) \quad \Theta(n \lg k) \quad (۴)$$

۱۷- یک جدول درهم‌سازی پویا با روش آدرس‌دهی باز (*Open Hashing*) پیاده‌سازی شده است. اگر هنگام درج یک عنصر، $\frac{3}{4}$ اندازه‌ی جدول پر باشد، جدولی به اندازه‌ی ۲ برابر جدول فعلی ایجاد می‌شود، عناصر فعلی به این جدول منتقل، جدول قبلی حذف و سپس عنصر جدید در آن درج می‌شود. اگر با حذف یک عنصر، تعداد عناصر موجود در جدول از $\frac{3}{8}$ ام اندازه‌ی جدول کم‌تر شود، جدولی به اندازه‌ی نصف جدول فعلی ایجاد و همه‌ی عناصر جدول فعلی به آن منتقل شده و جدول قبلی حذف می‌شود. فرض کنید که هزینه‌ی درج و حذف مستقیم هر عنصر در جدول $O(۱)$ است. با فرض این که در حال حاضر جدول n عنصر دارد و قرار است n عمل درج یا حذف صورت گیرد، هزینه‌ی سرشکن شده‌ی هر کدام از این اعمال کدام است؟

$$\Theta(۱) \quad (۱) \quad \Theta(\lg n) \quad (۲) \quad \Theta(\sqrt{n}) \quad (۳) \quad \Theta(n) \quad (۴)$$

۱۸- الگوریتم زیر که عنصر کمینه‌ی آرایه‌ی n عضوی A را به دست می‌آورد در نظر بگیرید:

```

MINIMUM(A, n)
۱ min ← +∞
۲ for i ← ۱ to n
۳   do if min > A[i]
۴     then min ← A[i]
```

با فرض این که آرایه‌ی A به احتمال یکسان یکی از جای گشت‌های آن است، کدام یک از گزینه‌های زیر میانگین تعداد دفعاتی است که سطر ۴ الگوریتم بالا اجرا می‌شود؟ بهترین گزینه را انتخاب کنید.

$$\Theta(\lg n) \quad (۱) \quad \Theta(n) \quad (۲) \quad \Theta(\sqrt{n}) \quad (۳) \quad \Theta(\lg^2 n) \quad (۴)$$

[سؤال‌های سراسری IT سال ۹۲]

۱۹- داده ساختار D یک درخت دودویی کامل است که اعداد دل‌خواه $a_1, \dots, a_n$ را در برگ‌های خود ذخیره می‌کند. (درخت دودویی کامل درختی است که اگر برگ‌های سطح آخر آن را برداریم، درخت باقی‌مانده پر - یعنی کامل و کاملاً متوازن، خواهد بود و برگ‌های سطح آخر آن به صورت متوالی از چپ به راست قرار دارند. هرم یا *heap* چنین ساختاری دارد.) هم‌چنین می‌دانیم که هر گره داخلی فقط مقدار کوچک‌ترین عدد دو فرزندش را ذخیره می‌کند.

چند تا از اعمال زیر را می‌توان در این داده ساختار در مرتبه‌ی $O(\lg n)$ انجام داد؟

- حذف یک عنصر دل‌خواه از D
- درج یک عنصر دل‌خواه در D
- کاهش مقدار یک عنصر موجود در D

(۱) ۰ (۲) ۱ (۳) ۲ (۴) ۳

۲۰- چه تعداد از الگوریتم‌های زیر هر آرایه‌ی متشکل از n عدد را به صورت صعودی مرتب می‌کند؟

- ابتدا $\frac{2n}{3}$ عنصر اول را به صورت صعودی مرتب کن، سپس $\frac{2n}{3}$ عنصر آخر را به صورت صعودی مرتب کن، در پایان $\frac{n}{3}$ عنصر اول را دوباره به صورت صعودی مرتب کن.
- ابتدا $\frac{n}{4}$ عنصر اول را به صورت صعودی مرتب کن، سپس عناصر بین $\frac{n}{4}$ و $\frac{3n}{4}$ را به صورت صعودی مرتب کن، در پایان $\frac{n}{4}$ عنصر اول را دوباره به صورت صعودی مرتب کن.
- ابتدا عناصر بین $\frac{n}{3}$ و $\frac{2n}{3}$ را به صورت صعودی مرتب کن، سپس $\frac{n}{3}$ عنصر اول را به صورت صعودی مرتب کن، در پایان $\frac{n}{3}$ عنصر آخر را به صورت صعودی مرتب کن.
- ابتدا $\frac{2n}{3}$ عنصر اول را به صورت صعودی مرتب کن، سپس $\frac{2n}{3}$ عنصر آخر را به صورت صعودی مرتب کن، در پایان $\frac{2n}{3}$ عنصر اول را دوباره به صورت صعودی مرتب کن.

(۱) ۱ (۲) ۲ (۳) ۳ (۴) ۴

۲۱- دو آرایه‌ی مرتب A و B به ترتیب با طول‌های n و m داده شده‌اند. می‌خواهیم میانه‌ی $A \cup B$ را به دست آوریم. برای این کار الگوریتم زیر را پیشنهاد می‌کنیم:

$$(۱) \text{ فرض کنید } k = \left\lfloor \frac{n}{2} \right\rfloor$$

(۲) عنصر میانه‌ی A به نام x را به دست آور

(۳) با یک جست‌وجوی دودویی اندیسی از B به نام i را به دست آور که همه‌ی عناصر $B[1..i]$ از x کم‌تر و بقیه بیش‌تر با مساوی x می‌باشند.

$$(۴) \text{ فرض کنید } k = \left\lfloor \frac{(n+m)}{2} \right\rfloor$$

(۵) اگر $i < \left\lfloor \frac{n}{2} \right\rfloor + 1$ به طور بازگشتی r امین عنصر را بین $A\left[1.. \left\lfloor \frac{n}{2} \right\rfloor\right]$ و $B[1..i]$ به دست آورد.

(۶) وگرنه به طور بازگشتی $k-r$ امین عنصر را بین $A[k+1..n]$ و $B[i+1..m]$ به دست آورد. کدام یک از رابطه‌های بازگشتی زیر، زمان اجرای این الگوریتم را نشان می‌دهد؟

$$(۱) T(n, m) = T\left(\frac{n}{2}, m\right) + \lg m + O(1)$$

$$(۲) T(n, m) = T\left(n, \frac{m}{2}\right) + \lg m + O(1)$$

$$(۳) T(n, m) = T\left(\frac{n}{2}, \frac{m}{2}\right) + \lg m + O(1)$$

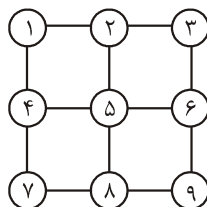
$$(۴) T(n, m) = T\left(\frac{n}{2}, \frac{m}{2}\right) + \lg(n+m) + O(1)$$

۲۲- کدام یک از دنباله‌های زیر می‌تواند پیمایش پیش ترتیب یک درخت دودویی جست‌وجو با ۱۲ گره باشد؟

$$(۱) (12, 8, 5, 10, 9, 20, 14, 13, 18, 15, 16, 19) \quad (۲) (12, 8, 5, 10, 9, 20, 14, 15, 18, 13, 16, 19)$$

$$(۳) (20, 14, 13, 12, 8, 5, 9, 10, 18, 19, 15, 16) \quad (۴) (20, 14, 13, 12, 8, 5, 9, 18, 10, 19, 15, 16)$$

۲۳- کدام یک از دنباله‌های زیر نمی‌تواند بیانگر ترتیب ملاقات گره‌های گراف شکل مقابل توسط الگوریتم جست‌وجوی عمق اول (DFS) باشد.



(۱) از چپ به راست: ۱, ۴, ۷, ۸, ۹, ۶, ۵, ۲, ۳

(۲) از چپ به راست: ۱, ۲, ۳, ۶, ۹, ۸, ۷, ۴, ۵

(۳) از چپ به راست: ۱, ۲, ۴, ۳, ۵, ۷, ۶, ۸, ۹

(۴) از چپ به راست: ۱, ۲, ۳, ۶, ۵, ۴, ۷, ۸, ۹

[سؤال‌های مهندسی کامپیوتر سال ۹۳]

۲۴- در یک لیست خطی یک سویه با عناصر $(x_1, x_2, \dots, x_n)$ اگر x_1 عنصر اول لیست باشد، جست‌وجو برای عنصر x_i برابر i واحد هزینه خواهد داشت. (*Move to Front*) MTF روشی برای کاهش میانگین زمان جست‌وجو است. در این روش هر عنصری که مورد جست‌وجو قرار می‌گیرد به ابتدای لیست منتقل می‌شود. لیستی با ۱۰۰ عنصر $(x_1, x_2, \dots, x_{100})$ را در نظر بگیرید. ابتدا به ترتیب عناصر زوج x_2 تا x_{100} و سپس عناصر فرد x_1 تا x_{99} را در A جست‌وجو می‌کنیم. اگر جست‌وجوی لیست به روش‌های (الف) عادی و (ب) MTF مدیریت شود، اختلاف میانگین هزینه‌ها بین این دو روش چقدر است؟

- (۱) ۰ (۲) ۱۲۷۵ (۳) ۳۳۷۵ (۴) ۲۵۵۰

۲۵- مخزنی با n لیتر آب داریم. هر بار $1/k$ از آب مخزن را برمی‌داریم. حداقل چند بار باید این کار را تکرار کنیم تا میزان آب به یک لیتر یا کم‌تر از آن برسد؟ فرض کنید $k > 2$.

- (۱) $\left\lceil \log_{\frac{k}{k-1}} n \right\rceil$ (۲) $\left\lceil \log_{1-\frac{1}{k}} n \right\rceil$ (۳) $\left\lceil \log_{1+\frac{1}{k}} n \right\rceil$ (۴) $\left\lceil \log_k n \right\rceil$

۲۶- بر روی لیست پیوندی و دوسویه‌ی Q که عناصر آن عدد هستند و اشاره‌گر به عنصر اول و آخر آن را داریم، اعمال زیر تعریف شده‌اند.

- $Delete(k)$: عنصر ابتدای Q را به ترتیب حذف می‌کند.
- $Append(c)$: عنصر آخر Q را نگاه می‌کند، اگر مقدارش از c بیش‌تر بود آن را حذف می‌کند. این کار را تکرار می‌کند تا عنصر انتهایی کم‌تر یا مساوی c شود (یا Q تهی شود).
- در آن صورت عنصر c را به انتهای صف درج می‌کند.
- اگر دنباله‌ای از n تا از این دو عمل را با ترتیب دلخواه بر روی یک لیست تهی Q انجام دهیم. مجموع کل هزینه‌ها به کدام گزینه‌ی زیر نزدیک‌تر است؟

- (۱) $n - k$ (۲) n (۳) $2n$ (۴) $3n$

۲۷- کمینه و بیشینه‌ی ارتفاع یک درخت «بی» (h) با ۱۰۰۰ عنصر با شرایط زیر چقدر است؟ هر بلوک برگ بین ۱ تا ۴ رکورد از عناصر را می‌تواند ذخیره کند. تعداد فرزندان گره‌های داخلی بین ۳ و ۵ و تعداد فرزندان ریشه بین ۲ تا ۵ است.

- (۱) $4 \leq h \leq 8$ (۲) $3 \leq h \leq 8$ (۳) $3 \leq h \leq 7$ (۴) $4 \leq h \leq 7$

۲۸- اگر $acdfbeg$ (از چپ به راست) پیمایش پیش ترتیب ($preorder$) یک درخت دودویی باشد، کدام یک از گزینه‌های زیر نمی‌تواند پیمایش میان ترتیب ($inorder$) آن باشد؟

(۱) $fdecabg$ (۲) $cabhgde$ (۳) $fdbcgae$ (۴) $adcbfge$

۲۹- هزار عنصر با کلیدهای ۱ تا ۱۰۰۰ را با تابع درهم‌سازی $h(i) = i^3 \bmod 10$ در آرایه‌ای به اندازه‌ی ۱۰ (و با اندیس‌های ۰ تا ۹) به روش زنجیره‌ای تخصیص می‌دهیم. احتمال آن که دو عنصر دلخواه (از کلیدهای داده شده) به یک درایه نگاشت شوند چقدر است؟ نزدیک‌ترین گزینه را انتخاب کنید.

(۱) $0/01$ (۲) $0/02$ (۳) $0/01$ (۴) $0/02$

سؤال‌های مهندسی IT سال ۹۳

۳۰- در مسئله‌ی ژوزفوس n نفر با شماره‌های ۱ تا n به صورت ساعت‌گرد دور یک میز دوار نشسته‌اند و با الگوریتم زیر یکدیگر را می‌کشند. اسلحه ابتدا در دست فرد شماره‌ی ۱ است. الگوریتم از شماره‌ی ۱ شروع می‌کند و در جهت ساعت‌گرد در هر مرحله از یک نفر زنده عبور کرده و فرد زنده‌ی بعدی خود را می‌کشد. این الگوریتم تا زمانی که یک نفر باقی بماند ادامه پیدا می‌کند. شماره‌ی فرد زنده‌ی آخر را $f(n)$ می‌نامیم. مثلاً اگر $n = 9$ ، به ترتیب شماره‌های ۲، ۴، ۶، ۸، ۱، ۵، ۹ و ۷ کشته شده و ۳ زنده می‌ماند. یعنی $f(9) = 3$. کدامیک از رابطه‌های بازگشتی زیر درست است؟

(۱) $f(1392) = 2f(696) - 1$ (۲) $f(1392) = 2f(696) + 1$
(۳) $f(1393) = 2f(696) - 1$ (۴) $f(1393) = f(1392) - 2$

۳۱- یک درخت دودویی جست‌وجوی متوازن شامل n عدد متمایز داده شده است. فرض کنید که به دلیل وجود نوپز عدد داخل یکی از گره‌ها تغییر می‌کند. با چه مرتبه‌ی زمانی می‌توان تشخیص داد که آیا درخت جدید هم‌چنان یک درخت دودویی جست‌وجوی معتبر هست یا خیر؟ بهترین گزینه را انتخاب کنید.

(۱) $O(\log n)$ (۲) $O(n)$ (۳) $O(n \log n)$ (۴) $O(n^2)$

۳۲- درستی گزاره‌های زیر را تعیین کنید.

الف) در هر الگوریتم مرتب‌سازی مبتنی بر مقایسه، دو عددی که اختلاف مرتبه‌ی آنها یک است، حتماً با یکدیگر مقایسه می‌شوند.

ب) در هر الگوریتم مرتب‌سازی مبتنی بر مقایسه، کوچک‌ترین و بزرگ‌ترین عدد حتماً با یکدیگر مقایسه می‌شوند.

۱) الف: نادرست و ب: نادرست ۲) الف: درست و ب: درست

۳) الف: نادرست و ب: درست ۴) الف: درست و ب: نادرست

۳۳- یک هرم پیشینه ($max\text{-}heap$) به اندازه‌ی m و یک هرم کمینه ($min\text{-}heap$) به اندازه‌ی n موجودند. بهترین الگوریتم برای ادغام این دو و ایجاد یک هرم پیشینه از کدام مرتبه است؟

۱) $O(n + m)$ ۲) $O(n \log m + m \log n)$

۳) $O(n \log n + m \log m)$ ۴) $O(\min\{n \log m, m \log n\})$

۳۴- چند تا از گزینه‌های زیر در مورد یک عبارت ریاضی E با عملگرهای دودویی و یکانی ($unary$) درست‌اند؟

الف) تنها با داشتن نگارش پیشوندی ($prefix$) E می‌توان در $O(n)$ درخت عبارت آن را به دست آورد.

ب) از نگارش پیشوندی E می‌توان مستقیماً و در $O(n)$ نگارش پسوندی ($postfix$) آن را به دست آورد.

ج) از نگارش میانوندی با پرانتز کامل ($infix$) E می‌توان در $O(n)$ درخت عبارت آن را به دست آورد.

۱) ۰ ۲) ۱ ۳) ۲ ۴) ۳

۳۵- چند تا از گزینه‌های زیر درست‌اند؟

الف) داده ساختاری برای n عنصر وجود دارد که بتوان اعمال Pop ، $Push$ ، یافتن مقدار عنصر کمینه‌ی موجود را هر کدام در $O(1)$ انجام دهد.

ب) داده ساختاری برای n عنصر وجود دارد که بتوان اعمال Pop ، $Push$ ، یافتن مقدار عنصر کمینه و یافتن مقدار عنصر بیشینه‌ی موجود را هر کدام در $O(1)$ انجام دهد.

ج) داده ساختاری برای n عنصر وجود دارد که بتوان اعمال Pop ، $Push$ و حذف عنصر کمینه‌ی موجود را هر کدام در $O(1)$ انجام دهد.

۱) ۰ ۲) ۱ ۳) ۲ ۴) ۳

سؤال‌های مهندسی کامپیوتر سال ۹۴

۳۶- فرض کنید در الگوریتم مرتب‌سازی سریع پس از عمل بخش‌بندی (*Partition*) آرایه‌ای $(۳, ۱, ۲, ۴, ۵, ۸, ۷, ۶, ۹)$ به دست آمده است. چند عدد از بین ۹ عدد در آرایه ممکن است محور این بخش‌بندی قرار گرفته باشند؟

- (۱) ۳ (۲) ۲ (۳) ۵ (۴) ۴

۳۷- می‌خواهیم مجموعه‌ای از n لیست خطی داشته باشیم که بتوانیم اعمال زیر را بر روی آن‌ها انجام دهیم:

- **Insert (x,i):** درج عنصر جدید x در لیست i ، هزینه‌ی این کار ۱ واحد است.
- **Sum (i):** جمع همه‌ی عناصر لیست i را به دست آورده و کل لیست را با یک عنصر با مقدار جمع به دست آمده جایگزین می‌کند. هزینه‌ی این کار برابر تعداد عناصر موجود در لیست i هنگام اجرای عمل فوق است.

اگر با لیست‌های تهی آغاز کنیم و اعمال گفته شده را به ترتیب دل‌خواه انجام دهیم، هزینه‌ی سرشکن هر یک از اعمال بالا متناسب با کدام گزینه است؟

- (۱) درج: ۲، جمع: ۱ (۲) درج: ۱، جمع: ۲ (۳) درج: ۱، جمع: n (۴) درج: n ، جمع: ۱

۳۸- فرض کنید برای درهم‌سازی از روش زنجیره‌ای با یک جدول به اندازه‌ی m استفاده شده است. تابع درهم‌ساز رکورد با کلید k را به خانه‌ی $k \bmod m$ نگاشت می‌کند، اگر بدانیم

کلید رکوردها زیرمجموعه‌ی $\{i^2 \mid 1 \leq i \leq 100\}$ است، به ازای کدام یک از m ‌های زیر هزینه‌ی جست‌وجو در بدترین حالت کم‌تر است؟

- (۱) ۱۱ (۲) ۷ (۳) ۹ (۴) ۱۲

۳۹- کدام گزینه حل تابع بازگشتی زیر است؟

$$T(n) = T(\log n) + O(1), T(1) = 1$$

- (۱) $O(\log n)$ (۲) $O(\log^2 n)$ (۳) $O(\log^* n)$ (۴) $O\left(\frac{n}{\log n}\right)$

۴۰- فرض کنید گره x باید بعد از گره n در یک لیست دوسویه درج شود. کدام گزینه به

درستی اشاره‌گرها را مقداردهی می‌کند. (ترتیب عملیات‌ها از چپ به راست است و فرض کنید $next[n]$ وجود دارد.)

$$next[x] = next[n]; prev[x] = prev[next[n]]; next[n] = x; prev[next[n]] = x; (۱)$$

$$next[n] = x; prev[x] = n; next[prev[n]] = x; prev[next[x]] = x; (۲)$$

next[n]=x; prev[x]=n; next[prev[x]]=x; prev[next[x]]=x; (۳)

next[x]=next[n]; prev[x]=n; next[n]=x; prev[next[x]]=x; (۴)

۴۱- یک درخت دودویی جست‌وجوی متوازن با n رأس را در نظر بگیرید. در هر گره، تعداد عناصر موجود در زیر درخت به ریشه‌ی آن گره را ذخیره کرده‌ایم. چند تا از اعمال زیر را می‌توان در زمان $O(\log n)$ انجام داد؟

• یافتن مرتبه‌ی یک عنصر داده شده

• یافتن تعداد عناصر بین a و b ($a < b$) داده شده

• یافتن جمع عناصر بین a و b ($a < b$) داده شده

(۱) ۰ (۲) ۱ (۳) ۲ (۴) ۳

سؤال‌های مهندسی IT سال ۹۴

۴۲- فرض کنید یک هرم کمینه با یک درخت متوازن پیاده‌سازی شده است. کدام گزینه نادرست است؟

(۱) بزرگ‌ترین عنصر حتماً در یک برگ ذخیره شده است.

(۲) سومین کوچک‌ترین عنصر حتماً فرزند ریشه است.

(۳) دومین بزرگ‌ترین عنصر لزوماً در برگ ذخیره نمی‌شود.

(۴) هر سه گزینه‌ی بالا.

۴۳- برچسب هر گره از یک درخت دودویی یکی از حروف الفبای انگلیسی است. می‌دانیم هر حرف حداکثر دو بار به عنوان برچسب استفاده شده است. اگر رشته‌ی حاصل از پیمایش پیش ترتیب قرینه‌ی رشته‌ی حاصل از پیمایش پس ترتیب باشد، در مورد این درخت چه می‌توان گفت؟

(۱) پیمایش میان ترتیب درخت آینه‌ای است. (۲) درخت متوازن است.

(۳) چنین درختی با حداقل ۶ گره وجود ندارد. (۴) هیچ‌کدام.

۴۴- در روش درهم‌سازی باز با واریسی خطی، تابع درهم‌سازی برای عناصر به صورت زیر است:

key: A B C D E F

hash: ۲ ۴ ۲ ۳ ۵ ۴

اگر در ابتدا جدول درهم‌سازی با اندازه‌ی ۶ تهی باشد، به چند روش مختلف می‌توان این ۶

کلید را درج کرد تا جدول نهایی برابر $H[0..5] = [F B C D A E]$ شود؟

(۱) ۰ (۲) ۲ (۳) ۴ (۴) ۸

۴۵- درخت قرمز سیاه با ۱۰۲۳ عنصر، دست کم چند گره قرمز باید داشته باشد؟

- (۱) ۰ (۲) ۱۰ (۳) ۵ (۴) ۱۱

۴۶- فرض کنید مجموعه‌ی A شامل n عدد مرتب شده (صعودی) به همراه عدد c داده شده است. با چه مرتبه‌ی زمانی می‌توان فهمید که آیا دو عدد در A وجود دارند که جمع آن‌ها برابر c شود؟

- (۱) $O(\log n)$ (۲) $O(n)$ (۳) $O(n \log n)$ (۴) $O(n^2)$

۴۷- می‌خواهیم k کوچک‌ترین عنصر یک آرایه‌ی نامرتب n عضوی را از کوچک به بزرگ به دست آوریم. زمان اجرای بهترین الگوریتم برای این کار چیست؟

- (۱) $O(n + k \lg k)$ (۲) $O(n \lg n + k)$
(۳) $O(n \lg k)$ (۴) $O(n + k \lg n)$

۴۸- رویه‌ی زیر a را به توان b می‌رساند (هر دو عدد صحیح هستند):

FASTPOWER(a, b)

۱ if $b == 1$

۲ return a

۳ $c = a \times a$

۴ $\text{ans} = \text{FASTPOWER}(c, [b / 2])$

۵ if b is odd

۶ $\text{ans} = a \times \text{ans}$

۷ return ans

تعداد دفعات فراخوانی این رویه چیست؟

- (۱) $O(\sqrt{b})$ (۲) $O(\log b)$ (۳) $O(b \log b)$ (۴) $O(b)$

۴۹- فرض کنید گره x باید بعد از گره n در یک لیست یک سویه درج شود. کدام گزینه به درستی اشاره‌گرها را مقداردهی می‌کند. (ترتیب عملیات‌ها از چپ به راست است و فرض کنید $\text{next}[n]$ وجود دارد.)

(۱) $\text{next}[n] = x; \text{next}[x] = \text{next}[n];$

(۲) $\text{next}[n] = x; \text{next}[x] = \text{next}[\text{next}[n]];$

(۳) $\text{next}[x] = n; \text{next}[n] = x;$

(۴) $\text{next}[x] = \text{next}[n]; \text{next}[n] = x;$

۵۰- فرض کنید صف Q با یک آرایه‌ی حلقوی به اندازه m پیاده‌سازی شده است که اندیس‌های آن از ۰ تا $m-1$ است و عناصر آن به صورت چرخه‌ای و در جهت ساعت‌گرد ذخیره شده‌اند. مؤلفه‌های $front(Q)$ و $rear(Q)$ به ترتیب اندیس اولین عنصر و عنصر بعد از آخرین عنصر صف Q را ذخیره می‌کنند. تعداد عناصر داخل صف و شرط پر بودن صف کدام گزینه زیر است؟

$$(۱) \quad rear(Q) - front(Q) + 1 \bmod m \quad \text{و} \quad front(Q) = rear(Q)$$

$$(۲) \quad rear(Q) - front(Q) \bmod m \quad \text{و} \quad front(Q) = rear(Q)$$

$$(۳) \quad rear(Q) - front(Q) + 1 \bmod m \quad \text{و} \quad front(Q) = rear(Q) + 1 \bmod m$$

$$(۴) \quad rear(Q) - front(Q) \bmod m \quad \text{و} \quad front(Q) = rear(Q) + 1 \bmod m$$

۵۱- چند تا از گزاره‌های زیر در مورد همبندی قوی یک گراف جهت‌دار $G = (V, E)$ درست است؟

- اگر G دور نداشته باشد، تعداد اجزای همبند قوی آن برابر V است.
- اگر یک یال از G حذف شود، تعداد اجزای همبند قوی G حداکثر ۲ واحد کم می‌شود.
- در الگوریتم یافتن اجزای همبندی قوی G ، می‌توان به جای الگوریتم DFS از BFS استفاده کرد.

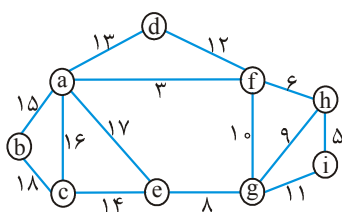
(۱) ۰ (۲) ۱ (۳) ۲ (۴) ۳

۵۲- کدام گزینه حل تابع بازگشتی زیر است؟

$$T(n) = 2T\left(\left\lfloor \frac{n}{4} \right\rfloor\right) + O(\log n), \quad T(1) = 1$$

(۱) $O(\log n)$ (۲) $O(\log^2 n)$ (۳) $O(\sqrt{n})$ (۴) $O(\sqrt{n} \log n)$

۵۳- مجموع وزن یال‌های درخت فراگیر کمینه‌ی گراف زیر چند است؟



(۱) ۶۸

(۲) ۷۲

(۳) ۷۴

(۴) ۷۷

پاسخنامه سؤال‌های مهندسی کامپیوتر سال ۹۱

(۱) گزینه (۳). با توجه به $T(n)$ داده شده، $T(n)$ حداقل از مرتبه n^2 می‌باشد ($f(n)$ هر چه باشد) پس امکان ندارد $T(n) = \theta(n \lg n)$ شود. ولی سه تابع دیگر ممکن است. مثلاً:

$$T(n) = 9T\left(\frac{n}{3}\right) + n = \theta(n^2)$$

$$T(n) = 9T\left(\frac{n}{3}\right) + n^2 \lg n = \theta(n^2 \lg^2 n)$$

$$T(n) = 9T\left(\frac{n}{3}\right) + n^3 = \theta(n^3)$$

(۲) گزینه (۴). تعداد برگ‌های درخت برابر تعداد برگ‌های زیر درخت چپ به علاوه تعداد برگ‌های زیر درخت راست است. اگر T فقط یک نود باشد آنگاه $LEAFC(left[T])$ و $LEAFC(right[T])$ برابر صفر و $ISLEAF(T)$ برابر یک می‌شود. در غیر این صورت، دو پارامتر اول تعداد برگ‌ها را محاسبه می‌کنند و $ISLEAF(T)$ برابر صفر می‌شود.

(۳) گزینه (۳). ریشه باید ۱۱ باشد و از ۱۰ عدد باقیمانده باید ۷ عدد برای زیردرخت چپ انتخاب شود که $\binom{10}{7}$ حالت دارد. حال از این ۷ عدد، بزرگترین باید ریشه باشد و از ۶ عدد باقیمانده ۵ عدد برای زیردرخت راست آن انتخاب شود که $\binom{6}{5}$ حالت دارد و از این ۵ عدد، بزرگترین باید ریشه باشد و از ۴ عدد باقیمانده ۳ عدد برای زیردرخت چپ انتخاب شود که $\binom{4}{3}$ حالت دارد و این ۳ عدد خود ۲ حالت دارند. در ضمن ۳ عددی که به زیردرخت راست ریشه می‌روند نیز ۲ حالت دارند پس:

$$\text{جواب} = \binom{10}{7} \binom{6}{5} \binom{4}{3} \times 2 \times 2 = 11520$$

(۴) گزینه (۱). در این تست قواعد زبان c نادیده گرفته شده است. و هر بار که حلقه for اجرا می‌شود، $++$ نیز اجرا می‌شود.

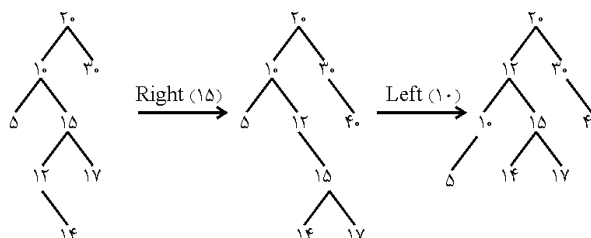
(۵) گزینه (۲). هزینه درج ۱۶ عمل اول در جدول زیر آمده است:

عمل شماره i	۱	۲	۳	۴	۵	۶	۷	۸	۹	۱۰	۱۱	۱۲	۱۳	۱۴	۱۵	۱۶
هزینه	۱	۲	۱	۴	۱	۲	۱	۸	۱	۲	۱	۴	۱	۲	۱	۱۶

- هزینه شماره‌های فرد برابر یک است پس n عمل دارای $\frac{n}{۲}$ هزینه است از شماره‌های باقیمانده اگر از ۲ فاکتور بگیرید شبیه همین جدول می‌شود پس اگر هزینه n عمل $T(n)$ باشد، هزینه شماره‌های زوج $۲T(\frac{n}{۲})$ است پس $T(n) = ۲T(\frac{n}{۲}) + \frac{n}{۲}$ که از مرتبه $\theta(n \cdot \lg n)$ است.
- (۶) گزینه (۳). این درخت ۷ سطح دارد پس ۶ عدد بزرگ (۶۴ و ۶۳ و ۶۲ و ۶۱ و ۶۰ و ۵۹) حتماً در سطوح ۱ تا ۶ هستند بنابراین بزرگترین عددی که در سطح آخر می‌تواند باشد ۵۸ است.

[پاسخنامه سؤال‌های IT سال ۹۱]

- (۷) گزینه (۳). اگر e وارد نشود، f در درایه ۵۱ قرار می‌گیرد. g که با درایه ۴۷ تصادف می‌کند به درایه ۵۰ وارد می‌شود.
- (۸) گزینه (۲). طبق قضیه Master مرتبه $\theta(n^2)$ می‌شود که $\Omega(n^2)$ نیز صحیح است.
- (۹) گزینه (۱). در گزینه ۱، با کمک A رئوس B و C و ملاقات می‌شوند حال باید با کمک B رأس E ملاقات شود که چنین نشده است.
- (۱۰) گزینه (۴). اگر کلید ریشه از i کوچکتر باشد پس باید به زیر درخت راست ریشه حرکت کنیم ($A = b \rightarrow \text{Right child}$) اگر کلید ریشه از j بیشتر باشد پس باید به زیر درخت چپ ریشه حرکت کنیم ($B = b \rightarrow \text{Left child}$) ولی اگر کلید ریشه بین i و j باشد آنگاه هم باید با زیر درخت چپ و هم زیر درخت راست ریشه، الگوریتم فراخوانی شود که جواب درست در گزینه ۴ مشاهده می‌شود.
- (۱۱) گزینه (۴). گزینه‌های ۱ و ۲ و ۳ جملات درستی را بیان می‌کنند. ولی DFS برای گرافی که با ماتریس مجاورتی ساخته شده از مرتبه $O(n^2)$ است.
- (۱۲) گزینه (۳). کلید ۱۴ سمت راست ۱۲ قرار می‌گیرد پس چون توازن کلید ۱۰ برابر ۲- شده است ابتدا باید ۱۵ را دوران به راست و سپس ۱۰ را دوران به چپ داد.



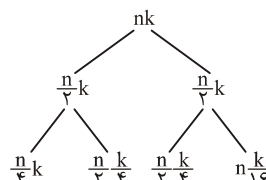
درخت حاصل در گزینه ۳ آمده است.

پاسخ سؤال‌های مهندسی کامپیوتر سال ۹۲

(۱۳) حذف شد. فرض کنید مثلاً ۹۹ عمل درج و یک عمل حذف انجام داده ایم. هزینه ۹۹ عمل درج برابر ۹۹ عمل $push$ است. هزینه عمل حذف برابر ۹۹ عمل pop از S_1 سپس ۹۹ عمل $push$ در S_2 و یک عمل pop از S_2 است پس جمع هزینه‌ها $۲۹۸ = ۱ + ۳ \times ۹۹$ است.

(۱۴) گزینه (۱). با توجه به درخت بازگشت، جمع نودهای سطح اول nk ، سطح دوم $\frac{۳nk}{۴}$ ، سطح سوم $\frac{۹nk}{۱۶}$ و ... می‌باشد. پس جمع کل هزینه‌ها برابر است با:

$$nk + \frac{۳}{۴}nk + \left(\frac{۳}{۴}\right)^2 nk + \left(\frac{۳}{۴}\right)^3 nk + \dots$$

$$= nk\left(1 + \frac{۳}{۴} + \left(\frac{۳}{۴}\right)^2 + \dots\right) = \theta(nk)$$


دقت کنید ضرایب nk ، یک تصاعد هندسی نزولی هستند که جمعشان عدد ثابت است و ارتفاع درخت تأثیری در مرتبه ندارد.

(۱۵) گزینه (۲). الف) BST متوازن که در هر نود آن علاوه بر کلید، سایز نیز ذخیره شده است. منظور از سایز نود، مجموع تعداد عناصر زیر درخت چپ و راست و خود نود است. ب) اگر چنین ساختمان داده‌ای وجود داشته باشد آنگاه مرتب‌سازی مقایسه‌ای را می‌توان با مرتبه کمتر از $n \lg n$ انجام داد که می‌دانیم ممکن نیست. زیرا کفایت با مرتبه $O(n)$ ساختمان داده مذکور را بسازیم سپس n عمل حذف مینیمم هر یک با مرتبه $o(\lg n)$ (ای کوچک) انجام دهیم که اعداد مرتب شوند!

(۱۶) گزینه (۳). اعداد مذکور را اگر در مبنای n فرض کنیم، حداکثر k رقمی می‌شوند و با $radixsort$ با مرتبه $\theta(k(n + n)) = \theta(nk)$ مرتب می‌شوند.

(۱۷) گزینه (۴). فرض کنید اندازه جدول m باشد و $n = \frac{۳}{۴}m$. در این صورت با اولین عمل درج، اندازه جدول $۲m$ می‌شود و با مرتبه $\theta(n)$ عناصر جدول قبلی به جدول جدید کپی می‌شوند. حال اگر یک عمل حذف انجام شود، مجبوریم جدولی به اندازه نصف بسازیم $(n = \frac{۳}{۸}(۲m))$ و با مرتبه $\theta(n)$ عناصر را کپی کنیم. یعنی بدترین حالت وقتی است که درج، حذف، درج، حذف و ... انجام دهیم که هر یک $\theta(n)$ می‌شوند.

(۱۸) گزینه (۱). در الگوریتم اشکال وجود دارد و باید $-\infty \leftarrow \min$ باشد. به ازای هر i دستور سطر ۴ وقتی اجرا می‌شود که $A[i]$ در بین عناصر $A[1..i]$ می‌نیم باشد که احتمال آن $\frac{1}{i}$ است پس میانگین تعداد دفعات اجرای این دستور برابر $\sum_{i=1}^n \frac{1}{i} \approx Lnn = \theta(Lgn)$ است.

[پاسخ سؤال‌های IT سال ۹۲]

- (۱۹) گزینه (۴). همانند *Minheap* می‌توان تمام این عملیات را با مرتبه $O(Lgn)$ انجام داد.
- (۲۰) گزینه (۱). فقط مورد چهارم صحیح است.
- مورد ۱: مثلاً اگر آرایه نزولی باشد، $\frac{n}{3}$ عنصر آخر که می‌نیم هستند به ابتدای آرایه منتقل نمی‌شوند.
- مورد ۲: $\frac{n}{4}$ عنصر آخر آرایه اصلاً مرتب نمی‌شوند.
- مورد ۳: مثلاً اگر می‌نیم انتهای آرایه باشد به ابتدای آرایه منتقل نمی‌شود.
- (۲۱) گزینه (۱). مرحله ۳ با مرتبه Lgm جستجوی دودویی می‌کند و عمل بازگشت در مراحل ۵ و ۶ در بدترین حالت $T(\frac{n}{3}, m)$ است.
- (۲۲) گزینه (۲). در گزینه ۱، ۱۲ ریشه است و ۹ و ۱۰ و ۵ و ۸ زیر درخت چپ و سایر اعداد زیردرخت راست هستند. در زیر درخت راست ۲۰ ریشه است و اعداد بعد از آن زیردرخت چپ ۲۰ هستند که ۱۴ ریشه است و ۱۹ و ۱۶ و ۱۳ و ۱۸ و ۱۵ باید سمت راست آن باشند که مکان ۱۳ غلط است. سایر گزینه‌ها نیز به راحتی قابل بررسی هستند.
- (۲۳) گزینه (۳). در گزینه ۳ بعد از ۲ باید ۳ یا ۵ ملاقات شوند و ۴ قابل ملاقات نیست.

[پاسخ سؤال‌های مهندسی کامپیوتر سال ۹۳]

- (۲۴) گزینه (۲). با روش عادی هزینه جستجو برابر $1 + 2 + 3 + \dots + 100$ است و با روش *MTF* هزینه جستجو برابر $(100 + 51 + 52 + 53 + \dots + 100) + (2 + 4 + 6 + \dots + 100)$ است که اختلاف اینها ۱۲۷۵ می‌باشد.
- (۲۵) گزینه (۱). هر بار $\frac{1}{k}$ از آب مخزن برداشته می‌شود، پس $1 - \frac{1}{k}$ می‌ماند. اگر x بار این عمل انجام شود باید داشته باشیم:

$$\left(1 - \frac{1}{k}\right)^x \cdot n \leq 1 \rightarrow n \leq \left(\frac{k}{k-1}\right)^x \rightarrow x \geq \log_{\frac{k}{k-1}} n$$

$$x = \left\lceil \log_{\frac{k}{k-1}} n \right\rceil \text{ پس}$$

- (۲۶) گزینه (۳). هر عمل به طور متوسط ۲ عنصر را بررسی می‌کند.
- (۲۷) گزینه (۴). کمینه ارتفاع وقتی است که در هر نود حداکثر عنصر باشد و می‌دانیم نود غیربرگی که x عنصر دارد، دارای $x + 1$ زیردرخت است. همچنین بیشینه ارتفاع وقتی است که در هر نود، حداقل تعداد عنصر قرار گیرند.
- (۲۸) گزینه (۱). a ریشه است. با توجه به گزینه ۱، g زیردرخت راست و $fdec b$ زیردرخت چپ است. c ریشه زیردرخت چپ است که fde سمت چپ آن و b راست c است ولی با توجه به گزینه ۱ امکان‌پذیر نیست. در ضمن در گزینه ۲ متأسفانه h بجای f تایپ شده است.
- (۲۹) گزینه (۳).

پاسخ سؤال‌های مهندسی IT سال ۹۳

- (۳۰) گزینه (۱). داریم $f(2n) = 2f(n) - 1$ و $f(2n + 1) = 2f(n) + 1$
- (۳۱) گزینه (۲). در بدترین حالت $\theta(n)$ عنصر باید بررسی شوند.
- (۳۲) گزینه (۴).
- (۳۳) گزینه (۱). ادغام دو هیپ دودویی از هر نوع از مرتبه خطی است. درواقع باید با $n + m$ عنصر یک هیپ ساخته شود که مرتبه آن $O(n + m)$ است.
- (۳۴) گزینه (۴). هر ۳ عبارت جملات صحیحی هستند.
- (۳۵) گزینه (۳). فقط ج نادرست است. زیرا در صورت صحیح بودن، مرتبه مرتب‌سازی مقایسه‌ای از $n \lg n$ کمتر می‌شود.

پاسخنامه سؤال‌های مهندسی کامپیوتر سال ۹۴

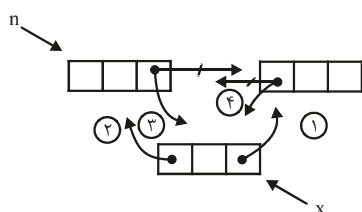
- (۳۶) گزینه (۱). بعد از پارتیشن عناصر سمت چپ محور از محور کوچک‌تر و عناصر سمت راست محور از محور بزرگ‌تر هستند. پس ۴ و ۵ و ۹ می‌توانند محور باشند.
- (۳۷) گزینه (۲). هزینه درج برابر یک واحد است. ولی هزینه جمع، متغیر است. اگر بر روی یک لیست حاوی n عنصر، n عمل جمع انجام دهیم، اولین عمل جمع هزینه n دارد و سایر عملیات جمع هزینه ۱ دارند که جمع کل هزینه $2n - 1$ است که اگر به n تقسیم کنید، ۲ حاصل می‌شود.
- (۳۸) گزینه (۱). مقدار مناسب برای m عددی اول است که نزدیک توان‌های ۲ نباشد که در نتیجه $m = 11$ مناسب است.

گزینه (۳) ۳۹.

$$T(n) = T(\lg n) + 1 = T(\lg \lg n) + 2 = T(\lg \lg \lg n) + 3$$

در واقع $T(n)$ برابر است با تعداد باری که از n لگاریتم می‌گیریم که به ۱ برسیم که همان $\lg^* n$ است.

گزینه (۴) ۴۰. باید تعداد ۴ اشاره‌گر عوض شود که گزینه‌ی ۴ به ترتیب این ۴ تا را عوض کرده است:

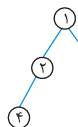


گزینه (۳) ۴۱. یافتن جمع عناصر بین a و b از مرتبه $O(n)$ است چون در بدترین حالت a برابر مینیمم و b برابر ماکزیمم است و باید همه اعداد جمع شوند. یافتن مرتبه یک عنصر داده شده از مرتبه $O(\lg n)$ است به شرطی که در هر نود تعداد نودهای زیر درختان آن ذخیره شده باشد. منظور از مرتبه یک عنصر موقعیت آن عنصر در ترتیب صعودی است، مثلاً عنصر مینیمم مرتبه‌اش ۱ است یا ماکزیمم مرتبه‌اش n است.

با مرتبه $O(\lg n)$ می‌توان تعداد اعداد بیش‌تر از a و کم‌تر از b را یافت.

[پاسخ سؤال‌های مهندسی IT سال ۹۴]

گزینه (۲) ۴۲. سومین کوچکترین عنصر می‌تواند فرزند ریشه باشد مثلاً با اعداد ۱ تا ۴، هرم



کمینه می‌تواند باشد که عدد ۳ سومین کوچکترین عنصر است. بزرگترین عنصر حتماً در یک برگ است. دومین بزرگترین می‌تواند پدر اولین بزرگترین باشد پس می‌تواند برگ نباشد.

گزینه (۴) ۴۳. یک درخت مورب با $n \geq 6$ گره که همه برچسب‌های آن متفاوت است، در نظر بگیرید. پیمایش پیش ترتیب این درخت، قرینه پیمایش پس ترتیب آن است و چنین درختی نقض گزینه‌های ۱ تا ۳ است.

گزینه (۴) ۴۴. عناصر C و D و E در آدرس طبیعی خود هستند، پس این عناصر باید ابتدا وارد شوند ولی می‌توانند به هر ترتیبی نسبت به یکدیگر باشند. بنابراین حداقل $3! = 6$ حالت

وجود دارد یعنی گزینه ۴ صحیح است. با بررسی دقیق، تعداد حالات ورود برابر ۸ است که عبارتند از: (از چپ به راست):

CDEAFB , CEDAFB , DECAFB , DCEAFB , ECDAFB , FDCAFB

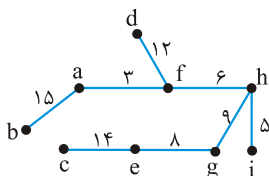
CDAEFB , DCAEFB

- (۴۵) گزینه (۱). چنین درختی می‌تواند درخت پر باشد و اصلاً گره قرمز نداشته باشد.
- (۴۶) گزینه (۲). یک روش این است که i را برابر ۱ و j را برابر n قرار دهیم (فرض می‌کنیم آرایه $A[1..n]$ است). سپس در یک حلقه، $A[i] + A[j]$ را محاسبه کنیم، اگر برابر C بود که کار تمام است، اگر کمتر از C بود، i را افزایش دهیم و اگر بیشتر از C بود، j را کاهش دهیم و این عمل را تا زمانی انجام دهیم که $i < j$. مرتبه این عمل به وضوح $O(n)$ است.
- (۴۷) گزینه (۱). کفایست k امین کوچکترین عنصر را یافته و با محوریت آن، آرایه را افراز کنیم که مرتبه این عمل $O(n)$ است و سپس k عنصر ابتدای آرایه را مرتب کنیم که با مرتبه $O(k \lg k)$ انجام می‌شود. پس کل عملیات از مرتبه $O(n + k \lg k)$ است.
- (۴۸) گزینه (۲). واضح است که در هر فراخوانی b نصف می‌شود پس مرتبه این کد $O(\lg b)$ است.
- (۴۹) گزینه (۴). دو اشاره‌گر باید عوض شوند. ابتدا باید لینک گره x ($\text{next}[x]$) به نود بعد از n اشاره کند: $\text{next}[x] = \text{next}[n]$. سپس لینک نود n به نود x اشاره کند: $\text{next}[n] = x$.
- (۵۰) گزینه (۴). اگر $f < r$ ، تعداد عناصر داخل برابر $r - f$ است و اگر $f > r$ ، تعداد عناصر داخل صف $m - (f - r)$ است، که می‌توان در هر حالت، تعداد عناصر صف را برابر $(r - f + m) \bmod m$ در نظر گرفت که گزینه‌ها همگی نادرست هستند. صف حلقوی وقتی پر است که فقط یک خانه خالی باشد که می‌توان شرط پر بودن را $f = (r + 1) \bmod m$ نوشت. کلید اولیه گزینه ۴ می‌باشد.
- (۵۱) گزینه (۲). فقط گزاره اول درست است. گراف n راسی فاقد دور، دقیقاً دارای n مولفه همبند قوی است یعنی هر رأس خود یک مولفه متصل قوی است. گزاره دوم غلط است، یک گراف n راسی که به شکل یک دور به طول n است، تصور کنید. این گراف دارای یک مولفه است، حال اگر یک یال از این گراف حذف شود آنگاه شامل n مولفه (هر رأس) می‌شود. وقتی یالی از گراف حذف می‌شود. مولفه‌ها یا تغییر نمی‌کند و یا زیاد می‌شود. گزاره سوم غلط است و فقط با DFS این عمل امکان‌پذیر است.
- (۵۲) گزینه (۳). طبق قضیه Master، $n^{\log_2 2} = n^{\frac{1}{2}} = \sqrt{n}$. رشد بیشتری از $\lg n$ دارد پس $T(n) \in O(\sqrt{n})$.

۴۱۳

ضمیمه ۲. سؤال‌های آزمون سراسری

۵۳) گزینه (۲). با هر الگوریتمی مثل کراسکال یا پریم، شکل MST این گراف به صورت زیر است که هزینه‌اش ۷۲ است:



$$3 + 5 + 6 + 8 + 9 + 12 + 14 + 15 = 72$$

سؤال‌های دکتری مهندسی کامپیوتر سال ۹۵

- ۱- اگر اعداد ۱ تا n را به ترتیب تصادفی در یک درخت جست و جوی دودویی درج کنیم، کدام رابطه بازگشتی در مورد میانگین ارتفاع این درخت صحیح است؟

$$h(n) = \frac{1}{n} \sum_{i=1}^n (h(i-1) + h(n-i)) \quad (۱)$$

$$h(n) = \frac{1}{n} \sum_{i=1}^n (h(i-1) \cdot h(n-i)) \quad (۲)$$

$$h(n) = 1 + \frac{1}{n} \sum_{i=1}^n (h(i-1) \cdot h(n-i)) \quad (۳)$$

$$h(n) = 1 + \frac{1}{n} \sum_{i=1}^n \max \{h(i-1), h(n-i)\} \quad (۴)$$

- ۲- جواب رابطه بازگشتی $T(n) = T\left(\frac{n}{4}\right) + O(\log^2 n)$ کدام است؟

$$O(\log n) \quad (۱) \quad O(\log^2 n) \quad (۲) \quad O(\log^3 n) \quad (۳) \quad O(\log^4 n) \quad (۴)$$

- ۳- فرض کنید برای مرتب‌سازی n عدد از مرتب‌ساز ادغامی استفاده کرده‌ایم. چند تا از گزینه‌های زیر درست هستند؟

– هر عدد حداکثر با $O(\log n)$ عدد مقایسه می‌شود.

– به طور متوسط هر عدد با $O(\log n)$ عدد مقایسه می‌شود.

– حتماً عددی وجود دارد که با $\Omega(\log n)$ عدد مقایسه شده است.

$$۳ \quad (۱) \quad ۲ \quad (۲) \quad ۱ \quad (۳) \quad ۰ \quad (۴)$$

- ۴- فرض کنید یک متن فقط از حروف a, b, c, d, e تشکیل شده است و تعداد تکرارهای این حروف به ترتیب ۳، ۱۶، ۸، ۴، ۱۰ است. طول کد هافمن (برحسب بیت) این متن کدام است؟

$$۸۸ \quad (۱) \quad ۸۹ \quad (۲) \quad ۹۷ \quad (۳) \quad ۱۲۳ \quad (۴)$$

- ۵- یک درخت دودویی (درخت ریشه‌دار که هر گره داخلی حداکثر دو فرزند دارد) را در نظر بگیرید که در هر گره آن یک زوج مرتب (x, y) نگهداری می‌شود. این درخت برحسب درایه اول (یعنی x) یک درخت دودویی جست و جو و برحسب درایه دوم یک هرم کمینه است (یعنی مقدار y هر گره از مقدار y فرزنداناش کمتر است) به این درخت، درخت خوب گوییم. به ازای n جفت (x_i, y_i) داده شده چند تا از گزینه‌های زیر درست‌اند؟
- درخت خوب آن یکتا است.

- ۶- لزوماً درخت خوب ندارد.
 - درخت خوبی وجود دارد که ارتفاع آن از مرتبه $O(\log n)$ است.
 - جست و جوی یک زوج داده شده در درخت خوب همیشه از مرتبه $O(\log n)$ است.
 ۱ (۳) ۲ (۲) ۳ (۱) ۴ (۰)
- ۷- چند تا از گزاره‌های زیر همیشه درست‌اند؟
 - بدون تغییر دیگری در درخت، گره‌های هر درخت دودویی جست و جو را می‌توان با رنگ‌های قرمز و سیاه رنگ کرد طوری که درخت حاصل قرمز - سیاه شود.
 - بدون تغییر دیگری در درخت، گره‌های هر درخت دودویی جست و جو با n عنصر و ارتفاع حداکثر $2 \log n$ را می‌توان با رنگ‌های قرمز و سیاه رنگ کرد طوری که درخت حاصل قرمز - سیاه شود.
 - یک درخت دودویی جست و جوی کاملاً متوازن را می‌توان فقط با رنگ کردن گره‌هایش به صورت قرمز - سیاه در آورد.
 ۱ (۲) ۲ (۳) ۳ (۴) ۴ (۰)
- ۸- داده ساختار صف با سه عملیات افزودن به ابتدای صف، حذف از انتهای صف و استخراج عنصر کمینه را در نظر بگیرید. بهترین پیاده‌سازی ممکن برای این داده ساختار هر یک از سه عملیات فوق را به صورت «سرشکن» در چه زمانی پشتیبانی می‌کند؟ بهترین گزینه را انتخاب کنید.
 ۱) هر سه عملیات $O(1)$
 ۲) هر سه عملیات $O(\log n)$
 ۳) درج و حذف $O(1)$ ، استخراج کمینه $O(\log n)$
 ۴) درج و حذف $O(\log n)$ ، استخراج کمینه $O(n)$

سؤال‌های دکتری علوم کامپیوتر سال ۹۵

- ۸- دو پیمایش $Preorder = \{A, B, C, D\}$ و $Postorder = \{A, B, C, D\}$ از درختی در دسترس هستند. تعداد درخت‌های پوشش‌دهنده این دو پیمایش چند تا است؟
 ۱ (۲) ۲ (۳) ۳ (۴) ۴ (۰)
- ۹- آرایه‌ای با ۸ عنصر را با $quicksort$ می‌خواهیم مرتب کنیم. بعد از اولین تکرار، داده‌ها به صورت زیر در آمده است. کدام داده $pivot$ بوده است؟
 ۱) ۷ یا ۹
 ۲) می‌تواند ۷ شکلی باشد ولی نمی‌تواند ۹ باشد.
 ۳) نمی‌تواند ۷ باشد ولی می‌تواند ۹ باشد.
 ۴) نه ۷ و نه ۹

پاسخنامه سؤال‌های دکتری مهندسی کامپیوتر سال ۹۵

۱- گزینه (۴). ارتفاع درخت را $h(n)$ می‌نامیم. فرض کنید عدد i ریشه است. در زیر درخت چپ اعداد ۱ تا $i-1$ قرار می‌گیرند که ارتفاع زیر درخت چپ برابر $h(i-1)$ و تعداد $n-i$ عدد در زیر درخت راست قرار می‌گیرند که ارتفاع زیر درخت راست برابر $h(n-i)$ می‌باشد و ماکزیمم بین این دو مقدار، تعیین کننده ارتفاع است. حال i می‌تواند از ۱ تا n تغییر کند و میانگین گرفته شود. در این سوال، ارتفاع درخت یک نودی، یک در نظر گرفته شده است.

۲- گزینه (۳). می‌دانیم

$$T(n) = aT\left(\frac{n}{b}\right) + n^{\log_b a} \cdot Lg^k n \quad k \geq 0 \quad \theta(n^{\log_b a} \cdot Lg^{k+1} n)$$

پس

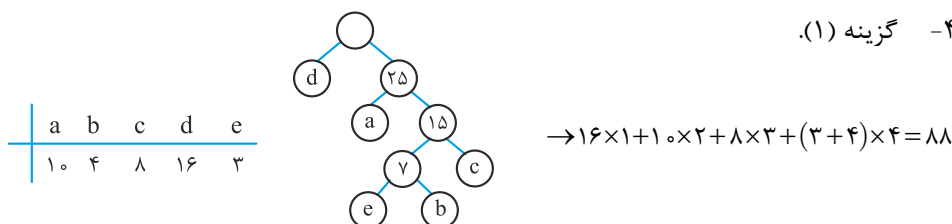
$$T(n) = T\left(\frac{n}{4}\right) + \log^2 n = \theta(\log^3 n)$$

۳- گزینه (۲). گزاره اول غلط است. مثلاً فرض کنید دو نیمه آرایه مرتب شده‌اند و حال قرار است ادغام شوند. اگر همه عناصر نیمه اول از همه عناصر نیمه دوم بزرگتر باشند آنگاه اولین عنصر نیمه اول با همه عناصر نیمه دوم مقایسه می‌شود یعنی عنصری وجود دارد که با $\theta(n)$ عنصر مقایسه می‌شود.

گزاره دوم درست است چون مرتبه مرتب‌سازی ادغامی $O(n \cdot \lg n)$ است پس به طور متوسط هریک از n عنصر با $\lg n$ عنصر مقایسه می‌شوند.

گزاره سوم درست است زیرا اگر همه اعداد با کمتر از $\lg n$ عنصر مقایسه شوند آنگاه مرتبه مرتب‌سازی از $n \cdot \lg n$ کمتر می‌شود که غیر ممکن است.

۴- گزینه (۱).



۵- گزینه (۳). اگر همه x_i ها متمایز باشند و y_i ها نیز متمایز باشند آنگاه درخت خوب (این درخت در واقع *treap* است) منحصر به فرد است ولی در غیر این صورت لزوماً چنین نیست (گزاره اول غلط).

گزاره دوم غلط است چون حداقل یک درخت خوب وجود دارد و روش ساخت این درخت به این صورت است که عنصر تازه وارد را همانند درج در T با توجه به x_i درج کنیم و پس از درج اگر مقدار y_i عنصر جدید از y_i پدرش کوچکتر باشد باید پدر عنصر جدید را دوران دهیم. ارتفاع درخت خوب می‌تواند از $\lg n$ تا n تغییر کند پس گزاره سوم درست و گزاره چهارم غلط است.

۶- گزینه (۲). شرط آنکه بتوان گره‌ها را قرمز سیاه کرد آن است که تعداد سطوح زیر درخت چپ هر نود حداکثر ۲ برابر (حداقل نصف) تعداد سطوح زیر درخت راست آن نود باشد که فقط گزاره سوم صحیح است.

۷- گزینه (۳).

پاسخنامه سؤال‌های دکتری علوم کامپیوتر سال ۹۵

۸- گزینه (۱). چنین درختی وجود ندارد.

۹- گزینه (۱). باید همه عناصر سمت چپ محور از محور کوچکتر باشند و همه عناصر سمت راست محور از محور بزرگتر باشند که ۷ و ۹ می‌توانند محور باشند.

[سؤال‌های مهندسی کامپیوتر سال ۹۵]

۱- در هر یک از رابطه‌های زیر به جای $\square$ کدام یک از نمادهای O ، o ، Θ یا Ω را بگذاریم؟ بهترین پاسخ را انتخاب کنید.

$$\log \log^* n = \square (\log^* \log n) \quad \text{الف)}$$

$$\sum_{i=1}^n \sqrt{i} = \square (n\sqrt{n}) \quad \text{ب)}$$

$$\log^{(i)} n = \log(\log^{(i-1)} n) \quad \text{و} \quad \log^* n = \min \{i \geq 0 : \log^{(i)} n \leq 1\} \quad \text{تعریف)}$$

$$\text{الف: } O \quad \text{و ب: } o \quad \text{الف: } \Omega \quad \text{و ب: } \Theta \quad \text{الف: } \log^* n \quad \text{و ب: } \log^{(i)} n$$

$$\text{الف: } \Theta \quad \text{و ب: } \Omega \quad \text{الف: } o \quad \text{و ب: } \Theta \quad \text{الف: } \log^* n \quad \text{و ب: } \log^{(i)} n$$

۲- آرایه‌ی n عضوی A تقریباً مرتب شده است، یعنی برای هر $i = 1, 2, \dots, n-k$ داریم $A[i] \leq A[i+k]$ برای مرتب‌سازی کامل آرایه چقدر زمان لازم است؟

$$O(n) \quad \text{الف)} \quad O(n \log n) \quad \text{ب)} \quad O(n \log k) \quad \text{ج)} \quad O(nk) \quad \text{د)}$$

۳- فرض کنید یک درخت دودویی با n گره داده شده است. درخت لزوماً متوازن نیست. به ازای هر گره u از درخت، اندازه دو زیر درخت سمت چپ و راست آن را محاسبه کرده و مینیمم این دو را به عنوان برجسب گره u در نظر می‌گیریم. منظور از اندازه یک زیر درخت تعداد گره‌های آن می‌باشد. اگر زیر درختی تهی باشد اندازه آن را صفر در نظر می‌گیریم، چند تا از گزینه‌های زیر درست است؟

- مجموع برجسب‌ها از مرتبه $O(n \log n)$ است.

- درختی وجود دارد که مجموع برجسب‌های آن از مرتبه $O(n)$ باشد.

- درختی وجود دارد که مجموع برجسب‌های آن از مرتبه $\Theta(n^2)$ باشد.

$$\text{الف)} \quad 0 \quad \text{ب)} \quad 1 \quad \text{ج)} \quad 2 \quad \text{د)} \quad 3$$

۴- بیشینه‌ی تعداد گره‌های با ارتفاع h در یک هرم با n عنصر برابر کدام گزینه است؟

$$\left\lceil \frac{n}{\sqrt{h+1}} \right\rceil \quad \text{الف)} \quad \left\lceil \frac{n}{\sqrt{h}} \right\rceil \quad \text{ب)} \quad \left\lfloor \frac{n}{\sqrt{h+1}} \right\rfloor \quad \text{ج)} \quad \left\lfloor \frac{n}{\sqrt{h}} \right\rfloor \quad \text{د)}$$

۵- n وزنه با ظاهری یکسان اما با وزن‌های متفاوت و نامشخص که تنها با برجسب‌های ۱ تا n از هم تفکیک شده‌اند و یک ترازوی دو کفه‌ای بدون وزنه داده شده است. می‌خواهیم تنها با توزین دو به دو وزنه‌ها و نوشتن نتایج بر روی برگه‌ای این وزنه‌ها را برحسب وزنشان مرتب کنیم. در بدترین حالت به چند بار توزین نیاز است؟ بهترین گزینه را انتخاب کنید.

$$\lceil \log_2 n! \rceil \quad \text{الف)} \quad \lceil n \log_2 n \rceil \quad \text{ب)} \quad \lfloor n \log_2 n \rfloor \quad \text{ج)} \quad n-1 \quad \text{د)}$$

- ۶- فرض کنید که داده ساختار مجموعه‌های مجزا را با درخت پیاده‌سازی کرده‌ایم. در این داده ساختار عمل $MakeSet$ یک مجموعه با یک عنصر را ایجاد می‌کند و عمل $Merge(i, j)$ دو درخت مربوط به مجموعه‌های i و j را در هم به این صورت ادغام می‌کند که درخت با ارتفاع کم‌تر را فرزند ریشه‌ی درخت دوم می‌کند. فرض کنید که n بار عمل $MakeSet$ و $n-1$ بار عمل $Merge$ را به ترتیبی نامشخص انجام داده‌ایم. سپس یک عمل $Find$ انجام می‌دهیم. هزینه‌ی این عمل $Find$ در بدترین حالت چقدر است؟
- (۱) $O(n)$ (۲) $O(1)$ (۳) $O(n \log n)$ (۴) $O(\log n)$

سؤال‌های سراسری IT سال ۹۵

- ۷- چند تا از گزاره‌های زیر درست‌اند؟

$$f(n) = \Theta(n) \wedge g(n) = \Omega(n) \Rightarrow f(n)g(n) = \Omega(n^2) \bullet$$

$$f(n) = \Theta(1) \Rightarrow n^{f(n)} = O(n) \bullet$$

$$f(n) = \Omega(n) \wedge g(n) = O(n^2) \Rightarrow g(n) / f(n) = O(n) \bullet$$

$$f(n) = O(n^2) \wedge g(n) = O(n) \Rightarrow f(g(n)) = O(n^2) \bullet$$

$$f(n) = O(\log n) \Rightarrow 2^{f(n)} = O(n) \bullet$$

$$(1) \quad (2) \quad (3) \quad (4)$$

- ۸- یک لیست پیوندی دو طرفه داده شده است. می‌خواهیم این لیست را بدون جابجا کردن مقادیر درون گره‌ها و تنها با تغییر اشاره‌گرهای بین گره‌ها مرتب کنیم. با کدام یک از روش‌های زیر می‌توان این لیست را با کم‌ترین تعداد تغییر اشاره‌گرها مرتب کرد؟
- (۱) مرتب‌سازی سریع (۲) مرتب‌سازی حبابی
(۳) مرتب‌سازی ادغامی (۴) مرتب‌سازی درجی
- ۹- فرض کنید که کلیدهای زیر با ترتیب دل‌خواهی در یک جدول درهم‌سازی به اندازه‌ی ۷ که در ابتدا تهی است درج می‌شود. فرض کنید که مقدار اولیه‌ی تابع درهم‌سازی به صورت زیر است و تصادم با وارسی خطی حل می‌شود:

کلید	اندیس درهم‌سازی
B	۳
C	۴
D	۵
F	۰
I	۳
N	۱
U	۱

چند تا از دنباله‌های زیر (از چپ به راست) می‌تواند محتوای جدول باشد؟

$F U N B C D I \bullet$

$C N F U I D B \bullet$

$C B I D N U F \bullet$

$F U I D C N B \bullet$

$F U N I B C D \bullet$

۱ (۱) ۲ (۲) ۳ (۳) ۴ (۴)

۱۰- اگر زمان اجرای یک الگوریتم با رابطه‌ای $T(n) = T(\log n) + O(n)$ تعریف شود، کدام گزینه مرتبه‌ی زمانی این الگوریتم را نشان می‌دهد؟

(تعریف: $\log^{(i)} n = \log(\log^{(i-1)} n)$ و $\log^* n = \min\{i \geq 0 : \log^{(i)} n \leq 1\}$)

۱ (۱) $O(n \log n)$ ۲ (۲) $O(n \log^* n)$ ۳ (۳) $O(n)$ ۴ (۴) $O(\log n)$

۱۱- فرض کنید $X[1..n]$ و $Y[1..n]$ دو مجموعه از اعداد دو به دو متمایزند و هر کدام به صورت صعودی مرتب هستند. در چه زمانی می‌توان میانه‌ی تمامی این $2n$ عدد را به دست آورد؟

۱ (۱) $O(n)$ ۲ (۲) $O(n \log n)$ ۳ (۳) $O(\log^2 n)$ ۴ (۴) $O(\log n)$

پاسخنامه سؤال‌های مهندسی کامپیوتر سال ۹۵

۱- گزینه (۴). الف) با توجه به تعریف لگ استار، می‌دانیم $\lg^*(\lg n) = \lg^* n - 1$ و با توجه به

اینکه x از $\lg x$ رشد بیشتری دارد پس $\lg^* n$ از $\lg \lg^* n$ رشد بیشتری دارد پس

$$\lg \lg^* n = o(\lg^* \lg n)$$

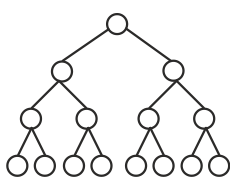
$$\text{ب) می‌دانیم } \sum_{i=1}^n i^p = \theta(n^{p+1}) \text{ که } p \geq 0 \text{ پس } \sum_{i=1}^n \sqrt{i} = \theta(n^{\frac{3}{2}})$$

۲- گزینه (۳). با توجه به شرط گفته شده، آرایه A شامل k تا زیرآرایه مرتب $\frac{n}{k}$ عنصری است که

باید این زیرآرایه‌های مرتب را با هم ادغام کنیم که با کمک هرم کمینه می‌توان این عمل را انجام داد. کافی است با عناصر ابتدای زیرآرایه‌ها یک هرم کمینه بسازیم که ارتفاعش $\lg k$ می‌شود و سپس همه n عنصر را در هرم درج و حذف کنیم که مرتبه $O(n \cdot \lg k)$ می‌شود.

۳- گزینه (۳). گزاره‌های اول و دوم صحیح هستند و گزاره سوم نادرست است. حداکثر مجموع برچسب‌ها وقتی حاصل می‌شود که درخت پر یا تقریباً پر باشد که در این صورت جمع

برچسب‌ها $\theta(n \lg n)$ خواهد شد. پس جمع برچسب‌ها همیشه از مرتبه $O(n \lg n)$ است. برای درخت مورب، جمع برچسب‌ها صفر است که از مرتبه $O(n)$ است پس فقط گزاره سوم نادرست است زیرا جمع برچسب‌ها از $n \lg n$ کمتر یا مساوی است.



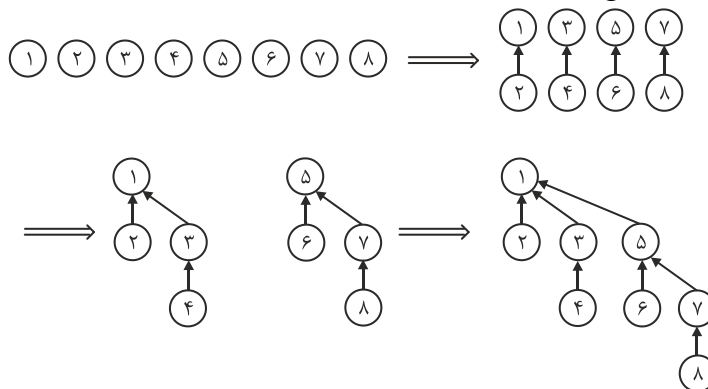
۴- گزینه (۱). برای درک بهتر، هرم پر با $n = ۱۵$ نود را در نظر بگیرید. در این هرم ۸ تا نود با ارتفاع صفر (برگ‌ها)، ۴ تا نود با ارتفاع یک، ۲ نود با ارتفاع ۲، یک نود با ارتفاع ۳ وجود دارد. که این مقادیر در $\left\lceil \frac{n}{2^{h+1}} \right\rceil$ صدق می‌کنند. با توجه به این رابطه اگر

$n = ۱۵$ و $h = ۰$ آنگاه $\left\lceil \frac{۱۵}{2^{0+1}} \right\rceil = ۸$ حاصل می‌شود که ۸ تا نود با ارتفاع صفر وجود دارد

و به همین ترتیب.

۵- گزینه (۱). درخت تصمیم برای این مسئله دارای $n!$ برگ است و ارتفاع آن حداقل $\lceil \lg n! \rceil$ است پس در بدترین حالت حداقل $\lceil \lg n! \rceil$ توزین نیاز است.

۶- گزینه (۴). با توجه به اینکه در عمل $merge$ (union)، درخت با ارتفاع کمتر را فرزند ریشه درخت دوم قرار می‌دهیم، حداکثر ارتفاع درخت $\lg n$ می‌شود که این ارتفاع وقتی حاصل می‌شود که دو به دو عمل $merge$ انجام شود مثلاً به ازای $n = ۸$ به عملیات زیر توجه کنید که در نهایت ارتفاع درخت $\lg ۸ = ۳$ شده است:



پاسخ سؤال‌های مهندسی IT سال ۹۵

۷- گزینه (۲). گزاره ۱ صحیح است. $f(n)$ با n هم رشد است و $g(n)$ رشد بیشتر یا مساوی n دارد پس $f(n).g(n)$ رشد بیشتر یا مساوی n^2 دارد.

گزاره ۲ صحیح نیست. مثلاً اگر $f(n) = ۲$ آنگاه $f(n) = ۲ \neq O(n)$ $n^{f(n)} = n^2$

گزاره ۳ صحیح است. $f(n)$ رشد بیشتر یا مساوی n دارد و $g(n)$ رشد کمتر یا مساوی n^2 دارد پس $\frac{g(n)}{f(n)}$ رشد کمتر یا مساوی n دارد.

گزاره ۴ صحیح نیست. مثلاً $f(n) = \frac{1}{n^4}$ و $g(n) = \frac{1}{n}$ آنگاه $f(g(n)) = n^4 \neq O(n^3)$

گزاره ۵ صحیح نیست. مثلاً اگر $f(n) = 2 \lg n$ آنگاه $f(n) = 2 \lg n \neq O(n)$

۸- گزینه (۴). در مرتب‌سازی درجی از عنصر اول تا n ام عمل درج انجام می‌شود. عنصری که درج می‌شود در جای صحیح خود درج می‌شود و نسبت به سایر روش‌ها تغییر اشاره‌گر کمتری دارد. مثلاً در روش سریع اگر عنصر اول را $pivot$ بگیریم ممکن است مجبور باشیم همه اشاره‌گرها را عوض کنیم و همه عناصر به قبل از $pivot$ منتقل شوند. در روش حبابی در هر دور ممکن است یک عنصر تا انتهای لیست حرکت کند (لیست معکوس مرتب باشد). در روش ادغامی، هنگام ادغام دو لیست ممکن است تغییر اشاره‌گر زیادی پیش آید.

۹- گزینه (۲). دنباله اول امکان‌پذیر است $\begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 \\ \text{F} & \text{U} & \text{N} & \text{B} & \text{C} & \text{D} & \text{I} \end{matrix}$ و اگر ترتیب ورود به صورت $FUNBCDI$ (چپ به راست) باشد، ایجاد می‌شود. دنباله‌های دوم و سوم صحیح نیستند، زیرا کلیدها به ترتیب وارد شوند F حتماً در حفره صفر باید قرار گیرد.

دنباله چهارم $\begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 \\ \text{F} & \text{U} & \text{I} & \text{D} & \text{C} & \text{N} & \text{B} \end{matrix}$ صحیح نیست زیرا ابتدا کلیدهای F و U که در جای طبیعی خود هستند وارد می‌شوند. پس از اینها اگر I وارد شود باید در حفره ۳ قرار گیرد، اگر D وارد شود باید در حفره ۵ قرار گیرد، اگر N وارد شود باید در حفره ۲ قرار گیرد، اگر B وارد شود باید در حفره ۳ قرار گیرد که چنین نشده‌اند. دنباله پنجم $\begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 \\ \text{F} & \text{U} & \text{N} & \text{I} & \text{B} & \text{C} & \text{D} \end{matrix}$ صحیح است و ترتیب ورود می‌تواند $FUNIBCD$ (چپ به راست) باشد.

۱۰- گزینه (۳).

$$\begin{aligned} T(n) &= T(\lg n) + n = T(\lg \lg n) + \lg n + n \\ &= T(\lg \lg \lg n) + \lg \lg n + \lg n + n \\ &\dots \\ &= \lg \lg \dots \lg n + \dots + \lg n + n = \theta(n) \end{aligned}$$

۱۱- گزینه (۴). این تست در طراحی الگوریتم چندین بار طرح شده است. کافی است عناصر وسط دو لیست مقایسه شوند و با عمل مقایسه نصف عناصر حذف می‌شوند پس

$$T(2n) \leq T(n) + \theta(1) = \theta(\lg n)$$

[سؤال‌های مهندسی کامپیوتر سال ۹۶]

۱- اگر زمان اجرای یک الگوریتم با رابطه‌ی بازگشتی $T(n) = \sqrt{n}T(\sqrt{n}) + n$ مشخص شود، کدام گزینه پیچیدگی زمانی الگوریتم را به درستی بیان می‌کند؟

- (۱) $\theta(n)$ (۲) $\theta(\sqrt{n} \log n)$ (۳) $\theta(n \log \log n)$ (۴) $\theta(n \log n)$

۲- برای درخت ریشه‌دار T ، درخت دودویی متناظر T' به شکل زیر تعریف می‌شود:

a. سمت چپ‌ترین فرزند هر گره در T فرزند چپ آن گره در T' است.

b. برادر سمت راست هر گره در T فرزند راست آن در T' است.

کدام گزینه در مورد پیمایش این دو درخت درست است؟

- (۱) $Preorder(T) = Postorder(T')$ (۲) $Inorder(T) = Inorder(T')$
(۳) $Postorder(T) = Postorder(T')$ (۴) $Preorder(T) = Preorder(T')$

۳- چند تا از عبارت‌های زیر در مورد درخت دودویی جست‌وجو (د.د.ج) درست است؟

a. اگر یک عنصر موجود در د.د.ج را حذف و بلافاصله درج کنیم، د.د.ج قبل و بعد از دو عمل فوق یکسان است.

b. هر د.د.ج را می‌توان با چند عمل چرخش (rotation) به یک د.د.ج متوازن تبدیل کرد.

c. عدد بلافاصله بعد از x در ترتیب صعودی، لزوماً در زیردرخت به ریشه‌ی گره‌ای که x در آن ذخیره شده قرار نمی‌گیرد.

- (۱) ۰ (۲) ۱ (۳) ۲ (۴) ۳

۴- کدام یک از گزینه‌های زیر در مورد پیچیدگی الگوریتم مرتب‌سازی شمارشی روی یک آرایه‌ی n تایی که کلیدهای آن اعداد صحیح و مثبت کمتر از عدد داده‌شده‌ی M هستند درست است؟ بهترین گزینه را انتخاب کنید. (فرض کنید چهار عمل اصلی در $O(1)$ انجام می‌شود.)

- (۱) $\theta(n)$ (۲) $\theta(n + M)$ (۳) $\theta(nM)$ (۴) $\theta(n \log M)$

۵- فرض کنید هزینه‌ی جست‌وجوی ناموفق در روش درهم‌سازی زنجیره‌ای و روش آدرس‌دهی باز با استفاده از واریسی خطی به ترتیب q و h باشند. همچنین فرض کنید در هر دو روش، از تابع درهم‌ساز یکنواخت استفاده شده است و تعداد عناصر درج شده کمتر از اندازه‌ی جدول است. کدام گزینه درست است؟

- (۱) $q = O(h)$ (۲) $q = \theta(h)$ (۳) $q = \Omega(h)$ (۴) $q = o(h)$

[سؤال‌های سراسری IT سال ۹۶]

۶- فرض کنید n رشته‌ی متمایز از صفر و یک داریم، طول بزرگ‌ترین رشته k است. از درخت ترای برای نگهداری این رشته‌ها استفاده شده است. ارتفاع درخت ترای از چه مرتبه‌ای است.

$$(۱) \theta(k) \quad (۲) \theta(\log n) \quad (۳) \theta(\log n + k) \quad (۴) \theta(\log k)$$

۷- یک درخت دودویی شامل n عنصر و ارتفاع $O(\log n)$ داده شده است. به ازای هر گره به فرزندان و پدر در $O(۱)$ می‌توان دسترسی پیدا کرد. به ازای دو گره‌ی داده شده‌ی u و v با چه مرتبه‌ی زمانی می‌توان کوتاه‌ترین مسیر بین آنها را پیدا کرد؟ بهترین گزینه را انتخاب کنید.

$$(۱) \theta(n) \quad (۲) \theta(\log n) \quad (۳) \theta(\log^2 n) \quad (۴) \theta(\log n \log n \log n)$$

۸- کدام یک از گزینه‌های زیر پیچیدگی درست برای $T(n, n)$ است که از رابطه‌ی بازگشتی زیر به دست آمده است؟

$$\begin{cases} T(x, c) = \theta(x) & c \leq 2 \\ T(c, y) = \theta(y) & c \leq 2 \\ T(x, y) = \theta(x + y) + T\left(\frac{x}{2}, \frac{y}{2}\right) \end{cases}$$

$$(۱) \theta(\log n) \quad (۲) \theta(n) \quad (۳) \theta(n \log n) \quad (۴) \theta(n \log^2 n)$$

۹- چند تا از گزینه‌های زیر در مور هرم کمینه که با یک درخت دودویی پیاده‌سازی شده است، درست است؟

- بزرگ‌ترین عدد همیشه در برگ ذخیره می‌شود.
- سومین کوچک‌ترین عنصر (با فرض وجود حداقل سه عنصر در درخت) حتماً در فرزند ریشه ذخیره می‌شود.
- عملیات حذف کوچک‌ترین عنصر در زمان $O(۱)$ انجام می‌پذیرد.
- k امین کوچک‌ترین عنصر نمی‌تواند در سطحی بیش از $k + ۱$ در درخت قرار بگیرد.

$$(۱) ۱ \quad (۲) ۲ \quad (۳) ۳ \quad (۴) ۴$$

۱۰- دنباله‌ی $jfcbadeghik$ پیش ترتیب یک درخت دودویی جست وجوی T است. کدام یک از گزینه‌های زیر دنباله‌ی پس ترتیب T است؟

(۱) $abdecihgfkj$ (۲) $badecihgfkj$ (۳) $abdecighgfkj$ (۴) $abedcihgfkj$

۱۱- با توجه به دو گزاره‌ی زیر، کدام گزینه صحیح است؟

(الف) هر الگوریتم مرتب‌سازی ناپایدار مبتنی بر مقایسه را می‌توان بدون افزایش پیچیدگی آن به گونه‌ی پایدار آن تغییر داد.

(ب) ارتفاع درخت تصمیم هر الگوریتم مرتب‌سازی مبتنی بر مقایسه $\Omega(n \log n)$ است که n تعداد اعداد می‌باشد.

(۱) (الف) درست، (ب) درست (۲) (الف) نادرست، (ب) درست

(۳) (الف) درست، (ب) نادرست (۴) (الف) نادرست، (ب) نادرست

پاسخنامه سؤال‌های مهندسی کامپیوتر سال ۹۶

۱- گزینه (۳). طرفین را به n تقسیم کرده و $\frac{T(n)}{n} = g(n)$ تعریف می‌کنیم:

$$g(n) = g(\sqrt{n}) + 1$$

حال $n = 2^m$ تعریف می‌کنیم و $g(2^m) = F(m)$ در نتیجه:

$$F(m) = F\left(\frac{m}{2}\right) + 1 = \theta(\lg m) \rightarrow g(n) = \theta(\lg \lg n) \rightarrow T(n) = \theta(n \cdot \lg \lg n)$$

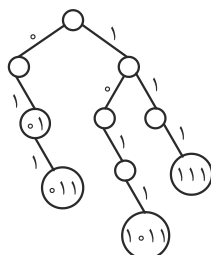
۲- گزینه (۴). در واقع درخت ریشه‌دار T را به درخت دودویی تبدیل کرده است و می‌دانیم، پیمایش پیش ترتیب درخت عمومی با پیش ترتیب درخت دودویی معادلش برابر است. همچنین پیمایش پس ترتیب درخت عمومی با پیمایش میان ترتیب درخت دودویی معادلش یکسان است.

۳- گزینه (۳). جملات دوم و سوم درست هستند. جمله اول نادرست است. اگر کلیدی از BST حذف شود و آن کلید برگ نباشد، پس از درج مجدد آن کلید، درخت حاصل با درخت قبل یکسان نخواهد بود. در مورد جمله سوم توجه کنید که $succ(x)$ ممکن است یکی از اجداد x باشد. در واقع اگر x فرزند راست نداشته باشد، $succ(x)$ اولین جد x که در زیر درخت چپ آن واقع است، می‌باشد.

۴- گزینه (۲). مرتب‌سازی شمارشی از مرتبه $\theta(n+k)$ است که n تعداد اعداد و k ماکزیمم عدد است.

۵- گزینه (۲). هزینه جستجوی ناموفق در هر دو روش یکسان است.

پاسخ سؤال‌های مهندسی IT سال ۹۶



۶- گزینه (۱). ارتفاع درخت برای به اندازه حداکثر طول رشته‌هاست. مثلاً درخت برای با رشته‌های ۰۱ و ۰۱۱ و ۱۱۱ و ۱۰۱۱ به شکل مقابل است که ارتفاع ۴ دارد.

۷- گزینه (۲). از u به v به سمت اجداد حرکت کنید تا به اولین جد مشترک u و v برسید. کوتاهترین مسیر از u به v مسیری است که از آن جد عبور می‌کند. مرتبه این عمل به اندازه ارتفاع درخت است.

۸- گزینه (۲).
$$T(x, y) = T\left(\frac{x}{2}, \frac{y}{2}\right) + c(x + y)$$

$$= T\left(\frac{x}{4}, \frac{y}{4}\right) + c\left(\frac{x}{2} + \frac{y}{2}\right) + c(x + y)$$

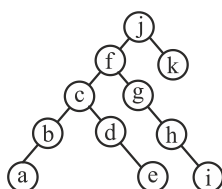
$$= T\left(\frac{x}{8}, \frac{y}{8}\right) + c\left(\frac{x}{4} + \frac{y}{4}\right) + c\left(\frac{x}{2} + \frac{y}{2}\right) + c(x + y)$$

⋮

$$= c(x + y) \underbrace{\left(1 + \frac{1}{2} + \frac{1}{4} + \dots\right)}_{\theta(1)} = \theta(x + y)$$

$$\rightarrow T(n, n) = \theta(n + n) = \theta(n)$$

۹- گزینه (۲). جملات اول و چهارم درست هستند. حذف عنصر کمینه از مرتبه $O(\lg n)$ است و یافتن عنصر کمینه از مرتبه $O(1)$ است. عنصر بیشینه حتماً در یکی از برگ‌هاست (فرض این است که کلیدها متمایز هستند)



۱۰- گزینه (۴). J ، ریشه است، حال اگر f از j بیشتر باشد آنگاه در پیمایش پس ترتیب، f باید قبل از j باشد که در گزینه‌ها چنین نیست، پس f از j کوچکتر است. با بررسی کامل می‌توان نشان داد که شکل درخت می‌تواند به صورت مقابل باشد:

۱۱- گزینه (۱). هر دو گزاره درست هستند.